现代计算机科学与技术教材系列

算法设计与分析

SUANFA SHEJI YU FENXI

张德富 编著

国防工业出版社
National Defense Industry Press

责任编辑：宋序一　xysong@ndip.cn
责任校对：钱辉玲
封面设计：彭建华　jhpeng@ndip.cn

算法设计与分析

SUANFA SHEJI YU FENXI

上架建议：计算机科学

http://www.ndip.cn

ISBN 978-7-118-06308-0

9 787118 063080 >

定价：36.00元

现代计算机科学与技术教材系列

算法设计与分析

张德富　编著

国防工业出版社

·北京·

内 容 简 介

本书主要取材于算法设计与分析领域的经典内容，并介绍了算法设计的发展趋势。内容主要包括非常经典的算法设计技术，例如递归与分治、动态规划、贪心、回溯、分支限界、图算法，也包括了一些高级的算法设计主题，例如网络流和匹配、启发式搜索、线性规划、数论以及计算几何。在算法分析方面，介绍了概率分析以及最新的分摊分析和实验分析方法。在算法的理论方面，介绍了问题的下界、算法的正确性证明以及 NP 完全理论等方面的内容。本书包括大量的问题实例，并给出了相应的设计与分析方法，书后精选了一些习题，供读者练习，以巩固所学的算法。工业应用领域的许多实际问题和疑难问题都需要有效的求解算法，本书提供了设计有效算法的基础以及大量的可供选择的解决途径。

本书内容基本上涵盖了目前程序设计竞赛所要掌握的算法，并在书后精选了部分 ACM 国际大学生程序设计竞赛的题目，供大家练习。

本书可作为计算机科学系、数学系、软件学院等专业本科及研究生课程的教材，特别适合于有志于参加程序设计竞赛的学生学习和训练。

图书在版编目(CIP)数据

算法设计与分析/张德富编著. —北京：国防工业出版社，2009.8

(现代计算机科学与技术教材系列)

ISBN 978-7-118-06308-0

Ⅰ.算… Ⅱ.张… Ⅲ.①电子计算机－算法设计－高等学校－教材②电子计算机－算法分析－高等学校－教材

Ⅳ.TP301.6

中国版本图书馆 CIP 数据核字(2009)第 062011 号

※

国防工业出版社出版发行

(北京市海淀区紫竹院南路 23 号 邮政编码 100048)

腾飞印务有限公司印刷

新华书店经售

*

开本 710×960 1/16 印张 21¾ 字数 416 千字

2009 年 8 月第 1 版第 1 次印刷 印数 1—4000 册 定价 36.00 元

(本书如有印装错误，我社负责调换)

国防书店：(010)68428422 发行邮购：(010)68414474

发行传真：(010)68411535 发行业务：(010)68472764

目　录

第1章　入门 …… 1
1.1　问题 …… 1
1.2　算法的概念 …… 1
1.3　算法的正确性 …… 3
1.4　算法的效率 …… 5
1.5　问题的下界 …… 9
1.6　小结 …… 10
习题 …… 11
实验题 …… 12
第2章　渐近符号 …… 13
2.1　Θ符号 …… 13
2.2　O符号 …… 15
2.3　Ω符号 …… 16
2.4　渐近符号的性质 …… 17
2.5　常用函数的直观含义 …… 18
2.6　小结 …… 18
习题 …… 19
第3章　算法分析方法 …… 20
3.1　概率分析 …… 20
3.2　分摊分析 …… 22
3.2.1　合计方法 …… 23
3.2.2　记账方法 …… 25
3.2.3　势能方法 …… 27
3.3　实验分析 …… 29
3.4　小结 …… 30
习题 …… 31
第4章　递归 …… 32
4.1　算法思想 …… 32
4.1.1　递归算法的应用 …… 33
4.1.2　递归与迭代 …… 40

4.2 递归方程的求解 …… 41
4.2.1 替换方法 …… 41
4.2.2 递归树方法 …… 43
4.2.3 公式法 …… 45
4.3 多项式求值实验 …… 47
4.4 小结 …… 48
习题 …… 48
实验题 …… 49
第5章 分治算法 …… 50
5.1 算法思想 …… 50
5.2 合并排序 …… 51
5.3 快速排序 …… 53
5.4 大整数乘法 …… 56
5.5 矩阵乘法 …… 58
5.6 残缺棋盘游戏 …… 59
5.7 快速傅里叶变换(FFT) …… 62
5.8 小结 …… 63
习题 …… 63
实验题 …… 65
第6章 动态规划 …… 66
6.1 算法思想 …… 66
6.2 装配线调度问题 …… 68
6.3 矩阵链乘法问题 …… 73
6.4 最长公共子序列问题 …… 77
6.5 0/1 背包问题 …… 81
6.6 最优二叉搜索树问题 …… 84
6.7 动态规划的基本性质 …… 88
6.8 小结 …… 92
习题 …… 92
实验题 …… 94
第7章 贪心算法 …… 95
7.1 算法思想 …… 95
7.2 任务选择问题 …… 95
7.3 背包问题 …… 100
7.4 哈夫曼编码问题 …… 102
7.5 缓存维护问题 …… 106

7.6 任务选择问题实验 …… 108
7.7 小结 …… 109
习题 …… 109
实验题 …… 111
第8章 图算法 …… 112
8.1 图的搜索问题 …… 113
8.1.1 宽度优先搜索 …… 113
8.1.2 深度优先搜索 …… 117
8.2 最小生成树问题 …… 121
8.2.1 Kruskal 算法 …… 122
8.2.2 Prim 算法 …… 124
8.3 最短路径问题 …… 125
8.3.1 单个源点的最短路径问题 …… 128
8.3.2 所有点对的最短路径问题 …… 131
8.4 小结 …… 134
习题 …… 135
实验题 …… 137
第9章 网络流与匹配 …… 138
9.1 最大流问题 …… 138
9.1.1 FordFulkerson 方法 …… 140
9.1.2 最短路径增广算法 …… 145
9.1.3 Dinic 算法 …… 148
9.1.4 MPM 算法 …… 151
9.1.5 最大流问题的变形 …… 152
9.2 最小费用流问题 …… 153
9.2.1 消除回路算法 …… 154
9.2.2 最小费用路算法 …… 156
9.2.3 最小费用路算法的改进 …… 158
9.3 匹配问题 …… 160
9.3.1 二分图匹配 …… 163
9.3.2 一般图的匹配 …… 167
9.4 小结 …… 172
习题 …… 172
实验题 …… 174
第10章 线性规划 …… 175
10.1 线性规划问题 …… 175

10.1.1 线性规划问题的标准形式 …… 176
10.1.2 线性规划问题的松弛形式 …… 178
10.2 求解算法 …… 180
10.2.1 图解法 …… 180
10.2.2 单纯形算法 …… 181
10.3 对偶 …… 188
10.4 小结 …… 192
习题 …… 192
实验题 …… 193
第 11 章 NP 完全理论 …… 194
11.1 判定问题 …… 195
11.2 P 和 NP …… 197
11.3 NPC …… 201
11.3.1 NPC 的定义 …… 201
11.3.2 电路可满足性问题 …… 203
11.4 NPC 的证明 …… 206
11.4.1 可满足性问题 …… 206
11.4.2 3-CNF 可满足性问题 …… 208
11.4.3 团问题 …… 210
11.4.4 顶点覆盖问题 …… 212
11.5 其他 NP 完全问题 …… 213
11.6 小结 …… 215
习题 …… 215
第 12 章 回溯 …… 218
12.1 算法思想 …… 218
12.2 装载问题 …… 222
12.3 0/1 背包问题 …… 226
12.4 着色问题 …… 228
12.5 n 皇后问题 …… 230
12.6 旅行商问题 …… 232
12.7 流水作业调度问题 …… 234
12.8 零件切割问题 …… 236
12.9 小结 …… 238
习题 …… 238
实验题 …… 240

第 13 章　分支限界 …… 241
13.1　算法思想 …… 241
13.2　装载问题 …… 243
13.3　0/1 背包问题 …… 251
13.4　可满足性问题 …… 254
13.5　旅行商问题 …… 256
13.6　流水作业调度问题 …… 258
13.7　0/1 背包问题实验 …… 260
13.8　小结 …… 261
习题 …… 262
实验题 …… 263
第 14 章　启发式搜索 …… 264
14.1　算法思想 …… 264
14.2　A*搜索 …… 265
14.2.1　最短路径问题 …… 267
14.2.2　8 数字问题 …… 268
14.3　博弈搜索算法 …… 271
14.3.1　α 和 β 剪支 …… 273
14.3.2　分硬币游戏 …… 275
14.3.3　井字博弈 …… 275
14.4　小结 …… 281
习题 …… 281
实验题 …… 283
第 15 章　数论 …… 284
15.1　数论基础 …… 284
15.2　最大公约数 …… 287
15.3　同余 …… 289
15.4　模幂运算 …… 294
15.5　数论的应用 …… 295
15.5.1　Hash 函数 …… 295
15.5.2　RSA 公钥加密系统 …… 296
15.6　小结 …… 299
习题 …… 299
实验题 …… 300
第 16 章　计算几何 …… 301
16.1　叉积及其应用 …… 301

16.2 计算任意线段的交点 …… 304
16.3 凸包 …… 306
16.3.1 礼物包装算法 …… 307
16.3.2 Graham 扫描法 …… 309
16.3.3 分治算法求凸包 …… 310
16.4 最近点对 …… 316
16.5 Voronoi 图 …… 318
16.5.1 Voronoi 图的定义及其性质 …… 318
16.5.2 Voronoi 图的构造 …… 321
16.5.3 Voronoi 图的应用 …… 325
16.6 小结 …… 327
习题 …… 327
实验题 …… 328
参考文献 …… 329

《现代计算机科学与技术教材系列》
编审委员会

主　　编	刘椿年	许满武		
执行副主编	陈国良	赵致琢		
编　　委	王志英	康立山	刘大有	刘椿年
	许满武	宋方敏	林作铨	陈向群
	李师贤	陈国良	洪先龙	应明生
	陆汝占	傅育熙	赵致琢	唐常杰
	张明义	彭群生	朱　洪	周龙骧
	李立武	承恒达	李西宁	陈　溧
	阎瑞琪			

总　序

民众多好饮酒，中外概莫能外。酒馆和酿酒坊伴随饮酒客而起，人类对酒的喜爱造就了酒文化和一个庞大的产业。好酒能卖好价钱，能使文人诗兴大发，催生佳作，还能解人间百难。于是，酿天下名酒自然成为不少人的毕生追求。

怎样才能酿出好酒呢？国人的看法不尽相同。崇信洋酒的人主张引进国外的生产工艺，学习洋人的生产和经营理念，而喜欢国酒的人则主张走自己的路，但不排除借鉴国外先进的科学技术和管理经验。这样的争论或许永远不会终结，但外国人重视科学酿酒，这一点是值得我们学习和借鉴的。

计算机科学教育，如同酿酒工业的生产一样，科学办学迄今还只是部分学者的一种理想。与国内一样，国外的计算机科学教育并没有像他们的科学酿酒业一样，实现科学办学。也许科学办学要远比科学酿酒困难得多。譬如，怎么实现科学办学？甚至怎么推出一套科学的系列教材都是一篇大文章。

这套教材的创作始于教育部面向21世纪教育与教学改革13－22项目的研究。2000年，在13－22项目研究工作即将完成之际，一些学者开始认识到面对计算机科学与技术的高速发展，我们亟需一套体现科学办学思想、反映内涵发展要求、服务教育与教学改革、参与构建学科人才培养科学体系的系列教材。强调系列教材是因为那时已经意识到计算机科学教育本质上是一项科学活动，但长期以来教师向学生传授科学技术知识的方式方法科学性不强。由于高等教育几百年来一直沿袭经验方式而非科学方式办学，大学教学的方式方法仍然还停留在古代作坊式的阶段，只不过今天使用的教学技术手段先进而已。在经验办学方式下，无论是研究型大学还是教学型大学，由于种种原因，教学活动的全过程存在着太多的漏洞和质量上的隐患。科学办学是对高等教育界传统的一个挑战，尽管在认识上，人们不难理解，科学办学是经验办学的最高形式，而经验办学应该成为科学办学的有益补充。

13－22项目组积极探索，率先倡导科学办学理念，初步构建了一个体现科学办学思想，反映内涵发展要求的计算机科学一级学科人才培养科学理论体系，为学科专业教育探索新天地，走向科学办学和发展学科系列教材提供了一个认知基础。

长期以来，学术界一直在探索计算机科学专业教育的规律。ACM和IEEE/CS的专家小组在走访了全美400多位著名计算机科学家的基础上，以学科方法论作

为切入点开展教学改革理论研究，于1989年发表了具有开创性意义的成果，尽管他们并未意识到自己的工作是以学科方法论的研究作为切入点，探讨内涵发展的道路。1990年前后，在迷宫中探索行走的专家小组经大师和精英群的指点实际上已经摸到了走出迷宫的大门，却没有打开并进入一个崭新的天地。这一点从他们在2000年网上公布的CC2001报告最先删除了CC1991报告中有关学科方法论的内容便不难看出（注：后经中国人的提醒又补充写入）。

与此同时，中外教材建设也一直没有停止探索，国内外出版社先后出版了种类繁多的计算机类专业教材。这些教材中不乏精品和上乘之作，但难觅具有鲜明特色、真正一体化设计并且符合科学办学要求的系列教材。多数丛书和系列教材基本上还只是出版社对出自作者个人创作的教材通过冠名“丛书”或“系列”的方法结集出版以求强势效应。尽管如此，不少优秀作者和学者理所当然地进入了编审委员会的视线。西方发达国家在计算机科学学科的领先优势曾使许多人不自觉地将目光转向海外，试图从世界名牌大学使用的教材中去寻找蓝本。遗憾的是与国内一样，经验办学并没有使西方大学在教材建设方面摆脱“各自为政，各行其是”的阴影。此时，我们如梦初醒，毕竟科学办学是前无古人的一项创举。随着学科的不断深入，在迈向深蓝知识海洋的今天，外国人未必比中国人在科学办学方面占有更多天时地利的优势。不经意中的发现使我们惊喜和激动，同时深感责任重大且平添担忧：即使能够写出系列教材的一体化设计，我们是否真能确认这项改革的正确性？真能推出科学的系列教材？可是，除了实践和试验，我们别无捷径可循。

从20世纪50年代起，我国几代学者苦苦追赶了西方发达国家半个世纪，依靠引进、学习、跟踪、改进、创新的高新技术发展思维定式曾使我们付出了高昂的学费和沉重的代价。固然，在高新技术领域，依靠“引进”和“泊来”，取得了长足的发展和进步，填补了不少国内的“空白”，但在水准上始终与发达国家保持着一段距离，一种在行业内部看来时长时短、难于逾越的差距。这种差距主要表现在对高精尖学科的发展我们缺乏思想、概念、理论、方法、技术、制度、规范和设计的原始创新和发展模式的全面创新，研究工作总是跟在别人后面亦步亦趋。我们缺乏在发展中另辟蹊径，走自己道路的机制和氛围，迷信洋人，盲目追随西方学术发展道路的习惯思维方式几乎导致国人失去了创新的机能，这是一个国家和民族发展高新技术学科和产业致命的硬伤。

高新技术领域竞争的成败，关键取决于人才与文化。现代科学技术的创新，已不单纯是一个学术问题，还是一个与文化、人文密切相关的问题。科学教育求真求是，技术教育求实求精，人文教育求灵求善，艺术教育求美求新。没有科学技术知识，人的认识和生活难免停留在原始社会，而没有人文精神和艺术的陶冶，科学技

术的创新必然失去力量的源泉。可见，走自己的道路，发展中国的科技创新体系，在某种程度和意义上，成败的关键在于大学能否真正培养一大批高素质的人才。高等学校要实现培养大量高素质计算机科学专业人才的目标，需要在前进中不断地进行系统的、科学的总结和深刻的反省，需要对遇到的问题进行科学的分析和判断，作出正确的决策。

工欲善其事，必先利其器。倘若教师不能在思想上摆脱陈旧的思维定式，用先进的理念武装头脑，勇于探索前人没有走过的发展道路，那么，即使采用了世界一流大学的全套教材，恐怕也难于培养出世界一流的人才。中西文化、人文传统之间的差异之大，中外教育思想之间的差异之大使得中国教育的现代化决不是一个引进和模仿就可以轻易解决的问题。教师的职业不是贩卖知识。授业、传道、启蒙、解惑技能的高低，不仅取决于教师知识的广博和深厚，更重要的在于远见、卓识、探索、创新、敬业、求真的本领和身先垂范。

身处21世纪，面对国家的期望，处在科学技术发展浪潮之巅的计算机科学教师，任重道远。我们就像茫茫林海中的探险者，环顾苍翠的群山，犹如身陷迷宫一样。计算机科学教育，敢问路在何方？其实，我们的出路或许只有一条，那就是系统总结前人的经验和教训，设法努力登上山峰，居高眺望，探寻走出林海的希望之路。

2001年初，《现代计算机科学与技术教材系列》编审委员会正式成立，一年后，计算机科学一级学科系列教材一体化设计研究报告的第一部分在《计算机科学》杂志第6期长篇发表。编审委员会为系列教材的出版制定了严格、详细的操作程序，并通过多种渠道，选择了第一批教材的作者。在体制创新方面本次教材出版设立了学术编审人，跟踪编审教材的创作内容，力求教材的尽善尽美。可以预期，《现代计算机科学与技术教材系列》将是一套基于计算机科学一级学科人才培养科学理论体系，体现内涵发展和科学办学要求，反映一体化设计的系列教材建设的一个尝试。然而，就像任何新生事物一样，她难免存在缺点和不足，我们诚恳地希望关心和使用本套系列教材的师生、读者，在使用中将批评或建议留下来，帮助改进教材建设工作，修正存在的不足之处。

一些学者对于一级学科人才培养科学理论体系的能行性表示怀疑。带着这个问题，在贵州大学和国内部分高等学校和一大批知名学者的支持下，从1999年夏天起在贵州大学连续举办了“计算机科学与技术高级研讨班”，向（博士）研究生和中青年教师陆续开设了研究生核心学位课程“高等计算机体系结构”、“并行算法设计基础”、“分布式算法设计基础”、“高等逻辑基础”和“形式语义学基础”，后来又进一步开设了本科生重点课程“算法设计与分析”、“数理逻辑基础”、“可计算性与计算复杂性”、“形式语言与自动机理论”等一系列课程。5年来，高级研讨班受

到全国广泛关注、响应和支持，先后吸引了几十所大学三百多人次的师生参加听讲和学习，目前已经发展为由教育部批准资助、15 所大学联合主办的高级研讨班。实践证明，高级研讨班为中国高等学校计算机科学教学改革和教育质量的提高正在发挥其独特的作用，并得到国内外一大批学者的充分肯定和好评。高级研讨班正在成为按照一级学科办学和教学改革的要求，对计算机科学系教师进行高起点、高标准、正规化研究生学位课程和本科重点课程培训的一个师资培训模式，有可能对未来计算机科学教育产生深远的影响。试想，如果高等学校的教师和培养的研究生普遍具有高级研讨班所开设的 3 门～4 门课程的共同基础，不仅科学办学面临的主要困难迎刃而解，各大学科研学术队伍的素质也将得到显著提高。

一些学者对高起点研究生学位课程的必要性提出疑问：是否这些课程都要学习？我们认为，应该看到在高等教育界从来就存在着两种不同的教育观，一种是专才教育观，一种是通才教育观。持这两种教育观的人尽管都主张基础知识的重要性，但在对学以致用原则的理解和解释方面存在差异。一般地说，专才教育观主张在一定的基础上，通过深入钻研某一方向的学问，逐步扩展和加深自己的知识，缺什么基础补什么知识，学以致用，逐步成长为一个学科的专家。通才教育观则不同，它不主张在具备一定的基础后，就匆忙沿着某一方向钻研学问，单线独进，而是主张在一级学科的范围内，通过尽可能系统地掌握从事本学科各个重要方向的研究所需要的共同的基础知识，能够站在学科的各个至高点上，沿着学科的一个方向，以单线独进、多线并进或整体推进的观点，逐步扩展和加深一级学科的知识，融会贯通，学以致用，逐步成长为一个学科的专家。两种教育观都有其代表人物。迄今为止，高等教育中研究生教育主要以培养专才为主，专才教育观是主流。但是，两种不同的教育观各有其特点。一般地说，当一个学科的发展处于早期时，专才教育比较容易跟上学科的发展步伐，比较容易出成果，也比较容易迅速地达到较深的学术层面。而当一个学科的发展比较成熟，发展速度比较平稳时，通才教育的优势就比较明显。因为，通才教育培养的人才可以在一级学科的范围内比较容易地向任何一个方向转向。特别是在胜任高难度重大创新人才的培养方面，在出综合性的大成果方面，在创立一套科学理论和开辟一个研究方向方面，通才教育的多种优势往往是专才教育所不具备的。当然，两种教育观谁优谁劣迄今并无定论，根据两种教育观的特点和现实情况，选择哪一种教育观实际仅反映了师生的一种选择策略。不过，实践告诉我们，尽管通才教育观的操作实现比较困难，但作为师资补充的来源，通才教育培养的人才更容易适应大学教学与科研的双重要求，理应更多地受到研究大学的青睐。在科学技术日益深化、高度分化又高度综合的今天，放眼未来，在高精尖学科中，通才教育观无疑有着更为宽广而美

好的发展前景。

伴随着学科教学改革理论研究与实践探索的推进，社会热切地期待着一套与教学改革方案相配套的高质量系列教材问世。总结过去教材建设成功和失败的经验和教训，使我们清楚地认识到：教材建设必须建立在科学研究基础之上，按照科学的运作程序，动员在第一线从事科学研究、功底深厚、学有所长的优秀教师参与到教材的创作中来，才有可能推出高质量并符合学科发展要求的系列教材。我们的主张是："让大学中的科学家来创作教材。"

冬去春来，年复一年。当我们终于从跟踪、学习、盲从西方大学教程的发展模式中走出时，感受到了一种从未有过的释然与激动，一种走自己的发展道路，独立自主的自豪与喜悦。这条道路虽然艰难，但前景光明。连续5年在贵州大学举办的全国计算机科学与技术高级研讨班的成功实践更进一步地坚定了我们对内涵发展模式与科学办学之路的认识与追求。

今天，经过编审委员会、作者和出版社的共同努力，《现代计算机科学与技术教材系列》终于开始陆续出版发行。在新年即将到来的时候，我们怀着喜悦的心情，向祖国和人民，向计算机科学界献上一份完全由华夏学者按照他们对高等教育和计算机科学的理解与认识，倾力创作的新年贺礼，一份建立在科学研究基础之上的教育与教学改革最新成果。

新的世纪已经到来，愿《现代计算机科学与技术教材系列》的出版能够为新一代的莘莘学子攀登现代科学技术的高峰成就未来。

《现代计算机科学与技术教材系列》编审委员会

2006年10月

前　言

算法是计算机科学的灵魂，Turing 奖获得者，算法大师 Donald E. Knuth 说过计算机科学的研究就是算法的研究。的确，计算机科学的每个领域，不管是软件、硬件还是具体的应用，例如集成电路的设计、操作系统的内存调度，计算机网络的路由问题、计算机图形用户界面等，都与算法密不可分。在某个领域算法上的突破，直接关系到该领域的突破和进展。到 2008 年底为止，在 55 名 Turing 奖（图灵奖，被称为计算机科学界的诺贝尔奖）获得者中，因为算法方面的贡献而获奖的就有 30 多个。可见算法的研究对推动计算机科学的发展起着至关重要的作用。

在厦门大学算法设计与分析课程的教学过程中，作者曾选用 Introduction to algorithms 以及 Algorithms Design Techniques and Analysis 这两本经典而且权威的英文教材，同时吸收其他算法教材的优点，这奠定了本书的内容和风格。本书将目前计算机科学领域出现的一些经典以及新颖的算法设计和分析技术合理地组织起来，进行一个全面的介绍，旨在帮助读者掌握基本的算法设计与分析技术，提高解决问题和分析问题的能力，进而对实际问题设计出简单有效的算法。

本书的主要内容如下：

第 1 章从问题入手，介绍了算法的基本概念及性质，计算模型的概念，以及如何具体分析一个算法的时间复杂度和算法正确性的证明方法，最后介绍如何衡量算法的效率，以及问题下界的概念，以便使读者能够明白，一个算法能否继续改进，何时才能达到最优。第 2 章介绍了分析算法复杂度所需要用到的渐近符号及其直观含义。第 3 章介绍了算法分析方法，具体介绍常用的概率分析以及新近出现的分摊分析和实验分析方法。第 4 章介绍了递归算法的设计以及递归算法的证明方法，并介绍求解递归算法时间复杂度的方法。第 5 章介绍了分治算法及其在排序、快速 FFT 等问题中的应用。第 6 章介绍了动态规划算法的基本过程及性质，其中最优子结构性质是一个问题能够用动态规划算法求解的必要条件；而重叠子问题性质是提高动态规划算法效率的关键。第 7 章通过任务选择问题引入贪心算法，介绍了贪心算法的过程，以及贪心选择性质的分析方法。第 8 章介绍了基本的图算法，图算法是网络流和匹配算法的基础。重点强调了分摊分析在图算法时间复杂度分析中的应用以及图算法正确性的证明方法，同时进一步巩固贪心算法以及动态规划算法的应用。第 9 章是图算法的进一步扩展，介绍了网络流中最大流问题、最小费用流问题以及匹配问题的求解算法。第 10 章介绍了计算机、经济以及

管理领域中经常用到的线性规划问题及其单纯形求解算法。第11章介绍了NP完全理论，具体介绍了P、NP和NPC的定义以及问题的分类，NP完全问题的证明方法。第12章介绍了求解NP-hard问题的回溯算法，通过回溯算法在具体问题中的应用，特别是剪支函数的设计，分析了提高回溯算法效率的关键。第13章介绍了基于宽度优先，同时借鉴回溯算法优点的分支限界算法，特别是剪支函数的设计。第14章介绍了一些启发式搜索算法的设计，特别详细介绍了A*搜索以及博弈搜索。第15章介绍了数论的基础知识、基本的数论算法及其应用。第16章介绍了计算几何的常用算法，例如线段交叉算法、凸包算法以及较新的内容Voronoi图及其应用，其中许多算法都利用了分治算法的思想。

本书具有如下特色：

（1）本书基本上涵盖了ACM国际大学生程序设计竞赛所要考查的内容。

（2）算法的正确性是软件可靠性的保证，本书为每种类型的算法给出其正确性的证明思路，这在大多数算法书中是看不到的。

（3）本书精选了大量的习题，读者可以有选择性地练习。同时，精选了ACM国际大学生程序设计竞赛的一些习题，作为课后实验，读者可以登录厦门大学XOJ：acm.xmu.edu.cn以及北京大学POJ:acm.pku.edu.cn进行练习。

（4）本书所讲解的算法以及大部分应用实例，都已经作为acm.xmu.edu.cn上的题目。

（5）本书不仅注重算法的应用设计，而且注重算法时间复杂度的分析，大部分算法都提供了具体例子，并详尽分析，以帮助对算法的理解。

（6）本书还提供英文课件，便于双语教学。中英文课件及各章习题的参考答案可从http://59.77.16.229上下载。

本书编写过程中，得到了同事赵致琢教授的关心。在2008年ACM杭州赛区现场，华南师范大学的师兄陈卫东副教授从专业的角度为本书提供了宝贵的建议。武汉大学著名ACM教练董文永教授从竞赛培训的角度提供了一些建议。本书内容曾在厦门大学计算机系2002级、2003级、2004级、2005级、2006级本科生中试用，也作为研究生选修课程试用。有些本科生、研究生为本书内容的完善提出了一些有益的建议。还有我的研究生魏丽军、陈青山、彭煜、陈竞驰、刘永凯、蓝旗花、周细岳、郑捷敏、林阳斌、吴博、华娴，ACM队员张真真、项光特以及沈健也阅读了部分章节，并进行了校对。魏丽军、彭煜、陈竞驰、张驰、叶志梅、林阳斌、项光特、沈健、张真真等，都是我的ACM队员，他们都上过我的算法课程，参加ACM国际大学生程序设计竞赛，获得了2金5银的好成绩，为厦门大学获得了荣誉，也为本书实验题目的选择提供了帮助。叶志梅、项光特、林阳斌还分别为本书的实验写了标程，陈锦辉写了部分ACM题目的解题报告。特别是项光特同学为本书的实验付出了辛勤的汗水，使得实验课程得以在厦门大学OJ系统acm.xmu.edu.cn上顺利进行。本书的习题解答也得到了本课程助教刘艳娟、陈小芬、李琳、曾丽群等研究生

们的整理。在此对他们一并表示感谢。我还要特别感谢我的妻子罗淀，她对书稿中的文字进行了录入和校对，还建议书不要写得太深奥复杂，这奠定了本书通俗易懂的风格。没有她的帮助，本书不可能顺利出版。

此外，本书的建设还得到厦门大学第二批双语教学项目“算法设计与分析课程”的资助。

限于水平和经验，书中难免有不妥和错误之处，还望广大读者指正，在此先表感谢。

谨以此书献给所有关心、鼓励和帮助过我的人们。

张德富
厦门大学计算机科学系
hustzdf@ sina. com

第1章 入 门

1.1 问 题

在日常生活中会碰到各种各样的问题，从众所周知的职业选择问题、投资决策问题、排序问题，到人类基因工程、网络安全、电子商务、战争等领域出现的各种比较深奥的问题，大都涉及到如何选取一个目标，在满足一定的约束条件下，使目标达到最优。为了研究方便，我们可以把这些问题形式化描述如下：

$$f(x) \qquad x \in S$$

其中，$f(x)$是一个目标函数，它表示解决问题的目的。如果目标是使问题达到某种最优，则称为最优化问题，例如最大化问题，它取目标函数的最大值。S称为解空间或搜索空间。$x \in S$称为约束条件。满足问题约束条件的解x称为可行解。使目标函数取得某种最佳值的可行解称为最优解。有些问题需要找出最优解，而有些问题只需要找到一个可行解即可。

当然实际生活中碰到的问题是具体的，而上述问题模型是抽象的。如何为一个实际问题建立一个数学模型，以方便求解，是一个算法高手所必须具备的素质。

一个具体的问题，常包括解决该问题所需要的所有已知条件，我们常把这个具体问题用问题实例来描述。问题实例有大有小，问题实例的大小称为问题的规模，它常用来描述解决一个问题实例的复杂程度。例如，对n个数排序，问题规模为n，有关图$G=(V,E)$的问题，其规模为图的大小，即V和E的大小。求解一个问题，通常是根据问题的已知条件（输入实例），求出问题的解（输出），因此算法也可以看作是输入实例与输出之间的函数。有些问题很难，而有些问题很容易。即使同样的问题，规模不一样，其难易程度也是有区别的。例如5个整数的相乘，跟10000个整数的相乘，两个问题的难易就不同。有些问题可解，而有些问题是不可解的。关于可解的概念，我们稍后给出。计算复杂性的概念，我们将在以后讨论。

1.2 算法的概念

给定一个问题，如何解决它是至关重要的。我们必须设计一个求解该问题的方法，也就是算法（Algorithm）。算法是解决问题的一种方法或一个过程，是一个由若干运算或指令组成的有穷序列。它一般具有下列特点：

(1) 有穷性:算法是有限的运算或指令序列。

(2) 可行性:每个运算都是可执行的。

(3) 确定性:每个运算有确定的含义,无二义性。

(4) 输入:0 个或多个输入,以描述运算对象的初始情况。

(5) 输出:1 个或多个输出,以描述对输入进行处理后的结果。

这里我们介绍的算法是确定性算法的定义,即对于同一个输入,不管算法运行多少次,得到的输出都是一样的。如果输出可能不一样,则称为非确定性算法(或称随机算法)。对于给定的某个问题,我们设计出一个求解该问题的算法,如果对该问题的所有实例,算法都能得到正确的输出,则我们说该算法是正确的。对一个问题,我们必须设计出一个正确的算法才有意义。当然,偶尔得到错误的输出结果的算法也是有用的,例如信息安全中测试一个大整数是否为素数的随机算法,对于一个实例,算法可能输出错误的结果。

下面我们给出一个算法设计实例。

考虑排序问题,即给定 n 个元素序列$\langle a_1, a_2, \cdots, a_n \rangle$,问题的目标是输出一个重新排过序的序列$\langle a'_1, a'_2, \cdots, a'_n \rangle$,序列值满足 $a'_1 \leqslant a'_2 \leqslant \cdots \leqslant a'_n$。

排序问题是一个经典问题,日常生活中随处可见。例如打扑克牌,假设左手握牌,右手从桌面取牌,插牌的目的是使左手中牌的大小从左到右,由小到大排列。初始时,左手为空,每当我们从桌上取一张牌,我们就会将该牌与左手中的牌依次进行比较,直到找到一个位置,使得插入的牌比左边的牌大,并不会比右边的牌大即可,如图 1.1 所示。

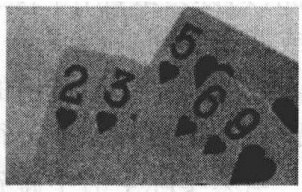


图 1.1

从日常打牌中得到启发,我们可以设计出一个自然而简单的插入排序(Insert Sort)算法:

```
InsertSort(A)
1  for j←2 to n do
2      key←A[j]
3      i←j-1
4      while i>0 and A[i]>key do
5          A[i+1]←A[i]
6          i←i-1
7      A[i+1]←key
8  return A
```

InsertSort(A)算法中,A 是一个数组 $A[1..n]$,表示给定的输入实例,n 是输入实例的大小。InsertSort(A)算法用伪代码描述,其中一些关键字,例如 **for**, **to**, **do**, **while**, **repeat**, **if**, **then**, **else** 等用黑体表示,代码中每个块结构用嵌套结构表示,可以方便地表示某个块结构内执行的语句。例如第 1 行的块结构 **for** 循

环，要执行第2、3、4、7行的语句，而第4行的块结构 **while** 循环，要执行第5、6行的语句。

InsertSort(A)算法的第1行对放在桌上的牌逐个取出，*key* 表示当前要插入的牌，当前的目的是要将 $A[j]$ 插入到已经有序的序列 $A[1..j-1]$ 中。第4行 **while** 循环的目的是要在有序序列 $A[1..j-1]$ 中找一个位置，以便 *key* 能够插入其中。*key* 从右至左逐个与 $A[i]$ 进行比较，如果 $A[i] > key$ 则表示 *key* 要插入到 $A[i]$ 的前面，因此 $A[i]$ 往右移动一个位置，然后 i 往左移一个位置。重复这个过程，直到找到一个元素使得 $A[i] \leqslant key$ 或者 $i=0$ 时停止循环。最后，在第7行将 *key* 插入到 $A[i+1]$。

例如，给定一个实例，如图1.2(a)所示。插入排序算法的运行过程如图1.2所示。图中阴影部分是当前要插入的数组元素。对于图1.2(a)，要插入的元素为2，2比 $A[i]$ 小，因此元素5往右移动一格，i 减1，此时 $A[i]$ 为空，停止执行第4行的 **while** 循环，将2插入 $A[i+1]$。继续执行 **for** 循环，此时 i,j 均加1，得到图1.2(b)。目前要插入的元素为4，4比 $A[i]$ 小，因此元素5往右移动一格，i 减1，此时 $A[i]$ 为2，比4小，将4插入到 $A[i+1]$ 位置。继续执行 **for** 循环，此时 i,j 均加1，如图1.2(c)所示。当前要插入的元素为6，6比 $A[i]$ 大，不移动，得到图1.2(d)。目前要插入的元素为1，1比 $A[i]$ 小，因此元素6往右移动一格，元素5往右移动一格，元素4和2依次往右移动一格，将1插入到 $A[1]$，得到图1.2(e)。当前要插入的元素为3，3比 $A[i]$ 小，因此元素6、5和4依次往右移动一格，将3插入到 $A[3]$，得到图1.2(f)。由于桌上所有牌都已经插入，因此算法终止。

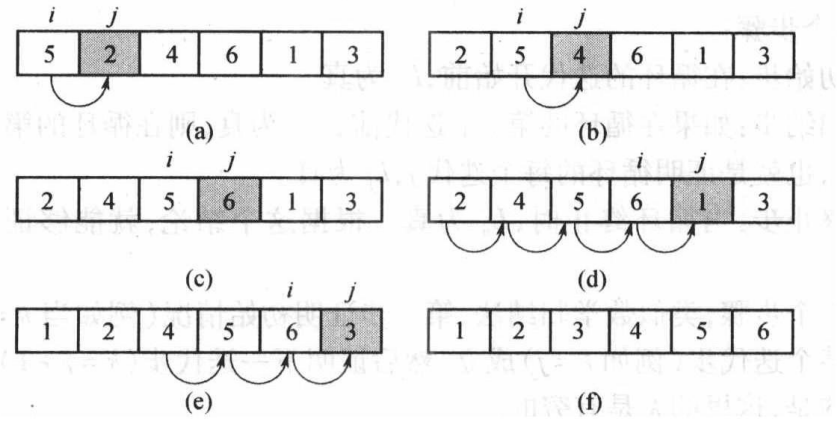


图1.2

1.3 算法的正确性

从上一节算法正确性(Correctness)的定义可以看出，要证明算法的正确性，我们只需证明对任意一个输入，算法能得到一个正确的输出。下面，我们证明 InsertSort(A)算法的正确性。

图 1.2 给出了 InsertSort(A)算法执行一个实例的过程。索引 j 表示当前要插入的牌的位置。在 **for** 循环每个迭代执行前,$A[1..j-1]$表示当前左手中已经排好序的牌。$A[j+1..n]$表示桌上待插入的牌。事实上,$A[1..j-1]$中的元素最初在位置 1 到 $j-1$,不过现在这些位置上的元素是有序的,即

在 **for** 循环第 j 个迭代执行前,子数组 $A[1..j-1]$由最初 $A[1..j-1]$中的元素构成,不过现在 $A[1..j-1]$是有序的。

我们把 $A[1..j-1]$中元素有序的这个性质称为循环不变量(Loop Invariants)。因此循环不变量是与程序变量(例如 $A[1..j-1]$)有关的一个语句,它在循环刚开始前,以及在循环的每个迭代执行后为真,特别是在循环结束后,仍然为真。下面利用循环不变量来证明算法的正确性。

令这个与循环的第 j 个迭代有关的循环不变量为 L_j,我们的目标就是证明在循环的每个迭代 j,L_j 为真。例如,对于 InsertSort(A)算法,L_j 为:在 **for** 循环第 j 个迭代执行前,子数组 $A[1..j-1]$由最初 $A[1..j-1]$中的元素构成,不过现在 $A[1..j-1]$是有序的。

这样在执行循环的第 $n+1$ 个迭代前,如果 L_n 为真,就能够证明插入排序算法的正确性。在证明之前,我们先介绍一下,利用循环不变量证明一个算法正确性的步骤。

我们能够利用循环不变量来证明一个算法为什么是正确的,证明的关键是寻找到循环不变量,即某个特性 L_j,然后证明循环不变量 $L_j(1\leqslant j\leqslant n)$为真,这显然可以利用类似数学归纳法的证明方法。因此要利用循环不变量证明一个算法是正确的,包括三个步骤:

(1) 初始步:在循环的迭代开始前,L_1 为真。

(2) 归纳步:如果在循环的第 j 个迭代前,L_{j-1}为真,则在循环的第 $j+1$ 个迭代前,L_j 为真,也就是证明循环的每个迭代 j,L_j 为真。

(3) 终止步:当循环终止时,L_n 为真。根据这个结论,就能够证明算法的正确性。

上述三个步骤,类似数学归纳法,第一步证明初始情况(例如当 $k=1$)成立,第二步假设某个迭代步(例如 $k=j$)成立,然后证明下一迭代步($k=j+1$)成立。与归纳法不同的是,这里的 k 是有穷的。

今后我们在证明算法的正确性时,一般采用循环不变量来证明。下面,我们利用循环不变量来证明 InsertSort(A) 算法的正确性。对任意的一个输入实例 $A[1..n]$:

(1) 初始步:当执行 **for** 循环的第 2 个迭代之前,此时 $j=2$,$A[1..j-1]$只有一个元素 $A[1]$,它显然是有序的,因此 L_1 为真。

(2) 归纳步:在执行 **for** 循环的第 k 个迭代前,此时 $j=k$,如果 L_{k-1}为真,即 $A[1..k-1]$是有序的。当执行迭代 $j=k$ 时,当前要插入的元素是 $key=A[j]$,反复

执行 **while** 循环，直到停止循环。最后，在第 7 行将 *key* 插入到$A[i+1]$。此时$A[1..k]$有序，因此，在下一个迭代$j=k+1$之前，L_k为真。

（3）终止步：当算法终止时，此时$j=n+1$，由于第二步的保证，因此，$A[1..n]$有序，且$A[1..n]$由最初$A[1..n]$中的元素构成，即L_n为真。此时，整个数组 A 都已经有序，这意味着对任意的输入 A，InsertSort(A)算法都得到一个正确的输出，因此 InsertSort(A)算法是正确的。

1.4 算法的效率

对于给定的问题，起初只是考虑只要找到一个算法解决该问题就行，而不管该算法花多长时间或者占用多大的存储空间。随着计算机技术的发展，人们发现，求解同一个问题的不同的算法，所需要的运行时间是不同的。例如对排序问题，除了上面介绍过的插入排序算法，我们还将介绍选择排序、合并排序以及快速排序算法。我们自然会问，哪个排序算法好呢？怎么样评价一个算法的好坏呢？这就涉及到算法的时间和空间资源的分析，即算法效率（Efficiency）的分析。

算法效率的分析，指的是算法求解一个问题所需要的时间和空间。分析一个算法，意味着预测该算法解一个问题时，需要花费多少时间和空间资源。空间资源一般指解决一个问题，需要多大的内存、硬盘空间等来存储输入输出数据等。时间资源一般指的是解决一个问题需要多长的时间。一般地，对一个问题，存在不同的求解算法，我们可以根据算法所需要的时空资源来确定出其中有效的算法。随着计算机硬件技术的发展，现在计算机的内存和硬盘空间基本上能满足问题的需要，即空间资源已经不是问题，因此，我们分析一个算法，主要考虑算法的计算时间，今后，我们也主要从时间资源方面对算法进行分析，即分析算法有多快。

在分析一个算法前，我们需要知道如何实现一个算法，这就涉及到计算模型的概念。如果对一个问题，利用计算模型能够实现一个求解算法，则该问题是可解的，否则是不可解的。排序问题是一个可解的问题，因为我们已经找到了许多排序算法。对于不可解的问题，由于无法找到其求解算法，对其进行算法研究也就没必要，本书探讨可解问题的算法设计与分析。

计算模型是一种实现算法的设备，比如最古老最简单的算盘、现在的计算机等。著名的抽象计算模型有歌德尔的递归函数、Turing 机以及 RAM（随机存储器）等。这些抽象计算模型在计算能力上是等价的。也就是说如果一个问题在其中的一个计算模型上是可解的，那么在其他模型上也是可解的。下面，我们考虑 RAM 计算模型作为实现算法的设备。在 RAM 模型中，指令的执行是串行的，没有并行运算。同时，RAM 包含一些通常在现实计算机上都具有的一些基本指令，例如算术运算、数据移动以及控制指令，每条这样的指令都花常数时间。虽然算法一样，但是如果计算模型不一样，算法解一个问题所花费的时空资源也是

大不相同的。一台286计算机跟现在的奔腾5计算机，用同样的算法解同样一个问题实例，显然后者会算得更快。因此，分析比较不同的算法必须在同样的计算模型下才有意义。

在一个计算模型上分析一个算法，是一件非常困难的事情。所需要的数学知识包括组合数学、概率理论、代数技巧以及在数学公式中确定主要项的能力。由于对每个输入实例，算法的运行行为可能都不同，因此我们需要一种办法来度量算法运行的行为。这就是下面要介绍的算法运行时间的估算。一个算法关于一个给定实例的运行时间是该算法执行基本运算（或步数）的数目。其中，基本运算应尽可能地依赖于给定的计算模型，并采用如下约定：在算法伪代码中，执行一行代码仅花常数时间。当然，不同行所花的常数时间可能不同，我们假设执行第 i 行花常数时间 c_i。这种假设与RAM一致，也反映了伪代码是如何在大多数现实的计算机上实现的。

有了上面的假设，我们可以分析一个算法的运行时间与问题规模之间的关系。一般地，度量一个算法的运行时间，有三种方式：

（1）最好情形时间复杂度。对任何规模为 n 的实例，算法求解某个实例所需要的最少基本运算次数。它表示存在某个实例，算法求解它所需要的基本运算次数最少。

（2）最坏情形时间复杂度。对任何规模为 n 的实例，算法求解某个实例所需要的最多基本运算次数。它表示存在某个实例，算法求解它所需要的基本运算次数最多。

（3）平均情形时间复杂度。对任何规模为 n 的实例，算法求解任意一个实例所需要的平均基本运算次数。它表示对任意一个实例，算法求解它，平均需要多少基本运算次数。一般都是通过将所有实例的计算时间求出来，然后取平均，从而得到求解任意一个实例所需要的平均基本运算次数。

下面对InsertSort(A)算法分析一下算法的时间复杂度。InsertSort(A)算法执行每一行所花的时间（Cost）显示如下：

	InsertSort(A)	Cost	Times
1	**for** $j \leftarrow 2$ **to** n **do**	c_1	n
2	$key \leftarrow A[j]$	c_2	$n-1$
3	$i \leftarrow j-1$	c_3	$n-1$
4	**while** $i>0$ and $A[i]>key$ **do**	c_4	$\sum_{j=2}^{n} t_j$
5	$A[i+1] \leftarrow A[i]$	c_5	$\sum_{j=2}^{n} (t_j-1)$
6	$i \leftarrow i-1$	c_6	$\sum_{j=2}^{n} (t_j-1)$
7	$A[i+1] \leftarrow key$	c_7	$n-1$
8	**return** A	c_8	1

上述伪代码中执行第 i 行花费常数时间 c_i，每行执行的次数(Times)显示在最右边一列，其中 t_j 表示在 **for** 循环的迭代 j 中，**while** 循环执行的次数。因此只要把 InsertSort(A)算法执行每行的次数与执行每行所需要的时间相乘，然后把所有时间求和，便可以得到算法实际运行的时间，即

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^{n} t_j + c_5 \sum_{j=2}^{n} (t_j - 1) + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7(n-1) + c_8 \tag{1.1}$$

由上述分析可知，对于插入排序算法，排序 5 个数跟排序 10000 个数，其运行时间是不一样的。一般地，算法运行的时间 $T(n)$ 随着问题规模 n 的增大而增加，因此很自然地把算法的运行时间 $T(n)$ 描述成关于问题规模 n 的一个函数。

对于排序问题，即使对于同样规模的问题实例，如果一个实例已经排序好，而另一个没有排序好，InsertSort(A)算法的运行时间也有可能不同。考虑一个由小到大已经排好序的问题实例，由于已经排好序，第 4 行 **while** 循环在 **for** 循环的每次迭代中只执行一次，即 $t_j = 1$，第 5 行和第 6 行根本不执行，将 $t_j = 1$ 代入式(1.1)，可得

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1) + c_8 \\ &= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7 - c_8) \end{aligned}$$

由于 c_i 为常数，因此 InsertSort(A)算法的运行时间 $T(n)$ 可以表示为关于问题规模 n 的线性函数，这是算法表现最好的情形。

如果给定的问题是逆序排列，即数组元素从大到小排列。这时待插入的元素 $A[j]$ 不得不与已经排好序的子数组 $A[1..j-1]$ 中的每个元素进行比较，对于给定的 j，**while** 循环执行的次数为 j，即 $t_j = j(j = 2, \cdots, n)$，将 $t_j = j$ 代入式(1.1)，可得

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^{n} j + c_5 \sum_{j=2}^{n} (j-1) + c_6 \sum_{j=2}^{n} (j-1) + c_7(n-1) + c_8$$

整理可得

$$T(n) = \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}\right)n^2 + \left(c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7\right)n - (c_2 + c_3 + c_4 + c_7 - c_8)$$

因此，InsertSort(A)算法的运行时间 $T(n)$ 可以表示为关于问题规模 n 的二次函数，这是算法表现最坏的情形。

上面我们分析了插入排序算法的最好和最坏两种情形，当然在今后某些算法分析中，例如随机算法，还将碰到平均(或者期望)情形分析。但是，我们今后将主

要考虑最坏情形分析，即对于任何规模为 n 的实例，算法运行时间最长的情形。这是因为

（1）最坏情形是任何规模为 n 的问题实例运行时间的上界，即任何规模为 n 的实例，其运行时间都不会超过最坏情形的运行时间。知道最坏情形运行时间后，我们就知道算法最差到什么程度。

（2）对某些算法，最坏情形经常发生。例如在某个数据库中查询不存在的某条数据。

（3）平均情形有时跟最坏情形差不多。例如，对于插入排序算法，随机选择 n 个数进行排序，假设在迭代 j，**while** 循环平均执行 $\frac{j}{2}$ 次，即 $t_j = \frac{j}{2}$，将其代入式(1.1)，我们仍然可以计算出 InsertSort(A)算法的平均运行时间 $T(n)$ 可以表示为关于问题规模 n 的二次函数，与最坏情形差不多。

从上面的分析我们也可以看出，分析一个算法的运行时间是非常复杂的。要比较两个算法运行时间的快慢，如果把每个算法的运行时间都具体表示出来，显然很复杂。因此，是否有更简单的表示方式以方便进行算法的比较呢？考虑插入排序算法的最好情形 $T(n) = cn - b$，这里 $c = c_1 + c_2 + c_3 + c_4 + c_7$，$b = c_2 + c_3 + c_4 + c_7 - c_8$ 是固定的常数，因此，算法运行时间随着问题规模 n 的增大而呈线性增长。随着规模 n 的增大，算法的运行时间主要由规模 n 决定，与常数项没有多大关系。

现在我们比较两个排序算法的效率，假设解排序问题的插入排序算法的运行时间为 $T(n) = c_1 n^2$，假设解排序问题的合并排序算法的运行时间为 $T(n) = c_2 n\lg n$，其中，合并排序算法将会在分治算法一章进行介绍，与数学符号表示不同的是，本书中 $\lg n$ 表示以 2 为底的对数。现在考虑两台计算机，计算机 A 每秒(s)执行 10^9 次运算，而计算机 B 每秒执行 10^7 次运算。可见计算机 A 比计算机 B 快 100 倍。取 $c_1 = 5$，$c_2 = 50$。考虑规模 $n = 10^6$ 的一个实例 I，我们现在用快的计算机 A 来执行插入排序算法，对于排序实例 I，在计算机 A 上，插入排序算法需要执行

$$\frac{5 \times (10^6)^2 \text{ 次运算}}{10^9 \text{ 次运算/s}} = 5000\text{s}$$

对于同样的问题实例，在计算机 B 上，合并排序算法需要执行

$$\frac{50 \times 10^6 \lg 10^6 \text{ 次运算}}{10^7 \text{ 次运算/s}} \approx 100\text{s}$$

这表明用一台速度快的计算机执行一个插入排序算法，其所需要的时间是慢的计算机执行合并排序算法所需时间的 50 倍。当一个排序问题实例的规模为 10^8 时，插入排序需要大约 13888.89h(这里 h 表示小时)才能得到答案，而合并排序算法则只需大约 3.69h。这主要是因为，插入排序算法的运行时间随着问题规模的增

大，其增长速度比合并排序算法快很多。由此可见，插入排序算法的运行时间取决于 n^2，而合并排序算法的运行时间取决于 $n\lg n$，常数 c_1，c_2 对算法运行时间的影响，与问题规模相比是微不足道的。而且不管 c_1 比 c_2 小多少，总存在某个自然数 n_0，当 $n > n_0$ 时，$c_1 n^2$ 比 $c_2 n\lg n$ 大。这表明求解排序问题的合并排序算法比插入排序算法的效率更高。对于更复杂的算法运行时间表达式，除了高阶项外，还涉及到许多常数项和很多低阶项，这里高阶项可以看作运行时间表达式里的主要项，其他的可看作低阶项，例如 $T(n) = c_1 n^2 \lg n + c_2 n^2$，主要项 $n^2 \lg n$ 可以看作是高阶项，n^2 是低阶项。由此可见，算法的时间复杂度取决于主要项。

上面的例子表明，算法的效率主要取决于算法本身，与计算模型（例如计算机）无关，这样可以通过分析算法的运行时间，从而比较出算法之间的快慢。但是实际上，许多算法是很难分析出时间复杂度的。因此，一种比较简单的方法是实验比较，即将算法写成程序，然后在程序的前后调用时间函数，通过在计算机上实现，算出得到计算结果所花的时间，然后根据算法所需时间的长短来确定算法的快慢。对于实验分析，下一章将会详细介绍。当然还有比较高级的随机算法与近似算法，不仅需要考虑算法的效率，还要关心解的质量，即分析算法的性能，其性能的比较会更复杂。

1.5 问题的下界

有些问题，例如著名的停机问题（对任意的输入，判断任意的算法是否会在某个时间停止的问题），根本无法用计算机来解决。因此我们下面讨论的都是能在计算机上得到解决的问题，即可计算的问题。即使是可计算的问题，不同的问题，其难易程度也是不一样的。即使是同一个问题，在很多情况下，也可以设计出具有不同效率的算法。所有这些都是计算复杂性所关心的问题。计算复杂性理论就是用于研究算法有效性和问题难度的一种工具，它是最优化问题的基础，涉及如何判断一个问题的难易程度。事先了解一个问题是否为难解问题自然会有很多好处。因为只有了解所研究问题的复杂性，才能更好地有针对性地设计有关算法，提高计算的效率。

直观地，如果一个问题，具有多项式时间求解算法，即该问题的时间复杂度可以表示为关于问题规模的一个多项式函数，则该问题属于简单问题（例如 P 类问题），否则该问题属于困难问题（例如 NP-hard 问题）。关于 P 和 NP 的定义，第 11 章将会给出，这里就不再详细叙述。

前面我们介绍过，一个问题，有许多不同的求解算法。下面，我们关心的是求解一个问题的任意算法中，怎样确定一个算法是最有效的算法。要回答这个问题，我们必须了解问题的下界（Lower Bounds），即任何一种算法解决一个问题所必需的最小运行时间。例如求 n 个元素最大值的问题，其下界为 cn，求两个矩阵相乘的

问题，其下界为 cn^2。当然，下界是不唯一的，例如排序问题的下界可以是常数时间 c，也可以是线性时间 cn，但是我们的目的是需要找有意义的下界，例如排序问题的下界 $cn\lg n$ 就是有意义的、最大的下界。而 cn^2 就不是排序问题的下界，因为存在堆排序算法其最坏情形时间为 $cn\lg n$。

知道了问题的下界，我们就可以方便地知道求解该问题的一个算法是否是最优算法，是否还有必要继续研究，以便寻求更优的算法。假定一个问题的下界为 $F(n)$，而当前解决该问题的最好的算法为 A，其最坏情形时间复杂度为 $W(n)$，如果 $F(n)=W(n)$，则认为 A 已经是最优算法，没有必要对其进行改进了。否则可以继续进行研究，例如问题的下界是否可以提高，算法 A 能否继续改进，使得 $W(n)$ 更小？因此，如果所解问题的下界比解决该问题的当前最好算法的时间复杂度低，那么可以通过提高问题的下界，或者也可以通过改进算法以便降低时间复杂度，或者两方面同时进行改进。如果问题的下界等于当前算法的最坏情形时间复杂度，那么下界和算法都不能再改进了。算法已经是一个最有效算法，下界也已经是最大下界。

例如，考虑在数组 A 中找最大元素的算法：

```
FindMax(A)
1  max←A[1]
2  for j←2 to n do
3      if A[j] > max then
4      max←A[j]
5  return max
```

FindMax(A)算法在最坏情形下至多需要比较 $n-1$ 次，即 $W(n)=n-1$。假定数组 A 中的元素两两不同，在数组 A 中找出最大元素，至少需要比较 $n-1$ 次，即 $F(n)=n-1$。因此 FindMax(A)算法是最有效算法，没必要对其进行改进。

从问题下界的描述可知，要想找出所有的算法以确定问题的下界，通常比找出有效的算法更难。

1.6 小　结

前面我们介绍了插入排序算法的设计与分析。如果将插入排序算法用具体的程序设计语言实现，就得到了程序。因此，程序与算法是有区别的，程序是用某种程序设计语言实现的算法，而算法是抽象的，它不依赖于具体的程序设计语言和硬件，也就是说，无论算法是用 C 语言编写在 586 上运行还是用 Basic 语言编写在笔记本电脑上运行，算法的思想都是一样的。Turing 奖获得者，Pascal 语言之父 Niklaus Wirth 曾说过“程序 = 数据结构 + 算法”。算法是程序背后的思想，是程序的核心。从这里可以看出，程序的本质是算法，是算法的具体体现。因此描述一个算法，我们不用具体的程序，而是用伪代码，主要是因为用伪代码描述的算法非常清

楚，而且这种表示可以方便地转化为各种语言编写的具体程序，例如 C、Java 等。因此，下面我们描述算法的时候，除非特别说明，一般采用伪代码。

从上面的分析可以看出，我们分析一个算法的运行时间应该主要关注与问题规模有关的主要项，其他低阶项，甚至主要项的常数系数都可以忽略。因此，我们必须引进新的记号来表示时间函数，以及时间函数随着问题规模增长的快慢程度。这些是下一章要介绍的内容。

本章部分内容取材于文献[1]，关于上界的内容参考了文献[2]。如何寻找问题的下界，读者可以参考文献[2,3]，这里就不再详细介绍。关于算法更高级的主题，例如随机算法、近似算法等，有兴趣的读者可以参考文献[4]。

习题

1.1 给出一个现实生活中的例子，该例子需要最优解才能满足问题的需要。另外给出一个近似最优解就足够满足问题需要的例子。

1.2 给定一个算法，其输入是一个整数集 S 和一个整数 m，输出是和为 m 的所有 S 的子集，算法的具体步骤如下：

(1) 列出 S 的全部子集，并求它们的和。

(2) 逐个查看步骤(1) 列出的子集，把每个和等于 m 的子集输出。

上述算法是否满足算法的特点？请说明理由。

1.3 给定一个数组 $A=\{15,12,1,3,7,19,12,15\}$，类似图 1.2，给出 InsertSort($A$) 的执行过程。

1.4 修改 InsertSort(A)，使之能对输入的任意序列，可以输出一个降序的排列。

1.5 类似插入排序算法运行时间的分析，分析找 n 个数最大值的算法 FindMax (A)的时间复杂度。

1.6 证明算法 FindMax(A)的正确性。

1.7 给出一个算法，使之能输出数组 $A[1],A[2],\cdots,A[n]$ 中最后出现的最大元素的下标。

1.8 利用循环不变量证明下列计算 a^n 算法的正确性：

```
Exp(a,n)
1  i←1
2  pow←1
3  while i≤n do
4        pow←pow × a
5        i←i+1
6  return pow
```

1.9 给定 n 个数，请设计一个找指定数 x 的算法，分析其时间复杂度并证明算法的正确性。所设计的算法是最有效算法吗？

1.10 设有两个在同一台机器上实现的算法，运行时间分别为 $100n^2$ 和 2^n。当 n

取何值时，前者比后者快？

1.11 假设在同一台计算机上比较插入排序和合并排序算法的效率。对于规模为 n 的输入，插入排序要运行 $8n^2$ 步，而合并排序要运行 $64n\lg n$ 步。当 n 取何值时，插入排序的效率要优于合并排序？

实验题

1.12 完成 XOJ 如下题目：1000，1001，1002，1003，1004。

第2章 渐近符号

前面我们已经介绍了如何计算一个算法的具体运行时间，从简单的插入排序算法的分析我们可以看出，要计算算法的运行时间是非常困难的，即使能分析出来，运行时间的表达式也非常复杂。一个算法的运行时间主要取决于问题的规模，对于足够大的输入，算法运行时间表达式里的那些低阶项以及高阶项的常数系数等，主要受高阶项所制约。为了表示算法的渐近有效性，我们需要引进新的符号，即渐近符号(Asymptotic Notation)，以便表示算法的运行时间与输入规模之间的主要关系，即渐近时间复杂性，来衡量算法的好坏。

由于输入实例的规模是整数，因此我们用来刻画算法运行时间渐近复杂度的符号是定义在自然数集上的一个函数。

2.1 Θ符号

为了表达算法平均时间复杂度的概念，我们引进Θ符号。

定义2.1 对于给定的函数$g(n)$，记$\Theta(g(n))=\{f(n)$：存在三个正常数c_1，c_2和n_0，对于任给$n\geqslant n_0$，满足$0\leqslant c_1g(n)\leqslant f(n)\leqslant c_2g(n)\}$的函数集。并记$f(n)=\Theta(g(n))$，它表示函数$f(n)$是集合$\Theta(g(n))$中的一个元素，这里的“=”表示属于的意思，接下来的定义均用到了这个符号。

定义2.1给出了两个函数$f(n)$和$g(n)$属于同一个数量级或者同阶的概念，即高阶项的增长次数相同。例如$f(n)=\frac{1}{2}n^2-3n$，$g(n)=1000n^2+50n-500$，其高阶项为n^2，高阶项的增长次数均为2，因此有$f(n)=\Theta(g(n))$。要证明$f(n)=\Theta(g(n))$，除了用定义证明外，还可以使用数学中极限的概念来证明，即

如果$\lim\limits_{n\to\infty}\frac{f(n)}{g(n)}$存在，且$\lim\limits_{n\to\infty}\frac{f(n)}{g(n)}=c(0<c<\infty)$，则有$f(n)=\Theta(g(n))$。

图2.1给出了用Θ符号表示的两个函数关系的直观含义，虽然当问题规模比较小时，即$n\leqslant n_0$，很难界定$f(n)$和$g(n)$之间的关系，但是随着问题规模的增大，当$n>n_0$时，$f(n)$总是落入$c_1g(n)$和$c_2g(n)$之间，即$c_1g(n)\leqslant f(n)\leqslant c_2g(n)$。我们也称$g(n)$是$f(n)$的渐近紧界，它既给出了算法时间复杂度的上界，也给出了下界。

值得注意的是，$\Theta(g(n))$的定义要求任意$f(n)=\Theta(g(n))$都是渐近非负的，

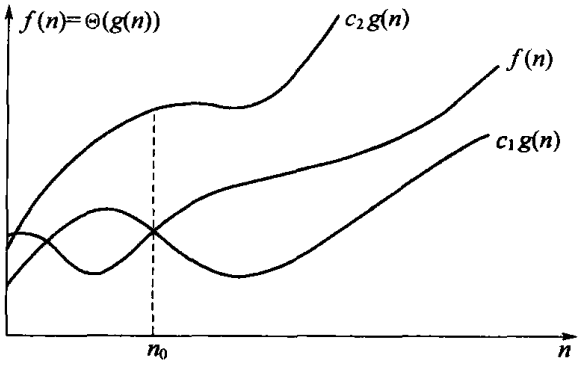


图 2.1

即当 n 足够大的时候 $f(n)$ 非负，因此，函数 $g(n)$ 本身也必须是非负的，否则集合 $\Theta(g(n))$ 为空。今后我们假定所讨论的函数也是渐近非负的。

例 2.1 已知 $f(n)=3n+3$，证明 $f(n)=\Theta(n)$。

证明：由于当 $n\geqslant 3$，有 $3n+3\leqslant 3n+n=4n$ 且当 $n\geqslant 0$，有 $3n+3\geqslant 3n$，因此存在正常数 $c_1=3$，$c_2=4$ 以及 $n_0=3$，对于任给的 $n\geqslant 3$，满足 $3n\leqslant 3n+3\leqslant 4n$，因此 $f(n)=\Theta(n)$。

也可以利用极限来证明。因为 $\lim\limits_{n\to\infty}\dfrac{3n+3}{n}$ 存在，且 $\lim\limits_{n\to\infty}\dfrac{3n+3}{n}=3$，因此 $f(n)=\Theta(n)$。

例 2.2 已知 $f(n)=\dfrac{1}{2}n^2-3n$，证明 $f(n)=\Theta(n^2)$。

证明：我们必须找到三个正常数 c_1，c_2 和 n_0，对于任意 $n\geqslant n_0$，使得

$$c_1n^2\leqslant\frac{1}{2}n^2-3n\leqslant c_2n^2$$

通过估计，只需要取 $c_1=1/14$，$c_2=1/2$，$n_0=7$ 即可。本例同样可以利用极限来证明，在此忽略。

例 2.3 已知 $f(n)=6n^3$，证明 $f(n)\neq\Theta(n^2)$。

证明：可用反证法证明。假设 $f(n)=\Theta(n^2)$，即存在 c_2 以及 n_0，对于任意 $n\geqslant n_0$，满足 $6n^3\leqslant c_2n^2$，由假设可以得到 $n\leqslant c_2/6$，由于 c_2 是常数，当 n 任意大时，该式不成立。故假设不成立。

直观上，渐近正函数的低阶项在估计其渐近紧界的时候可以被忽略掉，因为对于足够大的 n，它们显得无关紧要，高阶项的微小变化足够抵消低阶项的变化。所以，对于 Θ 符号定义中涉及的两个常数 c_1，c_2，选取 c_1 等于一个比高阶项系数稍小的数，再选取 c_2 等于一个比高阶项系数稍大的数，就可以满足 Θ 符号定义中的不等式。高阶项系数同样可以忽略，因为它的改变只会使高阶项的值发生常数系数倍的改变。

例如考虑二次函数 $f(n)=an^2+bn+c$，其中 a，b 和 c 为常数而且 $a>0$。舍去

低阶项 $bn+c$，再忽略高阶项系数就得到 $f(n)=\Theta(n^2)$。按照上面的说法，选取一个比高阶项系数 a 稍小一点的常数 c_1，再选取一个比高阶项系数 a 稍大一点的常数 c_2，例如选取 $c_1=a/4$，$c_2=7a/4$，然后读者就可以自己验证，对任意的 $n\geqslant n_0$，有 $0\leqslant c_1n^2\leqslant an^2+bn+c\leqslant c_2n^2$，其中，$n_0=2\max(|b|/a,\sqrt{|c|/a})$。证明可参考习题 2.4 的解答。

本书中用 $\Theta(1)$ 表示常数时间。

2.2 O 符号

Θ 符号描述了平均时间复杂度，并把一个函数限界在一个范围之内。为了描述算法的最坏情形时间复杂度以及上界的概念，我们引进 O 符号。

定义 2.2 对于给定的函数 $g(n)$，记 $O(g(n))=\{f(n)$：存在正常数 c 和 n_0，对于任意 $n\geqslant n_0$ 满足 $0\leqslant f(n)\leqslant cg(n)\}$ 的函数集。并记 $f(n)=O(g(n))$，它表示 $f(n)$ 是集合 $O(g(n))$ 中的一个元素。

同样可以利用极限来证明，即如果 $\lim\limits_{n\to\infty}\dfrac{f(n)}{g(n)}$ 存在，且 $\lim\limits_{n\to\infty}\dfrac{f(n)}{g(n)}\neq\infty$，则 $f(n)=O(g(n))$。

图 2.2 给出了 O 符号的直观含义。当 $n>n_0$ 时，$f(n)$ 的值总是落入 $cg(n)$ 的下面，即 $f(n)\leqslant cg(n)$。这时我们也称 $g(n)$ 是 $f(n)$ 的渐近上界，它表示算法时间复杂度的上界，即运行时间不可能比 $cg(n)$ 更大。

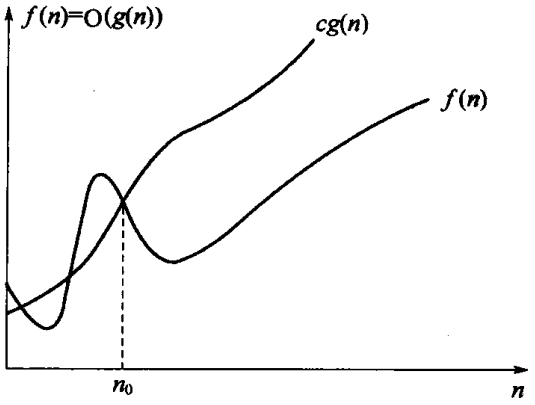


图 2.2

根据 Θ 符号和 O 符号的定义，如果 $f(n)=\Theta(g(n))$，则可以推出 $f(n)=O(g(n))$。

从前面的分析可知，插入排序算法的最坏运行时间为

$$T(n)=\left(\frac{c_4}{2}+\frac{c_5}{2}+\frac{c_6}{2}\right)n^2+\left(c_1+c_2+c_3+\frac{c_4}{2}-\frac{c_5}{2}-\frac{c_6}{2}+c_7\right)n-(c_2+c_3+c_4+c_7-c_8)$$

忽略常数项和低阶项，可得 $T(n)=\Theta(n^2)$，由此可得插入排序算法的最坏情形时间复杂度为 $T(n)=O(n^2)$。前面说过，O 符号描述给定算法在任意规模下运行时间的上界，即最坏情形时间复杂度，而从插入排序算法本身可以看出，算法伪代码中主要有两重循环，**for** 循环和 **while** 循环，两个循环最多各执行 n 次，因此很快得到插入排序算法运行时间的上界 $T(n)=O(n^2)$。尽管对于任意输入插入排序的运行时间不总是 $\Theta(n^2)$，但我们可以用 $\Theta(n^2)$ 来刻画插入排序的最坏运行时间。虽然，$T(n)=\Theta(n^2)$ 也可以限界最坏情形时间复杂度，但是，它并不限界任意规模为 n 的输入，其运行时间均为 $\Theta(n^2)$。事实上，对于顺序为从小到大的一个排列，排序它所需时间为 $\Theta(n)$。

从前面的分析可以看出，忽略掉常数项和低阶项，我们得到了插入排序算法的平均情形时间复杂度 $T(n)=\Theta(n^2)$ 以及最坏情形时间复杂度 $T(n)=O(n^2)$，特别是这样得到的最坏情形时间复杂度，与算法中分析主要循环而得到的时间复杂度没有什么差别。由此可见，当一个算法的最坏情形时间复杂度用渐近符号 O 表示时，分析一个算法的时间复杂度便容易许多。即没必要计算算法每一行执行的时间以及每行执行的次数，然后求和，只需要考虑算法中主要循环执行的次数即可，今后可类似分析其他算法的最坏情形时间复杂度。

2.3 Ω 符 号

为了描述算法的最好情形时间复杂度，我们引进 Ω 符号。

定义 2.3 对于给定的函数 $g(n)$，记 $\Omega(g(n))=\{f(n)$：存在正常数 c 和 n_0，对于任意 $n\geqslant n_0$，满足 $0\leqslant cg(n)\leqslant f(n)\}$ 表示函数的集合。并记 $f(n)=\Omega(g(n))$，它表示 $f(n)$ 是集合 $\Omega(g(n))$ 中的一个元素。

类似地，如果 $\lim\limits_{n\to\infty}\dfrac{f(n)}{g(n)}$ 存在，且 $\lim\limits_{n\to\infty}\dfrac{f(n)}{g(n)}\neq 0$，则有 $f(n)=\Omega(g(n))$。

图 2.3 给出了 Ω 符号的直观含义。当 $n>n_0$ 时，$f(n)$ 的值总是在 $cg(n)$ 的上

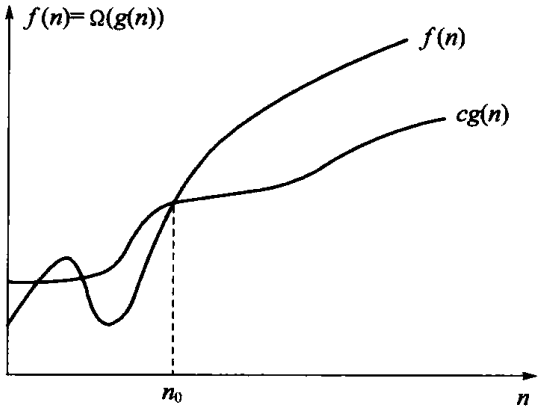


图 2.3

面，即$f(n) \geqslant cg(n)$。这时，我们也称$g(n)$是$f(n)$的渐近下界，它描述了算法时间复杂度的下界，即运行时间不可能比$cg(n)$更小。

Ω符号描述给定算法在任意输入规模下运行时间的下界，即最好情形时间复杂度。当我们说一个算法的时间复杂度是$\Omega(g(n))$时，意味着当n足够大时，不管n怎么取，算法的时间复杂度至少是$g(n)$的某常数倍。从前面插入排序算法的运行时间复杂度的分析可知，插入排序算法的时间复杂度介于$\Omega(n)$和$O(n^2)$之间。插入排序算法的最好情形时间复杂度不是$\Omega(n^2)$，因为存在一个实例使得算法的时间复杂度为$\Theta(n)$，因此，应该特别注意Ω符号所表达的算法在任意输入规模下运行时间下界的含义。

2.4 渐近符号的性质

根据前面渐近符号的定义，可得如下定理。

定理 2.1 对于任意给定的函数$f(n)$，$g(n)$，$f(n)=\Theta(g(n))$当且仅当$f(n)=O(g(n))$且$f(n)=\Omega(g(n))$。

证明可参考习题2.9的解答。

前面，我们证明了对于任意的常数a，b和c，且$a>0$有$an^2+bn+c=\Theta(n^2)$。由定理2.1可以马上得到$an^2+bn+c=\Omega(n^2)$和$an^2+bn+c=O(n^2)$。

定理 2.2 对于任意给定的函数$f_1(n)$，$f_2(n)$，如果$f_1(n)=O(g_1(n))$且$f_2(n)=O(g_2(n))$，则$f_1(n)+f_2(n)=O(\max\{g_1(n),g_2(n)\})$。

证明：由于$f_1(n)=O(g_1(n))$，因此存在正常数c_1和n_1，对于任意$n \geqslant n_1$满足$0 \leqslant f_1(n) \leqslant c_1 g_1(n)$。

由于$f_2(n)=O(g_2(n))$，因此存在正常数c_2和n_2，对于任意$n \geqslant n_2$满足$0 \leqslant f_2(n) \leqslant c_2 g_2(n)$。

令$c_3=\max\{c_1,c_2\}$，$n_0=\max\{n_1,n_2\}$，则

$$\begin{aligned} f_1(n)+f_2(n) &\leqslant c_1 g_1(n)+c_2 g_2(n) \\ &\leqslant c_3(g_1(n)+g_2(n)) \\ &\leqslant 2c_3 \max\{g_1(n),g_2(n)\} \end{aligned}$$

令$c=2c_3$，则存在正常数c和n_0，对于任意$n \geqslant n_0$满足$0 \leqslant f_1(n)+f_2(n) \leqslant c\max\{g_1(n),g_2(n)\}$。按照定义2.2，即可得结论。

如果某个算法A是由几个算法串行地组合起来的，如果其中每个算法的时间复杂度都已经知道，则由定理2.2很容易推出算法A的时间复杂度。比如，一个算法的时间复杂度为$O(n^2\lg n)$，另一个算法的时间复杂度为$O(n^2)$，则两个算法串行起来的时间复杂度为

$$O(n^2\lg n)+O(n^2)=O(n^2\lg n)$$

由定理2.2,可以得到如下结论。

推论2.1 对于任意给定的非负函数$f(n)$,$g(n)$,如果$f(n)\leqslant g(n)$,则$f(n)+g(n)=O(g(n))$。

前面的例子表明,我们用渐近符号可以很方便地表示插入排序算法的最好情形时间复杂度、平均情形时间复杂度以及最坏情形时间复杂度。这表明用渐近符号来研究时间复杂度可以忽略掉在计算算法运行时间时,所涉及的一些无关紧要的因素和复杂的细节,从而使分析时间复杂度变得简单可行。

2.5 常用函数的直观含义

下面我们经常提到的多项式时间算法,指的是该算法具有多项式时间复杂度,即算法的最坏情形时间复杂度可以表示为$O(n^c)$。指数级时间算法,指的是算法具有指数级时间复杂度,即算法的最坏情形时间复杂度可以表示为$O(c^n)$,当然还包括比c^n增长更快的时间函数,例如$n!$。其中,c是一个常数,n是问题的规模。图2.4给出了常用的几个函数,其中2^n和$n!$为指数级函数,其余几个为多项式时间函数。从图2.4可以看出,虽然问题规模n比较小时,指数级函数跟多项式函数没有什么差别。但是随着问题规模n的增大,指数级函数比多项式时间函数增长得快很多,即指数级函数会发生组合爆炸。

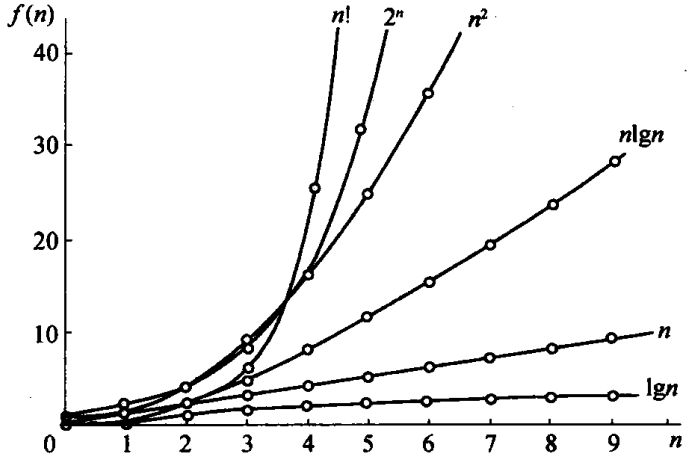


图2.4

因此,从算法设计的角度看,多项式时间算法互相之间虽有差距,但一般可以接受。而指数级时间算法对于较大规模的问题没有什么实用价值。

2.6 小 结

本章部分内容主要取材于文献[1],介绍了今后算法分析常会用到的Θ,O,Ω

符号，然后给出了几个重要的结论，最后给出了常用函数增长快慢的图形表示。关于渐近符号 θ, o, ω 的定义以及渐近符号的有关性质，有兴趣的读者可以参考文献[1]。

有了算法运行时间的渐近表示，我们就可以更方便地比较两个排序算法的性能。假设解排序问题的插入排序算法的运行时间为 $T(n)=c_1n^2$，可简写为 $T(n)=O(n^2)$。而解排序问题的合并排序算法的运行时间为 $T(n)=c_2n\lg n$，可简写为 $T(n)=O(n\lg n)$。随着问题规模 n 的增大，函数 n^2 将比 $n\lg n$ 增长得更快，因此合并排序算法比插入排序算法更快。也就是说，合并排序算法比插入排序算法效率更高，或者说合并排序算法优于插入排序算法。

值得注意的是，算法运行时间的渐近表示，忽略了高阶项的常数系数，因而可能缺乏应用价值，下一章将会提到。例如一个算法的运行时间为 n^2，另一个算法的运行时间为 $2^{100}n$。当 $n>2^{100}$，后一个算法的渐近效率显然比前一个算法高，但是实际上，很少有问题规模超过 2^{100}，因为此时计算机都无法表示这么大的数。后面提到的矩阵乘法，也会出现类似情况。这意味着，理论上渐近有效的算法，在实际应用中有时表现并不好。

此外，我们将在第4章中看到，使用渐近符号可以忽略掉一些无关重要的项，例如主要项的常数系数、低阶项等，从而可以方便地求解递归方程，分析出递归算法的时间复杂度。

习题

2.1 设 $f(n)=3n^2+2n-3, g(n)=5n^2-n+2$，证明 $f(n)=\Theta(g(n))$。

2.2 设 $f(n)$ 和 $g(n)$ 都是渐近非负函数。利用 Θ 符号的基本定义来证明 $\max(f(n),g(n))=\Theta(f(n)+g(n))$。

2.3 证明对于任意给定的实常数 a 和 b，其中 $b>0$，有 $(n+a)^b=\Theta(n^b)$。

2.4 设二次函数 $f(n)=an^2+bn+c$，其中 a, b 和 c 为常数而且 $a>0$，证明 $f(n)=\Theta(n^2)$。

2.5 设 $f(n)=n^2, g(n)=n^3$，证明 $f(n)=O(g(n))$。

2.6 请问 $2^{n+1}=O(2^n)$ 成立吗？$2^{2n}=O(2^n)$ 成立吗？

2.7 证明若 $f(n)=O(g(n))$，则 $g(n)=\Omega(f(n))$。

2.8 设 $g(n)=n^2, f(n)=3n^3$，证明 $n^3=\Omega(n^2)$。

2.9 证明定理2.1。

2.10 证明如果 $f_1(n)=O(g(n))$ 且 $f_2(n)=O(g(n))$，则 $f_1(n)\times f_2(n)=O(g(n)\times g(n))$。

第3章　算法分析方法

第1章已经分析了如何计算一个算法的时间复杂度，从中可以看出计算一个算法的时间复杂度是非常困难的。通过引进渐近符号，简化了时间复杂度的计算和分析。下面我们主要介绍常用的以及最新的一些算法分析方法。

3.1　概率分析

第1章已经介绍过算法最好情形和最坏情形的分析，本节将介绍如何分析算法的平均时间复杂度。平均时间复杂度可以通过将所有实例的计算时间求出来，然后取平均，从而得到求解某个实例所需要的平均基本运算次数。然而这种计算方法，由于实例太多，想要计算出每个实例的时间，显然不现实。这时候，我们借助概率分析(Probabilistic Analysis)，就可以方便分析出算法的平均时间复杂度。但是这种分析方法需要一个假定，即每个实例都有同样的机会作为算法的输入而被算法计算。由此可见，要分析算法的平均时间复杂度，必须掌握概率知识，特别要掌握关于求解问题输入分布的一些先验知识。

下面我们给出利用概率知识分析算法平均时间复杂度的一些例子。

第1章习题里已经涉及到线性搜索问题，即给定一个具有 n 个数的数组 A，问题就是要回答数组里是否存在一个指定的数 x，如果存在，给出该数在数组中的位置，如果不存在，则回答说不能找到。详细的算法如下：

```
LinearSearch(A,x)
1   k←1
2   while k≤n and x≠A[k] do
3       k←k+1
4   if k>n then return 0
5   else return k
```

LinearSearch(A,x)算法从数组 A 的第一个位置开始寻找，如果找到 x 或者 $k>n$，则退出 **while** 循环。

对于这个算法，其最好情形时间复杂度为$\Omega(1)$，其最坏情形时间复杂度为$O(n)$，下面分析其平均情形时间复杂度。为便于分析，我们假定数组 A 中的元素为 $1\sim n$ 之间的整数，而且两两互不相同。下面考虑数组 A 中含有元素 x，即 x 能成功搜索到的情形。

数组 A 中共有 n 个位置，要搜索的元素 x，有可能在 n 个位置中的任何一个位

置出现。我们假定 x 在 n 个位置中任意一个位置出现的机会相同，因此在任意一个位置 k，元素 x 被找到的概率为 $\frac{1}{n}$，如果元素在位置 k 找到，则算法比较的次数为 k。因此，算法找到元素 x 的平均比较次数为

$$T(n) = \sum_{k=1}^{n} k\frac{1}{n} = \frac{1}{n}\sum_{k=1}^{n} k = \frac{1}{n}\frac{n(n+1)}{2} = \frac{n+1}{2}$$

整个算法的平均时间复杂度由比较的次数决定，因此算法的平均情形时间复杂度为 $\Theta(n)$。下面我们给出另一个例子。

第 1 章我们介绍了插入排序算法，并分析了其最好情形时间复杂度和最坏情形时间复杂度，下面我们分析其平均情形时间复杂度。为便于分析，我们假定数组 A 中的元素为 $1 \sim n$ 之间的整数，而且两两互不相同。任意给定一个数组 A，其排列的数目为 $n!$。$n!$ 种排列中的任意一种排列具有同样的概率作为算法的输入。对于插入排序算法，考虑当前迭代要插入的元素为 $A[j]$，算法的目的是要将 $A[j]$ 插入到 $A[1] \sim A[j]$ 共 j 个位置中的任何一个位置。因此 $A[j]$ 插入到 j 个位置中的任何一个位置的概率为 $\frac{1}{j}$。假设 $A[j]$ 插入到 $A[k]$，现在我们分析为了插入到 $A[k]$，算法需要比较的次数。如果 $k=1$，则算法需要比较的次数为 $j-1$，如果 $1 < k \leqslant j$，则算法需要比较的次数为 $j-k+1$，因此将 $A[j]$ 插入到 $A[1] \sim A[j]$ 任何一个位置，平均比较的次数为

$$\frac{j-1}{j} + \sum_{k=2}^{j}\frac{j-k+1}{j} = \frac{j-1}{j} + \sum_{k=1}^{j-1}\frac{k}{j} = \frac{j}{2} - \frac{1}{j} + \frac{1}{2}$$

由于算法共迭代 $n-1$ 次，因此，整个算法需要的平均比较次数为

$$\sum_{j=2}^{n}\left(\frac{j}{2} - \frac{1}{j} + \frac{1}{2}\right) = \frac{n(n+1)}{4} - \frac{1}{2} - \sum_{j=2}^{n}\frac{1}{j} + \frac{n-1}{2}$$

$$= \frac{n^2}{4} + \frac{3n}{4} - \sum_{j=1}^{n}\frac{1}{j}$$

整个算法的平均时间复杂度由比较的次数决定，因此，插入排序算法的平均情形时间复杂度为 $\Theta(n^2)$。

考虑一个更实际的问题：假设你要聘用一个秘书。你对目前聘用的秘书不满意，你一直在试图寻找更合适的人选。因此，对当前的求职者面试之后，你决定，如果求职者的能力比当前秘书强，则解雇现在的秘书，并聘用该求职者。当然，面试的费用很低，而聘用的费用是昂贵的。你愿意支付这种策略所需要的费用，但是想预计整个费用是多大。

为了解决这个问题，我们可以把上面的策略形式化为如下算法：

```
HireAssistant(n)
1  best←0
```

```
2  for i←1 to n do
3      interview candidate i
4      if candidate i is better than candidate best then
5      best←i
6      hire candidate i
```

其中 HireAssistant(n)算法中,秘书是从 0 开始的,即原来没有秘书。*best* 表示当前最好的秘书。

令 C_i 表示面试的费用,C_h 表示聘用的费用,m 表示被聘用的人数,则 HireAssistant(n)算法的总费用为 $O(nC_i + mC_h)$。当第一个求职者就是最好的秘书时,可以得到算法最好情形时间复杂度为 $\Omega(nC_i + C_h)$,当应聘的求职者一个比一个好时,即后面来的求职者比前面的求职者好,可以得到算法的最坏情形时间复杂度为 $O(nC_i + nC_h)$。对于这个聘任问题,它的平均费用有多大? 为了进行概率分析,我们假定求职者以随机的顺序来应聘,并且能够比较两个求职者以确定哪一个更合格。对于当前的求职者,如果能够求得该求职者被聘用的概率,则我们可以估计出整个的费用。事实上,对于第 i 个求职者,前面 i 个人,每一个人都有同样的机会被聘用,因此第 i 个求职者被聘用的概率为 $1/i$。因此平均聘用的费用为

$$\sum_{i=1}^{n} C_h 1/i = C_h \ln n$$

对于这个聘任问题,算法平均情形复杂度为 $\Theta(nC_i + C_h \ln n)$。

从上面几个例子的分析可以看出,通过对输入的分布做一个假定,就可以利用概率分析来分析平均情形时间复杂度,而且平均情形时间复杂度确实接近最坏情形时间复杂度。本书第 5 章还将介绍概率分析的应用,这里不再介绍。

3.2 分摊分析

在上一节,我们介绍了如何分析一个算法的平均情形时间复杂度。但在一些实际应用中,分析一个算法的平均时间复杂度比较复杂,或者按照前面的最坏情形分析方法,可能得到比较高的时间复杂度。此时,我们可以借助分摊分析(Amortized Analysis),分析出合理的时间复杂度。分摊分析的思想,来源于会计学领域,它为算法分析提供了一种直观的比喻。在分摊分析中,通过研究执行一系列数据结构运算所需要的费用,来研究各个运算之间的关系。分摊分析能够用来证明,如果一系列运算的总费用是小的,则其中一个运算的分摊费用也是小的,即使一系列运算中某个运算的费用很大。分摊分析与平均情形分析的不同之处在于它不涉及复杂的概率分析,分摊分析保证了在最坏情形下每个运算具有的平均性能。

此外,对于一系列运算,最坏情形分析是找出其中运行时间最长的一个运算,然后对每个运算都按照这个最坏情形来估计,从而得到一系列运算的最坏情形时

间复杂度。这种估计可能导致比较高的时间复杂度，事实上，最坏情形并不总是频繁发生。比如，读大学的时候，父母会每个月给我们一定的零花钱，如果我们按照花钱最奢侈的一个月去估算，则父母一年里会给我们很多钱。事实上，每个月的消费会有多有少，前面几个月消费后剩下的钱，接下来几个月会继续利用。也就是说，某个月的消费取决于前面几个月是否有钱剩余。利用分摊分析的思想，父母可以在第1个月多给一些钱，如果第1个月的实际花费少于父母给的零花钱，则剩余的钱可以在第2个月再利用。上面的分析也表明，一系列数据结构的运算之间是相互关联的，从而才能利用分摊分析的思想。

下面介绍分摊分析的三种分析方法。

3.2.1 合计方法

在合计方法(Aggregate Method)中，我们就是要分析出由 n 个运算构成的序列在最坏情形下的运行时间总和 $T(n)$，因而在最坏情形下，每个运算的平均费用，或者分摊费用可定义为 $T(n)/n$。注意这个分摊费用的计算方法对每个运算都是成立的，即使当序列中存在几种类型的运算时也一样。

考虑进栈和出栈运算的例子，进栈和出栈运算为

Push(S,x)：将元素 x 压入栈 S。

Pop(S)：弹出并返回 S 的顶端元素。

因为这两个运算的运行时间都为 O(1)，故我们可以把每个运算的费用当作1。这样的话，一个包含 n 个 Push 和 Pop 运算的序列的总费用就为 n，因而这 n 个运算的实际运行时间就为 O(n)。

现在我们增加一个栈运算 MultiPop(S,k)，它弹出栈 S 顶部的 k 个元素，但如果栈中元素的个数小于 k，则此运算将把栈清空，其伪代码如下：

```
MultiPop(S,k)
1  while StackEmpty(S) ≠∅ and k≠0 do
2      Pop(S)
3      k ← k - 1
```

其中第1行，当 StackEmpty(S)不为空($\varnothing$)且 $k\neq0$ 时，元素出栈。现在，我们分析一下对大小为 s 的栈执行栈运算 MultiPop(S,k)，所需要的运行时间。如果栈中元素的个数 s 小于 k，则此运算将把栈清空，否则将弹出栈 S 顶部的 k 个元素，因此该运算的费用取决于 k 和 s 的大小，即该运算的费用为 min(s,k)。

下面给出一个例子，如图3.1所示。

图3.1(a)栈中有6个元素，执行 MultiPop(S,4)运算后，得到图3.1(b)，此时栈中有2个元素，执行 MultiPop(S,7)运算后，得到空栈，如图3.1(c)所示。

现在对一个初始为空的栈，执行一个由 n 个 Push、Pop 和 MultiPop 运算构成的序列所花费的时间进行分析。

在 n 个运算组成的序列中，MultiPop 运算可能会有 O(n)个，而堆栈的大小至

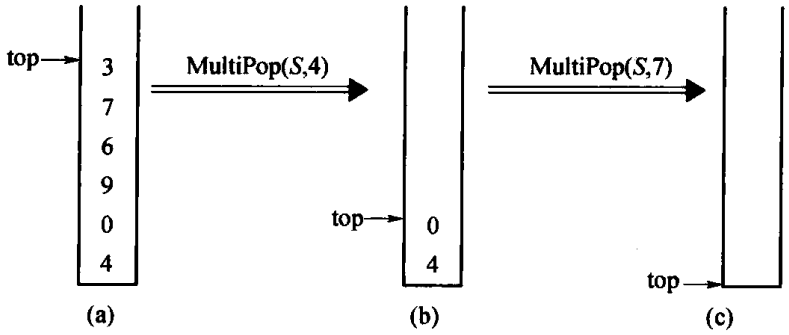


图 3.1

多为 n。由于每个运算的费用为 $O(n)$，因此 n 个运算的序列最坏情形的总费用为 $O(n^2)$，每个运算的平均费用是 $O(n^2)/n = O(n)$。虽然这个分析是正确的，但是这个上界太大。事实上，用分摊分析可以得到更好的上界。虽然某一次的 MultiPop 运算的费用可能较高，但是作用于初始为空的栈的任意一个包含 n 个 Push、Pop 和 MultiPop 运算的序列的费用至多为 $O(n)$；因为一个元素在每次被压入栈后至多被弹出一次，调用 Pop，包括 MultiPop 运算中的 Pop 的总次数，至多等于 Push 的次数 n。因此，整个序列的最坏情形的运行时间为 $O(n)$，因而每个运算的分摊费用为 $O(n)/n = O(1)$。

考虑实现一个由 0 开始向上计数的 k 位二元计数器的问题。令大小为 k 的数组 $A[0..k-1]$ 表示一个 k 位的计数器，存储一个二进制数 x，其低位在 $A[0]$，高位在 $A[k-1]$，因此

$$x = \sum_{i=0}^{k-1} A[i]2^i$$

对于初始状态：$x = 0, A[i] = 0 (i = 0, 1, \cdots, k-1)$，为了实现计数器的加，定义加运算如下：

```
Increment(A)
1  i←0
2  while i < k and A[i] = 1 do
3       A[i]←0
4       i←i + 1
5  if i < k then
6       A[i]←1
```

上述加运算 Increment(A)，总是从低位开始，执行加运算。第 2 行，当 $i < k$ 且 $A[i] = 1$，则 $A[i]←0$，再往前进一位。如果 $i < k$，则表明 $A[i] = 0$，因此 $A[i]←1$。

对于上述加运算，其时间复杂度主要取决于位改变的次数。如果数组 A 中的元素均为 1，则最坏情形下需要时间 $O(k)$。因此对于初始为 0 的计数器，一个具有 n 个加运算的序列最坏情形下需要时间 $O(nk)$，平均每个加运算的分摊费用为 $O(nk)/n = O(k)$。这个上界太大，我们可以进一步缩小这个上界。如果我们的分

析更精确些，则可得到 n 次 Increment 运算的序列的最坏情形费用为 $O(n)$，关键在于注意到每次调用 Increment 时，并不是所有的位都发生了变化。

表 3.1 给出了一个 8 位二进制计算器从 0 开始执行 Increment 运算，连续执行 16 次加运算的结果，最后一列给出了位发生变化的次数。从表 3.1 可以看出，$A[0]$ 位每次加运算都翻转一次，$A[1]$ 位每两次翻转一次，即 $A[1]$ 翻转 $\lfloor n/2 \rfloor$ 次，$A[2]$ 每四次翻转一次，即 $A[2]$ 翻转 $\lfloor n/2^2 \rfloor$ 次。一般地，对于初始为 0 的计数器，执行 n 次 Increment 运算，位 $A[i]$ 翻转 $\lfloor n/2^i \rfloor$ 次，其中 $i = 0, 1, \cdots, \lfloor \lg n \rfloor$。对于 $i > \lfloor \lg n \rfloor$，$A[i]$ 位不翻转，因此执行 n 次 Increment 运算，位翻转的总次数为

$$\sum_{i=0}^{\lfloor \lg n \rfloor} |n/2^i| < n \sum_{i=0}^{\infty} 1/2^i \approx 2n$$

因此，作用于一个初始为 0 的计数器上的 n 个 Increment 运算的最坏情形时间为 $O(n)$，而每个运算的分摊费用为 $O(n)/n = O(1)$。

表 3.1

Counter value	$A[7]$	$A[6]$	$A[5]$	$A[4]$	$A[3]$	$A[2]$	$A[1]$	$A[0]$	total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

3.2.2 记账方法

记账方法(Accounting Method)的思想是对不同的运算赋予不同的费用，某些

运算的费用可能比它们实际所需的费用多，也有可能少。对某一运算所赋予的费用，就记为该运算的分摊费用。当一个运算的分摊费用比它们的实际费用多时，多出来的费用（即余款）作为存款保存在数据结构中一些特定的对象中。这笔存款用来支付给分摊费用比实际所需费用少的某些运算。这样，我们可以把一个运算的分摊费用分成两部分，一部分用来支付实际所需的费用，另一部分作为存款（分摊费用有剩余）或者透支（分摊费用不够支付该运算的实际费用）保存在数据结构中某个特定的对象中，以备后用。

在选择每个运算的分摊费用时，需要非常小心，如果我们想利用分摊费用来证明在最坏情形下每个运算的平均费用是小的，运算序列的总的分摊费用必须为运算序列总的实际费用的一个上界，使得与该数据结构相关的存款始终为非负。令序列中第 i 运算的实际费用为 c_i，分摊费用为 $\hat{c}_i$，则需要保证

$$\sum_{i=1}^{n} c_i \leqslant \sum_{i=1}^{n} \hat{c}_i$$

从而使得在数据结构中总的存款 $\sum_{i=1}^{n} \hat{c}_i - \sum_{i=1}^{n} c_i$ 始终为非负。

为了描述记账方法的应用，重新考虑栈运算的例子，各栈运算的实际费用为

Push	1
Pop	1
MultiPop	$\min(k,s)$

现在我们给每个运算的分摊费用分别为

Push	2
Pop	0
MultiPop	0

假定我们用 1 元来表示单位费用，开始时栈为空。每次把一个元素压进栈，由于进栈运算的分摊费用为 2 元，1 元支付实际费用，另 1 元作为存款保存在该元素中。因此，在任何时候，栈中每个元素均有 1 元的存款。当需要一个元素出栈时，出栈的实际费用，就用该元素的存款来支付。这样，通过付给进栈运算 Push 多一点的分摊费用，我们就不必给出栈运算一笔分摊费用。同样，对于 MultiPop 运算也一样，不管该运算弹出几个元素，其实际的花费，均可以用弹出元素的存款来支付。由于栈中每个元素，均有 1 元存款，这就可以保证栈中存款总数为非负。因而，对于任意包含 n 个 Push、Pop 和 MultiPop 运算的序列，总的分摊费用就是其总的实际费用的一个上界。又因为总的分摊费用为 $O(n)$，故总的实际费用也为 $O(n)$。

下面给出另一个例子。

前面已经分析，计数器 Increment 运算的运行时间取决于位翻转的次数。要分析出位翻转的次数，还比较复杂。我们再次用 1 元代表单位费用。为进行分摊分析，我们规定将某一位设置为 1 的运算支付 2 元的分摊费用。1 元用于支付将该位设置为 1 的实际费用，而将另 1 元作为存款存在该位上，因此，若将该位复位为 0，

则其实际费用,用该位上的存款来支付就行了。

在 **while** 循环中复位的费用是由该位上的存款支付的,Increment 运算的第 6 行至多有一位被置为 1,因此,一个 Increment 运算的分摊费用至多为 2 元。计数器中 1 的数目总是非负数,因此计数器中存款的数目总是非负数。因而,对于 n 个 Increment 运算的序列,总的分摊费用为 $O(n)$,它是实际费用的上界,故总的实际费用也为 $O(n)$。

3.2.3 势能方法

前面我们已经介绍记账方法把余款作为存款存储在数据结构中某个特定对象中。下面介绍的势能方法把每个运算的余款表示成一种“势能”,存储在整个数据结构中,它在需要时可以用来支付后面运算所需要的费用。

势能方法(Potential Method)的思想是从一个初始的数据结构 D_0 出发,执行 n 个运算的序列。设 c_i 为第 i 个运算的实际费用,D_i 为对数据结构 D_{i-1} 作用第 i 个运算的结果,这里 $i=1,2,\cdots,n$。一个势能函数 Φ 将每个数据结构 D_i 映射为一个实数 $\Phi(D_i)$,即与数据结构 D_i 相联系的势能。

定义第 i 个运算的分摊费用为

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

则 n 个运算的总分摊费用为

$$\begin{aligned}\sum_{i=1}^{n}\hat{c}_i &= \sum_{i=1}^{n}(c_i + \Phi(D_i) - \Phi(D_{i-1}))\\ &= \sum_{i=1}^{n}c_i + \Phi(D_n) - \Phi(D_0)\end{aligned}$$

如果我们能定义一个势能函数 Φ 使得 $\Phi(D_n)\geqslant\Phi(D_0)$,则总的分摊费用就是总的实际费用的一个上界。事实上,我们不知道要执行多少个运算,如果能保证对所有的 i,有 $\Phi(D_i)\geqslant\Phi(D_0)$,则如同记账方法一样,我们可以预先支付。通常定义 $\Phi(D_0)=0$,然后证明对所有的 i,有 $\Phi(D_i)\geqslant 0$。直观地说,如果第 i 个运算的势能差 $\Phi(D_i)-\Phi(D_{i-1})>0$,则付给第 i 个运算的分摊费用过多,数据结构中的总势能将增加。如果势能差不大于 0,则付给第 i 个运算的分摊费用过少,则需要从总势能中支出一笔费用来支付实际的花费,因而数据结构中的总势能将减少。值得注意的是,上述所定义的分摊费用依赖于所选择的势能函数 Φ。不同的势能函数可能会产生不同的分摊费用,但它们都是实际费用的上界。因此,在选择一个势能函数时,常要做一些权衡,最佳势能函数的选择取决于想要得到的上界。下面我们给出势能方法的应用。

对于前面介绍过的栈运算,我们定义栈上的势能函数 Φ 为栈中元素的个数。开始时,我们要处理的是空栈 D_0,因此 $\Phi(D_0)=0$。因为栈中的元素个数始终是非

负的，故在第 i 个运算之后的栈 D_i 就具有非负的势能，即 $\Phi(D_i) \geqslant \Phi(D_0) = 0$。因此势能函数为 Φ 的 n 个运算的分摊费用的总和就表示了总实际费用的一个上界。

现在，我们计算各种栈运算的分摊费用，如果作用于一个包含 s 个元素的栈上的第 i 个运算是 Push 运算，则势能差为

$$\Phi(D_i) - \Phi(D_{i-1}) = (s+1) - s = 1$$

其分摊费用为

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + 1 \\ &= 2\end{aligned}$$

假设第 i 个运算是 MultiPop(s,k)，且弹出了 $k' = \min(k,s)$ 个元素。该运算的实际费用为 k'，势能差为

$$\Phi(D_i) - \Phi(D_{i-1}) = -k'$$

因此，MultiPop 运算的分摊费用为

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= k' - k' \\ &= 0\end{aligned}$$

类似地，Pop 运算的分摊费用为 0。因此每个运算的分摊费用均为 $O(1)$，对于任意包含 n 个 Push、Pop 和 MultiPop 运算的序列，总的分摊费用就是其总的实际费用的一个上界。因此该序列的最坏情形的费用为 $O(n)$。

对于前面介绍过的计数器 Increment 运算的例子，我们定义在第 i 次 Increment 运算后计数器的势能为 b_i，其中 b_i 为第 i 次运算后计数器中 1 的个数。

现在我们计算 Increment 运算的分摊费用。假设第 i 次 Increment 运算对 t_i 位进行了复位，即置 0。该运算的实际费用至多为 $t_i + 1$，因为除了将 t_i 个位复位外，它至多将一位设置为 1。若 $b_i = 0$，则第 i 个运算将所有 k 位都复位，因此有 $b_{i-1} = t_i = k$。若 $b_i > 0$，则有 $b_i = b_{i-1} - t_i + 1$。对上述任一情形，均有 $b_i \leqslant b_{i-1} - t_i + 1$。

第 i 次运算后，势能差为

$$\Phi(D_i) - \Phi(D_{i-1}) = b_i - b_{i-1} \leqslant (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i$$

因此分摊费用为

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &\leqslant (t_i + 1) + (1 - t_i) \\ &= 2\end{aligned}$$

计数器从 0 开始，因此 $\Phi(D_0) = 0$。因为计数器中 1 的个数始终是非负的，故在第 i 个运算之后，计数器就具有非负的势能，即 $\Phi(D_i) \geqslant \Phi(D_0) = 0$。因此势能函数为 Φ 的 n 个运算的分摊费用的总和就表示了总实际费用的一个上界。因此

该序列的最坏情形的费用为 $O(n)$。

势能方法为我们提供了一种方便分析计数器加运算序列的时间复杂度的方法，即使计数器不从 0 开始，也可以做类似分析，这里不再详述，可参考习题 3.10 的解答。

前面我们介绍的三种分摊分析方法，为我们分析算法提供了一种不同的、新的工具。分摊分析在许多算法分析以及许多优化设计中都有直接的应用，例如图算法时间复杂度的分析、计算几何算法的分析以及近似算法近似比的分析等，后面章节还将提及，这里不再介绍。对分摊分析在近似算法中的应用感兴趣的读者可参考文献[4]。

3.3 实验分析

前面介绍的几种分析都是基于渐近符号，分析算法的渐近复杂性，然而渐近复杂性也有不合理之处。渐近复杂性考虑随着问题规模无限增大时，算法的效率表现，而实际问题的规模总有一个范围，因而渐近复杂度低的算法在求解规模较小的问题时，并不一定优于一个渐近复杂度高的算法。习题 1.10 以及第 2 章都给出了例子进行说明。还有些问题，非常复杂，无法对输入做某种假设，很难对算法做理论分析。此外，对于一些非常困难的问题，也很难找到最优解，对算法的分析，不仅要考虑计算速度，而且要考虑解的质量，即分析算法的性能(Performance)。在这种情况下，就需要借助实验分析。

实验分析(Empirical Analysis)的目的是通过借助计算机，利用所设计的算法进行大量的实验测试，通过分析实验结果，来比较不同算法的性能。实验分析的优点是简单、实用。其基本过程是，首先选择一些具体问题的实例，然后利用算法对这些例子进行测试，最后记录算法计算这些例子所获得解的质量以及所需要的时间。通过对计算结果的统计分析，来分析比较算法的性能。通过分析算法的实际运行结果，可以知道现有的计算资源下算法可以求解多大的问题，解的质量如何，所需要的计算时间等。实验分析可以在完全真实的条件下进行，不需要像概率分析那样做各种假设。因此，实验分析一出现，就受到工程应用领域的青睐，成为算法分析的有力工具之一。

实验分析的缺点是容易受所选测试实例、计算模型、实验参数等的影响。例如，对于一些随机算法，即使是同一实例，如果多次运行算法，运行结果也可能不一样，因此，还必须多次运行，取平均结果。下面我们给出实验分析的主要步骤。

1. *数据的选择与生成*

由于实验分析容易受所选测试实例的影响，因此数据的选择必须保证具有足够多的测试数据，既要包括规模小的例子，又要包括规模大的实例；既要包括容易的例子，又要包括困难的例子。只有这样，选择的数据才具有代表性，相应的实验

分析才有说服力，也才能得到统计意义上的重要结果。

通常对一些困难的问题，网上都有一些公开的测试实例（benchmark），例如文献[26]就给出了一些经典 NP 问题的测试实例。因此，只需要测试这些公开的实例，就可以比较算法的性能，满足研究的需要。如果没有公开的测试实例，可以自己随机生成一些测试数据。然后，通过测试这些数据，来验证我们有关算法假设的一般性结论。

2. 实现算法

由于算法实现与具体的程序设计技巧有关，当用实验分析比较不同算法的性能时，必须假设实现不同算法的程序设计技巧相同，代码优化也一样。一旦实现算法后，同时，还要注意算法尽量保证在相同的计算机（计算模型）上测试数据。如果使用的计算机不同，则需要考虑 CPU 的速度、内存的大小等，这样，不同的算法才能进行公平的比较，所得的计算结果也才能有说服力。

3. 计算结果分析

通常借助图表将计算结果可视化。这样就可以非常清楚直观地比较不同算法的性能，发现算法的未知特性。此外，数据结果分析也可以帮助分析算法的时间复杂度。例如比率测试，方法是先估计算法的时间复杂度函数 $t(n)$，然后通过测试问题规模 n 从小到大的各种实例，计算算法所需要的时间与 $t(n)$ 的比值，如果该比值趋向于一个常数，那么就可以认为 $t(n)$ 与实际运行时间相吻合，是一个好的估计。由于这种估计是建立在精确的数量化结果的基础上，因而比纯粹的理论分析给人一种更为真实的感觉。

此外，实验分析也有助于研究算法的最好情形、平均情形和最坏情形的性能，方法是选择各种实例，例如容易的实例、特别困难的实例以及随机实例，在这些实例上测试算法的性能，则可以分别估计出算法最好情形、平均情形和最坏情形的性能，以及分析算法对实例的敏感性。

因此，实验分析不仅仅按理论分析的一些模式来分析算法，而且可以发现理论分析无法发现的结果和现象。特别是有些算法，例如模拟退火算法、遗传算法等，常常依赖一些参数的设计，利用实验分析还可以确定出比较好的参数设置，从而设计出更有效的算法。

3.4 小　结

本章分摊分析的内容主要取材于文献[1]，其他内容参考了文献[5,6]。第 2 章介绍了算法最好情形、最坏情形时间复杂度分析，本章主要介绍如何利用概率分析来研究算法的平均时间复杂度，然后介绍了目前算法分析里比较新的分析工具，即分摊分析。关于分摊分析的一些应用，感兴趣的读者可以阅读文献[1]。最后，我们介绍了目前流行的实验分析方法，由于目前启发式算法，例如遗传算法、模拟

退火、蚁群算法以及一些机器学习方法等，很难从理论上分析其时间复杂度，因此经常采用实验分析的方法。因此，今后介绍算法的时候，会涉及实验分析方法的应用。

习题

3.1 在 HireAssistant(n)中，假设应聘者以随机的顺序出现，正好雇用一次的概率是多少？正好雇用两次的概率是多少？正好雇用 n 次的概率又是多少？

3.2 请分析习题 1.5，找 n 个数最大值算法的平均时间复杂度。

3.3 如果一组栈运算中包括了一次 MultiPush 运算，它一次把 k 个元素压入栈内，那么栈运算分摊费用的界 $O(1)$ 是否还能保持？

3.4 对某个数据结构执行一个具有 n 个运算的序列。如果 i 为 2 的整数幂，则第 i 个运算的费用为 i，否则为 1。请利用合计方法确定每次运算的分摊费用。

3.5 对一个大小从不超过 k 的栈执行一系列的栈运算。在每 k 个运算后，复制整个栈的内容以留作备份。证明对各种栈运算分配合适的分摊费用后，n 个栈运算(包括复制栈的运算)的费用为 $O(n)$。

3.6 利用记账方法分析习题 3.4。

3.7 假设存在势函数 Φ，使得对所有的 i 都有 $\Phi(D_i) \geqslant \Phi(D_0)$，但是 $\Phi(D_0) \neq 0$。证明存在一个势函数 Φ'，使得对所有 $i \geqslant 1$，$\Phi'(D_0) = 0$，$\Phi'(D_i) \geqslant 0$，且用 Φ' 表示的分摊费用与用 Φ 表示的分摊费用相同。

3.8 用势能方法分析习题 3.4。

3.9 假设某个栈在执行 n 个栈运算 Push，Pop 和 MultiPop 之前包含 S_0 个对象，结束后包含 S_n 个对象，试求 n 个栈运算的总费用。

3.10 对于一个计数器加运算序列，如果计数器不从 0 开始，用势能方法分析该序列的时间复杂度。

第4章 递 归

4.1 算法思想

递归(Recurrence)是计算机、数学、运筹等领域经常使用的最强大的解决问题的方法之一,它用一种简单的方式来解决那些用其他方法解起来可能很复杂的问题,也就是说有些问题用递归算法来求解,会变得简单,而且容易理解;而用其他方法,则描述很复杂,甚至很难设计。递归的基本思想是把一个问题划分为一个或多个规模更小的子问题,然后用同样的方法解规模更小的子问题。值得注意的是,这些规模更小的子问题应该与原问题保持同一类型,这样才能用同样的方法求解。

递归算法的基本设计步骤:

(1) 找到问题的初始条件(递归出口),即当问题规模 n 小到某个值时,该问题变得很简单,能够直接求解。

(2) 设计一个策略,用于将一个问题划分为一个或多个一步步接近递归出口的相似的规模更小的子问题。

(3) 将所解决的各个小问题的解组合起来,即可得到原问题的解。

设计递归算法时需注意以下几个问题:

(1) 如何使定义的问题规模逐步缩小,而且始终保持同一问题类型?

(2) 每个递归求解的问题其规模如何缩小?

(3) 多大规模的问题可作为递归出口?

(4) 随着问题规模的缩小,能到达递归出口吗?

设计出一个递归算法后,如何证明递归算法的正确性呢? 我们可以利用循环不变量,采用归纳法来证明,其基本思路是:

(1) 检验作为递归出口的问题,递归算法是否能够正确解决它,即初始情况下,循环不变量为真。

(2) 证明每次递归调用能够一步步接近递归出口,即证明如果子问题的循环不变量为真,原问题也为真。

(3) 证明如果所有的小问题都能够得到正确的解决,那么最初的原始问题也能够正确地解决。当递归调用结束,循环不变量为真。

为了分析递归算法的时间复杂度,通常将递归算法的求解过程,构造成一棵递归树。借助递归树,可以方便地将求解一个问题的时间用求解该问题子问题的时间来表示,即写出递归算法时间复杂度的递归方程,然后利用求解递归方程的方

法,求出递归算法的时间复杂度。

4.1.1 递归算法的应用

1. 计算2的n次幂,即计算$f(n)=2^n$

当$n=1$时,问题$f(1)=2^1=2$很简单,可以做递归出口。当$n>1$时,原问题$f(n)$可以分解为$f(n)=2^n=2\times 2^{n-1}=2\times f(n-1)$,因此原问题$f(n)$的求解可以转化为求解规模更小的子问题$f(n-1)$,$f(n-1)$和$f(n)$具有同一问题类型,只是规模更小。因此这个问题可以用如下的递归方程表示:

$$f(n)=\begin{cases}2 & \text{if } n=1\\ 2\times f(n-1) & \text{if } n>1\end{cases}$$

根据这个递归方程,我们设计出如下递归算法:

```
f(n)
1    if n=1 then return 2
2    else return 2 × f(n-1)
```

递归算法$f(n)$中,第1行为递归出口,而第2行涉及到对自身的调用,只不过传递过去的参数为$n-1$,比原问题规模小1。当然这个问题也有别的划分方法,例如二分,这里不再详细介绍,读者可以进行思考。

下面来看$f(3)$的递归计算过程。如图4.1所示,沿着箭头的方向,需要求解的问题规模越来越小,当传递的参数为1,需要计算$f(1)$,就到了递归出口。递归调用结束,以后就是逐步把计算出来的子问题的值传递回去,以便得到原问题的解。

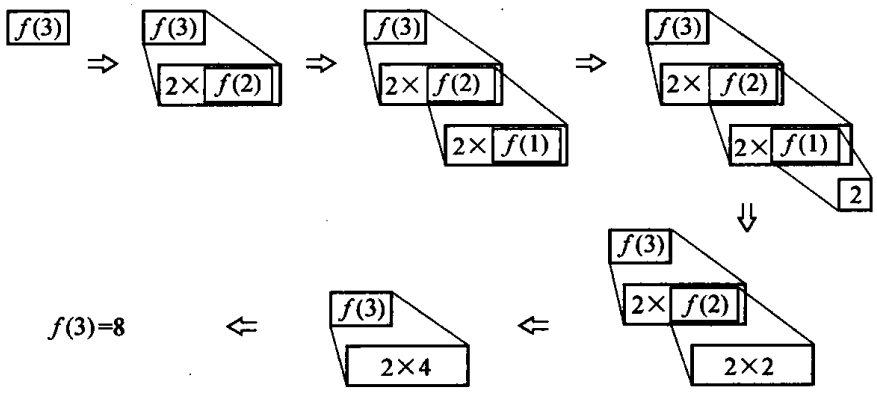


图4.1

递归算法$f(n)$的循环不变量为$f(k)=2^k$,下面我们证明如下:

1) 递归出口

当$n=1$时,根据算法$f(n)$,算法将返回2。而根据递归出口条件,当$n=1$时,$f(1)=2$,这表明算法正确地计算了$f(1)$。

2）归纳假设

假定当 $n=k$ 时，式子成立，即 $f(k)=2^k$。

我们要证明对于 $n=k+1$ 时式子成立，即有 $f(k+1)=2^{k+1}$，根据算法 $f(n)$，当 $n=k+1$ 时，算法将返回 $2\times f(k)$，而由归纳假设 $f(k)=2^k$，从而

$$f(k+1)=2\times f(k)=2\times 2^k=2^{k+1}$$

故所证成立。

从上面的程序可以看出 $f(n)=2\times f(n-1)$ 是一个递归函数，它是根据自身性质来定义的。从程序 $f(n)$ 的伪代码可以看出，如果没有递归出口 $f(1)=2$，程序就会永远递归下去。因此，递归出口作为结束条件，可以用来阻止程序死循环。

令 $T(n)$ 表示递归算法求解 2^n 所需要的计算时间，则有

$$T(n)=\begin{cases}\Theta(1) & \text{if } n=1\\ T(n-1) & \text{if } n>1\end{cases}$$

这个递归方程比较简单，很容易求得其解为 $T(n)=O(n)$。

2. Hanoi 问题

从前有座庙，庙里住着一群和尚，和尚们天天念经，颇感枯燥无味。庙前有三根柱子，其中有一根金字塔形的柱子 A，该金字塔由一些圆盘从大到小叠放而成，如图 4.2 所示。有一天，一个和尚望着门前的三根柱子，突发奇想，如果能够将柱子 A 上的所有圆盘借助柱子 B，一个接一个搬到另一根空柱子 C 上，那将是一件十分有趣的事情。

图 4.2

和尚搬圆盘的游戏规则如下：①一次只能搬动一个圆盘；②不能将大圆盘放在小圆盘上面。和尚们天天玩这个游戏，但是很少成功。在玩的过程中，和尚发现，当圆盘个数比较少的时候，还是很容易完成游戏的，当圆盘个数超过 5 以上，就很难完成游戏了。在经历了多次失败后，和尚们产生了非常悲观的想法，当解决了柱子上圆盘数为 64 的问题时，也就是世界末日到来的时候！

现在我们分析一下该问题的特点。假设有 n 个圆盘，如果能把 A 柱子上的 $n-1$ 个圆盘搬到 B 上，把 A 柱子上剩下的大圆盘搬到 C 柱子上，由于该圆盘是最大的，在以后的搬动过程中，它保持不动。这样，搬动 n 个圆盘的问题，就变为搬动 $n-1$ 个圆盘的问题。当问题规模为 1 个圆盘时，搬动很简单，可以直接求解。因此可以利用递归算法来求解圆盘搬动问题，其详细过程如下：

(1) 将前 $n-1$ 个圆盘从 A 柱借助于 C 柱搬到 B 柱。

(2) 将最后一个圆盘直接从 A 柱搬到 C 柱。

(3) 将 $n-1$ 个圆盘从 B 柱借助于 A 柱搬到 C 柱。

根据上述过程,我们可以写出如下递归算法:

```
Hanoi(n,A,B,C)
1    if  n=1  then  move(1,A,C)
2    else
3        Hanoi(n-1,A,C,B)
4        move(n,A,C)
5        Hanoi(n-1,B,A,C)
```

下面给出搬 3 个圆盘的执行过程,如图 4.3 所示。

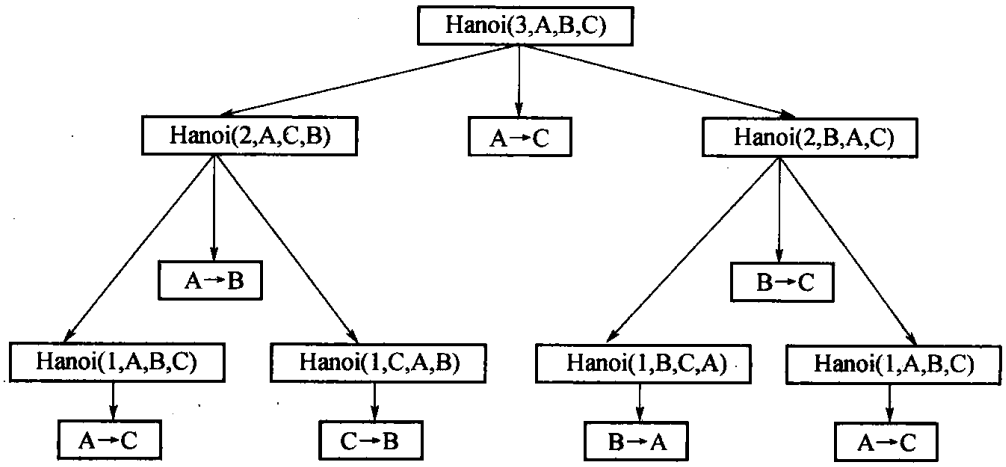


图 4.3

图 4.3 像一棵树,我们称之为递归树。事实上,对于递归算法,都可以构造出一棵递归树,借助递归树,我们很容易看出算法执行的过程,而且便于分析递归算法的时间复杂度。

根据程序 Hanoi(n,A,B,C),移动圆盘所花费的时间 T 可表示为

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n-1) + 1 & \text{if } n > 1 \end{cases}$$

关于这个递归方程的求解,我们稍后给出。

3. 选择排序问题

前面我们介绍了解决排序问题的插入排序算法,现在介绍选择排序算法。选择排序算法对排列规模比较小的元素序列非常有效,其基本思想就像排列手中的扑克牌一样:

(1) 把所有牌摊开,放在桌上,伸出左手,开始为空,准备拿牌。

(2) 将桌上最小的牌拾起,并把它插到左手所握牌的最右边。

(3) 重复步骤(2),直到桌上所有牌都拿在左手上,此时左手上所握的牌便是排好序的牌。

从上面的过程看出,可以用递归的思想来实现选择排序算法。假设要对 n 个元素排序,我们将这 n 个元素用数组 A 表示,这里元素与牌相对应。因为我们最终输出的是一个元素由小到大排列的数组,假设左手上握的牌为 $A[1],\cdots,A[i-1]$,其顺序由小到大已经排好,我们只需对桌上的牌 $A[i],\cdots,A[n]$ 进行排序即可。为了解决该问题,我们可以从牌 $A[i],\cdots,A[n]$ 中确定最小的一张牌 $A[k]$,如果 $k \neq i$,则将两张牌 $A[k]$ 和 $A[i]$ 交换,此时 $A[i]$ 为桌上最小的牌,将其插入到左手的最右边,这时手上的牌为 $A[1],\cdots,A[i]$,接下来只需对子问题 $A[i+1],\cdots,A[n]$ 进行排序,当桌上只剩下一张牌 $A[n]$ 时,直接拿到左手上即可。因此,选择排序(Selection Sort)算法的伪代码如下:

```
SelectionSort(i)
1  if i≥n then return 0
2  else
3       k←i
4       for j←i+1 to n do
5           if A[j] <A[k] then
6                k←j
7       if k≠i then A[i] ↔ A[k]
8       SelectionSort(i+1)
```

其中,符号↔表示交换的意思,即将数组中两个元素的值进行交换,后面用到这个符号的时候,就不再说明。从 SelectionSort(i)可知,当递归调用到 SelectionSort(n)时,程序停止,不再进行值的回传工作,是一种比较简单的递归。这样,我们要求解原问题,只需要调用 SelectionSort(1)即可。图 4.4(a)~(e)给出了一个运行实例,图中 k 指向当前桌子上最小的一张牌。

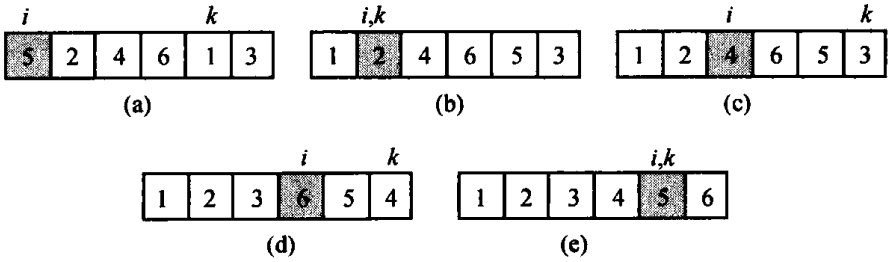


图 4.4

令 $T(n)$ 表示排序 n 个数所需要的时间,由于 n 个数中选择最小的数需要比较 $n-1$ 次,因此选择排序算法 SelectionSort(1)运行时间的递归方程为

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ T(n-1) + (n-1) & \text{if } n > 1 \end{cases}$$

上述递归方程的求解，我们将稍后给出。

4. 排列问题

给定 n 个元素 $\{1,2,\cdots,n\}$ 的一个序列，我们想产生该序列的所有排列(Permutation)。对于 n 个元素的排列，一共有 $n!$ 种。对于这个排列问题，我们如何设计其求解算法呢?

方法 1 固定位置放元素

假设我们能够生成 $n-1$ 个元素的所有排列，我们可以得到如下算法：

(1) 生成元素 $2,3,\cdots,n$ 的所有排列，并且将元素 1 放到每个排列的开头。

(2) 生成元素 $1,3,\cdots,n$ 的所有排列，并将数字 2 放到每个排列的开头。

(3) 重复这个过程，直到元素 $1,2,\cdots,n-1$ 的所有排列都产生，并将元素 n 放到每个排列的开头。

为了方便实现以及分析，我们用数组 $P[1..n]$ 表示 n 个元素 $\{1,2,\cdots,n\}$ 的一个序列。对于一般情形，我们要求 $P[m],P[m+1],\cdots,P[n]$ 的全排列，需要先考虑 $P[m]$，如果能够求出剩余元素 $P[m+1],P[m+2],\cdots,P[n]$ 的所有排列，我们只需将 $P[m]$ 放到每个排列的开头。然后考虑 $P[m+1]$，通过交换 $P[m]$ 和 $P[m+1]$，这样我们仍然只需考虑求剩余元素 $P[m+1],P[m+2],\cdots,P[n]$ 的所有排列即可。然后考虑 $P[m+2]$，通过交换 $P[m]$ 和 $P[m+2]$，这样我们仍然只需考虑求剩余元素 $P[m+1],P[m+2],\cdots,P[n]$ 的所有排列即可。然后依次考虑 $P[m+3],\cdots,P[n]$。当问题规模降为求一个元素 $P[m]$ 的全排列时，问题就极为简单，可作为递归出口。值得注意的是，将 $P[m]$ 和某个 $P[k]$ $(m<k\leqslant n)$ 交换，求出剩余元素的所有排列后，为了避免重复，发生混乱，必须将 $P[m]$ 和 $P[k]$ 交换回去，然后才能继续 $P[m]$ 和 $P[k+1]$ 的交换。按照上面的分析，求 $P[m],P[m+1],\cdots,P[n]$ 的全排列算法如下：

```
Perm1(m)
1  if m = n then output P[1..n]
2  else
3      for j←m to n do
4          P[j]↔P[m]
5          Perm1(m+1)
6          P[j]↔P[m]
```

因此，为了求出 $P[1],P[2],\cdots,P[n]$ 的全排列，我们只需要调用下列算法：

```
GeneratingPerm1( )
1  for j←1 to n do
2      P[j]←j
3  Perm1(1)
```

考虑求 $P[1],P[2],P[3]$ 的全排列，整个递归求解过程可用图 4.5 表示。从图 4.5 的递归树可以看出，求 n 个数的全排列，需要求解 n 个规模为 $n-1$ 的子问

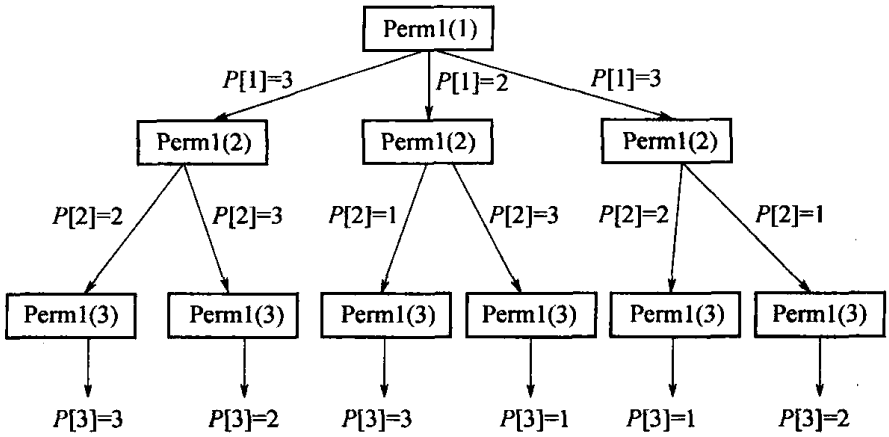


图 4.5

题,因此上述求全排列算法的时间复杂度可用下列递归方程表示:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ nT(n-1) + n & \text{if } n > 1 \end{cases}$$

上述递归方程的解,我们将稍后给出。

方法 2 固定元素找位置

在假设我们能够生成 $n-1$ 个元素的所有排列的情形下,通过交换数组的顺序,我们给出了一个能生成 n 个元素所有排列的算法,下面给出另外一种找全排列的算法。

(1) 把 n 放在 $P[1]$的位置上,并且用子数组 $P[2..n]$来产生前 $n-1$ 个数的排列。

(2) 将 n 放在 $P[2]$上,并且用子数组 $P[1]$和 $P[3..n]$来产生前 $n-1$ 个数的排列。

(3) 将 n 放在 $P[3]$上,并且用子数组 $P[1..2]$和 $P[4..n]$来产生前 $n-1$ 个数的排列。

(4) 重复上述过程,直到将 n 放在 $P[n]$上,并且用子数组 $P[1..n-1]$来产生前 $n-1$ 个数的排列。

为了实现上述过程,我们初始化数组 $P[1..n]$的值为 0,对于元素 n,可以依次把它放到数组的 $P[1],P[2],\cdots,P[n]$位置,在 n 放定一个位置$P[k]$后,剩下的 $n-1$个元素可以放在那些值为 0 的数组元素 $P[1..k-1]$和 $P[k+1..n]$上。依次递归下去,直到数组没有为 0 的元素为止。值得注意的是,在 n 放定一个位置 $P[k]$,找到剩下 $n-1$ 个元素的所有排列后,在找 n 的下一个可放置位置时,即把 n 放在位置 $P[k+1]$前,原来放 n 的位置 $P[k]$一定要置 0。否则,将有某些元素找不到位置。令 m 表示当前可放置的位置数,即数组 P 中,值为 0 的数目。则上述算法的伪代码如下:

```
Perm2(m)
1   if m=0 then output P[1..n]
2   else
3       for j←1 to n do
4           if P[j]=0 then
5               P[j]←m
6               Perm2(m-1)
7               P[j]←0
```

类似地，要求 n 个数的全排列，只需调用下列程序即可。

```
GeneratingPerm2( )
1   for j←1 to n do
2       P[j]←0
3   Perm2(n)
```

考虑求 $P[1],P[2],P[3]$ 的全排列，整个递归求解过程可用图 4.6 表示。

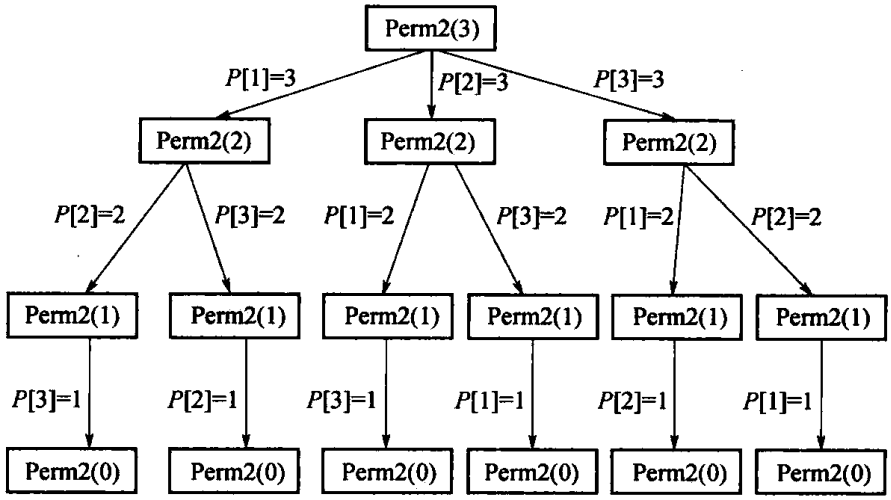


图 4.6

上述求全排列算法的时间复杂度可用下列递归方程表示

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ nT(n-1) + n & \text{if } n > 1 \end{cases}$$

上述递归方程与求全排列的 GeneratingPerm1( )算法的递归方程类似。

5. 多项式求值

假设多项式为 $p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$，要计算多项式 $p(x)$ 在点 x_0 处的值，可以使用 Horner 法则，即将多项式改写为 $p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-2} + xa_{n-1})\cdots))$。令

$$A_i = \begin{cases} a_{n-1} & i = 1 \\ A_{i-1}x_0 + a_{n-i} & i > 1 \end{cases}$$

并令 A 表示多项式系数的一个数组$\{a_0, a_1, \cdots, a_{n-1}\}$，则根据 Horner 法则，可写出如下递归算法：

```
Evalploy(A, x0, i)
1  if i = 1 then return a_{n-1}
2  else
3        return a_{n-i} + x0 × Evalploy(A, x0, i - 1)
```

要计算 $p(x)$ 在点 x_0 处的值，只需调用 Evalploy(A, x_0, n)，算法的时间复杂度为 $O(n)$，比直接计算 $p(x)$ 在点 x_0 处的值有效，4.3 节将对两种算法进行分析，这里就不再介绍。

4.1.2 递归与迭代

前面介绍了递归算法的基本思想及其应用，从上面的分析可以看出递归算法的三条设计规则：

（1）递归出口。必须要有一个递归出口（即结束条件），不用递归即可解决的情形，通常是一个可以直接求解的简单问题。

（2）递归的发展。对于所有的递归情形，每一次新的调用，必须保证不断地接近递归出口。

（3）设计规则。假定所有的递归调用都能起作用。

从上面的递归算法设计实例也可以看出，递归算法比较简单，而且对于很难求解的问题能够给出非常直观的解法，其缺点是由于不断地函数调用，需要保存中间结果以及进行参数的传递，因此需要更多的存储空间和时间。此外，设计递归需要不一样的思维方式。

虽然递归是一种非常强有力而且简单优美的求解技术，但是递归需要更多的存储空间和时间，因此，能够避免用递归的时候，尽量避免或者用相应的迭代算法（Iterated Algorithm）来实现。例如，求函数 $f(n) = 2^n$ 的程序，可改写为下列迭代算法：

```
f(n)
1  total←1
2  for i←1 to n do
3        total←total × 2
4  return total
```

前面介绍的选择排序算法 SelectionSort(i) 虽然简单，但仍然涉及到函数的递归调用，做了一些无用功。事实上，上述算法可以转化为更简单、更有效的迭代算法：

```
IterativedSelectionSort(A)
1  for i←1 to n - 1 do
2       k←i
```

```
3       for j←i+1 to n do
4           if A[j] <A[k] then
5               k←j
6       A[i] ↔ A[k]
```

从上面的例子可以看出，递归算法的计算过程是由复杂到简单再到复杂，而迭代算法的计算过程是由简单到复杂，因此迭代算法的效率更高，在实际的求解过程更常用。

4.2 递归方程的求解

前面我们已经介绍了递归算法的设计，从递归算法的运行时间分析可以看出，要确定递归算法的运行时间，需要求解递归方程。前面已经介绍，算法运行时间复杂度主要由关于问题规模的高阶项决定，因此当我们实际描述并解决一个递归方程时，我们可以忽略递归出口、顶和底等技术细节，例如

$$T(n)=\begin{cases}\Theta(1) & \text{if } n=1 \quad \text{递归出口}\\ T(\lfloor n/2\rfloor)+T(\lceil n/2\rceil)+\Theta(n) & \text{if } n\geq 2 \quad \begin{matrix}\text{顶}\\ \text{底}\end{matrix}\end{cases}$$

忽略了递归出口、顶和底后，得到简化的递归方程

$$T(n)=2T(n/2)+\Theta(n)$$

这样就便于求解，下面我们介绍解递归方程的方法。

4.2.1 替换方法

替换方法(Substitution Method)的主要思想是猜测出递归方程的解，然后代入递归方程，看是否存在满足的条件。虽然这种替换方法比较有效，但是它只应用于比较容易猜出递归解的情形。

替换方法解递归方程的步骤：

(1) 猜测出递归解的形式。

(2) 用数学归纳法找出使解真正有效的常数。

例 4.1 $T(n)=2T(\lfloor n/2\rfloor)+n$

首先猜测出解为 $T(n)=O(n\lg n)$，我们的目的就是为了证明存在某个常数 c，使得 $T(n)\leqslant cn\lg n$。假定这个解对$\lfloor n/2\rfloor$成立，即 $T(\lfloor n/2\rfloor)\leqslant c\lfloor n/2\rfloor\lg(\lfloor n/2\rfloor)$。将其对递归方程作替换，得

$$\begin{aligned}T(n)=2T(\lfloor n/2\rfloor)+n &\leqslant 2(c\lfloor n/2\rfloor\lg(\lfloor n/2\rfloor))+n\\ &\leqslant cn\lg(n/2)+n\\ &\leqslant cn\lg n-cn\lg 2+n\end{aligned}$$

$$= cn\lg n - cn + n$$
$$\leqslant cn\lg n$$

当 $c \geqslant 1$ 时，最后一步显然是成立的，按照 O 符号的定义，证明了猜测是正确的。

例 4.2 选择排序算法运行时间的递归方程为

$$T(n) = T(n-1) + (n-1)$$

为了求解它，猜测 $T(n) = O(n^2)$，假定 $T(n-1) \leqslant c(n-1)^2$。

$$\begin{aligned} T(n) &= T(n-1) + n - 1 \\ &\leqslant c(n-1)^2 + n - 1 \\ &= cn^2 - 2cn + c + n - 1 \\ &\leqslant cn^2 - 2cn + 2c + n - 1 \\ &= cn^2 - (2c-1)(n-1) \\ &\leqslant cn^2 \end{aligned}$$

当 $c \geqslant 1$ 时，最后一步显然是成立的，这就证明了猜测是正确的。

从上面解递归方程的过程可以看出，要利用替换方法来求解递归方程，需要做一个好的猜测。为了得到一个好的猜测，可以利用启发式方法，也可以使用递归树来帮助猜测。如果一个递归方程与先前解过的递归方程类似，那么就有可能猜测出一个类似的解，然后加以证明即可，例如

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$$

虽然该方程多了一个 17，但与前面解过的方程 $T(n) = 2T(\lfloor n/2 \rfloor) + n$ 类似，因此可类似猜测其解为 $T(n) = O(n\lg n)$，证明可参考习题 4.9 的解答。

另外，我们也可以先证出递归解的较宽松的上下界，然后再逐步缩小范围的不确定性。对于任何一个递归方程，我们都可以逐步降低其上界，提高其下界，直到达到正确的渐近界为止。

值得注意的是，在运用渐近符号的定义时，我们很容易犯错误。例如，对于递归方程 $T(n) = 2T(\lfloor n/2 \rfloor) + n$，可以猜测 $T(n) = O(n)$，如果渐近符号使用不当，可能会错误地证明 $T(n) = O(n)$。事实上，假设 $T(n/2) \leqslant c\lfloor n/2 \rfloor$，有

$$T(n) \leqslant 2(c\lfloor n/2 \rfloor) + n \leqslant cn + n$$

但是，由 $T(n) \leqslant (c+1)n$ 并不一定有 $T(n) \leqslant cn$，因而不能说明 $T(n) = O(n)$ 成立。因为 c 是一个常数，我们的错误就在于没有证明归纳假设的正确形式，即证明出 $T(n) \leqslant cn$。

我们再考虑解决 Hanoi 问题算法运行时间的递归方程

$$T(n) = 2T(n-1) + 1$$

我们猜测其解为 $O(2^n)$，即需要证明 $T(n) \leqslant c2^n$ 对于一个选择适当的常数 c 是成立的。将我们所做的猜测代入递归方程中，可得

$$T(n) \leqslant 2c2^{n-1} + 1 = c2^n + 1$$

上述推导并不能证明 $T(n) = O(2^n)$，怎么办呢？我们可以考虑缩小 $T(n)$ 的上界，即证明 $T(n) \leqslant c2^n - 1$ 成立，就可解决上述问题。

有些递归方程看起来有点复杂，通过变量代换，可以变为简单的递归方程，从而能利用替换方法来解。

例 4.3 考虑递归方程

$$T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \lg n$$

对上述递归方程，去掉底，并设 $m = \lg n$，可得

$$T(2^m) = 2T(2^{m/2}) + m$$

再设 $S(m) = T(2^m)$ 即得新的递归方程

$$S(m) = 2S(m/2) + m$$

利用例 4.1 的结果，$S(m) = O(m\lg m)$，因而可以得到

$$T(n) = T(2^m) = S(m) = O(m\lg m) = O(\lg n \lg\lg n)$$

4.2.2 递归树方法

虽然替换方法能够简捷地证明递归方程的解，但是，有时很难猜测出递归方程的解。递归树可以让我们方便地估计出一个好的猜测解，它在描述分治算法的运行时间复杂度时特别有用。用递归树方法(Recusion Tree Method)来求解递归方程的解时，通常是先利用递归树估计出一个好的猜测解，然后利用替换方法进行证明。当然在利用递归树估计猜测解时，如果估计比较准确，也没有必要证明。

构造递归树的方法就是展开递归方程，其展开形式通常用一棵树来表示，然后把每一层的时间进行求和，从而得到时间复杂度的估计。

例 4.4 解递归方程 $T(n) = 3T(\lfloor n/4\rfloor) + cn^2$。

构造递归树如图 4.7 所示，当递归树展开时，子问题的规模逐步缩小，当到达递归出口时，即当子问题的规模为 1 时，递归树不再展开。现在，我们计算一下，递归树总共有多少层。当递归树展开一层，其规模为 $n/4$，当递归树展开到第 2 层时，其规模为 $n/16 = n/4^2$，依次类推，当展开到第 k 层时，其规模 $n/4^k = 1$ 时，不再展开，由此可求得递归树的层数 $k = \log_4 n$。现在我们来计算一下，有多少个叶节点。第 1 层有 3 个节点，第 2 层有 3^2 个节点，依次类推，第 k 层有 3^k 个节点，当 $k = \log_4 n$，即为叶节点，因此叶节点的个数为 $3^{\log_4 n} = n^{\log_4 3}$，而每个叶节点需要的时间为 $T(1)$，因此，整个叶节点的时间为 $n^{\log_4 3}T(1)$。将递归树每一层的时间加起来，可得

$$\begin{aligned} T(n) &= cn^2 + (3/16)cn^2 + (3/16)^2cn^2 + \cdots + (3/16)^{\log_4 n-1}cn^2 + \Theta(n^{\log_4 3}) \\ &= \sum_{i=0}^{\log_4 n-1} (3/16)^i cn^2 + \Theta(n^{\log_4 3}) \end{aligned}$$

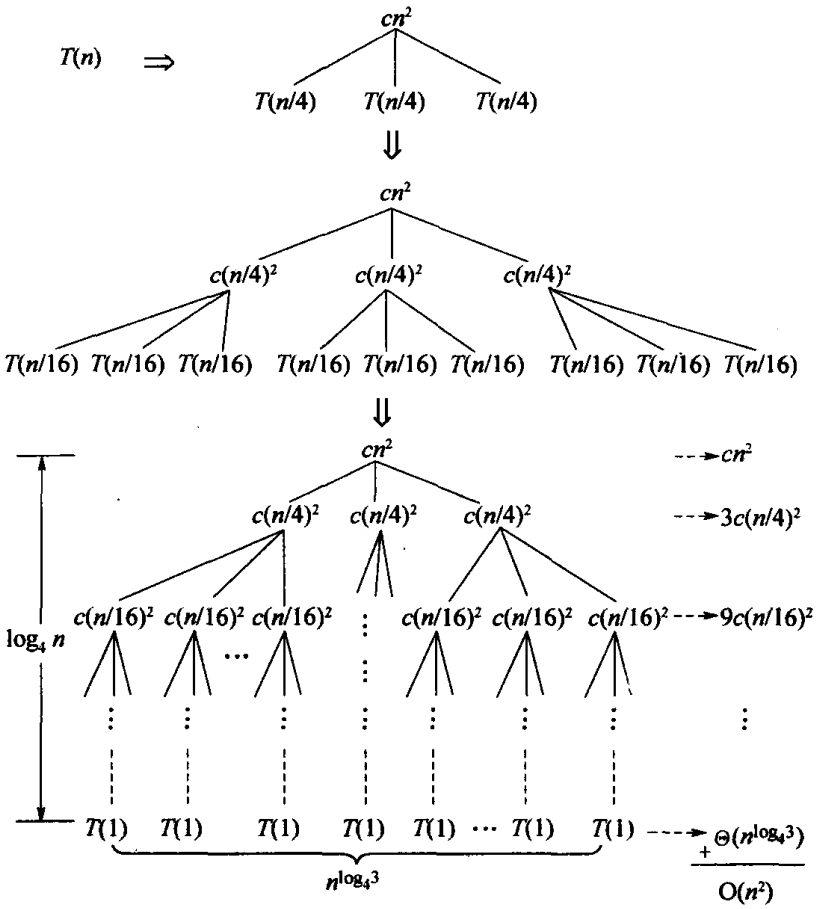


图 4.7

上式最后一步看起来很复杂，很难给出 $T(n)$ 的界。事实上，利用无穷几何级数，可以得到

$$T(n) = \sum_{i=0}^{\log_4 n-1} (3/16)^i cn^2 + \Theta(n^{\log_4 3}) < \sum_{i=0}^{\infty} (3/16)^i cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{1}{1-(3/16)} cn^2 + \Theta(n^{\log_4 3})$$

即

$$T(n) < (16/13)cn^2 + \Theta(n^{\log_4 3}) = O(n^2)$$

由递归树，我们能得到一个好的估计，即 $T(n) = O(n^2)$，现在我们可以用替换方法来验证估计的正确性，即证明 $T(n) = O(n^2)$ 是递归方程 $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$ 的一个上界。我们只需证明 $T(n) \leqslant dn^2$，对于某个常数 $d > 0$ 成立即可。对于以前使用的同一个常数 $c > 0$，我们有

$$T(n) \leqslant 3T(\lfloor n/4 \rfloor) + cn^2 \leqslant 3d\lfloor n/4 \rfloor^2 + cn^2$$

$$\leqslant 3d(n/4)^2 + cn^2 = (3/16)dn^2 + cn^2$$

要证明 $T(n) \leqslant dn^2$，只需证明 $(3/16)dn^2 + cn^2 \leqslant dn^2$ 即可。而当 $d \geqslant (16/13)c$ 时，显然有 $(3/16)dn^2 + cn^2 \leqslant dn^2$ 成立，故所证成立。

下面我们给出一个更复杂的例子。

例 4.5 解递归方程 $T(n) = T(n/3) + T(2n/3) + O(n)$。

构造的递归树如图 4.8 所示，从递归树的根到叶节点的最长的一条路径为 n，$(2/3)n, (2/3)^2 n, \cdots, 1$，第 k 层节点的值 $(2/3)^k n = 1$，因此递归树的层数为 $\log_{3/2} n$。故整个算法的时间复杂度为 $O(cn\log_{3/2} n) = O(n\lg n)$。类似地，可以用替换方法来证明所估计解的正确性，见习题 4.14。

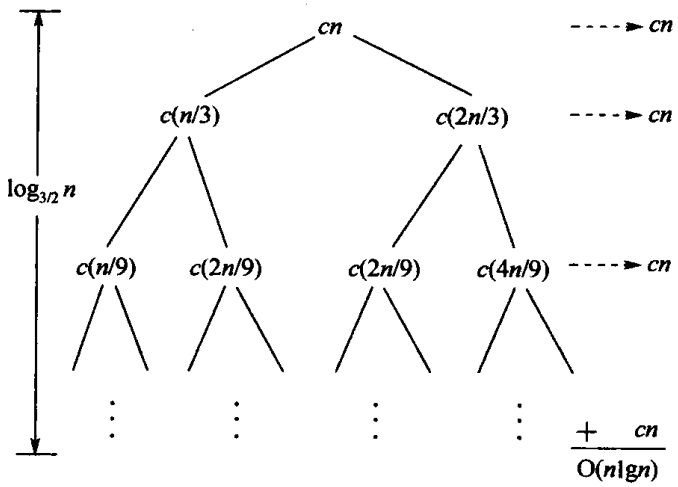


图 4.8

例 4.6 对于生成全排列的递归方程

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ nT(n-1) + n & \text{if } n \geqslant 2 \end{cases}$$

令 $T(n) = n!h(n)$（注意到 $h(1) = 0$），则 $n!h(n) = n(n-1)!h(n-1) + n$，进一步整理可得

$$h(n) = h(n-1) + n/n! = h(n-1) + 1/(n-1)!$$

按照递归树展开，上式可以估计为

$$h(n) = h(1) + \sum_{j=2}^{n} \frac{1}{(j-1)!} = \sum_{j=1}^{n-1} \frac{1}{j!} \leqslant \sum_{j=1}^{\infty} \frac{1}{j!} = (e-1)$$

$$T(n) = n!h(n) \leqslant n!(e-1) \leqslant 2n!$$

因此可猜测 $T(n) = O(nn!)$。

在猜测出递归方程的解后，我们可以采用前面类似的方法予以证明。

4.2.3 公式法

前面讲过的两种方法，需要估计出一个好的解，然后加以证明。因此使用起

来，不是很方便。事实上，对于下列形式的递归方程

$$T(n) = aT(n/b) + f(n) \tag{4.1}$$

其中，$a \geqslant 1$ 和 $b > 1$ 是常数，且 $f(n)$ 是一个渐近正函数，可以使用下面要介绍的公式法(Master Method)，从而更加方便快捷地求得递归方程的解。

递归方程(4.1)描述了如下算法的运行时间：将一个规模为 n 的问题划分为 a 个规模为 n/b 的子问题，其中 a 和 b 为正常数。分别递归地解决 a 个子问题，解每个子问题所需时间为 $T(n/b)$。划分原问题和合并子问题的解所需要的时间由 $f(n)$ 决定。上述算法正是下一章要介绍的分治算法。当然，n/b 可能不是整数，前面说过，忽略底或者顶，并不影响算法运行时间复杂度的分析。而在实际情形中，$T(n/b)$ 可能为 $T(\lfloor n/b \rfloor)$ 或者 $T(\lceil n/b \rceil)$。

定理 4.1 设 $a \geqslant 1, b > 1$，且 a, b 均为常数，设 $f(n)$ 为一个函数，并且假设 $T(n)$ 满足 $T(n) = aT(n/b) + f(n)$，其中 n/b 表示 $\lfloor n/b \rfloor$ 或者 $\lceil n/b \rceil$，那么 $T(n)$ 有如下三种情形的渐近界：

(1) 对某个常数 $\varepsilon > 0$，如果 $f(n) = O(n^{\log_b a - \varepsilon})$，则 $T(n) = \Theta(n^{\log_b a})$。

(2) 如果 $f(n) = \Theta(n^{\log_b a})$，则 $T(n) = \Theta(n^{\log_b a} \lg n)$。

(3) 对某个常数 $\varepsilon > 0$，如果 $f(n) = \Omega(n^{\log_b a + \varepsilon})$，且对某个常数 $c < 1$ 以及任意足够大的 n，有 $af(n/b) \leqslant cf(n)$，则 $T(n) = \Theta(f(n))$。

在以上每一种情形中，我们都把函数 $f(n)$ 与 $n^{\log_b a}$ 作比较。直观地，递归解是由两个函数中数量级较大的一个决定的。

在情形(1)中，函数 $n^{\log_b a}$ 数量级比较大，那么解为 $T(n) = \Theta(n^{\log_b a})$，而在情形(3)中，函数 $f(n)$ 数量级比较大，则解为 $T(n) = \Theta(f(n))$。对于情形(2)，两个函数属于同一个数量级，因此我们乘上一对数因子，得到解 $T(n) = \Theta(n^{\log_b a} \lg n)$。这只是我们的直觉，另外还有一些技术性问题需要加以理解。在情形(1)中，$f(n)$ 的数量级不仅小于 $n^{\log_b a}$，还必须是多项式地小。也就是说，对于某个常数 $\varepsilon > 0$，$f(n)$ 必须渐近地比 $n^{\log_b a}$ 小 n^{ε} 倍。在情形(3)中，$f(n)$ 不仅数量级上大于 $n^{\log_b a}$，而且必须比 $n^{\log_b a}$ 大多项式 n^{ε} 倍，且条件 $af(n/b) \leqslant cf(n)$ 满足。值得注意的是，我们后面碰到的大多数多项式有界的函数都满足条件 $af(n/b) \leqslant cf(n)$。

从定理 4.1 可以看出，设计一个更有效的分治算法，可以通过尽量减少子问题的个数或者减少 $f(n)$ 的数量级来实现。对于如何设计更有效的分治算法，我们下一章将会介绍。

从定理 4.1 也可以看出，公式法只要记住三种情形，就可以很容易地确定许多递归方程的解，通常不需要纸和笔的帮助。下面，我们给出一些应用的例子。

例 4.7 $T(n) = 9T(n/3) + n$。

由上式可得 $a = 9, b = 3, f(n) = n$，且 $n^{\log_b a} = n^{\log_3 9} = n^2 = \Theta(n^2)$。又因为对于 $\varepsilon = 1$，有 $f(n) = O(n^{\log_3 9 - \varepsilon})$，满足定理 4.1 的情形(1)，因此 $T(n) = \Theta(n^2)$。

例 4.8 $T(n)=T(2n/3)+1$。

由上式可得 $a=1, b=3/2, f(n)=1$，且 $n^{\log_b a}=n^{\log_{3/2}1}=n^0=1$。

又因为 $f(n)=\Theta(n^{\log_b a})=\Theta(1)$，满足定理 4.1 的情形(2)，因此 $T(n)=\Theta(\lg n)$。

例 4.9 $T(n)=3T(n/4)+n\lg n$。

由上式可得 $a=3, b=4, f(n)=n\lg n$，且 $n^{\log_b a}=n^{\log_4 3}=O(n^{0.793})$。

因为 $f(n)=\Omega(n^{\log_4 3+\varepsilon})$，其中 $\varepsilon\approx 0.2$。如果我们能够证明 $af(n/b)\leqslant cf(n)$ 对 $f(n)$ 能够成立，则满足定理 4.1 的情形(3)。事实上，对于足够大的 n，当 $c=3/4$ 时，有

$$af(n/b)=3(n/4)\lg(n/4)\leqslant(3/4)n\lg n=cf(n)$$

满足定理 4.1 的情形(3)，因此递归方程的解为 $T(n)=\Theta(n\lg n)$。

4.3 多项式求值实验

4.1.1 节中已经给出了利用 Horner 法则计算多项式 $p(x)$ 在某点 x_0 处的值。为了显示递归算法的效率，我们先设计计算多项式 $p(x)$ 在某点 x_0 处值的直接法，然后利用实验分析方法，对两个算法进行比较。

下面我们给出直接法(DirectPloy)：

```
DirectPloy(A, x0)
1  total←a0
2  for i←1 to n-1 do
3      total←total + ai × multipy(x0, i)
4  return total
```

其中，multipy(x_0, i) 计算 $(x_0)^i$。

两种算法用 C ++ 语言编程实现，并在 CPU 为 2.4GHz，内存为 512MB 的 PC 机上测试规模 n 分别为 600，800，1000，2000，4000，6000，8000，10000 的同样实例，算法的执行时间如表 4.1 和图 4.9 所示，其中时间用秒表示。图 4.9 清楚直观地表示了随着问题规模 n 的增大，执行时间的增长趋势。两种算法都得到同样的值

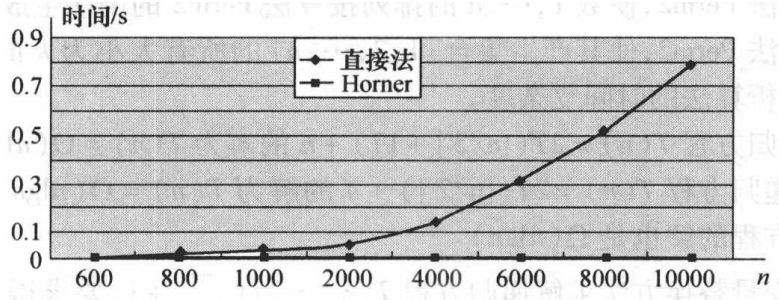


图 4.9

$p(x_0)$，但是从表 4.1 和图 4.9 可以看出，Horner 法则的速度更快，因此其效率更高。

表 4.1　直接法和 Horner 执行时间的实验结果

n	600	800	1000	2000	4000	6000	8000	10000
直接法	0.0	0.015	0.018	0.046	0.141	0.312	0.515	0.785
Horner	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

4.4 小　结

本章递归方程的求解主要取材于文献[1]，其他内容参考了文献[5]。递归技术是算法最基本的技术之一，本章首先介绍了递归算法的设计以及时间复杂度的分析方法，也介绍了如何证明递归算法的正确性。最后一节介绍了求解递归方程的三种方法。由于篇幅所限，公式法的理论证明，主要基于递归树的方法，本书没有介绍，感兴趣的读者可以阅读文献[1]。

习题

4.1　设计一个递归算法，在有 n 个元素的序列 $A[1..n]$ 中搜索元素 x。

4.2　给定含有 n 个元素的数组 A，请设计出一个算法找出数组 A 中出现次数最多的元素，并输出该元素出现的次数。例如，$A=\{1,2,2,2,3,5\}$。A 中出现次数最多的元素是 2，出现次数为 3。

4.3　将第 1 章的插入排序算法改写为递归算法，并分析其时间复杂度。

4.4　证明算法 GeneratingPerm1 的正确性。

4.5　修改算法 Perm1，使其能够按照字典序输出排列。

4.6　请仔细解释，为什么算法 GeneratingPerm2 中，当过程 Perm2 以 Perm2(m)($m>0$)形式调用时，数组 P 恰好包含 m 个 0，并且 Perm2($m-1$)将恰好执行 m 次。

4.7　修改算法 Perm2，使数 $1,\cdots,n$ 的排列按算法 Perm2 的倒序生成。

4.8　修改算法 Perm2，使其产生集合 $\{1,2,\cdots,n\}$ 的所有大小为 k 的子集，$1\leqslant k\leqslant n$，并分析算法的时间复杂度。

4.9　证明递归方程 $T(n)=2T(\lfloor n/2\rfloor+17)+n$ 的解为 $T(n)=O(n\lg n)$。

4.10　证明递归方程 $T(n)=2T(\lfloor n/2\rfloor)+n$ 的解为 $T(n)=O(n\lg n)$，并证明这个递归方程的解也是 $\Omega(n\lg n)$。

4.11　利用变量替换方法求解递归方程 $T(n)=2T(\sqrt{n})+1$，要求得到的解应当是渐近紧界，不必担心值是否为整数。

4.12 画出递归方程 $T(n)=4T(\lfloor n/2 \rfloor)+cn$ 的递归树，并给出其解的渐近紧界，然后用替换方法证明给出的界，其中 c 是一个常数。

4.13 利用递归树方法估计递归方程 $T(n)=T(n-a)+T(a)+cn$ 的渐近紧界，其中 a,c 是常数，且 $0<a<1, c>0$。

4.14 证明递归方程 $T(n)=T(n/3)+T(2n/3)+O(n)$ 的解为 $T(n)=O(n\lg n)$。

4.15 用公式法求解下列递归方程的渐近紧界：

(1) $T(n)=4T(n/2)+n$。

(2) $T(n)=4T(n/2)+n^2$。

(3) $T(n)=4T(n/2)+n^3$。

4.16 公式法能否应用于递归方程 $T(n)=4T(n/2)+n^2\lg n$？为什么？给出此递归方程的渐近上界。

实验题

4.17 完成 XOJ 如下题目：1005，1071，1082。

4.18 完成 POJ 如下题目：1007，1173，1187，1240，1426，2083，3601。

第5章 分治算法

5.1 算法思想

许多实用的算法都具有递归的结构：为解决一个给定的问题，它们递归地调用自己一次或者多次来解决与该问题紧密相关的子问题。分治算法(Divide and Conquer)就是这样一个具有递归结构的算法，其基本思想是把一个问题分解成若干个子问题(这些子问题与原问题相似，但是问题的规模小)，然后递归地解决子问题，最后把子问题的解组合成原问题的解。递归与分治亲同手足，互不分离，经常同时应用在算法设计之中，并由此产生许多高效的算法。

分治算法在求解问题的时候，通常遵循三个步骤：

(1) 把一个问题分解成若干个子问题。

(2) 通过递归地解决子问题来解决原问题。如果子问题的规模小到可以用直接的方法求出解，就停止递归。

(3) 把这些子问题的解组合成原问题的解。

当一个算法的过程中含有对本身的递归调用时，它的运行时间常用一个递归方程来表示。递归方程描述了求解一个规模为 n 的问题所需要的运行时间，可以由其子问题的运行时间来决定。对于递归方程，如果我们知道递归的初始边界，就能够很方便地利用第4章介绍的方法来求解递归方程。

分治算法的运行时间主要由它的三个求解步骤所决定。令 $T(n)$ 表示求解一个规模为 n 的问题所需要的运行时间。设问题的规模小到可以用直接的方法求出解，所花费的时间为 $\Theta(1)$，即存在一个正整数 c，使得当 $n \leqslant c$ 时，解这个问题非常简单，只需要常数级的时间就可以得到该问题的解。假设我们把问题分解成 a 个子问题，每个子问题的大小为 n/b，值得注意的是，n/b 可能是 $\lfloor n/b \rfloor$ 或者 $\lceil n/b \rceil$，这里忽略掉底和顶是因为第4章已经分析过，去掉底和顶，并不影响递归方程的求解。令把问题分解成 a 个子问题所花费的时间为 $D(n)$，将每个子问题的解组合起来，所花费的时间为 $C(n)$，则我们可以得到以下递归方程：

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leqslant c \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

如果将时间 $D(n)$ 和 $C(n)$ 合并成函数 $f(n)$，则许多分治算法的时间复杂度可以由第4章介绍的公式法求出。从上面的递归方程可以看出，分治算法的优点是它的运行时间常常可以很容易由其子问题的运行时间而得到。正如第4章所介绍

的，我们在设计分治算法的时候，尽量减少子问题的个数 a 或者减少 $f(n)$ 的数量级，从而可以设计出更有效的分治算法。

下面介绍分治算法的应用。

5.2 合并排序

前面介绍了插入排序算法和选择排序算法，现在我们利用分治算法的思想，给出一个合并排序(Merge Sort)算法，其基本步骤如下：

(1) 将包含 n 个元素的序列均分成各自包含 $n/2$ 个元素的子序列。

(2) 对两个子序列递归划分。

(3) 把两个已经排序的子序列合并成一个有序序列。

其详细的伪代码描述如下：

```
MergeSort(A,p,r)
1   if p < r then
2           q←⌊(p+r)/2⌋
3           MergeSort(A,p,q)
4           MergeSort(A,q+1,r)
5           Merge(A,p,q,r)
```

MergeSort(A,p,r)算法中，要排序的元素保存在数组 A 中，当前要排序的范围是数组 A 中，位置 p 到 r 之间的元素。当 $p \geq r$ 时，停止递归调用。否则，找出中间划分点 q，并分别递归求解两个子问题。两个子问题解决后，将两个子问题的解合并在一起，就得到原问题的解。求解原问题，只需调用 MergeSort$(A,1,n)$即可。

合并算法中的关键步骤就是如何把两个有序序列合并成一个有序序列。我们用了辅助函数 Merge(A,p,q,r)来完成这个步骤，其中 p,q,r 是数组下标，且有 $p \leqslant q < r$。该函数假定数组 $A[p..q]$ 和 $A[q+1..r]$ 是已排好序的，然后将它们合并为一个有序的子数组 $A[p..r]$。其详细的合并过程可以描述如下：

```
Merge (A,p,q,r)
1     n1←q-p+1
2     n2←r-q
3     for i←1 to n1 do
4           L[i]←A[p+i-1]
5     for j←1 to n2 do
6           R[j]←A[q+j]
7     L[n1+1]←∞
8     R[n2+1]←∞
9     i←1
10    j←1
11    for k←p to r do
```

```
12          if L[i] ≤ R[j] then
13                  A[k] ← L[i]
14                  i ← i + 1
15          else    A[k] ← R[j]
16                  j ← j + 1
```

Merge(A,p,q,r)算法用两个子数组$L[1..n_1+1]$,$R[1..n_2+1]$分别保存两个已经排好序的序列,以便腾出A中的位置来保存合并后的元素。同时,两个数组均多开辟一个存储单元来存储一个非常大的整数∞,其好处是方便元素比较。由于数组L和R中的元素均有序且从小到大排列,因此,我们只需依次比较两个数组中最前面的两个元素$L[i]$和$R[j]$,将小的一个元素存储在A数组中,重复上述过程,直到两个数组为空。图 5.1 给出了一个例子。其中实线箭头指向逐步划分的过程,而虚线箭头指向逐步合并的过程。

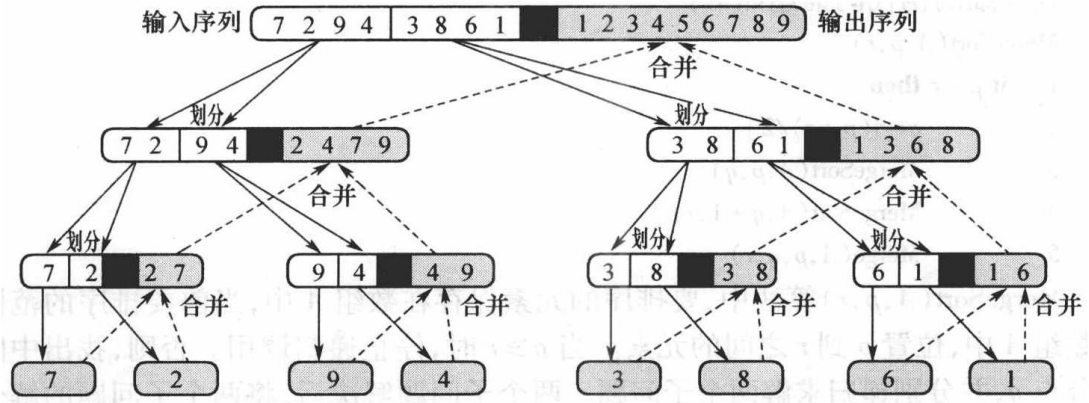


图 5.1

由于合并排序算法主要取决于合并算法,如果我们能证明合并算法的正确性,则合并排序算法的正确性就不难证得,下面我们证明合并算法的正确性。

考虑合并算法的循环不变量:

在 **for** 循环执行第k次迭代前,子数组$A[p..k-1]$中有序地存放了$L[1..n_1+1]$和$R[1..n_2+1]$中最小的$k-p$个元素,同时,$L[i]$和$R[j]$是各自数组中还未复制到A中的最小的元素。我们下面就是要证明,执行 **for** 循环的第k次迭代后,循环不变量依然为真。同样,这个循环不变量在算法结束时,依然为真,从而可以证明算法的正确性。

初始步:在 **for** 循环执行$k=p$之前,子数组$A[p..p-1]$中不存在元素。这个空数组包含了$k-p=0$个L和R中的最小元素,同时$i=j=1$,$L[1]$和$R[1]$即为还未复制到数组A中的两个数组中各自最小的元素,因此循环不变量为真。

归纳步:假设在执行 **for** 循环第k次迭代前,循环不变量为真,为了能清楚了解每次循环中不变量是否为真,不妨假定$L[i]\leqslant R[j]$。此时$L[i]$是还未复制到A的最小元素。因为假设$A[p..k-1]$有序地存放了$L[1..n_1+1]$和$R[1..n_2+1]$中最

小的 $k-p$ 个元素，执行 **for** 循环的第 k 次迭代后，在代码行第 13 行把 $L[i]$ 复制到 $A[k]$ 后，数组 $A[p..k]$ 将包含 $k-p+1$ 个最小元素。整数 k（在 **for** 循环中增加）和 i（代码行 14）在每次循环后都发生改变，因而代码第 13 至 14 行的执行使得循环不变量为真。即在执行 **for** 循环第 $k+1$ 次迭代前，循环不变量为真。

终止步：程序结束时 $k=r+1$，子数组 $A[p..k-1]$ 也就是 $A[p..r]$ 有序地包含了 $k-p=r-p+1$ 个 $L[1..n_1+1]$ 和 $R[1..n_2+1]$ 中最小的元素。数组 L 和 R 总共包含了 $n_1+n_2+2=r-p+3$ 个元素。此时，除了两个最大的元素没有合并到 A 中外，L 和 R 中的其他元素都已经有序地合并到 A 中了。因此循环不变量为真，合并算法正确地把两个有序的数组合并成一个有序的数组。

合并排序算法中，令 $T(n)$ 表示对大小为 n 的数组进行排序，对数组均匀划分后，可得到两个规模为 $n/2$ 的子问题，合并算法对两个子问题的解进行合并，最多需要时间 $\Theta(n)$，因此

$$T(n) = 2T(n/2) + \Theta(n)$$

利用前面介绍过的公式法，我们可以得到合并排序算法的时间复杂度为 $T(n)=O(n\lg n)$。由于排序问题的下界为 $\Omega(n\lg n)$，因此，合并排序算法是渐近最有效的算法，对解决规模大的问题比较有效。

5.3 快速排序

前面我们已经介绍了排序问题可以用一个分治算法来求解，下面我们可以设计另一个分治算法，即快速排序算法来求解排序问题。快速排序算法，是一个非常有创意的算法，对规模比较大的排序问题非常有效，因而被誉为 20 世纪最好的 10 个算法之一。C. Anthony R. Hoare 就因为其有代表性的贡献——快速排序（Quick Sort）算法，而获得 1980 年的 Turing 奖。

快速排序算法采用分治算法的思想。令对 n 个元素排序的问题用数组 $A[1..n]$ 表示，现在我们考虑一般问题 $A[p..r]$ 的分治求解过程：

（1）把问题 $A[p..r]$ 分解成两个子问题 $A[p..q-1]$，$A[q+1..r]$，并且满足 $A[p..q-1]$ 中的每个元素都小于或等于 $A[q]$，而 $A[q+1..r]$ 中的元素都比 $A[q]$ 大，其中 $A[q]$ 称为支点。计算索引 q 的过程就是划分子问题的过程。

（2）对 $A[p..q-1]$，$A[q+1..r]$ 这两个子问题分别调用 QuickSort 进行递归求解。

当每个子问题都已经解决时，数组就已经排好序了，每个元素都在所要求的正确位置，因此也就没有必要把子问题的解组合在一起了，最后整个数组也就全部排好序了。因此分治算法的第三步对于快速排序算法来说没有必要。下面我们给出快速排序算法的伪代码：

```
QuickSort(A,p,r)
1    if p < r then
```

```
2        q←Partition(A,p,r)
3        QuickSort(A,p,q-1)
4        QuickSort(A,q+1,r)
```

要解决原问题,只需要调用 QuickSort(A,1,n)即可。

QuickSort(A,p,r)算法的过程非常简洁,但是其中 Partition(A,p,r)过程非常关键,下面我们分析该过程。由于在分治算法划分的过程中,需要确定一个支点,这里我们简单地取 $A[r]$为支点,为了将比它大的元素以及不比它大的元素分离开来,我们需要对数组进行扫描,将数组元素逐一与 $A[r]$进行比较,同时还引进两个变量 i,j 来标记两个子问题 $A[p..i]$和 $A[i+1..j]$的大小。具体地,Partition(A,p,r)过程可以描述如下:

```
Partition(A,p,r)
1  x←A[r]
2  i←p-1
3  for j←p to r-1 do
4      if A[j]≤x then
5              i←i+1
6              A[i]↔A[j]
7  A[i+1] ↔ A[r]
8  return i+1
```

下面给出一个例子,分析一下快速排序算法的运行过程。

图 5.2(a)~(i)给出了一般问题 $A[p..r]$的划分过程,图 5.2(a)给出了执行扫描前的状态,$A[r]=4$ 为支点,将 $A[j]=2$ 与 $A[r]$比较,$A[j]<A[r]$,因此 i 往前移动一格,此时 $i=j$,$A[j]$跟自己交换,得到图 5.2(b)。此时 $A[j]=8$ 与 $A[r]$比较,$A[j]>A[r]$,因此不改变,j 往前移动一格,得到图 5.2(c)。重复上述过程,最终得到两个子问题 $A[p..q-1]$和 $A[q+1..r]$。下面我们给出快速排序的运行时间复杂度分析。

对快速排序算法来说,最坏的情形发生在 Partition(A,p,r)过程产生的两个子数组中,一个有 $n-1$ 个元素而另一个为 0,即有

$$T(n) = T(n-1) + O(n)$$

利用递归树方法展开上式可以得到

$$\begin{aligned} T(n) &= T(n-1) + O(n) \\ &= T(n-2) + O(n) + O(n-1) \\ &= \cdots = O(n^2) \end{aligned}$$

这样,我们可以估计出快速排序算法的最坏情形时间复杂度为 $O(n^2)$,然后可以利用替换方法加以证明,读者可以自己证明一下。

快速排序算法最好情形发生在“二等分”的时候,即 Partition(A,p,r)生成的两个子问题,一个子问题的规模为$\lfloor n/2 \rfloor$,另一个子问题的规模为$\lceil n/2 \rceil - 1$。在这种

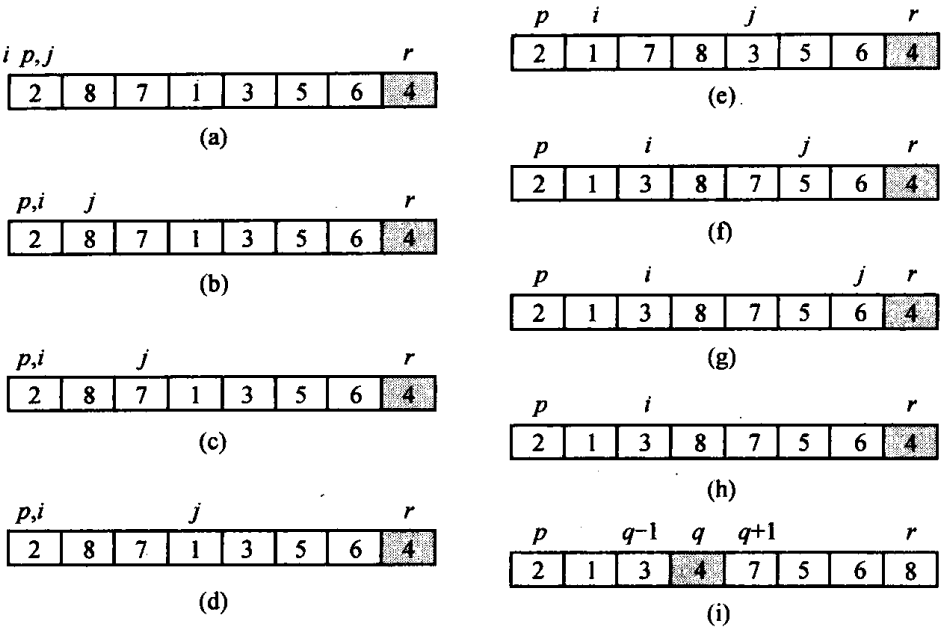


图 5.2

情况下,快速排序的速度是最快的。忽略底和顶,它的运行时间表达式为

$$T(n) = 2T(n/2) + \Theta(n)$$

利用第 4 章介绍的公式法,我们可以得到快速排序算法的最好情形时间复杂度为 $T(n) = \Omega(n\lg n)$。

下面介绍快速排序算法平均情形时间复杂度的分析。由于快速排序算法只与要排序的序列值的相对顺序有关,而与具体的值无关,因此不妨假设要排序的序列,其值是各不相同的。为进一步简化分析,我们假设序列的每种排列作为算法的输入,其机会是均等的。把输入序列看成是随机的,这就保证了数组中的每个元素被选为支点的概率是相等的,其概率为 $1/n$。假设第 q 小的元素被选择为支点,则原问题可以分解成两个子问题,一个规模为 $q-1$,另一个规模为 $n-q$。设 $T(n)$ 表示对一个大小为 n 的数组 A 进行快速排序的平均运行时间,则有

$$T(n) = \sum_{q=1}^{n} \frac{1}{n}(T(q-1) + T(n-q) + n - 1)$$

$$= \frac{1}{n}\sum_{q=1}^{n}(T(q-1) + T(n-q)) + n - 1$$

由于 $\sum_{q=1}^{n} T(q-1) = \sum_{q=1}^{n} T(n-q)$,因此有

$$T(n) = \frac{1}{n}\sum_{q=1}^{n}(T(q-1) + T(n-q)) + n - 1$$

$$= \frac{2}{n}\sum_{q=1}^{n} T(q-1) + n - 1$$

即

$$nT(n) = 2\sum_{q=1}^{n} T(q-1) + n(n-1) \tag{5.1}$$

类似有

$$(n-1)T(n-1) = 2\sum_{q=1}^{n-1} T(q-1) + (n-1)(n-2) \tag{5.2}$$

将式(5.1)和式(5.2)相减,可得

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2(n-1)$$

整理上式,可得 $nT(n) = (n+1)T(n-1) + 2(n-1)$,两边同除以 $n(n+1)$,可得

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)}$$

令 $D(n) = \frac{T(n)}{n+1}$,得到

$$D(n) = \begin{cases} 0 & \text{if } n = 1 \\ D(n-1) + \dfrac{2(n-1)}{n(n+1)} & \text{if } n > 1 \end{cases}$$

利用递归树的方法,可以得到

$$D(n) = 2\sum_{j=1}^{n} \frac{(j-1)}{j(j+1)} = \Theta(\lg n)$$

故快速排序算法的平均时间复杂度为

$$T(n) = (n+1)\Theta(\lg n) = \Theta(n\lg n)$$

快速排序算法虽然不是渐近最有效的算法,但是当求解规模比较大的问题时,效果不会比合并排序算法差,如果利用随机选择支点的策略,则效率会更高。

5.4 大整数乘法

设 u 和 v 表示两个 n 位的大整数,整数乘法问题是计算 uv 的值。对于这个问题,当两个整数的位数比较短时,这个问题很简单,但是当 n 非常大时,计算就变得非常复杂。按照通常的乘法,则需要 $\Theta(n^2)$ 次位运算,如图 5.3 所示。在密码系统与信息安全、数字信息、加密解密等领域,经常遇到大整数乘法问题,因此,寻找其更快的求解算法变得非常重要。下面介绍求解大整数乘法的分治算法。

假设 u 和 v 分别用 n 位的二进制表示,每个整数可分解为高位和低位两部分,每部分为 $n/2$ 位。假设整数 u 分成 w 和 x 两部分,整数 v 分成 y 和 z 两部分,如图

5.4 所示。

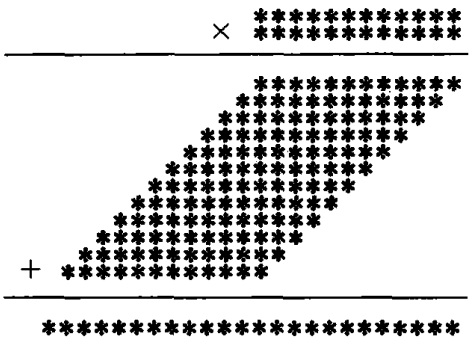


图 5.3

因此，整数 u 和 v 可以表示为

$$u = w2^{n/2} + x, v = y2^{n/2} + z$$

则

$$\begin{aligned} uv &= (w2^{n/2} + x)(y2^{n/2} + z) \\ &= wy2^n + (wz + xy)2^{n/2} + xz \end{aligned}$$

这样原问题就可以转化为位数更少的两个整数相乘的问题。值得注意的是，乘以 2^n 表示向左位移 n 位，这个运算耗时 $\Theta(n)$。

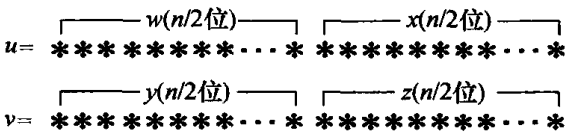


图 5.4

由上述递归方程可知，要计算出 $T(n)$，需要四次两个整数的乘法以及三次加法运算，而后者耗时 $\Theta(n)$，即

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 4T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

利用公式法可知 $T(n) = O(n^2)$。

上述算法从时间复杂度来看，比通常的乘法没有什么优势。如果我们能减少子问题的个数 a（这里 $a = 4$），则可以提高计算的效率。事实上，如果我们利用恒等式：$wz + xy = (w + x)(y + z) - wy - xz$，则可以得到如下算法：

```
Multiply2Int(u,v)
1  if |u| = |v| = 1 then return uv
2  else
3        A1←Multiply2Int(w,y); A2←Multiply2Int(x,z)
4        A3←Multiply2Int(w+x,y+z)
5        return A1·2^n + (A3 - A1 - A2)·2^(n/2) + A2
```

其中 $|u|$ 表示 u 的位数，上述 Multiply2Int(u,v) 算法虽然增加了加法运算的次数，但是加法运算的耗时不多，特别是减少了乘法运算次数，只需要三次乘法运算，即只需求解 3 个子问题，从而得到递归方程：

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

利用公式法，我们得到 $T(n) = O(n^{\lg 3}) = O(n^{1.59})$。这样，我们得到了具有更低时间复杂度的算法。

5.5 矩阵乘法

给出两个 $n \times n$ 的矩阵 $\boldsymbol{A}$ 和 $\boldsymbol{B}$，目的是要求矩阵的乘积，即计算 $\boldsymbol{C} = \boldsymbol{A} \times \boldsymbol{B}$。对于这个问题，可以用公式来计算，即

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

一般地，计算 $n \times m$ 矩阵 $\boldsymbol{A}$ 和 $p \times q$ 矩阵 $\boldsymbol{B}$ 乘积的算法，可以描述如下：

```
MatrixMultiply(A,B)
1   if m ≠ p then
2       print "Two matrices cannot multiply"
3   else for i←1 to n do
4                   for j←1 to q do
5                               C[i,j]←0
6                               for k←1 to m do
7                                       C[i,j]←C[i,j] +A[i,k] × B[k,j]
8   return C
```

MatrixMultiply(A,B)算法的运行时间复杂度为 $O(nmq)$。我们是否可以得到更有效的算法呢？现在我们考虑计算两个 $n \times n$ 的矩阵乘积，并且假定 $n = 2^k$ $(k \geqslant 0)$。如果 $n \geqslant 2$，那么 $\boldsymbol{A}$，$\boldsymbol{B}$ 可以被分成 4 个 $n/2 \times n/2$ 的矩阵：

$$\boldsymbol{A} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad \boldsymbol{B} = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad \boldsymbol{C} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

利用块矩阵的乘积，矩阵 $\boldsymbol{C}$ 可以表示为

$$\boldsymbol{C} = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

因此，原问题可以划分成计算 8 个子问题的乘积问题，因此，两个 $n \times n$ 矩阵乘积的计算量是 2 个 $n/2 \times n/2$ 矩阵乘积计算量的 8 倍，再加上 $n/2 \times n/2$ 阶矩阵相加的 4 倍，后者最多需要 $\Theta(n^2)$，因此有

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 8T(n/2) + \Theta(n^2) & \text{if } n > 1 \end{cases}$$

利用前面介绍的公式法，可以知道上述算法的时间复杂度为 $O(n^3)$，它跟两个矩阵直接相乘的计算量没有什么差别。是否可以算得更快呢？Strassen 通过仔细地研究，通过牺牲加、减的开销，来减少乘法的次数，提出了 Strassen 算法，其思路如下：

要计算矩阵乘积

$$C = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

只需要计算

$$C = \begin{pmatrix} d_1 + d_4 - d_5 + d_7 & d_3 + d_5 \\ d_2 + d_4 & d_1 + d_3 - d_2 + d_6 \end{pmatrix}$$

其中

$$d_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$
$$d_2 = (A_{21} + A_{22})B_{11}$$
$$d_3 = A_{11}(B_{12} - B_{22})$$
$$d_4 = A_{22}(B_{21} - B_{11})$$
$$d_5 = (A_{11} + A_{12})B_{22}$$
$$d_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$
$$d_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

由上面可知，两个 $n \times n$ 阶的矩阵相乘的计算量是两个 $n/2 \times n/2$ 矩阵相乘的7倍，加上它们进行加或减运算的18倍，加减运算共需要 $\Theta(n^2)$，因此有

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 7T(n/2) + \Theta(n^2) & \text{if } n > 1 \end{cases}$$

根据 Strassen 算法，原来需要求解8个子问题，现在只需要求解7个子问题，当然由此会带来加减运算量的增加。根据公式法，可得上述递归方程的解为 $T(n) = O(n^{\lg 7}) = O(n^{2.81})$，因而 Strassen 算法的效率更高。虽然 Strassen 算法的渐近效率很高，但是这些漂亮的算法由于常数系数很大，而缺乏应用价值，因此常用的还是矩阵直接相乘的算法。

5.6 残缺棋盘游戏

一个残缺的棋盘表示棋盘中有一个格子残缺，图 5.5(a) 表示一个 4×4 的残缺棋盘，其中黑格表示残缺格。给定一个 L 型的能覆盖三个格子的三格板，三格板有4个放置方向，如图 5.5(b) 所示。

残缺棋盘游戏问题就是给定一个 $2^n \times 2^n$ 的残缺棋盘，问如何放置三格盘，除了残缺格外，棋盘中其他格子被三格板覆盖，并满足放置的三格板互不重叠。现在我们很容易地计算出，对于一个 $2^n \times 2^n$ 的残缺棋盘，除了残缺格外，共需要放置 $(2^n \times 2^n - 1)/3$ 个三格板。图 5.6(a) ~ (e) 给出了一个放置过程，共需要5个三格板。

从图 5.7 可以看出，放置一个三格板后，棋盘中间区域就已经放置好，如果把这个三格板看成三个残缺格，原来的残缺棋盘就可以分成四个残缺棋盘。虽然有

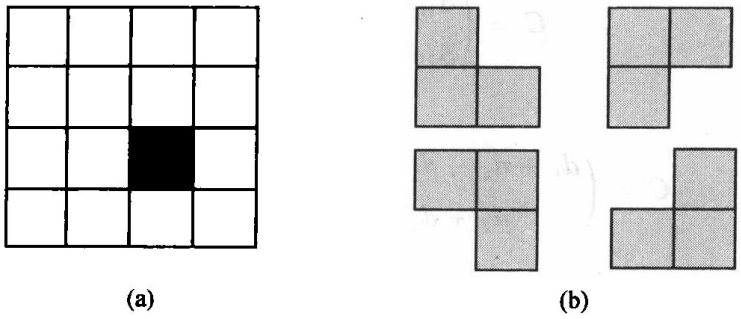


图 5.5

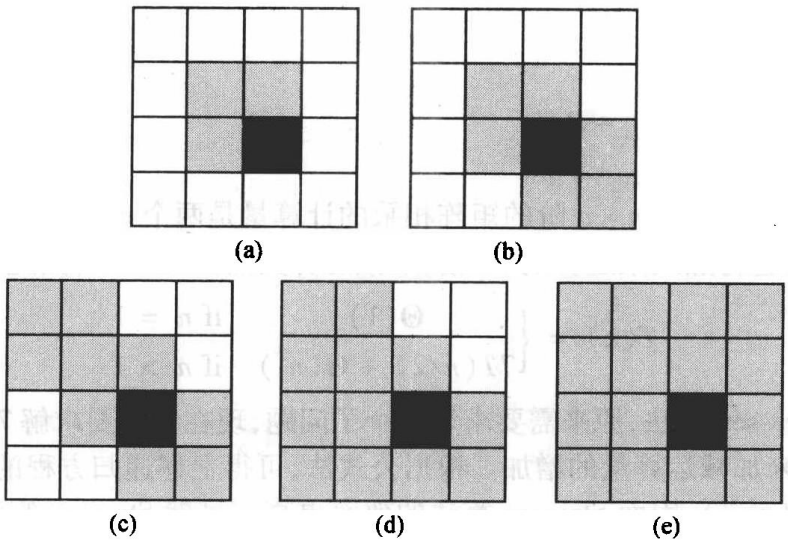


图 5.6

三个棋盘与原来棋盘不相似,但是通过放置一个三格板,可以将原来不相似的三个棋盘构造成残缺棋盘,这样这三个棋盘的放置方法可以采用类似原来残缺棋盘的放置方法。当残缺棋盘的规模降为 2×2 时,这时棋盘不再分解,递归终止,因为这时残缺棋盘只要放置一个三格板就可以铺满。这启示我们,残缺棋盘的游戏问题可以利用分治算法来求解。要完成残缺棋盘游戏,必须定位棋盘的位置和残缺格的位置,将棋盘一分为四,确定残缺格的位置后,我们便可以知道该对哪三个棋盘分别补一个残缺格。例如对图 5.7,确定残缺格的位置后,我们便可以知道,左上棋盘的右下角为残缺格,左下棋盘的右上角为残缺格,右上棋盘的左下角为残缺格。

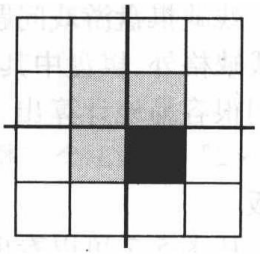


图 5.7

令二维整数数组变量 Board 表示棋盘,Board[0,0] 表示棋盘中左下角的方格,t 用来记录放置的三格板的数目,tr 表示残缺棋盘中左下角方格所在行,tc 表示棋

盘中左下角方格所在列，dr 表示残缺格所在行，dc 表示残缺格所在列，2^{size} 表示棋盘的行数或列数，则上述分治算法的伪代码可以描述如下：

```
TileBoard(tr,tc,dr,dc,size)
1       if  size = 1 return ok
2       tile←tile + 1; t←tile
3       s←size/2
4       if  dr < tr + s and dc < tc + s then
5               TileBoard(tr,tc,dr,dc,s)
6       else    Board[tr + s - 1,tc + s - 1]←t
7               TileBoard(tr,tc,tr + s - 1,tc + s - 1,s)
8       if  dr < tr + s and dc≥tc + s then
9               TileBoard(tr,tc + s,dr,dc,s)
10      else    Board[tr + s - 1,tc + s]←t
11              TileBoard(tr,tc + s,tr + s - 1,tc + s,s)
12      if  dr≥tr + s and dc < tc + s then
13              TileBoard(tr + s,tc,dr,dc,s)
14      else    Board[tr + s,tc + s - 1]←t
15              TileBoard(tr + s,tc,tr + s,tc + s - 1,s)
16      if  dr≥tr + s and dc≥tc + s then
17              TileBoard(tr + s,tc + s,dr,dc,s)
18      else    Board[tr + s,tc + s]←t
19              TileBoard(tr + s,tc + s,tr + s,tc + s,s)
```

TileBoard(tr,tc,dr,dc,size)算法的第 1 行，如果棋盘大小为 $2^1 \times 2^1$，则算法结束，否则，将棋盘划分为 4 个小棋盘，然后根据残缺格所在的位置，递归调用 4 次，以便解决 4 个子问题。其中，在算法实现时，tile 是全局变量，初始为 0。

下面分析一下 TileBoard(tr,tc,dr,dc,size)的时间复杂度。

令 $T(n)$ 表示完成一个规模为 $2^n \times 2^n$ 的残缺棋盘游戏所花费的时间，当棋盘规模为 2×2 时，游戏显然可以在常数时间 $\Theta(1)$ 内完成。因此，$T(n)$ 的递归方程为

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 4T(n-1) + \Theta(1) & \text{if } n > 1 \end{cases}$$

利用递归树的方法，将其展开，可得

$$\begin{aligned} T(n) &= 4T(n-1) + \Theta(1) \\ &= 4[4T(n-2) + \Theta(1)] + \Theta(1) \\ &= 4^2T(n-2) + 4\Theta(1) + \Theta(1) \\ &= 4^3T(n-3) + 4^2\Theta(1) + 4\Theta(1) + \Theta(1) \\ &= \cdots \\ &= 4^{n-1}T(1) + 4^{n-2}\Theta(1) + \cdots + 4\Theta(1) + \Theta(1) \end{aligned}$$

$$= \Theta(4^{n-1})$$

上述例子表明，分治法通过把问题分为较小的子问题来解决原问题，简化或减少了求解原问题的计算量。

5.7 快速傅里叶变换(FFT)

FFT 算法是求解离散傅里叶变换问题的快速算法，在数字信号处理、图像处理、计算大整数乘法、求解偏微分方程等领域具有广泛的应用。FFT 算法的影响是如此的巨大，因此被誉为 20 世纪最好的算法之一。

离散傅里叶变换问题(DFT)是计算下述公式

$$b_j = \sum_{k=0}^{n-1} a_k \omega^{kj}, j = 0,1,\cdots,n-1$$

其中 $\omega = e^{2\pi i/n}$ 为 n 次单位元根，$i = \sqrt{-1}$，且 $A = \{a_0, a_1, \cdots, a_{n-1}\}$ 已知。DFT 的逆变换为

$$a_k = \frac{1}{n}\sum_{j=0}^{n-1} b_j \omega^{-jk}, k = 0,1,\cdots,n-1$$

如果我们找到了 DFT 的快速求解算法，其逆变换可以同样快速求得。DFT 与多项式的计算关系紧密，事实上，考虑多项式 $p(x)$

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}$$

DFT 中的 b_j 是 $p(x)$ 在 $x = \omega^j$ 处的值。第 4 章中我们已经介绍过 Horner 法则，其计算 $p(x)$ 在某点的值，时间复杂度为 $O(n)$，因此要计算 $b_j, j = 0,1,\cdots,n-1$，其时间复杂度为 $O(n^2)$。利用 FFT 算法，其时间复杂度仅为 $O(n\lg n)$，下面我们介绍该算法。

将多项式 $p(x)$ 分解为奇次幂部分和偶次幂部分

$$\begin{aligned} p(x) &= (a_1 x + a_3 x^3 + \cdots + a_{n-1} x^{n-1}) + (a_0 + a_2 x^2 + \cdots + a_{n-2} x^{n-2}) \\ &= (a_1 + a_3 x^2 + \cdots + a_{n-1} x^{n-2}) x + (a_0 + a_2 x^2 + \cdots + a_{n-2} x^{n-2}) \end{aligned}$$

令 $y = x^2$，则

$$\begin{aligned} p(x) &= (a_1 + a_3 y + \cdots + a_{n-1} y^{\frac{n}{2}-1}) x + (a_0 + a_2 y + \cdots + a_{n-2} y^{\frac{n}{2}-1}) \\ &= q(y)x + r(y) \end{aligned}$$

因此 $p(x)$ 在 $x = \omega^j, j = 0,1,\cdots,\frac{n}{2}-1$ 处的值可表示为

$$p(\omega^j) = q(\omega^{2j})\omega^j + r(\omega^{2j})$$

根据复数的性质 $\omega^{j+\frac{n}{2}} = -\omega^j$，有

$$p(\omega^{j+\frac{n}{2}}) = -q(\omega^{2j})\omega^j + r(\omega^{2j})$$

上述推导过程说明，求解次数为 n 的多项式 $p(x)$ 在 $x=\omega^j, j=0,1,\cdots,n-1$ 处的值，可以转化为求两个次数为 $n/2$ 的多项式 $q(x)$ 和 $r(x)$ 在 $x=(\omega^2)^j, j=0,1,\cdots,n-1$ 处的值的问题。如果 n 是 2 的幂，则我们可以利用分治算法，继续划分下去，直到 $p(x)$ 是一个常数为止。下面给出算法的伪代码：

```
RecursiveFFT(A, ω)
1    if n = 1 then return A
2    x ← ω^0
3    A2 ← {a0, a2, ..., a_{n-2}}
4    A1 ← {a1, a3, ..., a_{n-1}}
5    q ← RecursiveFFT(A2, ω^2)
6    r ← RecursiveFFT(A1, ω^2)
7    for k ← 0 to n/2 - 1 do
8        b_k ← x × r_k + q_k
9        b_{k+n/2} ← -x × r_k + q_k
10       x ← xω
11   return b
```

其中 $b=\{b_0,b_1,\cdots,b_{n-1}\}$ 即为所求。令 $T(n)$ 表示计算 b 所需要的时间，则有

$$T(n) = 2T(n/2) + O(n)$$

利用公式法可得 $T(n)=O(n\lg n)$。

离散傅里叶变换与其逆变换具有相同的形式，因此 FFT 也可以用来计算逆变换。此外，利用 FFT 还可以在 $O(n\lg n)$ 时间内计算两个多项式的乘积。值得注意的是，上述 FFT 考虑多项式在一些特殊点 $x=\omega^j, j=0,1,\cdots,n-1$ 值的计算情形。

5.8 小 结

本章内容主要参考了文献[1,5,7]，介绍了分治算法的设计以及时间复杂度的分析方法。最后介绍了分治算法的应用，并特别介绍了如何减少子问题的个数来提高分治法的效率，另一个提高分治算法效率的方法是减少划分问题以及合并子问题解的计算量。当然，这里面也有一个矛盾的关系，即减少子问题的个数，可能会增加划分问题以及合并子问题解的计算量，这就需要根据具体问题来平衡考虑。分治算法是算法领域最基本的技术之一，虽然分治算法有时求解问题的效率不是太高，但是许多有效的算法都是基于分治的思想。分治的思想，在今后的章节中，例如在第 16 章还会介绍，这里就不再介绍它的应用。

习题

5.1 将合并排序算法改写成迭代算法（非递归算法），尽可能使算法更有效。

5.2 给出 Strassen 算法计算下列矩阵乘积

$$\begin{pmatrix}3 & 1\\4 & -1\end{pmatrix}\begin{pmatrix}2 & -5\\6 & -3\end{pmatrix}$$

的过程。

5.3 给定一个有序数组 $A[1..n]$，以及一个元素 x，设计一个寻找 x 的分治算法并分析其时间复杂度。要求返回 x 在数组中的位置。

5.4 设计一个求 n 个元素最小值和最大值的迭代算法，要求仅比较 $\frac{3n}{2}-2$ 次，这里的 n 是 2 的幂。

5.5 设计一个求 n 个整数数组 $A[1..n]$ 中所有元素和的分治算法，此算法应当把输入元素近似地划分成两半入手。

5.6 给定 n 个整数的数组 $A[1..n]$，以及一个整数 x，设计一个分治算法，求出 x 在数组 A 中出现的次数，并分析所设计算法的时间复杂度。

5.7 给出一个由 n 个元素组成的数组 $A[1..n]$ 及两个元素 x_1 和 x_2，设计一个寻找两个元素位置的分治算法。

5.8 按照下述思路修改 MergeSort 算法：算法首先把输入数组 $A[p..r]$ 划分成 4 个部分 A_1、A_2、A_3 和 A_4，取代原来的两部分，然后分别对每部分递归排序，最后将 4 个已排序部分合并得到一个有序的数组。为简单起见，假定 n 是 4 的幂。请设计出修改算法并分析其时间复杂度。

5.9 给定 n 个互不相同元素的数组 $A[1..n]$，要求找出数组中第 k 个最小元素的分治算法并分析其时间复杂度。

5.10 修改算法 QuickSort 使得它能够求解选择问题（习题 5.9），并分析新算法的最坏情形以及平均情形下的时间复杂度。

5.11 将算法 QuickSort 转化为迭代算法。

5.12 考虑在一个具有 n 个互不相同元素的数组 $A[1..n]$ 中找出所有前 k 个最小元素的问题，这里 k 不是常量，即它是输入数据的一部分。我们可以用排序算法解此问题并返回 $A[1..k]$，然而耗费的时间为 $O(n\lg n)$，试设计一个时间复杂度为 $\Theta(n)$ 的算法。

5.13 设 $x=a+bi$ 和 $y=c+di$ 是两个复数。只要做 4 次乘法就很容易计算出乘积 xy，也就是 $xy=(ac-bd)+(ad+bc)i$。设计一个算法，只用 3 次乘法就可以计算出 xy。

5.14 已知序列 $A=\{a_0,a_1,a_2,a_3\}$，给出 RecursiveFFT 算法的计算过程及结果。

5.15 设计一个分治算法，使之能判断两棵二叉树 T_1 和 T_2 是否相同。

5.16 设计一个分治算法，使之能计算一棵二叉树的高度。

5.17 设计一个分治算法，在一个具有 n 个数的数组中找出第二大的元素，并分析算法的时间复杂度。

5.18 说明怎样用三连格来平铺任何没有方格缺失的 $2i\times 3j$ 平板，其中 i 和 j 都是正整数。

实验题

5.19 编程序实现残缺棋盘游戏算法，并能用图形演示。

5.20 分别将插入排序、选择排序、合并排序和快速排序编程实现，并使用实验分析方法比较各种算法的效率。

5.21 完成 XOJ 如下题目：1004，1007，1017，1018，1022，1057。

5.22 完成 POJ 如下题目：1002，1947，2082，2282，2299，2318，2379，2418，2726，3122。

第6章 动态规划

6.1 算法思想

动态规划(Dynamic Programming,DP)算法采用分治算法的思想,将原问题分成若干个子问题,然后分别求解各个子问题,最后将子问题的解组合起来得到原问题的解。分治算法递归地求解各个子问题,可能重复求解某些子问题。与分治算法不同的是,动态规划算法不是递归地求解各个子问题,它是从简单问题的解入手,逐步求解,直至求解出原问题。动态规划算法的高明之处在于,它不会重复求解某些重复出现的子问题,即重叠子问题。为了说明这点,考虑 Fibonacci 序列 $F(n)$

$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

我们可以很容易地写出递归算法,其伪代码如下:

```
F(n)
1  if n=0 or n=1 then return 1
2  else return F(n-1)+F(n-2)
```

$F(n)$算法具有指数级时间复杂度,详细分析可参考习题 6.1 的解答。下面给出计算 $F(7)$的过程,如图 6.1 所示。

从图 6.1 可以看出,计算 $F(2)$重复计算了 8 次,见灰色框,计算 $F(3)$重复计算了 5 次,见黑色框。如果我们能够保存计算过的 $F(i)$,以后碰到它时,就不用再计算它,而是直接从保存的数据中得到相应的值,显然可以提高计算的效率。下面我们给出实现这种思想的伪代码:

```
F1(n)
1  if v[n]<0 then
2     v[n]←F1(n-1)+F1(n-2)
3  return v[n]
```

其中数组 $v[n]$表示计算过的值,$v[i]$对应 $F1(i)$,先初始化数组 v 的值为 -1,如果 $v[i]<0$,表示相应的 $F1(i)$没有被计算。

$F1(n)$算法虽然避免了大量的重复计算,但是依然有很多次函数调用,每一次调用都要花费很多时间用于参数的传递和动态链接等。事实上,计算 $F(i)$,我们仅需要知道 $F(i-1)$和 $F(i-2)$的值。如果我们从简单问题入手,就是由递归出口

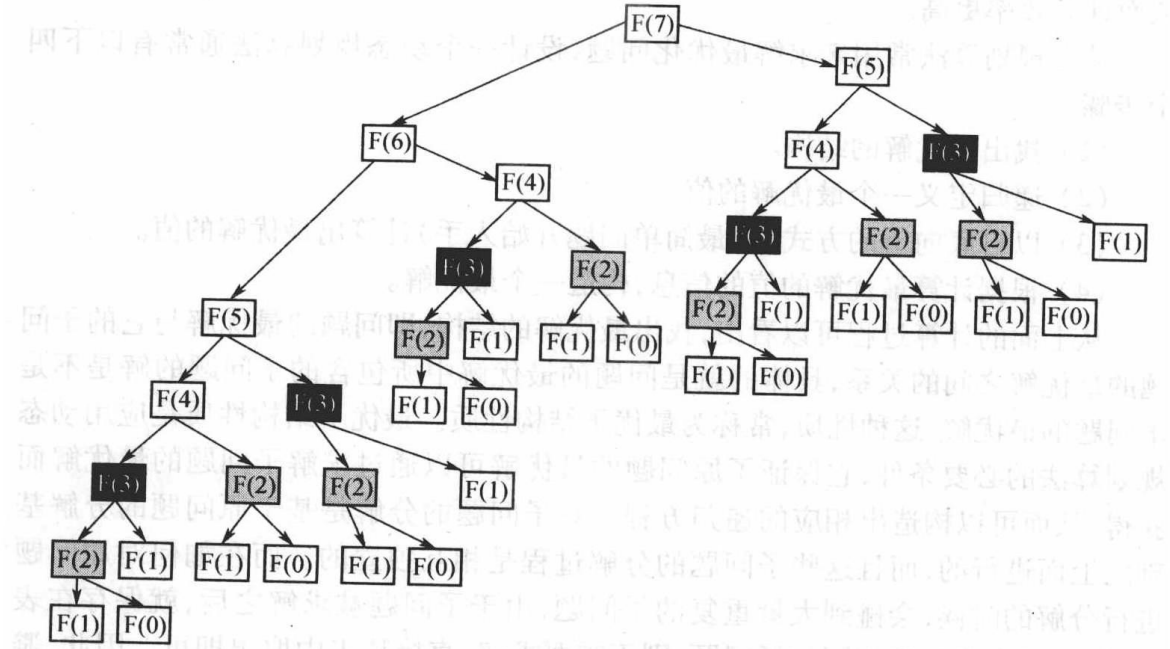


图 6.1

$F(0)=F(1)=1$ 开始，先计算 $F(2)$，然后计算 $F(3)$，$F(4)\cdots$，便可以得到下列迭代算法：

```
F2(n)
1  F[0]←1
2  F[1]←1
3  for i←2 to n do
4      F[i]←F[i-1]+F[i-2]
5  return F[n]
```

这个算法就是最简单的动态规划算法，其时间复杂度仅为 $O(n)$，比指数级时间复杂度的递归算法 $F(n)$更有效。从 $F2(n)$算法我们可以看出动态规划算法的计算步骤：

（1）构造出原问题和它的子问题之间的递归方程，例如 $F(n)=F(n-1)+F(n-2)$。

（2）将求出的子问题的值保存在一个表（一般用数组）中，以方便数据的保存和读取。

（3）以递归出口为计算的起点，以自底向上的方式将数组填满，这样就保证了当我们在解决一个问题时，所有比该问题更小的子问题的解已经保存在数组中，这样就可以直接利用已经求解过的子问题的值，来求出问题的解。

从上面的例子可以看出，在分治算法的计算过程中，子问题被独立地重复计算。而动态规划算法中，子问题的计算不是独立的，而且子问题仅仅被计算一次，

从而计算效率更高。

动态规划算法常用来求解最优化问题，设计一个动态规划算法通常有以下四个步骤：

（1）找出最优解的结构。

（2）递归定义一个最优解的值。

（3）以自底向上的方式（从最简单问题开始入手）计算出最优解的值。

（4）根据计算最优解的值的信息，构造一个最优解。

从上面的计算过程可以看出，找出最优解的结构，即问题的最优解与它的子问题的最优解之间的关系，具体的就是问题的最优解中所包含的子问题的解是不是子问题的最优解，这种性质，常称为最优子结构性质。最优子结构性质是应用动态规划算法的必要条件，它保证了原问题的最优解可以通过求解子问题的最优解而获得，从而可以构造出相应的递归方程。对子问题的分解是基于原问题的分解基础之上而进行的，而且这些子问题的分解过程是相互独立的。而在如何对原问题进行分解的时候，会碰到大量重复的子问题，由于子问题被求解之后，就保存在表中，如果下次再碰到相同的子问题，则不需要求解，直接从表中取出即可。因此，避免大量重叠子问题的重复计算，是提高动态规划算法效率的关键。上面的例子不是一个最优化问题，体现不出最优子结构性质，下面给出的实例将会重点说明。

6.2 装配线调度问题

有两条装配线，每一条装配线上有 n 个装配点，将装配线 i 上的第 j 个装配点记为 $S_i[j]$，设在装配点 $S_i[j]$ 的装配时间为 $a_i[j]$。假设要装配一辆汽车，将汽车底盘从进厂点送入装配线 $i(i=1$ 或 $2)$，需要时间 e_i。在装配点 $S_i[j]$ 装配后，如果汽车传送到同一条装配线的装配点 $S_i[j+1]$ 进行装配，则传送不需要时间。如果汽车从装配点 $S_i[j]$ 传送到另一条装配线进行装配，则需要传送时间 $t_i[j]$。汽车在装配点 $S_i[n]$ 装配后，将汽车成品从装配线上取下来，需要花费时间 x_i。装配线调度问题是如何确定每一个装配点的装配需要在哪条线上进行，使得当汽车成品出来时，花费的总时间最少。值得注意的是，两条装配线的第 j 个装配点都装配同样的汽车部件，只是装配效率不一样。

图 6.2 给出了装配的线路图，汽车可以沿着箭头所指的路线进行加工一直到成品出厂。装配线调度问题就是如何选择装配路线，使得汽车底盘从进厂点开始到出厂所需要的时间最短。

对这个问题，我们首先想到的是枚举法，列出每一条装配路线，然后计算出每一条路线花费的时间，最后选出花费时间最少的装配路线。由于有 n 个装配点，从每个装配点出发，有两种选择，因此有 2^n 条装配路线，当 n 很大时，枚举法就不实用了。

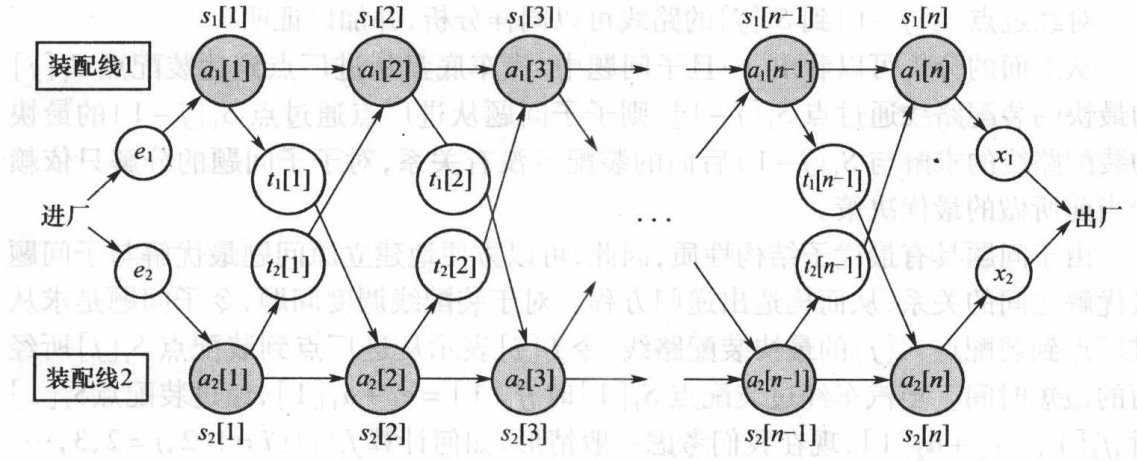


图 6.2

下面我们分析一下问题的最优子结构性质。考虑子问题，汽车底盘从进厂点到装配点 $S_1[j]$ 的最短装配时间问题。假定最快的装配路线通过点 $S_1[j]$，则有如下几种情形：

（1）如果 $j=1$，需要计算到达点 $S_1[1]$ 需要的时间。

（2）如果 $j>1$，到达点 $S_1[j]$ 有两种走法：

① 从前一个装配点 $S_1[j-1]$ 直接传送到点 $S_1[j]$ 装配；

② 从前一个装配点 $S_2[j-1]$ 装配后，传送到装配线 1 的装配点 $S_1[j]$ 装配。

对于情形(2)，我们有如下结论。

定理 6.1 假定过点 $S_1[j]$ 的最快装配路线是从进厂点经过路线 p 再过点 $S_1[j-1]$ 到达 $S_1[j]$，则从进厂点到点 $S_1[j-1]$ 的装配路线 p 也一定是从进厂点到点 $S_1[j-1]$ 的最快装配路线。

证明：可借助图 6.3，用反证法证明上述最优子结构性质。假定 p 不是最快的，则存在从进厂点到点 $S_1[j-1]$ 的最快装配路线 p'，将 p' 替换 p，则可以得到一条比最快装配路线（从进厂点经过 p 再过点 $S_1[j-1]$ 到达 $S_1[j]$）还要快的装配路线，这就产生了矛盾。故假设不成立，所证成立。

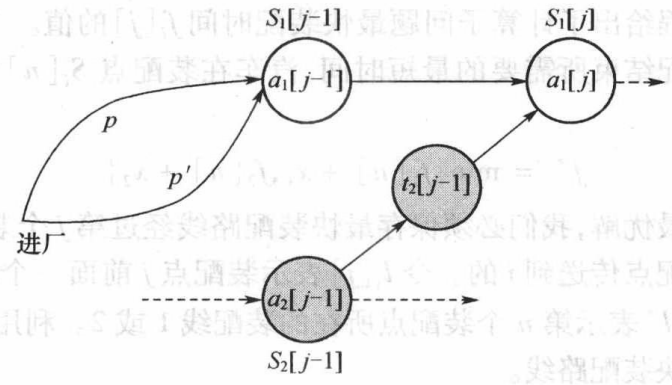


图 6.3

对经过点 $S_2[j-1]$ 到 $S_1[j]$ 的路线可以同样分析，并加以证明。

从上面的分析可以看出，一旦子问题中，汽车底盘从进厂点到达装配点 $S_1[j]$ 的最快的装配路线通过点 $S_1[j-1]$，则子子问题从进厂点通过点 $S_1[j-1]$ 的最快的装配路线的求解与 $S_1[j-1]$ 后面的装配点没有关系，对子子问题的分解只依赖于当前所做的最优决策。

由于问题具有最优子结构性质，因此，可以方便地建立原问题最优解与子问题最优解之间的关系，从而构造出递归方程。对于装配线调度问题，令子问题是求从进厂点到装配点 $S_i[j]$ 的最快装配路线，令 $f_i[j]$ 表示从进厂点到装配点 $S_i[j]$ 所经历的最短时间。当汽车经过装配点 $S_1[1]$ 时 $f_1[1]=e_1+a_1[1]$，经过装配点 $S_2[1]$ 时 $f_2[1]=e_2+a_2[1]$，现在我们考虑一般情形，如何计算 $f_i[j]$（$i=1,2, j=2,3,\cdots,n$），假定到点 $S_1[j]$ 的最快装配路线：

① 如果是从前一个装配点 $S_1[j-1]$ 直接到点 $S_1[j]$，则有 $f_1[j]=f_1[j-1]+a_1[j]$；

② 如果是从前一个装配点 $S_2[j-1]$ 装配后，传送到装配线 1 的装配点 $S_1[j]$ 装配，则有 $f_1[j]=f_2[j-1]+t_2[j-1]+a_1[j]$。

要计算经过点 $S_1[j]$ 的最短时间 $f_1[j]$，只需从上述两种情况中，选最优的装配点，即做最优的决策

$$f_1[j] = \min\{f_1[j-1]+a_1[j],\ f_2[j-1]+t_2[j-1]+a_1[j]\}$$

类似地，要计算过点 $S_2[j]$ 的最短时间 $f_2[j]$，只需计算

$$f_2[j] = \min\{f_2[j-1]+a_2[j],\ f_1[j-1]+t_1[j-1]+a_2[j]\}$$

总结上述各种情况，我们得到下列递归方程

$$f_1[j] = \begin{cases} a_1[1]+e_1 & \text{if } j=1 \\ \min\{f_1[j-1]+a_1[j], f_2[j-1]+t_2[j-1]+a_1[j]\} & \text{if } j>1 \end{cases}$$

$$f_2[j] = \begin{cases} a_2[1]+e_2 & \text{if } j=1 \\ \min\{f_2[j-1]+a_2[j], f_1[j-1]+t_1[j-1]+a_2[j]\} & \text{if } j>1 \end{cases}$$

上述递归方程给出了计算子问题最快装配时间 $f_i[j]$ 的值。令 f^* 表示汽车底盘从进厂点到装配结束所需要的最短时间，汽车在装配点 $S_i[n]$ 装配好出厂有两条路线，因此

$$f^* = \min\{f_1[n]+x_1, f_2[n]+x_2\}$$

为了构造出最优解，我们必须保存最快装配路线经过第 j 个装配点时，是从前面哪条线上的装配点传送到 j 的。令 $l_i[j]$ 表示装配点 j 前面一个装配点所在的装配线 1 或 2，并令 l^* 表示第 n 个装配点所在的装配线 1 或 2。利用 $l_i[j]$ 和 l^* 的值，就可以构造出最快装配路线。

根据递归方程，我们可以很容易地写出递归算法，但是这个递归算法的时间复

杂度为 $O(2^n)$，我们是否能够做得更好呢？从递归方程可以知道，对于 $j>1$，每一个 $f_i[j]$ 的值仅仅与 $f_1[j-1]$ 和 $f_2[j-1]$ 的值有关，而且，计算 $f_1[j]$ 和 $f_2[j]$ 都需要计算 $f_1[j-1]$ 和 $f_2[j-1]$，出现许多重叠子问题。因此我们可以采用自底向上的方式进行求解，从而避免大量的重复计算，动态规划算法的伪代码如下：

```
DPFastestWay(a,t,e,x,n)
1  f1[1]←e1 + a1[1];  f2[1]←e2 + a2[1]
2  for  j←2 to  n  do
3        if  f1[j-1] + a1[j] ≤ f2[j-1] + t2[j-1] + a1[j] then
4              f1[j]←f1[j-1] + a1[j]
5              l1[j]←1
6        else  f1[j]←f2[j-1] + t2[j-1] + a1[j]
7              l1[j]←2
8        if  f2[j-1] + a2[j] ≤ f1[j-1] + t1[j-1] + a2[j] then
9              f2[j]←f2[j-1] + a2[j]
10             l2[j]←2
11       else  f2[j]←f1[j-1] + t1[j-1] + a2[j] then
12             l2[j]←1
13 if  f1[n] + x1 ≤ f2[n] + x2 then
14       f*←f1[n] + x1
15       l*←1
16 else  f*←f2[n] + x2
17       l*←2
```

因为算法从第 2 行至 12 行，要执行 $n-1$ 次循环，因此 DPFastestWay(a,t,e,x,n)算法的时间复杂度为 $O(n)$，比指数级的递归算法有效多了。

给定一个装配线调度问题的一个具体例子如图 6.4 所示。图 6.5(a)给出了从进厂点到每个装配点的最短装配时间。图 6.5(b)给出了当前装配点装配时是从哪条装配线传送过来的。

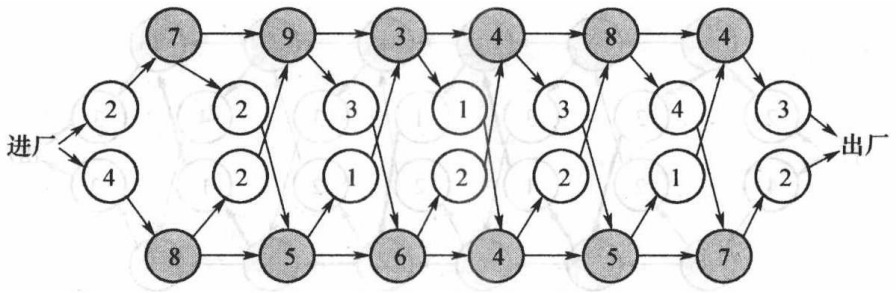


图 6.4

由 DPFastestWay(a,t,e,x,n)算法，可以计算出最快装配时间，以及一些额外的信息值。但是，我们想知道最快装配的路线，即具体在哪条装配线装配的，还必须根据有关信息构造出最快装配路线。

j	1	2	3	4	5	6
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37

$f^*=38$

(a)

j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	1	2

$l^*=1$

(b)

图 6.5

为了构造最快装配路线，我们必须知道每一装配点所在的装配线，这可以根据 $l_i[j]$ 和 l^* 得到下列构造最优装配路线的算法：

```
PrintStations(l, n)
1  i←l*
2  print "line" i ", station" n
3  for j←n downto 2 do
4          i←l_i[j]
5          print "line" i ", station" j − 1
```

我们可以利用图 6.5(b)中记录的 $l_i[j]$ 和 l^* 的值来构造最快装配路线。$l^*=1$ 表示是从装配线 1 的 $S_1[6]$ 出厂的，因此接下来我们找 $l_1[6]=2$，这表示前一个装配点是装配线 2 上的 $S_2[5]$，因此接下来找 $l_2[5]=2$，这表示前一个装配点是装配线 2 上的 $S_2[4]$。类似地，我们找 $l_2[4]=1$（装配点 $S_1[3]$），$l_1[3]=2$（装配点 $S_2[2]$），$l_2[2]=1$（装配点 $S_1[1]$），如图 6.6 所示。

j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2

$l^*=1$

图 6.6

图 6.7 显示了由图 6.6 构造出的最快装配路线：从进厂点沿着粗黑箭头方向一直到出厂。

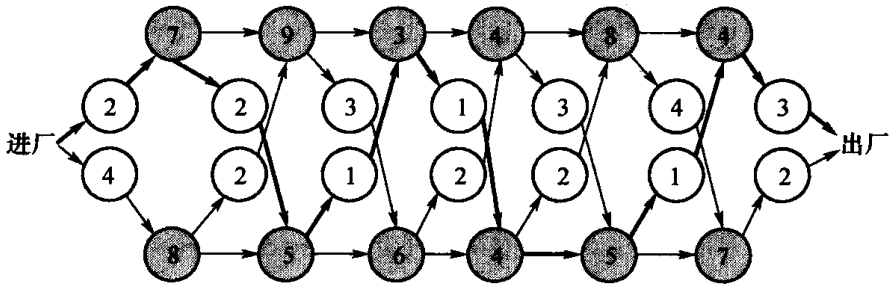


图 6.7

对于图 6.4 所示例子，PrintStations(l,n)程序打印出下列装配路线：

line 1，station 6

line 2, station 5
line 2, station 4
line 1, station 3
line 2, station 2
line 1, station 1

6.3 矩阵链乘法问题

给定一个有 n 个矩阵的矩阵链(Matrix Chain)$A_1, A_2, \cdots, A_i, \cdots, A_n$,矩阵 A_i($1 \leqslant i \leqslant n$)的维数为 $p_{i-1} \times p_i$,矩阵链乘法问题就是如何对矩阵乘积 $A_1A_2\cdots A_i\cdots A_n$ 加括号,使得它们的标量乘法次数达到最少。矩阵乘积可以加括号,指的是矩阵的乘积是可以完全括号化的,即要么是单一的矩阵,要么是两个完全括号化的矩阵相乘之后再加上括号而形成的。

例如,若矩阵链为 A_1, A_2, A_3, A_4,则 $A_1A_2A_3A_4$ 的乘积可以有以下五种不同的完全括号化的形式:$(A_1(A_2(A_3A_4)))$,$(A_1((A_2A_3)A_4))$,$((A_1A_2)(A_3A_4))$,$((A_1(A_2A_3))A_4)$,$(((A_1A_2)A_3)A_4)$。

对矩阵链乘积加括号的方式对计算量有很大的影响。由 5.5 节可知,只有当矩阵 A 的行数等于矩阵 B 的列数时,两个矩阵 A, B 能够相乘。现在假设 A 的维数为 $p \times q$,B 的维数为 $q \times r$,则计算出 $C = AB$ 的标量乘法次数为 pqr。现在,我们看看加括号的方式对矩阵链乘积计算量的影响。

考虑有三个矩阵的矩阵链 A_1, A_2, A_3 的乘积问题。假设它们的维数分别是 10×100,100×5 和 5×50,则

计算$((A_1A_2)A_3)$:

$A_1A_2 = 10 \times 100 \times 5 = 5000$,$A_1A_2$ 的维数为 10×5,因此

$$((A_1A_2)A_3) = 10 \times 5 \times 50 = 2500$$

总计 7500 次标量乘法运算。

计算$(A_1(A_2A_3))$:

$(A_2A_3) = 100 \times 5 \times 50 = 25000$,$A_2A_3$ 的维数为 100×50,因此

$$(A_1(A_2A_3)) = 10 \times 100 \times 50 = 50000$$

总计 75000 次标量乘法运算。

计算$(A_1(A_2A_3))$的乘法次数是计算$((A_1A_2)A_3)$的 10 倍,即一个数量级的差别。由此可见,加括号的方式对矩阵链乘积计算量的影响巨大。

给定一个矩阵链,我们想知道有多少种加括号的方式。令 $P(n)$ 表示有 n 个矩阵相乘的矩阵链加括号的方式数。一种很自然的加括号方式是采用递归的思想,因为对任意的 $k = 1, 2, \cdots, n-1$,我们可以将该矩阵链在第 k 个矩阵到第 $k+1$ 个矩

阵之间分开，然后对两个子矩阵链分别按照上述类似的方式加括号，我们得到下述递归方程

$$P(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geqslant 2 \end{cases} \tag{6.1}$$

上述递归方程的解为 $O(2^n)$，见习题 6.6，因此用枚举法来找到最优的加括号方式显然是不可取的。现在我们看看是否可以用动态规划算法来求解。

动态规划算法的第一步是找到最优子结构性质，以便用它来构造原问题的最优解与子问题最优解之间的关系。为了方便起见，我们首先要表示出子问题，令子问题表示如何对矩阵链 $A_i, A_{i+1}, \cdots, A_j$ 的乘积加括号，使得乘法次数最少。简记子问题为

$$A_{i..j} = A_i A_{i+1} \cdots A_j, i \leqslant j$$

假设 $A_{i..j}$ 的一种最优完全括号化方式是把乘积在 A_k 和 A_{k+1} $(i \leqslant k < j)$ 之间分开，则

$$\begin{aligned} A_{i..j} &= A_i A_{i+1} \cdots A_j \\ &= A_i A_{i+1} \cdots A_k A_{k+1} \cdots A_j \\ &= A_{i..k} A_{k+1..j} \end{aligned}$$

子问题 $A_{i..j}$ 分解成两个子问题 $A_{i..k}$ 和 $A_{k+1..j}$，我们有如下结论。

定理 6.2 假设 $A_{i..j}$ 的一种最优完全括号化方式是把乘积在 A_k 和 A_{k+1} $(i \leqslant k < j)$ 之间分开，则其中子问题 $A_{i..k}$ 和 $A_{k+1..j}$ 的加括号的方式也一定是最优的。

证明：我们可以用反证法来证明这个结论。假设子问题 $A_{i..k}$ 和 $A_{k+1..j}$ 的加括号方式不是最优的，则说明对子问题 $A_{i..k}$ 和 $A_{k+1..j}$ 分别有一种更优的加括号方式，它分别使得计算 $A_{i..k}$ 和 $A_{k+1..j}$ 的乘法次数更少，将该种加括号方式分别替换原来对子问题 $A_{i..k}$ 和 $A_{k+1..j}$ 的加括号方式，则子问题 $A_{i..j}$ 可得到一种新的加括号方式，它比原来最优加括号方式的乘法次数更少，显然矛盾。

从定理 6.2 可以看出，一旦子问题做出最优决策，在 k 处进行划分，则子子问题 $A_{i..k}$ 和 $A_{k+1..j}$ 的求解互相独立，且子问题加括号的方式也分别是最优的，对子子问题的分解只依赖于当前所做的最优决策。

现在我们用最优子结构性质来建立问题最优解与子问题最优解之间的关系，从而得到递归方程。

对于矩阵链乘积问题，我们的目的是找到一种最优的加括号方式使得总的乘法次数最小。如果我们能确定子问题 $A_{i..j}$ 最优加括号方式所需要的最　法次数，则原问题 $A_{1..n}$ 所需要的最小乘法次数也可确定。令 $m[i,j]$ 表示计算　问题 $A_{i..j}$ 需要的最少乘法次数，我们可以递归地定义如下。如果 $i=j, A_{i..i}=A_i$，问题很简单，根本不涉及乘法，因此 $m[i,j]=0(i=1,2,\cdots,n)$。如果 $i<j$，我们可以利用最优子结构性质，假设子问题 $A_{i..j}$ 最优加括号方式是把乘积在 A_k 和 A_{k+1} 之间分

开,即

$$A_{i..j} = A_{i..k}A_{k+1..j}(i \leqslant k < j)$$

则子问题$A_{i..j}$的最小乘法次数就等于两个子问题$A_{i..k}$和$A_{k+1..j}$的最少乘法次数之和再加上两个矩阵$A_{i..k}$和$A_{k+1..j}$相乘的乘法次数。由于矩阵$A_{i..k}$的维数是$p_{i-1} \times p_k$,矩阵$A_{k+1..j}$的维数是$p_k \times p_j$,因此矩阵$A_{i..k}$和$A_{k+1..j}$相乘的乘法次数为$p_{i-1}p_kp_j$。由上面分析可得

$$m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_k p_j$$

上述递归方程是假定子问题$A_{i..j}$最优加括号方式是在A_k和A_{k+1}之间划分而得到的。事实上,这种划分方式,我们事先并不知道,划分的方式有$j-i$种,即$k = i,i+1,\cdots,j-1$,因此,我们必须在$j-i$种划分中,选择使$m[i,j]$值最小的划分,因此,我们得到如下递归方程

$$m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \leqslant k < j}\{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

上述递归方程给出了子问题$A_{i..j}$的最小乘法次数的计算办法。为了构造最优解,即具体是如何加括号的,我们必须保存使子问题$A_{i..j}$的乘法次数最小的划分点,即最优的k值。令$s[i,j]$表示把乘积$A_iA_{i+1}\cdots A_j$划分以便得到最小乘法次数的划分点k的值。这样,我们根据$s[i,j]$的值就可以得到乘积$A_iA_{i+1}\cdots A_j$在何处加括号。

根据递归方程,我们很容易写出递归算法,然而这个递归算法需要指数级时间复杂度,它并不比枚举算法快多少。事实上,计算$m[i,j]$,需要计算两个规模更小的子问题的$m[i,k]$和$m[k+1,j]$,而且随着k值的不同,会碰到大量的重复计算。因此,我们采用动态规划算法,自底向上计算$m[i,j]$的值,详细的伪代码如下:

```
DPMatrixChain(p)
1    for i←1 to n do
2        m[i,i]←0
3    for c←2 to n do
4        for i←1 to n-c+1 do
5            j←i +c-1
6            m[i,j]←∞
7            for k←i to j-1 do
8                q←m[i,k] +m[k+1,j] +p_{i-1}p_kp_j
9                if q<m[i,j] then
10                   m[i,j]←q; s[i,j]←k
11   return m and s
```

其中,c是矩阵链的长度。DPMatrixChain(p)算法的输入是一个向量$p = \langle p_0, p_1,\cdots,p_n\rangle$,它表示矩阵链乘积中矩阵的维数。二维数组$m[1..n,1..n]$用来保存

已经计算过的 $m[i,j]$。二维数组 $s[1..n,1..n]$ 用来保存最优划分点 k 的值。填充 m 数组的方式是从左下角沿着箭头方向到右上角进行填充，一直到左上角位置 $m[1,n]$ 即为所求，如图 6.8(a)所示。除了从左下角沿着箭头方向到右上角的主对角线不填充外，s 左上角的值可类似填充，如图 6.8(b)所示。值得注意的是，两个数组右下角都不需要填充，如图 6.8(a)阴影部分所示。

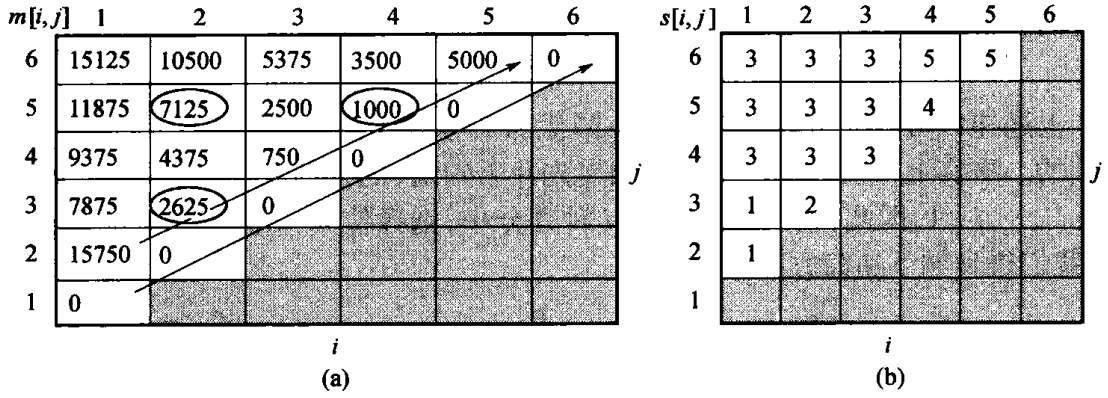


图 6.8

DPMatrixChain(p)算法从第 3 行直到第 11 行共有三重循环，因此算法的时间复杂度为 $O(n^3)$，比递归算法有效多了。

下面给出一个具体例子，看看 DPMatrixChain(p)算法的运算过程。

给定一个有 6 个矩阵的矩阵链 A_1,A_2,A_3,A_4,A_5,A_6，矩阵的维数分别为 $30\times35,35\times15,15\times5,5\times10,10\times20,20\times25$，则

$$m[2,5]=\min\begin{cases} m[2,2]+m[3,5]+p_1p_2p_5 = 0+2500+35\times15\times20 = 13000 \\ m[2,3]+m[4,5]+p_1p_3p_5 = 2625+1000+35\times5\times20 = 7125 \\ m[2,4]+m[5,5]+p_1p_4p_5 = 4375+0+35\times10\times20 = 11375 \end{cases}$$

$$= 7125$$

由上式可知，$m[2,5]$ 可由其右边的值 $m[4,5]$ 和其下边的值 $m[2,3]$ 决定，见图 6.8(a)带圈的值。按照表 m 的填充方式，它们都已经被计算，因此直接引用它们的值即可，从而避免了重复计算。类似地，其他 $m[i,j]$ 的值如图 6.8(a)所示。其中，j 从 6 到 1，是为了构造出上三角表格，后面都采取这种表示，就不再说明。

虽然 DPMatrixChain(p)算法能够计算出矩阵链乘积所需要的最小乘法次数，但是并没有告诉我们是如何加括号的。由于 $s[i,j]$ 记录了乘积 $A_iA_{i+1}\cdots A_j$ 的最优划分位置 k。这样，$A_iA_{i+1}\cdots A_j$ 就能够加括号为 $(A_iA_{i+1}\cdots A_k)(A_{k+1}\cdots A_j)$，对子问题我们可以根据 $s[i,k]$ 和 $s[k+1,j]$ 的值递归加括号，详细的加括号算法如下：

```
PrintParens(s,i,j)
1  if i=j then print "Ai"
```

```
2  else  print "("
3        PrintParens(s,i,s[i,j])
4        PrintParens(s,s[i,j]+1,j)
5        print ")"
```

根据图6.8(b)记录的$s[i,j]$的值,调用算法PrintParens(s,i,j),可以构造矩阵链A_1,A_2,A_3,A_4,A_5,A_6的最优加括号的方式如下:

$$(\Rightarrow((\Rightarrow((A_1\Rightarrow((A_1(\Rightarrow((A_1(A_2\Rightarrow((A_1(A_2A_3\Rightarrow((A_1(A_2A_3)\Rightarrow$$
$$((A_1(A_2A_3))\Rightarrow((A_1(A_2A_3))(\Rightarrow((A_1(A_2A_3))(\Rightarrow((A_1(A_2A_3))((\Rightarrow$$
$$((A_1(A_2A_3))((A_4\Rightarrow((A_1(A_2A_3))((A_4A_5\Rightarrow((A_1(A_2A_3))((A_4A_5)\Rightarrow$$
$$((A_1(A_2A_3))((A_4A_5)A_6\Rightarrow((A_1(A_2A_3))((A_4A_5)A_6)\Rightarrow$$
$$((A_1(A_2A_3))((A_4A_5)A_6))$$

因此$A_1A_2A_3A_4A_5A_6=((A_1(A_2A_3))((A_4A_5)A_6))$具有最小的乘法次数。

6.4 最长公共子序列问题

在生物工程应用中,我们经常要比较两个DNA序列的相似性,在软件工程领域也经常比较两个软件版本,以便确定版本的变化,所有这些相似性比较问题,都涉及最长公共子序列问题(Longest Common Subsequence,LCS)。为设计该问题的求解算法,我们先给出一些概念。

一个给定序列的子序列是在该序列中删去若干元素后得到的序列,形式的定义如下:

给定一个序列$X=\langle x_1,x_2,\cdots,x_m\rangle$,另一个序列$Z=\langle z_1,z_2,\cdots,z_k\rangle$是$X$的子序列,如果存在一个严格递增的$X$中元素下标的序列$\langle i_1,i_2,\cdots,i_k\rangle$,使得$x_{i_j}=z_j$ $(1\leqslant j\leqslant k)$。

给定两个序列X和Y,当另一序列Z既是X的子序列又是Y的子序列时,称Z是序列X和Y的公共子序列。最长公共子序列就是公共子序列中长度最长的子序列。

例如$X=\langle A,B,C,B,D,A,B\rangle$,$X$的子序列就是这个序列中的元素按顺序组成的一个子集,例如$\langle A,B,D\rangle$,$\langle B,C,D,B\rangle$等。另给定一个序列$Y=\langle B,D,C,A,B,A\rangle$,则$\langle B,C,B,A\rangle$,$\langle B,D,A,B\rangle$都是$X$和$Y$的长度为4的最长公共子序列,$\langle B,C,A\rangle$就不是$X$和$Y$的最长公共子序列。

给定两个序列$X=\langle x_1,x_2,\cdots,x_m\rangle$和$Y=\langle y_1,y_2,\cdots,y_n\rangle$,最长公共子序列问题就是找出$X$和$Y$的一个最长公共子序列。

对于最长公共子序列问题,枚举算法是对X的每个子序列,验证是否也是Y的一个子序列,X共有2^m个子序列,要验证每个子序列是否是Y的一个子序列,要花$\Theta(n)$的时间:从Y的第一个字母开始扫描,接着从第二个字母开始扫描,如此继续

下去，因此枚举算法的时间复杂度为 $\Theta(n2^m)$。利用枚举算法来求解，显然花费太大，下面给出动态规划求解算法。

我们先看最长公共子序列问题是否具有最优子结构性质。为方便起见，我们定义一些记号。给定一个序列 $X=\langle x_1,x_2,\cdots,x_m\rangle$，我们定义 X_i 为 X 的前 i 个元素，即

$$X_i=\langle x_1,x_2,\cdots,x_i\rangle(0\leqslant i\leqslant m)$$

例如，$X=\langle A,B,C,B,D,A,B\rangle$，$X_5=\langle A,B,C,B,D\rangle$。

有了上述定义，我们可以定义子问题：给定两个子序列 $X_i=\langle x_1,x_2,\cdots,x_i\rangle$ 和 $Y_j=\langle y_1,y_2,\cdots,y_j\rangle$，寻找它们的最长公共子序列。对于最长公共子序列问题，我们有如下结论。

定理 6.3 给定两个序列 $X_m=\langle x_1,x_2,\cdots,x_m\rangle$ 和 $Y_n=\langle y_1,y_2,\cdots,y_n\rangle$，并令 $Z_k=\langle z_1,z_2,\cdots,z_k\rangle$ 为 X_m 和 Y_n 的某个最长公共子序列，则

(1) 若 $x_m=y_n$，那么 $z_k=x_m=y_n$ 且 Z_{k-1} 就是 X_{m-1} 和 Y_{n-1} 的一个最长公共子序列；

(2) 若 $x_m\neq y_n$，且 $z_k\neq x_m$，那么 Z_k 就是 X_{m-1} 和 Y_n 的一个最长公共子序列；

(3) 若 $x_m\neq y_n$，且 $z_k\neq y_n$，那么 Z_k 就是 X_m 和 Y_{n-1} 的一个最长公共子序列。

证明：(1) 假设 $z_k\neq x_m$，则表示 x_m 不在 Z_k 里，将 x_m 附在 Z_k 后面，则得到一个比 Z_k 还长的公共子序列，这与 Z_k 已经是最长公共子序列矛盾，因此有 $z_k=x_m=y_n$。现在我们证明 Z_{k-1} 就是 X_{m-1} 和 Y_{n-1} 的一个最长公共子序列。假设 Z_{k-1} 不是 X_{m-1} 和 Y_{n-1} 的一个最长公共子序列，这表明 X_{m-1} 和 Y_{n-1} 存在一个比 Z_{k-1} 还长的公共子序列 Z，不妨设其长度为 k，将 x_m 附在 Z 后面，则得到一个比 Z_k 还长的公共子序列，这与 Z_k 已经是最长公共子序列矛盾。故所证成立。

(2) 若 $x_m\neq y_n$，且 $z_k\neq x_m$，假设 Z_k 不是 X_{m-1} 和 Y_n 的一个最长公共子序列，则 X_{m-1} 和 Y_n 存在一个比 Z_k 还长的公共子序列 Z。同样，Z 为 X_m 和 Y_n 的公共子序列，它比 Z_k 的长度还长，这与 Z_k 已经为 X_m 和 Y_n 的最长公共子序列矛盾。故所证成立。

(3) 可类似(2)证明。

定理 6.3 表明最长公共子序列问题具有最优子结构性质，因此我们可以很方便地构造出递归方程。考虑子问题找 $X_i=\langle x_1,x_2,\cdots,x_i\rangle$ 和 $Y_j=\langle y_1,y_2,\cdots,y_j\rangle$ 的最长公共子序列，令 $c[i,j]$ 表示 X_i 和 Y_j 的最长公共子序列的长度。

如果 $x_i=y_j$，并且能找到 X_{i-1} 和 Y_{j-1} 的最长公共子序列，把 x_i 添加到 X_{i-1} 和 Y_{j-1} 的最长公共子序列里，问题的最优解包含其子问题的最优解，因此有

$$c[i,j]=c[i-1,j-1]+1$$

如果 $x_i\neq y_j$，必须解决两个子问题：

(1) 找出 X_{i-1} 和 Y_j 的一个最长公共子序列；

(2) 找出 X_i 和 Y_{j-1} 的一个最长公共子序列。

然后比较，哪个子问题求得的公共子序列更长，因此我们有

$$c[i,j] = \max\{c[i-1,j], c[i,j-1]\}$$

当然，我们还必须考虑递归出口，即 $i=0$ 或 $j=0$ 的情况，此时有 $c[i,j]=0$。

综上所述，我们得到如下递归方程

$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1,j-1]+1 & \text{if } i,j > 0 \text{ and } x_i = y_j \\ \max\{c[i-1,j], c[i,j-1]\} & \text{if } i,j > 0 \text{ and } x_i \neq y_j \end{cases}$$

根据上述递归方程，我们可以很容易求得最长公共子序列的长度。现在，我们来看看重叠子问题性质，要找 X_i 和 Y_j 的一个最长公共子序列，我们需要找出 X_{i-1} 和 Y_j 以及 X_i 和 Y_{j-1} 的最长公共子序列，上述子问题的求解都包含找 X_{i-1} 和 Y_{j-1} 的最长公共子序列的问题，子问题又共享“子子问题”，因此，最长公共子序列问题具有重叠子问题性质是显然的。

根据递归方程，我们很容易写出递归算法，但是其时间复杂度太高。由于仅有 $\Theta(mn)$ 个子问题，因此，我们可以采用动态规划算法自底向上的方式求出子问题的最优解，其伪代码如下：

```
DPLCSLength(X,Y,m,n)
1   for i←1 to m do   c[i,0]←0
2   for j←0 to n do   c[0,j]←0
3   for i←1 to m do
4         for j←1 to n do
5               if x_i = y_j then
6                     c[i,j]←c[i-1,j-1]+1
7                     b[i,j]←"↖"
8               else if c[i-1,j] ≥ c[i,j-1] then
9                     c[i,j]←c[i-1,j]
10                    b[i,j]←"↑"
11              else      c[i,j]←c[i,j-1]
12                        b[i,j]←"←"
13  return c and b
```

DPLCSLength(X,Y,m,n)算法的初始输入为两个序列 X 和 Y，数组 $c[0..m, 0..n]$ 用来记录最长公共子序列的长度。为了构造出最长公共子序列，我们还必须引进一个数组 $b[1..m,1..n]$ 以便表示子问题的解 $c[i,j]$ 由哪个子问题的解（$c[i-1,j-1]$ 或者 $c[i-1,j]$ 或者 $c[i,j-1]$）决定。DPLCSLength(X,Y,m,n)算法只有两重循环，因此其时间复杂度为 $O(mn)$。填表的顺序如图 6.9 所示，沿着箭头先填充最上面一行，然后填充接下来的行。

下面我们给出一个具体的例子来演示 DPLCSLength(X,Y,m,n)算法的执行过程。

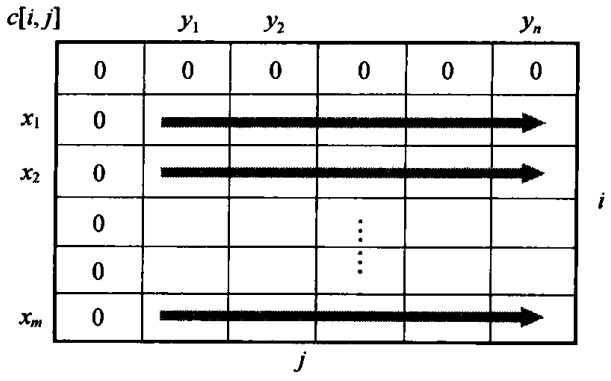


图 6.9

例如求 $X=\langle A,B,C,B,D,A,B\rangle$ 和 $Y=\langle B,D,C,A,B,A\rangle$ 的最长公共子序列，利用 DPLCSLength(X,Y,m,n)算法求得的 c,b 值如图 6.10 所示。

		0	1	2	3	4	5	6
	c[i,j]		B	D	C	A	B	A
0		0	0	0	0	0	0	0
1	A	0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C	0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	B	0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	D	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	B	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

图 6.10

计算出数组 b 的值以后，我们可以构造出一个最长公共子序列，其过程是从 $b[m,n]$ 开始，并根据箭头的方向来进行递归的查找，其伪代码描述如下：

```
PrintLCS(b,X,i,j)
1  if i=0 or j=0 then return 0
2  if b[i,j]="↖" then
3      PrintLCS(b,X,i-1,j-1)
4      print x_i
5  else if b[i,j]="↑" then
6          PrintLCS(b,X,i-1,j)
7      else PrintLCS(b,X,i,j-1)
```

上述算法的时间复杂度为 $O(m+n)$。图 6.10 给出了构造最长公共子序列的过程，具体地我们可以从图 6.10 右下角带圈的数字 4($c[7,6]$)开始沿着箭头所指的方向构造出最长公共子序列，图中灰色圈所代表的字符即构成了最长公共子序列$\langle B,C,B,A\rangle$。

设计出一个算法后，我们常常会考虑算法能否在时间和空间上有所改进。DPLCSLength(X,Y,m,n)虽然很简单，但是还可以对代码进行改进。我们先看看用什么方式来计算表 $c[i,j]$的值。由递归方程可知，$c[i,j]$只依赖于 $c[i-1,j-1]$，$c[i-1,j]$和 $c[i,j-1]$的值，因此我们可以不使用表 b 并在 $O(1)$的时间内计算要选用三个值中的哪一个来计算 $c[i,j]$。因此，我们不用表 b 也可以在 $O(m+n)$时间内构造出最长公共子序列。虽然我们可以节省 $O(mn)$的空间，但辅助空间的需求并没有渐近地减少，我们仍然需要表 c。如果我们只需要最长公共子序列的长度，则 DPLCSLength(X,Y,m,n)一次只需表 c 的两行：正在被计算的一行和前一行，因此我们可以通过只存储这两行来降低渐近空间的要求。

6.5 0/1 背包问题

给定一个物品集合 $s=\{1,2,\cdots,n\}$，物品 i 具有重量 w_i 和价值 v_i。背包能承受的最大载重量不超过 W。背包问题(Knapsack Problem) 就是找到一个物品子集 $s'\subseteq s$，使得 $\max\sum_{i\in s'}v_i$，且满足 $\sum_{i\in s'}w_i\leqslant W$。

如果物品不能被分割，即物品要么整个地选取，要么不选取，则问题称之为0/1背包问题。如果物品可以拆分，则问题称之为背包问题。为了便于分析，下面我们进一步假定所有物品的重量、价值以及 W 都是正整数，然后介绍 0/1 背包问题的动态规划求解算法。

对物品按照顺序编号，令子问题(i,w)表示求给定物品集合$\{1,2,\cdots,i\}$，背包载重量为 w 的最大价值问题，则我们有如下结论。

定理 6.4 假设子问题(i,w)的最优装载中含有物品 i，则其中子问题($i-1$，$w-w_i$)的装载方案也一定是最优的。

证明：可用反证法证明。假设子问题($i-1,w-w_i$)的装载方案 p 不是最优的，则有一个更优的装载方案 p'，将 p'替换 p 然后再加上物品 i 将会比原来的最优装载的价值更大，与假设矛盾。

由定理 6.4 可得 0/1 背包问题具有最优子结构性质，因此可以写出递归方程。令 $V[i,w]$表示将物品 1 到物品 i 的物品装入载重量为 w 的背包中所能获得的最大价值，即子问题(i,w)的最大价值。现在考虑两种情形：

(1) 物品 i 能够装进背包，则子问题(i,w)的最优解取决于物品 i 是否放进去。

① 如果物品 i 不放进去，则有 $V[i,w]=V[i-1,w]$；

② 若将物品 i 放进去，则有 $V[i,w]=V[i-1,w-w_i]+v_i$。

上述两种选择，我们可以做最优选择

$$V[i,w]=\max\{V[i-1,w],V[i-1,w-w_i]+v_i\}$$

(2) 物品 i 无法放入背包，则子问题$(i-1,w)$的最优解一定是子问题(i,w)的最优解，即

$$V[i,w]=V[i-1,w]$$

当子问题只剩下一个物品时

$$V[1,w]=\begin{cases}v_1 & \text{if } w_1 \leqslant w\\ 0 & \text{otherwise}\end{cases}$$

综上所述，我们可以写出下列递归方程

$$V[i,w]=\begin{cases}V[i-1,w] & \text{if } w_i > w\\ \max\{V[i-1,w],V[i-1,w-w_i]+v_i\} & \text{if } w_i \leqslant w\end{cases}$$

有了上述递归方程，我们就可以写出相应的递归算法，但该递归算法的复杂度太高。表 $V[0..n,0..W]$用来保存子问题(i,w)的最大价值。$b[1..n,1..W]$用来保存所做出的最优选择，以便构造最优解。

下面给出动态规划算法：

```
DPKnapsack(S,W)
1    for w←0 to w1 - 1 do V[1,w]←0
2    for w←w1 to W do V[1,w]←v1
3    for i←2 to n do
4        for w←0 to W do
5            if wi > w then
6                V[i,w]←V[i-1,w]
7                b[i,w]←"↑"
8            else if V[i-1,w] > V[i-1,w-wi] + vi then
9                                V[i,w]←V[i-1,w]
10                               b[i,w]←"↑"
11               else
12                               V[i,w]←V[i-1,w-wi] + vi
13                               b[i,w]←"↖"
14   return V and b
```

DPKnapsack(S,W)算法的输入为物品的集合 S 及背包的载重量 W。填表先后顺序如图 6.11 所示，沿着箭头的方向由左至右，自顶向下填充。

DPKnapsack(S,W)算法有两重循环，因此其时间复杂度为 $O(nW)$。

现在，我们给出一个例子来演示 DPKnapsack(S,W)算法的执行过程。

图 6.12(a)是待求背包实例。图 6.12(b)给出了表 V 的值，其最优价值为 37。

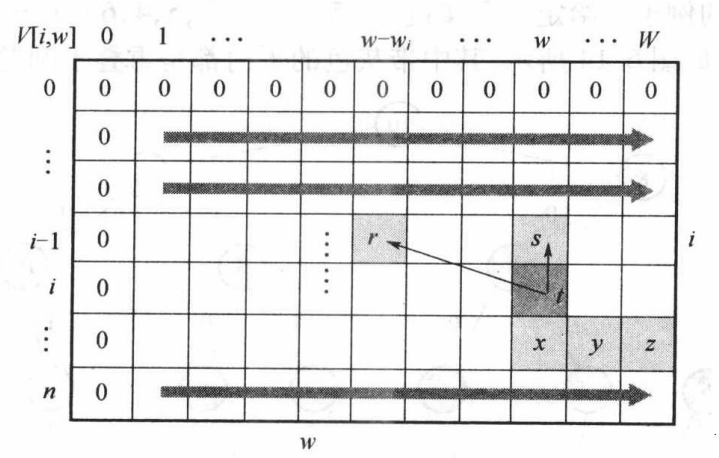


图 6.11

W=5

物品号i	w_i	v_i
1	2	12
2	1	10
3	3	20
4	2	15

(a)

V[i,w]

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

(b)

图 6.12

图 6.13 给出了表 b 的值，根据它我们可以构造最优解。

在计算最优解的时候，保存所做的最优决策，我们就可构造出最优解。下面给出打印最优解的伪代码：

```
PrintKnapsackItem(b,i,w)
1  if i=0 or w=0 then return 0
2  if b[i,w]="↖" then
3      PrintKnapsackItem(b,i-1,w-wi)
4      print i
5  else
6      PrintKnapsackItem(b,i-1,w)
```

b[i,w]

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	↑	⊖	↖	↖	↖
2	0	↖	↑	⊖	↖	↖
3	0	↑	↑	↑	↖	↖
4	0	↑	↖	↖	↑	⊖

图 6.13

对图 6.12(a)所示实例，调用 PrintKnapsackItem(b,4,5)，依次打印出物品号为 4,2,1，如图 6.13 中圆圈所在的行号，即可构造出最优解。

下面我们分析一下 0/1 背包问题的重叠子问题性质。如图 6.11 所示，在计算表中 x,y,z 的值时，都有可能用到表中 t 的值。为更清楚理解重叠子问题性质，我

们举一个具体的例子。给定一个实例 $n=5, v=[6,3,5,4,6], w=[2,2,6,5,4], W=10$，其递归树如图 6.14 所示，其中带灰色的子树都是重叠子问题。

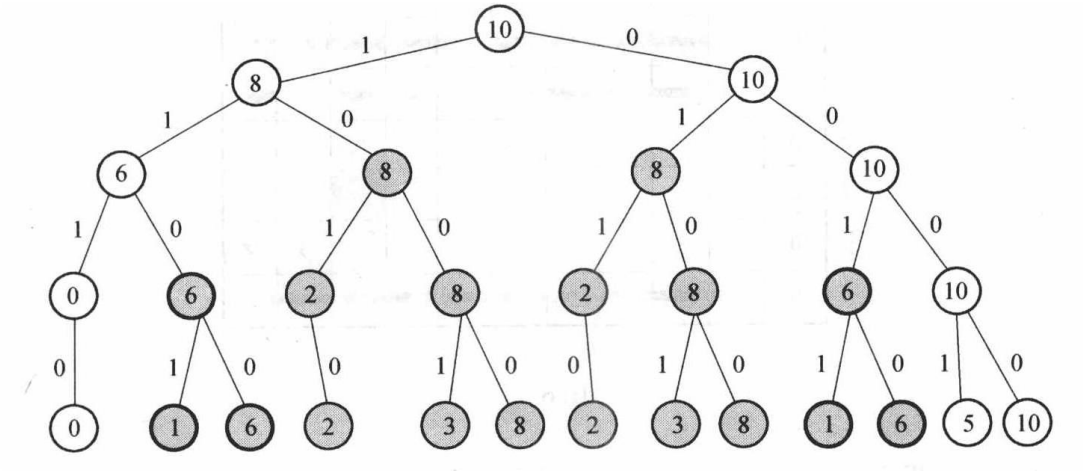


图 6.14

6.6 最优二叉搜索树问题

假定我们要设计一个将一段英文文本翻译成汉语的程序，对于文本中出现的每一个英文单词，我们需要查询它的汉语意思。为了执行查询操作，一个办法是将文本中出现的 n 个英文单词及其相应的汉语意思作为一个节点，然后建立一棵二叉搜索树。如果我们查询文本中每个英文单词的意思，我们必须查找整棵树，怎样才能使查找的代价更少？我们可以使用红黑树或者任何均衡的二叉搜索树，这样可保证 $O(\lg n)$ 的平均查找时间。然而，实际情况是有些英文单词经常出现，有些很少出现，如果使用上述的二叉树，当问题规模很大时，查找的效率仍然不高。因为查找一个节点，需要搜索的节点数为该节点的深度加 1。为了减少搜索的代价，我们希望文本中经常出现的单词应该离根节点更近。因此，在已知每个单词出现频率的情况下，如何构造一棵二叉树，使得所有搜索中，访问的节点数最小，就是我们需要解决的问题。

上述问题可以形式化为一个最优二叉搜索树（Binary Search Trees，BST）的问题：给定一个排好序的有 n 个不同关键字的序列 $K=\langle k_1,k_2,\cdots,k_n\rangle$ 且有 $k_1<k_2<\cdots<k_n$，对每个关键字 k_i，它被搜索到的概率为 p_i。由于有些单词不在序列 K 中出现，因此我们设 K 有 $n+1$ 个虚拟关键字 $d_0,d_1,d_2,\cdots,d_n$，以代表那些不在 K 中出现的单词。特别地，d_0 表示所有比关键字 k_1 值小的单词，d_n 表示所有比关键字 k_n 值大的单词，虚拟关键字 d_i 的值介于 k_i 和 k_{i+1} 之间，而且对每个 d_i，它被搜索到的概率为 q_i。问题是希望通过这些关键字构造一棵二叉搜索树。在建立问题的目标之前，我们先定义搜索一棵树的期望代价。由于某次搜索，要么找到某个关键字 k_i，要么

找到某个虚拟关键字 d_i，因此我们有 $\sum_{i=1}^{n} p_i + \sum_{i=0}^{n} q_i = 1$，其中 $\sum_{i=1}^{n} p_i$ 表示搜索成功的概率，而 $\sum_{i=0}^{n} q_i$ 表示搜索失败的概率。由于搜索每个关键字的概率已知，因此，我们能够确定搜索一棵二叉树的代价。假设一次搜索的实际代价为搜索过的节点数，也就是被搜索节点的深度加 1。那么对树 T 搜索一次的期望代价为

$$
\begin{aligned}
E(T) &= \sum_{i=1}^{n} (d_T(k_i) + 1) \times p_i + \sum_{i=0}^{n} (d_T(d_i) + 1) \times q_i \\
&= \sum_{i=1}^{n} d_T(k_i) \times p_i + \sum_{i=1}^{n} p_i + \sum_{i=0}^{n} d_T(d_i) \times q_i + \sum_{i=0}^{n} q_i \\
&= 1 + \sum_{i=1}^{n} d_T(k_i) \times p_i + \sum_{i=0}^{n} d_T(d_i) \times q_i
\end{aligned}
$$

其中，$d_T(k)$ 表示关键字 k 在树 T 中的深度。有了上述定义后，问题的目标也就明确了。即对一个给定关键字出现概率的集合，问题的目标是构造一棵使 $E(T)$ 最小的二叉搜索树，即最优二叉搜索树。

例如，给定如图 6.15 所示的关键字序列及其搜索的概率。图 6.16(a)给出了一棵二叉搜索树，其搜索的期望代价为 $E(T)=2.8$，图 6.16(b)给出了一棵最优二叉搜索树，其搜索的期望代价为 $E(T)=2.75$。由此可见，树的组织方式不同，得到的期望代价是不相同的。

i	0	1	2	3	4	5
p_i		0.15	0.1	0.05	0.1	0.2
q_i	0.05	0.1	0.05	0.05	0.05	0.1

图 6.15

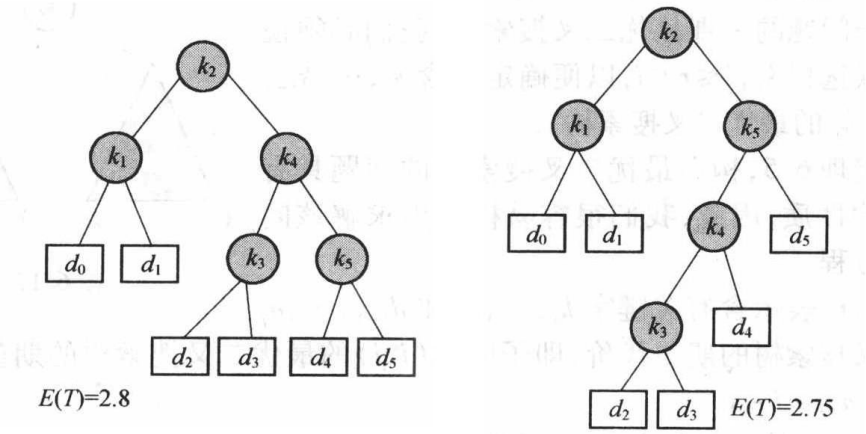


图 6.16

对于最优二叉搜索树的构造问题，我们来看看枚举搜索的代价。将关键字 k_1，$k_2,\cdots,k_n$ 作为任意一棵有 n 个节点的二叉树的节点，然后把那些虚拟关键字 d_0，$d_1,d_2,\cdots,d_n$ 作为叶子添加到二叉树中。对每棵二叉搜索树，计算其期望的搜索代价，然后选择期望代价最小的二叉树。由于共有 $\Omega(4^n/n^{3/2})$ 棵有 n 个节点的不同的二叉搜索树，因此枚举搜索显然代价太大。现在我们看看最优二叉搜索树问题的特点。

考虑一棵二叉搜索树的任一子树，它必须包含邻近的关键字 $k_i,\cdots,k_j$，且该子树也必须包含虚拟关键字 $d_{i-1},\cdots,d_j$ 作为它的叶子，其中 $1\leqslant i\leqslant j\leqslant n$。令子问题 (i,j) 表示对给定的关键字 $k_i,\cdots,k_j$ 和虚拟关键字 $d_{i-1},\cdots,d_j$ 构造最优二叉搜索树，则有如下结论。

定理 6.5 若一棵最优二叉搜索树 T 有一棵包含关键字 $k_i,\cdots,k_j$ 以及 $d_{i-1},\cdots,d_j$ 的子树 T'，那么这棵子树 T' 是子问题 (i,j) 的最优二叉搜索树。

证明：我们可以用反证法来证明。假设有一棵子树 T''，它的期望代价 $E(T'')$ 比树 T' 的 $E(T')$ 还要小，那么我们可以把 T' 从 T 中剪掉，然后将子树 T'' 粘贴上去，结果这棵新二叉搜索树的期望代价比 T 的期望代价还要小，这与 T 已经是一棵最优二叉搜索树矛盾。故所证成立。

现在，我们可以利用定理 6.5 来构造递归方程。考虑子问题 (i,j)，利用关键字 $k_i,\cdots,k_j$ 和 $d_{i-1},\cdots,d_j$ 构造最优二叉搜索树的问题。假定我们从关键字 $k_i,\cdots,k_j$ 中选择一个 $k_r(i\leqslant r\leqslant j)$ 作为最优二叉搜索树的根，则原问题可以分解成两个子问题：

(1) 子问题 $(i,r-1)$：构造包含 $k_i,\cdots,k_{r-1}$ 和 $d_{i-1},\cdots,d_{r-1}$ 的最优二叉搜索树（左子树）；

(2) 子问题 $(r+1,j)$：构造包含 $k_{r+1},\cdots,k_j$ 和 $d_r,\cdots,d_j$ 的最优二叉搜索树（右子树）。如图 6.17 所示。

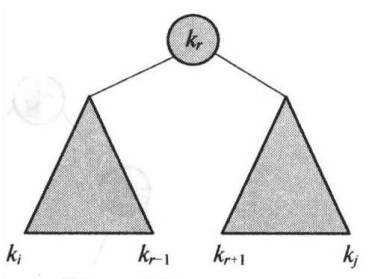


图 6.17

上面我们假设子问题的二叉树的根为 k_r，事实上，为了找出子问题的一棵最优二叉搜索树，我们必须检查所有的候选根 $k_r,i\leqslant r\leqslant j$，以便确定包含 $k_i,\cdots,k_{r-1}$ 和 $k_{r+1},\cdots,k_j$ 的最优二叉搜索树。

按照定理 6.5，构造最优二叉搜索树的问题具有最优子结构性质，因此，我们很容易构造出求解该问题的递归方程。

令 $e[i,j]$ 表示含有关键字 $k_i,\cdots,k_j$ 和 $d_{i-1},\cdots,d_j$ 的最优二叉搜索树的期望代价，即子问题 (i,j) 的最优二叉搜索树的期望代价，其中 $i\geqslant 1,j\leqslant n,j\geqslant i-1$。

若 $j=i-1$，该树只包含 d_{i-1}，因此 $e[i,j]=q_{i-1}$。

若 $j\geqslant i$，选择一个关键字 $k_r(i\leqslant r\leqslant j)$ 作为树根，递归地构造最优二叉搜索树，

将 $k_i,\cdots,k_{r-1}$ 作为左子树，将 $k_{r+1},\cdots,k_j$ 作为右子树。当左子树和右子树成为一个节点的子树时，对新构造出的子树的期望代价有什么影响呢？左子树和右子树中的每个节点的深度将增 1，因此按照期望代价的定义，当两棵子树成为一个节点的子树时，搜索期望代价增长为

$$w[i,j] = \sum_{l=i}^{j} p_l + \sum_{l=i-1}^{j} q_l \tag{6.2}$$

若 k_r 是一棵关键字为 $k_i,\cdots,k_j$ 的最优二叉搜索树的根，则

$$e[i,j] = e[i,r-1] + e[r+1,j] + w[i,j]$$

因此，我们应该选择使 $e[i,j]$ 值最小的关键字作为 k_r，即

$$e[i,j] = \begin{cases} q_{i-1} & \text{if } j = i-1 \\ \min\limits_{i\leqslant r\leqslant j}\{e[i,r-1] + e[r+1,j] + w[i,j]\} & \text{if } i \leqslant j \end{cases}$$

根据上述递归方程，我们很容易写出一个递归算法，但是其复杂度仍然太高。事实上，我们可以设计一个有效的动态规划算法。

我们开辟一个数组 $e[1..n+1,0..n]$ 来保存每个子问题的最优值 $e[i,j]$。由前面的分析可知，我们只需填充 $e[i,j], j \geqslant i-1$。同时，利用数组 $w[1..n+1, 0..n]$ 来保存子问题期望代价的增长 $w[i,j]$。为了计算 $w[i,j]$，可以采用下列递归方程

$$w[i,j] = \begin{cases} q_{i-1} & \text{if } j = i-1 \\ w[i,j-1] + p_j + q_j & \text{if } 1 \leqslant i \leqslant j \leqslant n \end{cases} \tag{6.3}$$

来提高计算的效率。此外，按照递归方程我们能够计算出最优值，但是，为了构造出最优解，我们必须引进一个数组 $root[i,j]$ 用来保存子问题含有关键字 $k_i,\cdots,k_j$ 的最优二叉搜索树的根 k_r 的下标值 r。下面我们给出动态规划算法的伪代码：

```
DPOptimalBST(p,q,n)
1  for i ←1 to n+1 do
2       e[i,i-1]←q_{i-1}
3       w[i,i-1]←q_{i-1}
4  for c←1 to n do
5       for i←1 to  n-c+1 do
6            j←i+c-1
7            e[i,j]←∞
8            w[i,j]←w[i,j-1]+p_j+q_j
9            for r←i to j do
10                t←e[i,r-1]+e[r+1,j]+w[i,j]
11                if t<e[i,j] then
12                     e[i,j]←t
13                     root[i,j]←r
```

14 **return** *e* and *root*

DPOptimalBST 算法主要有三重循环，因此其时间复杂度为 $O(n^3)$。

为了构造最优解，我们只需要调用下列程序：

PrintBST(*root*, *i*, *j*)

1 **if** $i=0$ or $j=0$ **then return** 0

2 **if** $i=j-1$ **then**

3 print $"d"_{i-1}$

4 **else** PrintBST(*root*, *i*, *root*[*i*, *j*] − 1)

5 print $"k"_{root[i,j]}$

6 PrintBST(*root*, *root*[*i*, *j*] + 1, *j*)

对于图 6.15 所示的例子，DPOptimalBST 算法计算的 $e[i,j]$ 如图 6.18(a)所示，$w[i,j]$ 如图 6.18(b)所示，*root*[*i*, *j*]如图 6.18(c)所示。

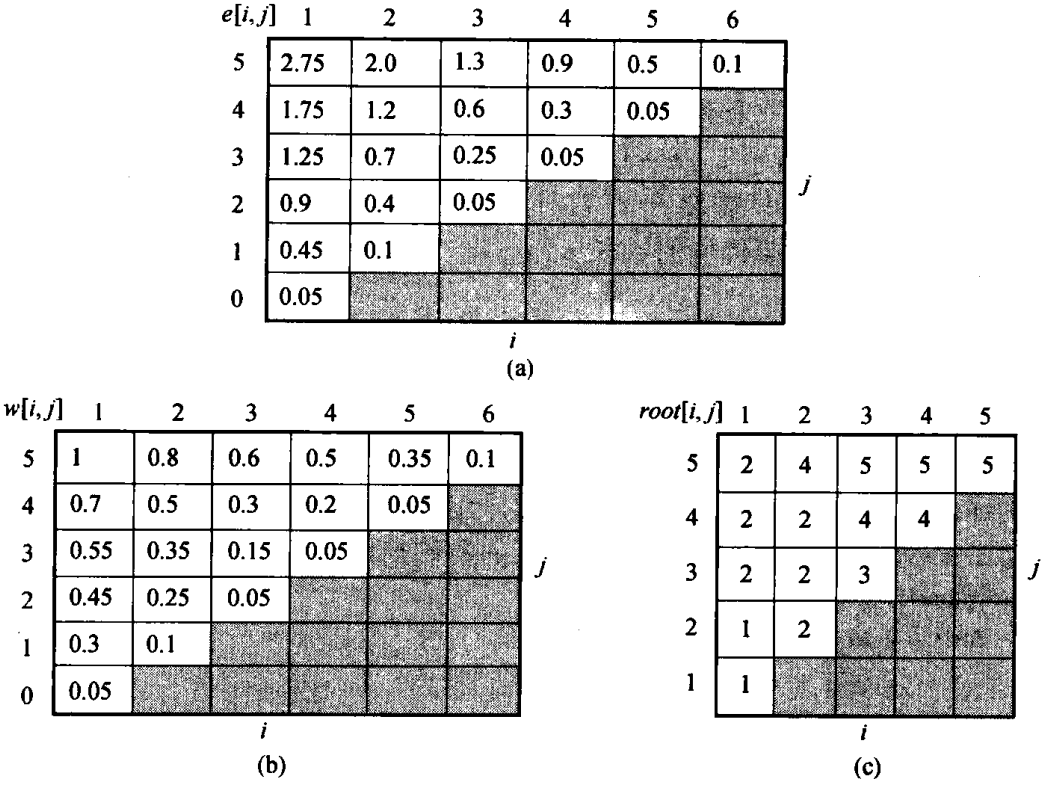


图 6.18

6.7 动态规划的基本性质

前面我们已经给出了应用动态规划来求解的例子，下面我们来看一个问题在什么情况下能够应用动态规划算法来求解。首先总结一下一个最优化问题能够应用动态规划求解的基本条件：最优子结构性质和重叠子问题性质。然后介绍一下

动态规划算法的变形。

应用动态规划算法的第一步是分析问题的最优子结构性质。前面几个例子已经给出了如何找最优子结构性质及其证明，找最优子结构性质具有下列规律：

（1）问题的划分依赖于一个决策，这个决策可以将原问题分解成一个或多个相似的子问题。

（2）假定一个导致最优解的决策，先不关心这个最优决策是如何得到的。

（3）对给定的这个最优决策，确定所划分的子问题以及如何方便地分析子问题解空间的变化。例如，在装配线调度问题中，子问题空间为 S_{1j} 和 S_{2j}，子问题空间只能一边缩小，而在矩阵链乘积问题中，子问题空间为 $A_{i..j}$，子问题空间可以两边缩小。

（4）然后证明，在问题的最优解 s 中用到的子问题的解 s_1，也一定是子问题的最优解。这可以使用“剪切和粘贴”技术来证明。即假定子问题的解 s_1 不是最优的，即存在一个比 s_1 更优的解 s'_1，将 s_1 从最优解 s 中“剪切”掉，将 s'_1“粘贴”上去，从而得到一个比 s 还优的解，而已知 s 已经是最优的，因此产生矛盾。故假设不成立。

最优子结构主要涉及下面两个问题：

（1）求原问题的一个最优解时要用到多少个子问题。

（2）确定哪些子问题的解包含在最优解中，有多少种选择。

例如装配线调度问题的最优解只用到一个子问题的解，但是为了确定最优解，我们必须考虑两个选择。而在矩阵链乘积问题中，最优解用到两个子问题，但是为了得到最优决策，我们必须考虑 $j-i$ 个选择。特别地，原问题在做出最优决策后而划分得到的子问题，对这些子问题的求解应该是独立的，下面将会给出一个例子进行说明。

由上面的分析可知，一个动态规划算法的运行时间取决于：

（1）所有子问题的数量；

（2）要在多少个选择里面做最优的决策。

例如在装配线调度问题里面，子问题的数量为 $O(n)$，在确定子问题时，只需从两个选择里面做决策，因此其时间复杂度为 $O(n)$。而矩阵链的乘积问题，子问题的数量为 $O(n^2)$，而在确定子问题时，最多从 $n-1$ 选择里做最优决策，因此算法的时间复杂度为 $O(n^3)$。

在使用动态规划算法来求解问题时，如果问题不具有最优子结构性质，而想当然地认为有，就会适得其反，达不到求解的效果。

给定一个有向图 $G=(V,E)$ 和顶点 $u,v\in V$，考虑下面两个问题：

（1）不带权的最短路径问题：寻找一条从 u 到 v 含有最少边数的路径。这样的路径必须是简单路径，即序列中顶点不重复出现的路径，要不然从这个路径中删去一个环会产生一条含有更少边数的路径。

(2) 不带权的最长简单路径:寻找一条从 u 到 v 含有最多边数的路径。这条路径必须是简单路径,要不然我们可以多次地绕着一个环遍历,从而得到一条含有任意多边数的路径。

对于不带权的最短路径问题,问题具有最优子结构性质。这可以通过使用"剪切和粘贴"技术来证明。假设从 u 到 v 的最短路径,包含了一个顶点 w,则从 u 到 w 的路径也一定是最短的。然而不带权的最长简单路径问题则不具有最优子结构性质。假设从 u 到 v 的最长简单路径,包含了一个顶点 w,则从 u 到 w 的路径也一定是最长简单路径吗?答案是否定的。例如图 6.19 给出了一个反例。假设从 u 到 v 的最长简单路径,包含了一个顶点 w,即 $u \rightarrow w \rightarrow v$,则 $u \rightarrow w$ 的路径不是最长简单路径,因为路径 $u \rightarrow x \rightarrow v \rightarrow w$ 更长。同样可以证明 $w \rightarrow v$ 也不是最长路径。虽然不带权的最短路径问题和不带权的最长路径问题都使用两个子问题,但是解这两个子问题,在不同的问题里面是不相同的。对于不带权的最长路径问题,解一个子问题跟解另一个子问题是不独立的,例如解子问题 u 到 w 的最长路径为 $u \rightarrow x \rightarrow v \rightarrow w$,而解子问题 w 到 v 的最长路径为 $w \rightarrow u \rightarrow x \rightarrow v$,均用到了顶点 x 和 v,因此是不独立的。将这两个子问题的解组合起来的路径就不是简单路径,因为顶点 x 和 v 出现了重复。而不带权的最短路径问题,解子问题 u 到 w 的最短路径以及 w 到 v 的最短路径是相互独立的。由此可见,将原问题划分为几个子问题,在解各个子问题时,互相独立,是问题具有最优子结构性质的关键。例如在装配线调度问题里,做了决策后,原问题只与一个子问题的最优解有关。在矩阵链乘积问题里,子问题 $A_{i..j}$ 在最优划分点 k 确定后,解子问题 $A_{i..k}$ 和子问题 $A_{k+1..j}$ 是相互独立的。

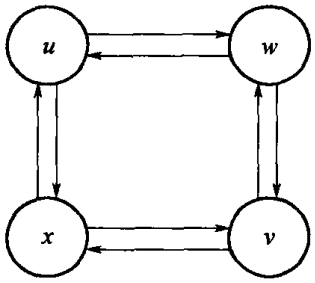


图 6.19

重叠子问题性质是保证动态规划算法有效求解一个问题的关键条件。当问题采用分治算法求解时,如果求解过程中需要反复地求解同一个子问题时,则说明这个问题具有重叠子问题性质。动态规划算法利用重叠子问题性质的特点,求解子问题一次,并将该值保存在表中,下次再碰到该问题的求解时,就只要从表中直接取值,而不需要重新计算该问题,从而避免大量的重复计算。

例如,对矩阵链的乘积问题,考虑下面的递归算法:

```
RecursiveMatrixChain(p,i,j)
1  if i = j then return 0
2  m[i,j] ← ∞
3  for k ← i to j - 1 do
4      q ← RecursiveMatrixChain(p,i,k) + RecursiveMatrixChain(p,k+1,j) + p_{i-1} p_k p_j
5      if q < m[i,j] then
6          m[i,j] ← q
7  return m[i,j]
```

上述递归算法的时间复杂度为 $O(2^n)$，涉及到大量的重复计算。例如当计算 $m[1,4]$ 时，其相应的递归树如图 6.20 所示，带灰色的节点，都是重叠子问题，在以前都计算过。利用动态规划算法则不用再计算。下面我们再介绍一下备忘录方法，它是动态规划算法的变形。其基本思想与递归的思想相似，采用自顶向下的策略，不同地是把第一次计算过的子问题的解保存在表中，以后每次遇到该子问题的时候只需查询之前表中填入的值即可。对于 Fibnoacci 序列问题，我们介绍了一个备忘录方法 $F1(n)$。

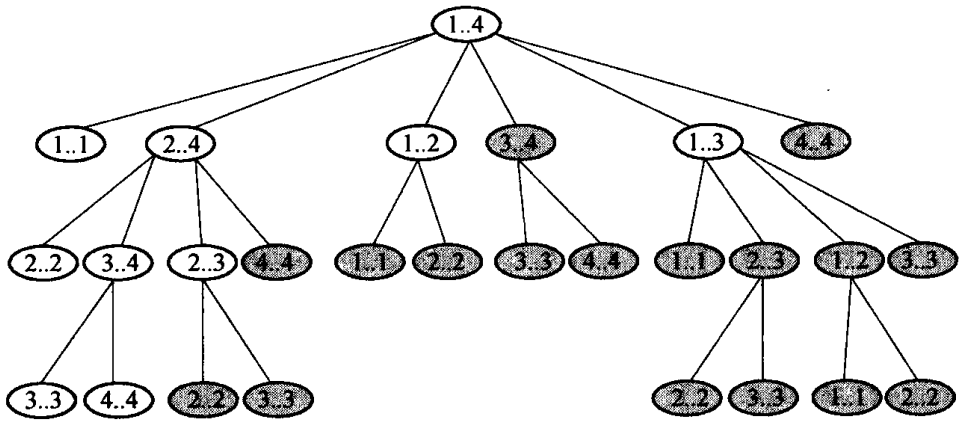


图 6.20

对于矩阵链乘积问题，其备忘录(Memoized)方法的伪代码如下：

```
MemoizedMatrixChain(p)
1  for i←1 to n do
2        for j←i to n do
3              m[i,j]←∞
4  return LookupChain(p,1,n)
```

备忘录方法先初始化 $m[1..n,1..n]$ 的值为 ∞，在调用 LookupChain$(p,1,n)$ 算法的过程中，如果 $m[i,j]<\infty$ 则表明子问题 $m[i,j]$ 已经计算过，直接返回。否则递归计算。对于图 6.20，带灰色的节点以前都计算过，其值被直接返回。LookupChain 算法如下：

```
LookupChain(p,i,j)
1  if m[i,j] < ∞ then return m[i,j]
2  if i = j then
3        m[i,j]←0
4  else for k←i to j - 1 do
5              q←LookupChain(p,i,k) + LookupChain(p,k + 1,j) + p_{i-1}p_kp_j
6              if q < m[i,j] then
7                 m[i,j]←q
8  return m[i,j]
```

现在我们比较一下动态规划算法与备忘录方法。与备忘录方法相比，动态规划算法采用自底向上的求解方式，不需要涉及函数的递归调用，维护表的代价较小，用表的形式降低了时间和空间的要求。而备忘录方法采用自顶向下的求解方式，有些子问题并不需要求解，但是仍然涉及函数调用和参数的传递。具体何时采用哪种方法，则根据具体问题来定。

6.8 小 结

前面所列举的例子说明了寻找一个问题的动态规划算法虽然没有什么规律可循，但是基本的求解框架是存在的：分析问题的最优子结构性质，构造出递归方程。其中分析最优子结构性质时，需要采用分治的思想。特别地，动态规划算法用填表的方法避免了大量重复的计算，提高了计算的效率。

本章内容主要取材于文献[1]，先介绍动态规划算法的基本步骤，然后通过6个典型的例子，介绍了动态规划算法的分析和设计过程。最后，介绍了动态规划问题的基本性质及其变形。动态规划算法在图问题中的应用，第8章将会介绍，这里不再详述。动态规划是一个非常强大、非常优美，也是一个非常讲究技巧性的方法，在计算机科学、运筹学以及经济管理领域，获得了大量的应用。此外，动态规划算法可以用来设计近似算法，也可以与其他方法，例如分支限界法结合使用。有兴趣的读者可以参考文献[4]。

习题

6.1 分析求 Fibonacci 序列递归算法 $F(n)$ 的时间复杂度。

6.2 说明应如何修改程序 PrintStations，使它能以装配点的递增顺序输出各装配点。（提示：利用递归。）

6.3 保存 $f_i[j]$ 和 $l_i[j]$ 值的表，总大小为 $4n-2$，说明如何把表的大小缩减到 $2n+2$，使之仍然能够计算出 f^*，并且仍然能够构造出最快装配路线。

6.4 对维数序列为 5,10,3,12,5,50,6 的矩阵链，找出其矩阵链乘积的一个最优加括号的方式。

6.5 给定矩阵链 $A_1, A_2, \cdots, A_i, \cdots, A_n$，由 DPMatrixChain 计算出的 s 以及下标 i 和 j，请设计一个求最优的矩阵链乘法的递归算法 MatrixChainMultiply(A, s, i, j)。（初始调用为 MatrixChainMultiply$(A, s, 1, n)$。）

6.6 使用替换法证明递归方程(6.1)的解为 $\Omega(2^n)$。

6.7 画出 MergeSort 算法排序 16 个元素的递归树。请解释在提高一个好的分治算法，例如 MergeSort 的效率时，利用备忘录方法为什么没有效果。

6.8 说明装配线调度问题具有重叠子问题性质。

6.9 已知原始序列 $X_m = \langle x_1, x_2, \cdots, x_m \rangle$ 和 $Y_n = \langle y_1, y_2, \cdots, y_n \rangle$，假设已计算出表 c，

说明不用表 b,也可以在 $O(m+n)$时间内构造一个 LCS。

6.10 请为 DPLCSLength 算法设计出一个运行时间为 $O(mn)$的备忘录算法。

6.11 设 X_m 和 Y_n 都是以 A 开始的字符序列,则 X_m 和 Y_n 的每一个最长公共子序列都从 A 开始,请予以证明。

6.12 给定一个具有 n 个数的序列,请设计出一个运行时间为 $O(n^2)$的算法,使之能找出该序列中最长的单调递增子序列。能设计出运行时间为 $O(n\lg n)$的算法吗?

6.13 如何修改 0/1 背包问题的动态规划算法,使得其运行时间复杂度为 $O(W)$,其中 W 为背包的最大载重量。如何修改动态规划算法,使其空间复杂度降低。

6.14 定义 0/1/2 背包问题为:$\max \sum_{i=1}^{n} v_i x_i$,使得 $\sum_{i=1}^{n} w_i x_i \leqslant W$,其中 $x_i \in \{0,1,2\}$,设其子问题的定义类似 0/1 背包问题中的定义,完成下列任务:

(1) 为 0/1/2 背包问题推出类似于 0/1 背包问题的递归方程。

(2)假设 w_i, v_i, W 为整数,设计一个类似于 0/1 背包的动态规划求解算法,并构造出最优解。

(3)分析上述算法的时间复杂度。

6.15 给定下表,请利用 DPOptimalBST(p,q,n)和 PrintBST($root,i,j$)算法分别求出最优二叉树的代价以及相应的二叉树。

i	0	1	2	3	4	5	6	7
p_i		0.04	0.06	0.08	0.02	0.1	0.12	0.14
q_i	0.06	0.06	0.06	0.06	0.05	0.05	0.05	0.05

6.16 如果不利用递归方程(6.3)计算表 $w[i,j]$,而是直接由公式(6.2)计算,试分析 DPOptimalBST(p,q,n)算法时间复杂度的变化。

6.17 著名计算机科学家 Knuth 已经证明,对任何 $1 \leqslant i < j \leqslant n$,最优子树的根总有 $root[i,j-1] \leqslant root[i,j] \leqslant root[i+1,j]$ 成立。用这个结论修改 DPOptimalBST(p,q,n)算法,使它的时间复杂度为 $O(n^2)$。

6.18 给定总金额为 A 的一张纸币,现要兑换成面额分别为 $a_1, a_2, \cdots, a_n$ 的硬币。硬币兑换问题是用最少枚数的硬币来兑换总金额为 A 的纸币。设计一个动态规划算法求解该问题,使得对任何可用的硬币种类总能求出最少枚数的硬币。

6.19 给定两个字符串 A 和 B 以及下列三种字符运算:

(1) 删除一个字符;

(2) 插入一个字符;

(3) 将一个字符改写为另一个字符。

问题是需要求出将字符串 A 转换为字符串 B，所需要的最少的字符运算数。请设计一个有效算法，对任给的 2 个字符串 A 和 B，输出将字符串 A 转换为字符串 B 所需的最少字符运算数。

6.20 在一个足球场的四周摆放着 n 堆货物，现要将周边货物合并成一堆。规定每次只能选相邻的两堆货物进行合并，合并的代价为新的一堆的货物数。试设计一个算法，求出将 n 堆货物合并成一堆所需要的最小总代价。

6.21 给定一个由 n 行数字组成的数字三角形，如图 6.21 所示。试设计一个算法，输出从三角形的顶至底的一条路径，使该路径经过的数字总和最大。

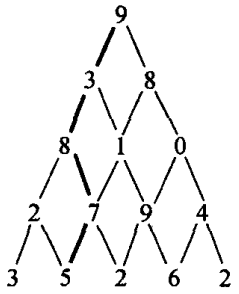


图 6.21

6.22 将 $3 \times n$ 的布条切割成 2×1 的方块，问有多少种切割方案？

6.23 一个机器人每步可以走 1 米、2 米或 3 米。编写一个动态规划算法，计算机器人走 n 米，有多少种走法（考虑步骤的次序）。

6.24 n 个人参加拔河比赛，每个人有自己的重量，现在需要把他们分成两组进行比赛，每个人属于其中的一个组，两组的人员个数相差不能超过 1。为使比赛公平，求使得两组重量差最小的分配。

实验题

6.25 分别实现矩阵链乘法的递归算法、动态规划算法及其备忘录算法，并用实验分析方法分析比较三种算法的效率。

6.26 完成 XOJ 如下题目：1010，1022，1023，1028，1029，1030，1031，1032，1033，1038，1041，1056，1059，1086。

6.27 完成 POJ 如下题目：1018，1050，1080，1088，1159，1160，1179，1276，1678，1742，2593，2614，2411，2778，3208。

第7章 贪心算法

7.1 算法思想

求解最优化问题的算法一般包括一系列的求解步骤，每一步骤中都要在许多选择中做一个决策，然后才能得到问题的一个解。贪心算法（Greedy Algorithm）总是在每一步骤中做最优的决策，希望通过这一系列的局部最优决策，获得问题的全局最优解。

贪心算法是一种非常有效的、简单的方法，在大多数情况下能找到问题的最优解。即使找不到问题的最优解，其近似解也令人非常满意。因此利用贪心算法来解决实际问题的例子，在日常生活中随处可见，例如股票投资、选择一所大学读书等。

本章先介绍用动态规划算法来求解任务选择问题，然后再将其改造成一个贪心算法，通过这个过程，让读者仔细体会贪心算法的简单、有效以及奥秘之处。然后介绍贪心算法在背包问题、哈夫曼编码问题中的实际应用。

7.2 任务选择问题

假定有一系列任务的集合 $S=\{a_1,a_2,\cdots,a_n\}$，这些任务竞争同一资源，这个资源一次只能被一个任务占用。任务占用该资源后，不能中断，直至完成。每个任务 a_i 有一个起始时间 s_i 和一个完成时间 f_i，且 $0\leqslant s_i<f_i<\infty$。如果任务 a_i 被选择占用资源，则该任务可在半开时间区间 $[s_i,f_i)$ 内占用资源。如果任务 a_i 的活动区间 $[s_i,f_i)$ 和任务 a_j 的活动区间 $[s_j,f_j)$ 互不重叠，则称任务 a_i 和 a_j 是相互兼容的，即它们占用资源时，互不冲突。

任务选择（Task Select）问题就是在任务占用资源互不冲突的条件下，选择尽可能多的任务占用资源，即要选择相互兼容任务的最大子集。

例如，给定任务集合 $S=\{1,2,3,4,5,6,7,8,9,10,11\}$，每个任务的开始时间和完成时间如图7.1所示。

图7.1中任务已经按照完成时间进行排序。其中相互兼容任务的子集为 $\{a_3,a_9,a_{11}\}$，最大相互兼容任务的子集为 $\{a_1,a_4,a_8,a_{11}\}$ 和 $\{a_2,a_4,a_9,a_{11}\}$。

对于任务选择问题，我们先介绍它的动态规划求解算法。首先我们看看这个问题是否具有最优子结构性质。

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

图 7.1

定义子问题空间

$$S_{ij} = \{a_k \in S : f_i \leqslant s_k < f_k \leqslant s_j\}$$

这个子问题包括了在 a_i 结束之后开始，a_j 开始之前结束的任务集合，如图 7.2 所示。

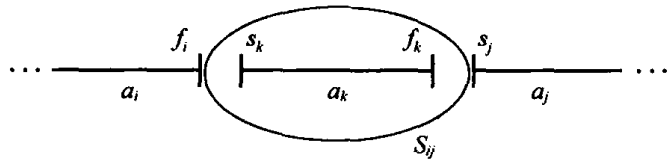


图 7.2

所有在 f_i 时都已经结束的任务以及所有不在 s_j 前开始的任务均和 S_{ij} 中的任务兼容。由子问题的定义可知，S_{1n} 并没有包括任务 a_1 和 a_n，因此要表示原问题，必须还要加入两个虚拟的任务 $a_0 = [-\infty, 0)$ 和 $a_{n+1} = [\infty, \infty + 1)$，这样子问题 $S_{0(n+1)} = S$ 就包括了原问题的所有任务。对子问题 $S_{ij}(0 \leqslant i, j \leqslant n+1)$，$i, j$ 变化的范围太大，为进一步缩小范围，我们假定集合 S 中的所有任务都按完成时间递增的顺序排序，即

$$f_0 \leqslant f_1 \leqslant \cdots \leqslant f_n < f_{n+1}$$

根据上述顺序，如果 $i \geqslant j$，即任务 a_i 排在 a_j 后面，则 $S_{ij} = \varnothing$。我们可以用反证法证明这个结论。由已知任务 a_i 排在 a_j 后面，必有 $f_i \geqslant f_j$，假设 $S_{ij} \neq \varnothing$，即存在一个任务 $a_k \in S_{ij}$，且 $f_i \leqslant s_k < f_k \leqslant s_j < f_j$。这与已知 $f_i \geqslant f_j$ 矛盾，故结论成立。

根据上述结论，我们只需要考虑求子问题 $S_{ij}(0 \leqslant i < j \leqslant n+1)$ 的最大相互兼容任务子集即可。

考虑从非空子问题 S_{ij} 中选择相互兼容任务的最大子集合，假定上述子问题 S_{ij} 的一个解包含任务 a_k，则上述子问题 S_{ij} 分解成两个子问题 S_{ik} 和 S_{kj}，如图 7.3 所示。

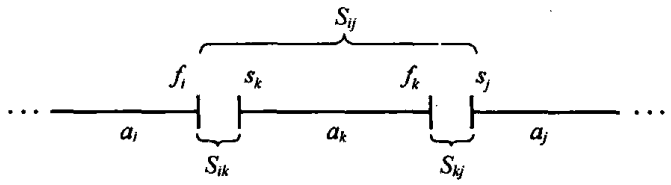


图 7.3

子问题 S_{ij} 的解，就由子问题 S_{ik}、S_{kj} 的解和 a_k 构成，即

$$\{\text{子问题 } S_{ij} \text{ 的解}\} = \{\text{子问题 } S_{ik} \text{ 的解}\} \cup \{a_k\} \cup \{\text{子问题 } S_{kj} \text{ 的解}\}$$

现在我们讨论最优子结构性质。

定理 7.1 假定子问题 S_{ij} 的最优解集合为 A_{ij} 且包括任务 a_k，则在最优解集合里的子问题 S_{ik} 的解 A_{ik} 以及子问题 S_{kj} 的解 A_{kj} 也一定是最优的。

证明：假设子问题 S_{ik} 存在一个解 A'_{ik}，它比 A_{ik} 含有更多的任务，则

$$|A'_{ik}| + 1 + |A_{kj}| > |A_{ik}| + 1 + |A_{kj}| = |A_{ij}|$$

这与我们假设 A_{ij} 是子问题 S_{ij} 的最优解矛盾。故所证成立。

从定理 7.1 可以看出，任务选择问题具有最优子结构性质。根据这个性质，我们来看看任何一个问题与其子问题最优解的关系。

令 $c[i,j]$ 表示子问题 S_{ij} 的最优解集合 A_{ij} 的大小，即 $|A_{ij}|$，显然

若 $S_{ij} = \varnothing$，则 $c[i,j] = 0 (i \geqslant j)$；

若 $S_{ij} \neq \varnothing$ 且假定 a_k 是最优解 A_{ij} 中的一个任务，则子问题 S_{ik} 的最优解 A_{ik} 的大小 $c[i,k]$ 以及子问题 S_{kj} 的最优解的大小 $c[k,j]$ 与最优解 A_{ij} 的大小 $c[i,j]$ 满足如下递归方程

$$c[i,j] = c[i,k] + c[k,j] + 1$$

在上述递归方程中，我们假设 a_k 是最优解 A_{ij} 中的一个任务而得到的，由 S_{ij} 的定义，a_k 可取 $a_{i+1}, a_{i+2}, \cdots, a_{j-1}$ 中任一个，k 共有 $j-i-2$ 个可能值，因此，我们考虑所有的 $a_k (i < k < j)$ 并选择使 $c[i,j]$ 最大的一个 a_k 即可，由此可得递归方程

$$c[i,j] = \begin{cases} 0 & \text{if } S_{ij} = \varnothing \\ \max\limits_{i<k<j, a_k \in S_{ij}} \{c[i,k] + c[k,j] + 1\} & \text{if } S_{ij} \neq \varnothing \end{cases} \tag{7.1}$$

根据上述递归方程，就可以写出一个动态规划算法了，见习题 7.1。第 6 章中介绍过，动态规划算法的效率还是很高的，但是对于任务选择问题，其计算量仍然过大。事实上，根据任务选择问题的特点，我们可以得到一个更简单的算法。我们先看一个定理。

定理 7.2 令子问题 $S_{ij} \neq \varnothing$ 且 a_m 为子问题 S_{ij} 中具有最早完成时间的任务，即

$$f_m = \min\{f_k : a_k \in S_{ij}\}$$

则

(1) a_m 一定包含在子问题 S_{ij} 中某个任务相互兼容的最大子集中，即存在含有 a_m 的最优解。

(2) 子问题 S_{im} 是空集，即选择 a_m 后，只剩下唯一一个可能具有非空解的子问题 S_{mj}。

证明：(1)假定子问题S_{ij}的最优解集合为A_{ij}，将集合A_{ij}中的任务按照完成时间由小到大进行排序。设A_{ij}中的第一个任务为a_k，如果$a_k = a_m$，则所证显然成立。如果$a_k \neq a_m$，则由于a_m是子问题S_{ij}中完成时间最早的，因此任务a_k的完成时间一定大于任务a_m的完成时间。从A_{ij}中去掉任务a_k，加入a_m后可得到一个新集合$A'_{ij} = \{A_{ij} - \{a_k\}\} \cup \{a_m\}$，$A'_{ij}$中的任务仍然是兼容的，而且$|A'_{ij}| = |A_{ij}|$。这样，我们得到了一个包含任务$a_m$的最优解$A'_{ij}$，故所证成立。

(2) 假设S_{im}不是空集，即存在一个任务$a_k \in S_{im}$，满足

$$f_i \leqslant s_k < f_k \leqslant s_m < f_m$$

这表明任务a_k的完成时间更早，这与已知a_m具有最早完成时间矛盾，故所证成立。

从定理7.2可知，通过选择具有最早完成时间的任务a_m，我们保证该任务包含在一个最优解中，因此没必要为做出选择而求解子问题S_{mj}。问题所具有的这种性质，我们称为贪心选择性质。因为问题具有贪心选择性质，我们没有必要像动态规划一样在$j-i-2$个选择中做最优决策，我们只需将S_{ij}中的任务按照完成时间进行排序，然后在每一步中，做贪心选择，即选择一个与前面已经选择的任务兼容且完成时间最早的任务即可。

除了减少子问题的个数以及做选择的次数外，定理7.2还带来另一个好处，即我们可以用自顶向下的方式解决每个子问题。为了求解子问题S_{ij}，我们只需选择其中具有最早完成时间的任务a_m，然后把该任务加入到最优集合A_{ij}，然后解子问题S_{mj}，重复上述过程，最后集合A_{ij}为所求。为了从子问题S_{ij}中选择任务相互兼容的最大子集A_{ij}，我们可以引出下列贪心算法：

(1) 选择具有最早完成时间的任务$a_m \in S_{ij}$(贪心选择)。

(2) 把a_m加到最优解的任务集合A_{ij}中。

(3) 对子问题S_{mj}重复上述过程。

值得注意的是，在步骤(1)中贪心选择的任务必须与已经选择的任务兼容，如果不兼容，则必须考虑接下来的一个。将$S_{0(n+1)}$中的任务按完成时间递增的顺序排序：

$$a_0, a_1, \cdots, a_n, a_{n+1}$$

则上述贪心算法思想，可以用递归算法描述如下：

```
RecursiveTaskSelect(s,f,i,j)
1  m←i+1
2  while m<j and s_m<f_i do
3              m←m+1
4  if m<j  then return {a_m} ∪ RecursiveTaskSelect(s,f,m,j)
5  else return ∅
```

因此，对原问题的求解，我们只需调用RecursiveTaskSelect(s,f,0,$n+1$)即可。

图7.4给出了一个递归算法解图7.1所示例子的过程，其中灰色的任务被选

择处理。首先调用 RecursiveTaskSelect(s,f,0,12),选择任务 a_1,其完成时间 $f_1=4$,如图 7.4(a)所示。调用 RecursiveTaskSelect(s,f,1,12),由于任务 a_2,a_3 的开始时间均小于 f_1,因此被抛弃,任务 a_4 被选择,如图 7.4(b)所示。调用 RecursiveTaskSelect(s,f,4,12),由于任务 a_5,a_6,a_7 的开始时间均小于 f_4,因此被抛弃,任务 a_8 被选择,如图 7.4(c)所示。类似地,可以得到图 7.4(d),最大相互兼容任务的子集为 $\{a_1,a_4,a_8,a_{11}\}$。

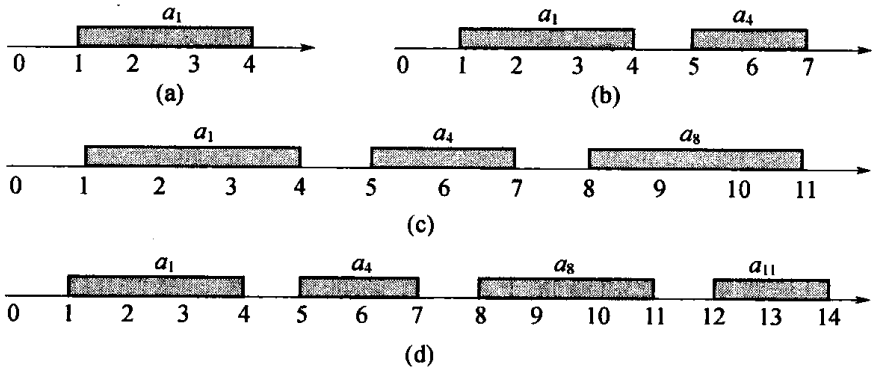


图 7.4

在任务已经按完成时间递增的顺序排序的前提下,只需调用 RecursiveTaskSelect(s,f,0,$n+1$)算法 $O(n)$次,即可完成求解,并且上述递归算法可以转化为更简单的迭代算法:

```
GreedyTaskSelect(s,f)
1  A←{a1}
2  i←1
3  for m←2 to n do
4        if sm≥fi then
5              A←A∪{am}
6              i←m
7  return A
```

GreedyTaskSelect(s,f)算法中,每个任务都被检查一次,因此其时间复杂度为 $O(n)$。

从前面的分析可知,求解任务选择问题的贪心算法如下:

(1) 分析问题的最优子结构性质。

(2) 构造递归方程,得到递归解。

(3) 证明在递归的每一阶段,最优选择可以通过贪心选择来实现,即做出贪心选择总是能得到问题的最优解。

(4) 证明做出贪心选择后,除了一个子问题外,所有其他子问题都为空。

(5) 用递归算法实现贪心策略。

(6) 把递归算法改造成迭代算法。

上述过程中，涉及分析问题具有最优子结构性质，并根据此性质构造递归方程。同时，分析贪心选择能否得到问题的最优解。最后将贪心选择与相应子问题的最优解组合在一起，就可以得到原问题的解。例如，在任务选择问题中，对于子问题 S_{ij}，做了贪心选择 a_m 后，得到子子问题 S_{mj}，则子问题 S_{ij} 的最优解 $A_{ij} = \{a_m\} \cup \{$子问题 S_{mj} 的最优解 $A_{mj}\}$。因此，我们可以得到贪心算法更一般的求解步骤：

（1）将最优化问题的求解转化成做一个选择，只留下一个待解决的子问题。

（2）证明对原问题总存在这样一个最优解，它包含贪心选择。

（3）在做出贪心选择后证明：贪心选择 + 子问题的最优解 = 原问题的一个最优解。

从任务选择问题的贪心算法，我们得到了贪心算法的一般求解步骤。现在，我们关心的是什么情况下，可以利用贪心算法求解最优化问题，通常情况下没有统一的判断标准，但是贪心选择和最优子结构性质是应用贪心算法的两个基本条件。如果一个最优化问题具有这两个性质，我们就可以应用贪心算法，从而得到问题的最优解。

应用贪心算法的第一个条件是贪心选择性质，即全局最优解可以通过局部最优选择得到，也就是说，在每一步，当我们考虑要做哪一个选择时，可以做当前看起来最好的选择，而无须考虑子问题的解。这也是贪心算法与动态规划算法的区别所在。在动态规划算法里，每一步的选择通常依赖于子问题的解，因此动态规划算法通常采用自底向上的求解方式，通过子问题的解来做决策，并进而获得原问题的解。贪心算法做选择时，只依赖到目前为止所做的选择，与将来的选择以及将来的子问题都没有关系，因此贪心算法通常采用自顶向下的求解方式。通过做贪心选择，从而得到类似的规模更小的子问题。贪心算法由于做贪心选择时，无须考虑子问题的解，从而比动态规划算法更有效。当然，我们必须证明，待求解的最优化问题具有贪心选择性质。

如果问题的最优解包括了子问题的最优解，则说明该问题具有最优子结构性质，这个性质是应用贪心算法和动态规划算法求解一个问题的最基本条件。例如，在任务选择问题中，如果子问题 S_{ij} 的最优解包括任务 a_k，则其最优解 A_{ij} 必定包括子问题 S_{ik} 的最优解 A_{ik} 以及子问题 S_{kj} 的最优解 A_{kj}。由此可见，应用贪心算法和动态规划算法都需要满足一个条件，即问题具有最优子结构性质，但是它们还是有很大的区别。下面我们再给出一些例子。

7.3 背包问题

对于 0/1 背包问题，动态规划一节已经证明该问题具有最优子结构性质。事实上，背包问题也具有最优子结构性质。考虑 n 个物品，重量不超过 W 的最有价

值的装载。如果我们在这个最优装载中，从物品 j 中取走部分重量 w，那么剩下的装载必定是重量不超过 $W-w$，$n-1$ 个剩余物品以及重量为 w_j-w 的物品 j 的价值最大的装载。这个最优子结构性质可用反证法类似证明（习题 7.4）。

现在我们考虑解背包问题的两个贪心策略。

（1）贪心策略 1：每次选择价值最大的物品装入背包。例如，给定 $W=50$，$w_1=10$，$v_1=60$，$w_2=20$，$v_2=100$，$w_3=50$，$v_3=150$，该策略解 0/1 背包问题可以得到总价值 150，最优装载的价值为 160。该策略解背包问题，可以得到总价值 150，事实上，最优装载达到总价值 220。

（2）贪心策略 2：每次选择单位重量价值 v_i/w_i 最大的物品。例如，给定 $W=50$，$w_1=10$，$v_1=60$，$w_2=20$，$v_2=100$，$w_3=30$，$v_3=120$。该策略解 0/1 背包问题可以得到总价值 160，而最优装载的价值为 220。该策略解背包问题，可得到总价值 240，如图 7.5 所示。

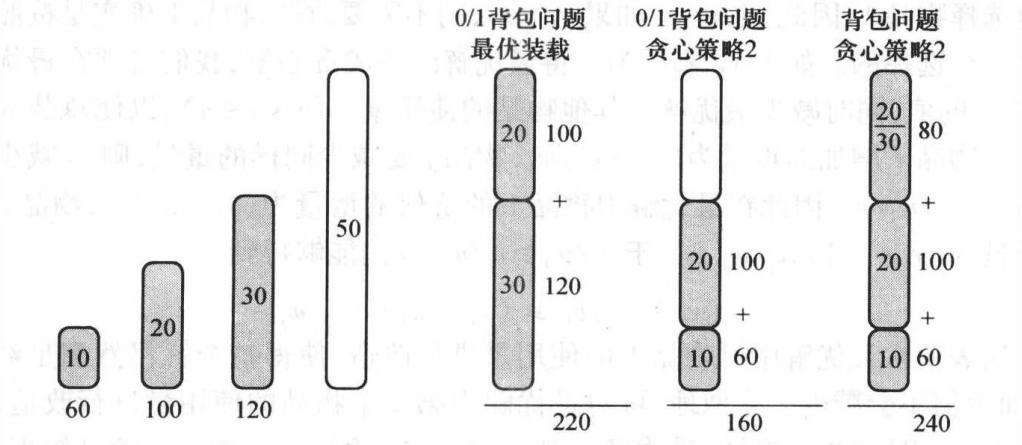


图 7.5

从上面的分析可知，两个贪心策略对 0/1 背包问题，都不能得到最优解。贪心策略 1 解背包问题，也不能得到最优解。虽然 0/1 背包问题和背包问题类型比较相似，但是 0/1 背包问题不具有贪心选择性质，因此无法利用贪心算法来得到问题的最优解，而背包问题虽然不能利用贪心策略 1 获得最优解，但是却可以用贪心策略 2 找到最优解。这说明即使贪心策略也并不一定得到背包问题的最优解。下面我们先给出贪心策略 2 的伪代码：

```
Greedy2Knapsack(W,v)
1   order items based on their value per pound v1/w1 ≥ v2/w2 ≥ ··· ≥ vn/wn
2   i←1; w←W
3   while w > 0 and as long as there are items remaining do
4       xi←min {1, w/wi}
5       remove item i from list
6       w←w − wixi
7       i←i + 1
```

Greedy2Knapsack(W,v)算法的时间复杂度主要取决于排序算法，因此为$O(n\lg n)$。下面证明该算法解背包问题具有贪心选择性质。

定理 7.3 解背包问题的 Greedy2Knapsack(W,v)算法具有贪心选择性质。

证明：设按照单位重量价值比从大到小排序后，物品的顺序为

$$1 \quad 2 \quad 3 \quad \cdots \quad i \quad \cdots \quad n$$

设最优解为

$$x_1 \quad x_2 \quad x_3 \quad \cdots x_i \quad \cdots x_n$$

贪心解为

$$y_1 \quad y_2 \quad y_3 \quad \cdots y_i \quad \cdots y_n$$

我们证明的思路是将最优解改造成贪心解，使得改造后的解满足问题的约束条件，而且总价值不会比最优解的总价值少。首先看物品 1，因为贪心选择尽可能多地选择物品 1，因此 $y_1 \geqslant x_1$。如果 $y_1 = x_1$，则不需要改造，物品 1 确实是按照贪心策略进行选择的。如果 $y_1 > x_1$，为了将最优解改造成贪心解，我们必须在最优解中增加 x_1 的值，同时减少最优解中其他物品的使用量 $x_j(1 < j \leqslant n)$，以使总装入重量不变。物品 1 增加的重量为$(y_1 - x_1)w_1$，物品 j 要减少同样的重量，则 x_j 减少的值为$(y_1 - x_1)w_1/w_j$，因此在最优解中物品 1 的价值的增量为$(y_1 - x_1)v_1$，物品 j 价值的减量为$(y_1 - x_1)w_1v_j/w_j$。由于 $v_1/w_1 \geqslant v_j/w_j$，我们能够得到

$$(y_1 - x_1)v_1 \geqslant (y_1 - x_1)w_1v_j/w_j$$

这表明在最优解中对物品 1 的使用量进行改造，使得整个解仍然满足约束条件，而价值不会减少。类似地，可对最优解中第 2 个物品的使用量进行改造，直到物品 $i(y_i = 0)$为止。这样，最优解就被改造成贪心解，这个贪心解满足约束条件，而且其价值不会比最优解的价值小，显然贪心解也是最优解。因此贪心算法 Greedy2Knapsack(W,v)具有贪心选择性质，从而能获得问题的最优解。

7.4 哈夫曼编码问题

哈夫曼编码是数据压缩算法中效率很高的一种压缩技术，在数据压缩领域具有广泛的应用。当然，压缩的效果取决于压缩的数据，一般压缩率能达到 20% ~ 90%。为方便分析，我们考虑一个字符串数据，哈夫曼贪心算法就是根据每个字符在串中出现的频率，用一个二进制串来代表每一个字符，以达到压缩的目的。

假设我们有一个包含 100000 个字符的数据文件需要压缩存储，该数据文件包含 $a \sim f$ 共 6 个字符，各字符出现的频率如图 7.6 所示。

有许多方式来表示这样的文件信息，下面我们用二进制字符码字来表示，即用一个二进制串来代表每一个字符。如果编码每个字符的二进制串的长度相等，则称这种编码方式是固定长度码字编码，如果二进制的长度不等，则称变长码字编

	a	*b*	*c*	*d*	*e*	*f*
频率	45	13	12	16	9	5
固定长度的码字	000	001	010	011	100	101
变长码字	0	101	100	111	1101	1100

图 7.6

码。从图 7.6 中可以看出，采用长度为 3 的固定长度码字编码，需要 100000 × 3 = 300000 位。我们能否做得更好呢？采用可变长度编码，需要 (45 × 1 + 13 × 3 + 12 × 3 + 16 × 3 + 9 × 4 + 5 × 4) × 1000 = 224000(位)，比固定长度编码节约 25%。事实上，上述编码是一种最优的编码方式。

在介绍哈夫曼编码之前，我们介绍一些概念。

前缀编码指的是任何一个编码码字都不是其他编码码字的前缀。可以证明，由字符编码获得的最优数据压缩总可用某种前缀编码来获得。哈夫曼就是利用前缀编码来实现最优的数据压缩。编码指的是将文件中代表各个字符的编码连接起来。例如 *abc*→0 · 101 · 100→0101100，其中 · 表示连接符。

前缀编码的好处在于它简化了编码与解码。利用前缀编码可以很容易地实现解码，因为任何一个编码码字都不是其他编码码字的前缀，因此解码一个文件的码字是明确的。例如 001011101→0 · 0 · 101 · 1101→*aabe*。

解码过程需要一种方便的形式来表示前缀码，二叉树就是一种好的表示方式。用二叉树的叶节点来代表每个字符。我们将一个字符的编码解释为从根至该字符的路径，其中 0 表示“转向左子节点”，1 表示“转向右子节点”，则我们可以构造出一棵与编码序列相符的二叉树。例如，图 7.6 中字符的编码就可以方便地用二叉树来表示，其中图 7.7(a)是固定长度编码的二叉树表示，图 7.7(b)是变长编码的二叉树表示。

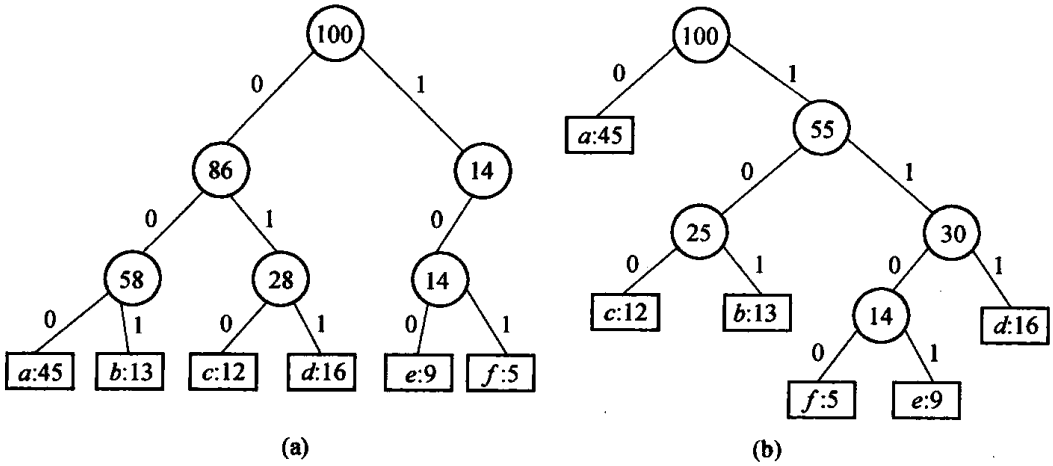


图 7.7

给定对应一种前缀编码的二叉树 T,很容易计算出编码一个文件所需的位数。对字符表 C 中的每个字符 c,设 $f(c)$ 表示 c 在文件中出现的频率,$d_T(c)$ 表示叶子 c 在树中的深度。注意 $d_T(c)$ 也是字符 c 的编码长度。这样编码一个文件所需的位数为

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

并将它定义为树 T 的代价。由于编码一个文件所需的位数取决于二叉树 T,因此问题的关键在于如何构造二叉树 T,使得 $B(T)$ 尽可能的小。

哈夫曼提出了一个贪心算法来构造最优的前缀编码,其正确性在于贪心选择性质和最优子结构性质,后面我们将给出证明。哈夫曼算法采用自底向上的方式来构造二叉树。先初始化每一个字符用一个单独节点表示,从而得到树的森林,然后在每一步选择频率最小的两棵树组合成一棵更大的树,其伪代码如下:

```
HuffmanCode(C)
1  Q←C
2  for i←1 to n-1 do
3        allocate a new node z
4        x←ExtractMin(Q)
5        y←ExtractMin(Q)
6        f(z)←f(x)+f(y)
7        Insert(Q,z)
8  return ExtractMin(Q)
```

其中 C 为一个字符表,其大小为 $n = |C|$,Q 是一个最小优先队列,可用最小堆来实现,ExtractMin(Q) 和 Insert(Q,z) 均是有关最小堆的运算。因为每个堆运算需要 O(lg$n$),需要循环 n 次,因此 HuffmanCode(C) 的时间复杂度为 O($n$lg$n$)。

下面,我们给出 HuffmanCode(C) 算法对字符表 $C = \{a,b,c,d,e,f\}$ 构造最优前缀编码的过程,如图 7.8 所示。图 7.8(a) 给出了每个字符出现的频率,并初始化为一座森林。然后,选择两个最小的节点 x 和 y,产生一个新节点 z,从而得到一棵新的子树,如图 7.8(b) 所示,图 7.8(b) ~ (f) 给出了详细的计算过程。

为了证明哈夫曼算法的正确性,我们需要证明确定最优前缀编码的问题具有贪心选择性质以及最优子结构性质。下面先证明问题具有贪心选择性质。

定理 7.4 令 C 为一个字符表。对任意的 $c \in C$,字符 c 在文件中出现的频率为 $f(c)$。令 x 和 y 为 C 中出现频率最小的两个字符。那么对 C 存在一个最优前缀编码。在这种编码中,x 和 y 的编码长度最长,且长度相等,只有最后一位不同。

证明:证明的思想是对于一棵表示任意最优前缀编码的二叉树 T,如果该树最深的两个叶节点为 x 和 y,则不需要证明。否则我们将该树进行修改,使得该树表示另一个由贪心算法构造的最优前缀编码,而且 x 和 y 出现在深度最深的且具有共同父节点的叶子上。假设树 T 表示任一种最优前缀编码,a 和 b 为树 T 中具有

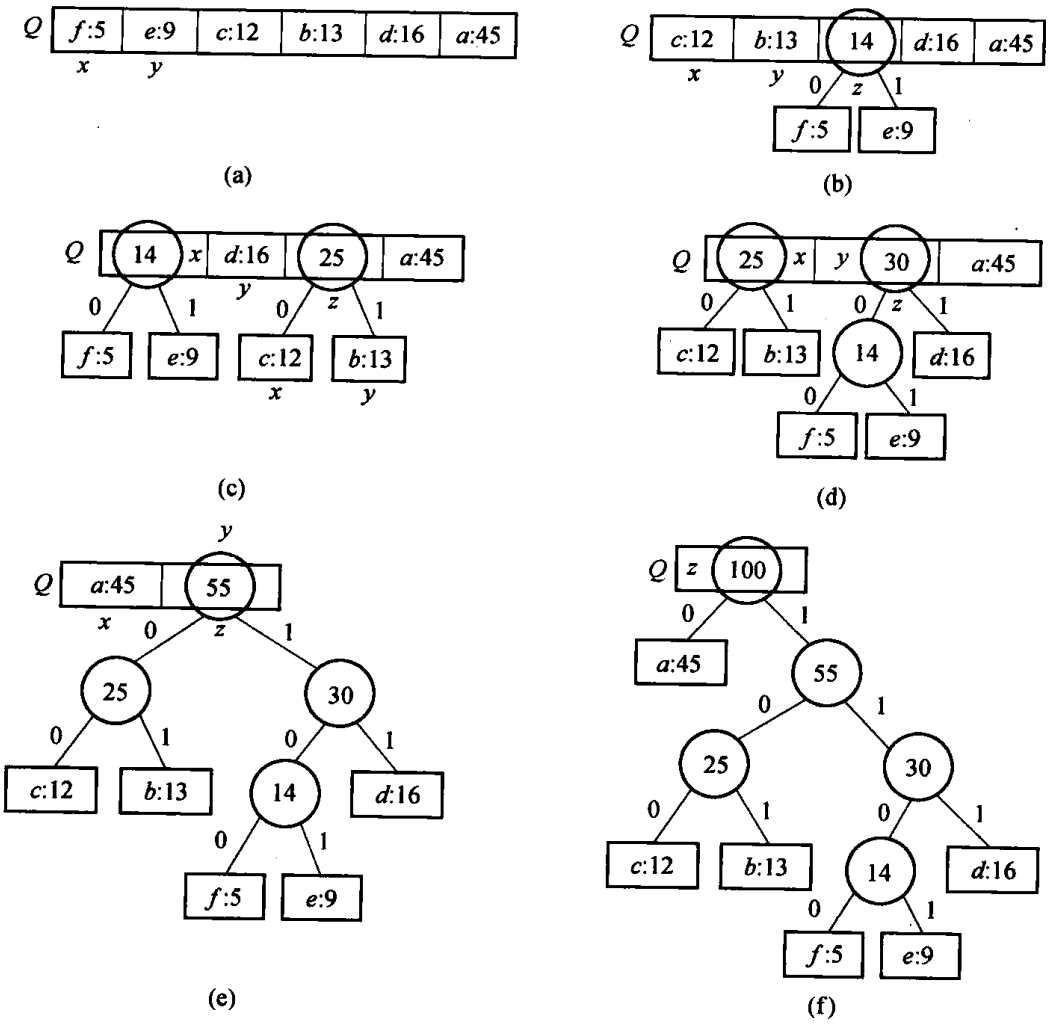


图 7.8

最大深度的节点，如图 7.9 所示。不失一般性，我们假定 $f(a) \leqslant f(b)$ 且 $f(x) \leqslant f(y)$。因为 x 和 y 是两个出现频率最低的字符，因此有 $f(x) \leqslant f(a)$ 和 $f(y) \leqslant f(b)$。将节点 a 和 x 互换得树 T'，如图 7.9 所示。

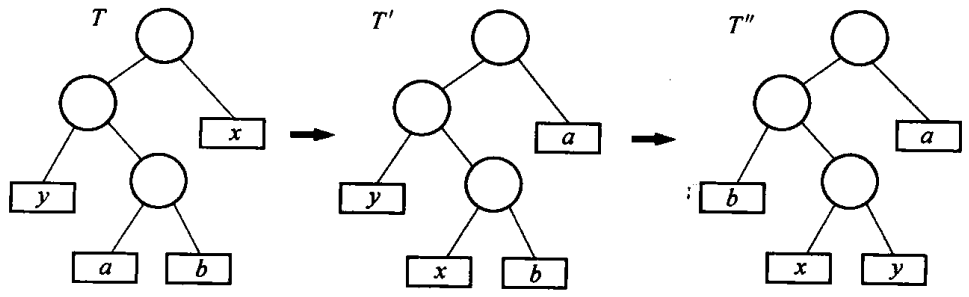


图 7.9

树 T 与 T' 的代价之差为

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \\ &= f(x) d_T(x) + f(a) d_T(a) - f(x) d_{T'}(x) - f(a) d_{T'}(a) \\ &= f(x) d_T(x) + f(a) d_T(a) - f(x) d_T(a) - f(a) d_T(x) \\ &= (f(a) - f(x))(d_T(a) - d_T(x)) \end{aligned}$$

由于 $f(x) \leqslant f(a)$ 且 $d_T(x) \leqslant d_T(a)$，因此 $B(T) - B(T') \geqslant 0$，即 $B(T) \geqslant B(T')$。

类似地，将 T' 中的 b 与 y 对换得到 T''，同理可得 $B(T') \geqslant B(T'')$。

由此可见，每次交换两个节点都不增加代价，所以 T'' 也是最优前缀编码树，其中 x 和 y 为具有最大深度的兄弟节点，从而证明了上述定理。

这个定理证明了构造最优前缀编码的问题具有贪心选择性质。下面我们证明构造最优前缀编码的问题具有最优子结构性质。

定理 7.5 令 C 为一个字符表。对任意的 $c \in C$，字符 c 在文件中出现的频率为 $f(c)$。令 x 和 y 为 C 中出现频率最小的两个字符。从 C 中删除 x 和 y 两个字符并加入新的字符 z，得到新的字符表 C'，即 $C' = \{C - \{x,y\}\} \cup \{z\}$。除了 $f(z) = f(x) + f(y)$ 外，C' 中字符频率的定义同 C。树 T' 代表 C' 的最优前缀编码，对树 T' 的叶节点 z，添加两个子节点 x 和 y，就可以得到树 T，则 T 代表 C 的最优前缀编码。

证明：为了证明最优子结构性质，我们必须将树 T 与 T' 的代价建立起关系。对 $c \in C - \{x,y\}$，我们有 $d_T(c) = d_{T'}(c)$，因此 $f(c)d_T(c) = f(c)d_{T'}(c)$。因为 $d_T(x) = d_T(y) = d_{T'}(z) + 1$，我们有

$$\begin{aligned} f(x)d_T(x) + f(y)d_T(y) &= (f(x) + f(y))(d_{T'}(z) + 1) \\ &= f(x) + f(y) + f(z)d_{T'}(z) \end{aligned}$$

由上式我们可得 $B(T) = B(T') + f(x) + f(y)$。

假设 T 并不是 C 的最优前缀编码，则由定理 7.4 可知，存在树 T''，使得 $B(T'') < B(T)$ 且 x 和 y 为兄弟节点。这样可以根据字符集 C' 构造一棵树 T'''，使得 $B(T'') = B(T''') + f(x) + f(y)$，由于 $B(T'') < B(T)$，因此可得 $B(T''') < B(T')$，这与 T' 代表 C' 的最优前缀编码矛盾。故 T 代表 C 的最优前缀编码，所证成立。

由定理 7.4 和 7.5 可知，HuffmanCode(C) 产生一种最优前缀编码，从而证明了算法 HuffmanCode(C) 的正确性。

7.5 缓存维护问题

缓存维护问题在许多实际应用领域中出现，具有不同的目标和约束条件。一般地，缓存维护问题可以描述如下：考虑 n 个页面，它驻留在主内存中，同时也

有一个高速缓存，它能容纳 k 个页面。现有来自于主内存中的 m 个页面请求序列 $I=I_1,\cdots,I_i,\cdots,I_m$，其中 $m>k$。对于这个要处理的序列，我们必须随时决定当前哪 k 个页面在缓存中。对于某个请求的页面 I_i，如果它在缓存中，我们可以很快地访问；如果它不在缓存中，就发生一次缺页，这时必须将它调入缓存，如果当前缓存未满，则直接将其调入缓存，否则必须决定将缓存中的哪个页面置换出来，以便为 I_i 腾出空间。缓存维护问题的目的是如何调度页面，使得缺页数尽可能少。

当要将页面 I_i 调入缓存时，我们必须考虑将缓存中的一个页面置换出，通常有下面贪心置换策略：

（1）先进先出（FIFO）策略：每次将缓存中驻留时间最长的一个页面置换出。

（2）请求最迟策略：每次将缓存中将来最迟被请求的一个页面置换出。

FIFO 策略比较简单，而且易于实现，因为它只需要一个先进先出队列来管理页面的置换。当主内存发出一个请求页面时，把页面插入队列，然后把页面放入缓存中。当需要置换出页面时，计算机简单地执行出队操作，以确定置换出的页面。因此，每次页面置换发生时，这个策略只需要 O(1) 个额外的计算量。同样，对于页面请求，FIFO 策略也不需要额外的开销。

请求最迟策略是一种很自然的策略，它总是置换出（evict）将来最迟被请求的一个页面。详细的请求最迟策略的算法如下：

```
GreedyCaching( )
1   the set of pages in cache is C
2   k←0
3   for i←1 to m do
4       if I_i ∈ C then evict nothing
5       else  select a page I_j ∈ C that is latest to be requested in the future
6             C←C − I_j
7             C←C∪I_i
8             k←k + 1
9   return k
```

其中 k 是缺页次数。GreedyCaching( ) 在第 1 行初始化缓存（cache）的页面集合 C，第 3 行对请求序列中的每个页面（page）I_i，如果 $I_i\in C$，则不需要置换（第 4 行），否则，在第 5 行选择一个将来最迟被请求的页面 $I_j\in C$。第 6 行置换出 I_j，第 7 行调入请求页面 I_i。

例如，考虑一个请求序列

$$I_1,I_2,I_3,I_4,I_1,I_4,I_5,I_1,I_4,I_2,I_3$$

$k=3$，最初缓存中有 $C=\{I_1,I_2,I_3\}$ 3 个页面。如果按照贪心置换策略（1），前面 3 步不需要置换。第 4 步，贪心置换策略（1）将置换出 I_1；第 5 步，将置换出 I_2；第 6

步不发生置换;第7步置换出I_3;第8步至第9步不需要置换;第10步置换出I_4;第11步置换出I_1。执行完整个页面请求序列,总共置换出5个页面,因此缺页次数为5。

如果按照贪心置换策略(2),第4步,由于缓存中的I_3将来最迟被请求,即它将在第11步才被请求,因此贪心置换策略(2)将置换出I_3。同理,第7步,将置换出I_2。第10步,由于当前缓存中的页面将来不会被请求,此时置换出缓存中的哪个页面,可以随机决定,也可以按照先进先出的规则,置换出一个页面即可。类似地,第11步也照此处理,置换出一个页面。执行完整个页面请求序列,总共置换出4个页面,因此缺页次数为4。事实上,这个贪心置换策略(2)是最优的。下面我们证明这个策略具有贪心选择性质。

定理7.6 GreedyCaching()算法具有贪心选择性质。

定理7.6的证明类似背包问题,布置为习题7.14,这里就不再给出。

上述贪心算法在使用前必须预先得知整个页面请求序列,而在实际应用中,是无法知道整个页面请求序列的。在这种情况下,我们必须考虑随机在线算法。这里就不再介绍,有兴趣的读者可以参考文献[4,22]。

7.6 任务选择问题实验

前面我们已经介绍了利用动态规划算法、递归贪心算法和贪心算法来求解任务选择问题。为了比较三种算法的效率,我们将三种算法用C++语言编程实现,并在CPU为2.4GHz,内存为512MB的PC机上测试规模n分别为100,200,400,600,800,1000的同样实例,这些实例均随机生成,算法的执行时间如表7.1和图7.10所示,其中时间用秒表示。图7.10清楚直观地表示了随着问题规模n的增大,执行时间的增长趋势。三种算法都能找到最大相容任务的子集,但是从表7.1和图7.10可以看出,递归贪心算法和贪心算法几乎不需要时间,在速度上没有什么区别,而动态规划算法,随着问题规模的增大,需要的时间越来越多,因此贪心算法效率更高。

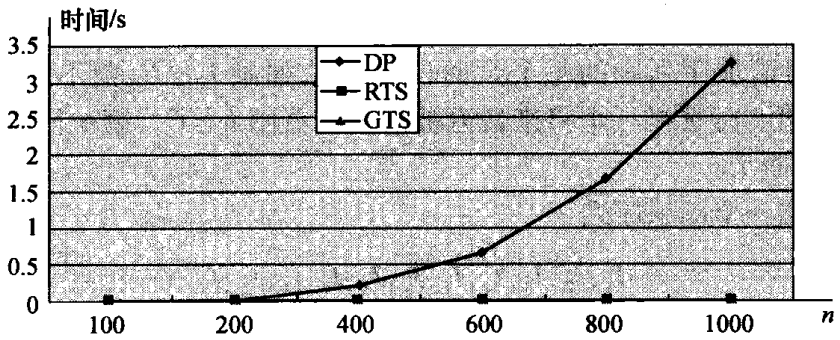


图7.10

表 7.1　求解任务选择问题的动态规划算法、递归贪心算法和贪心算法执行时间的实验结果

n	100	200	400	600	800	1000
动态规划算法(DP)	0.000	0.015	0.189	0.656	1.656	3.250
RecursiveTaskSelect(RTS)	0.000	0.000	0.000	0.000	0.000	0.000
GreeyTaskSelect(GTS)	0.000	0.000	0.000	0.000	0.000	0.000
最大相容任务数	19	24	30	40	46	52

7.7　小　结

前面已经介绍了证明贪心选择性质的方法，即将最优解改造成贪心解，然后证明贪心解不会比最优解差，从而证明贪心解也是最优的。当然，也可以利用循环不变量的方法来证明，这种证明方法将在图算法中介绍。

如果问题具有最优子结构性质和贪心选择性质，贪心法通过逐步地局部贪心选择，能够快速地得到问题的最优解。但是，许多实际问题不具有最优子结构性质或贪心选择性质，因此利用贪心算法常常得不到问题的最优解。然而有趣的是，对有些问题，虽然贪心算法得不到它们的最优解，但是求解速度非常快，而且解的质量也不错，因此贪心算法在工程应用领域受到普遍的重视。

本章内容主要取材于文献[1]，首先介绍了贪心算法的思想，最后通过 4 个经典问题，介绍了贪心算法的设计与分析，在第 8 章中，还将讨论贪心算法的应用，这里就不再介绍。关于贪心算法的理论基础，对实际问题的求解帮助不大，这里就不再介绍，有兴趣的读者可以参考文献[1]。此外，贪心算法是设计近似算法的主要方法之一，有兴趣的读者可以参考文献[4]。

习题

7.1　根据递归方程(7.1)，设计出任务选择问题的动态规划算法。要求算法能计算出 $c[i,j]$ 的大小以及最大相互兼容任务子集 A。

7.2　假设不是总是选择第一个要完成的任务，而是选择一个与之前选入任务兼容的最后一个要开始的任务。试说明这是一个贪心算法，并证明它能得到一个最优解。

7.3　假设有很多教室来对一组活动进行调度。我们希望使用尽可能少的教室来调度所有的活动。请给出一个有效的贪心算法，要求能确定哪一个活动应使用哪一个教室。

7.4 证明背包问题具有最优子结构性质。

7.5 假设在0/1背包问题中，按重量递增所排的物品的次序与按价值递减所排的次序一样，请给出求这种背包问题最优解的有效算法，并分析其正确性。

7.6 试证明一棵不满的二叉树不可能对应一种最优前缀编码。

7.7 如果对字母表中字符按其出现频率的单调递减顺序排序，则存在一个编码长度单调递增的最优编码，试证明上述结论。

7.8 将哈夫曼编码推广至三进制编码（即用符号0,1,2来编码），并证明它能产生最优三进制编码。

7.9 对上一章习题6.18，假定要用最少枚数的硬币来兑换总金额为A的货币，可用硬币的面额是1、5和10，设计一个有效算法，并分析你的结论。

7.10 设有n个客户同时等待一项服务，其中客户i需要的服务时间为t_i（$1 \leqslant i \leqslant n$），应如何安排$n$个客户的服务次序才能使平均等待时间达到最小。其中，平均等待时间是n个客户等待服务时间的总和除以n。

7.11 A驾驶着红旗牌轿车沿着107国道从深圳开往北京。A的车加满油时，可以跑n公里。A有一张地图，标出了沿途中加油站之间的距离。为节省时间，A希望在路途中尽量地少加油。请设计出一个有效算法帮助A决定该在哪些加油站加油，并证明你的算法能够得到最优解。

7.12 假设有两个集合A和B，并且每个集合都包含n个正整数。你可以按自己的意愿将集合重新排序。重新排序后，设集合A的第i个元素为a_i，集合B的第i个元素为b_i，则你可以获得的回报为$\prod_{i=1}^{n} a_i^{b_i}$。请设计出一个能求出最大回报的有效算法。证明算法的正确性，并给出其运行时间。

7.13 假设海岸是一条直线，陆地在海平面的一边有很多小岛（每个小岛可看作海里面的一个点）。在海岸上可布置若干雷达，每个雷达的控制范围为d，即如果小岛离雷达距离不超过d，那么小岛就在雷达的控制范围之内，如图7.11所示。假设x轴表示海岸，海在x轴上面，小岛用坐标(x,y)表示，给定一些小岛的坐标和雷达的控制距离d，求所需要的最少雷达数。（POJ1328）

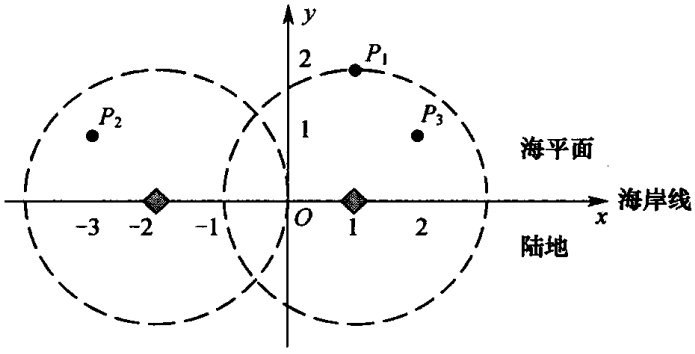


图7.11

7.14　证明定理7.6。

实验题

7.15　将背包问题分别用三种贪心算法实现，用实验分析方法分析哪个贪心算法更有效。

7.16　完成 XOJ 如下题目：1061，1062。

7.17　完成 POJ 如下题目：1017，1018，1042，1065，1083，1089，1230，1328，1659，1716，1744，2751，3069，3687。

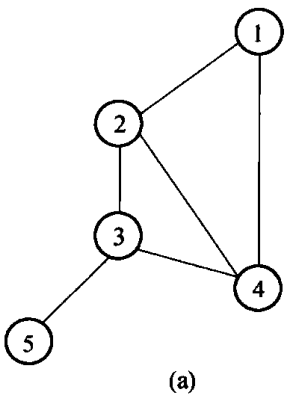
第8章　图算法

科学和工程领域里的许多问题，例如地理信息系统、道路交通网络、计算机网络等都可以形式化一个图问题，因此求解图问题的算法是计算机科学的基础。下面首先介绍图的表示，然后介绍一些基本的图算法，这些图算法是许多复杂图算法的基础。

给定一个图 $G=(V,E)$，其中 V 表示顶点集合，大小记为 $|V|$，E 表示边的集合，大小记为 $|E|$。如果图中的边是有方向的，则称该图为有向图，否则称为无向图。表示一个图 $G=(V,E)$ 通常有两种方式：邻接表和邻接矩阵。邻接表由 $|V|$ 个链表组成，每个顶点 u 有一个链表 $Adj[u]$，它由所有与 u 相邻的顶点 v 构成。假定图 $G=(V,E)$ 的顶点编号为 $1,2,\cdots,|V|$，其邻接矩阵由 $|V|\times|V|$ 的矩阵 $\boldsymbol{A}=(a_{ij})$ 构成，其中

$$a_{ij}=\begin{cases} w(i,j) & \text{if } (i,j)\in E \\ 0 & \text{otherwise} \end{cases}$$

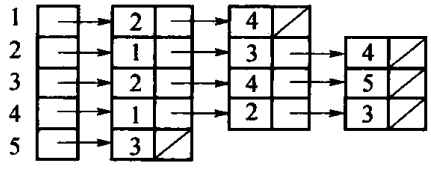
其中 $w(i,j)$ 表示边 (i,j) 的权值或者代价，如果不考虑边的权值，则 $w(i,j)=1$。图 8.1(b)和(c)分别给出了无向图 8.1(a)的邻接矩阵以及邻接表表示。图 8.2(b)和(c)给出了有向图 8.2(a)的邻接矩阵以及邻接表表示。



(a)

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	1	0
3	0	1	0	1	1
4	1	1	1	0	0
5	0	0	1	0	0

(b)



(c)

图 8.1

对于图的这两种表示，通常邻接表适用于描述那些 $|E|$ 远远小于 $|V|^2$ 的稀疏图，而邻接矩阵适用于描述那些 $|E|$ 很接近 $|V|^2$ 的稠密图。

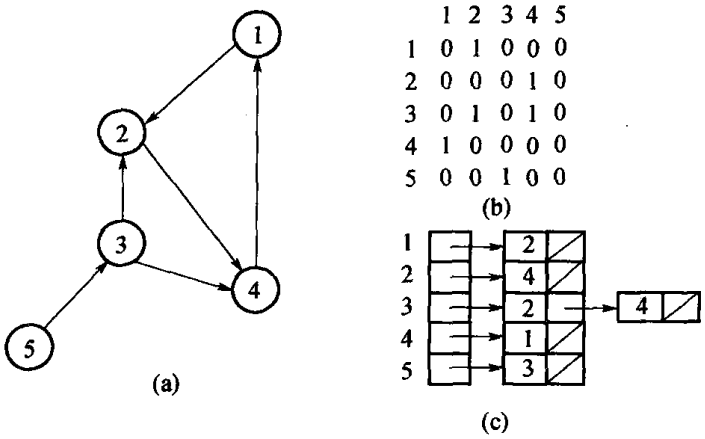


图 8.2

8.1 图的搜索问题

8.1.1 宽度优先搜索

给定图 $G=(V,E)$ 和一个固定的顶点 s，宽度优先搜索(Breadth First Search，BFS)系统地搜索图 G 的边，以找出每个从 s 可以到达的顶点，并且计算从 s 到每个可达顶点的距离(最小的边数)。它同时生成一棵从根 s 到所有可达顶点的宽度优先生成树。其算法可以描述如下：

```
BFS(G,s)
1   for each vertex u ∈ V - {s} do
2           color[u]←White
3           d[u]←∞
4           π[u]←NIL
5   color[s]←Gray
6   d[s]←0;π[s]←NIL;Q←∅
7   Enqueue(Q,s)
8   while Q≠∅ do
9       u←Dequeue(Q)
10      for each v ∈ Adj[u] do
11              if color[v] = White then
12                      color[v]←Gray
13                      d[v]←d[u] + 1
14                      π[v]←u
15                      Enqueue(Q,v)
16      color[u]←DarkGray
```

BFS(G,s)算法中,$color[u]$记录每个顶点 u 的颜色。如果该顶点还没有搜索过,则标记为白色(White),如果该顶点被搜索过,则标记为灰色(Gray),如果该顶点的所有邻接顶点都搜索过,则标记其为深灰色(DarkGray)。$d[u]$表示从顶点 s 到顶点 u 所经历的边数。$\pi[u]$表示搜索到顶点 u 之前所经历的距离 u 最近的顶点,即 u 的父顶点。用队列 Q 记录搜索过的灰色顶点。BFS(G,s)算法第 1 行 ~ 第 6 行进行初始化,初始时刻,队列 Q 里只有出发顶点 s。当 Q 不为空($\varnothing$)时,执行第 9 行 ~ 第 16 行,从队列里取出一个顶点,即 Dequeue(Q),然后对该顶点的每个邻接顶点 v,判断其颜色,如果为白色,则将其标记为灰色,保存到队列 Q 中,即 Enqueue(Q,v),并更新 $d[v]$,$\pi[v]$。当顶点 u 的每个邻接顶点 v 都搜索过后,将 u 标记为深灰色。

图 8.3 给出了 BFS(G,s)算法搜索一个图的过程。图 8.3(a)给出了一个无向图,并已经执行 BFS(G,s)算法第 1 行 ~ 第 6 行。对 s 的每个邻点搜索,并标记为灰色后,其深度为 1。s 的邻点都已经搜索过,s 被标记为深灰色,如图 8.3(b)所示。然后顶点 r 出队列,r 只有一个邻点 v,将其标记为灰色,$d[v]=2$,r 标记为深灰色,如图 8.3(c)所示。然后顶点 w 出队列,w 有两个邻点 t 和 x,将其标记为灰色,$d[t]=2$,$d[x]=2$,然后 w 标记为深灰色,如图 8.3(d)所示。然后顶点 v 出队

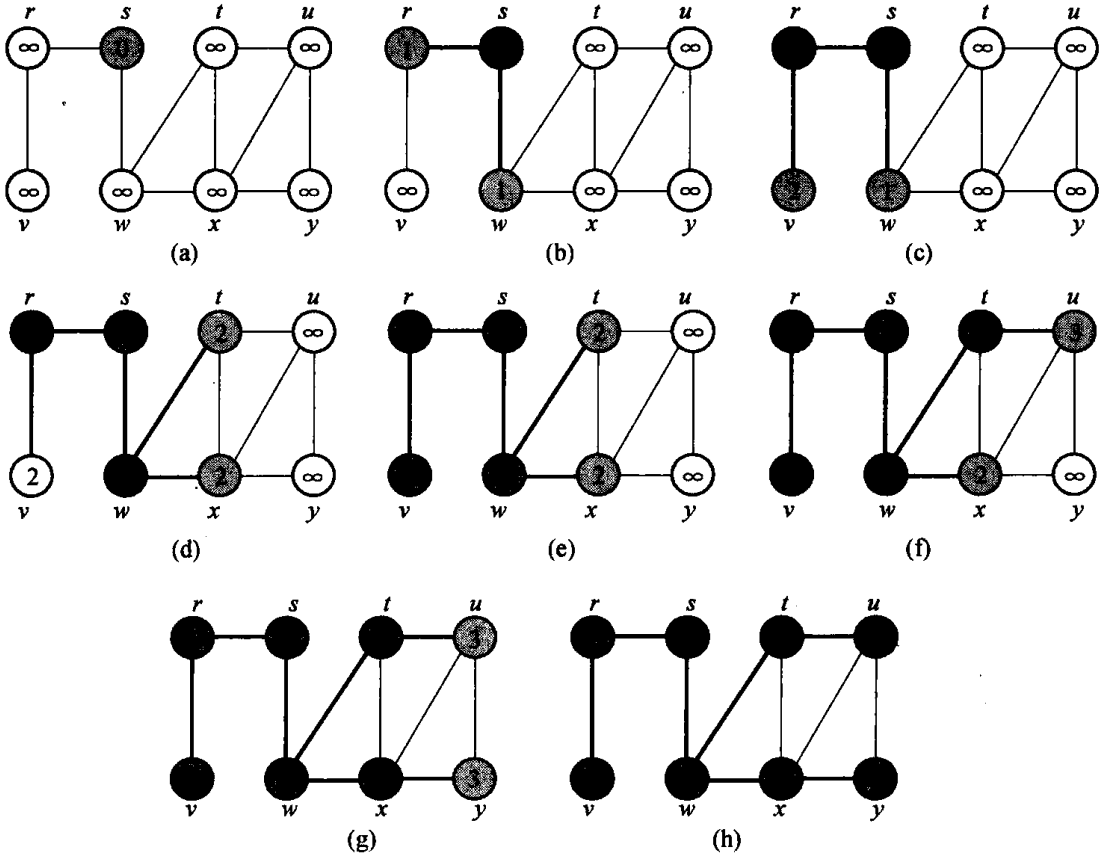


图 8.3

列，v 没有邻点，因此直接标记为深灰色，如图 8.3(e)所示。然后顶点 t 出队列，t 有一个白色邻点 u，将其标记为灰色，$d[u]=3$，如图 8.3(f)所示。然后顶点 x 出队列，x 只有一个白色邻点 y，将其标记为灰色，$d[y]=3$，x 被标记为深灰色，如图 8.3(g)所示。由于 u 和 y 没有邻接点为白色，因此直接标记为深灰色，如图 8.3(h)所示。

为了证明宽度优先搜索算法的正确性，我们先给出最短路径长度的定义。

定义 8.1 最短路径长度 $\delta(s,v)$ 为从 s 到 v 的所有路径中，最短路径所具有的边数。如果从顶点 s 到 v，不存在任何路径，那么 $\delta(s,v)=\infty$。

引理 8.1 设 $G=(V,E)$ 为一个有向或无向图，并且假设 $s\in V$ 为任意一个顶点。那么，对于任意的边 $(u,v)\in E$，有

$$\delta(s,v)\leqslant\delta(s,u)+1$$

证明：可分两种情况证明。如果 u 从 s 可达，则 v 从 s 可达，因此，从 s 到 v 的最短路径的长度不会比从 s 到 u 再经过边 (u,v) 的路径长度更长。如果 u 不能从 s 可达，则 $\delta(s,u)=\infty$。因此引理成立。

引理 8.2 设图 $G=(V,E)$ 是一个有向或无向图，假设从 G 的顶点 $s\in V$ 出发，运行 BFS。那么在算法终止之前，对于每个顶点 $v\in V$，由 BFS 计算得到的 $d[v]$ 值满足 $d[v]\geqslant\delta(s,v)$。

证明：我们用执行 Enqueue 运算的次数 n 进行归纳证明。

当 $n=1$ 时，$d[s]=0=\delta(s,s)$，并对于所有的 $v\in V-\{s\}$，有 $d[v]=\infty\geqslant\delta(s,v)$；

当 $n=k$ 时，考虑 u 的白色邻接顶点 v，假设 $d[u]\geqslant\delta(s,u)$；

当 $n=k+1$ 时，此时 v 着灰色，$d[v]=d[u]+1$，v 进队列，根据假设以及引理 8.1，有 $d[v]=d[u]+1\geqslant\delta(s,u)+1\geqslant\delta(s,v)$。

而且，此时 v 为灰色，它不会再次进队列，因此 $d[v]$ 值不会再改变，故所证成立。

引理 8.3 假设当 BFS 在一个图 $G=(V,E)$ 上执行时，队列 Q 包含如下顶点：$v_1,v_2,\cdots,v_r$，其中 v_1 是队列 Q 的头，并且 v_r 是队列的尾。那么，$d[v_r]\leqslant d[v_1]+1$，并且对于 $i=1,2,\cdots,r-1$，有 $d[v_i]\leqslant d[v_{i+1}]$。

证明：我们用执行队列运算的次数 n 进行归纳的方法来证明该引理在顶点进和出队列后仍然成立。

当 $n=1$ 时，队列中只有一个顶点 s，显然有 $d[s]\leqslant d[s]+1$；

当 $n=k$ 时，假设有 $d[v_r]\leqslant d[v_1]+1$ 且 $d[v_i]\leqslant d[v_{i+1}]$；

当 $n=k+1$ 时，考虑队列的两种运算：

(1) 如果执行出队列运算，队列的头 v_1 出队列，v_2 成为新的队列头。由归纳假设，有 $d[v_r]\leqslant d[v_1]+1$，又由假设 $d[v_1]\leqslant d[v_2]$，因此有 $d[v_r]\leqslant d[v_1]+1\leqslant d[v_2]+1$。

(2) 如果顶点 v 入队列，则它成为 v_{r+1}。对于刚出队列的顶点 u，根据 BFS，有 $d[v_{r+1}]=d[v]=d[u]+1$，根据归纳假设，新的队头 v_1 满足 $d[v_1]\geqslant d[u]$，从而有 $d[v_{r+1}]=d[v]=d[u]+1\leqslant d[v_1]+1$，又由假设 $d[v_r]\leqslant d[u]+1$，可得 $d[v_r]\leqslant d[u]+1=d[v_{r+1}]$。

由此可见，不管执行哪种队列运算，所证均成立。

由引理 8.3，马上可得如下结论。

推论 8.1 假设顶点 v_i 和 v_j 在 BFS 的执行过程中入队列，且 v_i 比 v_j 更先入队列，那么在 v_j 进队列后，有 $d[v_i]\leqslant d[v_j]$。

有了这些准备后，我们现在证明 BFS 的正确性，即 BFS 正确地计算了从 s 可达的顶点 v 的最短路径距离。

定理 8.1 设 $G=(V,E)$ 是一个有向或无向图，并且假设 BFS 运行在图 G 上，从一个给定的顶点 $s\in V$ 出发。那么在执行过程中，BFS 找出每个从 s 可达的顶点 $v\in V$，并且在终止时，对于所有的 $v\in V$，有 $d[v]=\delta(s,v)$。而且，对于任意的从 s 可达的顶点 $v\neq s$，从 s 到 v 的一条最短路径就是从 s 到 $\pi[v]$ 的最短路径，再沿着边 $(\pi[v],v)$ 到 v。

证明：假设某个顶点 v 得到一个不等于其最短路径长度的 d 值，即 $d[v]\neq\delta(s,v)$。显然 $v\neq s$。由引理 8.2 有 $d[v]\geqslant\delta(s,v)$，由假设可得 $d[v]>\delta(s,v)$，并且 s 可达 v。设 u 为 s 到 v 的一条最短路径上紧邻顶点 v 的一个顶点，则 $\delta(s,v)=\delta(s,u)+1$。

因为 $\delta(s,u)<\delta(s,v)$，且考虑到 v 的选取，对于其他顶点 u，有 $d[u]=\delta(s,u)$。根据这些结论，我们得到

$$d[v]>\delta(s,v)=\delta(s,u)+1=d[u]+1 \tag{8.1}$$

现在，我们考虑当 u 出队列时，顶点 v 的颜色变化情况：

如果 v 是白色的，由于 u 紧邻顶点 v，那么由 BFS 算法第 13 行得 $d[v]=d[u]+1$；

如果 v 是深灰色的，那么它已经先于 u 从队列中移走，即 v 比 u 更先入队列，由推论 8.1，有 $d[v]\leqslant d[u]\leqslant d[u]+1$；

如果 v 是灰色的，那么它是在某个顶点 w 出队列时被标记为灰色，其中 w 为从队列 Q 中移出得比 u 更早的一个顶点，且有 $d[v]=d[w]+1$。由推论 8.1，有 $d[w]\leqslant d[u]$，所以有 $d[v]\leqslant d[u]+1$。

综上所述，无论 v 为何种颜色，都会得到与式(8.1)相矛盾的结果。故假设不成立。

接下来，我们证明定理 8.1 的余下结论。如果 $\pi[v]=u$，则有 $d[v]=d[u]+1$。这样从 s 到 v 的一条最短路径经过 s 到 $\pi[v]$ 的最短路径，再沿着边 $(\pi[v],v)$ 到 v。故所证成立。

对于一个包含源点 s 的图 $G=(V,E)$，我们定义图 G 的子图为 $G_\pi=(V_\pi,E_\pi)$，

其中 $V_\pi = \{v \in V : \pi[v] \neq \text{NIL}\} \cup \{s\}$ 且 $E_\pi = \{(\pi[v], v) : v \in V_\pi - \{s\}\}$。当 V_π 由从 s 可达的顶点组成，并且对于所有的 $v \in V_\pi$，在 G_π 中仅有唯一的一条从 s 到 v 的最短路径，同时这也是 G 中从 s 到 v 的一条最短路径，则 G_π 是一棵宽度优先生成树。

现在，我们分析一下 BFS(G,s)算法的运行时间，如果按照一般的最坏情形分析方法，**while** 循环最多执行 $|V|$ 次，**for** 循环最多执行 $|E|$ 次，因此时间复杂度为 $O(|V||E|)$。然而，利用 3.2.1 小节介绍的合计方法进行分析，可以使时间复杂度估计得更准确。事实上，对每个顶点初始化的时间为 $O(|V|)$。由于初始化后，每个顶点被着成白色，BFS 算法在第 11 行保证了只有白色顶点才能进队列，同时该顶点被着成灰色，今后再也不会进队列，而且进出队列各最多一次。队列中最多 $|V|$ 个顶点，对每个顶点，**for** 循环最多执行 $|Adj[v]|$ 次，因此第 8 行 ~ 第 16 行总共执行

$$\sum_{u \in V} |Adj[u]| = O(|E|)$$

综上所述，我们有下列结论。

定理 8.2 给定图 $G = (V,E)$，BFS(G,s)算法的时间复杂度为 $O(|V| + |E|)$。

8.1.2 深度优先搜索

深度优先搜索(Depth First Search，DFS)是一种递归算法，它从一个顶点 v 出发，然后依次对 v 的邻接顶点产生递归调用，直到每个顶点都搜索完为止。深度优先搜索在执行的过程中，对每个顶点 v 做标记，每个顶点 v 有两个标记，一个是 $d[v]$，记录 v 第一次被访问的时间，而第二个标记 $f[v]$ 记录当完成对 v 的邻接表搜索的时间。详细的过程如下：

```
DFS(G)
1   for each vertex u ∈ V do
2           color[u]←White
3           π[u]←NIL
4   time←0
5   for each vertex u ∈ V  do
6           if color[u] = White then DFSVisit(u)
```

DFS(G)算法中，$color[u]$ 以及 $d[u]$ 的定义类似宽度优先搜索，$time$ 表示搜索的时间。DFSVisit(u)是一个递归算法，如果顶点 u 的某个邻接顶点 v 为白色，则产生递归调用。当对顶点 u 的递归调用结束后，将 u 标记为深灰色，并记录 u 的完成时间。DFSVisit(u)的递归过程如下：

```
DFSVisit(u)
1   color[u]←Gray
2   time←time + 1
```

```
3   d[u]←time
4   for each v∈Adj[u] do
5           if color[v] = White then
6                   π[v]←u
7                   DFSVisit(v)
8   color[u]←DarkGray
9   time←time +1
10  f[u]←time
```

下面给出一个例子，如图 8.4(a)所示，对顶点 u 开始搜索，其第一次访问的时间为 $d[u]=1$，如图 8.4(b)所示。然后对顶点 u 的一个邻点 v 进行搜索，其第一次访问的时间为 $d[v]=2$，如图 8.4(c)所示。然后对 v 的一个邻点 y 进行搜索，其第一次访问的时间为 $d[y]=3$，如图 8.4(d)所示。然后对 y 的一个邻点 x 进行搜索，其第一次访问的时间为 $d[x]=4$，如图 8.4(e)所示。由于 x 的邻点都已经搜索过，这样完成对 x 的搜索，其完成搜索的时间为 $f[x]=5$，并标记为深灰色，如图 8.4(f)所示，其中顶点 x 中的符号 4/5，4 表示第一次访问 x 的时间，5 表示完成对顶点 x 搜索的时间。然后回退至顶点 y，由于 y 没有其他邻点为白色，因此完成对 y 的搜索，其完成搜索的时间为 $f[y]=6$，并标记为深灰色，如图 8.4(g)所示。然后回退至顶点 v，由于 v 没有其他邻点为白色，因此完成对 v 的搜索，其完成搜索的时间为 $f[v]=7$，并标记为深灰色，如图 8.4(h)所示。然后回溯至顶点 u，由于 u 没有其他邻点为白色，因此完成对 u 的搜索，其完成搜索的时间为 $f[u]=8$，并标记为深灰色，如图 8.4(i)所示。如果图中还有白色的顶点，则选择一个顶点继续搜索，如图 8.4(j)选择白色顶点 w，类似前面搜索，分别得到图 8.4(k)，(l)和(m)。

类似宽度优先搜索的分析，我们分析一下深度优先搜索算法的运行时间，DFS(G) 中，由于初始化只需要 $O(|V|)$ 次，第 5 行中，对于每个顶点 u，除了调用 DFSVisit(u) 的时间外，只需要执行 $O(|V|)$ 次。对于调用 DFSVisit(u) 的时间，我们可以利用 3.2.1 小节介绍的合计方法进行分析。对每个顶点 u，只有当该顶点为白色时，才调用 DFSVisit(u)，因此 DFSVisit(u) 最多调用一次。在执行 DFSVisit(u) 时，第4行的**for**循环只执行 $|Adj[u]|$ 次，因此，DFSVisit(u) 算法中，第 4 行至第 7 行共执行 $\sum_{u\in V}|Adj[u]|=O(|E|)$ 次。因此，我们有下列结论。

定理 8.3 给定图 $G=(V,E)$，DFS(G)算法的时间复杂度为 $O(|V|+|E|)$。

深度优先搜索还具有许多有趣的性质，例如深度优先搜索的子图 G_π 构成的是一座森林，它由许多深度优先树构成。深度优先搜索完成后，将图 $G=(V,E)$ 的边分成四类：属于深度优先树中的边，常称为树边；不在深度优先树中，但是由树中的顶点指向其父顶点或者是指向顶点本身的边，常称为后向边(B)；不在深度优先树中，但是由树中的顶点指向其子辈顶点的边，常称为前向边(F)；其他的边统称为交叉边(C)，如图 8.4(n)所示。

下面我们介绍一下深度优先搜索的应用。

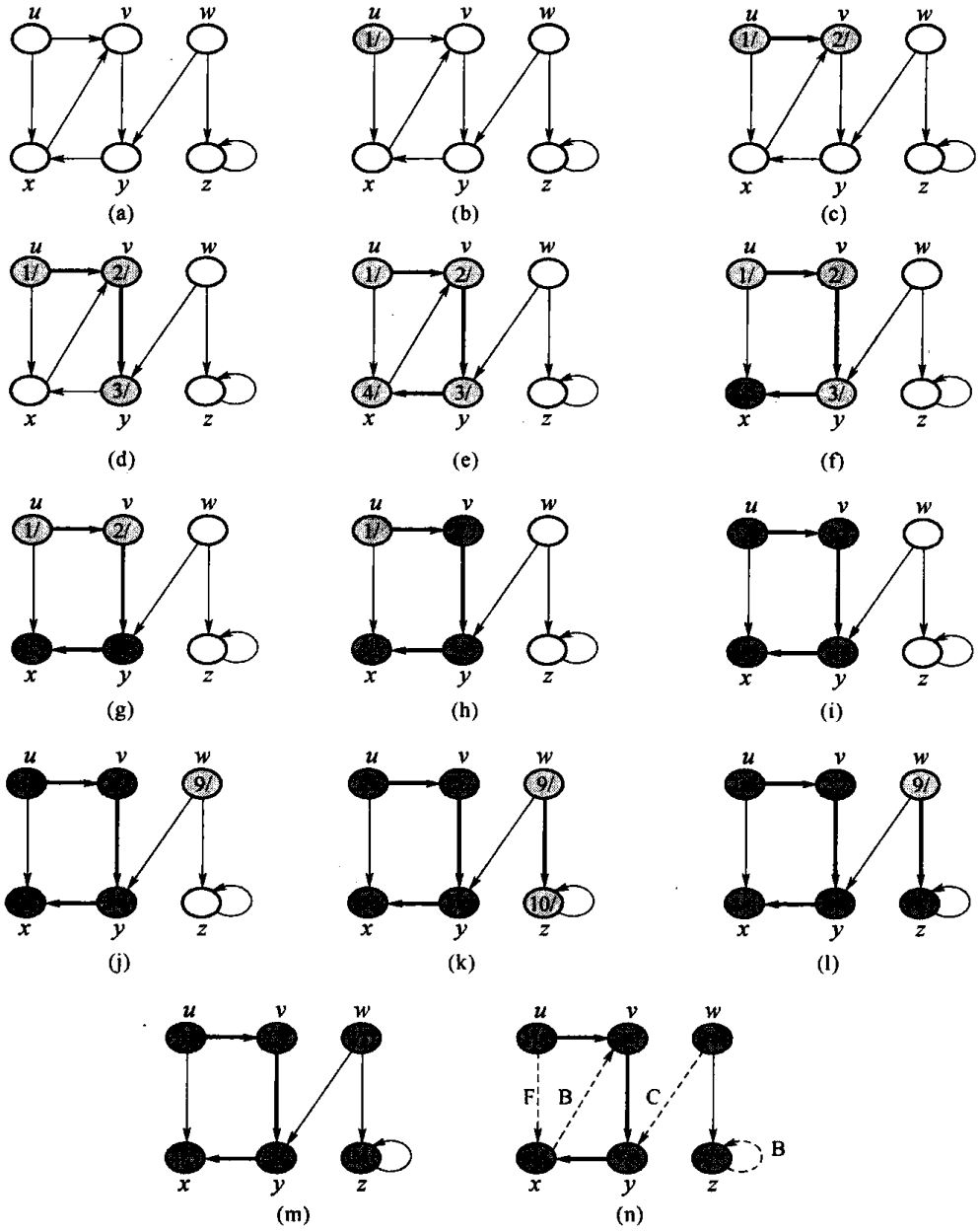


图 8.4

1. *拓扑排序*

给定一个有向非循环图 $G=(V,E)$，图 G 的拓扑排序是所有顶点的一个线性排序，它使得如果图 G 包含边 (u,v)，那么在该顶点序列中，u 一定在 v 的前面。图 G 的拓扑排序能够看作水平方向的一个顶点序列，使得所有的有向边从左指向右。它可以用来表示事件之间的优先关系。图 8.5(a) 给出了一个人起床时，各种事件之间的先后顺序关系，例如短裤指向长裤，表示应该先穿短裤，然后再穿长裤。

拓扑排序利用深度优先搜索算法，具体过程如下：

TopologicalSort(G)

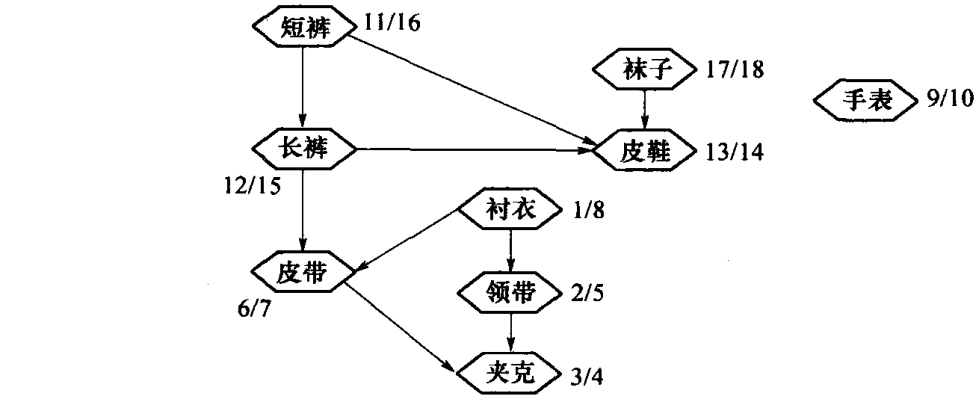
1 call DFS(G) to compute finishing times $f[v]$ for each vertex v

2 as each vertex is finished, insert it into the front of a linked list

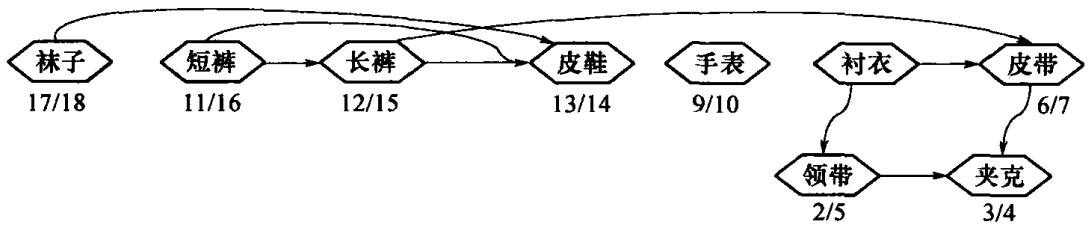
3 **return** the linked list of vertices

TopologicalSort(G)算法首先调用 DFS(G)以便计算出每个顶点 v 的完成时间 $f[v]$,当每个顶点都搜索完后,将它插入到链表(linked list)的最前端,最后返回链表的所有顶点即可。

图 8.5 给出了执行 TopologicalSort(G)的过程,调用 DFS(G)后,得到图 8.5(a),图 8.5(b)给出 TopologicalSort(G)算法的结果。



(a)



(b)

图 8.5

TopologicalSort(G)算法的运行时间主要由 DFS(G)决定,因此其时间复杂度为 $O(|V|+|E|)$。

2. 强连通分支

有向图 $G=(V,E)$ 的一个强连通分支是一个最大顶点集 $C \subseteq V$,使得对于 C 中的每一对顶点 u 和 v,它们都互相可达,即 u 可以到 v,v 也可以到 u。图 8.6 给出了一个例子,图中用圈圈起来的顶点集均构成连通分支。

为了找到图 $G=(V,E)$ 的所有连通分支,需要用到计算图 G 的转置图 $G^T=(V,E^T)$ 的算法,其中 $E^T=\{(v,u):(u,v)\in E\}$。假定图 $G=(V,E)$ 用邻接表表示,则可以在 $O(|V|+|E|)$ 时间内构造出 G^T,见习题 8.3。

找连通分支的算法如下:

StronglyConectedComponents(G)

1 call DFS(G) to compute finishing times $f[u]$ for each vertex u

2 compute G^T

3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $f[u]$

4 output the vertices of each tree in the depth – first forest formed in line 3 as a separate strongly connected component

StronglyConectedComponents(G)算法首先调用 DFS(G),计算出每个顶点的完成时间$f[u]$,然后在第 2 行计算图 G 的转置图 G^T,再调用 DFS(G^T),但在 DFS 的主循环中,按照$f[u]$递减的顺序进行递归调用。第 4 行输出第 3 行中生成的深度优先森林中每棵树的顶点,作为各自独立的强连通分支(strongly connected component)。

图 8.6 给出了一个找强连通分支的例子,图 8.6(a)中,强连通分支用大圈表示,例如 $C=\{a,b,e\}$。图(a)中每个顶点 u 的旁边都标记了 $d[u]/f[u]$,树边用粗箭头表示。图(b)给出了图(a)转置图的深度优先森林。灰色顶点就是深度优先搜索树的树根。输出深度优先森林中每棵树的顶点,便可以得到相应的连通分支,例如对于树 b,a,e,输出其顶点,便可以得到一个连通分支 $C=\{a,b,e\}$。

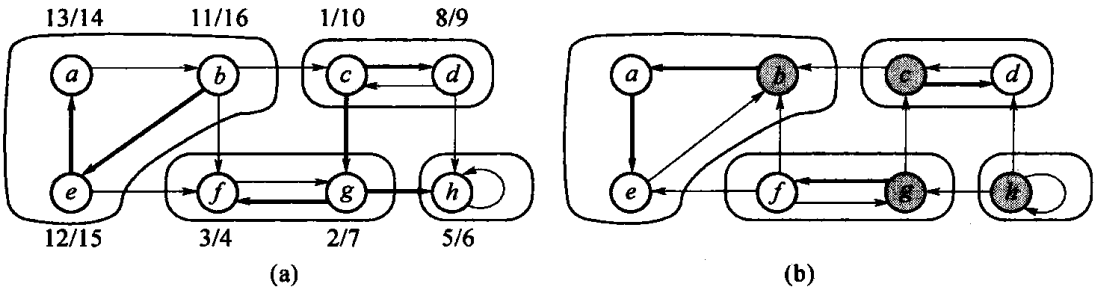


图 8.6

由于 StronglyConectedComponents(G)算法的时间复杂度主要取决于深度优先搜索算法以及找转置图的算法,因此其时间复杂度为 $O(|V|+|E|)$。

8.2 最小生成树问题

最小生成树(Minimum Spanning Trees,MST)问题可以描述如下:

给定一无向连通图 $G=(V,E)$,对每条边$(u,v)\in E$,都有一相应的权值 $w(u,v)$,表示连通 u 与 v 的代价。我们希望找到一个无环子集 $T\subseteq E$,其连通了图 G 所有的顶点且总权值 $\sum_{(u,v)\in T} w(u,v)$ 最小。

下面我们介绍找最小生成树问题的两个贪心算法,即 Kruskal 算法与 Prim 算法。

8.2.1 Kruskal 算法

Kruskal 算法是一个贪心算法，它每一次从剩余的边中选一条权值最小的边，然后加入到一个边的集合 A。其每一步加到 A 中的都是权值最小的边。这条边要么连接两棵树，要么加到一棵树上，要么生成一棵新树。当所有的顶点都连接后停止。详细的算法可以描述如下：

```
KruskalMST(G,w)
1  A←∅
2  for each vertex v ∈ V do
3       MakeSet(v)
4  sort the edges of E into nondecreasing order by weight w
5  for each edge(u,v) ∈ E do
6       if FindSet(u) ≠ FindSet(v) then
7            A←A∪{(u,v)}
8            Union(u,v)
9  return A
```

其中，A 表示最小生成树边的集合，MakeSet(v) 表示构造一个新的集合，其代表性元素为 v。FindSet(u) 返回包含 u 的集合的代表性元素，第 6 行测试顶点 u 和 v 是否属于同一棵树。而树的合并是通过 Union 运算来实现的。上述运算均是关于不相交集合数据结构的基本操作，详细的可以参考文献[1]，这里就不再介绍。KruskalMST(G,w) 算法首先初始化每个顶点为一个集合，然后对图中的边按照权值的大小由小到大进行排序。然后依次选择一条边，如果 FindSet(u) ≠ FindSet(v)，表示顶点 u 和顶点 v 不属于同一棵树，因此把边(u,v) 加入到最小生成树 A 中，然后合并顶点 u 和 v 到同一棵树中。

下面给出一个例子，对于图 8.7(a)，选择权值最小的边(g,f)，然后选择(a,b)，如图 8.7(b)所示。接着选择剩余边中权值最小的边(c,f)，因为 FindSet(c) ≠ FindSet(f)，因此加入到最小生成树中，如图 8.7(c)所示。然后选择剩余边中权值最小的边(a,d)，因为 FindSet(a) ≠ FindSet(d)，因此加入到最小生成树中，如图 8.7(d)所示。然后选择剩余边中权值最小的边(c,g)，因为 FindSet(c) = FindSet(g)，因此该边被抛弃，然后选择剩余边中权值最小的边(e,f)，因为 FindSet(e) ≠ FindSet(f)，因此加入到最小生成树中，如图 8.7(e)所示。然后选择剩余边中权值最小的边(a,e)，因为 FindSet(a) ≠ FindSet(e)，因此加入到最小生成树中，同时将两棵子树合并成一棵最小生成树，如图 8.7(f)所示。此时，所有顶点均连接，算法终止。

下面我们证明 KruskalMST 算法的正确性。

定理 8.4 任给带权连通无向图，KruskalMST 算法都能找到该图的一棵最小生成树。

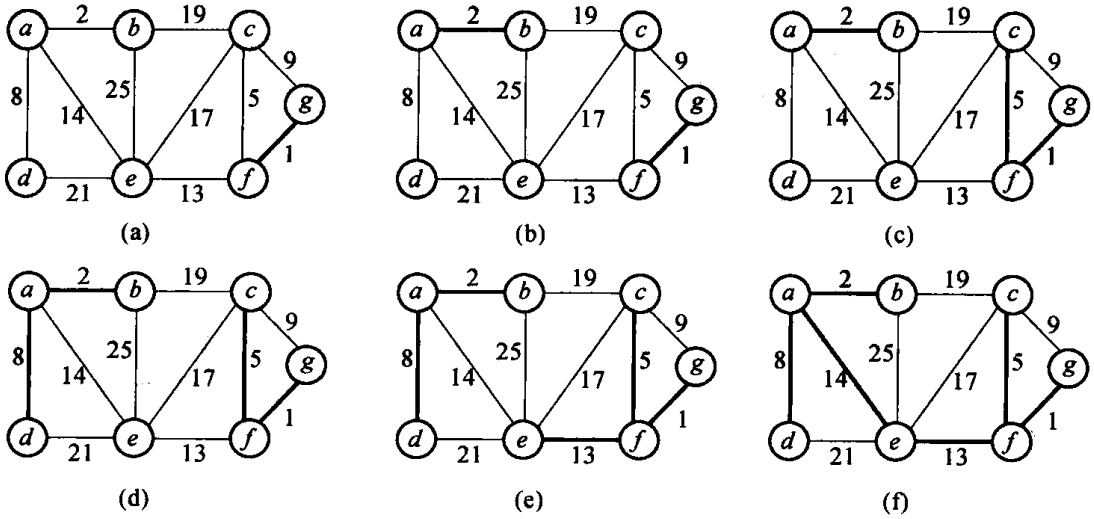


图 8.7

证明：为了证明这个结论，我们只需证明存在下述循环不变量：在每次往集合 A 中增加一条边时，集合 A 均是一棵最小生成树边集的一个子集。令 T 为一棵最小生成树，A^* 是 T 的边集，则只需证明，A 是 A^* 的一个子集即可，即 $A \subseteq A^*$。也就是说每一步贪心选择的边都在最小生成树中。

初始步：第 1 行结束后，集合 $A = \varnothing$，显然是 A^* 的子集，循环不变量为真。

归纳步：假设往 A 中加入边 (u,v) 前，A 是最小生成树边集 A^* 的一个子集，令 $A' = A \cup (u,v)$，我们只需证明 A' 也是 A^* 的一个子集即可。根据归纳假设，$A \subset A^*$。如果 A^* 包含 (u,v)，则结论显然成立。如果 A^* 不包含 (u,v)，由于 T 中顶点是连通的，则 $A^* \cup (u,v)$ 刚好包含一个回路，(u,v) 是其中的一条边，而且是当前选择要加入的边。令 X 是当前包含顶点 u 的子树的顶点集，因为 (u,v) 连接 X 中的一顶点 u 与 $V-X$ 中的另一顶点 v，A^* 必须包含另外一条边 (x,y)，使得 $x \in X$ 且 $y \in V-X$，且该边不在 KruskalMST 算法所得到的生成树中，这样的边一定能够找到，否则 A^* 的边会全部在 KruskalMST 算法所得到的生成树中，从而该生成树有回路，这就产生了矛盾。现在考虑边 (u,v) 和 (x,y) 权值的大小。如果 $w(u,v) > w(x,y)$，为了避免与 A 中的边构成回路，KruskalMST 算法不会选择边 (x,y)，而 $(x,y) \in A^*$，$A \subset A^*$，这样 A^* 中出现回路，这与 A^* 是最小生成树的边集矛盾，故 $w(u,v) > w(x,y)$ 的情形不会出现，只有 $w(u,v) \leqslant w(x,y)$ 这种可能，此时，如果我们构造 $A^{**} = \{A^* - (x,y)\} \cup (u,v)$，那么 A^{**} 也是一棵最小生成树。由于 $A' \subset A^{**}$，因此，A' 也是最小生成树边集的一个子集。

终止步：当 KruskalMST 算法结束时，所有被加到 A 中的边都在最小生成树中，所以第 9 行返回的 A 必为最小生成树。

下面我们分析一下 KruskalMST 算法的时间复杂度。KruskalMST(G,w) 算法的时间复杂度取决于数据结构不相交集运算的实现，MakeSet(v) 运算需要常数时间，

利用等级合并,或者等级合并和路径压缩技术,FindSet(u)和 Union(u,v)运算分别需要时间 O(lg|E|),详细地介绍可参考文献[1]。KruskalMST(G,w)算法第2行~第3行,共执行|V|次 MakeSet(v)运算,第5行共循环|E|次,因此 KruskalMST(G,w)算法第5行~第8行共需要时间 O(|E|lg|E|),因此,我们有如下结论。

定理 8.5 KruskalMST 算法的时间复杂度为 O(|E|lg|E|)。

8.2.2 Prim 算法

Prim 算法的特点是集合 A 中的边总是生成一棵单一的树,这棵树开始于任意一个根顶点 r,然后逐步生长,直到能包含所有 V 中的顶点为止,详细的算法可以描述如下:

```
PrimMST(G,w,r)
1  for each u ∈ V do
2      key[u] ← ∞
3      π[u] ← NIL
4  key[r] ← 0
5  Q ← V
6  while Q ≠ ∅ do
7      u ← ExtractMin(Q)
8      for each v ∈ Adj[u] do
9          if v ∈ Q and w(u,v) < key[v] then
10             π[v] ← u
11             key[v] ← w(u,v)
12             DecreaseKey(Q,v,key)
```

PrimMST 算法首先初始化每个顶点的 *key* 值为∞,并令根顶点 r 的 *key* 值为0,然后按照每个顶点的 *key* 值建立一个最小优先队列 Q。当队列 Q 不为空时,执行 ExtractMin(Q),它从队列里找出具有最小 *key* 值的顶点 u,然后对 u 的每个邻接顶点,依次更新 *key* 值。重复这个过程直到队列为空。当算法结束时,最小生成树的边集 $A=\{(v,\pi[v]):v\in V-\{r\}\}$。

图8.8给出了 PrimMST 算法运行的例子。对图8.8(a)初始化后,得到图8.8(b)。其中符号∞/b 表示顶点 b 的 $key(b)=\infty$。队列 Q 包含图8.8(a)的所有顶点。从队列 Q 中取出 *key* 值最小的顶点 a,接着对 a 的所有邻点更新其 *key* 值,然后从队列 Q 中取出 *key* 值最小的顶点 b,对 b 的所有邻点更新其 *key* 值,此时 $\pi[b]=a$,如图8.8(c)所示。从队列 Q 中取出 *key* 值最小的顶点 d,然后对 d 的所有邻点更新其 *key* 值,$\pi[d]=a$,如图8.8(d)所示。从队列 Q 中取出 *key* 值最小的顶点 e,然后对 e 的所有邻点更新其 *key* 值,$\pi[e]=a$,此时 $key[c]=17$,$key[f]=13$,如图8.8(e)所示。从队列 Q 中取出 *key* 值最小的顶点 f,然后对 f 的所有邻点更新其 *key* 值,此时,$\pi[f]=e$,$key[c]=5$,$key[g]=1$,如图8.8(f)所示。从队列 Q 中取出 *key*

值最小的顶点 g，然后对 g 的所有邻点更新其 *key* 值，$\pi[g]=f$，此时，顶点 c 的 *key* 值不变，如图 8.8(g)所示。此时 Q 中只有顶点 c，出队列，找到最小生成树，如图 8.8(h)所示的粗边。

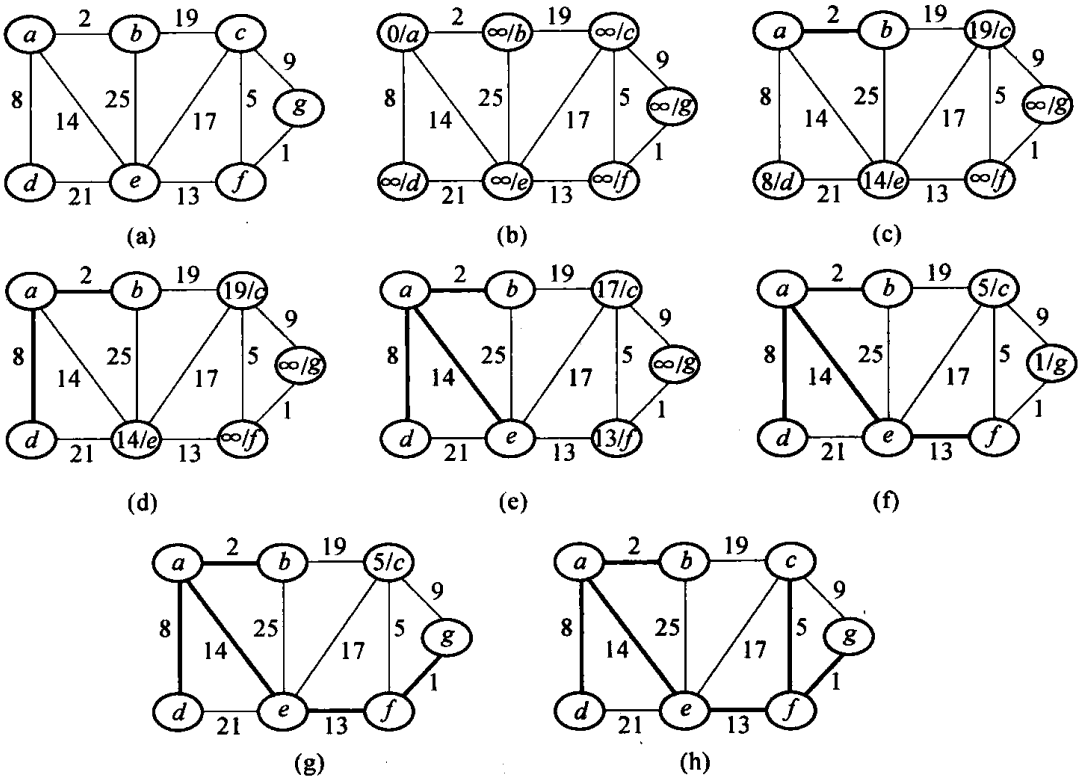


图 8.8

PrimMST 算法的性能取决于最小优先队列是如何实现的，如果用最小堆的数据结构来实现，利用建立最小堆的程序，PrimMST 算法中的第 1 行～第 5 行，可以在 $O(|V|)$时间内完成。第 6 行的 **while** 循环执行 $|V|$次，而每个 ExtractMin(Q)运算需要花费 $O(\lg|V|)$，因此第 7 行总共执行 $O(|V|\lg|V|)$次。由于每个顶点出队列一次，利用 3.2.1 小节介绍的合计方法进行分析。第 8 行的 **for** 循环对每个顶点 u 只执行 $|Adj[u]|$次，因此 PrimMST 算法中，第 8 行 **for** 循环共执行 $\sum_{u\in V}|Adj[u]|=O(|E|)$ 次。由于 DecreaseKey(Q,v,key) 需要时间 $O(\lg|V|)$，因此执行第 12 行共需要时间 $O(|E|\lg|V|)$。综上所述，可得如下结论。

定理 8.6 PrimMST 算法的时间复杂度为 $O(|E|\lg|V|)$。

同样，我们能够证明 PrimMST 算法的正确性，见习题 8.10。

8.3 最短路径问题

最短路径问题可以描述如下：给定一个带权有向图 $G=(V,E)$，图 G 的每条边

(u,v)都有一个实数权值$w(u,v)$，路径$p=\langle v_0,v_1,\cdots,v_k\rangle$的权值是路径$p$中边的总权值，即

$$w(p)=\sum_{i=1}^{k}w(v_{i-1},v_i)$$

从u到v的最短路径权值可以定义如下：

$$\delta(u,v)=\begin{cases}\min\{w(p):u\xrightarrow{p}v\} & \text{if there is one path } p \text{ from } u \text{ to } v\\ \infty & \text{otherwise}\end{cases}$$

因此，从u到v的最短路径是所有从u到v的路径中，具有最小权值$w(p)=\delta(u,v)$的路径p。

对于无向图的不带权的最短路径，可以利用BFS算法来计算，这里就不再介绍。下面我们主要考虑下列最短路径问题：

1）单个源点的最短路径问题

给定图$G=(V,E)$，我们想找出从一个给定的源点$s\in V$到每个顶点$v\in V$的最短路径。

2）两个顶点的最短路径问题

给定顶点u和v，寻找它们之间的最短路径。

3）所有点对的最短路径问题

对所有点对u和v，寻找它们之间的最短路径。

还有很多其他有关最短路径的问题，但是大多数问题可以看作是上述问题的特例或者扩展。在介绍求解这些问题的算法之前，我们先介绍一些有关概念。

定理8.7 给定一个带权有向图$G=(V,E)$，带权意味着对任给的边$(u,v)\in E$，都有一个实数值$w(u,v)$与边(u,v)对应。令从顶点v_1到顶点v_k的最短路径为$p=\langle v_1,v_2,\cdots,v_k\rangle$，对任意的$i$及$j$有$1\leqslant i\leqslant j\leqslant k$，令$p_{ij}=\langle v_i,v_{i+1},\cdots,v_j\rangle$为$p$中从顶点$v_i$到顶点$v_j$的子路径，则$p_{ij}$为从$v_i$到$v_j$的最短路径。

证明：可用反证法来证明。假设p_{ij}不是最短路径，则一定可以找到另外一条从顶点v_i到顶点v_j的子路径p'_{ij}，且有$w(p'_{ij})<w(p_{ij})$，这样，我们可以构造一条从v_1到v_k的路径p'，该路径除了经过路径p'_{ij}，其他与路径p相同，即有

$$\begin{aligned}w(p')&=\delta(v_1,v_i)+w(p'_{ij})+\delta(v_j,v_k)\\&<\delta(v_1,v_i)+w(p_{ij})+\delta(v_j,v_k)=w(p)\end{aligned}$$

这与p已经是最短路径矛盾。故所证成立。

最短路径通常用子图$G_\pi=(V_\pi,E_\pi)$表示，其中$V_\pi=\{v\in V:\pi[v]\neq\text{NIL}\}\cup\{s\}$，$E_\pi=\{(\pi[v],v)\in E:v\in V_\pi-\{s\}\}$。

接下来要介绍的算法，将用到一个很有用的松弛(relax)技术。对每个图中的顶点$v\in V$，我们在算法的每一步计算一个属性值$d[v]$，它是当前迭代步中，从源点s到顶点v的最短路径权值的上界。随着算法的运行，它将不断地被更新。令

$\pi(v)$表示当前路径中顶点 v 的父顶点。初始化每个顶点的属性值的算法如下：

```
InitializeSingleSource(G,s)
1   for each vertex v ∈ V do
2           d[v]←∞
3           π[v]←NIL
4   d[s]←0
```

为了不断地更新 $d[v]$ 值，我们利用 Relax(u,v,w) 过程来松弛一条边。它主要测试通过顶点 u，是否能改进我们到目前为止找到的一条路径，如果能改进，则更新 $d[v]$ 和 $\pi(v)$。一个松弛步可能减少最短路径估计值 $d[v]$ 以及改变 $\pi(v)$ 的值，对一条边进行松弛的过程如下：

```
Relax(u,v,w)
1   if d[v] > d[u] + w(u,v) then
2           d[v]←d[u] + w(u,v)
3           π[v]←u
```

例如图 8.9(a)，考虑边(u,v)，当前顶点 u 的估计值 $d[u]=5$，顶点 v 的估计值 $d[v]=9$，由于 $d[v]>w(u,v)+d[u]=5+2=7$，因此，更新 $d[v]$，得到 $d[v]=7$。图 8.9(b)中，由于 $d[v]\leqslant w(u,v)+d[u]=5+2=7$，因此，执行松弛过程后，$d[v]$ 值不变。

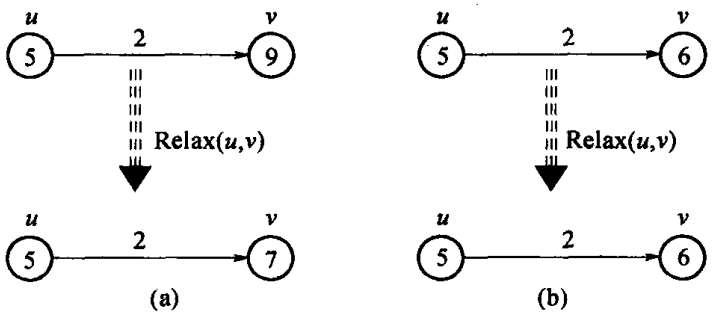


图 8.9

下面我们介绍几个经常用到的有关最短路径及松弛的性质：

1）三角不等式

对于任一条边$(u,v)\in E$，$\delta(s,v)\leqslant\delta(s,u)+w(u,v)$。

2）上界性质

对于所有的顶点 $v\in V$，有 $d[v]\geqslant\delta(s,v)$，并且一旦 $d[v]$ 的值达到 $\delta(s,v)$，它就不会再改变。

3）无路径的性质

若从 s 到 v 没有路径，则 $d[v]=\delta(s,v)=\infty$。

4）收敛性质

对于图 $G=(V,E)$ 中的顶点 $u,v\in V$，若 $s\rightsquigarrow u\rightarrow v$ 为图 G 的一条最短路径，并且

若 $d[u]=\delta(s,u)$，此时执行 Relax(u,v,w)，则有 $d[v]=\delta(s,v)$。

这里不再给出上述性质的证明，详细的证明可参考文献[1]。

8.3.1 单个源点的最短路径问题

1. BellmanFord 算法

给定一个源点为 s 的带权有向图 $G=(V,E)$，带权的含义同前。BellmanFord 算法对图 G 中的每条边进行松弛，最后返回一个布尔值，它表示是否有一条从源点 s 可达的回路，回路的总权值为负，这个回路也称负权回路。如果有一条这样的回路，则算法回答问题没有解；若没有这样的回路，则算法产生所有从源点到可达顶点的路径以及路径的权值。详细的算法如下：

```
BellmanFord(G,w,s)
1  InitializeSingleSource(G,s)
2  for i ←1 to |V| -1 do
3        for each edge (u,v) ∈E  do
4             Relax(u,v,w)
5  for each edge (u,v) ∈E  do
6        if d[v] >d[u] +w(u,v) then return False
7  return True
```

BellmanFord 算法首先初始化每个顶点 v 的估计值 $d[v]$，然后，第 2 行到第 4 行对每个从 s 可达的顶点 v，逐步减少其估计值 $d[v]$，直到 $d[v]=\delta(s,v)$。算法返回 True 当且仅当图 G 不含从 s 可达的负权回路。

由于第 1 行的初始化需要时间 $O(|V|)$，第 2 行迭代 $|V|$ 次，每次松弛 $|E|$ 边，因此算法 BellmanFord 的时间复杂度为 $O(|E||V|)$。

下面我们给出一个例子。给定图 8.10(a)，对图初始化后得到图 8.10(b)，然后对所有边执行一次松弛，得到图 8.10(c)。对所有边执行第二次松弛，得到图 8.10(d)。对所有边执行第三次松弛，得到图 8.10(e)。对所有边执行第四次松弛，得到图 8.10(f)。总共五个顶点，执行四次松弛后结束。由于图 8.10(f)中不含从 s 可达的负权回路，算法返回 True。图 8.10(f)中，每个顶点内的数字，即为从源点 s 到该顶点的最短路径权值。

BellmanFord 算法能够求解带负权边的单个源点的最短路径问题，但是这个算法的时间复杂度比较高，对于有些单个源点的最短路径问题，我们可以利用更有效的算法。

2. Dijkstra 算法

给定一个带权有向图 $G=(V,E)$，并且此图上的权值非负，对于图 G 的单个源点的最短路径问题，我们可以用 Dijkstra 算法来求解。

Dijkstra 算法维持一个集合 S，从源点 s 到集合 S 中每个点的最短路径都已确定；这个算法反复选取 $V-S$ 中具有最短路径估计的点 u，然后把 u 加入 S，并对所

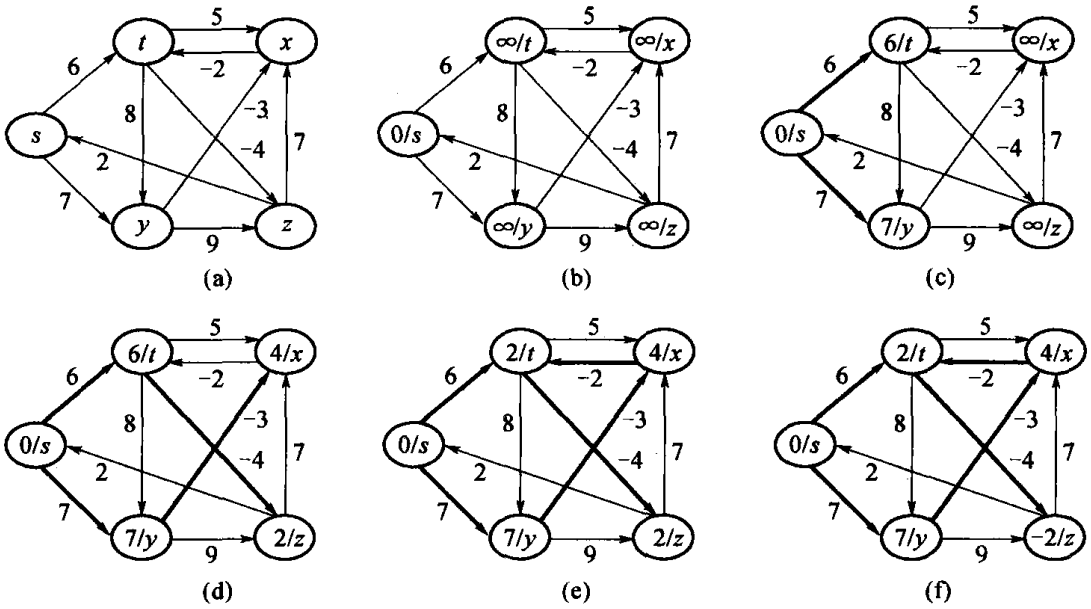


图 8.10

有与 u 相邻的边进行松弛。详细的算法过程如下：

```
Dijkstra(G,w,s)
1  InitializeSingleSource(G,s)
2  S←∅
3  Q←V
4  while Q≠∅ do
5        u←ExtractMin(Q)
6        S←S ∪ {u}
7        for each vertex v∈Adj[u] do
8              Relax(u,v,w)
```

Dijkstra(G,w,s)算法的第 1 行 ~ 第 3 行对图初始化，当队列 $Q\neq\varnothing$ 时，执行 ExtractMin(Q)，取出一个具有最短路径估计的点 u，然后对所有与 u 相邻的边进行松弛。

图 8.11(a)给出了一个例子，对该图初始化，得到图 8.11(b)，此时 $d[s]=0$，s 出队列，并加入集合 S，然后对 s 的邻边进行松弛，得到图 8.11(c)。此时顶点 z 出队列，并加入集合 S，然后对 z 的邻边进行松弛，得到图 8.11(d)。此时顶点 y 出队列，并加入集合 S，然后对 y 的邻边进行松弛，得到图 8.11(e)。此时顶点 t 出队列，并加入集合 S，然后对 t 的邻边进行松弛，得到图 8.11(f)。此时队列 Q 中只有一个顶点 x，出队列并加入集合 S，此时队列为空，算法结束。

定理 8.8 对带非负权函数 w，源点为 s 的有向图 $G=(V,E)$ 运用 Dijkstra 算法，算法终止时，对所有 $u\in V$ 有 $d[u]=\delta(s,u)$。

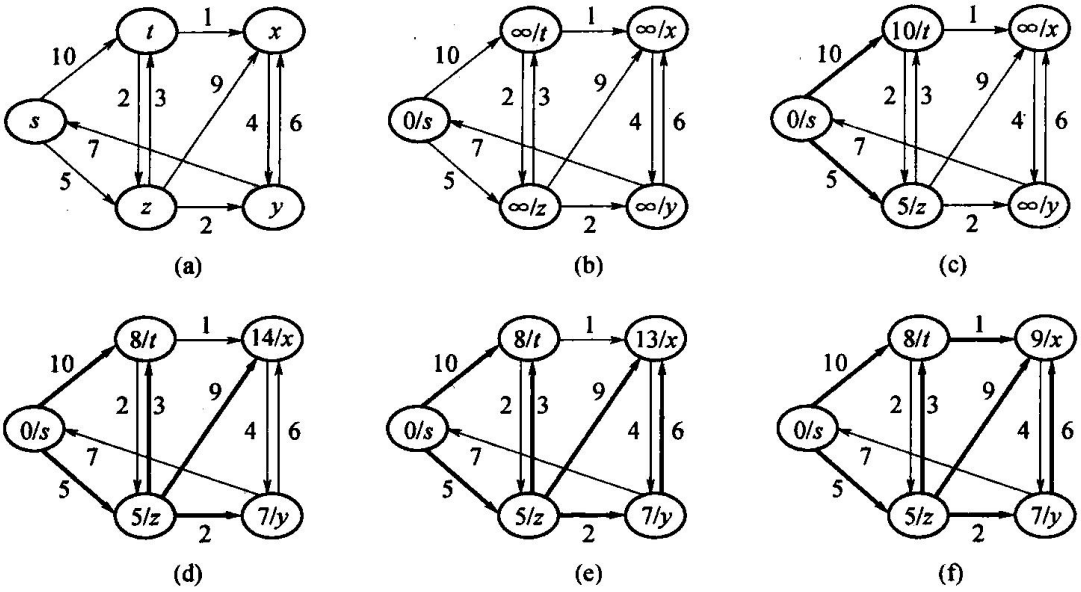


图 8.11

证明:考虑循环不变量。

在第 4 行 ~ 第 8 行 **while** 循环的每一次迭代开始之前,对任意的 $v \in S$,有 $d[v] = \delta(s,v)$。

初始步:当 **while** 循环开始执行之前,$S = \varnothing$,因此循环不变量为真。

归纳步:我们希望在每一次迭代中,对于加入集合 S 的顶点 u 有 $d[u] = \delta(s,u)$。为了证明此结论,我们可以利用反证法。为了导出矛盾,令 u 为迭代过程中加入集合 S 的顶点中第一个满足 $d[u] \neq \delta(s,u)$ 的顶点。

从 s 到 u 肯定存在最短路径 p,如果没有路径的话,则有 $d[u] = \delta(s,u) = \infty$,与假设矛盾。在把 u 加入 S 之前,路径 p 连接一个顶点 $s \in S$,到另一个集合 $V - S$ 中的点 u。如果路径 p 除了 u 之外,不含 $V - S$ 中其他顶点,则由收敛性质,可得 $d[u] = \delta(s,u)$,因此不妨设沿着从 s 到 u 的路径 p 中,第一个属于 $V - S$ 的顶点为 y,并设 $x \in S$ 为路径 p 中顶点 y 的父顶点。如图 8.12 所示。

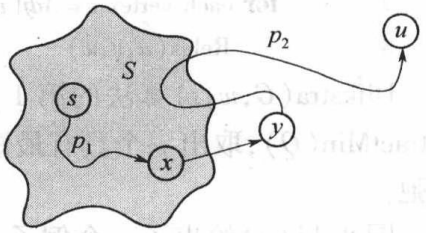


图 8.12

当 u 被加入到 S 时,由于 $d[x] = \delta(s,x)$,而 y 是路径 p 中紧挨着顶点 x 的点,根据最短路径的收敛性质,我们有 $d[y] = \delta(s,y)$。

现在,为了得到一个与假设 $d[u] \neq \delta(s,u)$ 矛盾的结果,我们只需证明 $d[u] = \delta(s,u)$。因为在从 s 到 u 的最短路径中,y 在 u 之前出现,并且所有边非负,我们有 $\delta(s,y) \leqslant \delta(s,u)$,并且根据最短路径的上界性质 $\delta(s,u) \leqslant d[u]$,我们有

$$d[y] = \delta(s,y) \leqslant \delta(s,u) \leqslant d[u]$$

但是由于当 u 被选中时(算法第5行),u 和 y 都属于 $V-S$,故有

$$d[u] \leqslant d[y]$$

根据上面两式,我们有

$$d[y] = \delta(s,y) = \delta(s,u) = d[u]$$

即 $d[u]=\delta(s,u)$,这与我们选择顶点 u 使得 $d[u]\neq\delta(s,u)$ 矛盾。因此我们断定在 u 被加入 S 后,肯定有 $d[u]=\delta(s,u)$,并且在之后的每一次迭代中都保持成立。

终止步:当 Dijkstra 算法终止时,队列 $Q=\varnothing$ 与循环开始前 $Q=V-S$ 一起,暗示着 $S=V$。因此,对所有点 $u\in V$ 有 $d[u]=\delta(s,u)$。

Dijkstra 算法的时间复杂度为 $O(|V|^2)$,可利用合计方法类似 PrimMST 算法进行分析,请参考习题 8.19 的解答。

8.3.2 所有点对的最短路径问题

我们可以运用 $|V|$ 次单个源点的最短路径问题的算法解决所有点对的最短路径问题,即以每一个顶点作为源点,调用一次单个源点的最短路径问题的求解算法,但是这样做,其时间复杂度为原来单个源点的最短路径问题算法时间复杂度的 $|V|$ 倍。

我们能得到效率更好的算法吗?这一节将给出一个复杂度更低的算法。与单个源点的最短路径问题算法用邻接表表示图的方法不同,下面我们用邻接矩阵来表示给定的图。

为方便起见,我们将顶点编号为 $1,2,\cdots,|V|$。令图的输入 $\boldsymbol{W}$ 为 $|V|\times|V|$ 的权值矩阵,代表有 $|V|$ 个顶点的有向图 $G=(V,E)$ 中每条边的权值,即 $\boldsymbol{W}=(w_{ij})$,其中

$$w_{ij}=\begin{cases}0 & \text{if } i=j\\ w(i,j) & \text{if } i\neq j \text{ and } (i,j)\in E\\ \infty & \text{if } i\neq j \text{ and } (i,j)\notin E\end{cases}$$

权值为负的边是允许的,但是我们假定图中没有负权的回路。

前面我们已经证明最短路径问题具有最优子结构性质,因此可以利用动态规划算法来求解所有点对的最短路径问题。

1. *类似矩阵乘法的动态规划求解算法*

考虑从顶点 i 到 j 的最短路径 p,并且假设路径 p 最多包括 m 条边。由于假设没有负权的回路,且 m 是有限数,若 $i=j$,那么路径 p 上没有边,其权和为0。若 i 与 j 不同,则我们把 p 分解成从 i 到 k 的路径 p' 和边 (k,j),如图 8.13 所示。其中路径 p' 至多包含 $m-1$ 条边,由前面的定理 8.7,p' 是从 i 到 k 的最短路径,且有

$$\delta(i,j) = \delta(i,k) + w_{kj}$$

图 8.13

令 $l_{ij}^{(m)}$ 为至多包含 m 条边的任意一条从 i 到 j 的路径 p 的最小权和。当 $m=0$ 时，有一条从 i 到 j 的不包含边的最短路径，当且仅当 $i=j$。因此

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } i \neq j \end{cases} \tag{8.2}$$

对于 $m \geqslant 1$，我们可以检查所有与顶点 j 相邻的顶点 k，然后选取具有最小权值的一条路径即可，即

$$l_{ij}^{(m)} = \min_{1 \leqslant k \leqslant |V|} \{ l_{ik}^{(m-1)} + w_{kj} \} \tag{8.3}$$

若图不包含负权边的回路，那么对每一满足 $\delta(i,j) < \infty$ 的点对 i 和 j，有包含至多 $|V|-1$ 条边的简单路径。一条多于 $|V|-1$ 条边的从 i 到 j 的路径不可能比最短路径 $\delta(i,j) = l_{ij}^{(|V|-1)}$ 的权值更小，即有 $\delta(i,j) = l_{ij}^{(|V|-1)} = l_{ij}^{(|V|)} = l_{ij}^{(|V|+1)} = \cdots$。令 $L^{(m)} = (l_{ij}^{(m)})$，我们可以依次计算 $L^{(1)}, L^{(2)}, \cdots, L^{(|V|-1)}$，最终的 $L^{(|V|-1)}$ 就是所求，它包含了所有点对之间最短路径的权值。

关于这个算法的伪代码及时间复杂度的分析可参考习题 8.23 的解答。事实上，这个算法的时间复杂度比较高，下面我们介绍一个更有效的动态规划算法。

2. FloydWarshall 算法

FloydWarshall 算法考虑一条最短路径的“中间”顶点，其中一条简单路径 p：$v_1, v_2, \cdots, v_l$ 的中间顶点是 p 中除了 v_1 及 v_l 之外的任意顶点，也就是集合 $\{v_2, \cdots, v_{l-1}\}$ 中的任意顶点。

前面我们已经假定对 G 的顶点进行编号，即 $V = \{1, 2, \cdots, |V|\}$，对某个给定的 k，考虑顶点子集 $\{1, 2, \cdots, k\}$。对任意的一对顶点 $i, j \in V$，考虑所有从 i 到 j 的路径，它的中间顶点为集合 $\{1, 2, \cdots, k\}$ 中的顶点。令 p 为其中的一条具有最小权值的路径，这里路径 p 为简单路径。

FloydWarshall 算法利用路径 p 与中间顶点取自 $\{1, 2, \cdots, k-1\}$ 的从 i 到 j 的最短路径之间的关系，这个关系取决于 k 是否为路径 p 上的中间顶点。

(1) 若 k 不是路径 p 上的中间顶点，则所有路径 p 的中间顶点都在集合 $\{1, 2, \cdots, k-1\}$ 中。这样，一条所有中间顶点都在集合 $\{1, 2, \cdots, k-1\}$ 的从顶点 i 到顶点 j 的最短路径，也是一条中间顶点取自 $\{1, 2, \cdots, k\}$ 的从 i 到 j 的最短路径。

(2) 若 k 是最短路径 p 上的中间顶点，其中 p 的中间顶点为集合 $\{1, 2, \cdots, k\}$ 中的顶点，那么我们把 p 分解成两条路径 p_1 和 p_2，如图 8.14 所示。则 p_1 为从 i 到 k 的最短路径，它的中间顶点为集合 $\{1, 2, \cdots, k-1\}$ 中的顶点，p_2 为从 k 到 j 的最短路径，它的中间顶点为集合 $\{1, 2, \cdots, k-1\}$ 中的顶点。

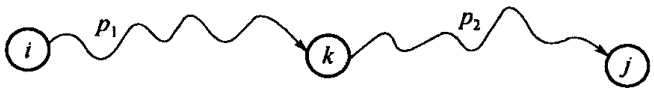


图 8.14

令 $d_{ij}^{(k)}$ 表示从 i 到 j，并且中间顶点在 $\{1,2,\cdots,k\}$ 中的最短路径的权值。当 $k=0$ 时，一条从 i 到 j 的路径没有编号大于 0 的中间顶点，这样的路径至多包含一条边，因此 $d_{ij}^{(0)}=w_{ij}$。当 $k\geqslant 1$，k 要么是路径 p 上的中间顶点，要么不是，因此有 $d_{ij}^{(k)}=\min\{d_{ij}^{(k-1)},d_{ik}^{(k-1)}+d_{kj}^{(k-1)}\}$。根据上面的分析，可得到递归方程

$$d_{ij}^{(k)}=\begin{cases}w_{ij} & \text{if } k=0\\ \min\{d_{ij}^{(k-1)},d_{ik}^{(k-1)}+d_{kj}^{(k-1)}\} & \text{if } k\geqslant 1\end{cases}$$

对于图中任意一条路径，其中间顶点均在 $\{1,2,\cdots,|V|\}$，令矩阵 $D^{(k)}=(d_{ij}^{(k)})$，则对任意点对 $i,j\in V$，$d_{ij}^{(|V|)}=\delta(i,j)$ 即为所求。

根据上述递归方程，我们可以得到求解所有点对最短路径问题的动态规划算法：

```
FloydWarshall(W)
1  D^(0) ← W
2  for k ← 1 to |V| do
3        for i ← 1 to |V| do
4              for j ← 1 to |V| do
5                    if d_ij^(k-1) < d_ik^(k-1) + d_kj^(k-1) then d_ij^(k) ← d_ij^(k-1)
6                    else     d_ij^(k) ← d_ik^(k-1) + d_kj^(k-1)
7  return D^(|V|)
```

FloydWarshall(W) 从 $D^{(0)}$ 开始，以至底向上的方式计算出 $D^{(|V|)}$，其时间复杂度为 $O(|V|^3)$。

为了构造出最短路径，我们可以用类似计算 $D^{(k)}$ 的方式来计算。令 $\Pi^{(k)}=(\pi_{ij}^{(k)})$，其中 $\pi_{ij}^{(k)}$ 表示从 i 到 j 的最短路径 p 上，顶点 j 的父顶点，其中 p 的中间顶点在 $\{1,2,\cdots,k\}$ 中。下面我们递归定义 $\pi_{ij}^{(k)}$。当 $k=0$，从 i 到 j 的最短路径上，根本没有中间顶点，因此

$$\pi_{ij}^{(0)}=\begin{cases}\text{NIL} & \text{if } i=j \text{ or } w_{ij}=\infty\\ i & \text{if } i\neq j \text{ and } w_{ij}<\infty\end{cases}$$

当 $k\geqslant 1$，如果从 i 到 j 的最短路径 p 含有顶点 k，则 p 中顶点 j 的父顶点与从 k 到 j 的最短路径中顶点 j 的父顶点相同，否则 p 中顶点 j 的父顶点与从 i 到 j 的最短路径 p' 中顶点 j 的父顶点相同，其中 p' 中的中间顶点为 $\{1,2,\cdots,k-1\}$ 中的顶点。即

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{if } d_{ij}^{(k-1)} \leqslant d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

将上述求父顶点的递归方程结合到 Floyd-Warshall(W)算法中,就可以构造出最优解,见习题 8.24。

给定图 8.15,下面给出了 FloydWarshall(W)算法求解的过程。

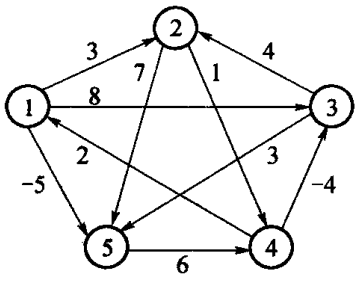


图 8.15

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & 3 \\ 2 & \infty & -4 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & 3 \\ 2 & 5 & -4 & 0 & -3 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 3 \\ 2 & 5 & -4 & 0 & -3 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 3 \\ 2 & 0 & -4 & 0 & -3 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & 0 & 4 & -5 \\ 3 & 0 & -3 & 1 & -2 \\ 7 & 4 & 0 & 5 & 2 \\ 2 & 0 & -4 & 0 & -3 \\ 8 & 6 & 2 & 6 & 0 \end{pmatrix} \quad D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 1 & -5 \\ 3 & 0 & -3 & 1 & -2 \\ 7 & 4 & 0 & 5 & 2 \\ 2 & 0 & -4 & 0 & -3 \\ 8 & 6 & 2 & 6 & 0 \end{pmatrix}$$

FloydWarshall(W)是一个非常有效的算法,Robert W. Floyd 因为在这个算法上的贡献而获得 1978 年的图灵奖。

8.4 小 结

图算法是计算机科学领域最基本的内容之一,本章内容主要取材于文献[1],介绍了图的遍历算法及其应用、最小生成树算法以及最短路径算法。对于最小生成树算法,文献[8]给出了当前渐近最快的随机算法,算法的期望运行时间为 $O(|E|)$。对于 Dijkstra 最短路径算法,如果采用 Fibonacci 堆,则可以使时间复杂度降为 $O(|E| + |V|\lg|V|)$,对于一些特殊图,还可以获得更有效的算法,参见文献[9]。对于稀疏图的所有点对的最短路径算法,有兴趣的读者可以参考文献[1]提及的 Johnson 算法,这里就不再介绍。对图算法感兴趣的读者,还可以参考文献[9,10]。

习题

8.1 完成以下练习：

(1) 对图 8.3(a)从顶点 y 开始，类似图 8.3 产生一棵宽度优先生成树。

(2) 对图 8.4(a)从顶点 y 开始，类似图 8.4 产生一棵深度优先生成树。

8.2 重写 DFS 算法，用数据结构栈来消除递归。

8.3 有向图 $G=(V,E)$ 的转置是图 $G^T=(V,E^T)$，其中 $E^T=\{(v,u)\in V\times V:(u,v)\in E\}$，即 G^T 就是 G 中所有边反向所构成的图。按邻接表和邻接矩阵两种形式分别设计一个由 G 计算 G^T 的有效算法，并分析算法的时间复杂度。

8.4 给出一个反例来说明下列猜想不成立：在有向图 G 中如果存在一条从 u 到 v 的路径，且对 G 进行深度优先搜索时，有 $d[u]<d[v]$，则在搜索产生的深度优先森林中，v 是 u 的子顶点。

8.5 给出一个算法来确定一给定无向图 $G=(V,E)$ 中是否包含回路，要求所设计算法的时间复杂度为 $O(|V|)$，与 $|E|$ 无关。

8.6 设计一个时间复杂度为 $O(|V|+|E|)$ 的算法来计算有向图 $G=(V,E)$ 的连通分支，并且保证所给出算法计算出的连通分支中的每两个顶点间至多只存在一条边。

8.7 给定一个无向图 $G=(V,E)$，它的传递闭包是一个 0/1 数组 T，当且仅当 G 中存在一条边数大于 1 的从 i 到 j 的路径时，$T[i,j]=1$。试设计一个计算 G 的传递闭包矩阵的算法，且使其时间复杂度为 $O(|V|^2)$。

8.8 假定图 $G=(V,E)$ 用邻接矩阵表示，试设计一个运行时间为 $O(|V|^2)$ 的 Prim 算法。

8.9 给定一个图 $G=(V,E)$ 及其一棵最小生成树，假定我们减少图中不属于最小生成树的一条边的权值，试设计一个算法，找出修改边后新图的一棵最小生成树。

8.10 证明 PrimMST 算法的正确性。

8.11 是否存在这样的图，对这些图，PrimMST 算法要慢于 KruskalMST 算法？

8.12 给出一个图 G，G 中边的权值可以为负数，对于顶点 i 和 j，从顶点 i 到顶点 j 没有最短路径，但从顶点 i 到顶点 j 仍然存在路径。

8.13 修改 BellmanFord 算法，使得对任意顶点 v，当从源点到 v 的某些路径上存在一个负权回路时，则设置 $d[v]=-\infty$。

8.14 试设计一个有效算法来统计有向无回路图中的全部路径数，并分析该算法。

8.15 令顶点 z 为源点，对图 8.10(a)所示的有向图运行 BellmanFord 算法。在每趟运行中，按图中的顺序对边进行松弛，显示每一趟运行后的 d 值和 π 值。现在把边 (z,x) 的权值改为 4，并把 z 作为源点再运行该算法的结果。

8.16 给定一个不含负权回路的有向带权图 $G=(V,E)$，对于所有顶点对 $u,v\in V$，

设 m 是所有从 u 到 v 的最短路径上所包含最小边数的最大值。这里最短路径是按权和,而不是按包含的边数来定义的。试对 BellmanFord 算法作简单修改,使其可在 $m+1$ 趟运算中完成计算。

8.17 令顶点 s 和顶点 y 分别为源点,对图 8.10(a)所示的有向图运行 Dijkstra 算法,类似图 8.11 所示的方式,给出 **while** 循环每次迭代后的 d 和 π 值以及集合 S 中的顶点。

8.18 给出一个带负权边的有向图的简单实例,说明 Dijkstra 算法计算该例子会产生错误的结果。如果允许图中边的权为负,说明定理 8.8 的证明不能成立的原因。

8.19 利用合计方法分析 Dijkstra 算法的时间复杂度。

8.20 假设将 Dijkstra 算法的第 4 行改为

4 **while** $|Q|>1$

这一修改使得 **while** 循环执行 $|V|-1$ 次而不是 $|V|$ 次,算法是否仍然正确?

8.21 给定一有向图 $G=(V,E)$,令其每条边 $(u,v)\in E$ 对应一实数值 $r(u,v)\in[0,1]$,表示从顶点 u 到顶点 v 的通信线路的可靠性,即 $r(u,v)$ 表示从 u 到 v 的线路不中断的概率,并假定这些概率是相互独立的。试设计一个找出两指定顶点间最可靠线路的有效算法。

8.22 设 $G=(V,E)$ 为一带权有向图,其权函数 $w:E\rightarrow\{0,1,\cdots,W\}$,其中 W 为某非负整数。修改 Dijkstra 算法,使其能够在 $O(W|V|+|E|)$ 时间内计算出给定源点 s 的最短路径。

8.23 根据递归方程(8.2)和(8.3),写出动态规划算法的伪代码,并分析其时间复杂度。

8.24 修改 FloydWarshall(W)算法,以便构造出最优解。

8.25 给定图 8.16,运行 FloydWarshall 算法,并写出外层循环中每次迭代所生成的矩阵 $D^{(k)}$。

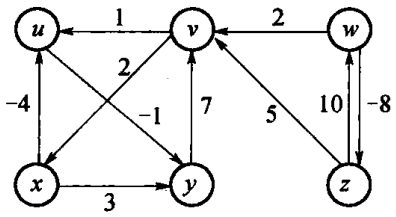


图 8.16

8.26 由于要计算 $d_{ij}^{(k)}$,$i,j,k=1,2,\cdots,|V|$,所以 FloydWarshall 算法的空间要求为 $O(|V|^3)$。给出仅仅去掉所有上标所得的算法:

```
FloydWarshall(W)
1  D←W
2  for k←1 to |V| do
3       for i←1 to |V| do
4            for j←1 to |V| do
5                 if d_ij < d_ik + d_kj then d_ij←d_ij
6                 else    d_ij←d_ik + d_kj
7  return D
```

证明该算法是正确的,而且所需空间仅为 $O(|V|^2)$。

8.27 如何利用 FloydWarshall 算法的输出来检测负权回路的存在?

8.28 令 $\phi_{ij}^{(k)}$ 是从 i 到 j 的最短路径中具有最高编号的中间顶点,其中 $i,j,k=1,2,\cdots,|V|$,在 FloydWarshall 算法中,利用 $\phi_{ij}^{(k)}$ 可以得到另一种构造最短路径的方法。写出 $\phi_{ij}^{(k)}$ 的递归方程,修改 FloydWarshall 以计算 $\phi_{ij}^{(k)}$,并用矩阵 $\boldsymbol{\Phi}=(\phi_{ij}^{(|V|)})$ 为输入构造最短路径的算法。矩阵 $\boldsymbol{\Phi}$ 与 6.3 节的矩阵链乘法问题中的 s 表有何相似之处?

实验题

8.29 编程实现求解最短路径问题的类似矩阵乘法的动态规划算法和 FloydWarshall 算法,并用实验分析方法比较两种动态规划算法。

8.30 完成 XOJ 如下题目:1073,1074,1075,1076,1077,1078,1040,1043,1049,1050,1063。

8.31 完成 POJ 如下题目:1062,1094,1125,1158,1161,1178,1181,1364,1639,1665,1679,1860,2394,2421,2553,2728,3009,3026,3259,3660。

第 9 章　网络流与匹配

9.1　最大流问题

我们可以把道路交通图转化为有向图，从而得到从一点到另一点的最短路径问题。我们也可以把有向图转化为网络流，从而解决某些有关物质流的实际问题，例如流经管子的液体流、装配线上的零件流、流过电路的电流、互联网上的信息流、运输网络的物品流，等等。因此，网络流是科学和工程领域中的一个非常重要的问题。

下面我们先给出网络流(Network Flow)的概念。

定义 9.1　给定有向图 $G=(V,E)$，对于图 G 的每一条边 $(u,v)\in E$，都有一个非负的容量 $c(u,v)\geqslant 0$，如果 $(u,v)\notin E$，则 $c(u,v)=0$。并且图中有两个特殊的顶点 s,t，其中顶点 s 只有出边，t 只有入边，并且图 G 中至少存在一条从 s 到 t 的路径。这样的有向图我们称为网络，其中顶点 s 称为网络的源点，t 称为网络的汇点。

图 9.1(a)就是一个网络的例子，它是实际的输油管道图 9.1(b)的抽象表示，其中图 9.1(a)中边上的数值表示该边的容量，例如 $c(v_1,v_2)=0$，$c(v_1,v_3)=3$。图 9.1(b)中输油管道的大小，对应管道容量的大小。

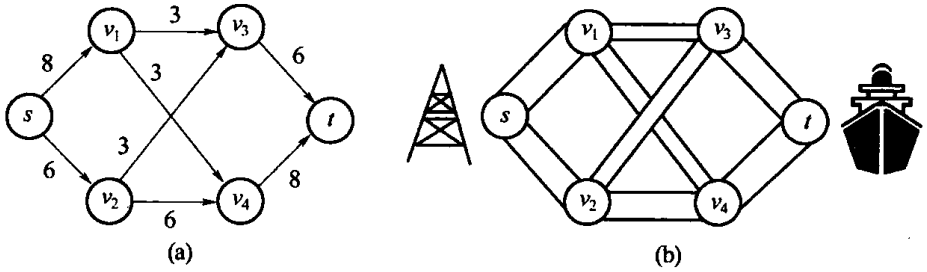


图 9.1

定义 9.2　给定一个网络 $G=(V,E)$，如果一个实值函数 $f:V\times V\to \boldsymbol{R}$ 满足下列三个性质：

(1) 容量约束性质，即任取 $u,v\in V$，均有 $f(u,v)\leqslant c(u,v)$；

(2) 反对称性质，即任取 $u,v\in V$，均有 $f(u,v)=-f(v,u)$；

(3) 流守恒性质，即任取 $u\in V-\{s,t\}$，均有 $\sum\limits_{v\in V}f(u,v)=0$。

则称 f 为网络 G 的流，其中 $f(u,v)$ 表示从顶点 u 到顶点 v 的流量。流 f 的大小可如

下定义

$$|f| = \sum_{v \in V} f(s,v)$$

$|f|$表示了从源点 s 流出的流量大小。流为 f 的网络，称之为流网络，并用 $f(u,v)/c(u,v)$ 表示边 (u,v) 具有流量 $f(u,v)$ 以及容量 $c(u,v)$。如果 $f(u,v)=0$，则省略掉 $f(u,v)/$，只用 $c(u,v)$ 表示。如果 $f(u,v)=c(u,v)$，则称边 (u,v) 为饱和边。

图9.2(a) 给出了一个流网络，其中边 (s,v_1) 上的值11/16表示流量/容量，前者表示 $f(s,v_1)=11$，后者表示 $c(s,v_1)=16$。边 (v_1,v_2) 上的值10表示该边的流量 $f(v_1,v_2)=0$，容量 $c(v_1,v_2)=10$。图9.2(a) 中每条边上的流都满足容量约束性质，例如边 (s,v_1) 上，$f(s,v_1)<c(s,v_1)$。由反对称性质可知，$f(v_1,s)=-f(s,v_1)=-11$。除了源点和汇点之外的其他顶点，其流进的流量和等于流出的流量和，例如对于顶点 v_2，有 $\sum_{v \in V} f(v_2,v)=f(v_2,v_1)+f(v_2,v_4)+f(v_2,s)+f(v_2,v_3)=1+11+(-8)+(-4)=0$。由此可见，图9.2(a) 满足流守恒性质。图9.2(a) 所示网络流的大小为 $|f|=\sum_{v \in V} f(s,v)=8+11=19$。

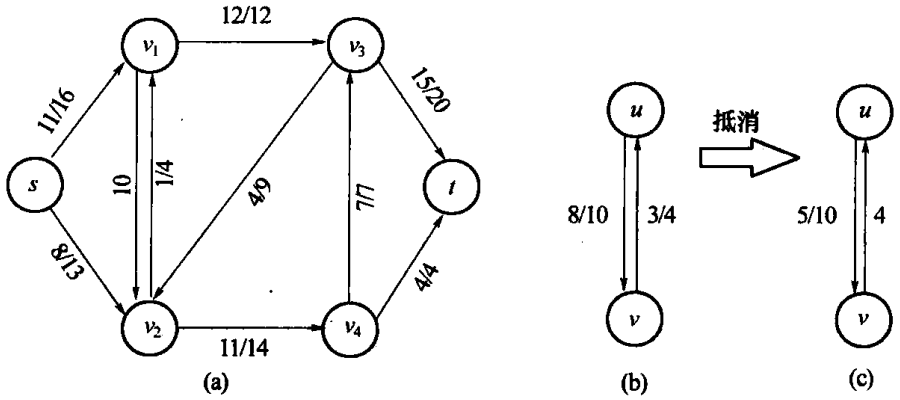


图9.2

值得注意的是，在一些实际问题中会出现如图9.2(b)所示的情况。这时 $f(u,v)=8$，$f(v,u)=3$，不满足反对称性质，这时可以抵消3个单位的流量，变成如图9.2(c)所示。直观上可以这样理解，顶点 u 送8个单位的流量给 v，v 送3个单位的流量给 u。事实上，顶点 u 送5个单位的流量给 v 就可以了。这样做，既避免了麻烦，又使得流满足反对称性质。

下面介绍实际问题中经常碰到的最大流问题，其问题描述为：给定一个网络 $G=(V,E)$，目的是找从源点到汇点的流 f，使其流量值 $|f|$ 最大。

下面我们介绍求解最大流问题的方法。

9.1.1 FordFulkerson 方法

FordFulkerson 方法是求解最大流问题的一个非常经典的方法，它的基本思想是从任何一个可行流开始，沿增广路径（augmenting path）对流进行增广，然后不断重复此过程，直到网络中不存在增广路径为止。问题的关键在于如何有效地找到增广路径，并保证算法在有限次增广后一定终止。这个方法包含三个重要的概念：剩余网络、增广路径和网络的割。在具体介绍 FordFulkerson 算法之前，我们先介绍上述概念。

定义 9.3 给定一个网络 G，其流为 f，容量函数为 c。关于流 f 的剩余容量函数 c_f 定义如下：对任意一对顶点 $u,v \in V$，$c_f(u,v) = c(u,v) - f(u,v)$。流 f 的剩余网络是一个有向图 $G_f = (V, E_f)$，其容量函数由 c_f 定义，边集由 $E_f = \{(u,v): c_f(u,v) > 0\}$ 确定。

由定义 9.3 可以看出，剩余容量 $c_f(u,v)$ 表示在超出边容量 $c(u,v)$ 之前，从顶点 u 到 v 还能够增加的流量的大小。例如图 9.3 给出了图 9.2(a)所示网络的剩余网络。图 9.2(a)中，$c(v_3,v_2)=9$ 且 $f(v_3,v_2)=4$，则在边 (v_3,v_2) 上我们能增加不超过 $c_f(v_3,v_2) = c(v_3,v_2) - f(v_3,v_2) = 5$ 个单位的流量。如果流 $f(u,v)$ 为负，则边的剩余容量 $c_f(u,v)$ 一定比 $c(u,v)$ 大。例如 $f(u,v) = -4$，$c(u,v) = 16$，则 $c_f(u,v) = 20$，这表示从 v 到 u 有 4 个单位的流量，通过从 u 到 v 送 4 个单位的流量，我们能够抵消这 4 个单位的流量。这样，我们从 u 到 v 能够送 20 个单位的流量。

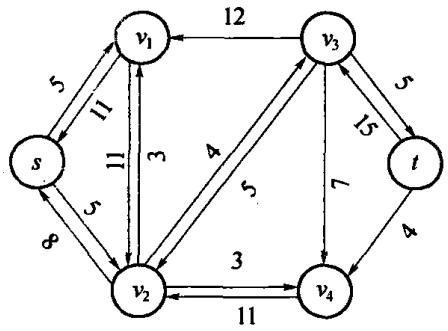


图 9.3

根据剩余网络的定义 9.3，剩余网络的每条边要么是原来网络 G 的边，要么是原来网络 G 中边的逆边（原来 G 中没有的边），剩余网络的每条边上的容量均大于零，例如图 9.3 中边 (v_2,v_3) 没有出现在图 9.2(a)中。根据剩余网络的定义，我们知道剩余网络本身也是一个网络。

定义 9.4 给定一个网络 G，令 f_1，f_2 是 G 的流，令 $(f_1+f_2)(u,v) = f_1(u,v) + f_2(u,v)$，则称 f_1+f_2 为流 f_1，f_2 的和。

引理 9.1 给定一个网络 $G=(V,E)$，令 f 为 G 的流。令 G_f 为 G 关于流 f 的剩余网络，令 f' 为 G_f 的流，则流和 $f+f'$ 仍然是 G 的流，且有 $|f+f'| = |f| + |f'|$。

证明：要证明流和 $f+f'$ 仍然是 G 的流，只需证明 $f+f'$ 满足流的三个性质即可。

对于容量约束性质，由于 f' 为剩余网络 G_f 的流，因此对于任意顶点 $u,v \in V$，有 $f'(u,v) \leqslant c_f(u,v)$。由流和的定义 9.4 可知

$$(f+f')(u,v) = f(u,v) + f'(u,v)$$

由上述两式可知

$$(f+f')(u,v) \leqslant f(u,v) + c_f(u,v) = f(u,v) + (c(u,v) - f(u,v)) = c(u,v)$$

因此，$f+f'$满足容量约束性质。

由于流f和f'满足反对称性质，因此有

$$\begin{aligned}(f+f')(u,v) &= f(u,v) + f'(u,v)\\ &= -f(v,u) - f'(v,u)\\ &= -(f(v,u) + f'(v,u))\\ &= -(f+f')(v,u)\end{aligned}$$

对任意的$u \in V-\{s,t\}$，我们有

$$\begin{aligned}\sum_{v\in V}(f+f')(u,v) &= \sum_{v\in V}(f(u,v) + f'(u,v))\\ &= \sum_{v\in V}f(u,v) + \sum_{v\in V}f'(u,v)\\ &= 0+0 = 0\end{aligned}$$

因此，$f+f'$满足流守恒性质。

最后，由流的定义9.2，我们有

$$\begin{aligned}|f+f'| &= \sum_{v\in V}(f+f')(s,v)\\ &= \sum_{v\in V}f(s,v) + \sum_{v\in V}f'(s,v)\\ &= |f| + |f'|\end{aligned}$$

定义9.5 给定一个网络G，令f为图G的一个流，则称剩余网络G_f中从s到t的一条有向路径p为增广路径。

由于剩余网络G_f的增广路径p上的每条边的容量均大于零，因此我们能够增加这条路径上的流量，在不违反容量约束的情况下，能增加多少呢？

定义9.6 给定剩余网络G_f中从s到t的一条增广路径p，我们将p上的最小剩余容量$c_f(p)$称为瓶颈容量，其中$c_f(p) = \min\{c_f(u,v):(u,v)\in p\}$，并记$|p|$表示路径$p$包含的边数。

瓶颈容量的定义9.6给出了增广路径p上，在不违反容量约束的情况下，能增加的最大流量。

例如图9.3中，路径$p:s,v_2,v_3,t$，就是一条增广路径，它的瓶颈容量$c_f(p)=4$。

引理9.2 给定一个网络$G=(V,E)$，令f为G的流。令G_f为G中关于流f的剩余网络，令p为G_f中从s到t的一条增广路径。定义函数$f_p:V\times V\to \boldsymbol{R}$如下

$$f_p(u,v) = \begin{cases} c_f(p) & \text{if } (u,v) \text{ is on } p\\ -c_f(p) & \text{if } (v,u) \text{ is on } p\\ 0 & \text{otherwise}\end{cases}$$

则f_p是G_f的流，且$|f_p| = c_f(p) > 0$。

证明可参考习题9.4的解答。

由引理9.1和9.2,可以得到如下推论。

推论9.1 给定一个网络$G=(V,E)$,令f为G的流,G_f为G中关于流f的剩余网络,p为G_f中从s到t的一条增广路径。f_p的定义见引理9.2,则$f+f_p$为G的流,且$|f+f_p|=|f|+|f_p|>|f|$。

图9.4给出了一个例子,图9.4(a)表示图9.3所示剩余网络G_f的流f_p,如粗线箭头所示。对原来的网络(图9.2(a))增加f_p后,得到如图9.4(b)所示的网络,此时$f=23$。

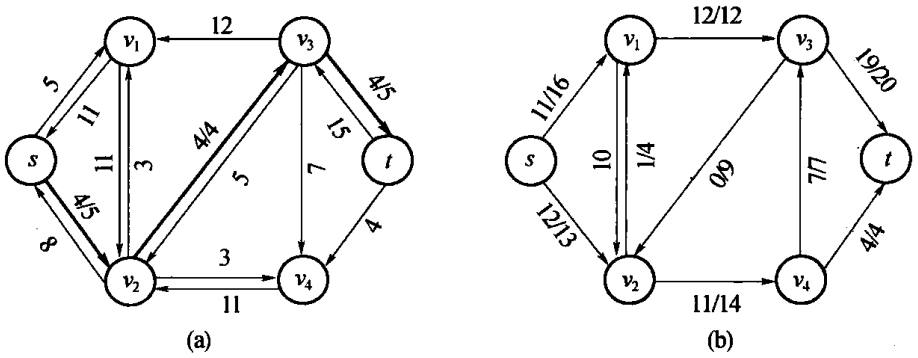


图9.4

推论9.1表明,如果能找到一条增广路径,我们就可以增加f_p到f上,从而可以得到流量值更大的网络G。重复这个过程,我们就能不断地增加网络的流量。问题是如果在剩余网络中再也找不到增广路径,那么所得流就是最大流吗?G的流最大是否等价于剩余网络G_f不再包含增广路径?下面要介绍的最大流最小切割定理将回答这些问题,为了证明该定理,我们先介绍流网络的割。

定义9.7 给定一个网络$G=(V,E)$,网络的一个割(S,T)是把网络G的顶点集V分成两个子集S和$T=V-S$且$s\in S,t\in T$,其中割(S,T)的容量记为$c(S,T)$,定义为$c(S,T)=\sum_{u\in S,v\in T}c(u,v)$,流过割$(S,T)$的流量记为$f(S,T)$,定义为$f(S,T)=\sum_{u\in S,v\in T}f(u,v)$。

图9.5给出了一个例子,其中割(S,T)见虚线,将V分为$S=\{s,v_1,v_2\}$和$T=\{v_3,v_4,t\}$,流经割(S,T)的容量和流量为

$$c(S,T)=c(v_1,v_3)+c(v_2,v_4)=12+14=26$$

$$f(S,T)=f(v_1,v_3)+f(v_2,v_4)+f(v_2,v_3)=12+11+(-0)=23$$

从割的定义9.7可知,越过割(S,T)的流包括从S到T的流,也包括从T到S的流,只是注意方向而已,而割的容量只包括从S到T的边的容量。

引理9.3 给定一个网络$G=(V,E)$,令f为G的流。对于G的任意一个割(S,T),有$f(S,T)=|f|$。

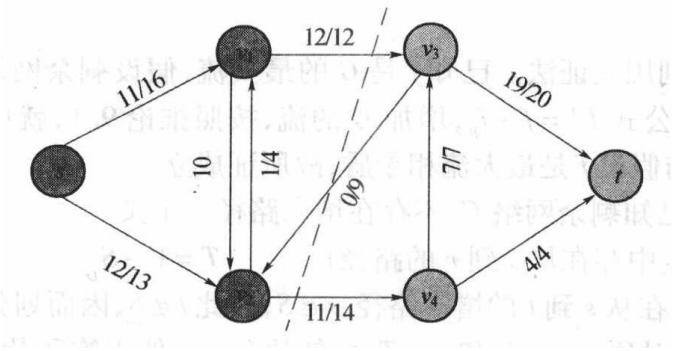


图 9.5

证明:对 S 的顶点数进行归纳证明。

如果 $S=\{s\}$,由定义 9.2 显然成立。

假定对割(S,T)成立,则可以证明当 $w\in T-\{t\}$ 时,结论对割$(S\cup\{w\},T-\{w\})$也成立。事实上:

$$\begin{aligned}f(S\cup\{w\},T-\{w\}) &= f(S,T-\{w\})+f(w,T-\{w\})\\ &= f(S,T)-f(S,w)+f(w,T)-f(w,w)\\ &= f(S,T)-f(S,w)+f(w,T)-0\\ &= f(S,T)+f(w,S)+f(w,T)\\ &= f(S,T)+f(w,V)\\ &= f(S,T)=|f|\end{aligned}$$

故所证成立。

引理 9.3 表明,流过网络的任意割的流量是相同的,且等于网络流的值。

推论 9.2 网络 G 中任意流 f 的值被 G 的任意割的容量所限制,即$|f|\leqslant c(S,T)$。

证明:

$$\begin{aligned}|f| &= f(S,T)\\ &= \sum_{u\in S}\sum_{v\in T}f(u,v)\\ &\leqslant \sum_{u\in S}\sum_{v\in T}c(u,v)\\ &= c(S,T)\end{aligned}$$

推论 9.2 表明,割的容量能用来限制网络流量的大小。下面我们给出最大流最小切割定理。

定理 9.1 给定一个网络 $G=(V,E)$,令 f 为 G 的流,则下列三个结论相互等价:

(1) f 是 G 的最大流;

(2) 剩余网络 G_f 不存在增广路径;

(3) 存在一个割(S,T),使得$|f|=c(S,T)$。

证明：

(1)⇒(2)：利用反证法。已知f是G的最大流，假设剩余网络G_f存在增广路径p，则可以通过公式$f'=f+f_p$，增加G的流，按照推论9.1，就可以得到一个比f更大的流f'，这与假设f是最大流相矛盾，故所证成立。

(2)⇒(3)：已知剩余网络G_f不存在增广路径。定义

$S=\{v\in V:G_f$中存在从s到v的路径$\}$　　$T=V-S$

因为G_f不存在从s到t的增广路径，$s\in S$，因此$t\notin S$，因而划分(S,T)是一个割(S,T)，对任意一对顶点$u\in S$和$v\in T$，可知边(u,v)处于饱和状态，即$f(u,v)=c(u,v)$，否则$(u,v)\in E_f$，顶点v将属于S，这与$v\in T$矛盾。因而$f(S,T)=c(S,T)$，按照引理9.3，可得$f(S,T)=|f|=c(S,T)$，故所证成立。

(3)⇒(1)：已知存在一个割(S,T)，使得$|f|=c(S,T)$，由于对任意割(S,T)，按照推论9.2，均有$|f|\leqslant c(S,T)$，这表示f是G的最大流，故所证成立。

推论9.1告诉我们如何增广流，而定理9.1则回答了当找不到增广路径后，我们就找到了最大流。因此，我们可再次重述FordFulkerson方法的基本思想：对于给定的网络$G=(V,E)$，从G的任意一个可行流f开始，求出该网络的剩余网络G_f，找到G_f的增广路径后，再沿增广路径对流进行增广，重复上述过程，直到网络中不存在增广路径为止。具体算法描述如下：

```
FordFulkerson(G, s, t, c)
1  for each edge (u,v) ∈ E do
2     f(u,v) ← 0
3  G_f ← G
4  while there is an augmenting path p in G_f do
5        let c_f(p) be the bottleneck capacity of p
6        for each edge (u,v) in p do
7           f(u,v) ← f(u,v) + c_f(p)
8        update the residual graph G_f
```

FordFulkerson方法的时间复杂度主要取决于第4行中，增广路径是如何确定的。如果增广路径选择不好，算法有可能不终止，即随着连续增广，它甚至不收敛到最大流量。如果增广路径选择适当，比如我们下面要介绍的最短路径方法，则算法的时间复杂度可以达到多项式时间复杂度。现在我们先给出FordFulkerson方法时间复杂度的上界。

算法第1行～第3行需要时间$O(|E|)$，第5行找增广路径p的瓶颈容量(bottleneck capacity)最多耗时$O(|E|)$。第6行沿着路径p增广以及第8行更新剩余网络G_f，其耗时都不会超过$O(|E|)$。由于第4行～第8行的**while**循环中，每次增加的流量至少为1，因此**while**循环至多执行$|f^*|$次，算法的复杂度为$O(|E||f^*|)$，这里$|f^*|$是算法能够找到的最大流量。实际应用中，如果容量是有理数，通常将其变为整数，从而导致很大的容量，因此FordFulkerson方法执行起来效率很

低，例如图 9.6，FordFulkerson 方法 **while** 循环要执行 20000 次。

图 9.6 给出了 FordFulkerson 方法执行的一个例子。图 9.6(a)给出一个网络，然后从 0 流开始初始化该网络，如图 9.6(b)所示，其剩余网络如图 9.6(c)所示，其中增广路径用粗箭头表示。沿着增广路径对流增广，得到如图 9.6(d)所示的网络。该网络的剩余网络如图 9.6(e)所示，其中增广路径用粗箭头表示。沿着增广路径对流增广，得到如图 9.6(f)所示的网络。该网络的剩余网络如图 9.6(g)所示，重复上述过程，增广 20000 次，才能得到最大流，如图 9.6(h)所示。

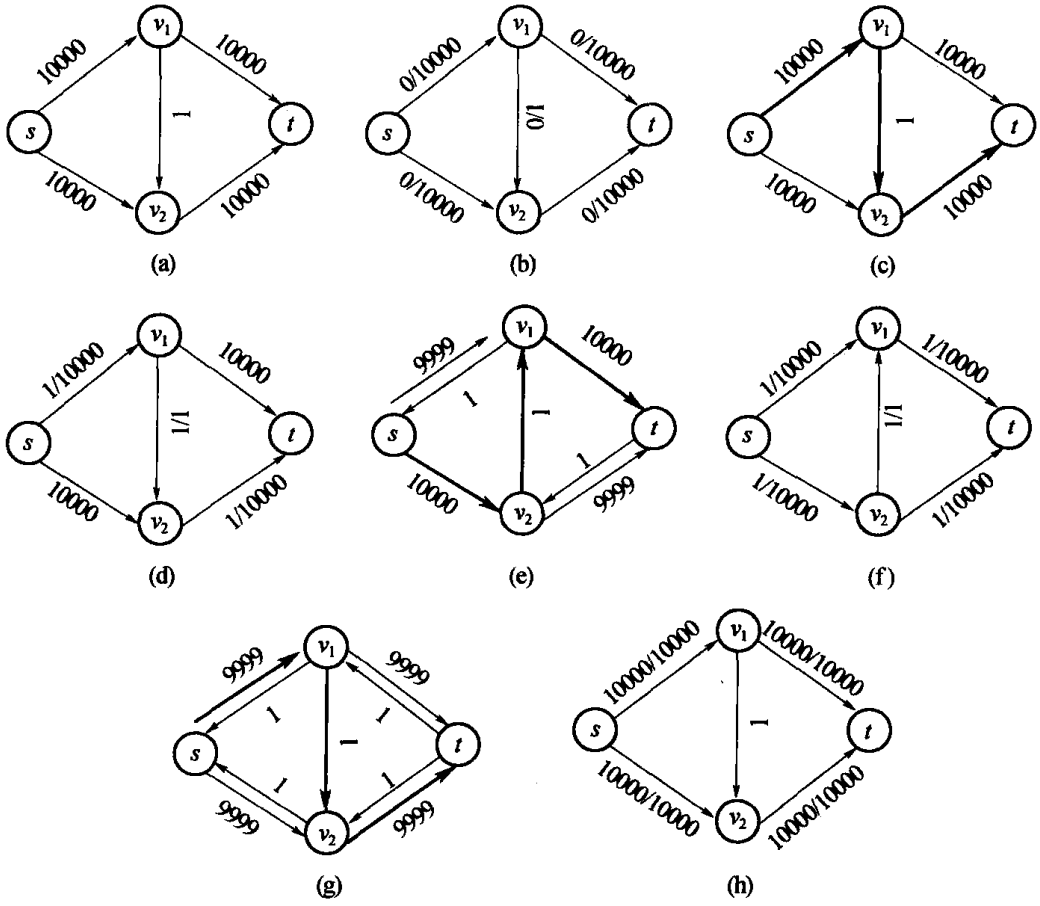


图 9.6

9.1.2 最短路径增广算法

下面我们介绍一个最短路径增广算法(Shortest Path Augmentation)，该方法提出了路径最短的增广路径先增广的思想，从而极大地改进了 FordFulkerson 方法的时间复杂度，而且克服了 FordFulkerson 方法可能依赖具体实例的缺点。

定义 9.8 给定一个网络 $G=(V,E)$，定义层次图 $G_L=(V,E')$，其中 $E'=\{(u,v):(u,v)\in E,\text{且 } d[v]=d[u]+1\}$，$d[v]$也称为顶点 v 的层次，是从源点 s 到

v 的路径中最短路径的边数。

图 9.7(a)给出了一个网络,图 9.7(b)是该网络的层次图。

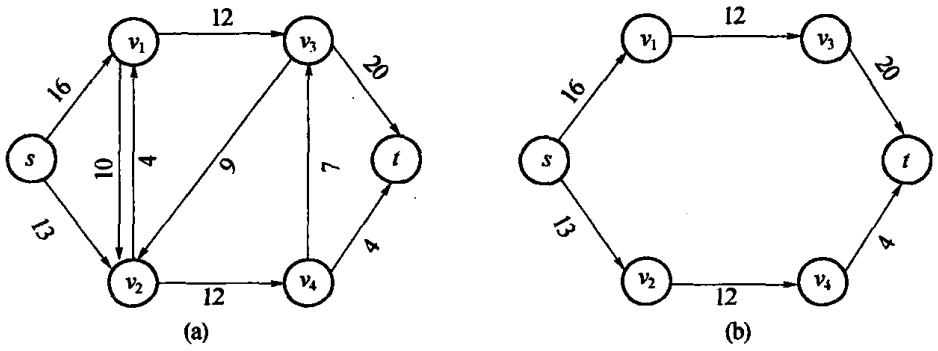


图 9.7

给定一个网络 $G=(V,E)$,我们如何找到该网络的层次图呢? 我们可以利用宽度优先算法来找到层次图,找到了层次图,我们就知道了找增广路径的方法。即我们利用宽度优先搜索找到具有最短路径长度的增广路径,并在当前的流上,沿着增广路径 p 增广 $c_f(p)$ 个单位的流量。详细的算法过程如下:

```
ShortestPathAugmentation(G, s, t)
1  for each edge (u,v) ∈ E do
2     f(u,v) ← 0
3  G_f ← G
4  find the level graph G_L of G_f
5  while t is a vertex in G_L do
6        while there is a path p from s to t in G_L do
7              let c_f(p) be the bottleneck capacity on p
8              augment the current flow f by c_f(p)
9              update G_L and G_f along the path p
10       use G_f to compute a new level graph G_L
```

ShortestPathAugmentation 算法开始先初始化流为 0 流,并设网络的剩余网络 G_f 为原始图 G,并计算其层次图(level graph),然后执行 **while** 循环。**while** 循环分阶段进行,每个阶段由下面两步组成:

(1) 只要 G_L 中有从 s 到 t 的路径 p,就用 f_p 对当前的流 f 进行增广,即 $f+f_p$,从 G_L 和 G_f 中移去饱和边,并相应地更新 G_L 和 G_f。

(2) 根据剩余网络 G_f 计算出层次图 G_L,若 t 不在 G_L 中,则停止,否则继续。

由定义 9.8 可知,层次图中从 s 到 t 的路径具有同样的长度,但在不同阶段中,层次图的最短路径长度都不同,而且随着阶段的进行,路径长度会逐渐增大。一阶段的路径长度会严格的比上一阶段的路径长度大。如果层次图中没有从 s 到 t 的路径,则算法停止。

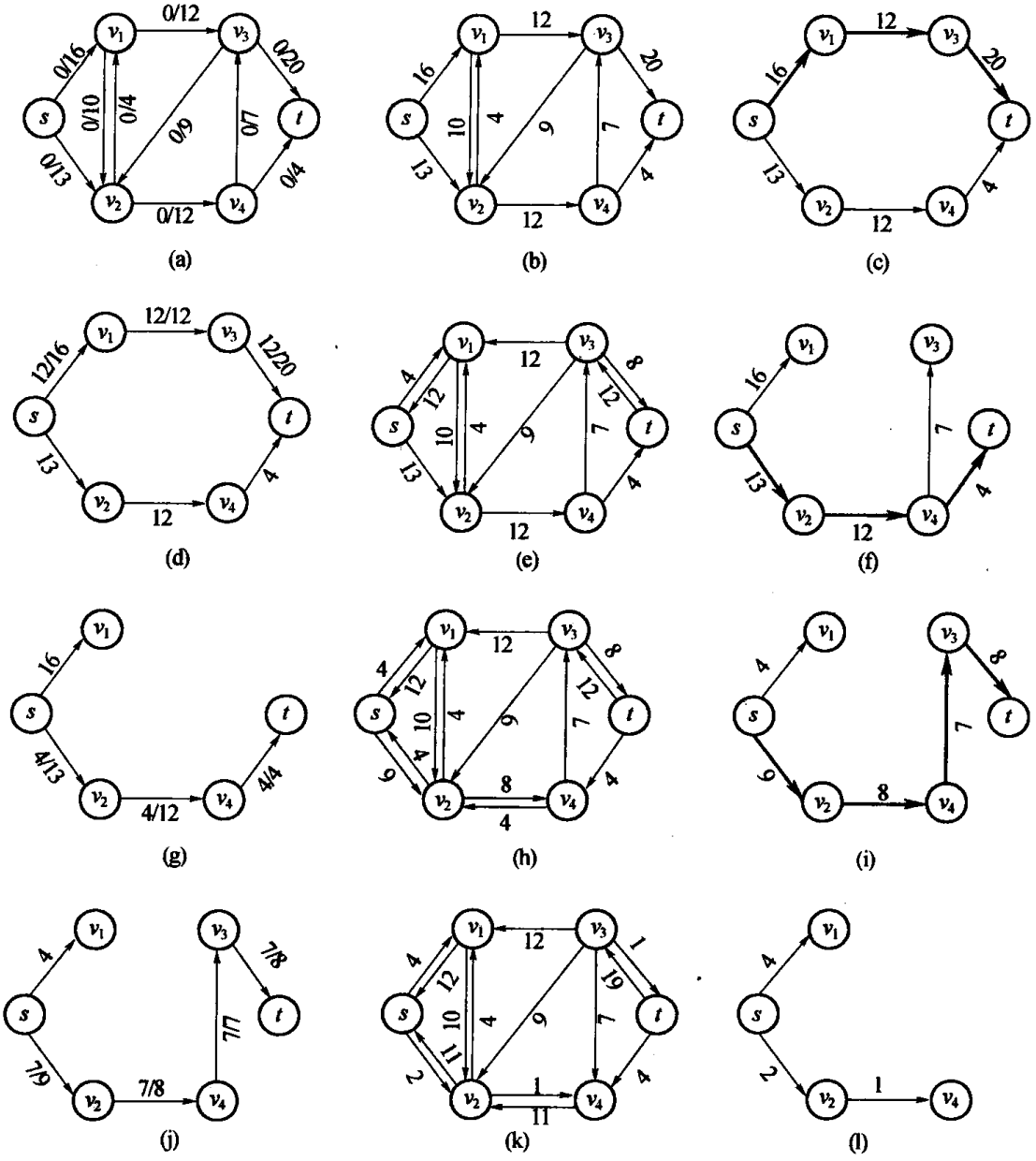


图 9.8

图 9.8 给出了 ShortestPathAugmentation(G, s, t)算法的计算过程。图 9.8(a)表示该算法从 0 流开始增广,图 9.8(a)的剩余网络也就是原始图,如图 9.8(b)所示,其层次图如图 9.8(c)所示。图 9.8(c)中存在一条从 s 到 t 的增广路径 p:s,v_1,v_3,t,其瓶颈容量 $c_f(p)=12$,沿着该路径进行增广,得到图 9.8(d)。沿着路径 p 更新图 9.8(b),得到如图 9.8(e)所示的剩余网络,其层次图如图 9.8(f)所示。图 9.8(f)中只有一条从 s 到 t 的增广路径 p:s,v_2,v_4,t,其瓶颈容量 $c_f(p)=4$,沿着该路径对流进行增广,得到图 9.8(g)。沿着路径 p 更新图 9.8(e),得到如图 9.8(h)

所示的剩余网络，其层次图如图 9.8(i)所示。图 9.8(i)中只有一条从 s 到 t 的增广路径 p:s,v_2,v_4,v_3,t，其瓶颈容量 $c_f(p)=7$，沿着该路径进行增广，得到图 9.8(j)。沿着路径 p 更新图 9.8(h)，得到如图 9.8(k)所示的剩余网络，其层次图如图 9.8(l)所示，图中找不到从 s 到 t 的增广路径，ShortestPathAugmentation(G, s, t)算法终止，最终 $|f|=23$。

下面我们分析一下 ShortestPathAugmentation(G, s, t)算法的时间复杂度。

引理 9.4 在 ShortestPathAugmentation(G, s, t) 算法中，第 5 行的 **while** 循环最多执行 $|V|$ 次。

证明：我们只需要证明 ShortestPathAugmentation 算法产生的层次图至多为 $|V|$ 个即可。

首先，我们证明该算法在前后层次图中找到的增广路径的长度序列是严格递增的。设 p 为当前层次图中任意一条增广路径，在对 p 增加流量后，路径 p 中至少有一条边将饱和，并在剩余网络中消失。至多 $|p|$ 条新边会在剩余网络中出现，但它们是后向边，因此不会影响从 s 到 t 的最短路径。当沿着同一层次图中所有增广路径对流依次进行增广后，下一层次图的增广路径会更长，故所证成立。

当 t 在 G_L 中再也不能从 s 到达时，则找不到增广路径。因为任何增广路径长度在 1 和 $|V|-1$ 间，因此用做增广的层次图个数最多为 $|V|-1$。因为 t 不出现的层次图也要计算一次，因此计算的层次图总数最多是 $|V|$。

定理 9.2 给定一个网络 G，用 ShortestPathAugmentation(G, s, t) 算法找到最大流所需要的时间为 $O(|V||E|^2)$。

证明：因为每次有一条边在层次图中消失，因此层次图中最多可能有 $|E|$ 条增广路径，即沿着同样长度的路径至多只能增广 $|E|$ 次，根据引理 9.4，用来增广而计算的层次图个数最多是 $|V|$，因此，增广的次数最多为 $|V||E|$。使用宽度优先搜索在 G_L 中找到一条最短增广路径需要 $O(|E|)$ 的时间。那么，寻找所有增广路径的时间为 $O(|V||E|^2)$，因此用 ShortestPathAugmentation(G, s, t) 算法找到最大流所需要的时间为 $O(|V||E|^2)$。

在 ShortestPathAugmentation(G, s, t) 算法中，每个层次图的增广路径中，找每条路径是分开进行并分别增广的，下面介绍的算法，能够更有效地找层次图的增广路径，降低了算法的时间复杂度，从而提高了计算的效率。

9.1.3 Dinic 算法

定义 9.9 给定一个网络 $G=(V,E)$，G_L 是 G 的子图，且包含 s 和 t，如果在 G_L 中每一条从 s 到 t 的路径中都至少有一条饱和边，则称 G_L 的流 f 为阻塞流。

Dinic 算法就是找出层次图，然后按照深度优先的方式找增广路径，再按照阻塞流对当前流进行增广。详细的过程如下：

```
Dinic(G, s, t, c)
1  for each edge (u,v) ∈ E do
2     f(u,v) ← 0
3  G_f ← G
4  find the level graph G_L of G_f
5  while t is a vertex in G_L do
6        u ← s; p ← {u}
7        while outdegree(s) > 0 do
8              while u ≠ t and outdegree(s) > 0 do
9                    if outdegree(u) > 0 then
10                         let (u,v) is an edge in G_L
11                         p ← p ∪ {v} ; u ← v
12                   else  delete u and all adjacent edges from G_L
13                         remove u from the end of p
14                         set u to the last vertex in p (u may be s)
15             if u = t then
16                   let c_f(p) be the bottleneck capacity along p
17                   augment the current flow along p by c_f(p)
18                   adjust residual graph and delete saturated edges from level graph G_L
19                   set u to the last vertex on p reachable from s (Note that u may be s)
20       compute a new level graph G_L from G_f
```

Dinic(G, s, t, c)算法的主要过程分为三个循环,外部 **while** 循环(第 5 行)的每次迭代对应一个阶段,按照引理 9.4,可知至多有$|V|$个阶段,中间 **while** 循环(第 7 行)执行深度优先搜索找增广路径,然后沿着增广路径对流进行增广。内部 **while** 循环(第 8 行)有两个条件判断,如果当前路径 p 的末端顶点 u 不是汇点,且有出边(outdegree(u) >0),则执行两个运算:如果顶点 u 有出边(u,v),则在第 11 行将顶点 v 加入路径 p 的后面,并令 $u \leftarrow v$,继续往下搜索。否则第 12 行从 G_L 中删除 u 以及与 u 相邻的边,第 13 行从路径 p 中去掉 u 点,第 14 行将路径 p 的末端顶点作为 u。

内部 **while** 循环执行完后,如果 u 点就是汇点 t,则执行增广过程。

Dinic(G, s, t, c)算法的高效之处在于,找到增广路径,并对其增广后,算法会回溯到路径上某个有可能继续增广的顶点(不一定是源点),然后开始寻找从该顶点出发的另一条增广路径。而 ShortestPathAugmentation(G, s, t)算法则需要从源点 s 出发找增广路径。

图 9.9 给出了 Dinic(G, s, t, c)算法计算一个例子的过程。Dinic 算法从 0 流开始计算,图 9.9(a)给出了关于 0 流的剩余网络,其对应的层次图如图 9.9(b)所示。由于 t 在层次图中,因此执行第 6 行,由于源点 s 的出度大于 0,因此执行第 8 行。满足 **while** 循环条件,继续执行第 9 行,同样满足 **if** 条件,因此执行第 10 行和

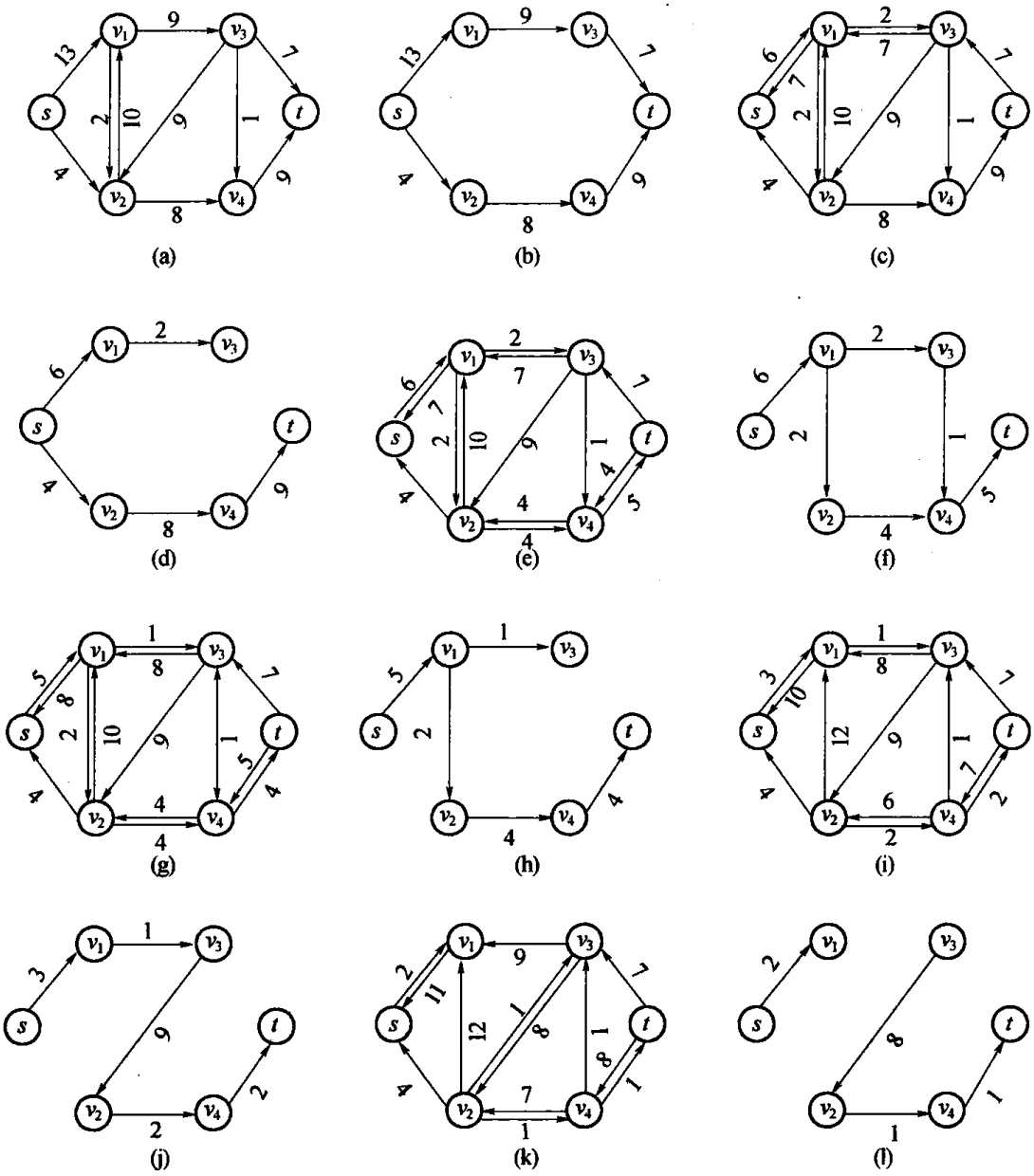


图 9.9

第 11 行，$p:s,v_1,u=v_1$。由于 $u\neq t$ 且 outdegree(u) >0，因此继续执行第 10 行和第 11 行，直到 $p:s,v_1,v_3,t,u=t$。由于 $u=t$，执行第 16 行～第 19 行，沿着增广路径对流进行增广，更新剩余网络，如图 9.9(c)所示，删除饱和边(v_3,t)，此时 $p:s,v_1,v_3,u=v_3$，如图 9.9(d)所示。继续执行第 8 行的 **while** 循环，由于 outdegree(u) $=0$，因此不断执行第 12 行～第 14 行，从 G_L 中删除 u 以及所有邻接 u 的边，从路径 p 中删除末端顶点 u，直到 $u=s$。由于源点 s 的出度大于 0，类似上述过程继续执行第 8 行的 **while** 循环。最后得到如图 9.9(e)所示的剩余网络。执行第 20 行，得到新的

层次图如图9.9(f)所示。由于t在层次图中,因此执行第6行,由于源点s的出度大于0,因此执行第8行。满足**while**循环条件,继续执行第9行,同样满足**if**条件,因此执行第10行和第11行,$p:s,v_1,u=v_1$。由于$u\neq t$且outdegree$(u)>0$,因此继续执行第10行和第11行,直到$p:s,v_1,v_3,v_4,t,u=t$。由于$u=t$,执行第16行~第19行,沿着增广路径对流进行增广,更新剩余网络,如图9.9(g)所示,删除饱和边(v_3,v_4),此时$p:s,v_1,v_3,u=v_3$,如图9.9(h)所示。继续执行第8行的**while**循环,由于outdegree$(u)=0$,因此执行第12行~第14行,从G_L中删除u以及所有邻接u的边,从路径p中删除末端顶点u,此时$u=v$。继续执行第8行的**while**循环,由于u的出度大于0,执行第4行,第11行,重复过程,直到$p:\ s,v_1,v_2,v_4,t,u=t$。最后得到如图9.9(i)所示的剩余网络。执行第20行,得到新的层次图如图9.9(j)所示。类似上述过程,得到如图9.9(k)所示的剩余网络。删除饱和边后,得到对应的层次图如图9.9(l)所示。此时,找不到增广路径,Dinic(G,s,t,c)算法终止,最终$|f|=15$。

Dinic(G,s,t,c)算法如同ShortestPathAugmentation(G,s,t)算法一样,分为$|V|$个阶段,每个阶段找一个层次图,然后找每个层次图的阻塞流,再按照当前阻塞流进行增广。对于Dinic(G,s,t,c)算法的时间复杂度,我们有如下结论。

定理9.3 给定一个网络$G=(V,E)$,Dinic(G,s,t,c)算法找到最大流的时间为$O(|V|^2|E|)$。

证明:用宽度优先搜索找层次图G_L,需要时间$O(|E|)$。第7行**while**循环最多执行$O(|E|)$次,第8行**while**循环最多执行$O(|V|)$次,第15行~第19行执行$O(|V|)$次,因此第7行~第19行共需要时间$O(|V||E|)$。第5行的**while**循环最多执行$|V|$次,因此执行第5行~第20行所需要的时间均不超过$O(|V|^2|E|)$,故所证成立。

9.1.4 MPM算法

虽然,Dinic算法找到最大流的时间为$O(|V|^2|E|)$,但是如果路径长度很长,用深度优先搜索来找阻塞流并增广,则需要花比较长的时间。Malhortra, Pramodh-Kumar, Maheshwari (MPM)对找阻塞流的方法进行改进,得到了一个更有效的算法。

定义9.10 给定一个网络$G=(V,E)$,对于网络G的顶点v,我们定义v的吞吐量$C(v)$为该顶点入边的总容量和出边的总容量的最小值。即对于$v\in V-\{s,t\}$

$$C(v)=\min\{\sum_{u\in V}c(u,v),\sum_{u\in V}c(v,u)\}$$

对于源点s和汇点t,我们分别定义其吞吐量为

$$C(s)=\sum_{v\in V-\{s\}}c(s,v)\quad 和\quad C(t)=\sum_{v\in V-\{t\}}c(v,t)$$

MPM算法找阻塞流的方法是,在层次图中找到吞吐量最小的顶点v,然后从该

顶点沿着出边推出 $C(v)$ 个单位的流量，即将流量分配到出边，先分配的边先饱和，直到有一条边部分饱和为止。类似地，对入边也类似分配。最后，将该顶点以及相邻的边删除。

详细的 MPM 算法如下：

```
MPM(G, s, t, c)
1  for each edge (u, v) ∈ E do f(u,v) ←0
2  G_f ←G; find the level graph G_L of G_f
3  while t is a vertex in G_L do
4        while t is reachable from s in G_L do
5              find a vertex v of minimum C(v)
6              push C(v) units of flow from v to t
7              pull C(v) units of flow from s to v
8              update f, G_L and G_f
9        use G_f to compute a new level graph G_L
```

MPM(G, s, t, c)算法在执行第 6 行时，从层次图的顶点 v 一路上推大小为 $C(v)$ 的流到 t，尽量使得 v 的出边中更多的边饱和。类似地，执行第 7 行，从层次图的源点 s 一路上拉大小为 $C(v)$ 的流到顶点 v，尽量使得 v 的入边中更多的边饱和。由于顶点 v 的吞吐量最小，删除所有饱和边后，顶点 v 和所有与顶点 v 邻接的边将从层次图中删除。对于 MPM 算法，有下列结论。

定理 9.4 给定一个网络 $G=(V,E)$，MPM(G, s, t, c)算法找到最大流的时间为 $O(|V|^3)$。

证明：用宽度优先搜索找层次图 G_L，需要时间 $O(|E|)$。在第 5 行从层次图中找具有最小吞吐量的顶点需要时间 $O(|V|)$。第 6 行和第 7 行中，包括部分饱和这些边、删除所有饱和边以及将大小为 $C(v)$ 的流从 s 推向 t，所需要的时间均不超过 $O(|V|)$。第 8 行更新 f，G_L 和 G_f 需要时间 $O(|V|)$。第 4 行的 **while** 循环最多执行 $|V|$ 次，因此执行第 5 行 ~ 第 8 行所需要的时间均不超过 $O(|V|^2)$，而第 3 行 **while** 循环最多迭代 $|V|$ 次，因此 MPM(G, s, t, c)算法找到最大流的时间为 $O(|V|^3)$。

9.1.5 最大流问题的变形

很多组合优化问题，可以很容易转化为最大流问题。这样便可以利用我们前面介绍的算法来求解。

在许多实际问题中，例如某些最大流问题可能不只一个源点和汇点。但是我们可以引进一个超级源点和一个超级汇点，从而可以把这种最大流问题转化为单源点和单汇点的最大流问题，如图 9.10 所示。图 9.10(a)所示为多源点多汇点的网络，通过引进超级源点和超级汇点，我们得到图 9.10(b)所示的单源点和单汇点的最大流问题。

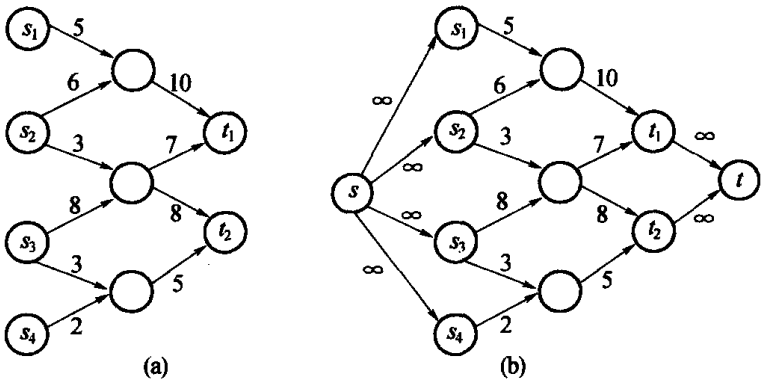


图 9.10

9.2 最小费用流问题

在带权网络 $G=(V,E)$ 上，除了给定容量函数外，还增加如下实值权函数 $w:V\times V\to \boldsymbol{R}$。任给 $(u,v)\in E$，对应的权值记为 $w(u,v)$，表示边 (u,v) 的单位流量的费用或成本，也就是说通过一条边的单位流所需要的费用。

给定一个流 f，流 f 的总费用定义为

$$w(f)=\sum_{(u,v)\in E} w(u,v)f(u,v)$$

如果 f 在流量为 $|f|$ 的所有流中具有最小的费用，则称流 f 为最小费用流。最小费用流问题就是在网络中寻找总费用最小的可行流。

最小费用流问题通常有两种表现形式，一种是在网络 $G=(V,E)$ 中计算给定流量为 $|f|$ 的最小费用流；另一种是不给定流量时，计算流量最大的最小费用流，此问题常称为最小费用最大流问题。事实上，前面介绍过的最短路径问题以及最大流问题都是最小费用流问题的特例。

在给出求解最小费用流问题的算法之前，类似最大流问题的剩余网络，带权网络流 f 的剩余网络是一个有向图 $G_f=(V,E_f)$，可定义如下：

定义 9.11 给定一个带有容量 c 以及权函数 w 的网络 $G=(V,E)$，任给 $(u,v)\in E$，我们用两条边 (u,v) 和 (v,u) 来代替原来的边 (u,v)。但是边 (u,v) 的权值 $w(u,v)$ 不变，其剩余容量为 $c_f(u,v)=c(u,v)-f(u,v)$，而新边 (v,u) 的权值为 $w(v,u)=-w(u,v)$，其剩余容量为 $c_f(v,u)=f(u,v)$。由所有剩余容量为正的边构成的带权网络称为流 f 的剩余网络 $G_f=(V,E_f)$。

图 9.11(a) 给出了一个网络，边 (u,v) 上的符号对应 $f(u,v)/c(u,v)$，$w(u,v)$，例如边 (s,v_1) 上的符号 3/4，2 表示 $f(s,v_1)=3$，$c(s,v_1)=4$，$w(s,v_1)=2$。图 9.11(b) 给出了相应的剩余网络。值得注意的是，剩余网络 G_f 中边上的符号对应 $c(u,v)$，$w(u,v)$，例如边 (s,v_1) 上的符号 1，2 表示 $c(s,v_1)=1$，$w(s,v_1)=2$。

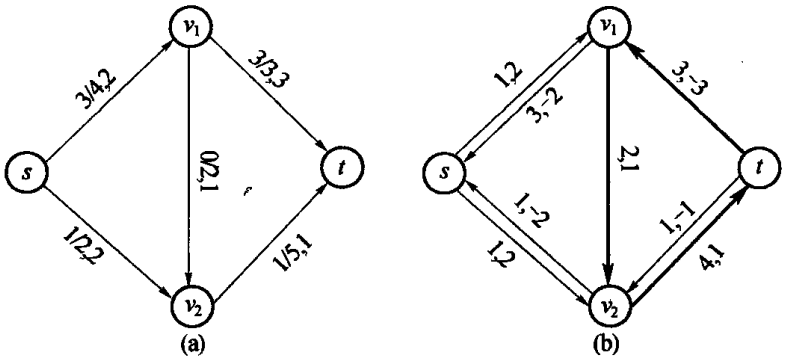


图 9.11

下面我们给出一些求解最小费用流问题的算法。

9.2.1 消除回路算法

定义 9.12 给定一个网络 $G=(V,E)$，相应剩余网络的增广回路(Cycle)是一条增广路径 p，路径 p 的起点跟终点相同。具体地说，它是一个顶点为 $v_1,v_2,\cdots,v_n(=v_1)$ 的有向回路，并且回路中每条边 (v_i,v_{i+1}) 的剩余容量 $c_f(v_i,v_{i+1})>0(1\leqslant i<n)$。定义增广回路中最小的剩余容量 $c_f(p)=\min\limits_{1\leqslant i<n}c_f(v_i,v_{i+1})$，则增广回路中能够增广的流量最多为 $c_f(p)$。定义增广回路 p 的费用为

$$w(p)=\sum_{(u,v)\in p}w(u,v)$$

图 9.11(b)中，增广回路 p 为 v_2,t,v_1,v_2，$c_f(p)=2$，$w(p)=-1$。

如果剩余网络 G_f 中存在费用为负的增广回路 p，即 $w(p)<0$，则可以沿增广回路 p 对当前流 f 进行增广，以便获得流量相等但费用更小的流。只要剩余网络中存在费用为负的增广回路 p，就可以重复上述过程，把一个最大流转化为另一个最大流，而且使得总费用逐步减少。

引理 9.5 设 p 为网络 $G=(V,E)$ 相应剩余网络的一条增广回路，则存在流量为 $|f|$ 的流 f'，且其费用为

$$w(f')=w(f)+w(p)c_f(p)$$

引理 9.5 说明，如果流 f 的剩余网络 G_f 中存在费用为负的回路，则该流就不是最小费用流。以下定理表明，它的逆命题也成立。

定理 9.5 可行流 f 为最小费用流当且仅当 G_f 中不存在费用为负的增广回路。

证明：已知可行流 f 为最小费用流，假设 G_f 中含有费用为负的增广回路，则由引理 9.5 可知，f 不可能是最小费用流，因为沿着该回路增广流，可以得到一个费用比 $w(f)$ 更低的流，与已知 f 为最小费用流矛盾，故所证成立。

现在已知 f 是可行流，G_f 中不存在费用为负的增广回路。假设 f 不是最小费用流，则存在不同于 f 的最小费用流 f' 且其流量值为 $|f|$。现在由 f 沿着增广回路通

过一系列的增广得到流f'，由于回路的费用非负，残余容量非负，因此，不论如何增广，都得不到流f'。要想得到流f'，必须存在一条费用为负的回路，这与已知矛盾，故假设不成立。

由定理9.5，可以得到如下消除回路(Canceling Cycle)算法：

CancelingCycle(G, s, t, c, w)

1 establish a feasible flow f in the network

2 **while** G_f contains a negative cycle **do**

3 use some algorithm to identify a negative cycle p

4 compute $c_f(p)$

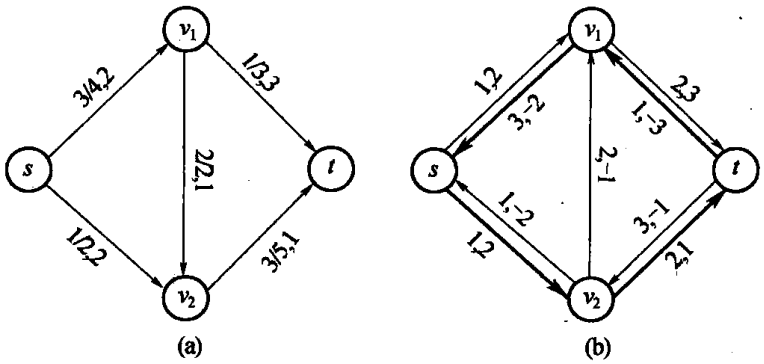
5 augment $c_f(p)$ units of flow in the cycle p and update G and G_f

CancelingCycle(G, s, t, c, w)算法的第1行，可用最大流算法找出一个可行流f。第2行确定G_f中是否含有一条费用为负的增广回路(negative cycle)，可用BellmanFord算法在$O(|V||E|)$时间内找出一条费用为负的回路。第4行计算增广回路中最小的剩余容量$c_f(p)$，第5行沿着增广回路增加$c_f(p)$个单位的流，然后更新G和G_f。令w'表示初始最大流f所具有的费用，从最大流f开始，可以在$O(|V||E|w')$时间内找到流量值为$|f|$的最小费用流，即我们可以得到下列结论：

定理 9.6 给定网络$G=(V, E)$以及一个最大流f，CancelingCycle(G, s, t, c, w)算法可以在$O(|V||E|w')$时间内找到流量值为$|f|$的最小费用流。

由定理9.6可以看出，CancelingCycle(G, s, t, c, w)算法的时间复杂度取决于初始最大流f所具有的费用w'，因此该算法不是多项式时间算法。如果按照一些特定次序消除回路，可得到一些多项式时间算法。

图9.12给出了CancelingCycle(G, s, t, c, w)算法计算一个例子的过程。图9.11(a)为初始网络，其对应的剩余网络如图9.11(b)所示。从该图中我们可以找出一条费用为负的增广回路p为v_2, t, v_1, v_2，$c_f(p)=2$，因此我们可以沿着该回路增广2个单位的流量，得到如图9.12(a)的网络。其对应的剩余网络如图9.12(b)所示。从该图中我们可以找出一条费用为负的回路$p: v_1, s, v_2, t, v_1$，$c_f(p)=1$，因此我们可以沿着该回路增广1个单位的流量，得到如图9.12(c)所示的网络。其对应的剩余网络如图9.12(d)所示，此时找不到费用为负的回路，算法终止，找到最小费用流。此时$f=4$，$w(f)=14$。



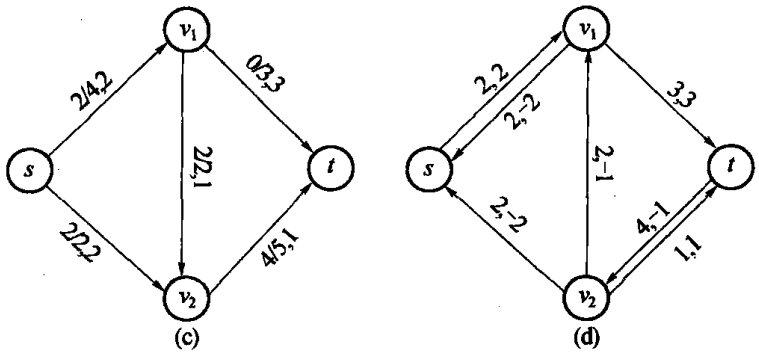


图 9.12

9.2.2 最小费用路算法

消除回路算法是在给定流 f 的网络 $G=(V,E)$ 中通过重复消去费用为负的增广回路，从而找到最小费用流。下面我们从一个 0 流开始，沿着具有最小费用的增广路径，通过一系列的增广过程，来获得具有最小费用的最大流。

对于剩余网络 G_f 的一条从 s 到 t 的有向路径 p，它一定对应于原网络 $G=(V,E)$ 的一条增广路径，即可以通过沿 p 对当前流 f 进行增广，获得流量值更大的可行流 f'。定义 p 的费用为

$$w(p)=\sum_{(u,v)\in p} w(u,v)$$

为了获得最小费用流，我们希望沿费用最小的增广路径对当前流 f 进行增广，以最小的费用增加获得流量值更大的可行流 f'。设最小费用路径 p 中最小的剩余容量为 $c_f(p)$，则有

$$w(f')=w(f)+w(p)c_f(p)$$

下面的定理保证了我们沿着最小费用路径对流进行增广，能够找到最小费用流。

定理 9.7 设 f 为流量值为 $|f|$ 的最小费用流，p 为相应剩余网络 G_f 中从 s 到 t 的一条最小费用增广路径，且沿 p 所能增广的流量为 $c_f(p)$，则增广后得到的流 f' 是最小费用流，且流量值为 $|f|+c_f(p)$。

证明：由定理 9.5 可知，只需证明关于流 f' 的剩余网络中不存在费用为负的增广回路即可。可以用反证法来证明。假设沿 p 增广后存在关于 f' 的费用为负的增广回路 H。由于 f 为最小费用流，根据定理 9.5 可知，关于流 f 的剩余网络中不存在费用为负的增广回路。由于除 p 以外的边及其流量在增广前后没有发生改变，于是 p 和 H 至少有一条公共边，这样该公共边发生改变后，才有可能与其他的边构成关于 f' 的费用为负的增广回路 H。不妨假设 p 和 H 有一条公共边 (u,v)，如图 9.13所示。如果 (u,v) 在 p 中是正向边，则在 H 中是反向边；反之，如果 (u,v) 在 p 中是反向边，则在 H 中是正向边，这样才能导致增广回路。因此 $\{p-(u,v)\}\cup\{H$

$-(v,u)\}$也是网络中关于f的增广路径，如图 9.13 所示。又由于$w(H)<0$，因此有

$$w(\{p-(u,v)\}\cup\{H-(v,u)\})=w(p)-w(u,v)+w(H)-w(v,u)$$
$$=w(p)+w(H)<w(p)$$

这表明关于流f的剩余网络G_f中存在一条比路径p的费用还小的路径，这与p是最小费用路径矛盾，故关于流f'的剩余网络中不存在费用为负的增广回路，由定理 9.5 可知，流f'是最小费用流，且流量值为$|f|+c_f(p)$。

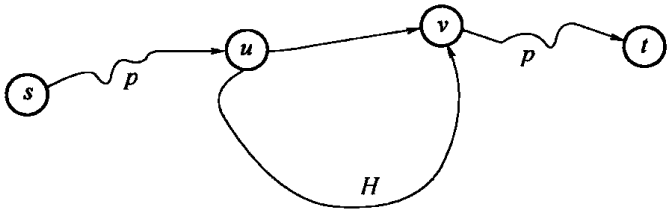


图 9.13

由定理 9.7，我们可以得到最小费用路(Minimum Weight Path)算法，即从 0 流开始，反复应用定理 9.7，计算一个具有最小费用的最大流。具体算法如下：

MinimumWeightPath(G, s, t, c, w)

1 **for** each edge $(u, v)\in E$ **do**
2 $f(u,v)\leftarrow 0$
3 $G_f\leftarrow G$
4 **while** there is a minimum weight path p in G_f **do**
5 let $c_f(p)$ be the bottleneck capacity on p
6 augment the current flow f by $c_f(p)$
7 update G_f along the path p

在第 4 行中，可以利用第 8 章介绍过的 BellmanFord 算法计算G_f的一条最小费用路径，其时间复杂度为$O(|V||E|)$。由于每次增加的流量至少为 1 个单位，因此 **while** 循环至多执行$|f^*|$次，这里$|f^*|$是算法能够找到的具有最小费用的最大流量。因此，MinimumWeightPath(G, s, t, c, w)算法的时间复杂度为$O(|f^*||V||E|)$。

例如给定一个流量为 0 的网络，如图 9.14(a)所示，其对应的剩余网络如图 9.14(b)所示。其最小费用路径p为s,v_2,v_1,t，$w(p)=3$，$c_f(p)=2$。因此沿着路径p增广 2 个单位的流量，得到网络图 9.14(c)，其对应的剩余网络如图 9.14(d)所示。其最小费用路径p为s,v_1,v_2,t，$w(p)=5$，$c_f(p)=1$。因此沿着路径p增广 1 个单位的流量，得到网络图 9.14(e)，其对应的剩余网络如图 9.14(f)所示，由于在剩余网络中找不到增广路径，算法终止。流量值为 3，费用为 11。

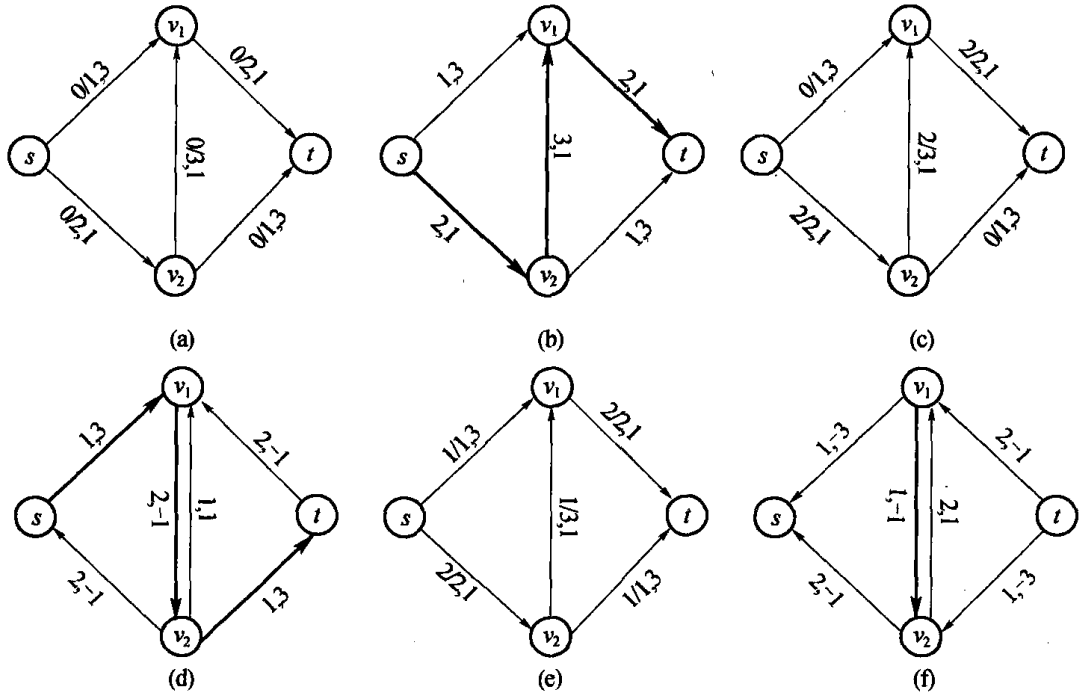


图 9.14

9.2.3 最小费用路算法的改进

在求剩余网络的最小费用路径时，由于剩余网络中，费用可能为负，只能利用第 8 章介绍过的时间复杂度为 $O(|V||E|)$ 的 BellmanFord 算法，如果能够改变剩余网络中边的费用，使得所有边的费用均为非负数，则可以调用第 8 章介绍过的时间复杂度为 $O(|E|\lg|V|)$ 的 Dijkstra 算法，从而得到更有效的最小费用路算法。下面介绍改变剩余网络中各边费用的方法。

定义 9.13 给定流为 f 的网络 $G=(V,E)$，其对应的剩余网络为 G_f，令 $d_f(v)$ 表示剩余网络 G_f 中，源点 s 到顶点 v 的最小增广费用路径的总费用。令 f' 表示沿着一条最小增广费用路径增大 f 后而得到的流，定义剩余网络 G_f 中任意边 (u,v) 的费用为

$$w'_{f'}(u,v) = w_f(u,v) + d_f(u) - d_f(v) \tag{9.1}$$

定理 9.8 剩余网络 $G_{f'}$ 中，每条边 (u,v) 的费用 $w'_{f'}(u,v) \geqslant 0$。且剩余网络 $G_{f'}$ 的最小费用路径对应 G_f 的最小费用路径。

证明：对于剩余网络 G_f 的边，有的在原来的 G_f 中，有的不在，因此我们可以分两种情况加以证明。

(1) 如果边 (u,v) 出现在 G_f 中，则源点 s 到顶点 v 的费用 $d_f(v)$ 一定不超过源点 s 到顶点 u 的费用 $d_f(u)$ 与 $w(u,v)$ 之和，即 $d_f(v) \leqslant d_f(u) + w(u,v)$，且有 $w_f(u,v) = w(u,v)$，由式(9.1)可得

$$w'_{f'}(u,v) = w_f(u,v) + d_f(u) - d_f(v) \geqslant 0$$

(2) 如果边(u,v)不在G_f中,则边(v,u)一定在增大流f得到流为f'的最小费用路径p上,且有$d_f(u) = d_f(v) + w(v,u)$,由于$w_f(u,v) = -w(v,u)$。由式(9.1)可得

$$w'_{f'}(u,v) = w_f(u,v) + d_f(u) - d_f(v) = 0$$

例如,如图9.14(d)所示,边(v_2,v_1)在原来的G_f中(图9.14(b)),因此有$d_f(v_1) \leqslant d_f(v_2) + w(v_2,v_1)$。边$(v_1,v_2)$不在原来的$G_f$中(见图9.14(b)),因此有$d_f(v_1) = d_f(v_2) + w(v_2,v_1)$。

下面我们证明,对G_f中各边的费用改变,得到$G_{f'}$后,G_f的最小费用路径仍然是G_f的最小费用路径。

给定G_f的任一条路径$p:s,v_1,v_2,\cdots,v_n,t$,该路径的费用为

$$w(p) = w(s,v_1) + \sum_{i=1}^{n-1} w(v_i,v_{i+1}) + w(v_n,t)$$

对$G_{f'}$中同样的一条路径$p:s,v_1,v_2,\cdots,v_n,t$,其费用为

$$w'(p) = w'(s,v_1) + \sum_{i=1}^{n-1} w'(v_i,v_{i+1}) + w'(v_n,t)$$

按照权值改变式(9.1)可得

$$\begin{aligned} w'(p) &= w(s,v_1) + d_f(s) - d_f(v_1) + \sum_{i=1}^{n-1} (w(v_i,v_{i+1}) + d_f(v_i) - d_f(v_{i+1})) + \\ &\quad w(v_n,t) + d_f(v_n) - d_f(t) \\ &= w(s,v_1) + d_f(s) + \sum_{i=1}^{n-1} w(v_i,v_{i+1}) + w(v_n,t) - d_f(t) \\ &= w(p) - d_f(t) \end{aligned}$$

上述式子表明G_f的任一条路径p的费用与$G_{f'}$的路径p的费用只差一个常数,因此G_f的最小费用路径仍然是$G_{f'}$的最小费用路径。

综上所述,故所证成立。

例如给定一个流量为0的网络图9.15(a),其剩余网络如图9.15(b)所示,顶点旁边的数字表示剩余网络中,源点s到达该顶点的最小费用路径的总费用,例如图9.15(b)中,顶点$d_f(v_1) = 2$表示从源点s到顶点v_1的最小费用路径的总费用。由于最小费用路径p为s,v_2,v_1,t,增广2个单位的流量,得到网络图9.15(c)。如果按照最小费用路算法,将会得到如图9.14(d)所示的剩余网络,将该网络与图9.15(b)中各顶点的费用按照式(9.1)计算,可以得到一个新的剩余网络,如图9.15(d)所示,其中各边费用非负。此时其最小费用路径p为s,v_1,v_2,t,$w(p) = 2$,$c_f(p) = 1$。因此沿着路径p增广1个单位的流量,得到网络图9.15(e),其对应的剩余网络如图9.15(f)所示,由于剩余网络找不到增广路径,算法终止。流量值为3,费用为11。

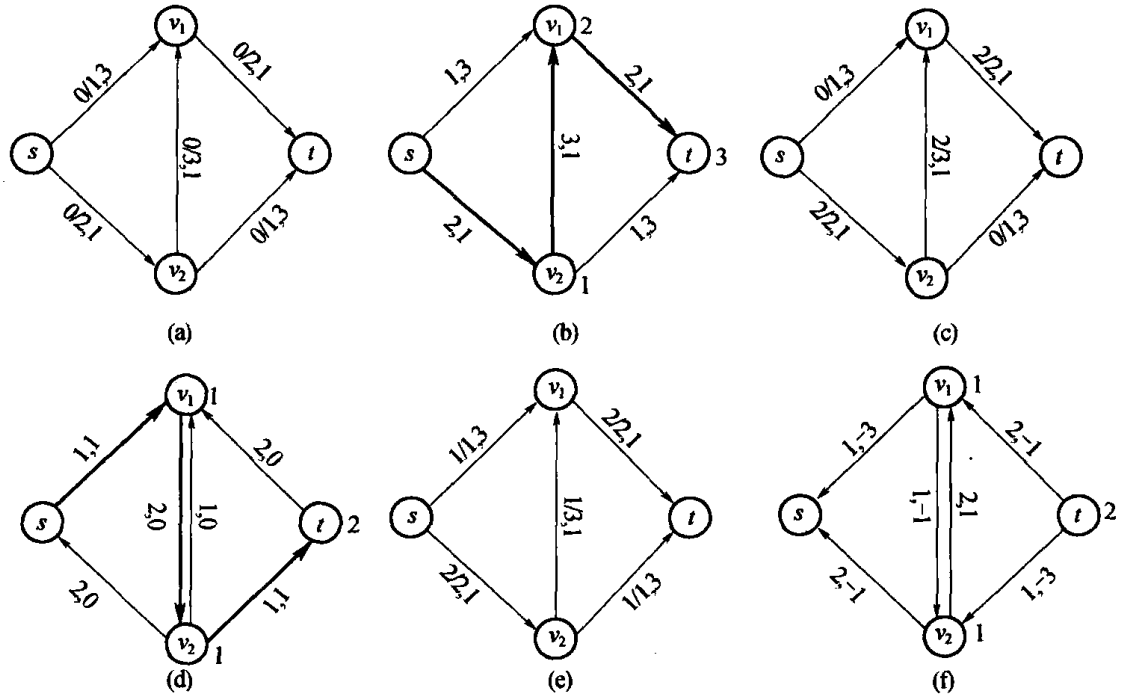


图 9.15

9.3 匹配问题

匹配问题(Matching Problem)在资源规划、机器调度以及人员分配等领域有许多实际的应用,下面介绍一些基本概念。

定义 9.14 给定一个无向图 $G=(V,E)$,一个匹配是一个边的集合 $M\subseteq E$,使得对所有顶点 $v\in V$,M 中至多有一条边与 v 相连。如果 M 中某条边与顶点 v 相连,则说顶点 v 被 M 匹配;否则说 v 是未匹配的。称 M 的边为匹配边,E 中不在 M 的边为未匹配边。

匹配 M 的大小 $|M|$,即 M 中边的条数,最大匹配问题就是在图 $G=(V,E)$ 中找出 $|M|$ 最大的集合 M。

定义 9.15 给定一个无向图 $G=(V,E)$,设 M 是图 G 的一个匹配,如果 G 的每一个顶点被 M 匹配,则称 M 是完全匹配(Perfect Matching)。

一般地,并不是每个无向图 $G=(V,E)$ 都存在完全匹配,即存在最大匹配的图 G 不一定存在完全匹配。反过来,如果图 G 存在完全匹配,则该完全匹配一定是最大匹配。

定义 9.16 给定一个无向图 $G=(V,E)$,设 M 是图 G 的一个匹配,则称 G 中由匹配边和未匹配边交错出现的一条简单路径为一条关于 M 的交错路径。如果交错路径 p 的起点和终点重合,则该路径称为交错回路。如果交错路径 p 的起点和终点都不是匹配顶点,则该路径称为关于 M 的一条增广路径。

例如，给定一个无向图 $G=(V,E)$，如图 9.16 所示，图 9.16(a) 中粗线给出了一个大小为 $|M|=3$ 的匹配，$M=\{(v_1,v_6),(v_2,v_7),(v_4,v_8)\}$，$M$ 的边为匹配边，其他边均为未匹配边。灰色顶点 v_1,v_2,v_6,v_7,v_8,v_4 被 M 匹配，白色顶点未匹配。路径 v_1,v_2,v_7,v_8,v_4,v_3 是交错路径，交错路径 v_1,v_2,v_7,v_6,v_1 为交错回路。交错路径 $v_5,v_6,v_1,v_2,v_7,v_8,v_4,v_3$ 为增广路径。图 9.16(b) 中粗线给出了一个大小为 $|M|=4$ 的匹配，$M=\{(v_1,v_2),(v_6,v_5),(v_7,v_8),(v_3,v_4)\}$，该匹配是最大匹配，而且是完全匹配。

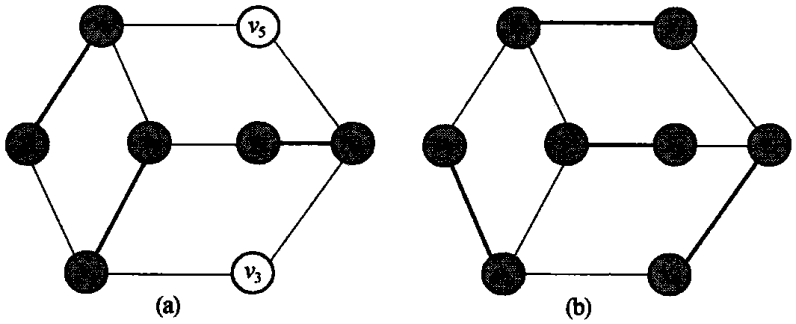


图 9.16

定义 9.17 给定两个边的集合 M_1 和 M_2，定义 $M_1 \oplus M_2=(M_1 \cup M_2)-(M_1 \cap M_2)$。

根据定义 9.17 可知，$M_1 \oplus M_2$ 是边的集合，其中边或者在集合 M_1 中，或者在集合 M_2 中，但不能同时在 M_1 和 M_2 中。例如图 9.16(a) 中，$M=\{(v_1,v_6),(v_2,v_7),(v_4,v_8)\}$，增广路径 $p=\{(v_5,v_6),(v_6,v_1),(v_1,v_2),(v_2,v_7),(v_7,v_8),(v_8,v_4),(v_4,v_3)\}$。则 $M \oplus p=\{(v_1,v_2),(v_6,v_5),(v_7,v_8),(v_3,v_4)\}$，恰好是图 9.16(b) 的最大匹配。值得注意的是，这里增广路径用边的集合表示，由于考虑的是无向图，因此边 (v_i,v_j) 与边 (v_j,v_i) 视为同一条边。

引理 9.6 给定一个无向图 $G=(V,E)$，设 M 是图 G 的一个匹配，p 是关于 M 的一条增广路径，则 $M \oplus p$ 是图 G 的一个匹配，且有 $|M \oplus p|=|M|+1$。

证明：首先证明 $M \oplus p$ 是图 G 的一个匹配，只要证明 $M \oplus p$ 的任意两条边没有公共的顶点即可。用反证法证明。假设 $M \oplus p$ 存在两条边有公共的顶点，则这两条边中一定有一条边属于 M，另一条边属于 p，即会出现如图 9.17 所示的情形，其中 $M \oplus p$ 的边集是图中去掉打叉的边后剩下的边集，v_3 是 $M \oplus p$ 中边 (v_3,v_4) 和 (v_3,v_6) 的公共顶点，值得注意的是，(v_3,v_6) 一定是匹配边，否则，它不会出现在 $M \oplus p$ 中。这表明 M 中有两条边与 v_3 相连，与匹配的定义 9.14 矛盾，故假设不成立。

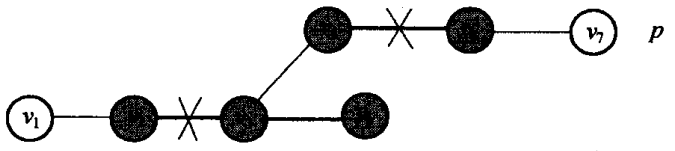


图 9.17

由 $M \oplus p$ 的定义 9.17 可知

$$|M \oplus p| = |M \cup p| - |M \cap p| = |M| + |p| - |M \cap p| - |M \cap p|$$
$$= |M| + |p| - 2|M \cap p|$$

又由于 p 是关于 M 的一条增广路径,因此有

$$|p| = 2|M \cap p| + 1$$

综合上面两个式子,可得 $|M \oplus p| = |M| + 1$。故所证成立。

定理 9.9 给定一个无向图 $G = (V, E)$,图 G 的一个匹配 M 是最大匹配的充要条件是 G 不包含关于 M 的增广路径。

证明:首先证明如果图 G 的一个匹配 M 是最大匹配,则 G 不包含关于 M 的增广路径。假设 G 包含关于 M 的增广路径,则可以根据引理 9.6,构造出一个比 M 还大的匹配,这与 M 是最大匹配矛盾,故结论成立。

下面证明 G 不包含关于 M 的增广路径,则 M 是最大匹配。假设 M 不是最大匹配,即存在一个比 M 还大的匹配,不妨设为 M'。考虑子图 $G' = (V, M \oplus M')$,则图 G' 中每个顶点至多连接两条边,而且图 G' 中某个顶点连接的两条边一定分属于集合 M 和 M'。这样,G' 中任何连通子图要么是一个孤立顶点,要么是一条路径,要么是一条具有偶数条边的回路,且所有这些路径是关于 M 和 M' 的交错路径。值得注意的是,连通子图不会是具有奇数条边的回路,否则回路中总有相邻的边来自于同一个匹配。对于每个回路,它来自 M' 的边数一定等于来自 M 的边数。因此,一定有一条具有奇数条边的路径,它来自 M' 的边数一定大于来自 M 的边数,而且路径的起点和终点是未匹配的,即这条路径是关于 M 的增广路径,与已知矛盾。故所证成立。

类似可以证明下列结论,分别见习题 9.15,9.16。

推论 9.3 给定一个无向图 $G = (V, E)$,M 和 M' 是图 G 的两个匹配,且有 $|M'| > |M|$,则 $M \oplus M'$ 至少包含 $|M'| - |M|$ 条顶点不相交的关于 M 的增广路径。

定理 9.9 给定一个无向图 $G = (V, E)$,M 是图 G 的一个匹配,s 是未匹配顶点,如果无向图中不存在以 s 为起点的增广路径,则图 G 中存在最大匹配 M^*,使得 s 关于匹配 M^* 也是未匹配顶点。

根据定理 9.9,我们可以得到下列找最大匹配(Maximum Matching)的一般方法:

```
MaximumMatching(G)
1  M←0
2  repeat
3      find an augmenting path p
4      M←M ⊕ p
5  until no augmenting path in G
6  return M
```

9.3.1 二分图匹配

现在我们考虑二分图的最大匹配问题。假定图 $G=(V,E)$ 的顶点能够被划分为两个顶点集合 L 和 R，即 $V=L\cup R$，且 L 的顶点没有边相连，R 的顶点也没有边相连。E 的边都跨过集合 L 和 R，V 的每个顶点至少有一条边与之相连。二分图的最大匹配问题就是找 $|M|$ 最大的一个匹配集合 M。如图 9.18 所示，$V=L\cup R$，图 9.18(a)中，$|M|=3$；图9.18(b)中，$|M|=4$。

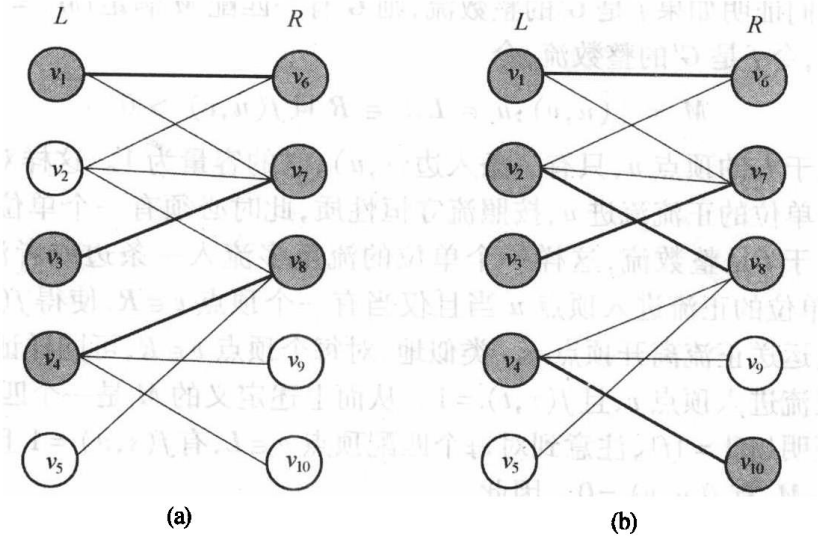


图 9.18

二分图匹配问题有许多实际的应用，例如给定一个机器的集合 L 和一个任务的集合 R，对于边 (u,v)，$u\in L,v\in R$ 意味着特定的机器 u 能够执行指定的任务 v，我们的目的是希望找到最大的匹配。下面我们介绍寻找二分图最大匹配的算法。

1. *最大流方法*

对于二分图(Bipartite Graph)的最大匹配问题，我们可以将该问题转化为最大流问题，从而可以利用最大流算法来求解。

给定二分图 $G=(V,E)$，相应的网络 $G'=(V',E')$ 可以构造如下：设源点 s 和汇点 t 是不属于 V 的新顶点，$V'=V\cup\{s,t\}$。如果 G 的顶点划分为 $V=L\cup R$，G'的有向边由下式给出

$$E'=\{(s,u):u\in L\}$$
$$\cup\{(u,v):u\in L,v\in R\text{且}(u,v)\in E\}$$
$$\cup\{(v,t):v\in R\}$$

同时，对 E'的每条边赋单位容量。

定理 9.11 给定二分图 $G=(V,E)$，其顶点划分为 $V=L\cup R$。设 $G'=(V',E')$ 是按照上述方法构造的网络。如果 M 是 G 的匹配，则 G'中存在一整数流 f，且

$|f| = |M|$；反过来，如果f是G'的整数流，则G中有一匹配M满足$|M| = |f|$。

证明：我们首先证明匹配M对应G'中一个整数流。定义流f如下：如果$(u,v) \in M$，则$f(s,u) = f(u,v) = f(v,t) = 1$且$f(u,s) = f(v,u) = f(t,v) = -1$。对所有其他的边$(u,v) \in E'$，定义$f(u,v) = 0$。容易验证流$f$满足流的三个性质。

由流f的定义9.2可知，对于匹配M的边$(u,v) \in M$对应G'的一个单位流，而且流过的路径为s,u,v,t。这些由匹配边引出的路径，除了源点和汇点外，其他顶点是独立的。这样，越过割$(L \cup \{s\}, R \cup \{t\})$的流量值等于$|M|$，按照引理9.3可得，$|f| = |M|$。

现在我们证明如果f是G'的整数流，则G有一匹配M满足$|M| = |f|$。为了证明这个结论，令f是G'的整数流，令

$$M = \{(u,v): u \in L, v \in R \text{ 且 } f(u,v) > 0\}$$

每个属于L的顶点u，只有一条入边(s,u)，它的容量为1。这样对每个$u \in L$，至多有1个单位的正流流进u，按照流守恒性质，此时必须有一个单位的正流流出u。而且，由于f是整数流，这样一个单位的流至多流入一条边或者流出一条边。因此，一个单位的正流进入顶点u当且仅当有一个顶点$v \in R$，使得$f(u,v) = 1$，且至多一条边运送正流离开顶点u。类似地，对每个顶点$v \in R$，可同样证明最多有一个单位的正流进入顶点v，且$f(v,t) = 1$。从而上述定义的M是一个匹配。

为了证明$|M| = |f|$，注意到对每个匹配顶点$u \in L$，有$f(s,u) = 1$且对于每条边$(u,v) \in E - M$，有$f(u,v) = 0$。因此

$$|M| = f(L,R) = f(L,V') - f(L,L) - f(L,s) - f(L,t)$$

由流守恒性质可得$f(L,V') = 0$，由反对称性质可得$-f(L,s) = f(s,L)$，由于t与L的顶点没有边相连，因此有$f(L,t) = 0$，由割的定义9.7，有$f(L,L) = 0$，可得

$$|M| = f(s,L) = |f|$$

故所证成立。

根据定理9.11，我们可以推出二分图G的最大匹配对应其相应网络流G'的最大流，因此我们可以利用前面介绍的求最大流的方法，求出G'的最大流，从而得到二分图G的最大匹配。

2. *匈牙利树方法*

MaximumMatching(G)算法给出了找最大匹配的方法，但是该算法并没有具体告诉我们如何找增广路径。定理9.10则告诉了我们如何寻找增广路径：从G的任意匹配(例如空匹配)M开始，首先对G中关于M的一个未匹配点，寻找关于M的一条增广路径。若关于M的所有未匹配点都不存在增广路径，则M为最大匹配；若存在，则将关于M的增广路径的匹配边与未匹配边互换，从而得到比M多一条边的匹配M'，再对M'重复上述过程。下面我们介绍如何利用上述方法来求解二分匹配问题。

给定二分图$G = (V,E)$，其顶点划分为$V = L \cup R$，M为G的匹配。下面我们介

绍一下如何构造一棵交错路径树(alternating path tree)。首先从 L 中找一个未匹配顶点 r,从 r 开始,加上一条连接顶点 r 的未匹配边(r,b),然后再找一条连接顶点 b 的匹配边(b,c),这样未匹配边和匹配边交替地加入,直到这条路径不再扩大,即路径已经遇到一个未匹配顶点或者所有的顶点都已经加入树中。再从顶点 r 出发,按照上述过程,搜索另一条交错路径。重复上述过程,直到从顶点 r 出发的所有交错路径都已经被搜索出为止。上述过程中,如果一条交错路径以未匹配顶点结束,则该交错路径是一条增广路径。如果交错路径树的叶子顶点均是匹配顶点,则交错路径树称为匈牙利树(Hungarian tree)。上述找交错路径树的方法是深度优先搜索,当然我们可以采用宽度优先搜索,来构造交错路径树。下面我们给出求最大二分匹配的匈牙利算法:

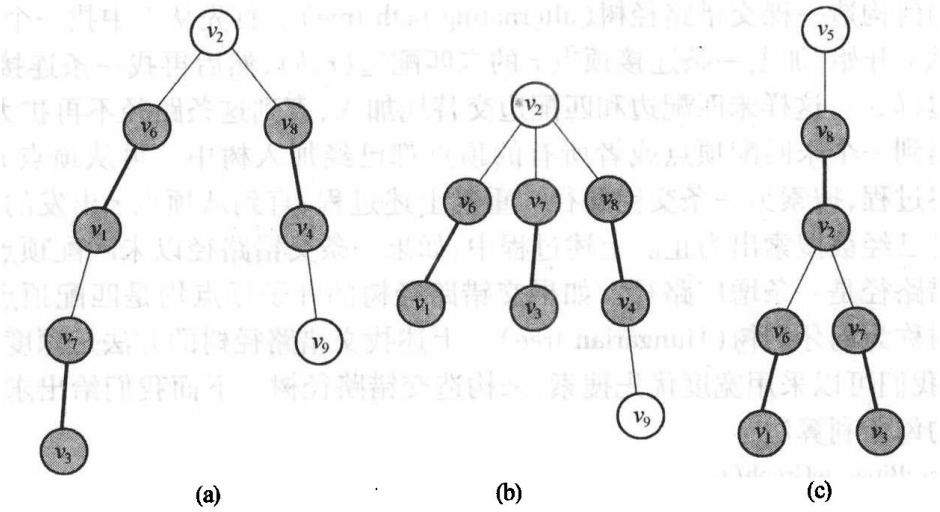
```
HungryBipartiteGraph(G)
1  begin with any matching M
2  while there exists an unmatched vertex in L and R respectively do
3     let r is an unmatched vertex in L
4     grow an alternating path tree T with root r using breadth first search
5     if T is a Hungarian tree then
6        G←G - T
7     else
8        find an augmenting path p in T
9        M←M ⊕ p
10 return M
```

HungryBipartiteGraph(G)算法的运行时间主要用于构造交错路径树,用宽度优先搜索需要 $O(|E|)$,由于匹配 M 的未匹配顶点最多有 $O(|V|)$,因此算法最多能构造 $O(|V|)$棵交错路径树。因此我们有下列结论。

定理 9.12 HungryBipartiteGraph(G)算法的时间复杂度是 $O(|V||E|)$。

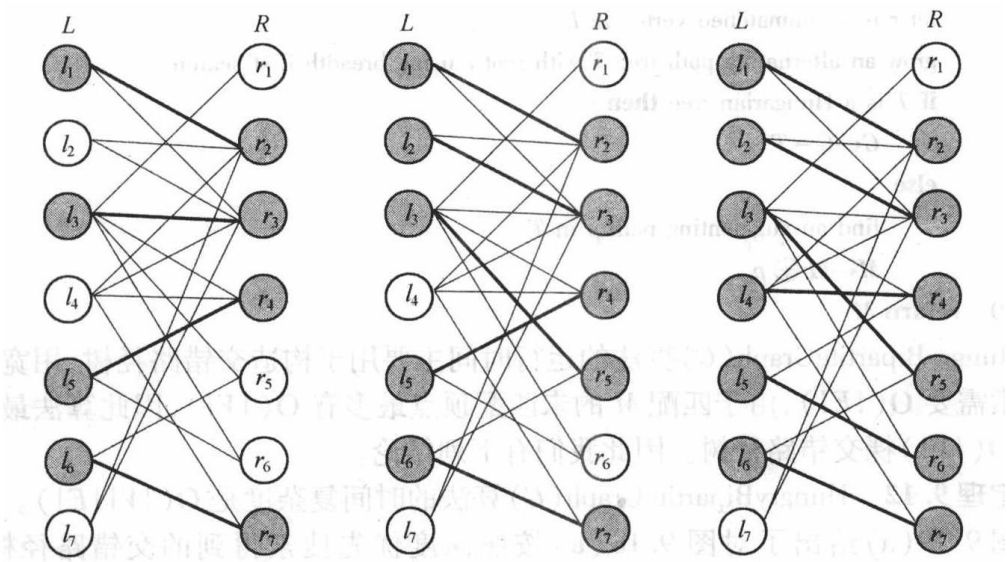
图 9.19(a)给出了对图 9.18(a)按照深度优先搜索得到的交错路径树,图 9.19(b)给出了对图 9.18(a)按照宽度优先搜索得到的交错路径树。图 9.19(c)给出了对图 9.18(b)按照宽度优先搜索得到的交错路径树,该树是一棵匈牙利树。从图中我们看出,在寻找交错路径树时,用宽度优先搜索更有效,因为它能帮助我们更快地找到一条增广路径。

现在给出一个例子,说明 HungryBipartiteGraph(G)算法找交错路径树,并对其中的增广路径进行增广的过程。例如,对于图 9.20(a),其交错路径树为图 9.20(d),树中有一条增广路径 $p=\{(l_2,r_3),(r_3,l_3),(l_3,r_5)\}$。将路径 p 的匹配边与未匹配边互换,得到图 9.20(b),其交错路径树如图 9.20(e)所示,树中有一条增广路径 $p=\{(l_4,r_4),(r_4,l_5),(l_5,r_6)\}$。将路径 p 的匹配边与未匹配边互换,得到图 9.20(c),其交错路径树如图 9.20(f)所示,为匈牙利树,算法终止,得到匹配 $M=\{(l_1,r_2),(l_2,r_3),(l_3,r_5),(l_4,r_4),(l_5,r_6),(l_6,r_7)\}$。

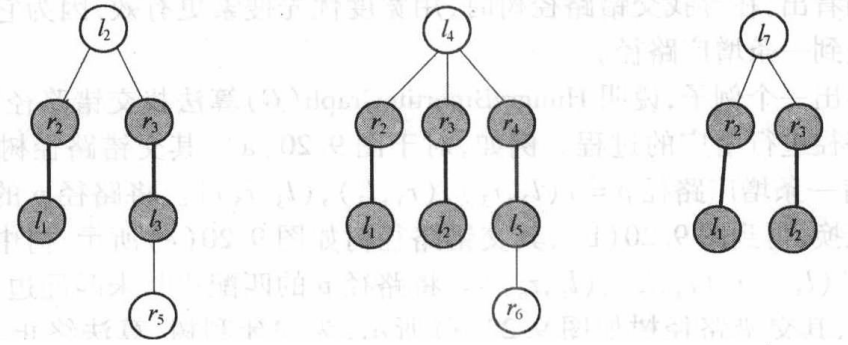


(a) (b) (c)

图 9.19



(a) (b) (c)



(d) (e) (f)

图 9.20

9.3.2 一般图的匹配

上一节中我们介绍了二分图的匈牙利算法，不难看出该算法不适合求解一般图的匹配问题。主要是因为一般图中可能存在含有奇数条边的回路，而在二分图中是不存在这样的回路的。对于图 9.21，如果从未匹配点 v_1 出发搜索交错路径树，将得到如图 9.21(b)所示的匈牙利树，算法停止。事实上，图 9.21(a)中存在一条增广路径 $v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8$。之所以出现这种找不到增广路径的情形，是因为图中存在一种花的结构。下面我们给出花的有关定义。

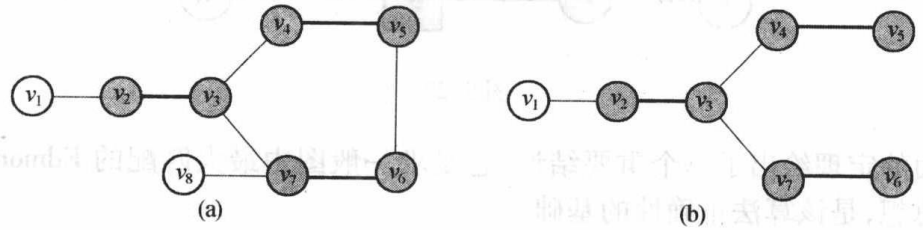


图 9.21

定义 9.18 给定一个无向图 $G=(V,E)$，M 是图 G 的一个匹配，花茎是一条由偶数条未匹配边和匹配边构成的交错路径。如果花茎的起点等于花茎的终点，则花茎为空。花是一条由奇数条未匹配边和匹配边构成的交错回路，花的起点和终点均为花茎的终点，除了花茎的终点外，花与花茎没有其他的公共顶点。花的起点和终点是同一顶点，这个顶点称为花基。

图 9.22 给出了两朵花，其中图 9.22(a)中花为 $v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_1$，花茎为空，花基为 v_1，图 9.22(b)花为 $v_3, v_4, v_5, v_6, v_7, v_3$，花茎为 v_1, v_2, v_3，花基为 v_3。

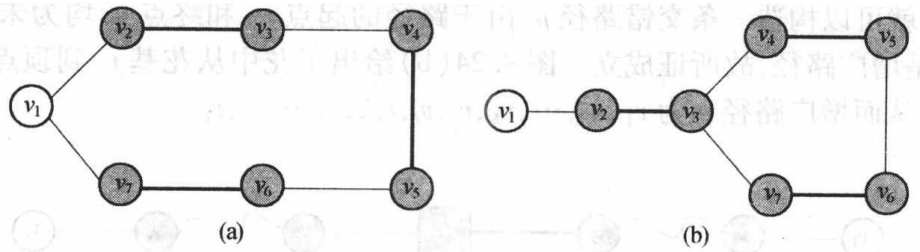


图 9.22

对于含有花的图，为了找到其增广路径，Edmonds J. 提出了一种非常大胆而有趣的想法，就是将花压缩成一个超级顶点，然后在图中继续寻找增广路径。

给定一个无向图 $G=(V,E)$，M 是图 G 的一个匹配，令 B 为超级顶点，用方形顶点表示，代表图 G 的一朵花。令 G' 表示将图 G 的一朵花压缩后得到的图。压缩花就是将花中的所有顶点用超级顶点 B 代替，同时保留花与其他顶点相连的边，删除花中的边。图 9.23 给出一个压缩图 9.21(a)中花的例子。花 $v_3, v_4, v_5, v_6, v_7, v_3$ 被超级顶点 B 代替，与花相连的边留下，花中的边被删除。这样我们就找到了一条

增广路径 v_1, v_2, B, v_8。为了得到图 9.21(a)的增广路径,我们只需要展开超级顶点 B 即可,由于花中与 v_8 相连的顶点是 v_7。由于 B 代表的花从花基 v_3 有两条路径过顶点 v_7 可以到达顶点 v_8,一条是具有奇数条边的路径 v_3, v_7,另一条是具有偶数条边的交错路径 v_3, v_4, v_5, v_6, v_7,我们只需要将具有偶数条边的路径直接取代增广路径 v_1, v_2, B, v_8 的超级顶点 B,从而得到图 9.21(a)的增广路径 $v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8$。事实上,上述扩展花的方法具有一般性。其合理性也可见定理 9.13 的证明过程。

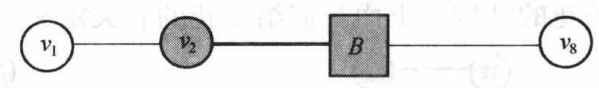


图 9.23

下面的定理给出了一个重要结论,它是求一般图中最大匹配的 Edmonds 算法的核心思想,是该算法正确性的基础。

定理 9.13 令 G'是将图 G 的花压缩后得到的图,则 G'含有一条增广路径当且仅当 G 含有一条增广路径。

证明:首先证明 G'含有一条增广路径,则 G 含有一条增广路径。令 G'的一条增广路径为 p',现在分两种情形给予证明。假设 p'与压缩花后的超级顶点 B 不相交,那 p'一定也是 G 的一条增广路径,结论显然成立。假设 p'与压缩花后的超级顶点 B 相交。不妨设路径 p'在与 B 相交前的第一个顶点为 v_h,相交后的第一个顶点为 v_n,根据花茎和花的定义,9.18 则(v_h, B)一定是匹配边,(B, v_n)一定是未匹配边,如图 9.24(a)所示。假设 B 与 v_n 相连的顶点为 v_k,路径 p'的起点为 v_1,终点为 v_t。因为(B, v_n)是未匹配边,只需要花中从花基到花中顶点 v_k 的交错路径具有偶数条边,就可以构造一条交错路径 p,由于路径的起点 v_1 和终点 v_t 均为未匹配点,因此 p 是增广路径,故所证成立。图 9.24(b)给出了花中从花基 v_i 到顶点 v_k 的交错路径,从而增广路径 p 为 $v_1, v_2, \cdots, v_h, v_i, v_j, v_k, v_n, \cdots, v_s, v_t$。

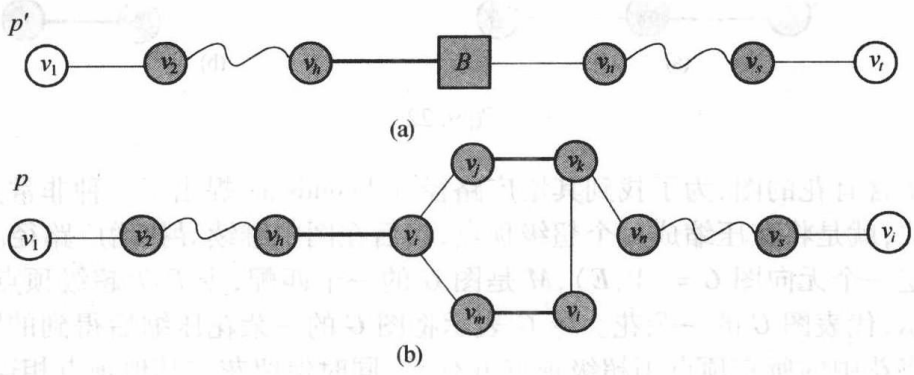


图 9.24

反过来，我们可以类似证明，见习题9.17。

定理9.13表明，如果图G有一条增广路径，对图G的花进行压缩后得到图G'，G'中仍然有一条增广路径。因此要在一般图中找到最大匹配，就看图中是否能够寻找到一条增广路径。为了找到增广路径，我们可以修改一下二分图的Hungry-BipartiteGraph(G)算法，使之可以发现花。一旦发现花，我们就将花压缩成一个超级顶点，然后在新图中继续寻找增广路径。如果图中还存在花，则继续上述过程，直到图中找到一条增广路径或者找不到增广路径。如果找到G'的增广路径，则可以把该路径上经过的超级顶点展开，以便寻找到图G的增广路径，从而得到新的匹配。如果找不到增广路径，则根据定理9.9，可知找到了最大匹配。

给定一个无向图$G=(V,E)$，M是图G的一个匹配。令U表示未匹配顶点的集合，T表示交错路径树。令B为超级顶点。为了方便找到花，我们可以从一个未匹配的顶点开始，依次对搜索过的顶点和边进行标记(mark)，匹配顶点被依次标记为内部的(inner)和外部的(outer)。

下面给出详细的算法：

```
GeneralGraphMatching(G)
1  begin with any matching M
2  maximum←False
3  while maximum = False do
4    determine the set of unmatched vertices U with respect to M
5    augment←False
6    while U≠∅ and augment = False do
7      empty stack, unmark edges and remove labels from vertices
8      let u be a vertex U
9      U←U - {u}; T←u
10     label u outer
11     hungarian←False
12     while augment = False do
13       choose an outer vertex u and an unmarked edge (u,v)
14       if (u,v) exists then mark (u,v)
15       else
16         hungarian←True
17         exit this while loop and go to line 28
18       if v is inner then do nothing
19       else if v is outer then
20                 place the blossom on top of the stack, shrink it
21                 replace the blossom with a vertex B and label B outer
22           else if v is an unmatched vertex then
23                   augment←True
24                   U←U - {v}
```

```
25                    else
26                        let (v,w) be in M and add (u,v) and (v,w) to T
27                        label v inner and w outer
28        if hungarian = True then G←G - T
29        else if augment = True then
30                construct p by popping blossoms from the stack
31                expand them and add the even - length path
32                M←M ⊕ p
33    if augment = False then maximum←True
34 return M
```

GeneralGraphMatching(G)算法主要包括三层 **while** 循环，最外层 **while** 循环（第3行～第33行）主要判断是否找到了最大匹配，由 maximum 判断。中间层 **while** 循环（第6行～第32行）判断未匹配顶点集合 U 是否为空，以及是否找到一条增广路径，由 augment 判断。内层 **while** 循环（第12行～第27行）通过逐步加入边来寻找增广路径，在每次循环中，它选择一个外部（outer）顶点 u 以及未标记（unmarked）的边 (u, v)。如果这样的边不存在，则找到了一棵匈牙利树（hungarian tree），退出内层循环。否则对边进行标记，同时根据顶点 v 的标记情况，依次判断：

（1）如果 v 是内部的（inner），则选择的这条边没有什么用。

（2）如果 v 是外部的（outer），则说明找到了花，对花进行压缩。

（3）如果 v 是未匹配的（unmatched），则说明找到了一条增广路径。

（4）否则，继续增广交错路径树。

第28行中，如果找到了一棵匈牙利树，则可以将该树从图中删除，而不影响匹配的大小。否则（第29行），对花进行扩展，以便构造出原图的增广路径。

下面我们分析 GeneralGraphMatching(G)算法的时间复杂度。由于搜索中可能找到花，而花至少含有三个顶点，因此每次搜索使得顶点数至少减少两个，因此每次搜索到一条增广路径所执行的压缩次数最多不超过 $|V|/2$ 次，展开超级顶点的次数也不超过 $|V|/2$ 次，因此压缩和展开花所需要的时间最多为 $O(|V|^2)$，而寻找一条增广路径和用这条路径来增广当前的匹配所需要的时间为 $O(|E|)$，因此我们有下列结论。

定理 9.14 GeneralGraphMatching(G)算法的时间复杂度为 $O(|V|^3)$。

下面我们给出 GeneralGraphMatching(G)算法计算一个实例的过程。给定一个无向图 $G=(V,E)$，M 是图 G 的一个匹配，如图 9.25(a)所示。$U=\{v_1, v_{10}\}$，从 U 中选择 v_1，此时 $U=\{v_{10}\}$，v_1 标记为外部的，选择 (v_1, v_2)，标记 (v_1, v_2)。v_2 未标记，执行第26行，加入边 (v_2, v_3)，执行第27行，v_2 标记为内部的，v_3 标记为外部的。然后选择 (v_3, v_5)，并标记它，v_5 未标记，故加入边 (v_5, v_4)，v_5 标记为内部的，v_4 标记为外部的。选择 v_4，选择 (v_4, v_3)，由于 v_3 标记为外部，找到了花 v_3, v_5, v_4, v_3，将其压

缩为超级顶点 B_1，并标记 B_1 是外部的，如图 9.25(b)所示。B_1 是外部的，继续加入边(B_1，v_7)，并标记它，由于 v_7 未标记，故加入(v_7，v_6)，并标记 v_7 为内部的，v_6 标记为外部的，加入边(v_6，B_1)，由于 B_1 是外部的，故找到了花，将其压缩为超级顶点 B_2，如图 9.25(c)所示。

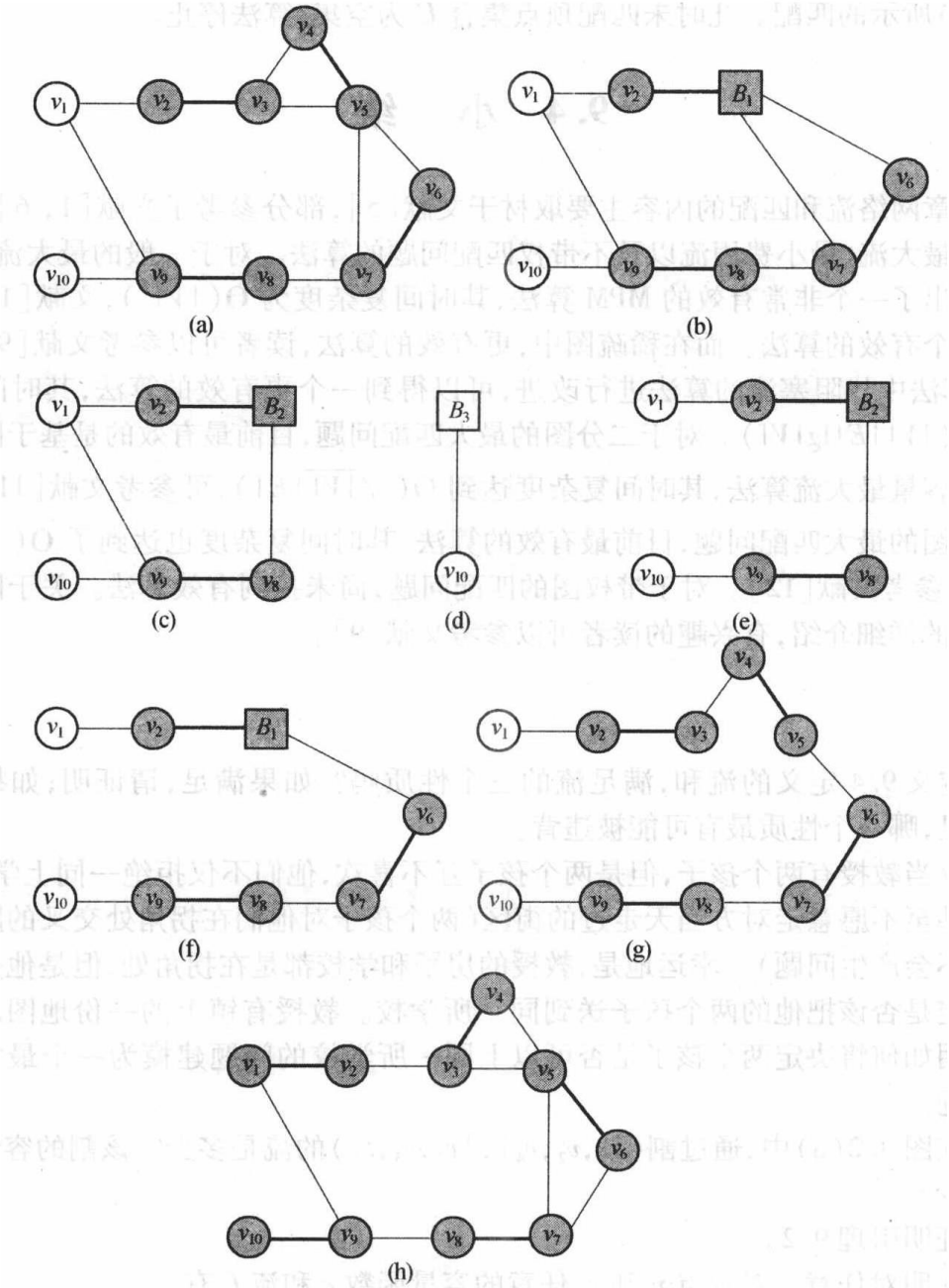


图 9.25

类似上述过程，我们可以得到图 9.25(d)。找到增广路径 B_3，v_{10}。为了构造出图 9.25(a)的增广路径，我们先对图 9.25(d)的超级顶点进行展开。由于顶点 v_{10}

与超级顶点 B_3 代表的顶点 v_9 相连，我们只需要将花 B_3 中从花基 v_1 到 v_9 的偶数条边的交错路径代替 B_3 即可，从而得到图 9.25(e)所示的路径。类似地，将 B_1, v_6, v_7 代替 B_2，得到如图 9.25(f)所示的路径。将 v_3, v_4, v_5 替换 B_1 可以得到图 9.25(g)所示的路径，从而找到了图 9.25(a)所示的增广路径，对其进行增广，得到如图 9.25(h)所示的匹配。此时未匹配顶点集合 U 为空集，算法停止。

9.4 小 结

本章网络流和匹配的内容主要取材于文献[5]，部分参考了文献[1, 6]，主要介绍了最大流、最小费用流以及不带权匹配问题的算法。对于一般的最大流问题，本章给出了一个非常有效的 MPM 算法，其时间复杂度为 $O(|V|^3)$，文献[1]也给出了几个有效的算法。而在稀疏图中，更有效的算法，读者可以参考文献[9]。对 Dinic 算法中找阻塞流的算法进行改进，可以得到一个更有效的算法，其时间复杂度为 $O(|V||E|\lg|V|)$。对于二分图的最大匹配问题，目前最有效的是基于网络流的单位容量最大流算法，其时间复杂度达到 $O(\sqrt{|V|}|E|)$，可参考文献[11]。对于一般图的最大匹配问题，目前最有效的算法，其时间复杂度也达到了 $O(\sqrt{|V|}|E|)$，可参考文献[12]。对于带权图的匹配问题，尚未找到有效算法。关于网络流和匹配的详细介绍，有兴趣的读者可以参考文献[9]。

习题

9.1 定义 9.4 定义的流和，满足流的三个性质吗？如果满足，请证明；如果不满足，哪一个性质最有可能被违背。

9.2 亚当教授有两个孩子，但是两个孩子互不喜欢，他们不仅拒绝一同上学，而且甚至不愿意走对方当天走过的街区(两个孩子对他们在拐角处交叉的路径并不会产生问题)。幸运地是，教授的房子和学校都是在拐角处，但是他并不确定是否该把他的两个孩子送到同一所学校。教授有镇上的一份地图。试说明如何将决定两个孩子是否可以上同一所学校的问题建模为一个最大流问题。

9.3 在图 9.2(a)中，通过割($\{s, v_2, v_4\}, \{v_1, v_3, t\}$)的流是多少？该割的容量是多少？

9.4 证明引理 9.2。

9.5 证明对任意一对顶点 u 和 v、任意的容量函数 c 和流 f，有

$$c_f(u,v) + c_f(v,u) = c(u,v) + c(v,u)$$

9.6 给定一个网络 $G=(V,E)$，证明 G 的最大流总可以被至多由 $|E|$ 条增广路径所组成的序列找到。(提示：找出最大流后再确定路径。)

9.7 若一个网络中所有的容量值都不同，则存在一个唯一的最小割，它把源点和

汇点分割开。该结论正确吗？如果正确，请证明；否则，请说明理由。

9.8 说明如何有效地在一个给定的剩余网络中，找到一条增广路径。

9.9 设计一个有效的算法，它在一个给定的有向无回路图中寻找具有最大瓶颈容量的增广路径。

9.10 设计一个有效算法以找出一个给定的有向无回路图的层次图(level graph)。

9.11 针对多个源点和多个汇点的问题，扩展流的性质和定义。证明多个源点和多个汇点的流网络的任意流，均对应于通过增加一个超级源点和超级汇点所得到的单源点单汇点的流，且流值相同。反之亦然。

9.12 在9.1节中，我们通过增加具有无限容量的边，把一个多源点多汇点的流网络转换为单源点单汇点的流网络。证明如果初始的多源点多汇点网络的边具有有限的容量，则转换后所得到的网络的任意流均为有限值。

9.13 假定多源点多汇点问题中，每个源点 s_i 产生 p_i 单位的流，即 $f(s_i,V)=p_i$。同时假定每个汇点 t_j 消耗 q_j 单位的流，即 $f(V,t_j)=q_j$，其中 $\sum p_i=\sum p_j$。说明如何把寻找一个流 f 以满足这些附加条件的问题转化为在一个单源点单汇点流网络中寻找最大流的问题。

9.14 有向图 $G=(V,E)$ 的一个路径覆盖是一个顶点不相交路径的集合 p，且 V 的每一个顶点仅属于 p 的一条路径。路径可以开始和结束于任意顶点，且长度也为任意值，包括0。G 的一个最短路径覆盖是指包含尽可能少的路径数的路径覆盖。

(1) 设计一个有效算法以找出有向无回路图 $G=(V,E)$ 的一个最短路径覆盖。(提示：假设 $V=\{1,2,\cdots,|V|\}$，构造图 $G'=(V',E')$，其中：

$$V'=\{x_0,x_1,\cdots,x_{|V|}\}\cup\{y_0,y_1,y_2,\cdots,y_{|V|}\}$$

$$E'=\{(x_0,x_i):i\in V\}\cup\{(y_i,y_0):i\in V\}\cup\{(x_i,y_j):(i,j)\in E\}$$

然后运行最大流算法。)

(2) 说明所给的算法是否适用于包含回路的有向图。

9.15 证明推论9.3。

9.16 证明定理9.10。

9.17 证明定理9.13结论的第二部分：G 含有一条增广路径，则 G' 含有一条增广路径。

9.18 史密斯教授评审了 n 篇论文，他将每篇论文的评审结果分别用 n 个信封将信(评审结果)装好，准备第二天邮寄出去。不幸的是，史密斯调皮的儿子小史密斯当晚就把这 n 封信都拿出了信封，玩过之后，他不知道如何将拿出的信正确地装回信封中了。着急的小史密斯只有求助于你，假设他所提供的 n 封信依次编号为 $1,2,\cdots,n$，且 n 个信封也依次编号为 $1,2,\cdots,n$。小史密斯唯一能提供的信息是：第 i 封信肯定不是装在信封 j 中。请设计一个有效的算法，帮助小史密斯尽可能多地将信正确地装回信封中。

9.19 在东方,男女相爱讲究的是缘分。只要月下老人做媒,就能千里姻缘一线牵,无论他们身处何地。而在西方,只能借助爱神丘比特之箭,射中的两个男女才能相爱。假定丘比特之箭的射程为 d,给定 n 对男女,已知每个男女的位置(x,y),以及男女之间的缘分值,请设计一个算法,使得每对男女射中一次,且每一对被射中的男女之间的缘分值的和最大。注意箭的轨迹只能是一条直线。

9.20 给定一个无向图 $G=(V,E)$,M 是图 G 的一个匹配,如图 9.26 所示。试写出算法 GeneralGraphMatching(G),求该图最大匹配的过程。

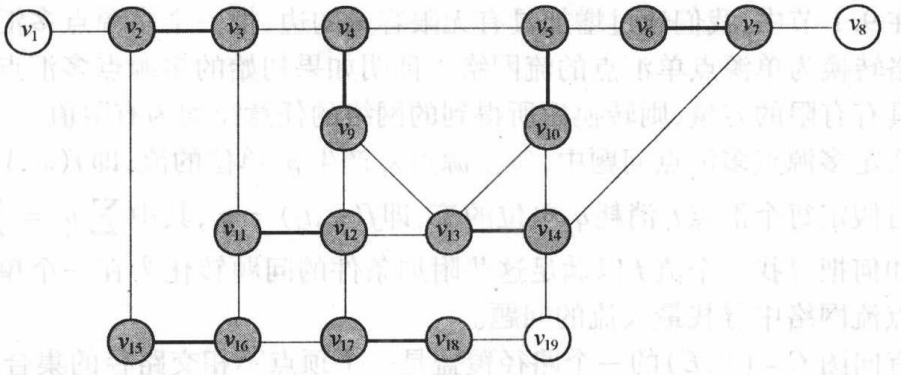


图 9.26

实验题

9.21 编程实现求解最大流问题的最短路径增广算法、Dinic 算法以及 MPM 算法,并用实验分析方法比较三种算法的效率。

9.22 完成 XOJ1089。

9.23 完成 POJ 如下题目:1087,1149,1273,1274,1325,1459,1637,1698,1719,2195,2239,2516,2771,3020,3041。

第 10 章　线性规划

在工业以及管理工程等领域中的许多实际问题，都可以建模为线性规划问题或者整数规划问题。对线性规划问题，利用单纯形等算法是可以很快找到其最优解的。对于整数规划问题，我们是无法在多项式时间内获得它的最优解。线性规划问题与整数规划问题密不可分，对整数规划问题的求解，可以转化为对线性规划问题的求解，从而有效地获得整数规划问题的近似解，进而快速求解一大批 NP 难问题，满足实际领域的需要。因此，下面介绍线性规划问题和整数规划问题的有关知识。

10.1　线性规划问题

把优化问题的目标函数指定为若干个变元的线性函数，并以等式或不等式的形式约束变量的范围，便可以建立起线性规划(Linear Programming，LP)模型。

例如，某工厂计划在下一季度生产 $A_1, A_2, \cdots, A_n$ 种产品，要消耗 $B_1, B_2, \cdots, B_m$ 种资源，已知产品 j 消耗资源 i 的数量为 a_{ij}，资源 i 的数量限制 b_i 以及产品 j 可获得的利润 c_j，问如何安排生产计划，才能充分利用现有的资源，使获得的总利润最大？

令产品 j 生产的数量为 x_j，则上述问题可以形式化如下

$$\max \quad \sum_{j=1}^{n} c_j x_j$$

s. t.

$$\sum_{j=1}^{n} a_{ij} x_j \leqslant b_i \quad \text{for} \quad i = 1, \cdots, m$$

$$x_j \geqslant 0 \quad \text{for} \quad j = 1, \cdots, n$$

其中，s. t. 表示并且满足的意思。

一般地，给定一组实数的集合 $a_1, a_2, \cdots, a_n$ 以及一组变量的集合 $x_1, x_2, \cdots, x_n$，关于这些变量的目标函数定义为 $f(x_1, \cdots, x_n) = a_1 x_1 + \cdots + a_n x_n = \sum_{j=1}^{n} a_j x_j$，其线性约束包括：

线性等式：$f(x_1, \cdots, x_n) = b$；

线性不等式：$f(x_1, \cdots, x_n) \leqslant b$ 及 $f(x_1, \cdots, x_n) \geqslant b$。

形式上，线性规划问题是基于有限个线性约束的求目标函数最大或最小值的

问题。例 10.1 便是一个最小化线性规划问题。

例 10.1

$$\begin{aligned} \min \quad & -2x_1 + 3x_2 \\ \text{s.t.} \quad & \\ & x_1 + x_2 = 7 \\ & x_1 - 2x_2 \leqslant 4 \\ & x_1 \geqslant 0 \end{aligned}$$

10.1.1 线性规划问题的标准形式

给定 n 个实数 $c_1, c_2, \cdots, c_n$，m 个实数 $b_1, b_2, \cdots, b_m$ 以及 $m \times n$ 个实数 $a_{ij}, i = 1, 2, \cdots, m, j = 1, 2, \cdots, n$。我们希望找到 n 个实数 $X = \{x_1, x_2, \cdots, x_n\}$，使得

$$\max \quad \sum_{j=1}^{n} c_j x_j$$

s.t.

$$\sum_{j=1}^{n} a_{ij} x_j \leqslant b_i \qquad \text{for} \quad i = 1, \cdots, m \tag{10.1}$$

$$x_j \geqslant 0 \qquad \text{for} \quad j = 1, \cdots, n \tag{10.2}$$

满足约束条件(10.1)和条件(10.2)的解 X 称为可行解，如果 X 使得至少有一个约束条件不满足，则称为不可行解。使目标函数值 $\sum_{j=1}^{n} c_j x_j$ 达到最大的可行解就是最优解，其目标函数值称为最大值 。如果一个线性规划问题没有可行解，则该问题是不可行的。如果一个线性规划问题有可行解，但目标函数值是无限的，则说线性规划问题无界。

令

$$c = \begin{pmatrix} c_1 \\ \vdots \\ c_n \end{pmatrix}, A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix}, x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

上述线性规划问题的标准形式(SLP)可以简记为

$$\begin{aligned} \max \quad & c^{\mathrm{T}} x \\ \text{s.t.} \quad & \\ & Ax \leqslant b \\ & x \geqslant 0 \end{aligned}$$

为了分析问题方便，我们今后一般基于标准形式进行讨论。如果一个线性规划问题不满足标准形式的定义，则可以将其转化为标准形式。事实上，任何其他形式的线性规划问题都可以很方便地转化为标准形式。具体转化方法如下：

（1）如果线性规划问题是关于目标函数的最小值问题，即

$$\min \quad f(x_1, x_2, \cdots, x_n)$$

即将目标函数值反号，求新目标函数的最大值即可，即令 $f' = -f$，原来的目标函数变为

$$\max \quad f'(x_1, x_2, \cdots, x_n)$$

对例 10.1 的目标函数反号，可得

$$\begin{aligned} &\min \quad -2x_1 + 3x_2 \\ &\text{s.t.} \\ &\quad x_1 + x_2 = 7 \\ &\quad x_1 - 2x_2 \leqslant 4 \\ &\quad x_1 \geqslant 0 \end{aligned} \qquad \text{转化为} \qquad \begin{aligned} &\max \quad 2x_1 - 3x_2 \\ &\text{s.t.} \\ &\quad x_1 + x_2 = 7 \\ &\quad x_1 - 2x_2 \leqslant 4 \\ &\quad x_1 \geqslant 0 \end{aligned}$$

（2）如果某个变元 x_j 没有非负约束，则可以引进两个新变元 x'_j 和 x''_j，然后将 $x'_j - x''_j$ 代替每个式子中的变元 x_j，同时增加两个新变元的非负约束 $x'_j \geqslant 0$ 和 $x''_j \geqslant 0$ 即可。具体地，如果目标函数包括项 $c_j x_j$ 则将其替换为 $c_j x'_j - c_j x''_j$；如果约束条件中包括项 $a_{ij} x_j$，则将其替换为 $a_{ij} x'_j - a_{ij} x''_j$。例如

$$\begin{aligned} &\max \quad 2x_1 - 3x_2 \\ &\text{s.t.} \\ &\quad x_1 + x_2 = 7 \\ &\quad x_1 - 2x_2 \leqslant 4 \\ &\quad x_1 \geqslant 0 \end{aligned} \qquad \text{将 } x_2 \text{ 用 } x'_2 - x''_2 \text{ 替换，得} \qquad \begin{aligned} &\max \quad 2x_1 - 3x'_2 + 3x''_2 \\ &\text{s.t.} \\ &\quad x_1 + x'_2 - x''_2 = 7 \\ &\quad x_1 - 2x'_2 + 2x''_2 \leqslant 4 \\ &\quad x_1, x'_2, x''_2 \geqslant 0 \end{aligned}$$

（3）如果约束条件是线性等式，即 $f(x_1, \cdots, x_n) = b$，则可将其转化为

$$f(x_1, \cdots, x_n) \leqslant b \text{ 且 } f(x_1, \cdots, x_n) \geqslant b$$

例如

$$\begin{aligned} &\max \quad 2x_1 - 3x'_2 + 3x''_2 \\ &\text{s.t.} \\ &\quad x_1 + x'_2 - x''_2 = 7 \\ &\quad x_1 - 2x'_2 + 2x''_2 \leqslant 4 \\ &\quad x_1, x'_2, x''_2 \geqslant 0 \end{aligned} \qquad \text{可以转化为} \qquad \begin{aligned} &\max \quad 2x_1 - 3x'_2 + 3x''_2 \\ &\text{s.t.} \\ &\quad x_1 + x'_2 - x''_2 \leqslant 7 \\ &\quad x_1 + x'_2 - x''_2 \geqslant 7 \\ &\quad x_1 - 2x'_2 + 2x''_2 \leqslant 4 \\ &\quad x_1, x'_2, x''_2 \geqslant 0 \end{aligned}$$

（4）如果线性约束为 $f(x_1, \cdots, x_n) \geqslant b$，则将两边同乘一个负号，变为 $-f(x_1, \cdots, x_n) \leqslant -b$。例如

$$
\begin{aligned}
&\max \quad 2x_1 - 3x'_2 + 3x''_2 \\
&\text{s. t.} \\
&\quad x_1 + x'_2 - x''_2 \leqslant 7 \\
&\quad x_1 + x'_2 - x''_2 \geqslant 7 \\
&\quad x_1 - 2x'_2 + 2x''_2 \leqslant 4 \\
&\quad x_1, x'_2, x''_2 \geqslant 0
\end{aligned}
\qquad \text{可以转化为标准形式} \qquad
\begin{aligned}
&\max \quad 2x_1 - 3x'_2 + 3x''_2 \\
&\text{s. t.} \\
&\quad x_1 + x'_2 - x''_2 \leqslant 7 \\
&\quad -x_1 - x'_2 + x''_2 \leqslant -7 \\
&\quad x_1 - 2x'_2 + 2x''_2 \leqslant 4 \\
&\quad x_1, x'_2, x''_2 \geqslant 0
\end{aligned}
$$

根据上述四步，我们能将例 10.1 所描述的线性规划问题转化为标准形式。为简便起见，将变元 x'_2 和 x''_2 依次用变元 x_2 和 x_3 替换，可得标准形式

$$
\begin{aligned}
&\max \quad 2x_1 - 3x_2 + 3x_3 \\
&\text{s. t.} \\
&\quad x_1 + x_2 - x_3 \leqslant 7 \\
&\quad -x_1 - x_2 + x_3 \leqslant -7 \\
&\quad x_1 - 2x_2 + 2x_3 \leqslant 4 \\
&\quad x_1, x_2, x_3 \geqslant 0
\end{aligned}
$$

值得注意的是，上述每一步转化中，读者不难验证老问题与新问题是等价的。

10.1.2 线性规划问题的松弛形式

为了有效地用单纯形算法来求解 LP 问题，我们需要将线性规划问题的标准形式转化为 LP 问题的松弛形式。具体转化步骤如下：

(1) 将不等式约束 $\sum_{j=1}^{n} a_{ij}x_j \leqslant b_i$，转化为

$$
\begin{aligned}
x_{n+i} &= b_i - \sum_{j=1}^{n} a_{ij}x_j \\
x_{n+i} &\geqslant 0
\end{aligned}
\tag{10.3}
$$

其中，变元 x_{n+i} 度量 b_i 和 $\sum_{j=1}^{n} a_{ij}x_j$ 之间的差值大小，因此被称为松弛变元。读者不难验证转化前后的问题是等价的。例如例 10.1 的标准形式按照步骤(1)，可以转化为

$$
\begin{aligned}
&\max \quad 2x_1 - 3x_2 + 3x_3 \\
&\text{s. t.} \\
&\quad x_4 = 7 - (x_1 + x_2 - x_3) \\
&\quad x_5 = -7 - (-x_1 - x_2 + x_3) \\
&\quad x_6 = 4 - (x_1 - 2x_2 + 2x_3) \\
&\quad x_1, x_2, x_3, x_4, x_5, x_6 \geqslant 0
\end{aligned}
$$

（2）忽略目标函数前面的最大值符号，引进新的变元 z 来表示目标函数的值。同时忽略每个变元的非负约束，这样便可以得到标准形式的 LP 问题的松弛形式

$$z = \sum_{j \in N} c_j x_j$$

$$x_{n+i} = b_i - \sum_{j \in N} a_{ij} x_j$$

等式左边新引进的变元 x_{n+i} 称为基本变元，等式右边的变元 x_j 称为非基本变元，它们出现在目标函数中，N 是非基本变元下标的集合。例如

$$\begin{aligned} &\max \quad 2x_1 - 3x_2 + 3x_3 \\ &\text{s. t.} \\ &\quad x_1 + x_2 - x_3 \leqslant 7 \\ &\quad -x_1 - x_2 + x_3 \leqslant -7 \\ &\quad x_1 - 2x_2 + 2x_3 \leqslant 4 \\ &\quad x_1, x_2, x_3 \geqslant 0 \end{aligned}$$

标准形式转化为松弛形式

$$\begin{aligned} z &= 2x_1 - 3x_2 + 3x_3 \\ x_4 &= 7 - x_1 - x_2 + x_3 \\ x_5 &= -7 + x_1 + x_2 - x_3 \\ x_6 &= 4 - x_1 + 2x_2 - 2x_3 \end{aligned}$$

其中，x_4, x_5, x_6 为基本变元，x_1, x_2, x_3 为非基本变元，$N = \{1,2,3\}$。

为了方便求解，令基本变元下标的集合为 $B = \{n+1, n+2, \cdots, n+m\}$，非基本变元下标的集合为 $N = \{1,2,\cdots,n\}$，同时引进 v 来表示目标函数中可选择的常数项，我们可以将松弛形式(10.3)转化为更简洁的形式

$$z = v + \sum_{j \in N} c_j x_j$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j, \text{其中 } i \in B$$

并简记为 (N, B, A, b, c, v)，值得注意的是，所有的变元 x_i 是非负的。

例 10.2 给定松弛形式

$$z = 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3}$$

$$x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3}$$

$$x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3}$$

$$x_4 = 18 - \frac{x_3}{2} + \frac{x_5}{2}$$

其中，$B = \{1,2,4\}$，$N = \{3,5,6\}$，$c = (c_3 \quad c_5 \quad c_6)^{\mathrm{T}} = (-1/6 \quad -1/6 \quad -2/3)^{\mathrm{T}}$，$v = 28$；

$$b=\begin{pmatrix}b_1\\b_2\\b_3\end{pmatrix}=\begin{pmatrix}8\\4\\18\end{pmatrix},A=\begin{pmatrix}a_{13}&a_{15}&a_{16}\\a_{23}&a_{25}&a_{26}\\a_{43}&a_{45}&a_{46}\end{pmatrix}=\begin{pmatrix}-1/6&-1/6&1/3\\8/3&2/3&-1/3\\1/2&-1/2&0\end{pmatrix}$$

10.2 求解算法

前面一节介绍了线性规划问题与其标准形式、松弛形式之间的关系，下面我们介绍线性规划问题的求解算法。

10.2.1 图解法

对于只涉及两个变元 x_1, x_2 的简单的线性规划问题，例如

$$\max \quad 2x_1 + 3x_2$$

s. t.

$$x_1 + x_2 \leqslant 6$$

$$x_1 + 2x_2 \leqslant 8$$

$$x_1 \leqslant 4$$

$$x_2 \leqslant 3$$

$$x_1 \geqslant 0, x_2 \geqslant 0$$

我们可以建立直角坐标系，然后将每个约束条件看作一直线方程，在坐标系中画出来，便可以得到一个凸形区域，如图 10.1 所示。这个凸区域正是由满足约束条件的解组成的，如图中阴影部分所示。凸区域以及边界上的点均为满足约束条件的可行解。我们可以将可行区域中的每一个点代入目标函数，求出其值，然后确定一个具有最大目标函数值的点作为最优解。然而由于可行区域中的点太多，这样计算需要的计算量太大。因此，我们需要更有效的计算方法。对目标函数引进变元 z 来表示目标函数值，即令 $z = 2x_1 + 3x_2$。由于直线 $z = 2x_1 + 3x_2$ 的斜率为 $-\frac{2}{3}$，该直线在可行区域中的滑动过程如图 10.1 虚线所示。

当 $z = 0$，直线 $0 = 2x_1 + 3x_2$ 为经过原点的虚线，当 $z = 14$，直线 $14 = 2x_1 + 3x_2$ 与可行区域的边界交于点(4,2)。事实上，目标函数的最大值一定可以在关于目标函数的直线与可行区域边界的交界之处，如图 10.1 所示的点(4,2)。由于直线 $14 = 2x_1 + 3x_2$ 只相交于一个顶点，因此上述线性规划问题有唯一解(4,2)，其目标函数值为 14。

当然，有时关于目标函数的直线与可行区域的边界可能不止一个顶点，则该线

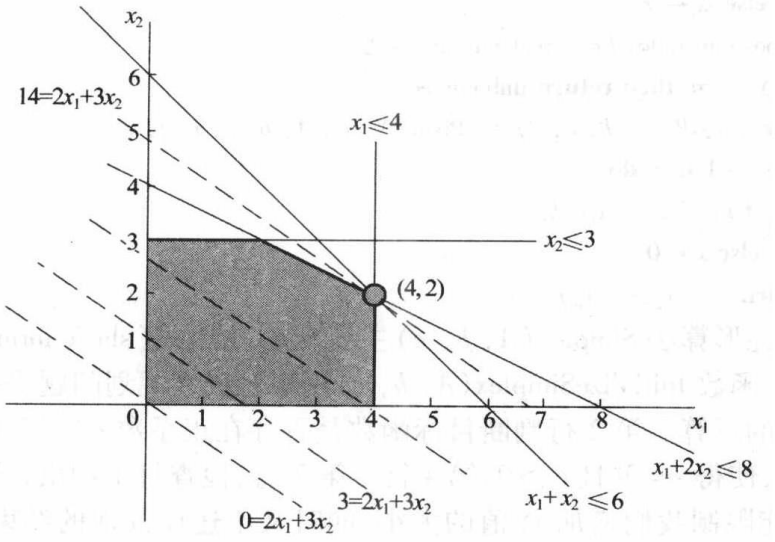


图 10.1

性规划问题有多个最优解。值得注意的是,图解法一般只适用于具有两个变元的简单线性规划问题,当变元数增多时,则很难画出图来,因此我们必须寻求更通用的求解算法。

10.2.2 单纯形算法

线性规划问题是一类比较容易处理的问题,可用单纯形算法来获得该问题的最优解。单纯形算法是一种能求解线性规划问题最优解的多项式时间算法。尽管算法实现起来有点复杂,但在实践中该算法是非常有效的,因而被誉为 20 世纪最好的 10 个算法之一。

单纯形算法(Simplex Algorithm)的主要思路是从线性规划问题的松弛形式开始,设置初始基本解为 0,即令每个非基本变元为 0,然后从等式约束中求出基本变元的值。为了增加目标函数的值,我们必须选择一个非基本变元,增加它的值能使得目标函数的值也增加。非基本变元的值该增加多少,取决于等式约束的强弱。选择等式约束强的一个方程,将非基本变元(与基本变元的角色互换)解出来,然后代入目标函数以及其他的等式约束中,得到一个新的松弛形式。重复上述过程,直到目标函数里变元的系数都为负或函数值不再增加为止。下面我们依次介绍一下单纯形算法的主要过程。

```
Simplex(A, b, c)
1  (N, B, A, b, c, v) ← InitializeSimplex(A, b, c)
2  while some index j ∈ N has c_j > 0 do
3      choose an index e ∈ N for which c_e > 0
4      for each index i ∈ B do
5          if a_ie > 0 then Δ_i ← b_i / a_ie
```

```
6        else Δ_i ← ∞
7    choose an index l ∈ B that minimizes Δ_i
8    if Δ_l = ∞ then return unbounded
9    else (N, B, A, b, c, v) ← Pivot(N, B, A, b, c, v, l, e)
10 for i ← 1 to n do
11       if i ∈ B then x̄_i ← b_i
12       else x̄_i ← 0
13 return (x̄_1, x̄_2, ⋯, x̄_n)
```

由于单纯形算法 Simplex(A, b, c)主要从松弛形式(slack form)开始,因此算法的第 1 行,函数 InitializeSimplex(A, b, c)用来对线性规划问题进行预处理,以便进行接下来的运算。第 2 行判断目标函数是否存在正系数 c_j。第 3 行选择一个非基本变元 x_e,使得 $e \in N$ 且 $c_e > 0$,第 4 行 ~ 第 7 行,检查每个约束,选择一个最强的约束,它最能限制我们增加 x_e 值的大小,同时又不违反其他的约束。然后将该约束方程左边的变元作为选择的基本变元 x_l。第 8 行判断如果 $\Delta_l = \infty$,则返回无界(unbounded),否则执行第 9 行,进行松弛形式的更新。第 10 行 ~ 第 12 行计算原始线性规划问题的解,我们将在下一节证明这个解正是线性规划问题的最优解。如果没有一个约束能限制我们增加 x_e 的值,则该线性规划问题是无界的。

下面我们介绍一下算法用到的两个子过程 InitializeSimplex(A, b, c)和 Pivot(N, B, A, b, c, v, l, e)。

1. *初始化基本可行解*

由于单纯形算法主要从松弛形式开始,而一般现实的线性规划问题有时连标准形式都不是。特别地,松弛形式的可行解,对原问题来说不一定是可行解,因此我们需要对线性规划问题进行预处理,即测试一个线性规划问题是否具有可行解,如果可行,将其转化为松弛形式,同时测试松弛形式的基本解是否可行,以便进行接下来的运算。

单纯形算法是从松弛形式的初始基本解开始的,一个线性规划问题是可行的,但是初始基本解可能是不可行的。例如对于线性规划问题

$$\begin{aligned} &\max \quad 2x_1 - x_2 \\ &\text{s.t.} \\ &\quad 2x_1 - x_2 \leqslant 2 \\ &\quad x_1 - 5x_2 \leqslant -4 \\ &\quad x_1, x_2 \geqslant 0 \end{aligned}$$

其松弛形式为

$$\begin{aligned} z &= 2x_1 - x_2 \\ x_3 &= 2 - (2x_1 - x_2) \\ x_4 &= -4 - (x_1 - 5x_2) \end{aligned}$$

松弛形式的初始基本解将令 $x_1 = 0, x_2 = 0$,然而这个解违反线性规划问题的约束条件 $x_1 - 5x_2 \leqslant -4$,因此这不是一个可行解。对于一个基本解是可行的松弛形式,为了判断初始的线性规划问题是否可行,我们引进一个辅助的线性规划问题进行判断。

引理 10.1 令 L_{aux} 表示下列具有 $n+1$ 个变元的线性规划问题

$$\max \quad -x_0$$

s. t.

$$\sum_{j=1}^{n} a_{ij}x_j - x_0 \leqslant b_i \quad \text{for} \quad i = 1,2,\cdots,m$$

$$x_j \geqslant 0 \qquad\qquad \text{for} \quad j = 0,1,\cdots,n$$

则标准形式的线性规划问题(SLP)是可行的,当且仅当 L_{aux} 的最优目标函数值为 0。

证明:首先我们证明 SLP 是可行的,则 L_{aux} 的最优目标函数值为 0。假定 SLP 有可行解 $\bar{x} = (\bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n)$,则满足 $\bar{x}_0 = 0$ 的解 $(\bar{x}_0, \bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n)$ 一定是 L_{aux} 的可行解,且 L_{aux} 的目标函数值为 0。因为 L_{aux} 中的约束条件有 $\bar{x}_0 \geqslant 0$,因此 L_{aux} 的最优目标函数值为 0。

反过来,如果有一个解 $(\bar{x}_0, \bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n)$ 使得 L_{aux} 的最优目标函数值为 0,则 $\bar{x}_0 = 0$,当然这个解是 L_{aux} 的可行解,由此推出解 $(\bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n)$ 满足 SLP 的约束条件,因此 SLP 是可行的。

综上所述,所证成立。

有了引理 10.1,就可以给出为 SLP 找初始基本可行解的算法:

```
InitializeSimplex(A, b, c)
1  let l be the index of the minimum b_i
2  if b_l ≥ 0 then return (N,B,A,b,c,0)
3  construct L_aux by 引理 10.1
4  let (N,B,A,b,c,v) be the resulting slack form for L_aux
5  (N,B,A,b,c,v) ← Pivot(N,B,A,b,c,v,l,0)
6  iterate the while loop of Simplex until an optimal solution to L_aux is found
7  if the basic solution sets x̄_0 = 0 then
8      return the slack form with x̄_0 removed and the original objective function restored
9  else return infeasible
```

InitializeSimplex(A, b, c)算法中,第 1 行选择最小 b_i 的下标 l,第 2 行判断,如果 $b_l \geqslant 0$,则返回松弛形式 $(N,B,A,b,c,0)$,否则执行第 3 行,按照引理 10.1 构造 L_{aux},第 4 行令 L_{aux} 的松弛形式为 (N,B,A,b,c,v)。第 5 行执行松弛形式的迭代过程,第 6 行反复执行,直到找到 L_{aux} 的最优解。第 7 行如果找到目标函数值为 0 的最优解,则执行第 8 行,为标准形式构造一个基本解可行的松弛形式,即从约束条件中删除 $\bar{x}_0$,恢复标准形式的目标函数。否则,第 9 行返回标准形式的线性规划问题是不可行的。

2. *松弛形式的迭代过程*

从单纯形算法的过程可以看出,松弛形式的迭代过程 Pivot(N, B, A, b, c, v,

l, e)主要根据选择的非基本变元 x_e 和基本变元 x_l，交换松弛形式(N,B,A,b,c,v)中两个变元 x_e 和 x_l 的角色，即从非基本变元变为基本变元，把基本变元变为非基本变元，从而得到新的松弛形式$(\hat{N},\hat{B},\hat{A},\hat{b},\hat{c},\hat{v})$，其算法描述如下：

```
Pivot(N, B, A, b, c, v, l, e)
1   b̂_e ← b_l / a_le
2   for each j ∈ N − {e} do â_ej ← a_lj / a_le
3   â_el ← 1 / a_le
4   for each i ∈ B − {l} do b̂_i ← b_i − a_ie b̂_e
5               for each j ∈ N − {e} do â_ij ← a_ij − a_ie â_ej
6               â_il ← − a_ie â_el
7   v̂ ← v + c_e b̂_e
8   for each j ∈ N − {e} do ĉ_j ← c_j − c_e â_ej
9   ĉ_l ← − c_e â_el
10  N̂ ← N − {e} ∪ {l}
11  B̂ ← B − {l} ∪ {e}
12  return (N̂, B̂, Â, b̂, ĉ, v̂)
```

Pivot(N, B, A, b, c, v, l, e)算法中，第 1 行～第 3 行，主要用于计算关于新基本变元 x_e 方程的系数。第 4 行～第 6 行通过替换原方程右边式子中的每个 x_e，以便得到新方程。第 7 行～第 9 行，将关于 x_e 的方程替换原目标函数中出现的 x_e。接下两行更新基本变元和非基本变元集合，最后返回得到的松弛形式。

例 10.3 给定松弛形式

$$z = 3x_1 + x_2 + 2x_3$$

$$x_4 = 30 - x_1 - x_2 - 3x_3 \tag{10.4}$$

$$x_5 = 24 - 2x_1 - 2x_2 - 5x_3 \tag{10.5}$$

$$x_6 = 36 - 4x_1 - x_2 - 2x_3 \tag{10.6}$$

此时，$B=\{4,5,6\}$，$N=\{1,2,3\}$，$c=(c_1 \quad c_2 \quad c_3)^{\mathrm{T}}=(3 \quad 1 \quad 2)^{\mathrm{T}}$，$v=0$，

$$b=\begin{pmatrix} b_4 \\ b_5 \\ b_6 \end{pmatrix}=\begin{pmatrix} 30 \\ 24 \\ 36 \end{pmatrix}, A=\begin{pmatrix} a_{41} & a_{42} & a_{43} \\ a_{51} & a_{52} & a_{53} \\ a_{61} & a_{62} & a_{63} \end{pmatrix}=\begin{pmatrix} 1 & 1 & 3 \\ 2 & 2 & 5 \\ 4 & 1 & 2 \end{pmatrix}$$

如果传进来的参数 $l=6$，$e=1$，即 $x_l=x_6$，$x_e=x_1$，调用 Pivot(N, B, A, b, c, v, l, e)执行第 1 行～第 3 行，从方程(10.6)解出 $x_1=9-\frac{x_2}{4}-\frac{x_3}{2}-\frac{x_6}{4}$，此时 $\hat{b}_1=9$，$\hat{a}_{12}=\frac{1}{4}$，$\hat{a}_{13}=\frac{1}{2}$，$\hat{a}_{16}=\frac{1}{4}$。接着执行第 4 行～第 6 行，将 $x_1=9-\frac{x_2}{4}-\frac{x_3}{2}-\frac{x_6}{4}$ 代入原方程(10.4)和(10.5)右边式子中的每个 x_1，可得

$$x_4 = 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4} \qquad \hat{b}_4 = 21, \hat{a}_{42} = \frac{3}{4}, \hat{a}_{43} = \frac{5}{2}, \hat{a}_{46} = -\frac{1}{4}$$

$$x_5 = 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2} \qquad \hat{b}_5 = 6, \hat{a}_{52} = \frac{3}{2}, \hat{a}_{53} = 4, \hat{a}_{56} = -\frac{1}{2}$$

将 $x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}$ 代入目标函数得 $z = 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4}$，从而 $\hat{c}_2 = \frac{1}{4}, \hat{c}_3 = \frac{1}{2}, \hat{c}_6 = -\frac{3}{4}$。因此，新的松弛形式为 $\hat{B} = \{1,4,5\}, \hat{N} = \{2,3,6\}, \hat{v} = 27$。

介绍了上述两个主要函数后，我们利用 Simplex(A, b, c)来求解下面的线性规划问题

$$\max \quad 3x_1 + x_2 + 2x_3$$

$$\text{s. t.}$$

$$x_1 + x_2 + 3x_3 \leqslant 30$$

$$2x_1 + 2x_2 + 5x_3 \leqslant 24$$

$$4x_1 + x_2 + 2x_3 \leqslant 36$$

$$x_1, x_2, x_3 \geqslant 0$$

其松弛形式为

$$z = 3x_1 + x_2 + 2x_3$$

$$x_4 = 30 - x_1 - x_2 - 3x_3$$

$$x_5 = 24 - 2x_1 - 2x_2 - 5x_3$$

$$x_6 = 36 - 4x_1 - x_2 - 2x_3$$

首先调用 InitializeSimplex(A, b, c)，找到最小的 b_i，这里 $b_5 = 24 \geqslant 0$，因此返回 (N, B, A, b, c, v)，这里 $B = \{4,5,6\}, N = \{1,2,3\}, v = 0$。由于存在某些 $c_j > 0 (j \in N)$，选择 c_1，它的增加能使得目标函数值增加最快，因此 $e = 1$。此时，$\Delta_4 = \frac{b_4}{a_{41}} = 30, \Delta_5 = \frac{b_5}{a_{51}} = 12, \Delta_6 = \frac{b_6}{a_{61}} = 9$，$\Delta_6$ 最小，因此 $l = 6$，交换 x_1 和 x_6 的角色，可得 $x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}$。

调用 Pivot(N, B, A, b, c, v, l, e)，可得

$$z = 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4}$$

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}$$

$$x_4 = 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4}$$

$$x_5 = 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2}$$

由于存在某些 $c_j > 0 (j \in N)$，选择 c_3，它的增加能使得目标函数值增加最快，因

此 $e=3$。此时，$\Delta_1=\frac{b_1}{a_{13}}=18$，$\Delta_4=\frac{b_4}{a_{43}}=8.4$，$\Delta_5=\frac{b_5}{a_{53}}=1.5$，$\Delta_5$ 最小，因此 $l=5$，交换 x_3 和 x_5 的角色，即从 $x_5=6-\frac{3x_2}{2}-4x_3+\frac{x_6}{2}$ 中解出 x_3，可得 $x_3=\frac{3}{2}-\frac{3x_2}{8}-\frac{x_5}{4}+\frac{x_6}{8}$。

调用 Pivot(N, B, A, b, c, v, l, e)，可得

$$z=\frac{111}{4}+\frac{x_2}{16}-\frac{x_5}{8}-\frac{11x_6}{16}$$

$$x_1=\frac{33}{4}-\frac{x_2}{16}+\frac{x_5}{8}-\frac{5x_6}{16}$$

$$x_3=\frac{3}{2}-\frac{3x_2}{8}-\frac{x_5}{4}+\frac{x_6}{8}$$

$$x_4=\frac{69}{4}+\frac{3x_2}{16}+\frac{5x_5}{8}-\frac{x_6}{16}$$

由于存在 $c_2>0$，只有选择它，它的增加才能使得目标函数值增加，因此 $e=2$。此时，$\Delta_1=\frac{b_1}{a_{12}}=132$，$\Delta_3=\frac{b_3}{a_{32}}=4$，因为 x_2 的增加，总能满足 $x_4=\frac{69}{4}+\frac{3x_2}{16}+\frac{5x_5}{8}-\frac{x_6}{16}$，因此 $\Delta_4=\infty$。Δ_3 最小，因此 $l=3$，交换 x_2 和 x_3 的角色，即从 $x_3=\frac{3}{2}-\frac{3x_2}{8}-\frac{x_5}{4}+\frac{x_6}{8}$ 解出 x_2，可得 $x_2=4-\frac{8x_3}{3}-\frac{2x_5}{3}+\frac{x_6}{3}$。

调用 Pivot(N, B, A, b, c, v, l, e)，可得

$$z=28-\frac{x_3}{6}-\frac{x_5}{6}-\frac{2x_6}{3}$$

$$x_1=8+\frac{x_3}{6}+\frac{x_5}{6}-\frac{x_6}{3}$$

$$x_2=4-\frac{8x_3}{3}-\frac{2x_5}{3}+\frac{x_6}{3}$$

$$x_4=18-\frac{x_3}{2}+\frac{x_5}{2}$$

此时不存在 $c_j>0(j\in N)$，停止执行 **while** 循环，执行第 10 行～第 12 行，$\hat{x}_1=8$，$\hat{x}_2=4$，$\hat{x}_4=18$，$\hat{x}_3=0$，$\hat{x}_5=0$，$\hat{x}_6=0$。解$(\hat{x}_1,\hat{x}_2,\hat{x}_3,\hat{x}_4,\hat{x}_5,\hat{x}_6)=(8,4,0,18,0,0)$被返回，此时，最优目标函数值 $z=28$。

关于单纯形算法正确性的证明，建议读者参看文献[1]，这里就不再给出证明。下面我们关心的是单纯形算法什么时候终止。我们先介绍一个有用的结论。

定理 10.1 令 I 是下标的集合。对于 I 中的任意 i，令 α_i 和 β_i 是实数，令 x_i 是实数值变量。令 γ 是任意的实数，假定对 x_i 的任何取值，有

$$\sum_{i\in I}\alpha_i x_i = \gamma + \sum_{i\in I}\beta_i x_i \tag{10.7}$$

则对 I 中的每个 i，有 $\alpha_i=\beta_i$，并且 $\gamma=0$。

证明：由于对 x_i 的任何取值，式(10.7)成立，我们不妨对每个 $i\in I$，令 $x_i=0$，能够推出 $\gamma=0$。现在选择一个任意的下标 $i\in I$，我们令 $x_i=1$，$x_k=0(k\neq i)$，则可以得到 $\alpha_i=\beta_i$。由于 i 的任意性，我们可得结论。

我们下面证明线性规划问题的松弛形式由基本变元的集合唯一确定。

引理 10.2 令 (A,b,c) 是标准形式的线性规划问题。给定一个基本变元的集合 B，相应的松弛形式被唯一确定。

证明：用反证法证明。假定有两个松弛形式具有同样的基本变元集合 B，则它们的非基本变元集合也是一样的。设第一个松弛形式为

$$z = v + \sum_{j\in N} c_j x_j \tag{10.8}$$

$$x_i = b_i - \sum_{j\in N} a_{ij}x_j, \text{其中 } i \in B \tag{10.9}$$

另一个松弛形式为

$$z = v' + \sum_{j\in N} c'_j x_j \tag{10.10}$$

$$x_i = b'_i - \sum_{j\in N} a'_{ij}x_j, \text{其中 } i \in B \tag{10.11}$$

将式(10.9)减去式(10.11)，可得

$$\sum_{j\in N} a_{ij}x_j = (b_i - b'_i) + \sum_{j\in N} a'_{ij}x_j$$

由定理 10.1，可得 $a_{ij}=a'_{ij}$ 且 $b_i=b'_i$。类似地，可以证明 $c_j=c'_j$ 且 $v=v'$，见习题 10.2。因此松弛形式被 B 唯一确定。

值得注意的是，单纯形算法 **while** 循环的每次迭代中，从一个松弛形式变到另一个松弛形式，如果目标函数值不发生改变，则有可能导致算法不会终止，而永远循环下去。下面，我们介绍一个结论。

定理 10.2 如果单纯形算法不能在 C^m_{n+m} 个迭代内终止，则算法循环。

证明：由引理 10.2 可知，松弛形式被基本变元集合 B 唯一确定。总共有 $n+m$ 个变元且 $|B|=m$，因此共有 C^m_{n+m} 种方式确定 B，因而有 C^m_{n+m} 种松弛形式。因此定理成立。

单纯形算法理论上有发生循环的可能，实际上几乎很少发生，当出现这种情况时，我们也能够采取一些办法，以避免循环的发生。例如，稍微扰动一下输入，使得不可能出现具有同样目标函数值的两个解。当出现具有两个相同目标函数值的迭代时，总是选择具有最小下标的索引等，这里就不再详细叙述，详细的内容可参考文献[1,13]。

10.3 对 偶

我们能够证明，在某种假定下，单纯形算法将会终止。为了证明算法终止后，返回的解是最优解，我们还必须介绍线性规划问题对偶性(Duality)的概念。事实上，对偶性是一个非常重要的概念，在优化领域里，问题对偶性的确定，能够帮助我们找到多项式时间求解算法。特别在近似算法的设计中，常常借助对偶性来设计多项式时间近似算法，关于对偶性在近似算法中的应用，请读者参看文献[4]，这里就不再讨论。下面，我们主要介绍利用对偶性，可以用来证明问题的一个解确实是最优的。

例如，求解最大流问题，我们需要找到一个流 f，现在我们需要知道流 f 是否就是一个最大流？按照第 9 章最大流最小切割定理 9.1，我们能够找到一个割，割的大小等于流值 $|f|$，然后我们证明，流 f 确实就是一个最大流。最大流问题与最小割问题就是一对偶问题。从这里我们也可以看出定义对偶问题的思路：给出一个最大值问题，为定义其对偶问题，我们只需要定义一个相应的最小值问题，使得两个问题具有同样大小的最优目标函数值即可。

给定一个求最大值的线性规划问题，我们将定义其对偶线性规划问题，它的目标是求最小值，且其最小值等于原始线性规划问题的最大值。下面，我们一般用主问题表示原始线性规划问题，这里原始线性规划问题用标准的线性规划问题表示。主问题与其对偶问题有如下关系

主问题：

$$\max \quad \sum_{j=1}^{n} c_j x_j$$

s. t.

$$\sum_{j=1}^{n} a_{ij} x_j \leqslant b_i \quad \text{for} \quad i = 1, \cdots, m$$

$$x_j \geqslant 0 \quad \text{for} \quad j = 1, \cdots, n$$

对偶问题：

$$\min \quad \sum_{i=1}^{m} b_i y_i$$

s. t.

$$\sum_{i=1}^{m} a_{ij} y_i \geqslant c_j \quad \text{for} \quad j = 1, \cdots, n$$

$$y_i \geqslant 0 \quad \text{for} \quad i = 1, \cdots, m$$

从上述式子，可以看出：

(1) 主问题求最大值，约束是 $\leqslant$，则相应对偶问题是求最小值，约束是 $\geqslant$。两个问题关于变量的非负约束不变。

(2) 主问题的约束系数矩阵为 A，有 m 个约束方程，则相应对偶问题的约束系数矩阵为 A^{T}，有 n 个约束方程。

(3) 两个问题中，b 和 c 恰好互换。

从上述两个问题的描述可以看出，两个问题可以互为对偶。即主问题的对偶是对偶问题，而对偶问题的对偶是主问题。

例 10.4 给定

主问题：		对偶问题：
$\max \quad 3x_1 + x_2 + 2x_3$		$\min \quad 30y_1 + 24y_2 + 36y_3$
s. t.		s. t.
$x_1 + x_2 + 3x_3 \leqslant 30$	其对偶为	$y_1 + 2y_2 + 4y_3 \geqslant 3$
$2x_1 + 2x_2 + 5x_3 \leqslant 24$		$y_1 + 2y_2 + y_3 \geqslant 1$
$4x_1 + x_2 + 2x_3 \leqslant 36$		$3y_1 + 5x_2 + 2y_3 \geqslant 2$
$x_1, x_2, x_3 \geqslant 0$		$y_1, y_2, y_3 \geqslant 0$

下面我们将证明，主问题的最优目标函数值将等于对偶问题的目标函数值。为证明此结论，我们先介绍弱对偶性，即主问题的任意可行解的目标函数值不会大于对偶问题任意可行解的目标函数值。这表明，主问题的任意可行解的目标函数值是对偶问题任意可行解的目标函数值的下界。

引理 10.3 令 $\bar{x} = (\bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n)$ 为主问题的任意可行解，令 $\bar{y} = (\bar{y}_1, \bar{y}_2, \cdots, \bar{y}_m)$ 为对偶问题的任意可行解，则 $\sum_{j=1}^{n} c_j \bar{x}_j \leqslant \sum_{i=1}^{m} b_i \bar{y}_i$。

证明：既然 $\bar{y}$ 为对偶问题的可行解，因此它满足对偶问题的约束方程 $\sum_{i=1}^{m} a_{ij} \bar{y}_i \geqslant c_j$，因此有

$$\sum_{j=1}^{n} c_j \bar{x}_j \leqslant \sum_{j=1}^{n} \left(\sum_{i=1}^{m} a_{ij} \bar{y}_i \right) \bar{x}_j$$

对上述不等式右边式子，交换求和的顺序，可得

$$\sum_{j=1}^{n} c_j \bar{x}_j \leqslant \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} \bar{x}_j \right) \bar{y}_i$$

而 $\bar{x}$ 为主问题的可行解，因此满足对偶问题的约束方程 $\sum_{j=1}^{n} a_{ij} \bar{x}_j \leqslant b_i$，代入上式，可得

$$\sum_{j=1}^{n} c_j \bar{x}_j \leqslant \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} \bar{x}_j \right) \bar{y}_i \leqslant \sum_{i=1}^{m} b_i \bar{y}_i$$

故所证成立。

由上述引理 10.3 可得如下推论。

推论 10.1 令 $\bar{x} = (\bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n)$ 为主问题的一个可行解，令 $\bar{y} = (\bar{y}_1, \bar{y}_2, \cdots, \bar{y}_m)$ 为对偶问题的一个可行解，如果有 $\sum_{j=1}^{n} c_j \bar{x}_j = \sum_{i=1}^{m} b_i \bar{y}_i$，则 $\bar{x}$ 和 $\bar{y}$ 分别是主问题和对偶问题的最优解。

如果我们能够找到对偶问题的一个可行解，它的目标函数值等于主问题可行

解的目标函数值，则根据推论 10.1，对偶问题的可行解一定也是最优的。当利用单纯形算法对主问题求解时，算法终止返回解，设最后一次得到的松弛形式为

$$z = v' + \sum_{j \in N} c'_j x_j$$

$$x_i = b'_i - \sum_{j \in N} a'_{ij} x_j, \text{其中 } i \in B$$

令

$$\bar{y}_i = \begin{cases} -c'_{n+i} & \text{if } (n+i) \in N \\ 0 & \text{otherwise} \end{cases} \tag{10.12}$$

则该解是对偶问题的最优解。在证明这个结论之前，我们先举一个例子，验证一下是否属实。

$$z = 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3}$$

$$x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3}$$

$$x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3}$$

$$x_4 = 18 - \frac{x_3}{2} + \frac{x_5}{2}$$

此时，$B = \{1,2,4\}$，$N = \{3,5,6\}$，$\bar{y}_1 = 0$，$\bar{y}_2 = -c'_5 = \frac{1}{6}$，$\bar{y}_3 = -c'_6 = \frac{2}{3}$，满足对偶问题的约束条件，而且代入对偶问题的目标函数，可得 28，恰好等于主问题的目标函数值，按照推论 10.1，因此解 $\bar{y}_i$ 是对偶问题的最优解。现在我们给出一个重要的结论。

定理 10.3 假定单纯形算法计算主问题返回的解为 $\bar{x} = (\bar{x}_1, \bar{x}_2, \cdots, \bar{x}_n)$，令 N 和 B 分别为单纯形算法结束后最后一次得到的松弛形式的非基本变元和基本变元下标的集合，$\bar{y} = (\bar{y}_1, \bar{y}_2, \cdots, \bar{y}_m)$ 由式(10.12)确定，则 $\bar{x}$ 是主问题的最优解，$\bar{y}$ 是对偶问题的最优解，且有 $\sum_{j=1}^{n} c_j \bar{x}_j = \sum_{i=1}^{m} b_i \bar{y}_i$。

证明：当利用单纯形算法对主问题求解时，算法终止返回解，设最后一次得到的松弛形式的目标函数为 $z = v' + \sum_{j \in N} c'_j x_j$，此时一定有 $c'_j \leqslant 0 (j \in N)$。

令 $c'_j = 0 (j \in B)$，则

$$z = v' + \sum_{j \in N} c'_j x_j \quad = v' + \sum_{j \in N} c'_j x_j + \sum_{j \in B} c'_j x_j \quad = v' + \sum_{j \in N \cup B} c'_j x_j$$

对于与最后松弛形式相关的基本解 $\bar{x}$，一定有 $z = v'$。由于所有的松弛形式都是等价的，将 $\bar{x}$ 代入主问题的目标函数中，则一定获得同样的目标函数值，即

$$\sum_{j=1}^{n} c_j \bar{x}_j = z = v'$$

事实上,对于所有的松弛形式,对于任意的解 $x = (x_1, x_2, \cdots, x_n)$,有

$$\sum_{j=1}^{n} c_j x_j = z = v' + \sum_{j \in N \cup B} c'_j x_j = v' + \sum_{j=1}^{m+n} c'_j x_j$$

$$= v' + \sum_{j=1}^{n} c'_j x_j + \sum_{j=n+1}^{n+m} c'_j x_j$$

$$= v' + \sum_{j=1}^{n} c'_j x_j + \sum_{i=1}^{m} c'_{n+i} x_{n+i}$$

由式(10.12)可得

$$\sum_{j=1}^{n} c_j x_j = v' + \sum_{j=1}^{n} c'_j x_j + \sum_{i=1}^{m} c'_{n+i} x_{n+i} = v' + \sum_{j=1}^{n} c'_j x_j + \sum_{i=1}^{m} (-\bar{y}_i) x_{n+i}$$

根据松弛形式 $x_{n+i} = b_i - \sum_{j=1}^{n} a_{ij} x_j$,可得

$$\sum_{j=1}^{n} c_j x_j = v' + \sum_{j=1}^{n} c'_j x_j + \sum_{i=1}^{m} (-\bar{y}_i)\left(b_i - \sum_{j=1}^{n} a_{ij} x_j\right)$$

$$= v' + \sum_{j=1}^{n} c'_j x_j - \sum_{i=1}^{m} b_i \bar{y}_i + \sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij} x_j \bar{y}_i$$

$$= v' + \sum_{j=1}^{n} c'_j x_j - \sum_{i=1}^{m} b_i \bar{y}_i + \sum_{j=1}^{n} \sum_{i=1}^{m} (a_{ij} \bar{y}_i) x_j$$

$$= \left(v' - \sum_{i=1}^{m} b_i \bar{y}_i\right) + \sum_{j=1}^{n} \left(c'_j + \sum_{i=1}^{m} a_{ij} \bar{y}_i\right) x_j$$

由定理 10.1,可得

$$c_j = c'_j + \sum_{i=1}^{m} a_{ij} \bar{y}_i, j = 1, 2, \cdots, n \tag{10.13}$$

$$v' - \sum_{i=1}^{m} b_i \bar{y}_i = 0 \tag{10.14}$$

由于 $c'_j \leqslant 0$,由式子(10.13),可得

$$c_j = c'_j + \sum_{i=1}^{m} a_{ij} \bar{y}_i \leqslant \sum_{i=1}^{m} a_{ij} \bar{y}_i, \quad j = 1, 2, \cdots, n$$

因此解 $\bar{y}$ 满足对偶问题的约束条件。此外,由于 $c_j \leqslant 0, j \in N \cup B$,因此 $\bar{y}_i \geqslant 0$。所以 $\bar{y}$ 是对偶问题的可行解。由式(10.14)可得对偶问题的目标函数值等于主问题的目标函数值。由推论 10.1 可得结论。

我们已经证明,给定一个可行的线性规划问题,如果 InitializeSimplex(A, b, c)

返回一个可行解，且 Simplex(A, b, c)算法正常终止，即没有返回无界，则算法返回的解确实是一个最优解。我们也介绍了怎样构造一个对偶问题的最优解。

10.4 小 结

本章内容主要参考文献[1]，介绍了线性规划问题的标准形式以及松弛形式，然后介绍线性规划问题的单纯形求解算法。最后介绍了线性规划问题的对偶形式及其相关结论。单纯形算法及其改进算法仍然是目前求解线性规划问题最有效的方法之一。当然求解线性规划问题，还有别的一些多项式时间求解算法，例如内点法以及 Ellipsoid 算法等，有兴趣的读者可以参考文献[13]。利用线性规划问题的求解方法来设计 NP 难问题的近似求解算法，一直是一个活跃的研究主题。关于线性规划算法在组合优化问题中的应用，有兴趣的读者可以参考文献[4]。

习题

10.1 将下列线性规划问题

$$\begin{aligned}
&\min \quad 2x_1 + 7x_2 \\
&\text{s. t.} \\
&\qquad x_1 = 7 \\
&\qquad 3x_1 + x_2 \geqslant 24 \\
&\qquad x_2 \geqslant 0 \\
&\qquad x_3 \leqslant 0
\end{aligned}$$

转化为标准形式。

10.2 证明引理 10.2 中，$c_j = c'_j$ 且 $v = v'$。

10.3 将下列线性规划问题

$$\begin{aligned}
&\max \quad 2x_1 - 6x_3 \\
&\text{s. t.} \\
&\qquad x_1 + x_2 - x_3 \leqslant 7 \\
&\qquad 3x_1 - x_2 \geqslant 8 \\
&\qquad -x_1 + 2x_2 + 2x_3 \geqslant 0 \\
&\qquad x_1, x_2, x_3 \geqslant 0
\end{aligned}$$

转化为松弛形式，并给出基本变元集和非基本变元集合。

10.4 图解法求解下述线性规划问题

$$\max \quad x_1 + 2x_2$$

s. t

$$x_1 + 2x_2 \leqslant 6$$

$$3x_1 + 2x_2 \leqslant 12$$

$$x_2 \leqslant 2$$

$$x_1, x_2 \geqslant 0$$

10.5 假定我们将标准形式的线性规划问题(A,b,c)转化为松弛形式，证明基本解是可行的当且仅当$b_i \geqslant 0 (1 \leqslant i \leqslant m)$。

10.6 用单纯形算法求解习题10.4。

10.7 利用单纯形算法求解下列线性规划问题

$$\max \quad 18x_1 + 12.5x_2$$

s. t.

$$x_1 + x_2 \leqslant 20$$

$$x_1 \leqslant 12$$

$$x_2 \leqslant 16$$

$$x_1, x_2 \geqslant 0$$

10.8 利用单纯形算法求解下列线性规划问题

$$\min \quad x_1 + x_2 + x_3$$

s. t.

$$2x_1 + 7.5x_2 + 3x_3 \geqslant 10000$$

$$20x_1 + 5x_2 + 10x_3 \geqslant 30000$$

$$x_1, x_2, x_3 \geqslant 0$$

10.9 证明推论10.1。

10.10 给出习题10.7的对偶形式。

10.11 给定一个非标准形式的线性规划问题，我们可以通过将其转化为标准形式，然后再求其对偶形式。如果能直接从非标准形式的规划问题得到对偶形式，则非常方便。试说明任给一个线性规划问题，我们能直接将其转化为线性规划问题的对偶形式。

实验题

10.12 编程实现单纯形算法，并完成10.7和10.8的求解。

第 11 章　NP 完全理论

前面我们介绍的大多数算法都是多项式时间算法，即对任何规模为 n 的实例，这些算法的最坏情形时间复杂度为 $O(n^k)$，其中 k 为常数。值得注意的是，我们下面讨论算法的复杂度，并对问题分类，都是从最坏情形时间复杂度方面进行讨论的。我们自然想知道，是否所有问题都能够在多项式时间内求解出来。答案是否定的。例如旅行商问题，虽然能找到指数级时间的求解算法，但是至今找不到多项式时间有效算法。此外，还有一些问题是不可解的，例如著名的停机问题，即要求确定对任意的输入，任意的确定性算法是否会在某个时间停止的问题，该问题不管花多长时间，在什么样的机器上，都无法求解。通常，我们把能够在多项式时间内求解的问题看作简单问题，而那些不能在多项式时间内求解的问题，看作是很难处理的问题。为什么我们能够把在多项式时间内求解的问题看作是简单问题呢？有三个原因：

（1）虽然把高阶多项式时间，例如 $\Theta(n^{100})$ 看作是难处理的问题是合理的，但是很少有实际问题具有这样高阶的多项式时间求解算法。多项式时间可计算的问题，实际计算时，需要花费的时间常常更少。经验表明，如果一个问题存在多项式时间求解算法，它常常伴随着一个更有效的多项式时间求解算法。例如，对一个问题，当前找到了一个最有效的多项式时间 $\Theta(n^{100})$ 求解算法，也许不久以后，对该问题可以找到一个更有效的算法，例如时间复杂度为 $\Theta(n^{90})$ 的算法。

（2）对许多合理的计算模型，如果一个问题在一个模型（RAM）上多项式时间可解，则在另外一个模型（Turing 机）上也是多项式时间可解的。

（3）多项式时间可解的算法常常具有好的封闭性质，因为多项式在加、减、乘等运算下，仍然是多项式的。例如，一个多项式算法的输出作为另一个多项式算法的输入，则该组合算法仍然是多项式算法。

这章我们主要介绍一类有趣的问题“NP 完全问题（NP-Complete，NPC）”。到目前为止，还没有找到这类问题的多项式时间求解算法，也没有人证明对这类问题找不到多项式时间算法。的确，现在找不到并不意味着将来也找不到。这引出了一个著名的“P 是否等于 NP”的问题，自从它在 1971 年被提出后，就成为计算机科学领域最深奥有趣的开放性研究问题之一。甚至有著名机构悬赏 100 万美金寻找该问题的答案，但至今仍然遥不可及。

有些问题看似相同，但是一些细微的差别，却使得求解它们的难易程度具有天壤之别。例如第 6 章介绍过的最短简单路径和最长简单路径问题，接下来介绍的

欧拉回路和汉密尔顿回路、2-SAT 问题和 3-SAT 问题等。最短简单路径、欧拉回路和 2-SAT 问题均是多项式时间可解的，而余下的问题却是 NP 完全问题，求解非常困难，到目前为止均找不到多项式时间有效求解算法。

在介绍 P、NP 和 NPC 的定义之前，我们先介绍一些概念。

11.1 判定问题

许多有趣的问题都是最优化问题(optimization problem)，问题的每个可行解都对应一个目标函数值，求解这类问题的目的是希望得到一个有最优目标函数值的可行解。例如，最短路径问题、0/1 背包问题。而判定问题(decision problem)则是回答是否存在一个满足问题要求的解，即解只是简单的回答“是(1)”或“否(0)”。例如图着色问题，给定一个图 $G=(V,E)$，问该图能否用 k 种颜色着色，使得相邻顶点互不同色？如果能，则回答“是”，如果不能，则回答“否”。

最优化问题与它相应地判定问题具有密切的联系，通过限界最优化问题要优化的目标函数值，通常可以把一个最优化问题转化为一个判定问题。例如，最短路径问题的目标是找一条从 u 到 v 的最短路径，优化的目标函数值是路径的边数。则相应的判定问题 Path 可以描述为：给定一个有向图 G，顶点 u 和 v，以及整数 k，问图 G 中从 u 到 v 是否存在一条最多 k 条边的路径。因此，最优化问题转化为相应的判定问题，主要看最优化问题目标是求最大值还是最小值，从而来决定判定问题的描述，如果目标是求最大值，则引进一个整数 k 来限界目标函数值，相应的判定问题可以描述为是否存在一个解，使其目标函数值至少为 k。

最优化问题和判定问题之间的关系也方便我们理解最优化问题的难易程度，判定问题在某种意义上，比最优化问题更容易或者至少不会比最优化问题更难。我们可以通过求解最优化问题来求解相应的判定问题。例如，我们求解了最短路径问题，即求得了最短距离，就可以将该值与判定问题中给定的 k 值比较，从而可以求解判定问题。这就是说，如果一个最优化问题是容易的，则相应的判定问题也就不难，反过来，如果判定问题是难的，则它相应的最优化问题也不容易。因此，我们可以从判定问题的角度来对问题的难易程度进行分类。

此外，如果我们找到了判定问题的多项式时间算法，我们能够在多项式时间内求解相应的最优化问题吗(不一定非要找到最优解)？在许多情形下，很容易给出肯定的回答。例如考虑图着色问题，假设我们有一个多项式时间复杂度的判定算法 ColorIsTrue(G,k)，其返回值为 True 当且仅当图 G 能够 k 着色，则求出图着色问题的最小着色数的算法如下：

```
MinColoring(G)
1   k ← 0
2   while ColorIsTrue(G,k) ≠ True do
3         k ← k + 1
```

4 **return** k

由于图 G 最多有 $|V|$ 个顶点，因此第 2 行最多执行 $|V|$ 次，由于 ColorIsTrue(G, k)算法是多项式时间算法，因此 MinColoring(G)算法的时间复杂度仍然为多项式时间复杂度。

上面分析一个最优化问题与相应判定问题的难易关系的方法也可以用来说明两个判定问题之间的难易关系。

考虑一个判定问题 A，我们想找到问题 A 的多项式时间求解算法，给定问题 A 的一个输入实例，例如，对 Path，输入实例包括图 G，顶点 u 和 v，以及整数 k。现在假定有一个不同的判定问题 B，我们已经知道 B 能够在多项式时间内求解。假定有一个过程能转化 A 的任何一个实例 α 到 B 的某个实例 β，且过程具有下列性质：

(1) 该转化过程只需要多项式时间就可以完成；

(2) 答案是一致的。也就是说，α 的答案是“是”当且仅当 β 的答案也是“是”。

我们称这样的过程是多项式时间约简(Reduction)算法，如图 11.1 所示。

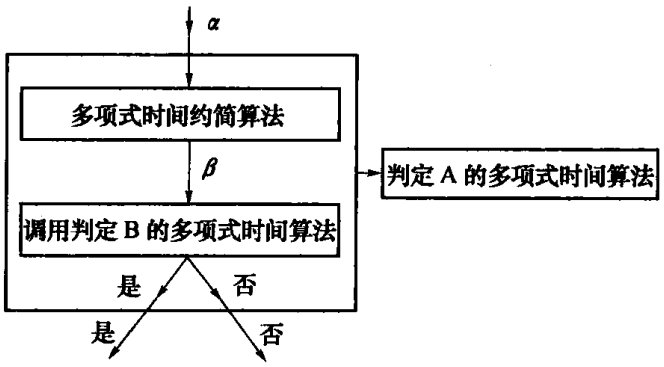


图 11.1

上述过程提供了一个在多项式时间内判定问题 A 的方法：

(1) 任给问题 A 的一个实例 α，用一个多项式时间约简算法把它转化为问题 B 的一个实例 β。

(2) 对实例 β，调用判定 B 的多项式时间算法。

(3) 用 β 的答案作为 α 的答案。

如果上述每一步骤都能在多项式时间内完成，则把每一步合在一起，就得到了一个判定 A 的多项式时间算法。也就是说，通过约简将 A 的求解转化为 B 的求解，用 B 的容易性来证明 A 的容易性。

特别地，我们能够用多项式时间约简算法的思想来证明特定的问题 B 不存在多项式时间算法。假设我们有一个判定问题 A，而且我们知道不存在求解 A 的多项式时间算法。假设有多项式时间约简算法可以把 A 的任意一个实例转化为 B 的一个实例，这样我们可以用简单的反证法来证明 B 没有多项式时间的算法。证明的思路是，假设 B 有多项式时间算法，则按照上述约简算法的思想，我们也能证

明 A 也有多项式时间算法，这与已知不存在求解 A 的多项式时间算法矛盾。

11.2 P 和 NP

我们知道，如果我们写一个程序来求解一个最优化问题，作为程序输入的问题实例必须表示成计算机能够理解的符号形式。即把问题实例编码成一个二进制串，由这些问题实例构成的问题常称为具体问题。一个最优化问题通过这种编码，就可以转化为一个具体问题。这个具体问题的实例集就是一个二进制串的集合，现在考虑具体问题的求解算法。给定具体问题的一个实例 I，令实例的规模为 $n=|I|$，一个算法在 $O(T(n))$ 的时间内得出解，那么就说该算法在 $O(T(n))$ 的时间内求解了一个具体问题。一个具体问题是多项式时间可解的，如果该问题存在一个多项式时间求解算法 $O(n^k)$，其中 k 为一个常数。因此我们可以定义复杂类 P 为多项式时间可解的具体判定问题的集合。有了具体问题的定义及相应求解算法的概念后，就可以从形式语言的角度给出 P 和 NP 的定义。

令字符集 Σ 是符号的一个有穷集合，字符集 Σ 上的语言是由 Σ 中字符所构成的字符串的集合。例如，给定 $\Sigma=\{0,1\}$，则 $L=\{10,11,101,111,1011,1101,10001,\cdots\}$ 是素数的二进制表示所构成的语言。令 ε 表示空串，ϕ 表示空语言，则 Σ 上所有串的语言表示为 Σ^*。例如，$\Sigma=\{0,1\}$，则 $\Sigma^*=\{\varepsilon,0,1,00,01,10,11,000,\cdots\}$，字符集 Σ 上的任一个语言是 Σ^* 的一个子集。

从形式语言的观点看，任何具体判定问题 Q 的实例集合仅仅是集合 Σ^* 的一个子集，其中 $\Sigma=\{0,1\}$，由于 Q 完全被产生答案 1(是)的实例所刻画，因此我们可以把 Q 看作是字符集 Σ 上的语言 L

$$L=\{x\in\{0,1\}^*:Q(x)=1\}$$

例如，对判定问题 Path，我们可以构造相应的语言：

PATH = $\{<G,u,v,k>:G=(V,E)$ is an undirected graph, $u,v\in V, k\geqslant 0$ is an integer, and there exists a path from u to v in G consisting of at most k edges$\}$ 为方便起见，有时判定问题及其相应的语言的名称不加区分。

形式语言的框架可以方便地表示判定问题及其相应求解算法之间的关系。我们先给出几个有用的定义。

定义 11.1 对于给定的输入串 $x\in\{0,1\}^*$，如果算法 A 的输出 $A(x)=1$，则说算法 A 接受串 x。算法 A 所接受的串的集合，称为算法 A 接受的语言 $L=\{x\in\{0,1\}^*:A(x)=1\}$。如果 $A(x)=0$，则算法 A 拒绝串 x。

从定义 11.1 可以看出，即使算法 A 接受一个语言 L，但是对于串 $x\notin L$，算法 A 不一定拒绝串 x，例如算法可能永远循环下去。

定义 11.2 如果算法 A 接受 L 中的每个串，且算法 A 拒绝不在 L 中的串，则说算法 A 能够判定语言 L。如果算法 A 接受语言 L，且对于任意的 $x \in L$，A 能够在多项式时间 $O(n^k)$（n 为串的长度，k 为一个常数）内接受 x，则说算法 A 在多项式时间内接受语言 L。如果算法 A 能够在多项式时间 $O(n^k)$（n 为串的长度，k 为一个常数）内正确判定 x 是否属于 L，则说算法 A 在多项式时间内判定语言 L。

从上面的定义可以看出，算法接受一个语言 L，算法只考虑语言 L 中的串，而判定一个语言，则需要考虑属于 L 中的串以及不属于 L 中的串。例如，有一个算法在多项式时间内接受语言 PATH，这个算法首先验证 G 是否表示一个无向图，验证 u 和 v 是否为 G 中的顶点，然后用宽度优先搜索算法计算 G 中 u 到 v 的路径，然后将计算得到的最小路径的边数与 k 比较。如果 G 表示一个无向图，且 G 中 u 到 v 的最短路径的边数至多为 k，则算法输出 1 且终止。否则，算法永远运行下去。这个算法并不判定语言 PATH，因为这个算法对那些最短路径边数超过 k 的实例，算法并不一定输出 0。语言 PATH 的判定算法一定拒绝不属于 PATH 的串。对于类似 PATH 的判定问题，它对应的判定算法是容易设计的：对那些最短路径边数超过 k 的实例，算法输出 0 并终止即可，其他类似于接受算法。对于停机问题，存在一个接受算法，但是没有判定算法存在。

现在，我们形式化的定义复杂类为语言的集合，成员资格由某种复杂类的度量来确定，例如判定一个给定串是否属于语言 L 所需要的运行时间。这样，我们可以定义复杂类 P。

定义 11.3 $P = \{L \in \{0,1\}^* : \text{there exists an algorthm } A \text{ that decides } L \text{ in polynomial time}\}$。

由定义 11.3 可知，P 类是由所有可以在多项式时间内所判定的问题组成。事实上，P 也是在多项式时间内所接受语言的集合。

定理 11.1 $P = \{L: L \text{ is accepted by a polynomial time algorithm}\}$。

证明：因为由多项式时间算法判定的语言类是多项时间算法可接受语言类的子集。我们只需要证明如果存在多项式时间算法接受语言 L，则存在一个多项式时间算法判定语言 L。假设某个多项式时间算法 A 接受语言 L。我们用一个经典模拟方法来构造另一个能判定语言 L 的多项式时间算法 A'，对某个常数 k，因为算法 A 在多项式时间 $O(n^k)$ 内接受语言 L，即存在一个常数 c 使得 A 在至多 $T = cn^k$ 步接受语言 L。对任何输入串 x，算法 A' 在时间 T 内模拟 A 的计算步骤。在过了时间 T 后，算法 A' 检查 A 的行为。如果 A 接受 x，则算法 A' 接受 x 并输出 1，如果 A 不接受 x，则算法 A' 拒绝 x 并输出 0。算法 A' 模拟 A 的行为，它的运行时间最多为 T 的常数倍。这样就构造出了一个算法 A'，它在多项式时间内判定语言 L。故所证成立。

我们现在寻找能验证语言 L 中成员的算法。例如，假定对于给定的判定问题

PATH 的一个实例 $<G,u,v,k>$，也假定有一条从 u 到 v 的路径 p。我们能够很容易验证路径 p 的边数是否至多为 k。如果情况属实，则我们可以把路径 p 看作该实例确实属于 PATH 的证书。对于判定问题 PATH，验证这个证书并不需要太大的代价，毕竟判定问题 PATH 属于 P。

PATH 能够在线性时间内得到解决，因此验证该证书的时间不会超过求解该问题所花费的时间。对于没有多项式时间判定算法的判定问题，我们希望验证其证书也是容易的。考虑下面汉密尔顿回路问题。

无向图 G 的汉密尔顿回路是通过 G 中每个顶点恰好一次的简单回路。汉密尔顿图(hamiltonian graph)是存在汉密尔顿回路的图。给定一个无向图 $G=(V,E)$，它含有汉密尔顿回路吗？这个问题已经研究了上百年。现在可用语言 HamCycle 定义该问题如下：

$$\text{HamCycle} = \{ <G> : G \text{ is a hamiltonian graph}\}$$

一个算法怎样判定语言 HamCycle？给定问题实例 $<G>$，一个可能的判定算法是穷举所有顶点序列的排列，然后检查给定的一个排列是否是汉密尔顿回路。这个算法的运行需要指数时间。现在考虑一个容易的问题，假定上帝告诉你一个给定的图 G 存在汉密尔顿回路 p，你的任务就是验证这条回路 p 是否是汉密尔顿回路。你只需要检查回路 p 中的顶点是否为图中顶点的一个排列，回路中的边是否确实在图中，这个验证算法确实能够在 $O(n^2)$ 时间内完成验证。因此，验证一个图 G 中给定的回路是否是汉密尔顿回路能够在多项式时间内完成。

我们定义验证算法为带两个参数的算法 A。一个参数是给定的输入串(即输入实例)，另一个参数 y 是一个证书(即解)。如果存在一个证书 y，使得 $A(x,y)=1$，则说算法 A 验证了输入串 y。因此，我们可以定义能够被算法 A 所验证的语言：

$$L = \{x \in \{0,1\}^* : \text{there exists } y \in \{0,1\}^* \text{ such that } A(x,y) = 1\}$$

直观地说，如果对任何的串 $x \in L$，存在证书 y，A 能够用该证书证明 $x \in L$，而且，对任何 $x \notin L$，没有证书能够证明 $x \in L$，则说算法 A 验证了语言 L。例如，考虑汉密尔顿回路问题，证书是一个汉密尔顿回路里的顶点序列。如果一个图是汉密尔顿图，则汉密尔顿回路本身提供了足够的信息验证这个事实。反过来，如果图不是汉密尔顿图，没有顶点序列能够愚弄验证算法相信图是汉密尔顿图。因为验证算法能够仔细检查给定的回路是不是汉密尔顿回路。

定义 11.4 复杂类 NP 指的是一类能够被多项式验证算法所验证语言的集合。

从定义 11.4 可以看出，一个语言 L 属于 NP 当且仅当存在两个输入的多项式时间算法 A 和常数 c，使得

$$L = \{x \in \{0,1\}^* : \text{there exists a certificate } y \text{ with } |y| = O(|x|^c) \text{ such that } A(x,y) = 1\}$$

意思是说，算法 A 在多项式时间内验证了语言 L。

当然 NP 问题的另一种定义是能够用非确定性多项式时间算法求解的判定问题的集合。其中非确定性算法包括两个步骤：

（1）非确定性猜测出一个解，也就是随机生成一个解。

（2）确定性验证该解是否为问题的解，也就是说用确定性算法验证该解是否为问题的一个解，如果是，则返回 True，否则返回 False 或者不停下来。

如果非确定性算法在第（2）步的运行时间为多项式时间复杂度，则称非确定性算法为非确定性多项式时间算法。例如，对于汉密尔顿回路问题，可以设计一个非确定性（Nondetermined）算法：

```
NondeterminedHamCycle(G)
1   S←V
2   p←∅
3   for i←1 to |V| do
4         randomly select one vertex v from S
5         p←p∪{v}
6         S←S-{v}
7   if verify(G,p) = 1 then return True
8   else return False
```

其中，verify(G,p) 验证顶点序列是否是判定问题汉密尔顿回路问题的解。V 是图 G 的顶点集合，算法 NondeterminedHamCycle(G) 是非确定性多项式时间算法，其中第 3 行～第 6 行是非确定性猜测阶段，而第 7 行和第 8 行是确定性验证阶段。对于一个具有汉密尔顿回路的问题，非确定性算法总会在某次执行中猜中一个解（例如多次的猜测），并使得算法返回 True。

事实上，NP 问题的两个定义其含义上没有什么多大差别，前面的定义是说明任意给定一个输入串（问题实例），存在一个证书（解），这个证书可以看作是上帝给出的解，使得多项式验证算法的输出为 1。而后者的定义，是猜中一个解，至于怎么样猜中，花多长时间则不管它，然后多项式验证该解使其输出为 True。

现在我们知道，NP 问题中包括了许多 P 问题，即如果 $L \in P$，则 $L \in NP$，即 $P \subseteq NP$，也就是说，NP 问题中有些问题是容易的。是否 $NP \subseteq P$ 是不知道的，但是大多数的研究者相信，P 和 NP 不同类，即 $P \neq NP$。NP 类由那些问题的解能很快验证的问题所构成。从我们的直觉和经验知道，解一个问题比验证该问题的解要困难得多。很多理论计算机科学家相信，NP 确实包括一些不属于 P 的语言，即 NPC。除了 P 是否等于 NP 这个问题外，还有一个有趣的问题，令 co-NP = $\{L : \bar{L} \in NP\}$，即 NP 类在补运算下是否封闭的问题。例如，HamCycle $\in$ NP，HamCycle 的补问题可以描述如下：对于给定的一个无向图 G，不存在一条汉密尔顿回路吗？co-NP 是否等于 NP 的问题，也仍然悬而未决，但是大多数研究者相信 co-NP $\neq$ NP。

11.3 NPC

11.3.1 NPC 的定义

大多数理论计算机科学家相信 P≠NP，是因为他们确实找到一类问题 NPC，这类问题有令人非常惊奇的性质，即如果 NPC 中任何一个问题能够在多项式时间内找到最优解，则 NP 中的每个问题都能在多项式时间内找到最优解，即 P = NP。尽管研究了大半个世纪，但是还没有找到任何一个 NPC 问题的多项式时间算法。在给出 NPC 的定义之前，我们先介绍问题约简的概念。

直观地，如果一个问题 Q 的任一实例能够容易地转化为另一个问题 Q' 的一个实例，则说问题 Q 能够约简到问题 Q'，这样问题 Q' 实例的解就可以看作问题 Q 相应实例的解。例如解线性方程的问题可以约简到解二次方程。例如线性方程的一个实例 $ax+b=0$ 可以转化为二次方程的一个实例 $0x^2+ax+b=0$，这样实例 $0x^2+ax+b=0$ 的解就是实例 $ax+b=0$ 的解。这样问题 Q 能够约简到问题 Q'，从某种意义上说，Q 不会比 Q' 更难解。

定义 11.5 给定一个函数 $f:\{0,1\}^*\rightarrow\{0,1\}^*$，对任意给定的 $x\in\{0,1\}^*$，如果存在一个多项式时间算法 A，算法产生输出 $f(x)$，则称 f 是多项式时间可计算的。

定义 11.6 给定语言 L_1 和 L_2，如果存在多项式时间可计算的函数 $f:\{0,1\}^*\rightarrow\{0,1\}^*$ 使得对任意的 $x\in\{0,1\}^*$，$x\in L_1$ 当且仅当 $f(x)\in L_2$，则说语言 L_1 能多项式时间约简到 L_2，记为 $L_1\leqslant_P L_2$。函数 f 称为约简函数，计算函数 f 的算法 A 称为约简算法。

定义 11.6 中的符号 $\leqslant_P$ 表示多项式时间约简，$L_1\leqslant_P L_2$ 意味着语言 L_1 在难度上不会超过语言 L_2。

图 11.2 描述了将一个语言 $L_1\subseteq\{0,1\}^*$ 多项式时间约简到语言 $L_2\subseteq\{0,1\}^*$ 的例子。约简函数 f 提供了一个多项式时间映射，使得如果 $x\in L_1$，则 $f(x)\in L_2$。

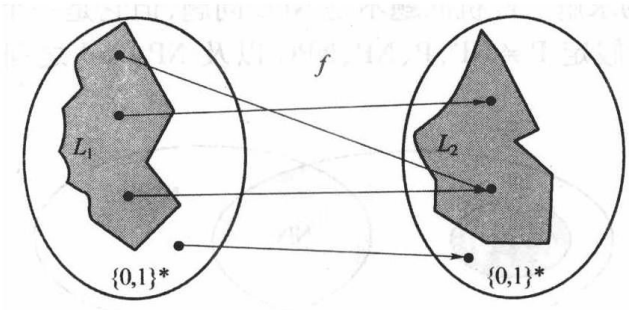


图 11.2

而且如果 $x \notin L_1$，则 $f(x) \notin L_2$。这样约简函数将由语言 L_1 表示的判定问题的任何一个实例 x，映射到由 L_2 表示的判定问题的一个实例 $f(x)$。因而，是否 $f(x) \in L_2$ 的答案提供了是否 $x \in L_1$ 的答案。

引理 11.1 如果语言 $L_1, L_2 \subseteq \{0,1\}^*$ 使得 $L_1 \leqslant_P L_2$，且 $L_2 \in \mathrm{P}$，则 $L_1 \in \mathrm{P}$。

证明：令 A_2 是判定 L_2 的多项式时间算法，令 A 是计算约简函数 f 的多项式时间约简算法，为了证明引理，只需构造一个能够判定 L_1 的多项式时间算法 A_1 即可。

图 11.3 描述了 A_1 的构造。对于一个给定的输入 $x \in \{0,1\}^*$，算法 A_1 用 A 将实例 x 转化为实例 $f(x)$，然后用 A_2 来测试是否 $f(x) \in L_2$。A_2 的输出作为 A_1 的输出。A_1 的正确性来自于定义 11.6。由于 A 和 A_2 为多项式时间算法，所以 A_1 也为多项式时间算法。故所证成立。

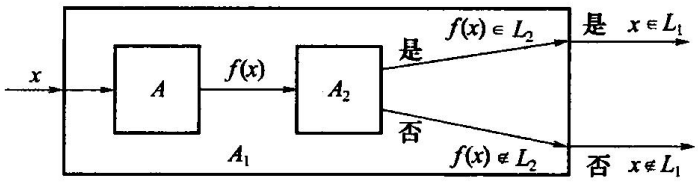


图 11.3

多项式时间约简提供了一种证明一个问题至少与另一个问题一样难的形式化方法。下面给出 NPC 的定义。

定义 11.7 给定语言 $L \subseteq \{0,1\}^*$，L 是 NP 完全的当且仅当

(1) $L \in \mathrm{NP}$；

(2) 对于所有 $L_1 \in \mathrm{NP}$，有 $L_1 \leqslant_P L$。

如果有一个语言 L 满足上述性质(2)，但不一定满足性质(1)，则称该语言是 NP 难的。并记 NPC 表示所有 NP 完全语言构成的语言类。NPC 是 NP 中最难的问题。从前面最优化问题以及判定问题的分析可知，NP 难问题(NP-hard)对应最优化问题。因此理论上分析问题的复杂性，我们利用 NPC 的概念。而现实问题中，则考虑 NP 难问题的求解。停机问题不是 NPC 问题，但它是一个 NP 难问题(为什么，见习题 11.9)。假定 $\mathrm{P} \neq \mathrm{NP}$，P、NP、NPC 以及 NP-hard 之间的关系如图 11.4 所示。

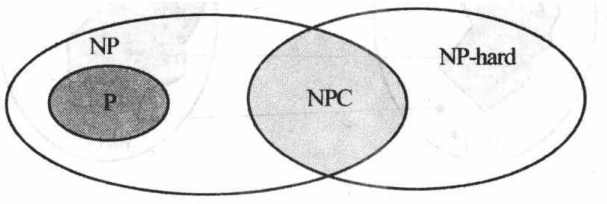


图 11.4

定理 11.2 如果任何一个 NPC 问题多项式时间可解，则 P = NP。等价地，如果 NP 中的任何问题没有多项式时间求解算法，则没有 NPC 问题是多项式时间可解的。

证明：任给 $L \in \text{NPC}$，且 $L \in \text{P}$。任给 $L_1 \in \text{NP}$，按照 NPC 的定义(2)，有 $L_1 \leqslant_P L$。由于 $L \in \text{P}$ 按照引理 11.1，有 $L_1 \in \text{P}$，故所证成立。

11.3.2 电路可满足性问题

布尔电路是一个无环的有向图，图中每个顶点对应一个逻辑门，它对应简单的布尔函数，例如 NOT、AND 和 OR。逻辑门的入边是其相应布尔函数的输入，如果入边的起点没有连接到一个逻辑门，则该入边为布尔电路的输入。逻辑门的出边是其相应布尔函数的输出，如果一个输出的终点不再连接到一个逻辑门，则它称为整个布尔电路的输出，如图 11.5 所示。

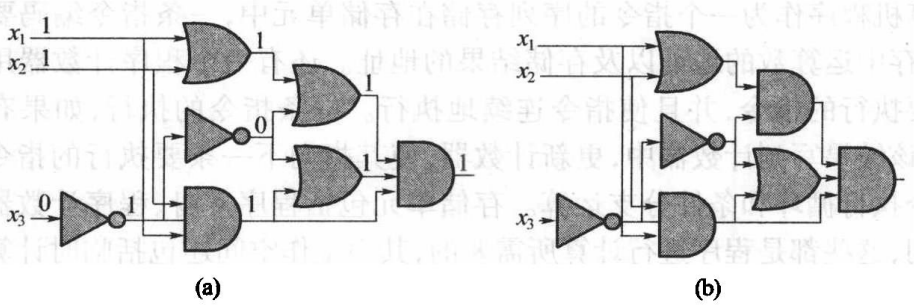


图 11.5

给定一个布尔组合电路，其输入为 x_1, x_2, x_3，是一组 0,1 赋值。具有一个输出的布尔组合电路是可满足的，指的是它有一组可满足的赋值，使得组合电路的输出为 1。

图 11.5 给出了电路可满足性问题的两个实例，其中图 11.5(a)给出了一个可满足的电路，赋值 $x_1 = 1, x_2 = 1, x_3 = 0$ 使得电路的输出为 1，因而是可满足的。图 11.5(b)给出了一个不可满足的电路，不管输入取何值，电路的输出总为 0，因而该布尔电路是不可满足的。

电路可满足性(Circuit Satisfiablity，CircuitSAT)问题指的是，给定一个由逻辑门 AND、OR 和 NOT 构成的组合电路 C，它可满足吗？从形式语言的角度，可以定义如下：

$$\text{CircuitSAT} = \{C : C \text{ is a satisfiable boolean combinational circuit}\}$$

引理 11.2 CircuitSAT 属于 NP。

证明：按照 NP 问题的定义，只要我们构造一个具有两个输入的多项式时间算法 A，它能在多项式时间内验证 CircuitSAT 即可。算法 A 的一个输入是一个布尔组合电路，另一个输入是一个证书，它对应布尔变元的一组赋值。

算法 A 可以构造如下：对于电路中的每个逻辑门，它检测逻辑门的输出是否是

相应布尔函数输入值的函数值，如果整个电路的输出为1，则算法 A 的输出为1，因为输入值使得电路 C 可满足，否则，算法 A 输出为0。

无论什么时候，一个可满足的电路作为算法 A 的输入，有一个证书，它的长度是关于电路大小的多项式，它使得算法 A 输出为1。无论什么时候，一个不可满足的电路作为算法 A 的输入，则没有一个证书能愚弄算法 A，使得算法 A 输出为1，即电路是可满足的。算法 A 是多项式时间算法，这样 CircuitSAT 能够在多项式时间内得到验证，因此 CircuitSAT 属于 NP。

要证明电路可满足性问题是 NP 完全的，还需要证明电路可满足性问题是 NP 难的。严格地证明电路可满足性问题是 NP 难问题是非常困难的，这里我们给出一个虽然不是很严格，但是容易理解的证明。

在证明之前，我们先介绍一些概念。前面我们说过，任何一个算法都可以用简单的计算模型（例如 RAM）实现。通常，计算模型由 CPU 和可寻址的存储单元组成。计算机程序作为一个指令的序列存储在存储单元中，一条指令编码要执行的运算、内存中运算数的地址以及存储结果的地址。还有一个程序计数器用来跟踪下一条要执行的指令，并且使指令连续地执行。一条指令的执行，如果有输出结果，就将该结果写到计数器中，更新计数器，使其指向下一条要执行的指令。可以允许指令执行循环和条件分支运算。存储单元包括程序本身、程序计数器、输入、工作空间，这些都是程序进行计算所需要的，其中工作空间还包括临时计算所需要的寄存器等辅助设备。因此计算的整个状态用存储单元确定。为方便起见，我们把计算机存储单元的任何一个特别的状态称为一个格局。指令的执行能够看作映射一个格局到另一个格局。完成这种映射的计算机硬件能够实现为一个组合电路 M。下面，我们给出电路可满足性问题是 NP 难的证明。

引理 11.3 CircuitSAT 是 NP 难的。

证明：令 L 是 NP 中的任何一个语言，我们将描述一个多项式时间算法 F，它计算约简函数 f。对于每个二进制串 x，f 能够将其映射到一个组合电路，即 $C=f(x)$，使得 $x\in L$ 当且仅当 $C\in$ CircuitSAT。

因为 $L\in$ NP，一定存在一个算法 A，它能在多项式时间内验证 L。我们将构造的算法 F 是一个具有两个输入的算法，它能计算约简函数 f。

对于长度为 n 的输入串 x，令 $T(n)$ 表示算法 A 计算 x 的最坏情形的运行时间。这里算法 A，包括两个输入，输入 x 以及证书 y，且 $|y|=|x|^c$。因此 $T(n)=O(n^k)$，k 是一个常数。算法 A 的计算可以看作一个格局序列，从初始格局 c_0 出发，每个格局 c_i 被组合电路 M 映射到下一个格局 c_{i+1}，当算法 A 完成计算的时候，算法的输出0或1被写到工作带的某个指定的位置。由于算法 A 最多运行 $T(n)$ 步，因此算法的输出出现在格局 $c_{T(n)}$ 的某个位置。整个算法 A 的过程如图 11.6 所示。

约简算法 F 构造一个大的组合电路 C，从初始格局开始，计算所有的格局。想法就是把 $T(n)$ 个组合电路连接在一起，构成一个大的组合电路 C。第 i 个电路的

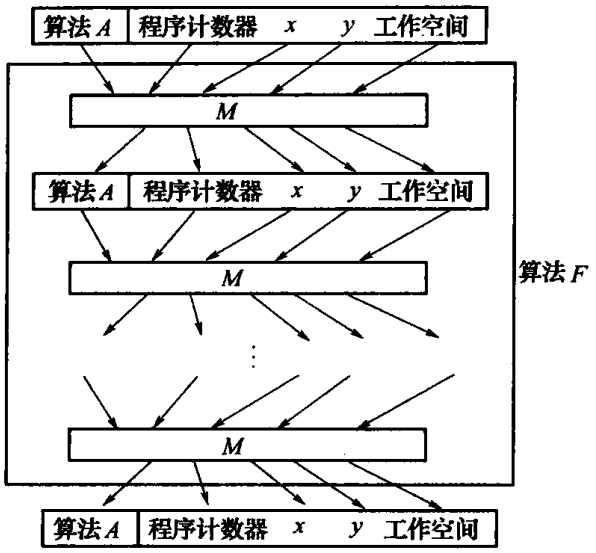


图 11.6

输出 c_i 直接作为第 $i+1$ 个电路的输入。

现在来看看算法 F，给定一个输入 x，它首先计算电路 $C=f(x)$，C 是可满足的当且仅当存在一个证书 y 使得 $A(x,y)=1$。当 F 获得一个输入 x，它首先计算 $n=|x|$，然后构造一个组合电路 C。电路 C 的输入是一个初始格局，对应着计算 $A(x,y)$，输出为格局 $c_{T(n)}$ 上的某一位，对应着 A 的输出，其余中间的输出都不考虑。这样构造的电路 C，对任何长度为 $|x|^c$ 的证书 y，计算 $C(y)=A(x,y)$。对于给定的输入 x，约简算法 F 构造出这样的电路 C，然后输出它。

下面，我们必须证明，F 正确地计算了约简函数 f，也就是说，我们必须证明 C 是可满足的当且仅当存在一个证书 y，使得 $A(x,y)=1$。事实上，假定存在一个长度为 $|x|^c$ 的证书 y，使得 $A(x,y)=1$，则我们可以将 y 作为电路的输入，电路的输出为 $C(y)=A(x,y)=1$，因此电路 C 是可满足的。反过来，假定电路 C 是可满足的，则存在一个输入 y，使得 $C(y)=1$，因而可以推断出 $A(x,y)=1$。因此，算法 F 正确地计算了约简函数 f。

此外，我们还必须证明算法 F 运行在多项式时间内。算法 A 本身的时间复杂度为 $T(n)=O(n^k)$，由于算法至多运行 $O(n^k)$，因此其时间复杂度仍为多项式时间，因此工作空间也是多项式大小。因此，需要表示一个格局的位数也是多项式大小。因此，实现计算机硬件的组合电路的大小是关于格局数的多项式，因此也是关于 n 的多项式。整个电路 C 最多由 $O(n^k)$ 个 M 组成，因此电路 C 的大小也是 n 的多项式。构造 C 的每一步需要多项式时间，因此 C 的构造能够由算法 F 在多项式时间内完成，因此算法 F 运行在多项式时间内。

综上所述，电路可满足性问题是 NP 难的。

根据 NP 完全问题的定义，我们马上可以得到。

定理 11.3 CircuitSAT 是 NP 完全的。

11.4 NPC 的证明

前面一节，我们根据 NP 完全问题的定义，证明了电路可满足性问题是 NP 完全的。下面我们介绍一个非常有用的结论，用它可以很方便地证明一个问题是 NP 完全的。

引理 11.4 如果 L 是一个语言，对某个语言 $L' \in \text{NPC}$，使得 $L' \leqslant_P L$，则 L 是 NP 难的。而且，如果有 $L \in \text{NP}$，则 $L \in \text{NPC}$。

证明：因为 L' 是 NP 完全的，对任意的 $L'' \in \text{NP}$，我们有 $L'' \leqslant_P L'$，按照假设有 $L' \leqslant_P L$，这样按照传递性，我们有 $L'' \leqslant_P L$，这表明 L 是 NP 难的。如果 $L \in \text{NP}$，则由 NP 完全的定义，可知 $L \in \text{NPC}$，故所证成立。

换句话说，通过将一个已知的 NPC 约简到 L，我们简化了将 NP 中的每一个语言约简到 L 的证明过程。从而利用引理 11.4，我们得到证明一个语言属于 NP 完全的方法：

（1）首先证明 $L \in \text{NP}$。

（2）最后证明 L 是 NP 难的，其证明过程如下：

① 首先选择一个已知属于 NP 完全的语言 L'；

② 设计一个计算函数 f 的算法，能将 L' 的每个实例 $x \in \{0,1\}^*$ 映射到 L 的实例 $f(x)$；

③ 证明上述转换函数 f 满足，对任意的 $x \in \{0,1\}^*$，$x \in L'$ 当且仅当有 $f(x) \in L$；

④ 证明计算函数 f 的算法是多项式时间算法。

下面，我们根据上述证明思路，来证明几个 NP 完全问题。

11.4.1 可满足性问题

给定一个布尔公式 ϕ，其构成如下：

（1）n 个布尔变元：$x_1, x_2, \cdots, x_n$。

（2）m 个布尔连接符：任何一个布尔连接符（或称布尔函数）连接一个或两个输入，具有一个输出。常见的布尔连接符有 $\wedge$，$\vee$，$\neg$，$\rightarrow$，$\leftrightarrow$。

（3）括号。

我们可以很容易地编码一个布尔公式，使其长度为 $n+m$ 的多项式。类似布尔组合电路，一个布尔公式的真值赋值是关于布尔变元的一组取值，一个可满足的赋值是一个真值赋值，它使得布尔公式的值为 1。如果一个公式具有可满足赋值，则称该公式是可满足的。可满足性问题（Satisfiablity，SAT）指的是一个给定的布尔公式是否是可满足的。用形式语言可以描述如下：

$$\text{SAT} = \{\phi: \text{公式 } \phi \text{ 是可满足的}\}$$

则可满足性问题可以描述为,对于给定的 ϕ,ϕ 是否属于 SAT。

例如,给定布尔公式

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

赋值,$\langle x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1\rangle$ 是一个可满足的赋值。因为

$$\phi = ((0 \rightarrow 0) \vee \neg((\neg 0 \leftrightarrow 1) \vee 1)) \wedge \neg 0 = (1 \vee \neg(1 \vee 1)) \wedge 1 = 1$$

因此上述公式 $\phi \in$ SAT。

定理 11.4 SAT 是 NP 完全的。

证明:首先我们证明 SAT $\in$ NP。对于一个布尔公式 ϕ 以及它的一个可满足赋值构成的证书 y,验证算法 A 只要将公式 ϕ 中的变元用其赋值代替,然后计算公式 ϕ 的值,这个过程显然能够在多项式时间内完成,如果 $\phi = 1$,则它是可满足的。因此 SAT $\in$ NP。

再证 SAT 是 NP 难的。对于已知的 NP 完全问题 CircuitSAT,如果我们能够证明 CircuitSAT $\leqslant_P$ SAT,则问题得证。首先要证明 CircuitSAT 的任意一个实例能够在多项式时间内约简到可满足性问题的一个实例。我们可以按照下列方式进行约简,对于组合电路 C 的每个输入 x_i,布尔公式 ϕ 有对应的变元 x_i。电路中一个逻辑门能够表达为一个公式,公式由逻辑门的输入和输出决定。例如输出 AND 逻辑门能够表达成公式 $x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)$,如图 11.7 所示。

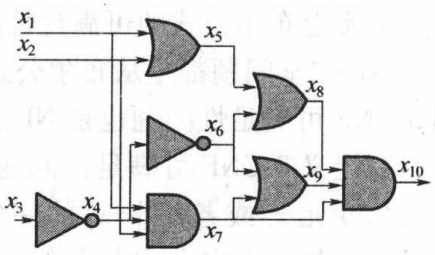


图 11.7

对于图 11.7 所示的布尔电路 C,其对应的布尔公式如下

$$\begin{aligned}\phi = {} & x_{10} \wedge (x_4 \leftrightarrow \neg x_3) \\ & \wedge (x_5 \leftrightarrow (x_1 \vee x_2)) \\ & \wedge (x_6 \leftrightarrow \neg x_4) \\ & \wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4)) \\ & \wedge (x_8 \leftrightarrow (x_5 \vee x_6)) \\ & \wedge (x_9 \leftrightarrow (x_6 \vee x_7)) \\ & \wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9))\end{aligned}$$

上述转换能够在多项式时间内完成。由此可见,任意给定一个布尔电路 C,我们能够在多项式时间内将其约简为布尔公式 ϕ。这样如果 C 有一个可满足的赋值,电路的每条输入线都有一个确定的值,电路的输出为 1。因此每条输入线的赋值对应布尔公式的一个赋值,使得公式中的每个子句的值均为 1,所有子句的合取仍然为 1。反过来,如果有一组赋值使得布尔公式 ϕ 取值为 1,则类似可证明相应

电路 C 是可满足的。因此,我们就证明了 SAT 是 NP 难的。综上所述,SAT 是 NP 完全的。

定理 11.4 就是著名的 Cook-Levin 定理,简称 Cook 定理。虽然,本书是从证明 CircuitSAT 属于 NP 完全开始的,是因为 CircuitSAT 的证明比较容易理解,而直接证明 SAT 属于 NP 完全则要复杂些,但是证明它们的思路是一样的。事实上,SAT 才是第一个被证明的 NP 完全问题。Cook 定理揭开了计算复杂性中 NP 完全理论的研究。在此基础上关于 NP 完全问题的研究,成为过去几十年来计算机科学最活跃和重要的研究领域之一。Stephen A. Cook 因为其在计算复杂性领域的开创性贡献而获得 1982 年的图灵奖。

11.4.2 3-CNF 可满足性问题

事实上布尔公式的可满足性问题是第一个 NP 完全问题,而且非常经典。因为许多 NP 完全问题都是从布尔公式的可满足问题出发而得到证明的。下面我们证明 3-CNF 可满足性问题也是 NP 完全的。

为定义 3-CNF 可满足性问题,我们先给出一些术语。布尔公式中的一个文字是布尔变元 x_i 或者其非 $\neg x_i$ 的一个出现。如果一个布尔公式,是一些子句的合取,而且子句是一个文字或者多个文字的析取,则该公式称为合取范式(CNF)。如果 CNF 中每个子句都精确地有 3 个不同的文字,则该公式称为 3-CNF。

例如,布尔公式

$$(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$$

是 3-CNF,其中每个子句含有 3 个不同的文字。

从形式语言的角度定义 3-CNF 可满足性问题如下

$$\text{3-CNF-SAT} = \{\phi : \text{3-CNF 公式 } \phi \text{ 是可满足的}\}$$

3-CNF-SAT 问题就是给定一个 3-CNF 公式,它是否是可满足的。

下面我们证明 3-CNF-SAT 问题是 NP 完全的。

定理 11.5 3-CNF-SAT 是 NP 完全的。

证明:类似定理 11.4 的证明,我们能够证明 3-CNF-SAT 属于 NP。下面我们只需证明 $\text{SAT} \leqslant_P \text{3-CNF-SAT}$。为此,我们需要构造一个约简算法,它由以下三步组成。

第一步,对于任意给定的输入公式 ϕ,我们构造一棵二叉树,文字作为叶子,连接符作为内部节点,对每个连接符引进一个新变元 y,作为连接符的输出。图 11.8 给出了由公式 $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$ 构造出的一棵二叉树。

这棵二叉树现在能够看成一个计算函数值的组合电路。类似定理 11.4 的证明,我们能够把原始的布尔公式写成根变元和子句的合取,即

$$\begin{aligned}\phi' = & y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ & \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\ & \wedge (y_4 \leftrightarrow \neg y_5) \\ & \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))\end{aligned}$$

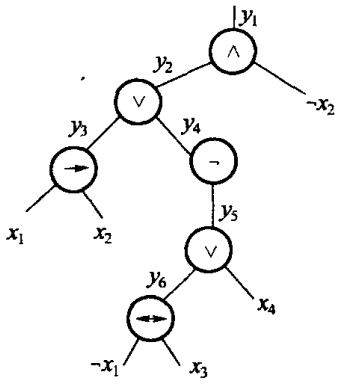


图 11.8

上述公式 ϕ' 是子句 ϕ'_i 的合取，每个子句 ϕ'_i 最多 3 个文字。

第二步，将上述公式 ϕ' 转换为合取范式。对每个子句 ϕ'_i，我们可以构造真值表。真值表的每一行是子句 ϕ'_i 的变元的一组可能的赋值，以及在这个赋值下子句 ϕ'_i 的值。对真值表中子句 ϕ'_i 的值为 0 的赋值，我们可以建立一个析取范式(DNF)，它是子句的析取，子句为文字的合取，它等价于 $\neg\phi'_i$。通过 DeMorgan 定律(对所有文字取反，然后将 $\vee$ 变成 $\wedge$，$\wedge$ 变成 $\vee$)，可以得到一个 CNF。下面给出一个子句的转化例子。对公式 ϕ' 的一个子句 $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$，我们转化如下。

首先对该子句构造真值表，如图 11.9 所示。

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$	y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0	0	1	1	1
1	1	0	1	0	1	0	0
1	0	1	0	0	0	1	1
1	0	0	0	0	0	0	1

图 11.9

由真值表，可以得到 DNF 公式

$$\begin{aligned}&(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \\ &\vee (\neg y_1 \wedge y_2 \wedge \neg x_2)\end{aligned}$$

应用 DeMorgan 定律，我们得到 CNF 公式

$$\begin{aligned}\phi''_i = & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \wedge \\ & (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2)\end{aligned}$$

上述公式等价于子句 $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$。

公式 ϕ' 中的每一个子句 ϕ'_i 都可以类似地转化为 CNF 公式 ϕ''_i，而且 ϕ''_i 中最多含有 3 个文字，令 ϕ'' 为上述 ϕ''_i 的合取。从而，公式 ϕ' 等价于 CNF 公式 ϕ''。

约简算法的第三步是进一步转化公式 ϕ''，使得 ϕ'' 中的每个子句精确含有 3 个不同的文字。为了完成这一步，我们引进两个辅助变元 p 和 q。对公式 ϕ'' 的子句

ϕ''_i，公式 ϕ''' 包括下列子句：

（1）如果 ϕ''_i 有 3 个不同的文字，只需要将 ϕ''_i 加入到 ϕ''' 的子句中即可。

（2）如果 ϕ''_i 有 2 个不同的文字，例如 $\phi''_i = l_1 \vee l_2$，只需要将子句 $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ 加入到 ϕ''' 的子句中即可。

（3）如果 ϕ''_i 仅有 1 个文字 l，只需要将子句 $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$ 加入到 ϕ''' 的子句中即可。

事实上，可以验证上述（2）、（3）步中，ϕ''_i 与新生成的子句是等价的。可以看出，通过仔细验证约简算法的三步，可以得到 3-CNF 公式 ϕ''' 是可满足的当且仅当 ϕ 是可满足的。事实上，上述每一步都是等价的。

下面我们证明上述约简算法能够在多项式时间内完成约简。事实上，从 ϕ 构造出 ϕ'，对公式 ϕ 中的连接符，至多多出一个变元和一个子句。由于 ϕ' 的每个子句至多 3 个变元，真值表至多为 8 行，因而从 ϕ' 构造出 ϕ'' 至多多出 8 个子句。从 ϕ'' 构造出 ϕ'''，至多多出 4 个子句。因此公式 ϕ''' 的规模是原始公式 ϕ 编码长度的多项式，而且每个构造都能很容易地在多项式时间内完成。故所证成立。

11.4.3 团问题

令 V' 为无向图 $G=(V,E)$ 顶点 V 的子集，当且仅当对于 V' 中的任意顶点 u 和 v，(u,v) 是图 $G=(V,E)$ 的一条边时，V' 定义了一个完全子图，一个团就是图 G 的一个完全子图，一个团的大小就是该团所含顶点的数目。

以最优化形式描述的团（Clique）问题：给定一个无向图 $G=(V,E)$，问题就是在图 $G=(V,E)$ 中找一个团，使其所含的顶点数最多。

以判定形式描述的团问题：给定一个无向图 $G=(V,E)$ 和一个正整数 k，判定图 G 是否包含一个大小为 k 的团，即是否存在 V 的子集 V'，使得对任意 $u \in V'$，$v \in V'$，有 $(u,v) \in E$ 且 $|V'|=k$？从形式语言的角度，定义语言

$$\text{Clique} = \{ <G,k> : \text{图 } G \text{ 有大小为 } k \text{ 的团} \}$$

定理 11.6 Clique 是 NP 完全的。

证明：我们首先证明 Clique $\in$ NP。对于给定的一个无向图，我们用团的顶点集 $V' \subseteq V$ 作为证书。检查 V' 是否是一个团，我们只需要检查一个 V' 中任意两个顶点 u 和 v，(u,v) 是否属于 E，这个过程显然可以在多项式时间内完成，因此，Clique $\in$ NP。

下面我们利用 3-CNF-SAT $\leqslant_P$ Clique 来证明 Clique 是 NP 难的，约简算法可以构造如下。

对于 3-CNF-SAT 的任何一个实例 $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$，子句 $C_r (1 \leqslant r \leqslant k)$ 精确地有 3 个不同的文字 l'_1，l'_2 和 l'_3。下面，我们构造一个图 G 使得 ϕ 是可满足的，当且仅当 G 有大小为 k 的团。图 G 可以构造如下：对每个子句 $C_r = (l'_1 \vee l'_2 \vee l'_3)$，我们将 l'_1，l'_2 和 l'_3 看成图 G 中的 3 个顶点，对于两个顶点 l'_i 和 l'_j，如果下列两个条件

成立：

（1）l_i^r 和 l_j^s 属于不同的子句，即 $r \neq s$。

（2）文字 l_i^r 不是 l_j^s 的非。

则在图 G 中，将顶点 l_i^r 和 l_j^s 用一条边连接起来。这个图可以在多项式时间内构造出来。下面，我们举一个例子。

给定 3-CNF 公式 $\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$，例如子句 C_1 中，文字 x_1 可与子句 C_2 中的 x_2 和 x_3 用边相连，还可与子句 C_3 中的文字 x_1、x_2 和 x_3 用边相连，按照上述办法，我们构造出的图 11.10。公式 ϕ 的一个可满足的赋值为 $x_2 = 0, x_3 = 1, x_1$ 的值为 1 或者 0 均可。这个赋值 $x_2 = 0$ 使得子句 $(x_1 \vee \neg x_2 \vee \neg x_3)$ 可满足，赋值 $x_3 = 1$ 使得子句 $(\neg x_1 \vee x_2 \vee x_3)$ 和 $(x_1 \vee x_2 \vee x_3)$ 可满足。对应一个团集 $\{\neg x_2, x_3, x_3\}$，大小为 3，如图 11.10 中灰色顶点所示。

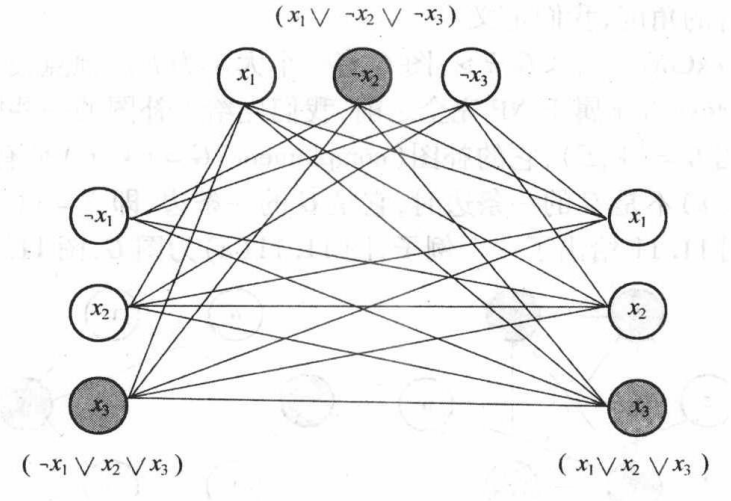


图 11.10

现在我们证明，ϕ 是可满足的当且仅当 G 有大小为 k 的团。

假定 ϕ 是可满足的，即公式 ϕ 存在一个可满足的赋值，则每个子句 C_r 中，至少有一个文字 l_i^r，其值为 1，这样的文字对应图 G 中的顶点 l_i^r。从每个子句中选择一个赋值为 1 的文字，这样就构造了一个大小为 k 的顶点集合 V'，现在我们证明该集合 V' 是一个团。事实上，对 V' 中的任意两个顶点 l_i^r 和 $l_j^s (r \neq s)$，由于其对应文字的赋值为 1，因此文字 l_i^r 不是文字 l_j^s 的非，所以按照图的构造，$(l_i^r, l_j^s) \in E$，因此集合 V' 是一个团。

反过来，假定集合 V' 是一个大小为 k 的团，按照图的构造，同一个子句里的文字，其对应的顶点在图中是没有连接的。因此 V' 中的任意两个顶点，其对应的文字不属于同一个子句。即每个子句都有一个文字，其相应的顶点属于 V'。因此，我们只要对团中的顶点其对应的文字取值为 1，便可以使得每个子句为可满足的。因此所证成立。

3-CNF-SAT 和 Clique，初看起来，它们之间没有什么关系，但是只要认真思考，就会发现它们之间存在紧密的联系，从而方便地将 3-CNF-SAT 约简到 Clique，并给出证明。

11.4.4 顶点覆盖问题

给定一个无向图 $G=(V,E)$，寻找 V 的一个最小子集 V'，使得如果 $(u,v)\in E$，则有 $u\in V'$，或者 $v\in V'$（或者 u,v 均属于 V'）。顶点覆盖 V' 的大小 $|V'|$ 即为它包含的顶点个数。

顶点覆盖（Vertex Cover）问题的判定问题可以描述为：给定一个无向图 $G=(V,E)$ 和一个整数 k，问是否存在一个子集 V'，使得如果 $(u,v)\in E$，则有 $u\in V'$，或者 $v\in V'$（或者 u,v 全部属于 V'），且其大小为 k？

从形式语言的角度，我们定义

$$\text{VertexCover}=\{<G,k>:\text{图 } G \text{ 有一个大小为 } k \text{ 的顶点覆盖}\}$$

在证明 VertexCover 属于 NP 完全之前，我们先给出补图的一些概念。

对于任意图 $G=(V,E)$，它的补图（complement）$\overline{G}=(V,\overline{E})$ 是有同样顶点集的图，当且仅当 (u,v) 不是 G 的一条边时，它是 $\overline{G}$ 的一条边，即 $\overline{E}=\{(u,v):u,v\in V \text{ 且 } (u,v)\notin E\}$。图 11.11 给出了一个例子，图 11.11(a) 为图 G，图 11.11(b) 为图 $\overline{G}$。

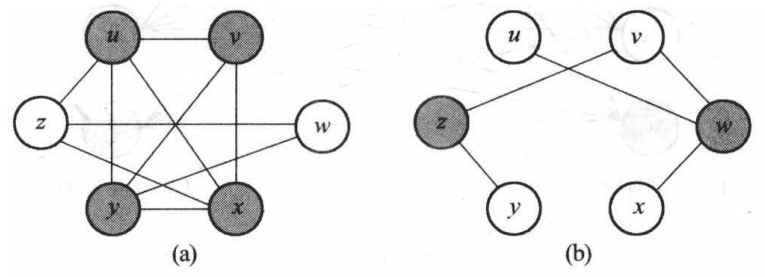


图 11.11

定理 11.7 VertexCover 是 NP 完全的。

证明：首先，我们证明 VertexCover $\in$ NP。假设给定一个无向图 $G=(V,E)$ 以及一个整数 k。我们选择的证书是一个顶点覆盖 $V'\subseteq V$，验证算法只需要先确认 $|V'|=k$，然后对每条边 $(u,v)\in E$，检查是否有 $u\in V'$，或者 $v\in V'$，显然验证可在多项式时间内完成。故 VertexCover $\in$ NP。

其次，通过将 Clique $\leqslant_P$ VertexCover 来证明 VertexCover 是 NP 难的。约简算法主要基于补图的概念。对于团问题的任意一个实例 $<G,k>$，我们很容易在多项式时间内构造出相应的补图 $\overline{G}$，从而可以得到顶点覆盖问题的一个实例 $<\overline{G},|V|-k>$。下面只需证明图 G 有一个大小为 k 的团当且仅当图 $\overline{G}$ 有一个大小为 $|V|-k$ 的顶点覆盖。例如图 11.11(a)，团集 $V'=\{u,v,x,y\}$，其大小为 4，图 11.11(b) 为图 G 的补图，其顶点覆盖为 $\{w,z\}$，其大小为 $6-4=2$。

假定 G 有一个团 $V' \subseteq V$ 且其大小为 k。我们首先证明 $V - V'$ 是图 $\bar{G}$ 的顶点覆盖。任给 $(u,v) \in \bar{E}$，则 $(u,v) \notin E$，由于 V' 中任意一对顶点都是相连的，因而 u 或者 v 至少有一个顶点不在 V' 中，否则与 $(u,v) \in \bar{E}$ 矛盾。也就是说，u 或者 v 至少有一个顶点在 $V - V'$ 中，因此边 (u,v) 被集合 $V - V'$ 覆盖。从而 $\bar{E}$ 中任一条边都被 $V - V'$ 覆盖。因此，$V - V'$ 是图 $\bar{G} = (V,\bar{E})$ 的一个顶点覆盖，且其大小为 $|V| - k$。

反过来，假定图 $\bar{G} = (V,\bar{E})$ 有一个顶点覆盖 V'，且其大小为 $|V| - k$。则对任意的 $u \in V, v \in V$，如果 $(u,v) \in \bar{E}$，则 $u \in V'$，或者 $v \in V'$ 或者均在 V'。反过来，对任意的 $u \in V, v \in V$，如果 $u \notin V'$ 且 $v \notin V'$，则 $(u,v) \in E$，这意味着 $V - V'$ 构成图 G 的一个完全子图，其大小为 $|V| - |V'| = k$。故 VertexCover 是 NP 难的。

综上所述，故所证成立。

11.5 其他 NP 完全问题

到目前为止，已经找到了大约 4000 个 NP 完全问题，下面只介绍几个经典的 NP 完全问题。

图着色(Graph Coloring)问题，其最优化问题可以描述为：给定一个无向图 $G = (V,E)$，需要对图 G 中的每个顶点着色，使得相邻的两个顶点有不同的颜色，求最小的颜色数。相应的判定问题(GraphColoring)可以描述为：给定一个无向图 $G = (V,E)$ 以及一个正整数 m，问顶点能否用 m 种颜色进行着色，使得相邻的两个顶点有不同的颜色。

集合覆盖(Set Cover)问题，其最优化问题可以描述为：给定一个有限集合 X，F 为 X 的子集的集合且子集 F 覆盖了 X 的元素，即 $X = \bigcup_{S \in F} S$。集合覆盖问题就是，找出含 X 的子集个数最少的子集 $C \subseteq F$，使得 C 覆盖了 X 中的所有元素。其判定问题(SetCover)可以描述为，给定一个有限集合 X，F 为 X 的子集的集合，以及一个正整数 k，问能否找到大小为 k 的集合 C ⊆ F，使得 C 覆盖了 X 中的所有元素。

子集合(Subset Sum)问题，其最优化问题可以描述为：任给一个正整数的集合 $S = \{x_1, x_2, \cdots, x_n\}$ 以及一个正整数 t。子集和问题是要找到 S 的一个子集 S_1，使得其和不超过 t，又尽可能地接近 t。该问题的判定问题(SubsetSum)可以描述为：是否存在 S 的一个子集 S_1，使得 $\sum_{x \in S_1} x = t$。

0/1 背包问题，其最优化问题可以描述为：给定一个物品集合 $s = \{1,2,\cdots,n\}$，物品 i 具有重量 w_i 和价值 v_i，背包能承受的最大重量不超过 W。问题就是找到一个物品子集 $s' \subseteq s$，在不超过背包载重量的条件下，使得 $\sum_{i \in s'} v_i$ 最大。其相应的判定问题(Knapsack)可以描述为：给定一个物品集合 $s = \{1,2,\cdots,n\}$ 以及一个整数 k，在不超过背包载重量的条件下，能否找到一个子集 $s' \subseteq s$，使得 $\sum_{i \in s'} v_i \geqslant k$。

旅行商问题(Traveling Salesman Problem,TSP),其最优化问题可描述为:给定一个完全无向带权图 $G=(V,E)$,其每一边 $(u,v)\in E$ 有一非负整数权值(或代价) $w(u,v)$。目的是要找出 G 的一条经过每个顶点一次且仅经过一次的回路,且使回路的总权值最小。相应的判定问题(TSP) 可以描述为:给定一个完全带权图 $G=(V,E)$ 和一个整数 k,问是否存在一个总权值至多为 k 的汉密尔顿回路。

划分问题(Partition),可以描述为:给定一个物品的集合 $S=\{s_1,\cdots,s_i,\cdots,s_n\}$,其中 s_i 为正整数,问题是 S 能否划分为大小相等的两部分,即是否有一个划分 $S=\{A,B\}$,使得 $\sum_{s_i\in A}s_i=\sum_{s_i\in B}s_i$。

装箱(Bin Packing)问题,其最优化问题可以描述为:给定 n 个物品的集合 $S=\{s_1,\cdots,s_i,\cdots,s_n\}$,要将这些物品装入载重量为 W 的箱子中,在不超过箱子载重量的条件下,问如何装载,使得所用的箱子数最小? 其判定问题(BinPacking)可以描述为:给定 n 个物品的集合 $S=\{s_1,\cdots,s_i,\cdots,s_n\}$ 以及 k 个箱子,每个箱子的载重量为 W,问能否将 n 个物品装入 k 个箱子中。

零件切割(Strip Packing)问题,其最优化问题可以描述为:给定一块宽度为 W 的矩形板,矩形板的高度不受限制。现需要从板上分别切割出 n 个高度为 h_i,宽度为 w_i 的矩形零件,其中 W, h_i, w_i 均为正整数。切割的规则是零件的高度方向与矩形板的高度方向保持一致,在保证切割的零件互不重叠的条件下,问如何切割使得所使用的矩形板的高度 h 最小。其相应的判定问题(StripPacking)可以描述为:给定一块宽度为 W 的矩形板,n 个高度为 h_i,宽度为 w_i 的矩形零件,以及一个正整数 k,问高度为 k 的矩形板能否切割出所需求的零件。

调度问题包括各种类型,例如并行机器的调度问题、流水作业调度问题以及作业车间调度问题等。下面,我们只介绍并行机器的调度问题,其最优化问题可以描述为:给定 n 个任务的集合 $J_1,\cdots,J_n$ 和 m 台机器 $M_1,\cdots,M_m$。每个任务 J_i,其处理时间为 $t_i>0$,都必须由一台机器不间断地处理,每台机器在一个时间段里最多处理一个任务。并行机器调度问题就是如何将任务分配给机器,使得处理完所有任务的完成时间(makespan)最短。其判定问题(ParallelScheduling)可以描述为给定 n 个任务的集合 $J_1,\cdots,J_n$ 和 m 台机器 $M_1,\cdots,M_m$,以及一个整数 k,问是否存在一个调度,其 makespan 最多为 k。

上述介绍的判定问题,均是 NP 完全问题,我们可以利用约简算法对它们进行证明。详细的证明过程,我们留给有兴趣的读者去完成,这里我们只给出将一个已知的 NP 完全问题约简到相应待证明的问题,如图 11.12 所示。从图 11.12 可以知道,当我们利用构造法证明了 CircuitSAT 属于 NPC 后,通过将 CircuitSAT$\leqslant_p$SAT,可证明 SAT 属于 NPC,将 SAT$\leqslant_p$3-CNF-SAT,可证明 3-CNF-SAT 属于 NPC,将 3-CNF-SAT$\leqslant_p$Clique,可证明 Clique 属于 NPC,将 Clique$\leqslant_p$VertexCover,可证明 VertexCover 属于 NPC。类似地,可证明 SubsetSum、GraphColoring、Partition、BinPacking、StripPacking、ParallelScheduling、HamCycle 以及 TSP 属于 NPC,其中 Partition 和 Bin-

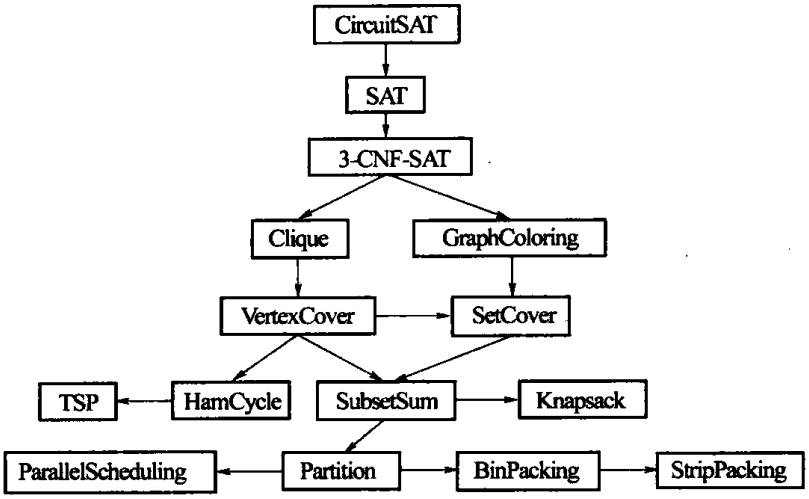


图 11.12

Packing 的证明可参考习题解答。其他类型的调度问题，例如流水车间调度问题以及作业车间调度问题等，都可以从 Partition 约简过来，从而得到证明。当然约简算法采用的技术各不相同，而且有些问题之间可以相互约简。例如除了 Clique $\leqslant_P$ VertexCover，利用 3-CNF-SAT $\leqslant_P$ VertexCover 同样可以证明 VertexCover 属于 NPC。利用问题之间的相互约简，目前已经证明了大量的 NP 完全问题。这里就不再详细叙述，有兴趣的读者可参考文献[6,14]。

11.6 小 结

本章内容主要取材于文献[1]，介绍了最优化问题和判定问题的概念，P，NP，NPC 的定义，Cook 定理的证明，以及如何证明一个问题是 NPC 问题。证明一个问题是 NPC 问题的方法很多，详细地可以参考文献[6, 14]。关于计算复杂性的最新结果、随机复杂性类以及概率可验证明，读者可以参考文献[15]。了解一个问题是否是 NPC 问题非常重要，它使得我们认识到，设计 NPC 问题的多项式时间有效求解算法，至少在目前是不可行的，从而转向其近似求解，以满足实际问题的需要。关于 NPC 问题近似算法的设计，请读者参考文献[4, 14]，这里就不再介绍了。

习题

11.1 0/1 背包问题的动态规划算法是否是一个多项式时间算法？

11.2 定义最优化问题 LongestPathLength 为：给定一个无向图以及两个顶点，问题是求这两个顶点间的最长简单路径所包含的边数。定义判定问题 LongestPath = $\{ <G,u,v,k> : G=(V,E)$ 是一个无向图，$u,v \in V, k \geqslant 0$ 是一个整数，G 中从 u 到 v 存在一条长度至少为 k 的简单路径$\}$。证明最优化问题 Longest-

PathLength 可以在多项式时间内解决，当且仅当 LongestPath 能够在多项式时间内解决。

11.3 假设 TSP 问题的求解算法是多项式时间算法，试设计一个算法，求出旅行商问题最短回路的权值。

11.4 给定一个无向图，最长简单回路问题是寻找图中最长的简单回路（其中没有重复出现的顶点）。为该问题分别给出最优化问题和相应判定问题的描述，并给出与该判定问题对应的语言。

11.5 证明对于一个多项式时间的算法，当它调用一个多项式时间的子程序的次数至多为常数次时，则该算法仍然是多项式时间算法，但是当调用子程序的次数为多项式次时，该算法就可能变成一个指数级时间算法。

11.6 考虑语言 GraphIsOmorphism = $\{ <G_1, G_2> : G_1$ 和 G_2 是同构图$\}$，描述一个可以在多项式时间内验证该语言的算法以证明 GraphIsOmorphism $\in$ NP。

11.7 证明如果 HamCycle $\in$ P，则按顺序列出一个汉密尔顿回路中各个顶点的问题是多项式时间可解的。

11.8 图 G 中的汉密尔顿路径是一条简单路径，它经过图中每个顶点仅一次。假设语言 HamPath = $\{ <G, u, v> :$ 图 G 中存在一条从 u 到 v 的汉密尔顿路径$\}$，证明 HamPath $\in$ NP。

11.9 试解释说明停机问题不是 NPC 问题，是一个 NP 难问题。

11.10 证明 $\leqslant_P$ 关系是语言上的一种传递关系。即证明：如果有 $L_1 \leqslant_P L_2$，且 $L_2 \leqslant_P L_3$，则有 $L_1 \leqslant_P L_3$。

11.11 证明可满足性赋值可以当作证书来使用，从而得到引理 11.2 的另一种证明。哪一个证书可以使证明更容易些？

11.12 Jagger 教授提出，在定理 11.5 的证明中，可以通过仅利用真值表技术，无需其他步骤，就能证明 SAT$\leqslant_P$3-CNF-SAT。具体策略为：取布尔公式 ϕ，形成有关变元的真值表，根据该真值表导出一个 3-DNF 形式的、等价于 $\neg\phi$ 的公式，再对其取反，并运用 DeMorgan 根定律，从而可以得到一个等价于 ϕ 的 3-CNF 公式。证明该策略不能产生多项式时间约简。

11.13 证明确定析取范式形式的布尔公式的可满足性问题是多项式时间可解的。

11.14 假设有一个判定公式可满足性的多项式时间算法。试说明如何利用该算法在多项式时间内找出一组可满足赋值。

11.15 假设 2-CNF-SAT 为 CNF 形式的、每个子句中恰有 2 个文字的可满足公式的集合。证明 2-CNF-SAT $\in$ P。要求所给出算法的效率应尽可能地高。（提示：注意 $x \vee y$ 等价于 $\neg x \rightarrow y$。将 2-CNF-SAT 约简为一个在有向图上能有效可解的问题。）

11.16 已知一个 $m \times n$ 整数矩阵 A 和一个 m 维整数向量 b，0/1 整数规划问题是：是否存在其元素属于集合 $\{0,1\}$ 的一个 n 维整数向量使得 $ax \leqslant b$。证明

0/1整数规划问题是 NP 完全的。（提示：约简到 3-CNF-SAT。）

11.17 证明 Partition $\in$ NPC。

11.18 证明 BinPacking $\in$ NPC。

11.19 证明 ParallelScheduling $\in$ NPC。

11.20 对于子集和问题，证明如果目标值 t 表示成一元形式，那么子集和问题就是多项式时间可求解的问题。

11.21 证明习题 11.4 给出的最长简单回路问题是 NP 完全的。

第 12 章　回　溯

在许多实际问题中，我们都要面临在很多选择中做一些决策，但是没有足够的信息帮助我们做出好的决策。同时，在做出一个决策后，接下来又要在许多新的选择中做一个决策。经过一系列的决策后，才有可能得到问题的解。对于这样的问题，当无法利用前面介绍过的算法进行求解时，一个好的解决方法就是利用回溯算法(Backtracking Algorithm)。

12.1　算法思想

在详细介绍回溯法之前，我们先介绍树的概念。我们知道，树由一系列的节点构成，在本书中，我们对节点在树中的角色，给它以不同的名称，如图 12.1 所示。椭圆形表示根节点，浅灰色的圆圈表示内部节点，深灰色的圆圈表示叶节点。有了树的概念后，我们就可以方便地用树来描述回溯法。树的每个节点，有若干个分支，可以看作面临许多选择，从一个节点沿着某分支进入下一个节点，表示在做了一个决策后，到达一个新节点，即将又面临许多选择。类似上述过程，一直到达叶节点。因此回溯法可以看作是从根节点出发，经过一系列的节点，到达一个个的叶节点。从根节点到达某个叶节点的路径可以看作问题的一个解，而叶子可以看作搜索的目标。回溯法就是搜索出所有的叶节点，然后选择一个满足要求的叶节点。

回溯是一种简单而非常强大的求解问题的方法，它系统地在所有可能的选择中搜索问题的一个最优解。如果把做的决策序列用一个向量(x_1，x_2，…，x_i，…，x_n)表示，回溯法就是以深度优先的方式逐步确定 x_i 的值，直到找到问题的解。回溯法尝试不同的决策序列，直到找到一个决策序列满足问题的目标。回溯通过枚举所有的决策序列保证解的正确性。当然，决策序列的多少，即解的多少极大地影响回溯法的效率。

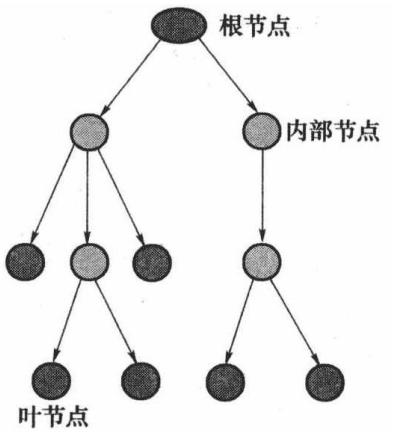


图 12.1

为了获得问题的解(x_1，x_2，…，x_i，…，x_n)，我们从 x_1 出发，逐步扩展部分解(x_1，x_2，…，x_i)，通过尝试解元素 x_{i+1} 的一个值，然后测试扩展出的(x_1，x_2，…，x_i，x_{i+1})是否还是一个部分解，如果是，则继续往下搜索，如果不是，则尝试 x_{i+1} 的其他值，依次类推，直到找到

问题的一个解。一般地，用回溯法求解问题的步骤如下：

（1）定义问题的解空间，这个解空间必须至少包含问题的一个最优解。

令 S_i 表示解元素 x_i 的取值范围，则问题的解空间为 $S_1 \times S_2 \times \cdots \times S_n$，通常，这个解空间很大，因此搜索一个目标解的计算量常常是无法想象的。为了提高回溯法的效率，我们必须有效地组织解空间。

（2）组织解空间以便更容易地搜索问题的解，典型的解空间组织方式是一棵树或一个图。

（3）用深度优先的方式搜索问题的解，并用剪支函数来避免去搜索一个无解的子空间。

下面我们分析回溯算法中所涉及的一些关键概念。

先介绍解空间的概念，任何时候，问题的解包括做一系列的决策，如果我们把决策序列用图画出来，它就像一棵树。前面我们已经介绍了树能够很好地描述回溯法的求解过程，因此，把问题的解空间组织成一棵树，把树的叶子当作解空间中的一个解，可以很方便地进行回溯搜索。这棵树也常称为解空间树。

有时问题是要从一个集合的所有子集中搜索一个集合，作为问题的解。有时从一个集合的排列中搜索一个排列，作为问题的解。回溯可以很方便地遍历一个集合的所有子集或所有的排列。当问题是需要求 n 个元素的子集，以便达到某种优化目标时，我们可以把这个解空间组织成一棵子集树。一个有 n 个元素的集合有多少个子集？如果每个 S_i 的大小为 k，对每个 $x_i \in S_i$，则有 k^n 个子集。当 n 很大时，解空间是非常巨大的。子集树的结构如图 12.2 所示。图 12.2 中，解元素 x_i 的值对应第 $i-1$ 层节点的一个分支。

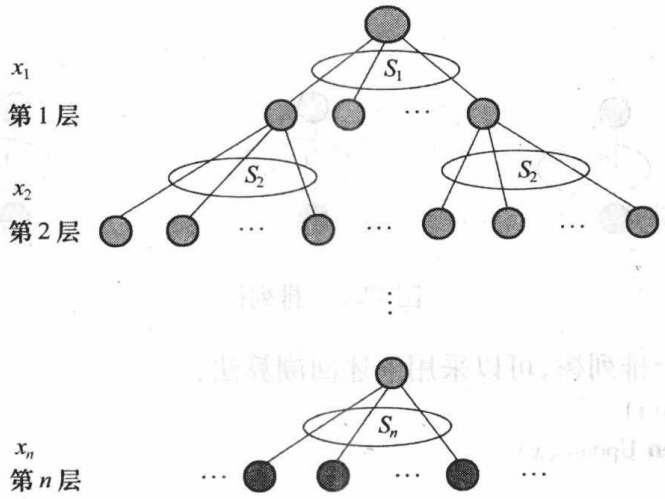


图 12.2　子集树

对于子集树，回溯法的伪代码如下：

```
Backtrack(i)
1   if i > n then Update(x)
```

```
2  else
3      for each a ∈ S_i do
4          x_i ← a
5          if C(i) and B(i) then
6              Backtrack(i+1)
```

其中，i 表示搜索的层数，即假定已经搜索到部分解$(x_1,x_2,\cdots,x_{i-1})$，通过尝试解元素 x_i 的一个值，然后测试扩展出的$(x_1,x_2,\cdots,x_i)$是否还是一个部分解，这由约束函数 $C(i)$和限界函数 $B(i)$决定，这两个函数也常统称为剪支函数，后面将根据具体例子说明如何设计约束函数和限界函数。第 1 行判断是否到了叶节点，如果是则 Update(x)，即更新有关解 x 的信息。第 3 行，对 x_i 取值范围 S_i 中的每个值 a，执行第 4 行后，在第 5 行判断是否满足约束条件和限界条件，如果满足，则回溯下一层。

当问题是需要求 n 个元素的一个排列，以便达到某种优化目标时，我们可以把这个解空间组织成一棵排列树。n 个元素的集合有多少个排列？建立一个含 n 个元素的数组 A，x_i 的值是数组 A 中下标从 1 到 n 的一个元素，并且之前从未出现过，对应排列中的第 i 个元素，那么共有 $n!$ 个排列，排列树的结构如图 12.3 所示。图 12.3 排列树中，集合 S_i 中的元素是数组 A 的下标。

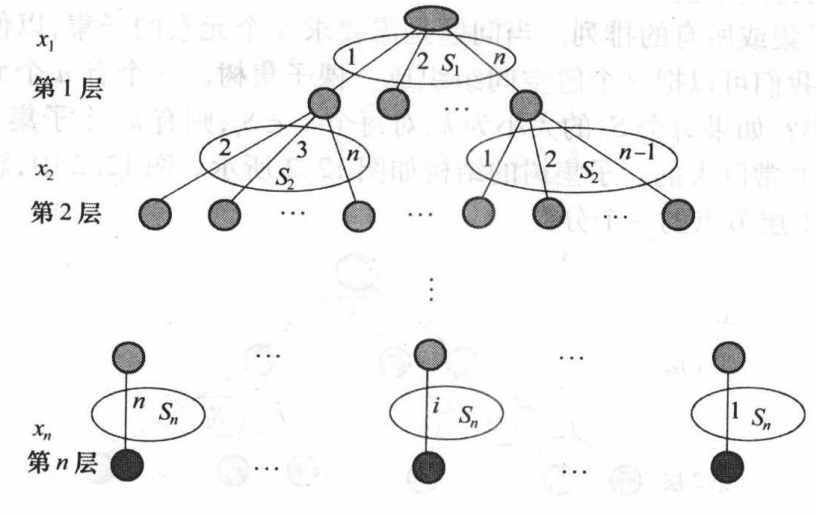


图 12.3　排列树

事实上，对于排列树，可以采用下述回溯算法：

```
BacktrackPerm(i)
1  if i > n then Update(x)
2  else
3      for j ← i to n do
4          x_i ↔ x_j
5          if C(i) and B(i) then
6              BacktrackPerm(i+1)
7          x_i ↔ x_j
```

BacktrackPerm(i)算法中，搜索是从一个给定的排列($x_1, x_2, \cdots, x_n$)开始的。假定已经搜索到部分解($x_1, x_2, \cdots, x_{i-1}$)，通过尝试解元素 x_i 的一个值，其中 x_i 属于 $\{x_i, x_{i+1}, \cdots, x_n\}$。为了保证排列中每个元素不同，可以通过交换 $x_i \leftrightarrow x_j$ 来实现。然后在第5行利用 $C(i)$ 和 $B(i)$ 来测试扩展出的($x_1, x_2, \cdots, x_i$)是否还是一个可行的部分解。

从 Backtrack(i)算法可以看出，回溯法事实上是一个递归算法，因此，我们可以利用更有效率的迭代算法来实现，即迭代回溯算法，例如，对于子集树的递归算法，其相应的迭代回溯算法如下：

```
IterateBacktrack( )
1  i ← 1
2  while i > 0 do
3        while S_i is not exhausted do
4              x_i ← next element in S_i
5              if i = n then Update(x)
6              else if C(i) and B(i) then i ← i + 1
7        i ← i - 1
```

IterateBacktrack( )算法中，搜索从第1层开始，第3行表示对 x_i 的取值范围 S_i 中的每个元素进行枚举，第4行中，令 x_i 为 S_i 中的下一个元素，第5行判断如果搜索到叶节点，则 Update(x)，否则如果满足约束条件和限界条件，则继续往下搜索。第7行进行回溯。

从上面的回溯算法可以看出，在每一层节点，能否往下搜索，取决于约束函数 $C(i)$ 和限界函数 $B(i)$。利用约束函数和限界函数来剪除无用的分支，集中搜索那些最有希望获得解的分支，是提高回溯算法搜索效率的关键。因此为了改进搜索的性能，利用回溯至少需要考虑以下三点：

(1) 如何找到约束函数。

(2) 对于求最大值(最小值)问题，怎样计算上界(下界)，从而利用这个信息来构造限界函数；

(3) 如何利用约束函数和限界函数来剪除无用的分支。

对于第(1)点，约束函数一般可以从问题的描述中找到，如果找不到，经过仔细分析，也是比较容易构造出约束函数的。对于第(2)点，设计限界函数是需要技巧的。一般来说，对于求解最大值问题，我们计算当前节点 i 的上界值 $B(i)$，这个 $B(i)$ 通常与问题的目标函数值有关，它包括从根节点到当前节点 i 的部分解的目标函数值，以及当前节点 i 到叶节点这部分解目标函数值的上界。如果当前最大目标函数值 *bestc* 不小于 $B(i)$，那么剪掉该分支，否则继续。因为从根节点 R 到叶节点 L 的目标函数值一定不大于 $B(i)$，所以如果 $B(i) \leqslant bestc$，那么它表明正在搜索的节点 i 是无希望的，所以要剪掉当前节点 i，如图 12.4 所示。当然估计当前节点到叶节点这部分解目标函数值的上界是比较困难的，也是需

要技巧的。一种计算 $B(i)$ 的方法是只考虑根节点到当前节点部分解目标函数值，这种方法简单，但是效率不高。后面的例子，我们会介绍如何根据具体的问题来估计 $B(i)$。

类似地，对于求解最小值问题，我们计算当前节点 i 的下界值 $B(i)$。如果当前最大目标值 *bestc* 不大于 $B(i)$，那么剪掉该分支，否则继续。因为从根节点 R 到叶节点 L 的目标值必须不小于 $B(i)$，所以如果 $B(i) \geq$ *bestc*，那么它表明正在搜索的节点 i 是无希望的，即死节点，所以要剪掉当前节点 i。如图 12.4 所示。

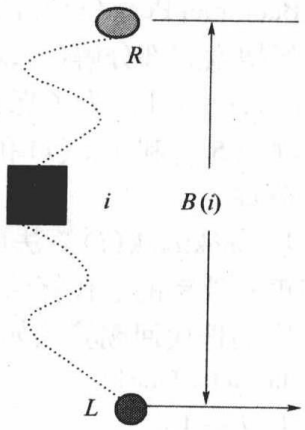


图 12.4

下面介绍回溯算法的应用。

12.2 装载问题

给定 n 个集装箱，集装箱 i 的重量为 w_i，以及一艘船，船的载重量为 W。集装箱装载问题是在不超过船的载重量的前提下装载尽可能重的集装箱。

由于集装箱问题是从 n 个集装箱里选择一部分集装箱，而对每个集装箱，可装上船，也可不装。假设解可以由向量 $(x_1, x_2, \cdots, x_n)$ 表示，$x_i \in \{0,1\}$，$x_i = 1$ 表示集装箱 i 被装上船，$x_i = 0$ 表示集装箱 i 不装上船。因此集装箱装载问题可以描述如下

$$\max \sum_{i=1}^{n} w_i x_i$$

s. t.

$$\sum_{i=1}^{n} w_i x_i \leqslant W$$

由于解元素 x_i 表示集装箱 i 可装可不装，即 $|S_i| = 2$，因此解空间的大小为 2^n，可以把解空间组织成一棵有 2^n 个叶子的子集树，如图 12.5 所示。每个内部节点对应两个分支，1 分支和 0 分支。如果是 1 分支，表示相应集装箱装上船。否则不装上船。

令 $cw(i)$ 表示从根节点到第 i 层节点为止装入船的重量，即部分解 $(x_1, x_2, \cdots, x_i)$ 的重量，$cw(i)$ 可计算如下

$$cw(i) = \sum_{j=1}^{i} w_j x_j$$

假设已经知道部分解 $(x_1, x_2, \cdots, x_{i-1})$，由于 $S_i = \{0,1\}$，则 x_i 试探取值 1，即从

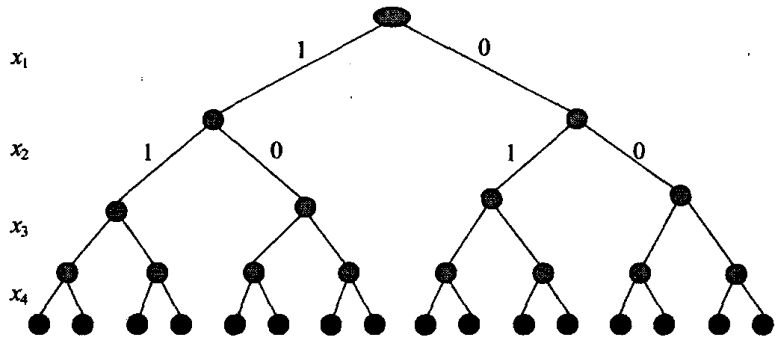


图 12.5

第 $i-1$ 层节点往 1 分支走的约束函数为

$$C(i) = cw(i-1) + w_i$$

若 $C(i) > W$,则停止搜索 1 分支,否则,继续搜索。此外,当 x_i 试探取值 0,显然不会违反约束,因此可以继续往下搜索。详细的递归回溯算法如下:

```
BacktrackLoading(i)
1  if i > n then
2      if cw > bestw then
3          bestw←cw
4  else
5      if C(i) ≤ W then
6          cw←cw + w[i]
7          BacktrackLoading(i + 1)
8          cw←cw - w[i]
9      BacktrackLoading(i + 1)
```

BacktrackLoading(i)算法中,第 1 行判断如果到了叶节点,则表明已经找到一个解,如果该解的装载重量比当前最好的 *bestw* 更大,则更新当前最好的重量。第 4 行表示,还没有搜到叶节点,第 5 行试探走 1 分支,计算 $C(i) = cw + w[i]$,判断约束函数值 $C(i)$是否超过了船的载重量 W,如果没有,则继续往下搜索,并更新当前装载的重量 cw,否则,进行剪支。第 9 行对 0 分支,无条件进行回溯。

由于装载问题的子集树中叶节点的数目为 2^n,因此 BacktrackLoading(i)算法的时间复杂度为 $O(2^n)$。

例 12.1 给定一个装载实例:$n=4, w=[8,6,2,3], W=12$,我们可以图示回溯的过程,如图 12.6 所示。

初始化 $cw=0$,首先调用 BacktrackLoading(1),$C(A)=8 \leqslant 12$,因此执行第 6 行,$cw=8$,调用 BacktrackLoading(2),$C(B)=cw+6>12$,不满足约束条件,因此停止搜索第 1 层节点的 1 分支及其下面层,即 B 节点被剪支。然后回溯至 A 节点,搜索 0 分支,调用 BacktrackLoading(2),$C(C)=cw \leqslant 12, cw=8$,调用 BacktrackLoading(3),试探搜索至 D 节点,$C(D)=cw+2 \leqslant 12, cw=10$,调用 BacktrackLoading(4),

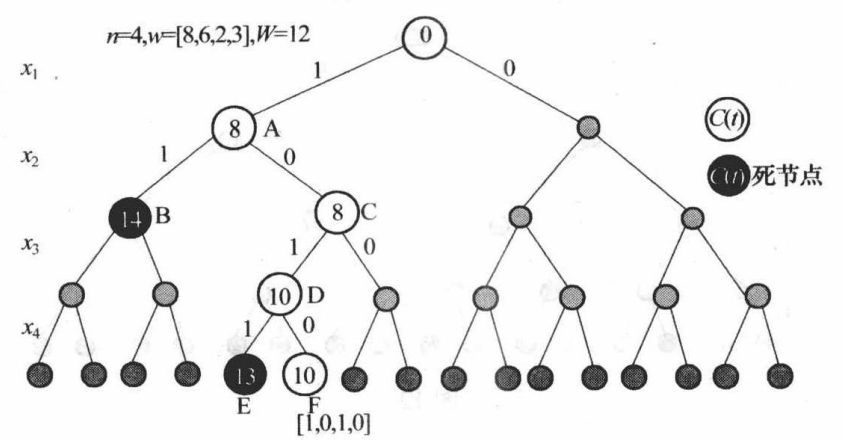


图 12.6

$C(\mathrm{E}) = cw + 3 > 12$，不满足约束条件，因此停止搜索第 3 层节点的 1 分支及其下面的层。然后回溯至 D 节点，搜索 0 分支，调用 BacktrackLoading(4)，这样可以得到一个解[1，0，1，0]。

回溯算法 BacktrackLoading(i)没有利用限界函数，下面我们给出一个改进的回溯算法。其基本思想是在 BacktrackLoading(i)的基础上利用限界函数

$$B(i) = C(i) + r(i)$$

其中，$r(i)$表示剩余集装箱的总重量，它可计算如下

$$r(i) = \sum_{j=i+1}^{n} w_j$$

若 $B(i) \leqslant bestw$，则停止搜索第 i 层及其下面的层，否则，继续搜索。其中，$bestw$ 表示到目前为止所得到的最大重量。下面给出改进回溯法的伪代码：

```
ImprovedBacktrackLoading(i)
1   if i > n then
2       if cw > bestw then bestw←cw
3   else
4       r←r - w[i]
5       if C(i) ≤ W then
6                   cw←cw + w[i]
7                   ImprovedBacktrackLoading(i + 1)
8                   cw←cw - w[i]
9       if B(i) > bestw then
10                  ImprovedBacktrackLoading(i + 1)
11      r←r + w[i]
```

ImprovedBacktrackLoading(i)算法与 BacktrackLoading(i)算法不同的是在第 9 行，对于 0 分支，需要判断是否满足限界函数，以及增加估计下界的过程，因此整个时间复杂度与 BacktrackLoading(i)算法相同。

对例 12.1，我们利用改进的回溯法进行求解，图解过程如图 12.7 所示。搜索到 A 节点，此时约束函数值为 8，限界函数值为 19，此时，试探装集装箱 2，往 1 分支搜索的时候，违背约束条件，节点 B 被剪支。回溯回去，往 0 分支，可以搜索到节点 C，此时往 1 分支走，没有违反约束条件，继续递归搜索到 D，此时往 1 分支走，会违背约束条件，因此节点 E 被剪支。回溯至 D 节点往 0 分支走，搜索到 F，得到一个解[1，0，1，0]，*bestw* = 10。回溯到 C 节点，往 0 分支走到 G，可以继续搜索到 H 节点，获得解[1，0，0，1]，*bestw* = 11。由于找到了更好的解，因此解被更新。回溯至 G，其 0 分支节点 I，$B(I) < bestw$，因此 I 被剪掉。回溯到根节点，往 0 分支走，由于 J 节点的限界函数值为 11，不会比当前最好的载重量 *bestw* = 11 大，因此节点 J 被剪支。通过利用限界函数，极大地提高了搜索的效率。

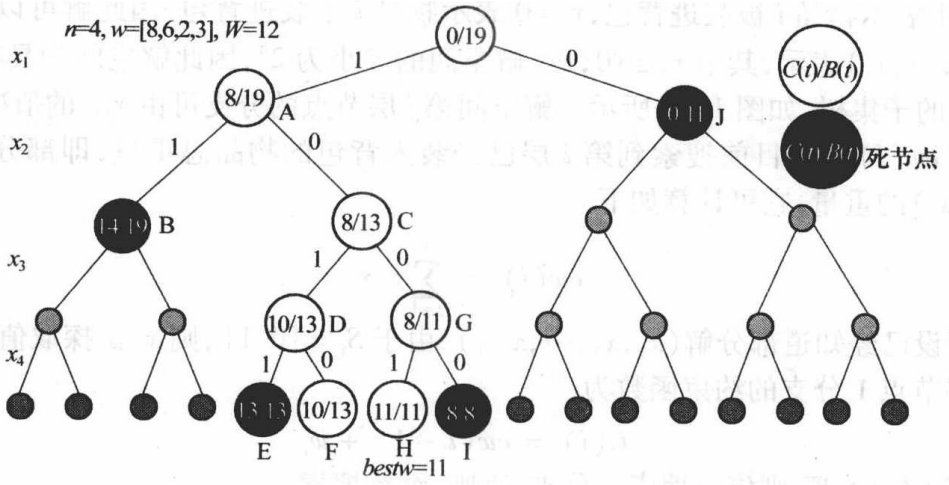


图 12.7

上述两个回溯算法，只给出了最优装载重量的计算，为了构造最优解，我们必须在伪代码中添加保存解的过程，对上面的回溯算法稍加修改，就可以得到如下算法：

```
ImprovedBacktrackLoading1(i)
1  if i > n then
2        if cw > bestw then
3              bestw←cw
4              for j←1 to n do
5                    bestx[j]←x[j]
6  else
7        r←r - w[i]
8        if C(i) ≤ W then   x[i] ←1
9                           cw←cw + w[i]
10                          ImprovedBacktrackLoading1(i + 1)
11                          cw←cw - w[i]
```

```
12      if B(i) > bestw then x[i] ←0
13                          ImprovedBacktrackLoading1(i+1)
14      r←r+w[i]
```

ImprovedBacktrackLoading1(i)算法在第 3 行需要考虑保存最好的解，因此需要时间 O(n)，因此 ImprovedBacktrackLoading1(i)算法的时间复杂度为 O($n2^n$)。

12.3 0/1 背包问题

第 6 章已经介绍了 0/1 背包问题，并且给出了一个有效的动态规划算法。现在，我们给出 0/1 背包问题的回溯算法。由于每个物品可以装进背包，也可不装，令 $x_i=1$ 表示物品 i 被装进背包，$x_i=0$ 表示物品 i 不装进背包，因此解可以由向量 $(x_1,x_2,\cdots,x_n)$ 表示，其中 $x_i\in\{0,1\}$，解空间的大小为 2^n，因此解空间为具有 2^n 个叶节点的子集树，如图 12.5 所示。解空间第 j 层节点的分支可由 x_{j+1} 的值决定。

令 $cw(i)$ 表示目前搜索到第 i 层已经装入背包的物品总重量，即部分解 $(x_1,x_2,\cdots,x_i)$ 的重量，它可计算如下

$$cw(i)=\sum_{j=1}^{i}w_jx_j$$

假设已经知道部分解 $(x_1,x_2,\cdots,x_{i-1})$，由于 $S_i=\{0,1\}$，则 x_i 试探取值 1，则第 $i-1$ 层节点 1 分支的约束函数为

$$C(i)=cw(i-1)+w_i$$

因此若 $C(i)>W$，则停止搜索 1 分支，否则，继续搜索。

如果仅用约束函数，我们已经能够写出回溯算法，但是效率不高，为了提高搜索的效率，我们引进限界函数

$$B(i)=cv(i)+r(i)$$

其中，$cv(i)$ 表示目前到第 i 层节点已经装入背包的物品价值

$$cv(i)=\sum_{j=1}^{i}v_jx_j$$

$r(i)$ 表示剩余物品的总价值

$$r(i)=\sum_{j=i+1}^{n}v_j$$

若 $B(i)\leqslant bestv$，则停止搜索第 i 层的节点及其下面的层，否则，继续搜索。其中，$bestv$ 表示目前所得到的最好价值。

当 $B(i)\leqslant bestv$ 时，则停止搜索，因此 $r(i)$ 越小，$B(i)$ 越小，剪掉的分支就越多。我们是否可以剪掉更多的分支呢？答案是可以，我们只要构造更小的 $r(i)$ 即可。具体地说，我们可以通过构造更好的限界函数来实现。

首先，先将物品以单位重量价值递减的顺序进行排列

$$v_1/w_1 \geqslant v_2/w_2 \geqslant \cdots \geqslant v_n/w_n$$

考虑第 i 层，背包的剩余容量为 $W-cw(i)$，用贪心算法把剩余的物品放进背包，直到装不进背包的物品 k 为止，即物品 k 放不进去。因此

$$r(i) = \sum_{j=i+1}^{k-1} v_j + (W - cw(i) - \sum_{j=i+1}^{k-1} w_j)(v_k/w_k) \tag{12.1}$$

按照第7章贪心算法的结论，上述剩余物品的价值已经是最优的，因为对剩余物品的装载不存在比上述贪心装载方案还优的方案。因此，由式(12.1)计算的 $r(i)$ 已经最优，它的值比原始 $r(i)$ 值小，因此可以剪掉更多的分支。详细地计算 $r(i)$ 的伪代码如下：

```
r(i)
1  rw← W - cw
2  b ← cv
3  while i+1≤n and w[i+1]≤rw do
4     rw← rw - w[i+1]
5     b← b + v[i+1]
6     i←i+1
7  if i+1≤n then b← b + v[i+1]/w[i+1] × rw
8  return b
```

其中，cw，cv 分别表示搜索到第 i 层，部分解所具有的当前重量和价值，rw 为背包的剩余重量，下面，我们给出结合上述限界函数的回溯算法：

```
BacktrackKnapsack(i)
1   if i > n then
2       if cv > bestv then
3           bestv←cv
4           for j←1 to n do
5                bestx[j] ←x[j]
6   else
7       if C(i)≤W then x[i] ←1
8                      cw←cw + w[i]; cv ←cv + v[i]
9                      BacktrackKnapsack(i+1)
10                     cw←cw - w[i];cv←cv - v[i]
11      if B(i) > bestv then x[i] ←0
12                        BacktrackKnapsack(i+1)
```

BacktrackKnapsack(i)算法除了更新当前价值 cv 外，其伪代码与 ImprovedBacktrackLoading1(i)类似，时间复杂度的分析也类似，这里就不再说明。

例 12.2 给定一个背包实例：$n=4$，$W=7$，$v=[9,10,7,4]$，$w=[3,5,2,1]$。其单位重量价值比 $v/w=[3,2,3.5,4]$。按单位重量价值比排序可得一个新的实例：$n=4$，$W=7$，$v=[4,7,9,10]$，$w=[1,2,3,5]$，$v/w=[4,3.5,3,2]$。

从图 12.8 可知，沿着子集树的左分支一直搜索到第 3 层的节点 C，由于该节点 1 分支的约束函数值为 11，超过了背包的载重量，因此剪掉该分支。第 3 层节点的 0 分支节点 D 总是可行的，因此我们可以搜索到一个解[1,1,1,0]，其总价值为 20，即到目前为止找到的最大价值。然后回溯到第 2 层节点 B 的 0 分支，由于 $B(E)=19$ 小于当前找到的最大价值 20，因此，剪掉该节点 E。重复上述过程，节点 F 和 G 都被剪掉。由于剪掉了大量的分支，搜索的效率得到了明显的提高。

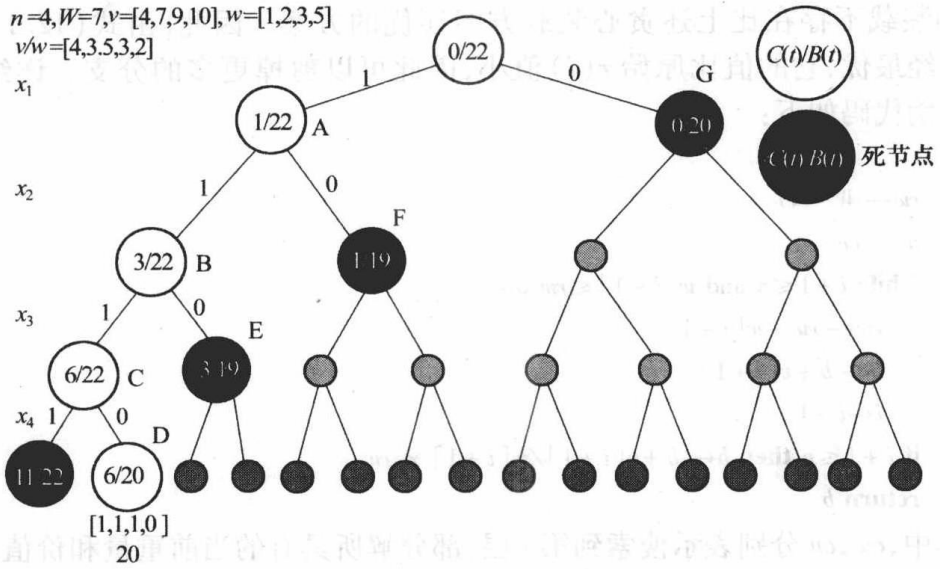


图 12.8

12.4 着色问题

地图着色是图论中最经典的问题之一，其描述是给定一幅地图（平面图），给地图上的国家着色，使得具有公共边界的任何一对国家有不同的颜色，问需要多少种颜色，就可以达到着色的目的。

上个世纪最著名的一个问题是猜想所有平面图仅用四种颜色就能对平面图着色，使得相邻的区域有不同的颜色。对大多数地图，找到合法的着色方案是很容易的。对某些地图，要找到合法的着色方案却是相对困难的。四色猜想吸引了众多科学家的兴趣和热情，然而一直悬而未决，直到 1976 年，美国计算机科学家 Appel 和 Haken 借助计算机，通过枚举分析，证明了此猜想是成立的，即所有平面图仅用四种颜色就可着色。

将国家看作一个顶点，如果两个国家相邻，则在两个国家之间引入一条边，这样地图着色问题就可以转化为顶点着色问题，如图 12.9 所示。因而问题转化为需要多少种颜色来着色顶点使得相邻的顶点有不同的颜色。例如图 12.9(a)和(b)只需要四种颜色就可以满足问题的要求。

下面我们考虑更一般的 m 着色问题。

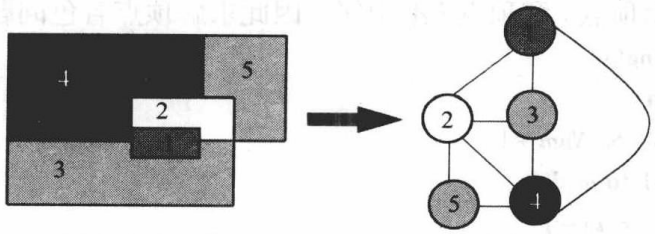


图 12.9

给定一个无向图 $G=(V,E)$，需要对图 G 中的每个顶点用 m 种颜色中的一种进行着色，使得相邻的两个顶点有不同的颜色。

从问题的描述可知，相邻的顶点必须用不同的颜色，我们没有足够的信息来选择颜色，每个选择会得到新的选择子集。一个或多个选择序列可以得到一个解，也有可能得不到，因此 m 着色问题可以用回溯算法来求解。

对 m 种颜色编号为 $1,2,\cdots,m$，由于每个顶点可从 m 种颜色中选择一种着色，如果图 $G=(V,E)$ 的顶点数为 n，则解空间的大小为 m^n 种，解空间是非常巨大的。与前面几个问题的解空间树有所不同的是，着色问题的解空间树为一棵多叉树。图 12.10 给出 $3n=4,m=3$ 时的解空间树。由于着色问题的解空间太大，因此，寻找合适的剪支函数无疑会提高搜索的效率。然而，着色问题没有明显的约束函数。而且着色问题也不是最优化问题，它只要求找到一种可行的着色方案就可以了，限界函数在这里也不起作用。唯一可用的约束函数是相邻的两个顶点需要着不同的颜色。

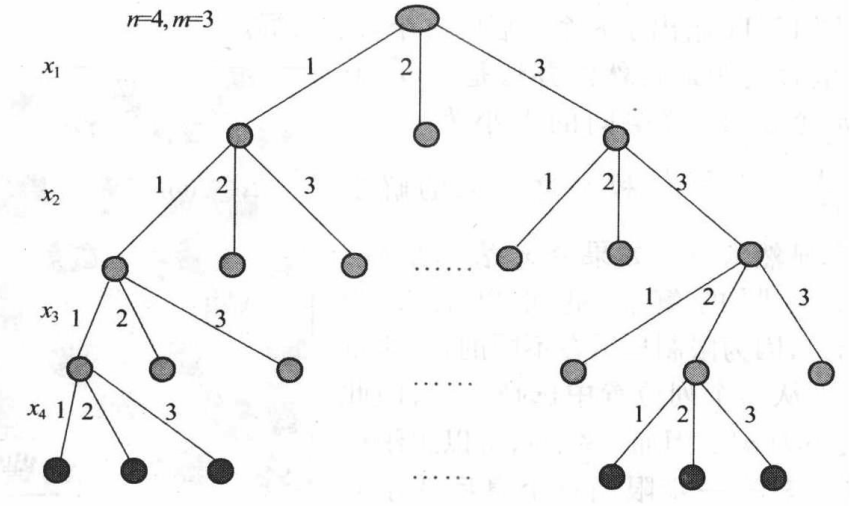


图 12.10

令顶点 i 的颜色为 x_i，因此解向量为 $(x_1,x_2,\cdots,x_n)$，$x_i\in\{1,2,\cdots,m\}$，同时，我们令数组 $w[i,j]$ 表示顶点 i 和顶点 j 是否相邻，如果 $w[i,j]=1$，表示顶点 i 和顶点 j 相

邻，$w[i,j]=0$ 表示顶点 i 和顶点 j 不相邻。因此求解顶点着色问题的回溯算法如下：

```
BacktrackColoring(i)
1  if i > n then
2     SolNum ← SolNum + 1
3  else for j←1 to m do
4              x[i]←j
5              if Ok(i) = True then BacktrackColoring(i + 1)
```

其中，*SolNum* 表示可行着色方案的数目，约束函数 Ok(i) 的伪代码如下：

```
Ok(i)
1     for j←1 to i - 1 do
2         if w[i, j] = 1 and x[j] = x[i] then return False
3     return True
```

BacktrackColoring(i) 算法中，对每个内部节点，其子节点的一种着色是否可行，需要判断子节点的着色与相邻的 n 个顶点的着色是否相同，因此共需要 $O(mn)$，而整个解空间树共有 $\sum_{i=0}^{n-1} m^i$ 个内部节点，因此，BacktrackColoring(i) 算法的时间复杂度为 $O(m^n n)$。

12.5 n 皇后问题

给定一个 $n \times n$ 的棋盘，n 皇后问题就是在棋盘上放置 n 个皇后，使得 n 个皇后两两之间互不攻击。任何两个处于同一列、同一行、同一条对角线上的皇后都会互相攻击。图 12.11 给出了 8 个皇后的一个合法布局。

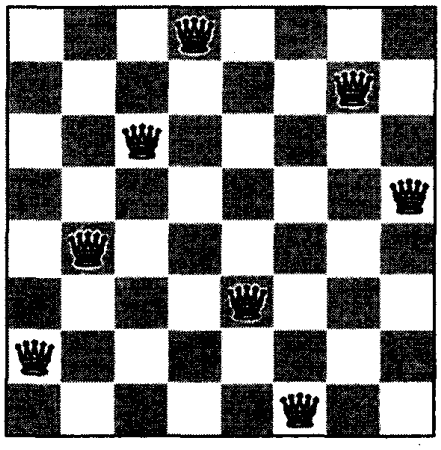


图 12.11

求解 n 皇后问题最自然的办法是从 n^2 个位置中选 n 个位置，解空间的大小为 $C_{n^2}^n = \frac{n^2(n^2-1)\cdots(n^2-n+1)}{n!}$，枚举这么大的解空间，计算效率显然太低。如果令 x_i 表示皇后 i 的列位置，问题的解向量可以表示为 $(x_1, x_2, \cdots, x_n)$，因为限制皇后在不同的行，对每个皇后 i，x_i 可从 n 个列位置中选择一个，因此解空间的大小为 n^n。因而，解空间可以组织成一棵子集树。若进一步限制每个皇后在不同列，解空间大小可以从 n^n 减少到 $n!$，因而，解空间可以组织成一棵排列树。由此可见，不同的表示，其解空间的大小也不一样。下面考虑解空间用子集树来表示的情形。如图 12.12 所示，图中黑色节点是死节点，是考虑了约束函数得到的子集树。下面，我们进一步形式化约束函数。

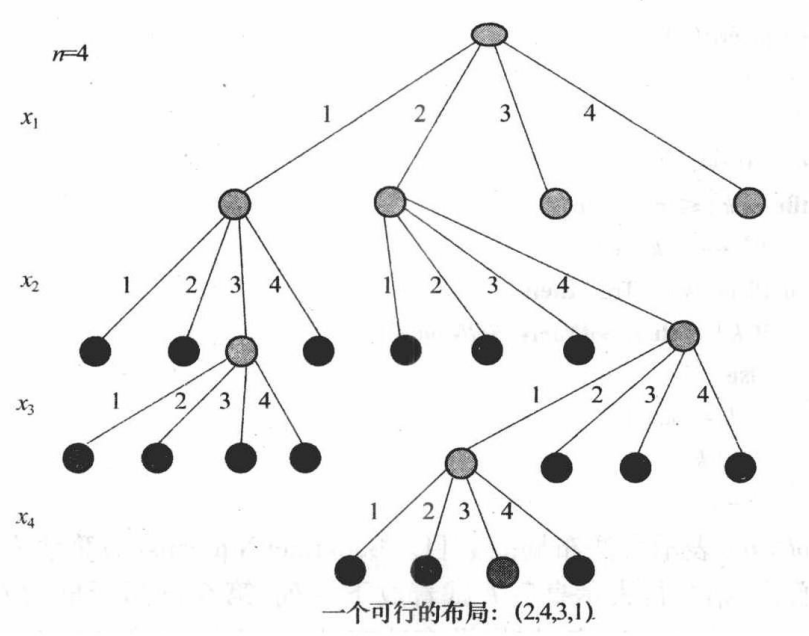


图 12.12

由于任意两个皇后不允许在同一行、同一列、同一对角线上。由于已经排除在同一行的情形，因此只考虑在同一列或在同一对角线上的条件。两个皇后在同一列的条件，比较容易判断。假设两个皇后被放在位置 x_k 和 x_j，若它们在同一列当且仅当 $x_k = x_j$。从解析几何可知，两个皇后在同一条对角线上当且仅当它们所在直线的斜率为 1 或者 -1，即

$$\frac{k-j}{x_k - x_j} = 1 \text{ 或者} \frac{k-j}{x_k - x_j} = -1$$

将上述条件改写一下，可得

$$| x_k - x_j | = | k - j |$$

因此，问题的约束条件变为

$$x_k = x_j \text{ 或者} | x_k - x_j | = | k - j |$$

根据上述约束函数，用来判断两个皇后是否在同一列、同一对角线上的伪代码如下：

```
Place(k)
1  for j←1 to k-1 do
2      if |x_k - x_j| = |k-j| or x_k = x_j then
3          return False
4  return True
```

程序 Place(k) 的输入为 k，表示当前要放置的是皇后 k，皇后 k 能否放置在列 $x[k]$，就要判断放置在 $x[k]$ 的皇后 k 是否与前面已经放置的皇后产生冲突。

有了判断任意两个皇后产生冲突的方法后，我们就可以写出下列回溯算法：

```
BacktrackNqueens( )
1  x[1] ← 0
2  k ← 1
3  while k > 0 do
4     while x[k] ≤ n - 1 do
5        x[k] ← x[k] + 1
6        if Place(k) = True then
7           if k = n then SolNum←SolNum + 1
8           else
9              k ← k + 1
10             x[k]← 0
11    k ← k - 1
```

其中，*SolNum* 表示合法布局的数目。BacktrackNqueens( )算法中，第 1 行与第 2 行进行初始化，第 5 行表示皇后 k 试着放下一列，第 6 行判断皇后 k 是否与已经放置的前面 $k-1$ 个皇后冲突，如果没有冲突，则第 7 行测试皇后 k 是否已经到了叶节点，如果到叶节点，则返回一个合法解，否则放置下一个皇后。第 11 行执行回溯。

与前面介绍的递归回溯算法不同的是，BacktrackNqueens( )算法是一个迭代回溯算法，由于该算法避免了参数的反复传递以及中间值的保存，效率要比递归回溯高。因此，平时在设计回溯算法时，应该尽量采用迭代的方式来实现。

12.6 旅行商问题

旅行商问题(TSP)是组合优化领域中的著名难题之一，也是计算复杂性理论、图论等许多理论中的一个经典问题。旅行商问题最早在 20 世纪 20 年代由著名数学家兼经济学家 Karl Menger 提出。问题的一般描述为：旅行商从驻地出发，经过每个所要访问的城市一次且只经过一次，并最终返回驻地。问如何安排旅行的路线才能使旅行的总路程最短。旅行商问题在军事、通信、电路板的设计、大规模集成电路、基因排序等领域具有广泛的应用。其形式化的描述如下：

给定一个完全无向带权图 $G=(V,E)$，其每一边 $(u,v)\in E$ 有一非负整数权值(或代价) $w(u,v)$。目的是要找出 G 的一条经过每个顶点一次且仅经过一次的回路，即汉密尔顿回路 $v_1,v_2,\cdots,v_n$，使回路的总权值最小。即

$$\min\sum_{i=1}^{n-1} w(v_i,v_{i+1}) + w(v_n,v_1)$$

图 12.13 给出了一个旅行商问题实例。其中回路有(1,2,4,3,1)，(1,4,2,3,1)，(1,3,2,4,1)等，其中(1,3,2,4,1)的总权值为 25，为最优回路。

既然回路是包括所有顶点的环，我们可以选择任意一个顶点为起点(该顶点

也是终点)。假设将 n 个顶点编号为 $1,2,\cdots,n$,并选择顶点 1 为起点,则每个回路可以被描述成顶点序列 $1,x_2,\cdots,x_n,1$,顶点序列 $x_2,\cdots,x_n$ 为顶点 $2,3,\cdots,n$ 的一个排列,因此解空间的大小为 $(n-1)!$,而且解空间可以组织成一棵排列树,如图 12.14 所示。其中,第 1 层节点有 3 个分支,分别是选择顶点 2,3,4。

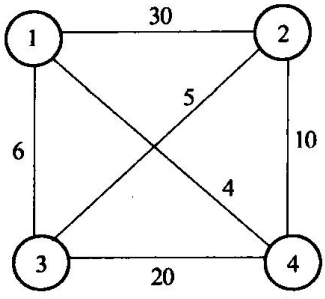


图 12.13

图 12.14 中,每条从根到叶节点的路径定义一条路线,解空间大小为 3!。令 $w[i,j]$ 表示从顶点 i 到顶点 j 的权值,若 $w[i,j]=\infty$,表示顶点 i 和顶点 j 之间没有边。因此,我们马上得到一个很自然的约束条件。如果当前顶点 j 与当前路径中的末端点 i 没有边相连,即 $w[i,j]=\infty$,则不必搜索顶点 j 所在的分支,否则继续。当然,这个约束函数不是很有效,假如是一个完全图,则该约束函数没有什么用。而解空间很大,因此我们必须构造有效的限界函数。

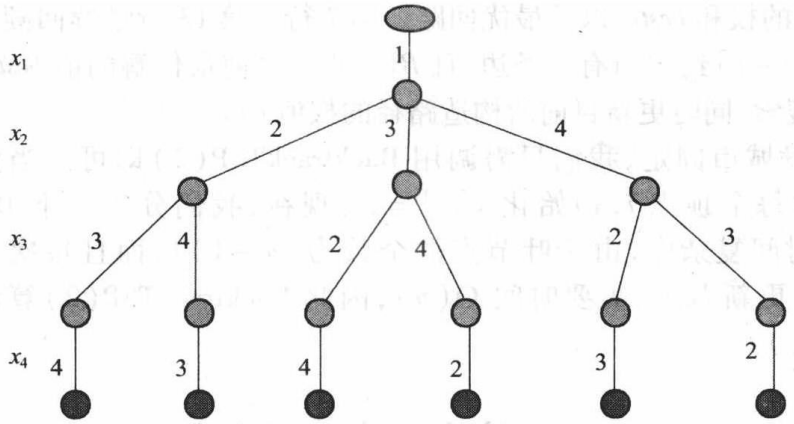


图 12.14

令到第 i 层为止所构造路径的权和为

$$cw(i) = \sum_{j=2}^{i} w[x[j-1],x[j]]$$

假设已经知道部分解 $(x_1,x_2,\cdots,x_{i-1})$,则从第 $i-1$ 层节点选择顶点 $x[i]$ 往下走的限界函数为

$$B(i) = cw(i-1) + w[x[i-1],x[i]]$$

若 $B(i) \geqslant bestw$,则停止搜索 $x[i]$ 分支及其下面的层,否则继续搜索。其中,*bestw* 表示目前为止找到的最佳回路的权和。有了限界函数,我们可以写出下列回溯算法:

```
BacktrackTSP(i)
1  if i = n then
2     if w[x[n-1], x[n]] ≠ ∞ and w[x[n],1] ≠ ∞ then
```

```
3        if cw + w[x[n-1], x[n]] + w[x[n],1] < bestw then
4          bestw←cw + w[x[n-1], x[n]] + w[x[n],1]
5          for j ← 1 to n do
6            bestx[j] ← x[j]
7    else for j ← i to n do
8      if w[x[i-1], x[j]] ≠ ∞ and cw + w[x[i-1], x[j]] < bestw then
9            x[i] ↔ x[j]
10           cw ←cw + w[x[i-1], x[i]]
11           BacktrackTSP(i+1)
12           cw ←cw - w[x[i-1], x[i]]
13           x[i] ↔ x[j]
```

BacktrackTSP(i)算法中，第 1 行测试如果 $i=n$ 时，表示已经搜索到叶节点。如果从 $x[n-1]$到 $x[n]$以及从 $x[n]$到起点 $x[1]$有一条边，则找到了一条回路。此时，第 3 行需要判断该回路是否是目前发现的最优回路。如果是，则第 4 行～第 6 行更新回路的权和 *bestw* 以及最优回路。第 7 行～第 13 行继续回溯搜索，在第 8 行，如果从 $x[i-1]$到 $x[j]$有一条边，且 $B(i)$小于当前最优解的值 *bestw*，则表示可以继续往下搜索，同时更新目前所构造路径的权值 *cw*。

由于出发城市固定，我们只需调用 BacktrackTSP(2)即可。当然在调用之前，还必须对每个顶点 i，初始化 $x[i]=i$。现在，我们分析一下 BacktrackTSP(2)算法的时间复杂度，由于叶节点的个数为$(n-1)!$，而且每次找到一个更好的回路时，更新 *bestx*，需要时间 $O(n)$，因此 BacktrackTSP(2)算法的时间复杂度为 $O(n!)$。

12.7 流水作业调度问题

给定 n 个作业 $J=\{j_1,j_2,\cdots,j_n\}$，每个作业 j_i 有两道工序，分别在两台机器上处理。一台机器一次只能处理一道工序，并且工序一旦开始就必须进行下去直到完成。此外，一个作业只有在机器 1 上的处理完成以后，才能由机器 2 处理，也就是说，每个作业只能在机器 1 和机器 2 上轮流进行。假设作业 i 在机器 j 上的处理时间为 $t[i,j]$。给定一个调度，令 $F[i,j]$表示作业 i 在机器 j 上的结束时间，则所有作业在机器 2 上的结束时间总和定义如下

$$f=\sum_{i=1}^{n}F[i,2]$$

由于总共只有两台机器，作业的处理顺序极大地影响结束时间 f。流水作业调度问题就是确定作业的处理顺序，使得 f 最小。

由于每个作业都要处理，只是每个作业在机器上的处理顺序不同而已，因此流水作业调度问题的一个候选解是 n 个作业的一个排列。设解向量为$(x_1,\cdots,x_n)$，

我们就是确定最优解是 n 个作业$(1,2,3,\cdots,n)$的哪一个排列。因此,解空间能够组织成一棵排列树,如图 12.15 所示。

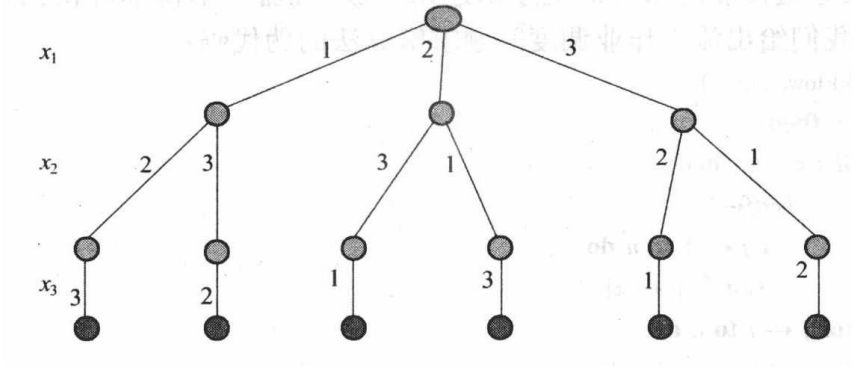


图 12.15

解空间的大小为 $n!$,如果在搜索过程中不考虑任何剪支函数,则回溯算法的效率显然不会太高。由于问题里没有显式的约束函数,因此我们考虑限界函数。令到第 i 层为止所得到的已经处理的作业在第 2 台机器上的结束时间为

$$B(i) = \sum_{j=1}^{i} F[x[j],2]$$

若 $B(i) \geqslant bestf$,则停止搜索第 i 层及其下面的层,否则继续搜索。其中,$bestf$ 表示目前为止得到的最小结束时间。

图 12.16 给出一个调度问题实例,其中作业的每一个排列都给出了相应的结束时间。从图 12.16 中可以看出,解(1,3,2)是最优调度,结束时间为 18。

$t[i,j]$	机器 1	机器 2
j_1	2	1
j_2	3	1
j_3	2	3

解 (1,2,3) 的结束时间是 19
解 (1,3,2) 的结束时间是 18
解 (2,1,3) 的结束时间是 20
解 (2,3,1) 的结束时间是 21
解 (3,1,2) 的结束时间是 19
解 (3,2,1) 的结束时间是 19

图 12.16

对于解(1,2,3),机器加工过程如图 12.17 所示。

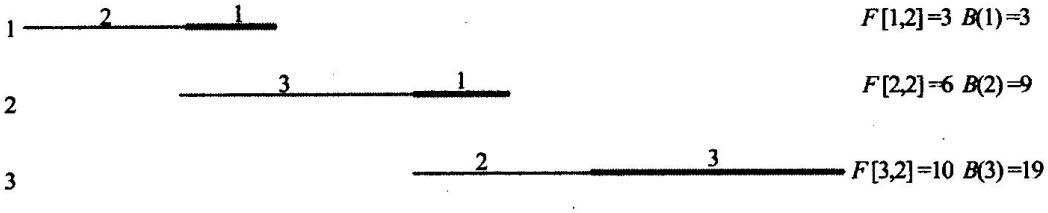


图 12.17

从图 12.17 可以看出，第 3 个作业的第 2 道工序何时开始处理，取决于第 2 台机器何时空闲以及第 3 个作业的第 1 道工序的结束时间。图中，虽然机器 2 已经空闲，但是第 3 道作业的第 1 道工序却还未完成，机器 2 不得不等待，直至该工序结束。下面我们给出流水作业调度问题求解算法的伪代码：

```
BacktrackFlowshop(i)
1  if i > n then
2      if f < bestf then
3          bestf←f
4          for j ← 1 to n do
5              bestx[j] ← x[j]
6  else for j ← i to n do
7      f1 ← f1 + t[x[j], 1]
8      if f2[i-1] > f1 then f2[i]←f2[i-1] + t[x[j], 2]
9      else f2[i]←f1 + t[x[j], 2]
10     f ←f+f2[i]
11     if f < bestf then   x[i] ↔x[j]
12                         BacktrackFlowshop(i+1)
13                         x[i] ↔ x[j]
14     f1 ←f1 - t[x[j], 1]
15     f ←f -f2[i]
```

BacktrackFlowshop(i)算法中，$f2[i]$即$f[i, 2]$。初始化$f1=0$，$f2[0]=0$。第 7 行～第 9 行表明，对于当前要加工的作业$x[j]$，在机器 1 上处理后，记录其结束时间$f1$，机器 2 能否接着处理作业$x[j]$，取决于当前机器 2 是否空闲。如果机器 1 上处理作业$x[j]$后，前一个作业在机器 2 上还没有处理完，作业$x[j]$只能等待机器 2。同样，如果机器 2 早就结束了对前一个作业的处理，而机器 1 仍然还在处理当前作业$x[j]$，因此，机器 2 必须等待机器 1 完成处理作业$x[j]$后，才能处理$x[j]$。因此，我们必须比较机器 2 完成前一个作业的完成时间$f2[i-1]$与$f1$的大小，然后才能决定机器 2 从什么时候开始处理$x[j]$。解原问题只需调用 BacktrackFlowshop(1)即可。

12.8 零件切割问题

给定一块宽度为 W 的矩形板，矩形板的高度不受限制。现需要从板上分别切割出 n 个高度为 h_i，宽度为 w_i 的矩形零件。切割的规则是零件的高度方向与矩形板的高度方向保持一致，问如何切割使得所使用的矩形板的高度 h 最小？

由于每个矩形零件都需要从矩形板上切割出来，只是每个矩形零件被切割的先后顺序不同而已，因此零件切割问题的一个候选解是 n 个矩形零件的一个排列。设解向量为$(x_1, \cdots, x_n)$，将 n 个矩形零件编号为 $1, 2, 3, \cdots, n$，我们就是确定最优解

是 n 个零件(1,2,3,…,n)的哪一个排列。因此,解空间能够组织成一棵排列树,如图 12.15 所示。

解空间的大小为 $n!$,如果在搜索过程中不考虑任何剪支函数,则回溯算法的效率显然不会太高。问题的约束条件是切割的零件不能重叠,这个比较容易实现,下面我们考虑限界函数。令到第 i 层为止已经切割的零件所使用的板的高度为 h,令到目前为止所得到的最优高度为 $besth$,若 $h \geqslant besth$,则停止搜索第 i 层及其下面的层,否则继续搜索。

除了零件切割的顺序外,另一个需要考虑的是如何从板上切割一个零件。对于零件切割问题,比较复杂,我们先介绍一个零件切割的过程 cut(i):每次从最左下角的位置开始切割一个零件 i,如果当前位置不可行,即切不出该零件,则必须寻找下一个可行位置,重复此过程,直到该零件能找到一个可行位置切割。下面,我们给出一个例子,给定一块板,其高度很长(可以足够使用),宽 $W=8$,给定三个矩形零件,零件 1 的大小为 $h_1=3, w_1=2$,零件 2 的大小为 $h_2=2, w_2=4$,零件 3 的大小为 $h_3=1, w_3=5$,如图 12.18 所示,其中矩形零件中的数字是零件的编号。图 12.18(a)中黑点 A 是最左下角位置,假定放置的矩形序列为 2,1,3,则矩形 2 可以放置在 A 点,如图 12.19(a)所示。图中黑点 B 是最左下角位置,当前矩形 1 可以放置在该位置,如图 12.19(b)所示。图 12.19(b)中最下面的黑点 C,矩形 3 不能放在该位置,因为矩形 3 的宽度方向只能沿着矩形板的宽度方向放置,$w_3=5$ 大于 CF。这时必须寻找新的最左下角位置,即图 12.19(b)中上面的黑点 D,同样该位置不能放零件 3。因此必须寻找新的最左下角位置,即图 12.19(c)中上面的黑点 E,这时零件 3 可以放在该位置,如图 12.19(d)所示。

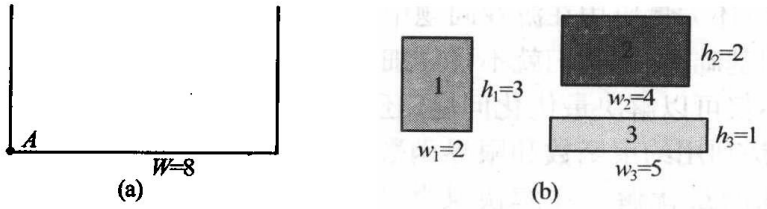


图 12.18

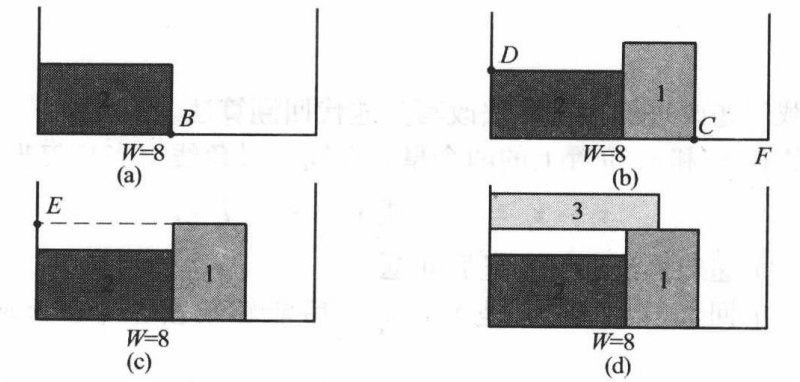


图 12.19

下面我们给出零件切割(StripPacking)问题的回溯算法的伪代码:

```
BacktrackPacking(i)
1  if i > n then
2       if h < besth then
3          besth←h
4          for j ← 1 to n do
5             bestx[j] ← x[j]
6  else for j ← i to n do
7       x[i] ↔ x[j]
8       h← cut(x[i])
9       if h < besth then
10         BacktrackPacking(i+1)
11      x[i] ↔ x[j]
```

上述算法的剪支过程非常简单,如果能设计出更好的剪支函数,例如考虑当前的浪费空间,则可以得到更有效的算法。此外,切割的过程是对于当前考虑的零件,我们寻找一个可行的位置,如果我们考虑当前最左下角的位置,来寻找一个可切割的矩形,效果是否可以更好呢? 回溯的过程是否需要修改,读者可以思考,见习题 12.16。

12.9 小 结

本章内容主要参考了文献[7, 16],介绍了回溯算法在一些问题中的应用,事实上,回溯算法还大量应用在游戏问题中,例如典型的迷宫问题等,它也是第 14 章启发式搜索的基础,这里我们就不再详细介绍。

回溯法不仅可以解决最优化问题,还可以用来解决判定问题。对于求解最优化问题,回溯法利用约束函数和限界函数剪支,从而跳过大量无须搜索的分支,较快地得到问题的最优解。在解决某些无启发式信息的问题时,应用很广,也很有效。

习题

12.1 将装载问题改进的递归算法改写为迭代回溯算法。

12.2 证明放在 x_i 和 x_j 位置上的两个皇后在同一对角线上当且仅当

$$x_i - x_j = i - j \text{ 或 } x_i - x_j = j - i$$

12.3 设计一个递归算法求解 8 皇后问题。

12.4 设计一个回溯算法,确定是否 k 个王后能够攻击一个 $n \times n$ 棋盘的所有位置。

12.5 设计一个回溯算法求 $\{1,2,\cdots,n\}$ 的所有子集。

12.6 给出一个 8×8 的棋盘，按照中国象棋马的走法，一个放在棋盘某个位置上的马是否可以恰好访问每个方格一次，并回到起始位置上？设计一个回溯算法求解上述问题。

12.7 设计 3-CNF-SAT 问题的回溯算法。

12.8 设计一个回溯算法求解汉密尔顿回路问题。

12.9 将旅行商问题的递归回溯算法改写为迭代回溯算法。

12.10 n 个雇员被分配做 n 件工作，分配第 i 个人做第 j 件工作的代价为 $c[i,j]$，其中 $c[i,j]\geqslant0$，$1\leqslant i,j\leqslant n$，设计一个回溯算法找出一种分配使得总代价最少。

12.11 用回溯法求解并行机调度问题（见第 11 章）。

12.12 设有 n 个独立的作业 $\{1,2,\cdots,n\}$，每个作业 i 都有一个开始时间 r_i、一个处理时间 p_i 和一个退出时间 q_i。现有一台机器 M 要加工这 n 个作业，每个作业 i 只能在 r_i 后开始加工，且加工后不能中断，直到加工完成，而且每个作业加工完成后，必须有一个退出时间。问怎样安排 n 个作业的加工顺序，使得加工完 n 个作业的完成时间最短？

12.13 将一个 6×10 的木块，划分成 12 个小图形，每个小图形的形状虽然互不相同，但是均由 5 个小方块构成，如图 12.20 所示。设计一个回溯算法，求出将这 12 个小图形再重新拼接成原 6×10 木块的所有可行方案。

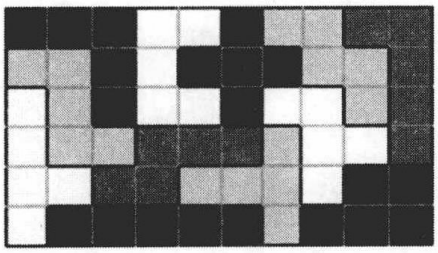


图 12.20

12.14 一个羽毛球队有男女运动员各 n 人，给定 2 个 $n\times n$ 矩阵 P 和 Q。$P[i,j]$ 是男运动员 i 和女运动员 j 配对组成混合双打时的竞赛优势；$Q[i,j]$ 则是女运动员 i 和男运动员 j 配合时的竞赛优势。显然，由于技术的配合和心理状态等各种因素的影响，$P[i,j]$ 不一定等于 $Q[i,j]$。设计一个回溯算法，计算出男女运动员的最佳配对法，使各组男女双方竞赛优势乘积的总和达到最大。

12.15 木匠史密斯准备了一些长度相等的棍子，用来做家具，可是他不懂事的儿子小史密斯，随机地把它们锯成了每一段长度都小于 m 的小棍子。为了不挨骂，现在他想把这些小棍子还原成初始的状态，可是他忘记了棍子初始的长度和数目。请设计一个算法帮他计算出初始棍子最小的可能长度。

12.16 请为零件切割问题的回溯算法设计一个考虑浪费的剪支函数或者改进切割过程的策略。

实验题

12.17 对旅行商问题,分别用动态规划法和回溯算法求解,用实验分析方法分析哪个算法更有效。

12.18 完成 XOJ 如下题目:1008,1027,1039,1063,1064。

12.19 完成 POJ 如下题目:1010,1011,1020,1062,1167,1190,1085,1753,2078,2488,2677。

第 13 章　分支限界

13.1　算 法 思 想

1985 年，Richard M. Karp 因为提出分支限界法而获得 Turing 奖。从提出到现在，分支限界（Branch and Bound）算法已经在许多问题中得到广泛的应用，其典型的应用就是求解最优化问题。当分支限界法用于求解最优化问题时，其效率主要取决于以下几个方面：

（1）对所给的问题，利用合适的方法构造出上界和下界。

（2）在每个节点利用限界函数来杀死节点以及扩展节点。

（3）使用目前最好的目标函数值来进行剪支，直到所有节点被遍历或是杀死。

假设我们考虑求解一个最小化问题。分支限界的主要思想是，在搜索的过程中，对每个可行的节点，始终维护一个上界（到目前为止最小的目标函数值）和下界（由限界函数得到的目标函数的估计值），然后每次选择一个具有最小下界值的节点进行扩展。经过一系列的扩展之后，搜索过程到达一个节点，在这个节点上，如果得到的下界大于或等于上界，那么就没有必要再扩展这个节点，即不需再延伸这个分支。对于最大化问题，剪支规则正好相反：一旦上界小于或等于先前确定的下界，那么就剪掉这个分支。由此可见，分支限界是对一个最优化问题的部分可行解进行剪支的一个方法，它试着把搜索集中在可以得到解的分支上。也就是说，它基于已经建立起的上下界以及约束函数来进行节点的扩展，其理由是扩展这样的分支可以得到更好的上下界，这样就会剪去更多的分支。总之，分支限界算法不是单纯以深度优先或宽度优先来进行搜索，而是把它们结合起来进行搜索，从而提高了搜索的效率。

分支限界算法的计算步骤如下：

（1）对所求的问题定义一个解空间，这个解空间至少包含问题的一个最优解。

（2）对解空间进行合理组织，以方便搜索。比较常见的组织方式是利用图或是一棵树。

（3）以宽度优先或者宽度结合深度的优先方式搜索解空间，并利用限界函数来避免搜索那些不能得到解的子空间。

分支限界算法的解空间树，常见的组织方式与回溯法一样，都包括子集树和排列树。只是展开解空间树的方式与回溯方法不一样。回溯法中，扩展一个节点是沿着该节点的某个分支一直搜索下去，到达叶节点后，再退回搜索，扩展节点有可

能被多次重复搜索。而分支限界算法则不一样，搜索过程是不断从活节点表中取出一个节点进行扩展。一个节点扩展完后，就被杀死，以后再也不会搜索该节点。对于扩展出的节点，通过剪支函数来决定其是否放入活节点表中。如果一个节点不能放入活节点表，意味着该节点及其分支不会被再次搜索，从而达到了剪支的目的。例如图 13.1 所示解空间树。黑圆圈是死节点，白圆圈代表活节点表中的活节点，而灰正方形是扩展节点。

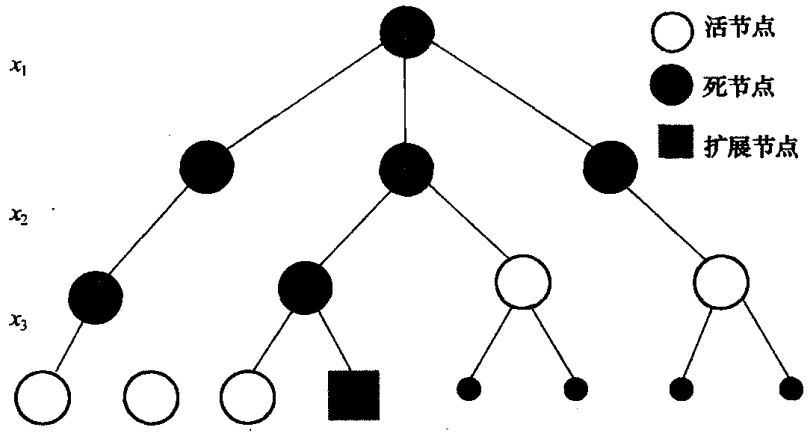


图 13.1

与回溯法一样，使用分支限界至少需要考虑以下三点：

（1）对于最大值（或者最小值）问题，需要考虑如何估算上界（或者下界）值。

（2）怎样从活节点表中选择一个节点作为扩展节点。

（3）怎样展开一棵搜索树，是宽度优先搜索，还是宽度优先结合深度优先进行混合搜索。

为了提高分支限界算法的效率，应该充分利用限界函数和约束函数来剪去无效的分支，并把搜索集中在有希望得到解的分支上。根据最优化问题的不同，我们相应地采取不同的剪支策略。类似回溯算法中的分析，一般来说，对于求解最大值问题，我们维护一个活节点表，从活节点表中选择一个节点作为扩展节点，对扩展节点的每个分支节点 i，计算其上界值 $B(i)$。如果当前最大目标函数值 *bestc* 不小于 $B(i)$，那么节点 i 就不会放入活节点表，否则放入。类似回溯算法限界函数的定义，从根节点 R 到叶节点 L 的目标函数值一定不会大于 $B(i)$，所以如果 $B(i) \leqslant bestc$，那么它表明正在搜索的节点 i 是无希望的（即死节点）。要剪掉扩展节点的分支节点 i，只要不把该节点放入活节点表即可。由于节点 i 不在活节点表中出现，因此它就没机会展开，从而达到剪支的目的，如图 13.2 所示。

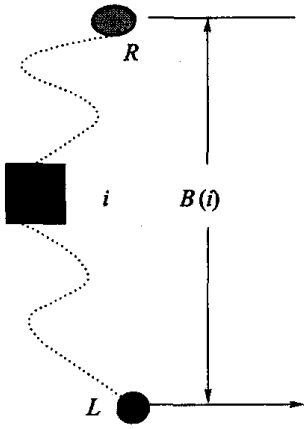


图 13.2

一般来说,对于求解最小值问题,我们维护一个活节点表,从活节点表中选择一个节点作为扩展节点,对扩展节点的每个分支节点 i,计算其下界值 $B(i)$。从根节点 R 到叶节点 L 的目标值一定不小于 $B(i)$,所以如果 $B(i) \geqslant bestc$,那么它表明正在搜索的节点 i 是无希望的(即死节点),所以要剪掉分支节点 i,否则继续。如图 13.2 所示。

分支限界同回溯一样,都是系统搜索一个解空间的方法。与回溯不同的是,在节点的扩展方法上,它主要以宽度优先的方式进行扩展,每个活节点变成扩展节点只有一次。当一个活节点变成扩展节点时,我们一次性把从它可以到达的节点全部展开,并把那些不能得到可行解的节点去掉(成为死节点),把剩下来的节点加到活节点表中,然后,从活节点表中选一个节点作为下一个扩展节点。重复上述过程,直到找到解或活节点表为空时为止。

下面我们讨论怎么样从活节点表中选择扩展节点,通常有两种方法来选择下一个扩展节点。

1) 先进先出 (FIFO)

这个方法是按节点放进活节点表中的次序从活节点表中选择节点。这个活节点表可看作是一个队列,使用宽度优先来搜索整棵解空间树。

值得注意的是,FIFO 方法与宽度优先的不同之处在于它不搜索那些不可行的节点。

2) 最小费用或者最大收益

用一个优先队列代替一个 FIFO 队列。这个方法与每个节点的费用或收益有关。如果我们求解最小值问题,需要搜索最小费用的解时,那么活节点表就用数据结构最小堆来表示。每次从活节点表中选择一个费用最小的节点进行扩展。如果我们求解最大值问题,需要得到最大收益的解时,活节点表可用一个最大堆来表示。每次从活节点表中选择一个收益最大的节点进行扩展。

根据选择扩展节点的不同,可以得到不同的分支限界法,例如先进先出分支限界算法、最小费用分支限界算法等。对于采取优先队列形式的分支限界法,一般需要确定评价一个节点的策略,以便优先展开哪个节点。通常,我们可以通过计算活节点表中节点的上界或下界值,作为选择一个活节点的标准。例如,对于最大值问题,可以计算活节点表中节点的上界值,并每次从活节点表中选择上界值最大的活节点进行扩展,便可以构造出最大收益分支限界算法。值得注意的是,活节点表中的节点,其上界或者下界值并不是该节点放入后才计算的,而是在放入之前就已经计算出。

下面,我们介绍分支限界算法的应用。

13.2 装载问题

上一章已经介绍了装载问题的回溯算法,现在我们来看如何用分支限界算法

来求解装载问题。不难分析装载问题的解空间以及约束函数，与回溯算法中的一样，即

$$C(i) = cw(i-1) + w_i$$

其中，$cw(i)$表示第 i 层当前扩展节点的重量

$$cw(i) = \sum_{j=1}^{i} w_j x_j$$

因此，如果 $C(i) > W$，那么第 $i-1$ 层扩展节点的 1 分支节点不放入活节点表。因此，下面给出基于约束函数的分支限界算法，其扩展节点的方式采用先进先出的方法，算法伪代码如下：

```
FIFOMaxLoading(w, W, n)
1    i ← 1
2    Enqueue(Q, -1)
3    cw ← 0; bestw ← 0
4    while Q≠∅ do
5      if C(i)≤W then
6            SaveQueue(Q, C(i), bestw, i)
7      SaveQueue(Q, cw, bestw, i)
8      cw←Dequeue(Q)
9      if cw = -1 then
10           if Q = ∅ then return bestw
11           Enqueue(Q, -1)
12           cw←Dequeue(Q)
13           i ← i+1
14   return bestw
```

FIFOMaxLoading(w, W, n)算法的输入参数是集装箱的重量 w，船的载重量 W，集装箱的个数为 n。第 1 行初始化当前层 i。第 2 行，Q 是一个先进先出队列，初始化队列，在其尾部增加一个 -1 标记。第 3 行初始化当前船已装集装箱的重量 cw，以及到目前为止，最佳解的载重量 $bestw$。第 5 行判断 1 分支的节点能否放入活节点表 Q 中，如果 1 分支的约束函数值 $C(i) = cw + w_i$ 没有超过船的载重量 W，则将 1 分支的节点加入活节点表中。第 7 行表示 0 分支的节点总是可以加入活节点表中。第 8 行表示从活节点表中取出一个活节点作为扩展节点。第 9 行判断如果取出的活节点是 -1 标记，则需要判断活节点表是否为空（∅），如果为空，则要返回 $bestw$，否则在活节点表后加一个 -1 标记，并重新取出一个节点作为扩展节点，并同时进入下一层。一个节点进队列需要判断该节点是否是一个叶节点，如果是，则需要更新到目前为止最佳解的载重量 $bestw$，否则，进队列，其伪代码如下：

```
SaveQueue(Q, wt, bestw, i)
1  if i = n then
2       if wt > bestw then
```

```
3          bestw ← wt
4   else
5          Enqueue(Q, wt)
```

从 SaveQueue(Q, wt, $bestw$, i)过程可知,进队列的,事实上是与该节点对应的当前载重量 wt。

例 13.1 给定一个装载实例,$n=3$,$w=[8, 6, 2]$, $W=12$。初始化队列如图 13.3(a)所示。此时队列非空,节点 B 和 C 依次进队列,如图 13.3(b)所示,值得注意的是,实际进队列的是数值 8 和 0,这里用节点 B 和 C 分别代替 8 和 0,每个节点所代表的数值见其旁边矩形中的数字,也就是约束函数值。B 节点约束函数值为 8,表示当前目标函数值的下界为 8。-1 出队列,而且此时队列非空,因此 -1 进队列,同时节点 B 出队列,如图 13.3(c)所示。展开节点 B,由于 B 节点的 1 分支节点的约束函数值为 14,大于船的载重量 12,因此 1 分支被剪支,B 节点的 0 分支节点 E 无条件进队列,如图 13.3(d)所示。C 节点出队列,该节点为扩展节点,如图 13.3(e)所示。展开节点 C,C 节点的两个分支节点 F 和 G 均可以进入队列,如图 13.3(f)所示。从队列中取出扩展节点 E,如图 13.3(g)所示。将节点 E 展开,其两个分支节点 J 和 K 均为叶节点,不再展开,如图 13.3(h)所示。此时,E 节点的 1 分支节点 J 的约束函数值为 10,表示当前目标函数值的下界更新为 10,得到一个解[1, 0, 1],其重量为 10。从队列中依次取出扩展节点 F 和 G,将其展开,其分支节点为叶子,如图 13.3(i)所示。此时队列为空,搜索停止,如图 13.3(j)所示。

上述 FIFO 分支限界算法,只利用了约束函数,如果利用限界函数,则可以提高分支限界算法的效率。考虑限界函数

$$B(i) = C(i) + r(i)$$

其中,$r(i)$为剩下集装箱的重量

$$r(i) = \sum_{j=i+1}^{n} w_j$$

如果 $B(i) \leqslant bestw$,那么当前第 i 层节点不放入活节点表,否则放入。其中 $bestw$ 表示到目前为止,最佳解的载重量。下面给出算法的伪代码:

```
ImprovedFIFOMaxLoading(w, W, n)
1  i←1
2  Enqueue(Q, -1)
3  cw ← 0; bestw ← 0; r ←0
4  for j ← 2 to n do r ← r + w[j]
5  while Q≠∅ do
6     if C(i) ≤ W then
7              if C(i) > bestw then bestw←C(i)
8              if i < n then Enqueue(Q, C(i))
9     if B(i) > bestw and i < n then Enqueue(Q, cw)
```

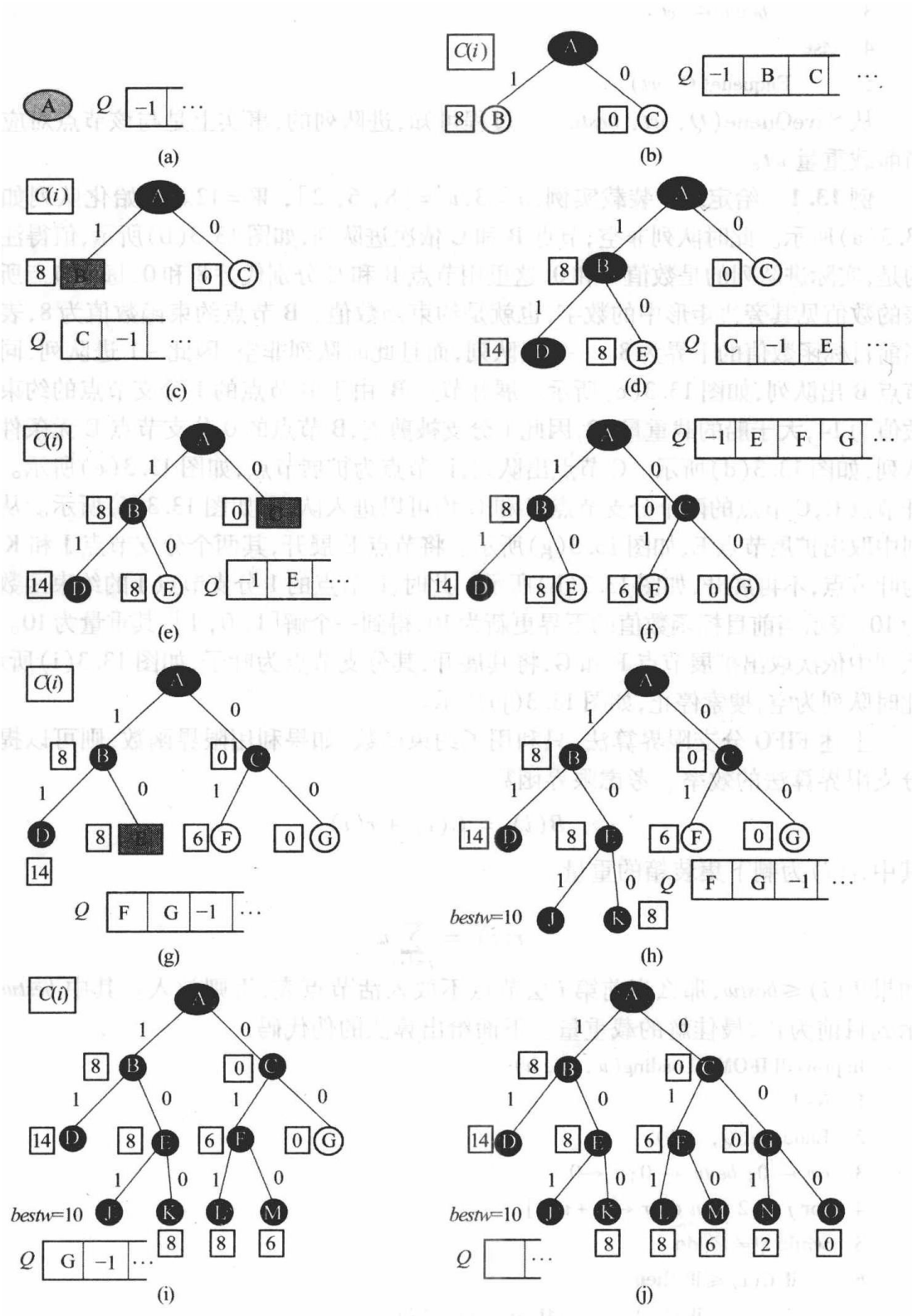


图 13.3

```
10      cw←Dequeue(Q)
11      if cw = -1 then
12              if Q = ∅ then return bestw
13              Enqueue(Q, 1)
14              cw←Dequeue(Q)
15              i ← i+1
16              r ← r-w[i]
17  return bestw
```

其中第4行从第2个物品开始计算下界。根节点为A,其两个分支节点,考虑第1个物品的重量,装还是不装。第6行判断1分支的约束条件是否满足,如果满足,则更新当前最好的重量值*bestw*。如果不是叶节点,则进队列。第9行当限界函数值比当前最好的重量值大,表示有希望找到更好的解。如果该节点不是叶节点,则进队列。第15行~第16行,考虑下一层节点时,其剩余重量要减去该层所考虑物品的重量。ImprovedFIFOMaxLoading(w, W, n)与FIFOMaxLoading(w,W,n)算法不同的是引进了限界函数,并且在算法第7行时刻更新*bestw*的值,因为*bestw*更新得越大,在第9行利用限界函数剪支的时候,就可以更早更多地剪掉一些无效的分支,从而提高了搜索的效率。

对例13.1,利用ImprovedFIFOMaxLoading(w,W,n)算法,其搜索过程如图13.4所示。初始化队列如图13.4(a)所示。此时队列非空,节点B进队列。值得注意的是,与前面装载问题的分析一样,实际进队列的是数值8,这里我们用节点B代替8,每个节点所代表的数值见其旁边矩形中的前面一个数字,也就是约束函数值。在B节点进队列的时候,我们已经得到目前最好的载重量*bestw*=8,而且其限界函数值为16,前者是目标函数的下界,而后者是目标函数的上界。对于A节点的0分支,由于其限界函数值(即上界)为8,不会比目前得到的*bestw*的大,因此被杀死,如图13.4(b)所示。-1出队列,而且此时队列非空,因此-1进队列,同时节点B出队列,成为扩展节点,如图13.4(c)所示。展开节点B,由于B节点的1分支节点违反约束条件,因此1分支被剪支,B节点的0分支节点E的限界函数值为10,有希望得到更好的解,因此无条件进队列,如图13.4(d)所示。-1出队列,此时队列非空,-1再进队列,从队列中取出节点E,节点E成为扩展节点,如图13.4(e)所示。将节点E展开,其分支节点J的约束函数值和限界函数值分别为10和10,已经是叶节点,不再进入队列Q,而节点K的限界函数值为8,没有超过当前最好的*bestw*,也不再进入队列,如图13.4(f)所示。此时得到一个解[1, 0, 1],其重量为10,搜索很快停止。因此在搜索的过程中,节点目标函数值的上下界是不断变化的,下界会不断地增加,而上界会不断地减少。

上述两个分支限界算法都是只算出最优解的值,如果要构造最优解的值,还必

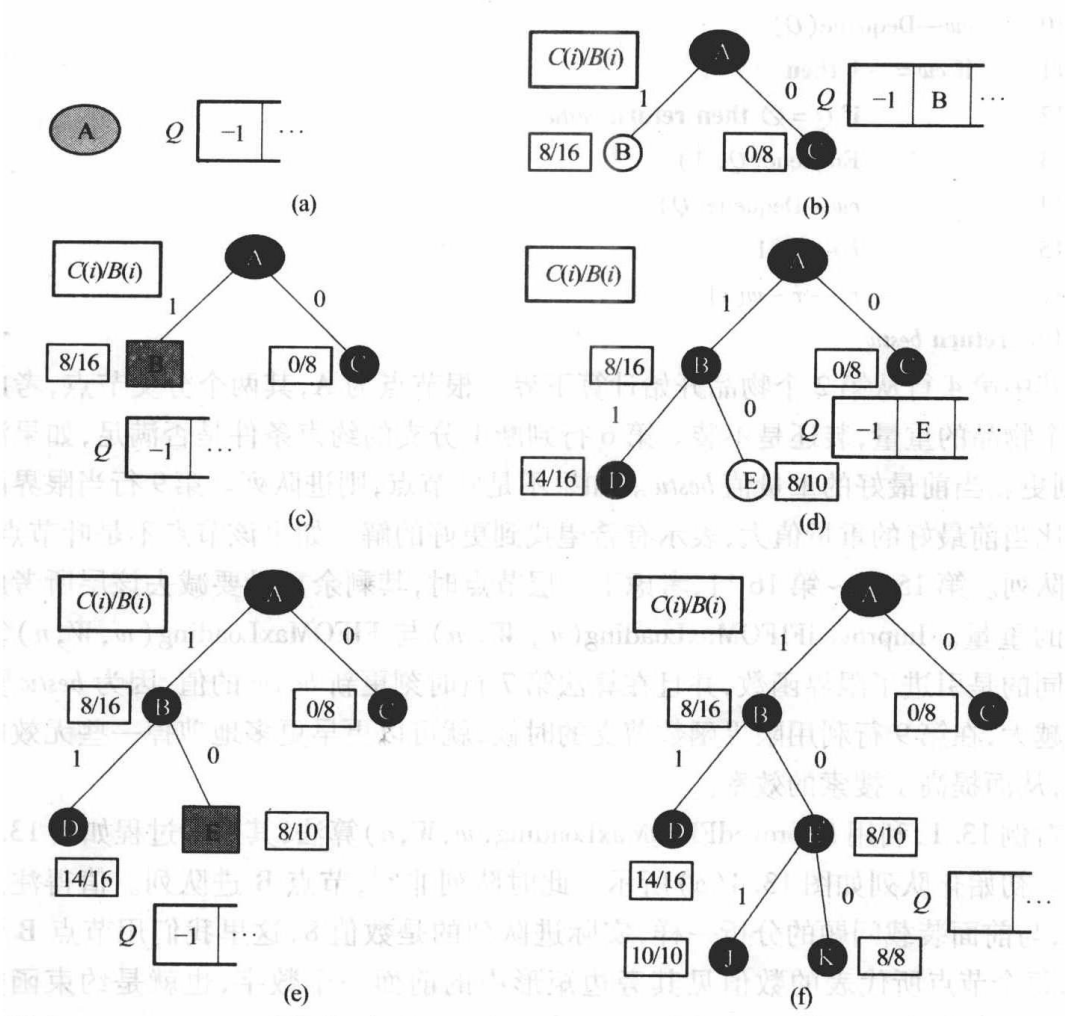


图 13.4

须保存有关解的信息。保存最优解的详细伪代码如下：

```
SolutionFIFOMaxLoading( )
1   i ← 1
2   Enqueue(Q, -1)
3   cw ← 0; bestw ← 0; r ← 0
4   for j ← 2 to n do r ← r + w[j]
5   while Q ≠ ∅ do
6       if C(i) ≤ W then // x[i] = 1
7           SaveQueue(Q, C(i), i, bestw, E, bestE, bestx, 1)
8       if B(i) > bestw then
9           SaveQueue(Q, cw, i, bestw, E, bestE, bestx, 0)
10      E←Dequeue(Q)
11      if E = -1 then
12          if Q = ∅ then return bestw
```

```
13          Enqueue(Q, -1); E←Dequeue(Q); i←i+1; r←r-w[i]
14      cw←E. weight
15  for j←n-1 downto 1 do
16      bestx[j]←bestE. Lchild
17      bestE←bestE. parent
18  return bestw
```

与前面介绍的两个分支限界算法不同的是，SolutionFIFOMaxLoading( )算法不仅保存当前最好的装载量 *bestw*，而且记录最好的叶节点 *bestE*，以及每个节点是从哪个分支过来的。详细地由 SaveQueue 函数记录与每个扩展节点有关的信息，例如，该节点的限界函数值或者约束函数值(*wt*)、该节点的父节点以及该节点是其父节点的那一个分支(*ch*)。算法中第 15 行～第 17 行从当前具有最大载重量的节点出发，沿着找其父节点的路线，完成对解的构造。SaveQueue 函数的伪代码如下：

```
SaveQueue(Q, wt, i, bestw, E, bestE, bestx, ch)
1  if i = n then
2      if wt > bestw then
3              bestE ← E
4              bestw ← wt
5              bestx[n] ← ch
6  else
7      b. weight ← wt
8      b. parent ← E
9      b. Lchild ← ch
10     Enqueue(Q, b)
```

b 可用一个结构体数据类型实现，表示活节点表中节点的数据类型。

前面，我们介绍了先进先出队列形式的分支限界算法，在选择节点进行扩展时，只是按照活节点进入活节点表的先后顺序扩展节点，没有利用每个活节点的限界函数值。事实上，活节点的限界函数值越大，越有希望更快找到最优解。下面介绍基于最大收益的分支限界算法。活节点表是一个最大优先队列，每个节点 i 都有一个限界函数值，即上界 $B(i)$，这个上界重量是与节点 i 相关的重量加上剩下集装箱的重量，即 $B(i)=C(i)+r(i)$，其中，$C(i)$，$r(i)$ 与前述定义一样。每个活节点都有 $B(i)$值，并以 $B(i)$值递减的次序成为扩展节点。值得注意的是，根据上界值 $B(i)$的计算可知，节点 i 的子树中没有节点的限界函数值比 $B(i)$还大。

特别地，如果一个叶节点 L 的装载重量等于它的上界重量，那么当该叶节点 L 以最大收益分支限界方式成为扩展节点时，没有其他活节点能产生一个叶节点 M，M 获得的装载重量比叶节点 L 的装载重量更大，因此，可以提前结束分支限界搜索。下面给出最大收益分支限界算法求解装载问题的伪代码：

```
MaxCostLoading( )
1  i ← 1
2  r[n] ← 0
3  for j ←n - 1 downto 1 do r[j] ←r[j + 1] + w[j + 1]
4  while i ≠n + 1 do
5      if C(i) ≤W then
6          AddLiveNode(Q, E, C(i) + r[i], 1, i + 1)
7      AddLiveNode(Q, E, cw + r[i], 0, i + 1)
8      N←ExtractMax(Q)
9      i ←N. level
10     E ←N. ptr
11     cw ←N. weight - r[i - 1]
12 for j ← n downto 1 do
13     bestx[j] ←E. Lchild
14     E ←E. parent
15 return cw
```

其中第3行的$r[j]$表示物品$j+1$到物品n的重量和，即剩余集装箱的总重量。第5行判断第$i-1$层的1分支节点A是否满足约束函数，如果满足，则将该节点A插入活节点表中，此时，A的限界函数值为$cw+w[i]+r[i]$。0分支的节点A总是可以加入活节点表，此时A的限界函数值为$cw+r[i]$。第8行从最大堆中选择一个具有最大收益值(最大重量值)的活节点。第12行~第14行是构造最优解。往活节点表中加入节点的算法如下：

```
AddLiveNode(Q, E, wt, ch, lev)
1    b. parent ← E
2    b. Lchild ← ch
3    N. weight ← wt
4    N. level ← lev
5    N. ptr ← b
6    Insert(Q, N)
```

AddLiveNode(Q, E, wt, ch, lev)算法的传递参数包括最大堆Q，扩展节点E，扩展节点的上界值wt，1分支还是0分支的标记ch，节点的层标记lev。其中，b是一个结构体数据类型，N是一个最大堆数据类型。

对于例13.1，最初解空间树如图13.5(a)所示，图13.5(b)中，两个白色节点B和C是活节点，B节点具有最大收益值16，因此B是扩展节点。优先扩展B，得到图13.5(c)。由于节点D不满足约束条件，因此不会放入活节点表，即它被杀死，当前活节点为C和E，由于E具有更大的上界函数值，因此E被优先扩展，得到图13.5(d)。E的1分支节点J已经是叶节点，J不会放入活节点表，被杀死。此时J获得的载重量为10，等于其上界函数值10，而J的上界函数值是当前活节点中上

界值最大的,因此扩展其他节点不会得到比 10 还大的载重量。因此分支限界搜索被终止。获得最优解[1,0,1],其最优载重量为 10。

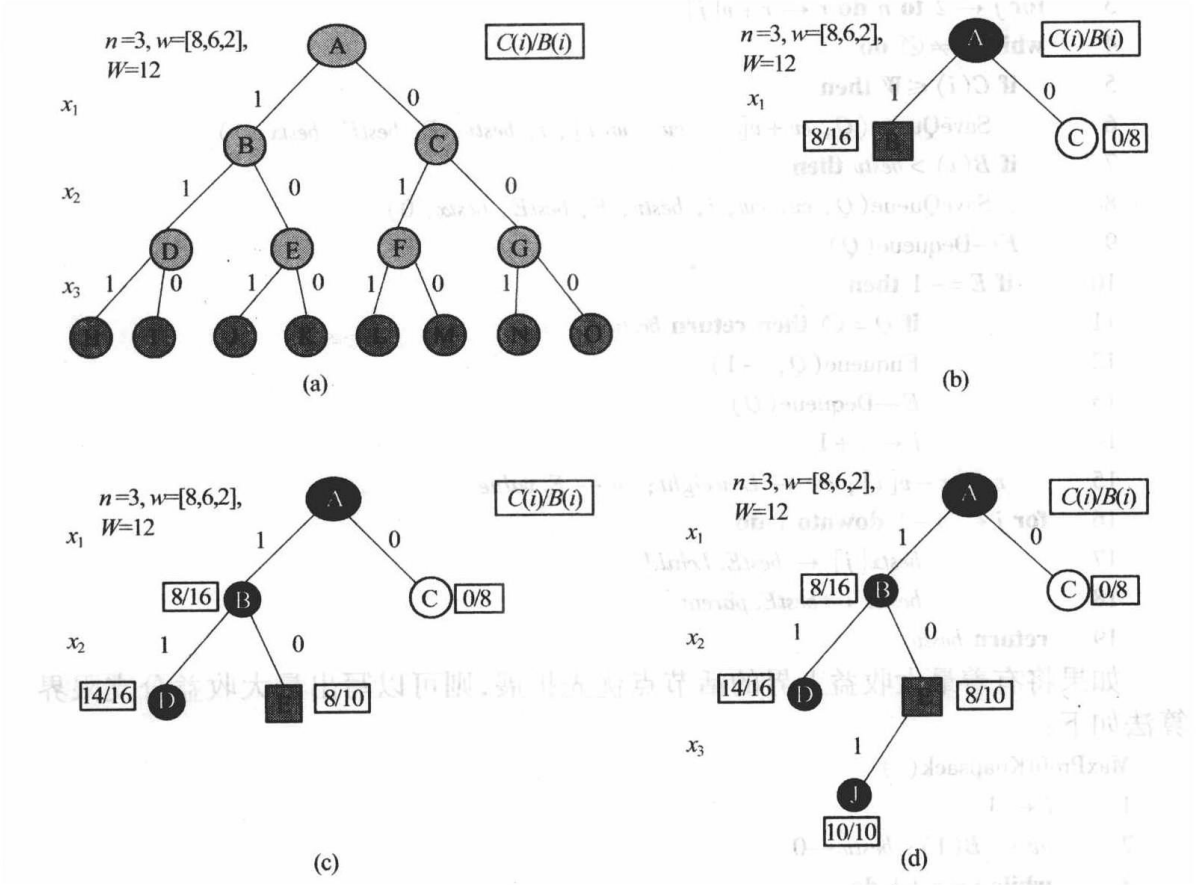


图 13.5

13.3 0/1 背包问题

第 12 章已经介绍了 0/1 背包问题的回溯算法,现在介绍求解背包问题的最大收益分支限界算法。背包问题的解空间为一棵子集树,第 12 章已经介绍过。令 $cw(i)$ 表示到第 i 层扩展节点的总重量,即

$$cw(i) = \sum_{j=1}^{i} w_j x_j$$

那么第 $i-1$ 层节点 1 分支节点的约束函数为 $C(i) = cw(i-1) + w_i$,如果 $C(i) > W$,则不用将 1 分支节点放入活节点表,否则放入。下面,我们考虑限界函数,对于第 i 层上的节点 A,我们估计它的收益的上界值为 $B(A) = cv(i) + r(i)$,$cv(i)$ 和 $r(i)$ 的定义与第 12 章的相同。如果 $B(A) \leqslant bestv$,那么就不用将 A 放入活节点表,否则放入。根据上述描述,类似最优装载问题,可以得到 FIFO 分支限界算法:

```
FIFOKnapsack( )
1    Enqueue(Q, -1)
2    i ← 1; cw ← 0; cv ← 0; bestv← 0; r ← 0
3    for j ← 2 to n do r ← r+v[j]
4    while Q≠∅ do
5      if C(i) ≤W then
6        SaveQueue(Q, cv+v[i], cw+w[i], i, bestv, E, bestE, bestx, 1)
7      if B(i) >bestv then
8      . SaveQueue(Q, cv, cw, i, bestv, E, bestE, bestx, 0)
9      E←Dequeue(Q)
10     if E =-1 then
11             if Q =∅ then return bestv
12             Enqueue(Q, -1)
13             E←Dequeue(Q)
14             i ← i+1
15       r ← r-v[i]; cw ← E. weight; cv ← E. value
16   for j ← n-1 downto 1 do
17             bestx[j] ← bestE. Lchild
18             bestE ← bestE. parent
19   return bestv
```

如果将有着最大收益上界的活节点优先扩展,则可以写出最大收益分支限界算法如下:

```
MaxProfitKnapsack( )
1    i ← 1
2    uv ← B(1); bestv ←0
3    while i≠n+1 do
4        if C(i) ≤W then
5                if cv+v[i] >bestv then bestv ← cv+v[i]
6                AddLiveNode(uv, cv+v[i], C(i), 1, i+1)
7                uv ← B(i)
8        if B(i) >bestv then
9                AddLiveNode(B(i), cv, cw, 0, i+1)
10       N←ExtractMax(Q); E←N. ptr; cw←N. weight
11       cv ← N. value; uv ← N. upvalue;   i ← N. level
12   for  j ←n  dounto 1 do
13       bestx[j] ←E. Lchild;   E ← E. parent
14   return bestv
```

其中,SaveQueue(Q, $cv+v[i]$, $cw+w[i]$, i, $bestv$, E, $bestE$, $bestx$, 1)和 AddLiveNode(E, uv, cv, wt, ch, lev)函数与前面类似。下面我们给出 MaxProfitKnapsack( )运行的例子。

例 13.2 给定一个背包实例:$n=3$,$W=30$,$w=[20,15,15]$,$v=[40,25,25]$。图 13.6 给出了最大收益分支限界算法运行的过程。其中图 13.6(a)已经扩展了

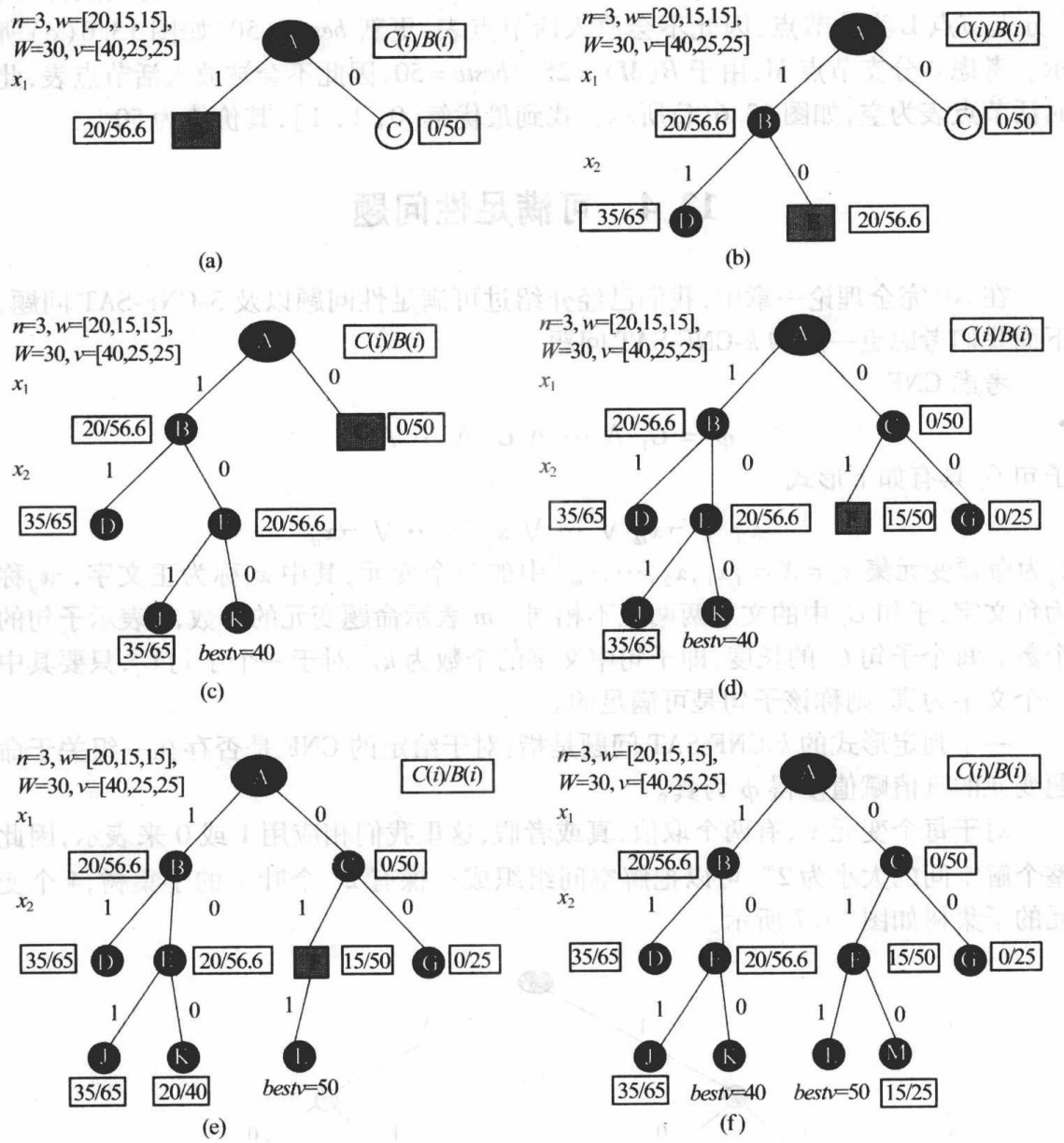


图 13.6

根节点,得到两个活节点 B 和 C。由于 B 节点的上界值比 C 的大,因此优先扩展 B 节点。同时,B 节点的 1 分支节点 D 不满足约束条件,因此 D 不会被放入活节点表,而 0 分支节点 E 可以放入,得到图 13.6(b)。对于当前活节点 E 和 C,E 的上界值比 C 的大,因此 E 被优先扩展,由于 E 节点的 1 分支节点 J 不满足约束条件,因此 J 不会被放入活节点表,而 0 分支节点 K 可以放入,但是由于 K 已经是叶节点,不会放入活节点表,但是需要保存目前最好的 *bestv* = 40,得到图 13.6(c)。当前只有一个活节点 C,将 C 扩展,由于其 0 分支 G 节点的上界值比 *bestv* 小,因此 G 不会

放入活节点表，即被杀死，得到图 13.6(d)。由于只有一个活节点 F，将其展开，其 1 分支节点 L 为叶节点，因此不会放入活节点表，更新 $bestv=50$，如图 13.6(e)所示。考虑 0 分支节点 M，由于 $B(M)=25<bestv=50$，因此不会被放入活节点表，此时活节点表为空，如图 13.6(f)所示。找到最优解[0，1，1]，其价值为 50。

13.4 可满足性问题

在 NP 完全理论一章中，我们已经介绍过可满足性问题以及 3-CNF-SAT 问题，下面我们考虑更一般的 k-CNF-SAT 问题。

考虑 CNF

$$\phi = C_1 \wedge \cdots \wedge C_i \wedge \cdots \wedge C_n$$

子句 C_i 具有如下形式

$$x_{i1} \vee \neg x_{i2} \vee \cdots \vee x_{ij} \vee \cdots \vee \neg x_{ik}$$

x_{ij}为命题变元集 $x_{ij} \in X=\{x_1,x_2,\cdots,x_m\}$ 中的一个变元，其中 x_{ij}称为正文字，$\neg x_{ij}$称为负文字，子句 C_i 中的文字两两互不相同。m 表示命题变元的个数，n 表示子句的个数。每个子句 C_i 的长度，即子句中文字的个数为 k。对于一个子句 C_i，只要其中一个文字为真，则称该子句是可满足的。

一个判定形式的 k-CNF-SAT 问题是指：对于给定的 CNF 是否存在一组关于命题变元的真值赋值使得 ϕ 为真。

对于每个变元 x_i，有两个取值，真或者假，这里我们相应用 1 或 0 来表示，因此整个解空间的大小为 2^m，可以把解空间组织成一棵有 2^m 个叶子的子集树，4 个变元的子集树如图 13.7 所示。

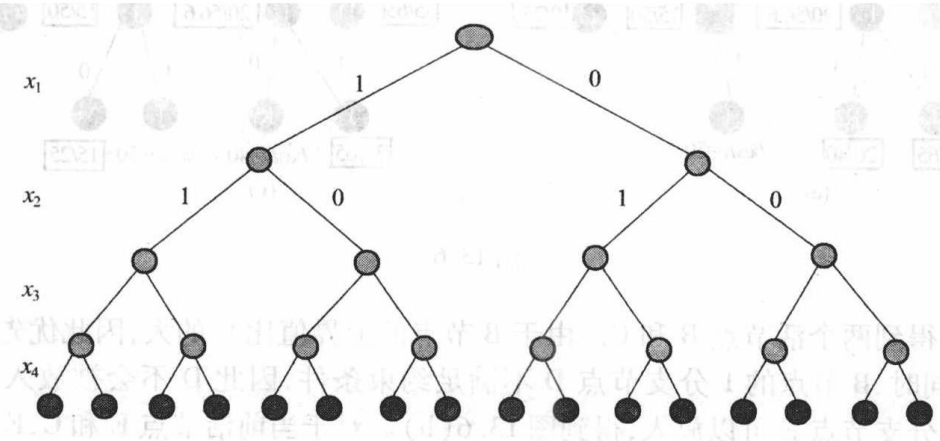


图 13.7

求解的过程可以从第 1 个变元，逐个变元进行赋值。约束函数为：当前部分变元$\{x_1,x_2,\cdots,x_{i-1}\}$的赋值已经确定，对当前变元 x_i 赋值，例如 $x_i=1$，如果有某个子

句,在当前赋值下,该子句为假(0),则可以剪掉该分支。类似0/1背包问题,我们可以用先进先出的分支限界算法来求解。但是这种算法的效率不高,因此我们可以利用最大收益分支限界算法来求解。即计算每个活节点的收益,活节点的收益值为在当前赋值下,能满足的子句数。我们的目标是满足所有的子句,因此,当前收益大的活节点被优先扩展。详细的算法过程如下:

```
MaxProfitSAT( )
1     i ← 1
2     while i≠m+1 do
3           cv←ok(i)
4           if cv>0 then
5             AddLiveNode(cv, 1, i+1)
6           cv←ok(i)
7           if cv>0 then
8             AddLiveNode(cv, 0, i+1)
9           N←ExtractMax(Q); i ← N.level
10    for j ←m downto 1 do
11          bestx[j] ←E.Lchild;   E ← E.parent
```

其中AddLiveNode(cv, 1, $i+1$)过程类似0/1背包问题,ok(i)函数如下:

```
ok(i)
1     cn←0
2     for j←1 to n do
3         if C_j=0 then return 0
4         else if C_j=1 then cn←cn+1
5     return cn
```

其中,cn表示在当前赋值下,可满足子句的个数。

下面我们给出一个3-CNF-SAT例子

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$$

求解的过程如图13.8所示。当前活节点表的节点为B和C,如图13.8(a)所示。由于C节点满足的子句数为2,因此被优先扩展,节点F和G被放入活节点表,其满足的子句数均为3,如图13.8(b)所示。节点F被优先扩展,由于其1分支节点使得第1个子句为0,因此该节点不放入活节点表,即被剪支,如图13.8(c)所示。扩展F的0分支节点,这时我们已经找到了一个可满足的赋值[0, 1, 0],使得$\phi=1$,算法终止,如图13.8(d)所示。

对于k-CNF-SAT,我们是否可以设计更好的限界函数呢?是否可以优先选择满足最多子句的变元呢?这些留作习题13.5。

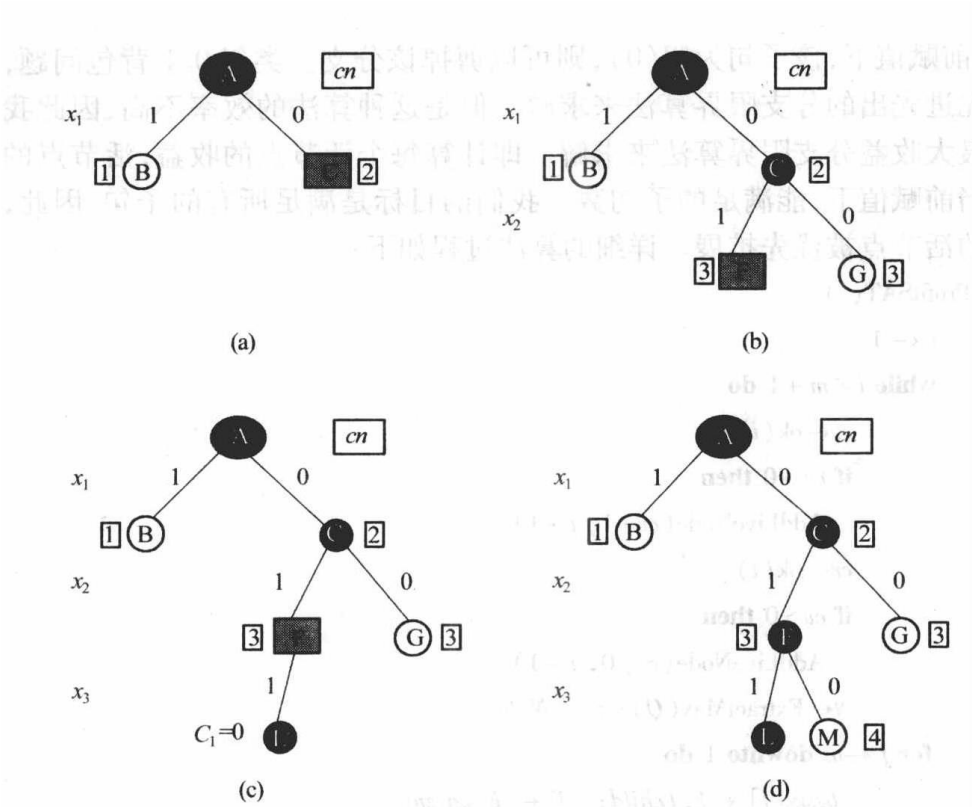


图 13.8

13.5 旅行商问题

第 12 章已经介绍了旅行商问题的回溯算法，现在我们介绍求解旅行商问题的最小权值分支限界算法。前面已经介绍了限界函数，令扩展节点 i 的权值为 $cw(i) = \sum_{j=2}^{i} w[x[j-1],x[j]]$。

假设第 $i-1$ 层节点选择顶点 $x[i]$往下走的限界函数为

$$B(i) = cw(i-1) + w[x[i-1],x[i]]$$

若 $B(i) \geqslant bestw$，则不用把 $x[i]$放入活节点表，否则放入。其中，$bestw$ 表示目前为止找到的最佳回路的权和。同时，具有最小权值 $B(i)$的活节点优先扩展。上述限界函数值只考虑搜索过来的路径的权值，并没有考虑后续路径的权值。如果我们能够构造出更大的 $B(i)$，则显然可以剪掉更多分支。

令扩展节点 i 的权值为 $B(i) = cw(i) + rw(i)$。$cw(i)$如上定义，$rw(i)$表示从每个剩余节点出发的权值最小边的权值总和，即

$$rw(i) = \sum_{j=i+1}^{n} \min_{i<k<n,k\neq j} \{w[x[j],x[k]]\}$$

如果 $B(i) \geq bestw$，则不用把 $x[i]$ 放入活节点表，否则放入。同时，具有最小权值 $B(i)$ 的活节点优先扩展。下面我们给出按照上述改进思路写出的基于优先队列的分支限界算法：

```
MinWeightTSP( )
1    MinSum ← 0
2    for i ← 1 to n do
3        Min ← ∞
4        for j ← 1 to n do
5            if w[i, j] ≠ ∞ and w[i, j] < Min then Min ← w[i, j]
6        if Min = ∞ then return ∞
7        MinOut[i] ← Min
8        MinSum ← MinSum + Min
9    for i ← 1 to n do E.x[i] ← i
10   E.s ← 1; E.cw ← 0; E.rw ← MinSum;   bestw ← ∞
11   while E.s < n do
12     if E.s = n - 1 then
13         if w[E.x[n-1], E.x[n]] ≠ ∞ and w[E.x[n], E.x[1]] ≠ ∞ and
           E.cw + w[E.x[n-1], E.x[n]] + w[E.x[n], E.x[1]] < bestw then
14             bestw ← E.cw + w[E.x[n-1], E.x[n]] + w[E.x[n], E.x[1]]
15             E.cw ← bestw; E.lw ← bestw
16             E.s ← E.s + 1
17             Insert(Q, E)
18     else for i ← E.s + 1 to n do
19         if w[E.x[E.s], E.x[i]] ≠ ∞ then
20             cw ← E.cw + w[E.x[E.s], E.x[i]]
21             rw ← E.rw - MinOut[E.x[E.s]]
22             B(i) ← cw + rw
23             if B(i) < bestw then
24                 for j ← 1 to n do N.x[j] ← E.x[j]
25                 N.x[E.s + 1] ← E.x[i]
26                 N.x[i] ← E.x[E.s + 1]
27                 N.cw ← cw;   N.s ← E.s + 1
28                 N.lw ← B(i);   N.rw ← rw
29                 Insert(Q, N)
30     E ← ExtractMin(Q)
31   if bestw = ∞ return ∞
32   for i ← 1 to n do bestx[i] ← E.x[i]
33   return bestw
```

MinWeightTSP( )算法在第 1 行～第 8 行计算每个顶点最小出边的权和。第 9 行～第 10 行初始化扩展节点的有关信息，到扩展节点为止路径的权和 $E.cw$，扩展

节点剩余路径的最小权和 $E.rw$，以及目前为止，最佳路径的权和 $bestw$。第 11 行～第 30 行重复扩展节点直到叶节点 $E.s = n-1$ 为止。第 13 行，判断扩展节点的分支顶点与前面路径以及与出发顶点是否相通，如果相通，然后比较该回路的权和 $B(i)$ 与目前最佳回路的权和，如果该回路的权和更小，则更新最佳回路，然后将该分支叶节点插入最小堆，否则删除该叶节点。如果不是叶节点，则考虑扩展节点的分支顶点 $E.x[i]$ 与到扩展节点的路径的末端顶点是否相通，如果相通，则估计扩展节点的权值 $B(i)$，如果 $B(i) \geqslant bestw$，则将该扩展节点的分支剪掉。第 32 行将最优路径保存到数组 $bestx$。最后，将最优回路的权和返回。

13.6 流水作业调度问题

前面我们已经介绍了流水作业调度问题的回溯算法，现在我们介绍求解流水作业调度问题(Flow Shop Problem)的分支限界算法。虽然回溯算法里面已经介绍了一个限界函数，但是其效率不高，我们可以对其进行改进。

令 $x = \{x[1], x[2], \cdots, x[i]\}$ 表示到扩展节点 i 为止已经处理过的作业集合。那么 $f = \sum_{j=1}^{i} F[x[j],2] + rf(i)$。其中，$rf(i) = \sum_{j=i+1}^{n} F[x[j],2]$，精确计算 $rf(i)$ 是非常困难的，我们能估计出它的下界吗？

考虑以下两种情形：

(1) 每个剩余的作业都可以在机器 1、机器 2 上连续地进行处理，也就是说一个作业在机器 1 上处理完后可以不需要等待继续在机器 2 上进行处理，此时有

$$rf1(i) = \sum_{j=i+1}^{n} (F[x[i],1] + (n-j+1)t[x[j],1] + t[x[j],2])$$

显然有 $rf(i) \geqslant rf1(i)$。

(2) 机器 2 没有等待时间，即机器 2 在完成某个作业后，它可以不用等待处理下一个作业，此时有

$$rf2(i) = \sum_{j=i+1}^{n} (\max\{F[x[i],2], F[x[i],1] + \min_{i \leqslant k \leqslant n} t[x[k],1]\} + (n-j+1)t[x[j],2])$$

显然有 $rf(i) \geqslant rf2(i)$。

对于情形(1)，若我们将作业按 $t[j,1]$ 非递减的顺序排列，那么可得到更小的 $rf1(i)'$，即

$$rf1(i)' \leqslant rf1(i)$$

对于情形(2)，若我们将作业按 $t[j,2]$ 非递减的顺序排列，那么可得到更小的 $rf2(i)'$，即

$$rf2(i)' \leqslant rf2(i)$$

因此

$$f = \sum_{j=1}^{i} F[x[j],2] + rf(i) \geqslant$$

$$\sum_{j=1}^{i} F[x[j],2] + \max\{rf1(i),rf2(i)\} \geqslant$$

$$\sum_{j=1}^{i} F[x[j],2] + \max\{rf1(i)',rf2(i)'\}$$

令节点 i 的权值为 $B(i) = \sum_{j=1}^{i} F[x[j],2] + \max\{rf1(i)',rf12(i)'\}$，若 $B(i) \geqslant bestf$，则不用将节点 i 放入活节点表，否则放入，与此同时，权值 $B(i)$ 最小的活节点优先扩展。下面，我们给出基于上述限界函数的分支限界算法：

```
MinCostFlowShop( )
1   E.s←1; E.f ← 0; E.f2 ← 0; E.f1 ← 0
2   for j←1 to n do E.x[j] ←j
3   bestf←∞
4   while E.s≤n do
5      if E.s = n then
6         if E.f < bestf then
7            bestf←E.f
8            for i←0 to n do bestx[i] ←E.x[i]
9            return bestf
10     else for i←E.s + 1 to n do
11              for j←1 to n do
12                    N.x[j] ←E.x[j]
13              N.x[E.s + 1] ← E.x[i]
14              N.x[i] ← E.x[E.s + 1]
15              f1 ← E.f1 + t[N.x[E.s +1], 1]
16              if E.f2 >f1 then f2←E.f2 +t[N.x[E.s + 1], 2]
17              else f2 ←f1 +t[N.x[E.s + 1], 2]
18              N.f ← E.f +f2
19              N.s ← E.s +1
20              N.f2 ←f2
21              N.f1 ←f1
22              if B(i) < bestf then Insert(Q, N)
23        E←ExtractMin(Q)
24  return bestf
```

其中优先队列仍然用最小堆来实现，其中 N 是一个最小堆节点。第 10 行，对

当前扩展节点 $E.s$ 的每个子节点，依次判断，看能否加入最小堆中。MinCostFlowShop()算法的过程与旅行商问题的 MinCostTSP()过程类似，这里就不再详细描述。

对于图 12.16 给出的调度实例，图 13.9(a)给出了排列树的初始状态。图 13.9(b)的三个活节点中，B 节点和 D 节点的 $B(i)$ 最小，因此 B 节点为扩展节点，将其展开，得到图 13.9(c)。图 13.9(c)的活节点中，F 节点具有最小的 $B(i)$ 值，因此成为当前扩展节点，将其展开，得到一个叶节点 L，此时 $bestf=18$ 等于其下界值，因此，算法直接终止。

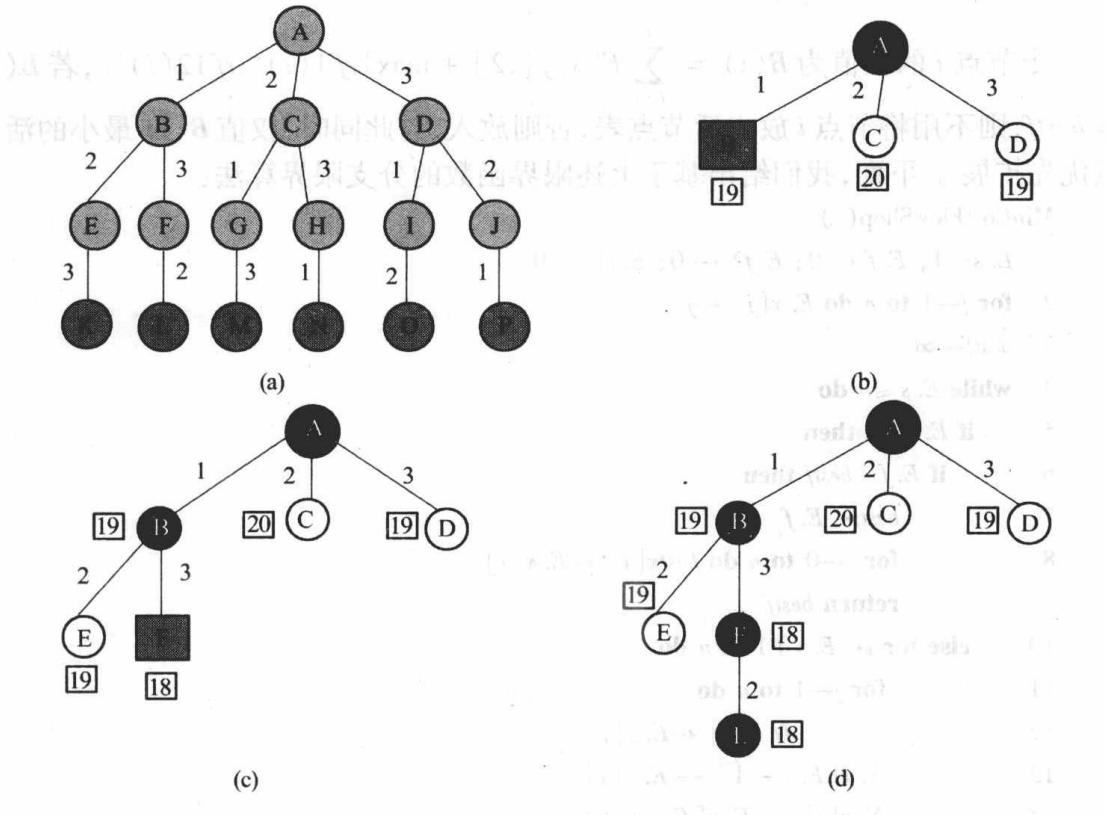


图 13.9

13.7 0/1 背包问题实验

前面已经介绍了求解 0/1 背包问题的动态规划算法(DPKnapsack)、回溯算法(BacktrackKnapsack)和最大收益的分支限界算法(MaxProfitKnapsack)。为了比较三种算法的效率，我们将三种算法用C++ 语言编程实现，并在 CPU 为 2.4GHz，内存为 512MB 的 PC 机上测试规模 n 分别为 1000，1200，1400，1600，1800，2000，2200，2400 的同样实例，这些实例均随机生成，算法的执行时间如表 13.1 和图 13.10 所示，其中时间用秒(s)表示。图 13.10 清楚直观地表示了随着问题规模 n

的增大,执行时间的增长趋势。三种算法都能找到最大价值的物品的子集,但是从表 13.1 和图 13.10 可以看出,动态规划算法随着问题规模的增大,需要的时间越来越多,回溯算法的计算速度比动态规划算法快。虽然回溯算法与分支限界算法的计算速度差不多,但是后者相对更快,因此分支限界算法的效率更高。前面我们已经分析过,动态规划算法的时间复杂度为 $O(nW)$,算法的计算速度取决于背包的载重量 W,因此该算法的效率还与具体的实例有关。分支限界算法是用空间换时间,从而计算速度更快,这与前面我们对三个算法的理论分析一致。

表 13.1 DPKnapsack、BacktrackKnapsack 和 MaxProfitKnapsack 的实验结果

n	1000	1200	1400	1600	1800	2000	2200	2400
DPKnapsack	0.109	0.187	0.203	0.296	0.421	0.578	1.125	1.218
BacktrackKnapsack	0.031	0.063	0.078	0.063	0.11	0.14	0.14	0.109
MaxProfitKnapsack	0.015	0.015	0.031	0.031	0.031	0.062	0.046	0.046
最优价值	282000	414610	455339	607732	748955	940129	1305502	1312372

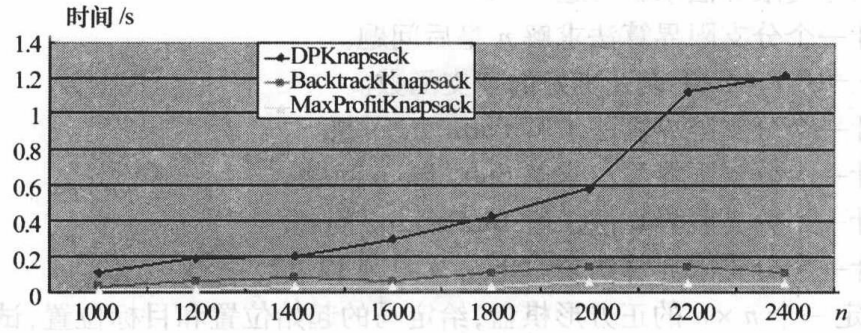


图 13.10

13.8 小 结

本章内容主要参考文献[7,16],介绍了分支限界算法的设计及其应用。

从前面的分析可知,回溯法一般适于求解要找出所有满足约束条件的解的问题,有通用的解题方法之称,而分支限界法一般适于求解最优化问题。回溯法以深度优先的方式搜索一棵解空间树,分支限界常常以宽度优先的方式搜索解空间树。一般来说,回溯需要更多的时间,基本不需要什么空间,而分支限界法需要更多的空间,但是效率更高。特别是,采用优先队列的分支限界法,可以看作是对回溯算法求解最优化问题的一个改进,通常具有更高的效率。

此外,回溯法和分支限界算法基本上都用到了剪支函数,剪支函数的好坏直接

关系到算法的性能。然而，不是所有的分支都可以剪掉，这需要设计出合理的剪支函数。在设计剪支函数时，一般需要注意不能剪去含有正确解的分支，如果不能保证这一点，剪支也就失去了意义。在此基础上，应该具体问题具体分析，设计有效的剪支函数以提高搜索的效率。

习题

13.1 对于旅行商问题，应用分支限界算法求解图 13.11 所示实例。

$$\begin{pmatrix} \infty & 5 & 2 & 10 \\ 2 & \infty & 5 & 12 \\ 3 & 7 & \infty & 5 \\ 8 & 2 & 4 & \infty \end{pmatrix}$$

图 13.11

$$\begin{pmatrix} 3 & 5 & 2 & 4 \\ 6 & 7 & 5 & 3 \\ 3 & 7 & 4 & 5 \\ 8 & 5 & 4 & 6 \end{pmatrix}$$

图 13.12

13.2 对于上一章的习题 12.10，应用分支限界算法求解以下实例：给定 4 个雇员和 4 件工作，其代价矩阵如图 13.12 所示，其中，第 i 行对应第 i 个雇员，第 j 列对应第 j 项工作。

13.3 利用分支限界法求解习题 12.11。

13.4 设计一个分支限界算法求解 n 皇后问题。

13.5 为 k - CNF - SAT 设计更好的剪支函数。

13.6 设计一个分支限界算法求解 Partition 问题。

13.7 设计一个分支限界算法求解 SubsetSum 问题。

13.8 设计一个分支限界算法求解 BinPacking 问题。

13.9 设计一个分支限界算法重解上一章习题 12.12。

13.10 给定一个 $n \times n$ 的正方形棋盘，给定马的起始位置和目标位置，试设计一个算法求解马从起始位置跳到目标位置需要的最少步数。在任意一个位置，马跳一步，可以从该位置跳到如图 13.13 所示的 8 个可能位置中。(PKU1915)

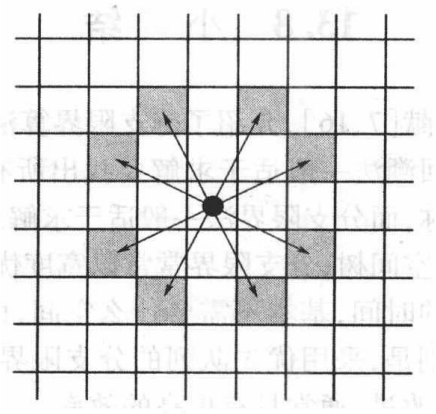


图 13.13

实验题

13.11 动态规划算法、回溯算法与分支限界算法各有什么优缺点？编程实现三个算法，并求解旅行商问题，用实验分析方法说明你的结论。

13.12 完成 XOJ 如下题目：1063，1064，1066。

13.13 完成 POJ 如下题目：1041，1077，1103，1167，1324，1475，1817，2908，3322，3635，3662。

第 14 章　启发式搜索

14.1　算 法 思 想

在搜索过程中，并没有利用获取的启发式信息，总是按照预定的搜索策略实行搜索，这种搜索方式称为盲目搜索，反之，称为启发式搜索。前面我们介绍的深度优先搜索以及宽度优先搜索都是盲目搜索，而回溯和分支限界算法可以称为启发式搜索(Heuristic Search)。

本章提及的启发式信息，也称为中间信息或者面向问题领域的知识，常常是一种经验或者直觉。借助启发式信息，可以避免盲目搜索，提高搜索的效率。启发式搜索一般有两类：一类是借助于启发式信息，一步一步构造出问题的近似解或者最优解；另一类是从一个初始解(一般是随机产生的初始解)出发，借助于启发式信息，不断迭代改进问题的解，直到找到合乎问题要求的近似解。前者包括回溯以及分支限界，后者包括现代启发式算法，例如遗传算法、模拟退火以及禁忌搜索等。由于现代启发式算法非常依赖启发式信息，不管如何设计，都有可能找不到问题的最优解，有时甚至连解也找不到。本章主要介绍第一类启发式搜索算法，这类启发式算法如果设计得当，它通常能找到问题的最优解。

前面我们介绍过，为方便搜索，常将搜索空间组织成一棵树或者图的形式。在搜索树或者图时，启发式信息主要用来控制搜索的过程，例如：

(1) 如何选择当前要展开的节点？

(2) 如何展开，是部分展开还是全部展开？

(3) 如何决定舍弃还是保留新生成的节点？

(4) 如何决定停止或继续搜索？

(5) 如何定义启发式函数？

(6) 如何决定搜索的方向？

上述问题，主要通过一个评估函数来解决。因此，启发式搜索与盲目搜索最大的区别，就在于选择当前要展开的节点，是根据一个评估函数，对节点进行评估，然后根据节点评估值的大小来进行选择，选择评估值最优的节点进行展开。如果有几个节点，其评估值相同，则一般根据深度信息来选择。

一般地，节点 n 的评估函数 $f(n)$ 可表示如下

$$f(n) = h(n) + d(n) \tag{14.1}$$

$h(n)$ 是启发式函数，与启发式信息相关。启发式函数 $h(n)$ 的设计非常重要，它无

固定的设计模式，主要取决于面向具体问题结构的知识和求解的技巧。$d(n)$是开始节点到节点 n 的深度信息。要设计出满足(14.1)的启发式函数，一般比较困难，需要很高的技巧。前面我们在回溯以及分支限界算法中提到的剪支函数都利用到了启发式信息，而且设计剪支函数的技巧性很强。特别地，剪支函数设计的好坏，直接影响算法的效率。

启发式搜索在游戏、理论证明、问题求解领域都有大量的实际应用，启发式搜索包括很多算法，例如，局部搜索、最好优先搜索、A* 以及博弈搜索等。这些算法都使用了启发式函数，但是在具体选取最佳搜索节点时的策略却有不同。下面介绍常用的 A* 搜索算法和博弈搜索算法。

14.2 A* 搜索

A* 搜索算法，是一个著名的启发式算法，已经被广泛应用于求解组合优化问题，特别是图的搜索问题。A* 搜索算法最为核心的部分，就在于评估函数的设计。在 A* 搜索算法的评估函数中，$d(n)$表示从起始节点到当前节点的代价，通常用当前节点在搜索树中的深度来表示。启发式函数 $h(n)$表示当前节点到目标节点的评估值，是启发式搜索算法最为关键的部分。$h(n)$设计的好坏，直接影响着启发式算法能否找到问题的最优解。因为对于宽度优先搜索来说，$h(n)=0$，一点启发式信息也没有，虽然能找到最优解，但是运行时间代价太大。$h(n)$的启发式信息越多，则剪掉的节点也越多，但是设计也更复杂，更讲究技巧。

一般地，A* 搜索算法的评估函数包括两个约束条件：

(1) $h(n)$不大于节点 n 到目标节点 t 的实际最小代价 $h^*(n)$，即 $h(n) \leqslant h^*(n)$。

(2) 任意节点的评估值 f 必须不小于父节点的 f 值，即 f 单调递增。

当评估函数 f 满足上述两个约束条件时，按正确的启发式搜索算法，在合理的限制条件下，可证明 A* 能够获得问题的最优解。这就是常说的 A* 搜索算法所具有的优势。

由于节点 n 到目标节点 t 的实际最小代价 $h^*(n)$是无法知道的，因此要设计出满足条件(1)的启发式函数是很难的，但是一般希望设计的 $h(n) \leqslant h^*(n)$，并且尽量接近 $h^*(n)$。例如，对于图 14.1，我们要寻找从 s 到 t 的一条最短路径，$d(n)$表示从起始顶点 s 到当前顶点 n 的距离，从 s 出发可以到达 3 个顶点 v_1, v_2, v_4，因此 $d(v_1)=3, d(v_2)=4, d(v_4)=5$。对于顶点 v_1，从该顶点到目标顶点的实际最短距离 $h^*(v_1)$是多少呢？事实上，从 v_1 到 t 有

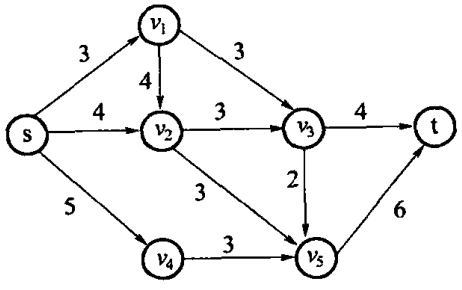


图 14.1

5 条路径:v_1,v_3,t,其长度为 7;v_1,v_3,v_5,t,其长度为 11;v_1,v_2,v_3,t,长度为 11;v_1,v_2,v_3,v_5,t,其长度为 15;v_1,v_2,v_5,t,其长度为 13;因此,$h^*(v_1)=7$。由此可见,$h^*(n)$在通常情况下是很难知道的,即使知道怎么求,求起来也相当复杂。事实上,从顶点 v_1 出发到 t 有两条可能的路径,从顶点 v_1 出发到 v_2 或者到 v_3,这表示从顶点 v_1 出发到 t,其距离最小为 3(v_1 到 v_3),我们可以把这个距离作为 $h^*(v_1)$的估计,即 $h(v_1)=3$。关于距离的估计,接下来会详细给出。

不过,对于许多问题,例如我们下面介绍的图的最短路径问题和 8 数字问题,其相应的启发式函数一般满足约束条件(1)和(2),因而启发式搜索在这两个问题中得到了成功的应用。

一般地,A* 搜索算法可以描述如下:

```
A*()
1  open ← {s};
2  closed ←∅
3  while open≠∅ do
4      remove leftmost state from open, call it x
5      if x is a goal node then return success
6      else
7          generate children of x
8          for each child c do
9              if c is already on open list then
10                 if c was reached by a shorter path then
11                     update f(c)
12             else if c is already on closed list then
13                 if c was reached by a shorter path then
14                     remove c from closed list
15                     add c to open list and update f(c)
16             else
17                 assign c a heuristic value f(c)
18                 add c to open list
19         put x on closed list and reorder states on open by f
20 return failure
```

A* 搜索算法采用最小代价优先的策略,即在所有将扩展的节点中,选择具有最小代价的节点作为下一个扩展节点。其终止准则是如果选择的节点是目标节点,即停止 A* 搜索算法。A*()算法中 *open* 表是一个优先队列,保存即将要展开的节点。*closed* 表是一个数据结构,例如数组或者链表,保存展开过的节点。s 是开始节点。A* 搜索算法在第 3 行判断 *open* 表是否为空,如果 $open\neq\varnothing$,意味着当前有可以扩展的节点,算法在第 4 行选择 *open* 表中最左的节点,即代价最小的节点 x,如果 x 是目标节点,则返回成功(success),否则产生 x 的子节点。第 8 行,对 x

的每个子节点，依次执行第 9 行 ~ 第 18 行。对 x 的某个子节点 c，如果该节点在 *open* 表中，且有一条更短的路径搜索到节点 c，则更新 $f(c)$；如果该节点在 *closed* 表中，且有一条更短的路径搜索到节点 c，则从 *closed* 表中移走 c，将其重新放入 *open* 表中，并更新 $f(c)$；否则，将 c 放入 *open* 表中。如果 x 的所有孩子节点都已经搜索过，则将其放入 *closed* 表中，然后对 *open* 表中的节点按照代价 f 进行排序，使代价最小的节点在最左边。显然，*open* 表可用数据结构最小堆实现。

下面，我们介绍 A* 搜索算法的应用。

14.2.1 最短路径问题

给定一个有向图 $G=(V,E)$ 以及所有边 (u,v) 的权值 $w(u,v)$，要寻找给定顶点 s 到 t 的最短路径问题，可以利用图算法一节介绍的 Dijkstra 算法来求解。由于 Dijkstra 算法的时间复杂度是 $O(|V|^2)$，虽然能够在多项式时间内找到最短路径，但是当顶点很多并且需要实时计算的话，Dijkstra 算法就无法满足要求了。而 A* 算法在处理这类有实时需求的问题时，则显示出独特的优势。

前面一节，我们已经分析了对于最短路径问题，求一个顶点 n 到目标顶点的最短路径的权和 $h^*(n)$ 是非常复杂的，因此我们只要给出它的一个估计 $h(n)$ 即可。由于一个顶点 n 到目标顶点的最短路径的权和不会少于顶点 n 的出边的最小权值，因此，我们令

$$h(n) = \min_{v \in V} \{w(n,v) : (n,v) \in E\}$$

上述估计满足约束条件(1)，因此，顶点 n 的评估函数可以构造如下

$$f(n) = h(n) + d(n)$$

其中，$d(n)$ 表示从起始顶点 s 到当前顶点 n 的距离，即路径的权和。上述评估函数满足约束条件(2)，因此，利用 A* 搜索算法能够找到最短路径。

图 14.2 给出了 A* 搜索算法找图 14.1 中 s 到 t 的最短路径的过程。初始 *open* 表包含一个顶点 s，取出它并将其展开，得到图 14.2(a)，此时 *open* = $\{v_1, v_2, v_4\}$，*closed* = {s}。将评估值最小的顶点 v_1 取出，将其展开，其孩子顶点 v_2 已经在 *open* 表中，而且路径 s, v_1, v_2 的长度不会比路径 s, v_2 更短，因此 v_2 的评估值不会更新。v_3 放入 *open* 表，其评估值为 8。如图 14.2(b)所示，此时 *open* = $\{v_2, v_4, v_3\}$，*closed* = $\{s, v_1\}$。将评估值最小的顶点 v_2 取出，将其展开，对 v_2 的孩子顶点 v_3，由于它已经在 *open* 表中，而且其路径 s, v_2, v_3 的长度不会比原来的路径长度更短，因此不需要更新。对于另一个孩子顶点 v_5，其不在 *open* 表中，也不在 *closed* 表中，因此将其添加到 *open* 表中，得到图 14.2(c)，此时 *open* = $\{v_4, v_3, v_5\}$，*closed* = $\{s, v_1, v_2\}$。将评估值最小的顶点 v_4 取出，将其展开，对 v_4 的孩子顶点 v_5，由于它已经在 *open* 表中，而且其路径 s, v_4, v_5 的长度不会比原来的路径长度更短，因此不需要更新。同时，将顶点 v_4 放入 *closed* 表中，得到图 14.2(d)，此时 *open* = $\{v_3, v_5\}$，*closed* = $\{s, v_1, v_2, v_4\}$。将评

估值最小的顶点 v_3 取出，将其展开，对 v_3 的孩子顶点 v_5，由于它已经在 *open* 表中，而且其路径 s，v_4，v_5 的长度不会比原来的路径长度更短，因此不需要更新。对 v_3 的另一个孩子顶点 t，由于其不在 *open* 表中，也不在 *closed* 表中，因此将其添加到 *open* 表中，得到图 14.2(e)，此时 $open = \{t, v_5\}$，$closed = \{s, v_1, v_2, v_4, v_3\}$。将评估值最小的顶点 t 取出，由于该顶点是目标顶点，A* 搜索算法终止，如图 14.2(f)所示。

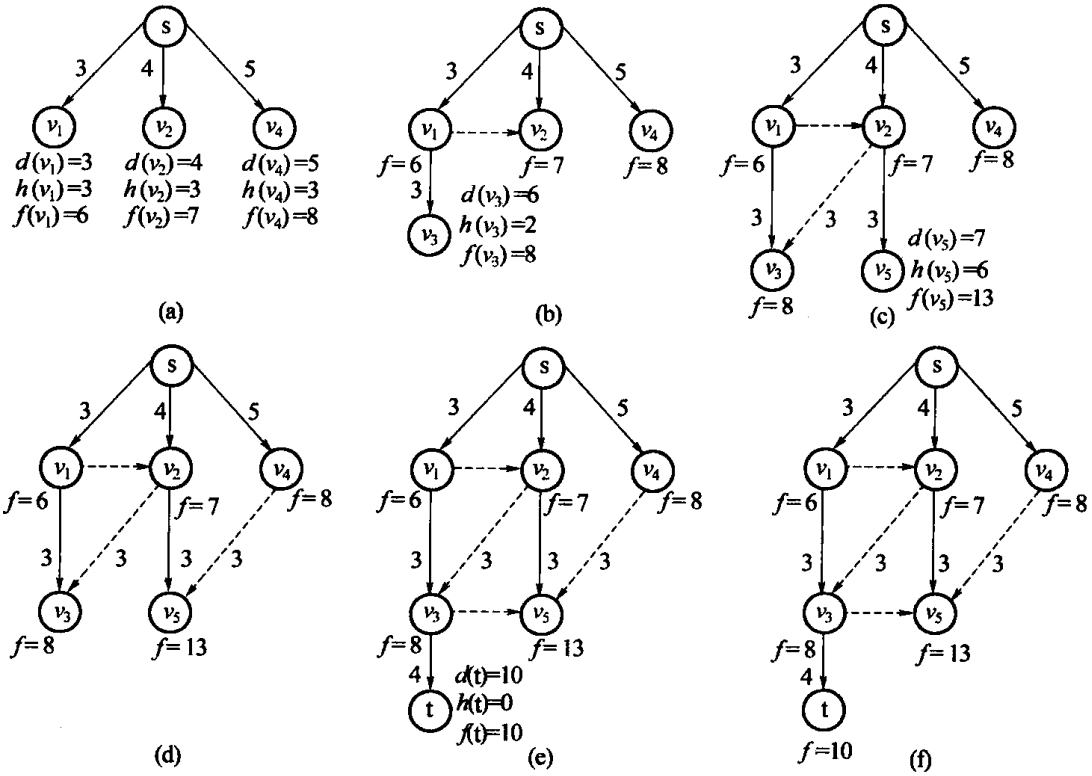


图 14.2

14.2.2 8 数字问题

给定一个 3×3 的棋盘，棋盘除了一格没有数字外，其他 8 个格子都放有一个数字，空格周边格子中的数字可以移到该格中。已知一个目标格局，8 数字问题的目标就是找到一个数字移动序列使初始的棋盘格局变为一个目标格局。

问题描述中，数字的移动可看成空格的左、右、上、下移动。图 14.3 给出了一个初始格局、目标格局以及从初始格局出发，移动一步所能到达的格局。

对于 8 数字问题，我们可以设计出几种评估函数，一种直观的启发式信息，是把当前格局 n 与目标格局相比，不在正确位置的数字个数作为启发式函数值，这里不考虑深度信息，即

$$f(n) = \text{位置不正确的数字个数}$$

随着 $f(n)$ 的减少，搜索越来越逼近目标格局。但是这种评估函数没有利用距离的信息，也容易使搜索陷入死胡同。另外一种是只考虑距离信息，即当前格局

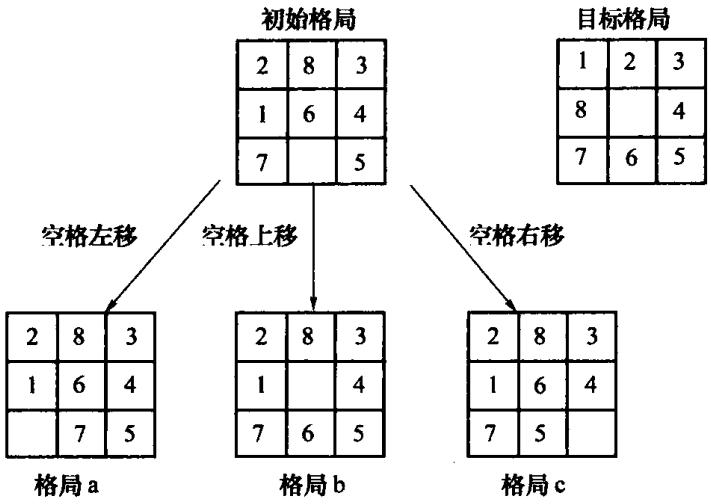


图 14.3

中,不在正确位置的数字到达目标格局必须移动的次数,这种评估函数计算相对比较麻烦。上述两种评估函数,都不是很理想。我们的目标是希望利用有限的启发式信息,更快地找到目标格局。

为了避免由于过分的优化试探而陷入"死胡同",我们可以把上述两种启发式信息结合起来,即在第一种评估函数中,考虑深度信息,即构造出一种新的评估函数

$$f(n) = h(n) + d(n)$$

其中,$h(n)$ = 位置不正确的数字个数(与目标格局相比),$d(n)$ = 搜索图中从初始格局到当前格局 n 的深度。值得注意的是,评估函数$f(n)$满足上述提及的两个约束条件(1)和条件(2),因此,按照上述评估函数进行启发式搜索,可以找到目标格局。

在图 14.3 中,格局 a 的启发式值 $h(a) = 5$,深度值 $d(a) = 1$,因此评估值 $f(a) = 6$。类似地,我们可以计算出 $h(b) = 3, d(b) = 1, f(b) = 4; h(c) = 5, d(c) = 1$,因此评估值$f(c) = 6$。

给定初始格局 s 和目标格局 t,整个 A* 搜索过程如图 14.4 所示。

A* 搜索算法具体执行过程中,*open* 表以及 *closed* 表的变化情况如下:

(1) *open* = {s}, closed = { }

(2) *open* = {b4, a6, c6,}, *closed* = {s}

(3) *open* = {d5, e5, a6, c6, f6}, *closed* = {s, b4}

(4) *open* = {e5, a6, c6, f6, g6, h7}, *closed* = {s, b4, d5}

(5) *open* = {i5, a6, c6, f6, g6, h7, j7}, *closed* = {s, b4, d5, e5}

(6) *open* = {k5, a6, c6, f6, g6, h7, j7}, *closed* = {s, b4, d5, e5, i5}

(7) *open* = {l5, a6, c6, f6, g6, h7, j7, m7}, *closed* = {s, b4, d5, e5, i5, k5}

(8) success, m = t

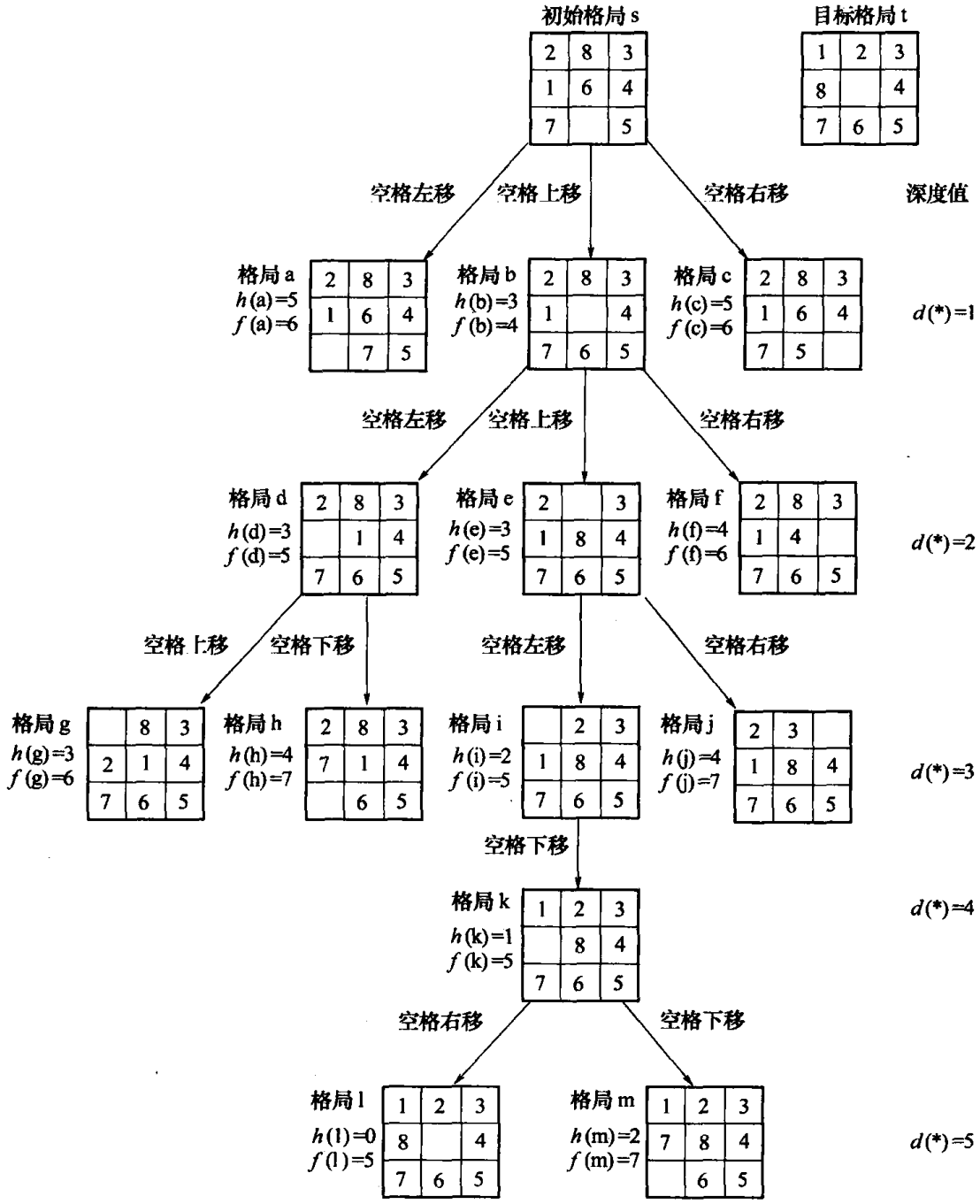


图 14.4

其中，对于 *open* 和 *closed* 表中的格局 ni，n 表示格局，i 表示格局 n 的评估函数值，例如 b4 表示格局 b 的评估函数值为 4。*open* 表中的格局，按照评估值的大小排序，评估值小的被排在表的前面，*open* 表中最左的格局被优先展开。

从上述例子可以看出，评估函数中，启发式函数值能够使当前的搜索朝着目标格局前进，而深度信息却能避免搜索陷入看不到希望的“死胡同”。对新产生的格

局，要检查该格局是否以前访问过，从而避免循环搜索。

从8数字问题的例子，我们也可以看出，搜索路径的深度以及展开的格局数，启发式函数 $h(n)$ 的设计及计算量，均会影响 A* 搜索算法的效率，如何平衡这些因素，提高搜索的效率，是值得考虑的问题，设计合适的评估函数，无疑是其中最关键的地方。

14.3 博弈搜索算法

博弈(Game)是启发式搜索算法最重要的一个应用领域。无论是下棋游戏，还是真正的战争，它们都为启发式算法的设计者提供了展示自己的舞台。设计博弈算法，既有趣又充满挑战。博弈是两方或多方的一种竞争。为方便研究，下面假设博弈的各方清楚地了解自己的目标和利益所在。在竞争中，各方总是力图采用最佳策略以便更好地达到自己的目标和获得想要的利益。

由于博弈双方完全熟悉自己所处的环境，并且知道自己和对方可能的移动方式及其所带来的影响。因此，博弈者为了取胜，必须针对竞争中可能出现的各种情况制定相应的对策，从而做出对自己有利的决策。这些对策常称为博弈策略，一个完整的博弈策略应该包括：

(1) 能够对竞争中可能出现的情况，进行估计和分析。

(2) 针对竞争中可能出现的每一种情况，博弈者可能采取的一系列决策。

许多博弈问题往往包含着多个回合的交锋，因此包含着一系列的决策。为了便于搜索，常常把一系列的决策(解空间)构造成一棵博弈树。博弈树由一系列的节点和分支组成。下面我们均讨论两方博弈，并令博弈一方为 MAX，令一方为 MIN，之所以这样取名，我们将随后可以看到，MAX 总想最大化自己的好处，而 MIN 则总想最小化对方的好处。假设 MAX 先走，双方博弈中，MAX 和 MIN 轮流移动，直到其中一方获得胜利，另一方失败，或者双方出现和局。通常用层数表示博弈树的搜索深度，它可以表示出向前预测的 MAX 和 MIN 交锋的回合数。博弈树的根节点深度为0，即0层。层数为偶数的节点，称为 MAX 节点，轮到 MAX 走；层数为奇数的节点，称为 MIN 节点，轮到 MIN 走。因此，双方博弈树具有如下特点：

(1) 奇层节点，即 MIN 层节点，是 MAX 走后的格局，下一步轮到 MIN 方走。偶层节点是 MIN 方走后的格局(除了初始格局外)，下一步轮到 MAX 方走。

(2) 博弈树的叶节点表示双方交锋的最终结果，即终止格局，表示要么取胜，要么和局。如果奇层节点是叶节点，则 MAX 方胜或者和局。如果偶层节点是叶节点，则 MIN 方胜或者和局。

博弈双方的目的，就是设计出合适的博弈策略，在博弈树的每一个分支中，做出合适的决策，从而使得博弈一方尽快使对手处于叶节点，从而击败对手。

现在的任务是为博弈者找最佳的走法。假设 MAX 先移动，MAX 的最佳首步

如何确定？博弈树中每一层的节点，如何评估？MAX 和 MIN 如何走步？都是博弈搜索需要考虑的非常关键的问题。虽然博弈双方完全熟悉所处的环境，并且知道自己和对方可能的移动方式及其所带来的影响。但是，对节点进行评估，特别是设计出有效的评估函数，还是非常困难的。一般地，设计静态评估函数还是比较容易的。静态评估函数通过对博弈树的叶节点进行评估，然后倒推出其父节点的评估值。这种评估值通常由影响这个值的许多不同特性来决定。通常分析博弈树时，对 MAX 有利的格局，其评估函数值通常为正数；对 MIN 有利的格局，其评估函数值通常为负数，接近零的值表示该格局对 MAX 和 MIN 都一样。

一个最佳首步可以由一个最大最小化过程产生。假设轮到 MAX 从搜索树的 MIN 叶节点中选取，MAX 肯定选择拥有最大值的节点，因为节点的评估函数值越大，对它越有利。因此，MIN 叶节点的 MAX 父节点的倒推值就等于叶节点的最大静态评估值。另一方面，MIN 从 MAX 叶节点中选取时，必然选值最小的节点。即 MAX 叶节点的 MIN 父节点的倒推值等于叶节点静态评估值的最小值。在所有叶节点的父节点的倒推值都确定后，开始倒推上一层。当整棵博弈树中每个节点的倒推值都确定后，MAX 将选择具有最大倒推值的 MIN 节点，作为其走法，而 MIN 会选择有最小倒推值的 MAX 节点，作为其走法。假定 MAX 先走，MAX 将选择有最大倒推值的节点作为它的首步。

整个最大化和最小化过程的有效性基于这样的假设：博弈树内部节点的倒推值比直接从静态评估函数中得到的评估值更可靠。这是因为倒推值基于在博弈树中的预先推算，并且取决于在博弈结束时发生的一些特性，这些值往往更加切合实际。

整个最大最小化过程如下：

```
MinMax(d)
1  if mode = MAX then return Max(d)
2  else return Min(d)
```

其中，Max 函数如下：

```
Max(d)
1     best← - ∞
2     if d≤0 then return Evaluate( )
3     generate successor node Successor[1.. n]
4     for i ← 1 to n do
5          move to Successor[i]
6          v ← Min(d - 1)
7          unmove to Successor[i]
8          if v > best then best ← v
9     return best
```

其中 Max 函数中，*best* 记录节点的最大倒推值，第 2 行判断是否搜索到深度为 0 的节点，如果是，则执行 Evaluate( )，直接计算该节点的评估值。第 3 行产生后继

节点(successor node),第 4 行,对每个后继节点:在第 5 行试着移动到后继节点 Successor[i]。由于其父节点是求最大倒推值,因此在第 6 行,调用 Min($d-1$),执行完后,退回到父节点,如果节点 Successor[i]的倒推值比当前最大的倒推值 *best* 大,则更新 *best*,然后考虑下一个后继节点。Min 函数如下:

```
Min(d)
1  best ← ∞
2  if d≤0 then return Evaluate( )
3  generate successor node Successor[1..n]
4  for i ← 1 to n do
5    move to Successor[i]
6    v←Max(d-1)
7    unmove to Successor[i]
8    if v < best then best←v
9  return best
```

Min 函数的分析,类似 Max 函数,这里就不再介绍。

上述最大化和最小化过程中,博弈树的深度变量 d 直接影响博弈搜索的效率。对于搜索空间比较小的博弈问题,我们可以直接对博弈树的叶节点进行评估,然后进行倒推,逐步获得内部各节点的评估值。当博弈树非常大时,如果从叶节点进行倒推,计算速度将会很慢。因此,确定 d 的值非常关键,必须综合考虑时间以及存储空间的限制。

14.3.1 α 和 β 剪支

最大最小化过程中,需要确定博弈树的深度以及如何计算倒推值。搜索过程需要展开整棵博弈树,博弈树的产生和节点的评估完全分离,只有在博弈树完全产生后才能对节点进行评估。这种分离导致算法繁琐,而且搜索效率低。如果将叶节点的评估、计算倒推值与博弈树的产生同时进行,就可能大量减少搜索空间,提高搜索的效率。

α 和 β 剪支的过程是一种智能搜索技术,它采用深度优先的方式进行搜索,通过剪支,能够大量减少搜索的节点数目。其中,α 表示 MAX 节点的下界,它等于其所有后继节点最终倒推值的最大值。在搜索的过程中,α 的值(包括开始节点)决不会减少。β 表示 MIN 节点的上界,它等于其所有后继节点中最终倒推值的最小值。在搜索的过程中,β(包括开始节点)决不会增加。假定一个 MAX 节点的 α 值为 5,则 MAX 不需要考虑任何其后继 MIN 节点中值小于或等于 5 的其他节点。类似地,如果 MIN 节点的 β 值为 5,则 MIN 不需要考虑任何其后继 MAX 节点中值大于或等于 5 的其他节点。通常,当一个节点的后继节点有了倒推值之后,其倒推值的界限就会被修改,通过记录倒推值的界限达到减少搜索的目的。

如图 14.5 所示,假设有一棵博弈树,只展开 3 层,现在 B 节点及其后继节点

C,D,E,F 都已产生，而 G 节点还未产生，B 节点的倒推值为 -1。由此可知，开始节点 A 的倒推值不小于 -1。也就是说，不管 A 的其他后继节点的倒推值如何，A 的倒推值的下界值 $\alpha=-1$。继续深度优先搜索，直到产生节点 G 及其第一个后继节点 H。如果 H 节点的评估值为 -1。则可知 G 节点的倒推值不会比 -1 大。因此 G 节点倒推值的上界 $\beta=-1$。此时可以看出 G 节点的最终倒推值不会超过 A 节点的值。因此，可以终止 G 节点以下的搜索，达到剪支的目的。

根据以上分析，可以得出如下有用的剪支规则：

（1）α 剪支：当任何 MIN 节点的值不大于父节点 MAX 的值，则可中止该 MIN 节点以下的搜索。该 MIN 节点的最终倒推值即为它的 β 值。

（2）β 剪支：当任何 MAX 节点的值不小于它的父节点 MIN 的值时，则可中止该 MAX 节点以下的搜索，该 MAX 节点的最终倒推值等于它的 α 值。

利用以上规则对博弈树剪支的过程，常称为 α 和 β 剪支。当开始节点的所有后继节点都得到倒推值后，过程终止。最佳首步移动即往具有最大倒推值的后继节点移动。该过程产生的移动方案与简单使用最大最小化过程搜索相同深度得到的方案是相似的。唯一不同的是，由于 α 和 β 剪支可以减少搜索的数目，因此它通常可以更快地找到最佳首步移动。

如图 14.6 给出了一个利用上述规则剪支的例子。图 14.6 中，叶节点评估值给定后，首先进行深度优先搜索，得到 MAX 节点 C 的倒推值 3，从而可以得出 MIN 节点 B 的 $\beta=3$，由于 D 节点的倒推值不会比 5 小，按照 β 剪支，就可以停止 D 剩余分支的搜索。由于 B 节点的倒推值为 3，因此 A 节点的 $\alpha=3$（其倒推值不小于 3），而 MIN 节点 E 和 G 的倒推值分别不大于父节点 MAX 节点 A 的值，因此产生 α 剪支，停止节点 E 和 G 剩余分支的搜索。

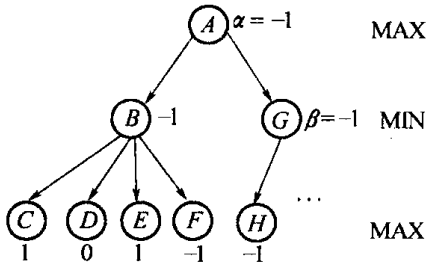


图 14.5

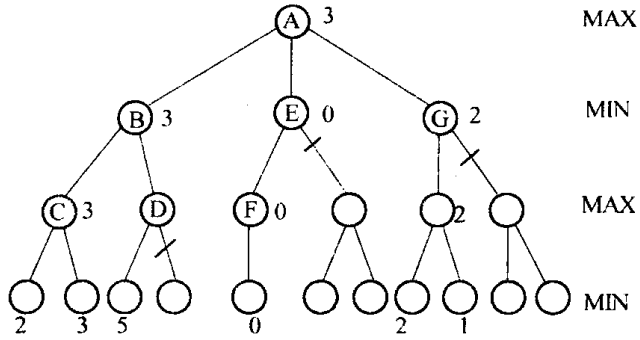


图 14.6

为了完成 α 和 β 剪支，α 和 β 过程通常利用深度优先搜索，至少将部分博弈搜索树产生至最大深度处，然后利用静态评估函数对最大深度处的叶节点进行评估，因此 α 和 β 剪支过程的搜索效率取决于早期 α 和 β 的值与最终倒推值的近似程度。下面，给出几个博弈搜索的例子。

14.3.2 分硬币游戏

给定 n 个硬币，游戏者 MAX 和 MIN 轮流分硬币，直到一方不能再分时，该方便输掉游戏。博弈双方分硬币的规则是：每次分的两堆硬币大小不等，而且任一堆不能为空。

图 14.7 给出了分 7 个硬币的博弈树。从图 14.7 可以看出，MAX 先走，沿着最左边的分支进行，MAX 将会获胜。

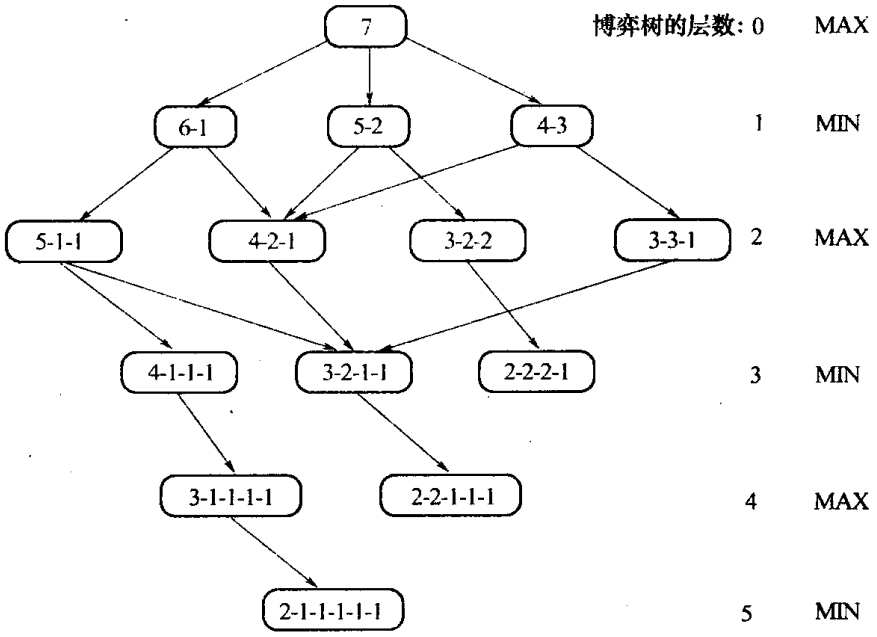


图 14.7

假定 MAX 和 MIN 用同样的信息，显然，MAX 总是试图划分硬币，使得产生的格局对 MIN 来说是最坏的格局。为了实现这种博弈策略，我们对博弈树的叶节点的评估值直接定义如下：如果是 MAX 节点，则定义其值为 1，如果是 MIN 节点，则定义其值为 -1。

按照最大最小化过程：如果父节点是 MAX 节点，则取其子节点中倒推值最大的，如果父节点是 MIN 节点，则取其子节点中倒推值最小的。从叶节点直接往上逐层倒推各内部节点的倒推值，如图 14.8 所示。从图 14.8 可以看出，MAX 先走，沿着粗线走，MIN 将获胜。如果开始走最左边的，则 MAX 获胜。

14.3.3 井字博弈

给定一个井字形的 3 ×3 棋盘，博弈者 MAX 和 MIN 在棋盘中轮流标记，一个标记“X”，一个标记“O”。先用同样标记填满一行、一列或一条对角线的博弈者，将获得胜利。下面假设 MAX 标记“X”，MIN 标记“O”，MAX 先开始标记。

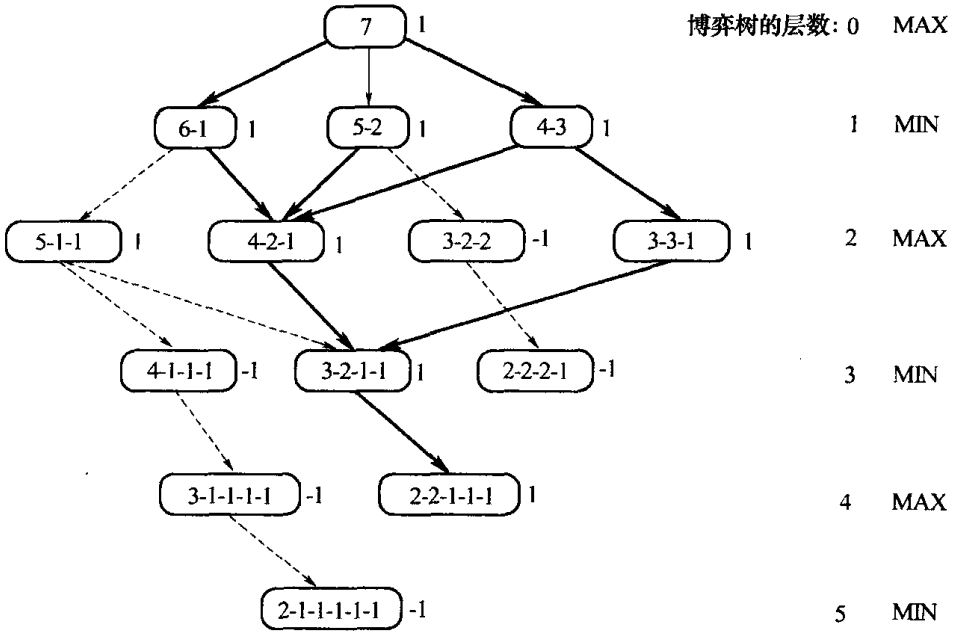


图 14.8

对于这个问题,如果与分硬币游戏一样,构造整棵博弈树,则搜索空间太大。我们可以构造深度为 d(有限的整数)的博弈树,对这棵树进行宽度优先搜索,直到搜索完深度为 d 的节点为止,然后对这些节点代表的格局采用静态评估函数对节点进行评估。

格局 n 的静态评估函数 $f(n)$ 可如下给出:

假如对任何一方,n 都不是获胜格局,则

$f(n)$ = MAX 可能获胜的(行数 + 列数 + 对角线数) - MIN 可能获胜的(行数 + 列数 + 对角线数)

假如对 MAX 来说,n 是获胜格局,则

$$f(n) = \infty$$

假如对 MIN 来说,n 是获胜格局,则

$$f(n) = -\infty$$

其中∞表示一个非常大的正数。图 14.9(a)给出了一个格局 n,MAX 可能获胜的(行数 + 列数 + 对角线数) = 2 + 2 + 2 = 6,如图 14.9(b)所示。MIN 可能获胜的(行数 + 列数 + 对角线数) = 2 + 2 + 1 = 5,如图 14.9(c)所示。因此格局 n 的评估函数值 $f(n) = 6 - 5 = 1$。

图 14.10 演示了井字棋的最大最小化过程。图 14.10 给出了一棵深度为 2 的博弈树。

图 14.10 中,在产生任意一个格局的后继节点时,采用了对称法。即对称的节点中,只留下一个节点。通过排除对称格局,能够大大缩小搜索空间。叶节点的评

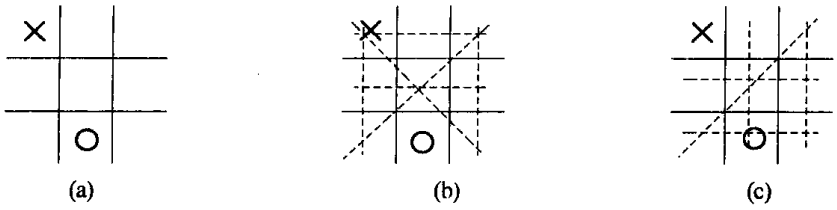


图 14.9

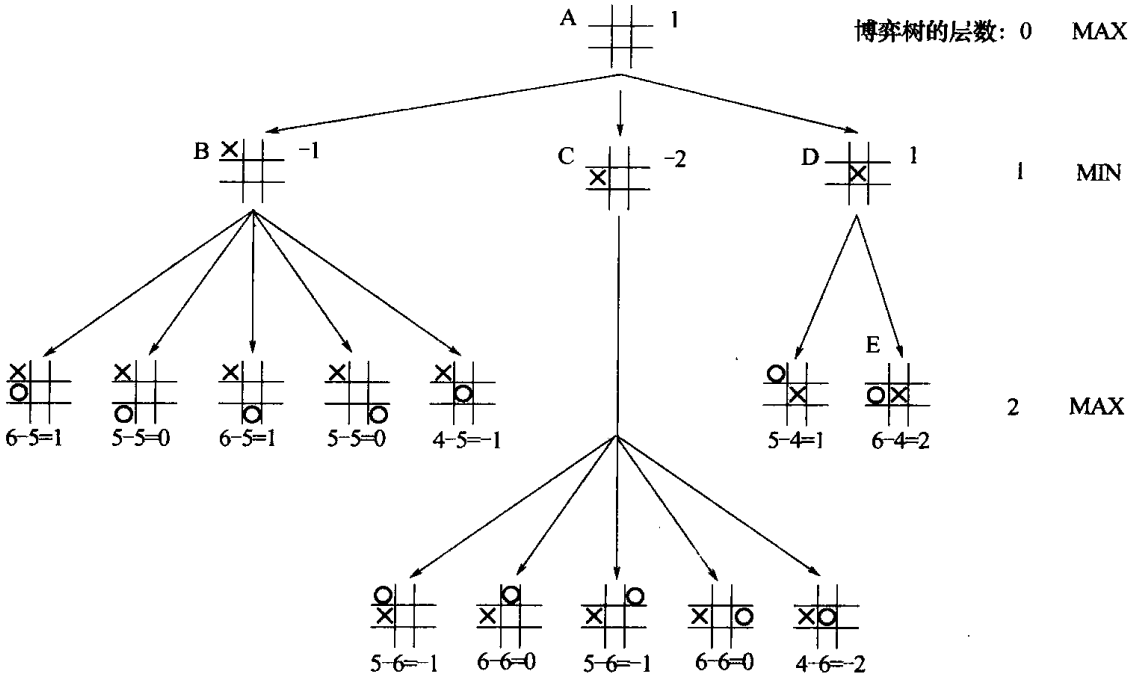


图 14.10

估值被显示在格局的下面,而节点的倒推值显示在格局的右边。节点 A 的倒推值为 1,其后继节点 B,C,D 中,D 节点具有最大的倒推值,因此 MAX 的最佳首步移动是选择到 D 节点。这时轮到 MIN 移动,假设 MIN 在 X 的左边标记"O"(对 MIN 来说,这可能不是一个好的走法,因为它可能没有利用好的搜索策略),如图 14.10 中节点 E 所示。这时为了确定 MAX 的最佳走法,同样产生一棵深度为 2 的博弈树,如图 14.11 所示。

图 14.11 中节点 F 和 H 都有最大倒推值 1,因此是最佳移动,不妨 MAX 走 H。MIN 为了避免失败,马上移动到节点 J。接下来为了给 MAX 找到一个好的走法,同样可以构造一棵深度为 2 的博弈树。如图 14.12 所示。

图 14.12 中一些 MIN 节点的后继节点,例如 P,Q 的评估值为 $-\infty$,因此是 MIN 必胜节点。J 的倒推值为 1,因此 MAX 选择往 K 移动,这是 MAX 避免失败的一步,同时这将是 MAX 必胜的一步。

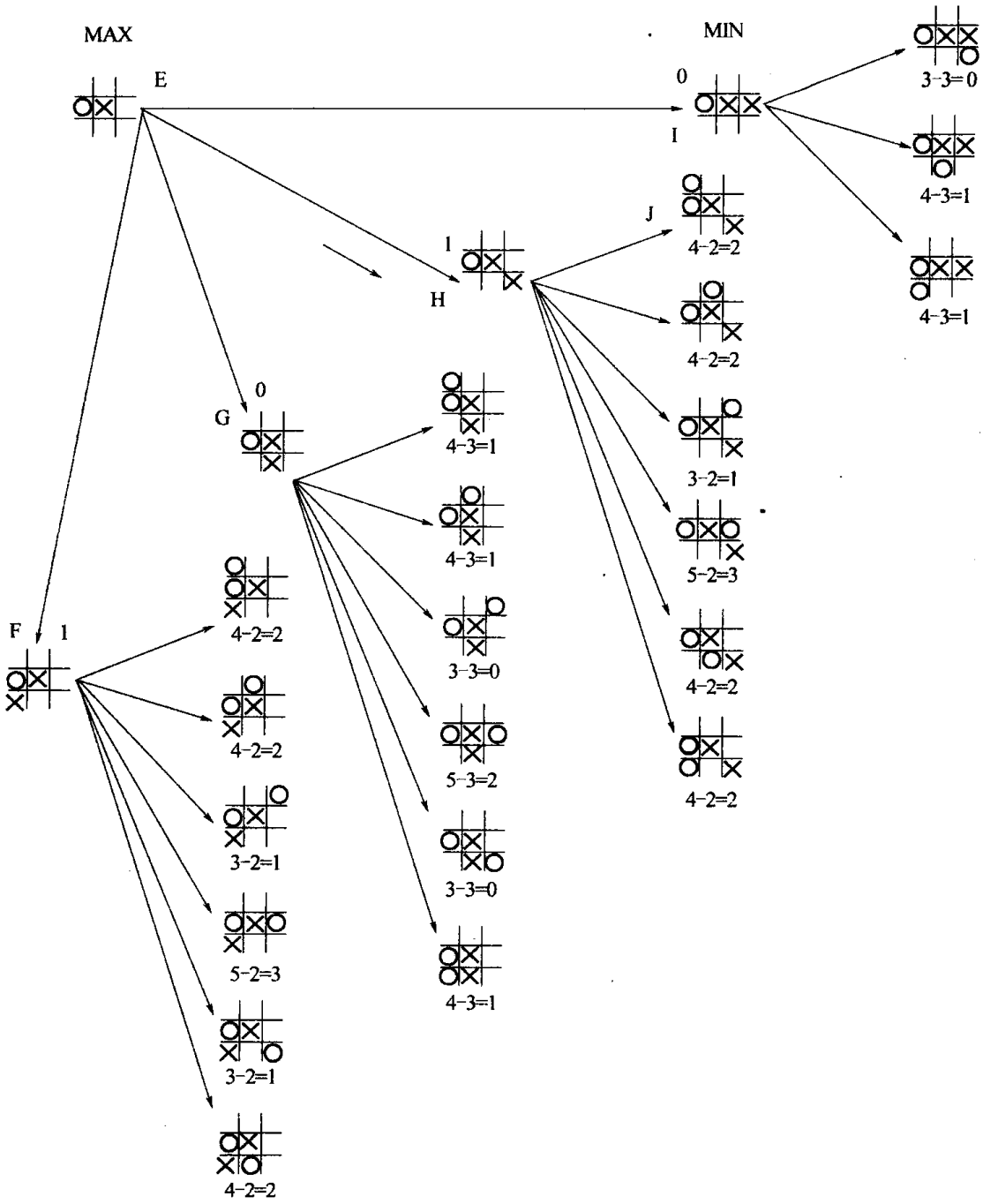


图 14.11

上面的分析可知,如果采用最大最小化搜索,必须展开深度为 d 的博弈树,这种搜索效率是非常低下的,如果将叶节点的评估、计算倒推值与博弈树的产生同时进行,就可能大量减少搜索空间,提高搜索的效率。例如,对图 14.12,如果叶节点 Q 一产生便评估,其值为 $-\infty$,则 MIN 节点肯定选择往 Q 节点移动,因此不用评估其他节点 R,S,T,而且 MAX 的移动也不会受其影响,当博弈树的深度比较深时,这

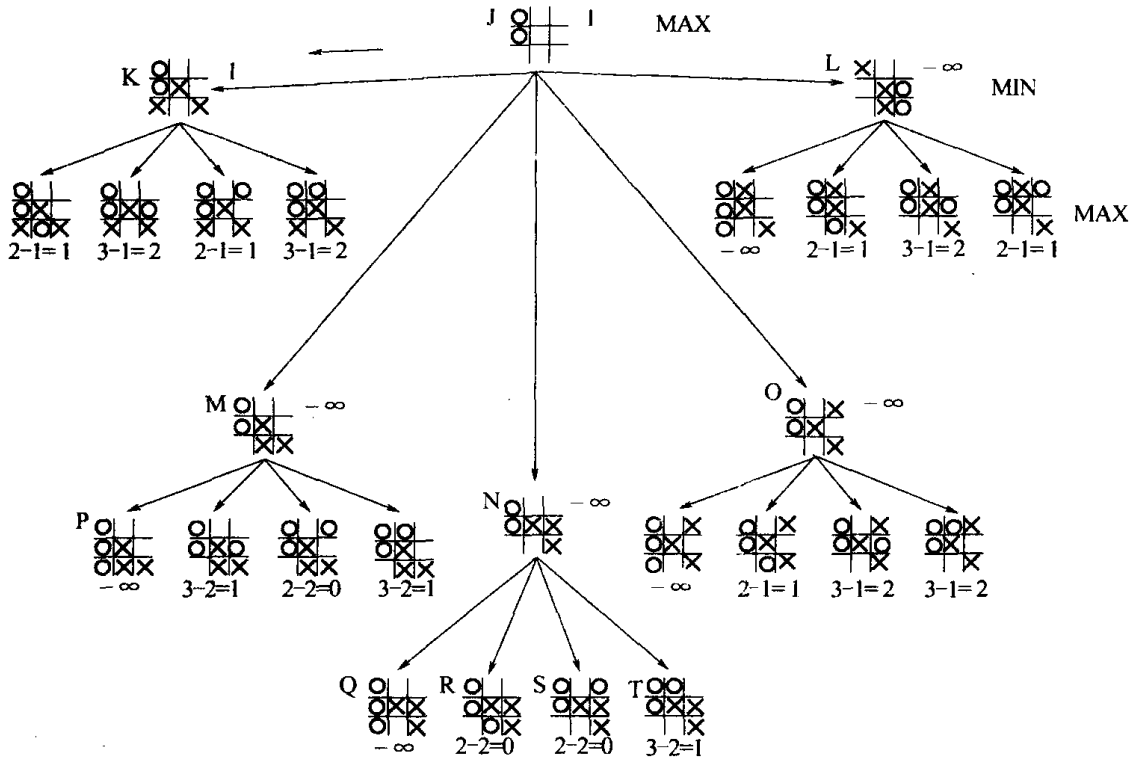


图 14.12

种策略剪掉的分支节点会更多。

下面给出井字棋的带 α 和 β 剪支过程的博弈搜索算法：

```
Ticktacktoe( )
1   mode← MAX
2   s. value ← − ∞
3   while Goal(s) =0 and step < maxstep do
4         if mode = MAX then MAXgo( )
5         else repeat
6                   output " MIN go!"
7                   input information of MAX go
8            until going is feasible
9            s. state ←new state
10           mode ←MAX
11       step ←step + 1
12  if Goal(s) =− ∞ then output "MIN win"
13  else if Goal(s) = ∞ then output "MAX win"
14       else "A draw"
```

Ticktacktoe( )是博弈搜索的主过程，用 *mode* 表示博弈者 MAX 或者 MIN。*step* 表示博弈的步数。第 2 行初始化初始格局的倒推值，第 3 行 Goal 过程表示博弈者

输赢的值，可实现如下：

```
Goal(q)
1    if q. state is win for MAX then return ∞
2    else if q. state is win for MIN then return - ∞
3        else return 0
```

MAXgo()过程表示如何决定 MAX 移动，其详细过程如下：

```
MAXgo()
1    compute successor and save subnode into successor[1..n]
2    value←- ∞
3    for i ← 1 to n do
4        successor[i]. value← Search(successor[i], MIN, value, 0)
5        if value≤successor[i]. value then
6            value←successor[i]. value
7    for i ← 1 to n do
8        if value = successor[i]. value then
9            s← successor[i]
10           mode ← MIN
11           output "MAX has put a chess"
12           break
```

其中，n 表示当前 MAX 后继节点的数目，对每个后继节点计算其倒推值，然后选择具有最大倒推值的节点进行移动。Search(q, *mode*, *oldvalue*, d)是一个带 α 和 β 剪支规则的最大最小化过程。通过后序遍历的方法构造以 q 节点为根的博弈树，详细过程可以描述如下：

```
Search(q, mode, oldvalue, d)
1  if d < maxd and Goal(q) =0 then
2    if mode = MAX then value←- - ∞
3    else value ←∞
4    while q is not completely extend do
5      if ((mode = MAX and value > oldvalue) or (mode = MIN and value < oldvalue)) is
       false then
6            extend node q according to mode value and obtain subnode successor
7            if mode = MAX then
8                temp←Search(successor, MIN, value, d +1)
9                if value≤temp then value←-temp
10           else temp←Search(successor, MAX, value, d +1)
11               if value≥temp then value←-temp
12 else if Goal(q) ≠ 0 then value←-Goal(q)
13     else value←-f (q)
14 ruturn value
```

其中，q 是当前节点，*mode* 表示当前谁走子，*oldvalue* 表示当前节点其父节点的倒推值，*value* 表示当前节点的倒推值。d 表示当前节点的深度，$f(q)$ 给出节点的静态评估函数值。(*mode* = MAX and *value* > *oldvalue*) or (*mode* = MIN and *value* < *oldvalue*) 是剪支条件，如果条件成立，则剪支，否则，继续往下搜索。第 1 行如果当前深度 $d < maxd$ 且 q 不是必胜节点，则继续执行第 2 行 ~ 第 11 行，否则执行第 12 行。第 2 行判断如果当前节点的走子对象 *mode* = MAX，则必须找后继节点的最大倒推值，并初始化 *value* = $-\infty$，否则初始化 *value* = ∞。第 4 行，当节点 q 没有被完全扩展，则继续执行第 5 行。第 5 行，如果剪支条件不成立(is false)，则执行第 6 行，扩展节点 q 以获得后继节点，然后在第 7 行，根据走子对象是 MAX 还是 MIN，决定倒推值的计算以及 q 节点倒推值的更新。如果 *mode* = MAX，则从后继节点中选择大的倒推值，否则从后继节点中选择小的倒推值。第 12 行，如节点 q 为一方获胜的状态，则返回最大值或最小值，如果扩展到了最大深度(*maxd*)节点，则计算该节点的评估函数值。

14.4 小 结

本章内容主要参考文献[17,18,25]，介绍了 A* 搜索算法和博弈搜索算法。关于启发式搜索的一些最新进展，例如迭代加深策略，最佳搜索策略以及 beam 搜索等，有兴趣的读者可以阅读文献[18]。此外，本章介绍的启发式搜索，通常情况下是能够得到问题的最优解。关于启发式算法的理论分析，例如什么情况下，会得到最优解，这里就不再详细叙述，有关内容，可以参考人工智能方面的文献[17,18]。此外，对于不能保证得到最优解的启发式搜索算法，例如模拟退火算法、遗传算法、禁忌搜索等，读者可以参考文献[4]。

习题

14.1 A* 搜索算法一直搜索到一个目标节点被选择扩展才会终止。然而，到达目标节点的一条路径可能在那个节点被选择扩展前就找到了。一旦目标节点被发现，为什么不终止搜索呢？用一个例子说明你的答案。

14.2 考虑 4 皇后问题，假设用下面的问题空间解决这个问题：开始节点由一个空的 4×4 数组表示，后继函数产生新的 4×4 数组，该数组包含一个皇后的附加合法放置，它可在数组的任何位置，预计目标是只要四个皇后在数组中的位置合法即可。

(1) 根据皇后到达目标的剩余步数，为该问题设计一个启发式的函数 h，注意所有的目标节点离开始节点恰好四步！

(2) 使用基于该启发式函数的 A* 搜索算法搜索一个目标节点。画出包含搜索产生的所有 4×4 数组的搜索树，用 g 和 h 标出每一个数组。注意

的是，考虑到对称性，我们只要产生开始节点的三个后继就可以了。

14.3 为什么在博弈搜索算法中搜索总是从当前位置向前而不是从目标向后搜索？

14.4 考虑图 14.13 的博弈树，叶节点的静态评估值都是从第一个博弈者的角度得出的，假设第一个博弈者为 MAX 一方。

(1) 第一个博弈者将怎样移动？

(2) 假设节点按从左到右顺序搜索，考虑 α 和 β 剪支过程，哪些节点不需要搜索？

14.5 对图 14.14：

(1) 执行最大最小化过程。

(2) 分别执行从左到右以及从右到左的 α 和 β 剪支过程，讨论为什么出现不同的剪支？

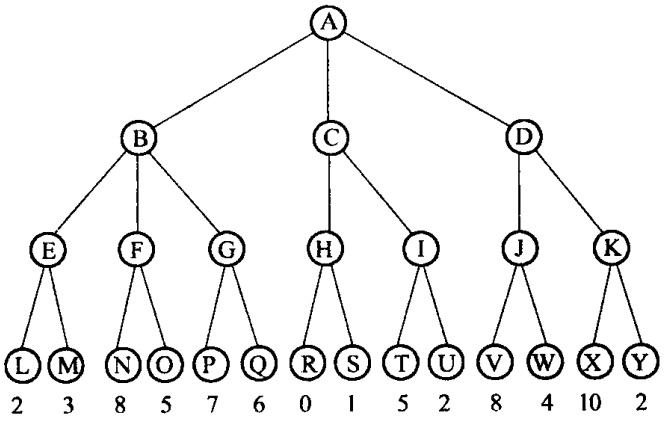


图 14.13

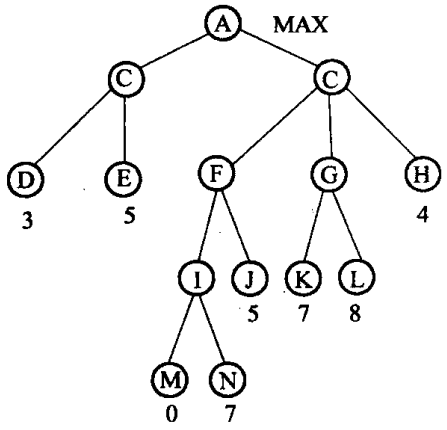


图 14.14

14.6 假设用 α 和 β 剪支过程来决定博弈树中的移动值，而且已断定节点 n 及其后继节点可被截断。假如博弈树实际上是一个图，并且有另一条路可到达节点 n，该节点还能被删除吗？如果能，请给出证明，否则，请给出一个反例。

14.7 对图 14.10，14.11 和 14.12 分别执行 α 和 β 剪支过程，每种情形下，有多少叶节点被剪掉？

14.8 给定一个数字棋盘，如图 14.15 所示。对弈双方各执黑白棋子。某方子落入的那格中的数字为某方所占有。黑方先下。黑方落子后，若黑棋占有的所有数字之和小于白棋占有的所有数字之和，则黑方接着下；若黑棋占有的所有数字之和大于或等于白棋占有的所有数字之和，则轮到白方下。同理，白方落子后，若白棋占有的所有数字之和小于黑棋占有的所有数字之和，则白方接着下；若白棋占有的所有数字之和大于或等于黑棋占有的所有数字之和，则轮到黑方下。双方按照上述规则轮流下子，直到 32 个数均被占为

止。局终所占数字的总和较大的一方为胜方。上述问题能用博弈搜索解决吗？如果能，请设计出相应的博弈搜索算法，如果不能，请设计一个有效算法求解它。

14.9 三三棋是一种流传很广的民间游戏。棋盘由三个不同的正方形，并由8条线段连接在一起，共有24个连接点，如图14.16所示。游戏规则是：一方执黑，一方执白，双方轮流在棋盘24个点上放子，先放子者后走。放子时，某方棋子（没有被压住的棋子）放成三点一线，就叫“三”，并用棋子压住对方一棋子，使之不能成三。双方被压住的棋子，在走子前都拿掉。走子时，棋子只能从一个点走到相邻的没有棋子的点上，一方棋子走成三点一线，就叫“三”，可拿掉对方一子。如果一方无子可走或者棋子无法走成三子一线，就为输。当然，还有更复杂的规则，这里就不再介绍，请按照上述游戏规则设计一个博弈算法。

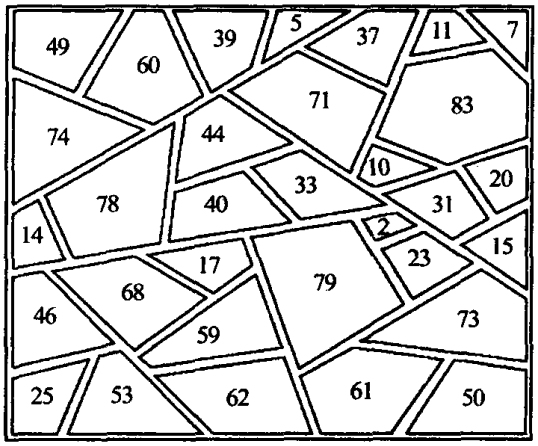


图14.15

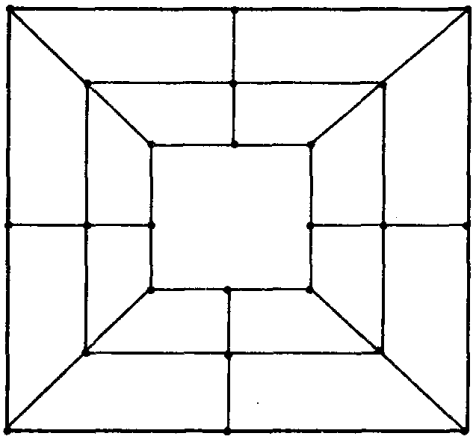


图14.16

实验题

14.10 编程实现井字棋游戏。

14.11 分别用回溯算法、分支限界算法以及A*搜索算法求解旅行商问题，用实验分析方法比较算法的效率。

14.12 完成XOJ1053。

14.13 完成POJ如下题目：1067，1077，1084，1085，1143，1167，1184，1324，1475，1740，3317，3460。

第 15 章　数论

数论(Number Theory)过去一度被认为是好看不中用的纯数学理论。今天,有关数论的算法已经被广泛应用于信息安全领域,其原因在于数论中的许多问题,例如大整数的素性测试以及大整数的分解等,是信息安全的基础和核心。本章将介绍一些基本的数论知识和相关的算法,它们都是数论能实际应用的基础。

为了方便分析数论的一些算法,我们需要介绍一下本章的问题规模与前面各章所述问题规模的不同之处。本章中,问题规模的大小不仅指数的多少(例如排序算法对多少个数进行排序),而且还要考虑数的大小(这个数是 2008 还是 1987602008)。因此,本章将根据表示输入数所需要的位数以及输入中包含的整数个数来衡量问题的规模。这里的位数一般用输入数相应二进制位数的长度来表示。例如,具有整数输入 $a_1, a_2, \cdots, a_k$ 的算法是多项式时间算法意味着其运行时间复杂度能表示成关于 $\lg a_1, \lg a_2, \cdots, \lg a_k$ 的多项式,即它是输入数的二进制位数的多项式。前面在分析算法时间复杂度的时候,一般通过计算基本运算(例如乘法、除法等)的次数来获得。由于要考虑输入数的大小,特别是当输入很大时,基本运算也可能很耗时,因此,衡量一个数论算法的时间复杂度,还需要考虑算法所需要的位运算的次数。例如,用普通的方法进行两个 β 位整数的乘法需要进行 $\Theta(\beta^2)$ 次位运算,而不是以前考虑的 $\Theta(1)$ 次乘运算。下面在分析算法时,一般既要考虑基本运算的次数,也要考虑算法所需要的位运算的次数。

15.1　数论基础

本节将简单地介绍有关整数集合 $\mathbf{Z} = \{\cdots, -2, -1, 0, 1, 2, \cdots\}$ 和自然数集合 $\mathbf{N} = \{0, 1, 2, \cdots\}$ 上的最基本的数论概念。

定义 15.1　给定整数 a, b,对某个整数 k,有 $a = kb$,即 a 是 b 的倍数,则称 b 整除 a,记为 $b \mid a$。如果 b 不能整除 a,则记为 $b \nmid a$。如果 $b \mid a$ 并且 $b \geqslant 0$,则称 b 是 a 的约数。

根据定义 15.1,$b \mid a$ 当且仅当 $-b \mid a$,因此不失一般性,可定义约数为非负整数,因为 a 的任何约数的相应负数同样能整除 a。一个整数 a 的约数最小为 1,最大为 $|a|$。例如,30 的约数有 1,2,3,5,6,10,15 和 30。

定义 15.2　任何整数 a 都可以被 1 和 a 整除,1 和 a 称为整数 a 的平凡约数。a 的非平凡约数称为 a 的因子。

例如,30 的因子有 2,3,5,6,10 和 15。

定义 15.3 对于某个整数 $a>1$,如果它仅有因子 1 和 a,则称 a 为素数(或质数)。

素数具有许多有趣的性质,在数论中具有重要的地位,例如,a 是素数,则 $b|a$ 意味着 $b=1$ 或者 $b=a$。2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53 等都是素数。

定理 15.1 素数有无穷多个。

证明:假设素数只有有限的 n 个,这些素数从小到大依次为 $p_1,p_2,\cdots,p_n$,令 $p=(p_1p_2\cdots p_n)+1$,则 p 显然不能被 $p_1,p_2,\cdots,p_n$ 中的任何一个素数整除,因此 p 也是一个素数,这和素数只有有限的 n 个矛盾,故所证成立。

定义 15.4 整数 $a>1$ 且 a 不是素数,则称 a 为合数。

例如,49 是一个合数(7|49)。由定义 15.3 和 15.4 可得,整数 1,0 以及所有负整数不是素数也不是合数。

定义 15.5 对任意整数 a 和正整数 n,定义 $a \bmod n=a-\lfloor a/n\rfloor n$,$r=a \bmod n$ 称为除法的余数,$q=\lfloor a/n\rfloor$ 称为除法的商。

这里 r 总是集合 $Z_n=\{0,1,2,\cdots,n-1\}$ 中的一个数。下面我们给出一个非常重要的定理,它是数论的基本定理之一。这里不给出证明,请读者思考。

定理 15.2 任给整数 a 和正整数 n,存在唯一的整数 q 和 r,使得 $0<r\leqslant n$,且有 $a=qn+r$。

推论 15.1 任给整数 a 和 n,$n|a$ 当且仅当 $a \bmod n=0$。

定义 15.6 如果 d 是 a 的约数,也是 b 的约数,则 d 是 a 与 b 的公约数。

例如,24 的约数为 1,2,3,4,6,8,12,24,而 30 的约数为 1,2,3,5,6,10,15,30。因此 24 与 30 的公约数为 1,2,3 和 6。

公约数有一些很重要的性质:

(1) 对任意整数 x 和 y,如果 $d|a$ 且 $d|b$,则 $d|(ax+by)$。

(2) 如果 $a|b$,则 $|a|\leqslant|b|$ 或者 $b=0$。进一步地,如果 $a|b$ 且 $b|a$,则 $a=\pm b$。

定义 15.7 任意给定两个不同时为 0 的整数 a 和 b,整除 a 和 b 的最大整数称为 a 和 b 的最大公约数,记为 $\gcd(a,b)$。如果 a 和 b 同时为 0,则定义 $\gcd(0,0)=0$。

例如,$\gcd(24,30)=6$,$\gcd(0,9)=9$,$\gcd(-12,78)=6$。

如果 a 与 b 不同时为 0,则 $1\leqslant\gcd(a,b)\leqslant\min(|a|,|b|)$。最大公约数还有如下基本性质:

(1) $\gcd(a,b)=\gcd(b,a)$

(2) $\gcd(a,b)=\gcd(-a,b)$

(3) $\gcd(a,b)=\gcd(|a|,|b|)$

(4) $\gcd(a,0)=|a|$

对任意的 $k \in \mathbf{Z}, \gcd(a,ka) = |a|$。

定理 15.3 如果 a 和 b 是两个不同时为 0 的任意整数，则 $\gcd(a,b)$ 是 a 和 b 的线性组合集合 $\{ax + by : x,y \in \mathbf{Z}\}$ 中的最小正整数。

证明：设 d 是 a 和 b 的线性组合集合中的最小正整数，并且对 $x,y \in \mathbf{Z}$，有 $d = ax + by$，设 $q = \lfloor a/d \rfloor$，即 q 是一个满足 $a = qd + a \bmod d$ 的整数，因此有

$$\begin{aligned} a \bmod d &= a - qd \\ &= a - q(ax + by) \\ &= a(1 - qx) + b(-qy) \end{aligned}$$

由于 $1 - qx$ 和 $-qy$ 都是整数，因此，$a \bmod d$ 也是 a 和 b 的一种线性组合。由模运算的定义 15.5，有 $a \bmod d < d$。由于 d 是 a 和 b 的线性组合的集合中的最小正整数，故有 $a \bmod d = 0$。由推论 15.1 可得 $d \mid a$。类似可推得 $d \mid b$。因此，d 是 a 和 b 的公约数，从而有 $\gcd(a,b) \geqslant d$。由于 $\gcd(a,b) \mid a$ 和 $\gcd(a,b) \mid b$，由公约数的性质(1)，可得 $\gcd(a,b) \mid (ax + by)$，又由于 d 是 a 和 b 的线性组合，因而有 $\gcd(a,b) \mid d$。又由公约数的性质(2)及 $d > 0$，可得 $\gcd(a,b) \leqslant d$。综上所述，$\gcd(a,b) = d$，因此 $\gcd(a,b)$ 是 a 和 b 的线性组合集合中的最小正整数。

推论 15.2 对任意整数 a 和 b，如果 $d \mid a$ 且 $d \mid b$，则 $d \mid \gcd(a,b)$。

证明可参考习题 15.1 的解答。

定义 15.8 如果两个整数 a 和 b 仅有公约数 1，即如果 $\gcd(a,b) = 1$，则称 a 和 b 互素。

例如 9 和 20 互素，因为 9 的约数为 1,3,9，而 20 的约数为 1,2,4,5,10,20。定理 15.4 说明如果两个整数中每一个数都与一个整数 p 互素，则它们的积与 p 互素。

定理 15.4 对任意整数 a,b 和 p，如果 $\gcd(a,p) = 1$ 且 $\gcd(b,p) = 1$，则有 $\gcd(ab,p) = 1$。

证明：由定理 15.3 可知，存在整数 x,y,x',y'，使得

$$\begin{aligned} ax + py &= 1 \\ bx' + py' &= 1 \end{aligned}$$

将上面两个等式两边相乘，可得

$$ab(xx') + p(ybx' + y'ax + pyy') = 1$$

由于 xx' 和 $ybx' + y'ax + pyy'$ 是整数，因此 1 是满足 ab 和 p 的线性组合的最小正整数，由定理 15.3 可得结论。

下面我们给出一个重要的结论，它是整数分解的基础。

定理 15.5 任给素数 p、以及整数 a,b，如果 $p \mid ab$，则 $p \mid a$ 或者 $p \mid b$，或者 $p \mid a$ 且 $p \mid b$。

证明：用反证法证明。假设结论不成立，即 $p \nmid a$ 且 $p \nmid b$。由于 p 的约数只有 1 和 p，而且又假设 p 既不能整除 a 也不能整除 b，因此有 $\gcd(a,p) = 1$ 且 $\gcd(b,p) = 1$。根据定理 15.4 可知，$\gcd(ab,p) = 1$，这就与已知 $p \mid ab$ 矛盾，故所证成立。

由定理 15.5 可以推出，一个整数分解为素数因子的乘积的分解式是唯一的，即有如下定理。

定理 15.6 任意的整数 a 能唯一表示成下述乘积的形式

$$a = p_1^{e_1} \times p_2^{e_2} \times \cdots \times p_r^{e_r}$$

其中，$p_1, p_2, \cdots, p_r$ 是 a 的素数因子且 $p_1 \leqslant p_2 \leqslant \cdots \leqslant p_r$，$e_1, e_2, \cdots, e_r$ 是非负整数，$r \geqslant 1$。

例如，$a = 600$ 可以唯一分解成 $a = 600 = 2^3 \times 3 \times 5^2$。

定理 15.6 非常重要，许多计算理论中的结论都可以由这个定理推出，这里就不再详细叙述。

定义 15.9 给定关于正整数 n 的一个算术函数 $f(n)$，如果 $f(1) = 1$ 且当 a 和 b 互素时，有 $f(ab) = f(a)f(b)$，则函数 $f(n)$ 称为积性函数。若某算术函数 $f(n)$，如果 $f(1) = 1$ 且 $f(ab) = f(a)f(b)$，则函数 $f(n)$ 称为完全积性函数。

积性函数有下列性质：如果 a 能分解成素数因子的乘积，即 $a = p_1^{e_1} \times p_2^{e_2} \times \cdots \times p_r^{e_r}$，令 $f(a) = a$，则有 $f(a) = f(p_1^{e_1}) \times f(p_2^{e_2}) \times \cdots \times f(p_r^{e_r})$。如果 f 为积性函数且 $f(p_i^{n_i}) = f(p_i)^{n_i}$，则 f 为完全积性函数。

前面介绍的最大公约数 $f(a) = \gcd(a, c)$，其中 c 为常数，以及后面介绍的欧拉函数，均属于积性函数。

15.2 最大公约数

本节我们介绍能有效计算两个整数最大公约数的欧几里得算法（Euclid's algorithm），该算法也称辗转相除法。由最大公约数的性质(3)，我们只需考虑求两个非负整数最大公约数的算法即可。

对于求两个整数的最大公约数这个问题，可以利用定理 15.6，分别将两个整数分解，然后求其最大公约数。由于整数分解本身就是一个很难解决的问题，显然这种办法不可行。下面我们给出一个有用的结论。

定理 15.7 对于任意的非负整数 a 和正整数 b，有 $\gcd(a, b) = \gcd(b, a \bmod b)$。

证明：如果能证明 $\gcd(a, b)$ 和 $\gcd(b, a \bmod b)$ 能互相整除，则根据公约数的性质(2)，可证得结论。

首先我们证明 $\gcd(a, b) \mid \gcd(b, a \bmod b)$。令 $d = \gcd(a, b)$，则有 $d \mid a$ 且 $d \mid b$。由于 $a \bmod b = a - \lfloor a/b \rfloor b$，因而 $a \bmod b$ 是关于 a 和 b 的线性组合，按照公约数的性质(1)，可得 $d \mid a \bmod b$。又因为 $d \mid b$，由推论 15.2 可得 $d \mid \gcd(b, a \bmod b)$，故有 $\gcd(a, b) \mid \gcd(b, a \bmod b)$。

类似地，可证明 $\gcd(b, a \bmod b) \mid \gcd(a, b)$，令 $d = \gcd(b, a \bmod b)$，则有 $d \mid b$ 且 $d \mid a \bmod b$。由于 $a = \lfloor a/b \rfloor b + a \bmod b$，因而 a 是关于 b 和 $a \bmod b$ 的线性组合，

按照公约数的性质(1),可得,$d|a$。又因为 $d|b$,由推论 15.2 可得 $d|\gcd(a,b)$,故结论成立。

上述结果表明,$\gcd(a,b)$ 和 $\gcd(b,a \bmod b)$ 能互相整除,按照公约数的性质(2),故定理成立。

由定理 15.7,我们可以得到如下欧几里得算法:

```
Euclid(a,b)
1  if b=0 then return a
2  else return Euclid(b,a mod b)
```

例如求 30 和 21 的最大公约数,则 Euclid 算法的运行过程为:Euclid(30,21)= Euclid(21,9) = Euclid(9,3) = Euclid(3,0) =3。

Euclid(a,b)算法是一个递归算法,因此其时间复杂度与递归调用的次数成正比。为了方便分析该算法的时间复杂度,不失一般性,我们不妨假设 $a>b\geqslant 0$。也就是 Euclid(a,b)算法中,第一个参数总是大于第二个参数。设 F_i 是第 i 次调用 Euclid(a,b)算法中的第一个参数,则第二个参数为 F_{i+1},这样我们得到一个参数序列 $F_1,F_2,\cdots,F_n$,并且有

$$F_{i+2} = F_i \bmod F_{i+1}$$

由于序列 F_i 满足 $F_{i+2}=F_i \bmod F_{i+1}$,因此有 $F_{i+1}<F_i$,这表明序列 F_i 是一个严格递减序列。

事实上,这个序列 F_i 递减得更快,即有

$$F_{i+2} < \frac{1}{2}F_i$$

证明可参考习题 15.5 的解答。

由此可见,每隔一次调用 Euclid(a,b)算法,第一个参数的值将会减少一半,当 $F_n=0$ 时,递归终止。按照第 4 章递归树的分析方法,可得如下结论。

定理 15.8 给定两个整数 a,b,且 $a>b\geqslant 0$,则 Euclid(a,b)算法的时间复杂度为 $O(\lg a)$。

下面给出一个修改的欧几里得算法,该算法不仅能计算出最大公约数 d,而且能计算出额外的有用信息,即在算法结束时,能获得两个整数(可能为 0 或负数)x 和 y 使等式 $d=\gcd(a,b)=ax+by$ 成立。当 $b=0$ 时,$d=a,x=1,y=0$,否则,由定理 15.3,有

$$\gcd(b,a \bmod b) = bx' + (a \bmod b)y'$$

根据定理 15.7 有 $\gcd(a,b)=\gcd(b,a \bmod b)$,从而有

$$ax + by = bx' + (a \bmod b)y'$$

由于 $a \bmod b = a - \lfloor a/b \rfloor b$,因此 $ax+by=bx'+(a \bmod b)y'=bx'+(a-\lfloor a/b \rfloor b)y'$,整理可得

$$ax + by = ay' + b(x' - \lfloor a/b \rfloor y')$$

由恒等式可得

$$x = y'$$
$$y = x' - \lfloor a/b \rfloor y'$$

详细的算法可以描述如下：

ExtendedEuclid(a, b)
1 **if** $b = 0$ **then return** (a, 1, 0)
2 (d', x', y') ← ExtendedEuclid(b, a mod b)
3 (d, x, y) ← (d', y', $x' - \lfloor a/b \rfloor y'$)
4 **return**(d, x, y)

ExtendedEuclid(a, b)算法第 1 行，如果 $b=0$，则返回 $d=a, x=1, y=0$，如果 $b\neq 0$，则执行第 2 行～第 3 行，最后返回满足 $d=\gcd(a,b)=ax+by$ 的三元组(d, x, y)。

ExtendedEuclid(a, b)算法与 Euclid(a, b)算法的时间复杂度在渐近意义下相同，差别仅在于常数因子，而由此得出的 x 和 y 却对计算模乘法的逆非常有用，这将在公钥密码系统中用到。

图 15.1 给出一个例子，图中每一行代表一层递归，调用 ExtendedEuclid(99, 78)，算法最终返回三元组(d, x, y) = (3, −11, 14)。

a	b	$\lfloor a/b \rfloor$	d	x	y
99	78	1	3	−11	14
78	21	3	3	3	−11
21	15	1	3	−2	3
15	6	2	3	1	−2
6	3	2	3	0	1
3	0		3	1	0

图 15.1

ExtendedEuclid(a, b)算法是一个递归算法，为了提高算法的效率，读者可以将其转化为迭代算法，见习题 15.6。

15.3 同　余

定义 15.10　任给整数 a, b 和 n，如果(a mod n) = (b mod n)，则称 a 和 b 对模 n 同余，记为 $a\equiv b \pmod n$。

例如，$15\equiv 10 \pmod 5$，$16\equiv -4 \pmod 5$。

由定义 15.10，可知同余具有下述性质：

(1) 自反性　$a\equiv a \pmod n$。

(2) 对称性　若 $a\equiv b \pmod n$，则 $b\equiv a \pmod n$。

(3) 传递性　若 $a\equiv b \pmod n$，$b\equiv c \pmod n$，则 $a\equiv c \pmod n$。

定理 15.9　如果 $a\equiv b \pmod n$，则 $n \mid (b-a)$。

证明可参考习题 15.7 的解答。

定理 15.10　若 $ac\equiv bc \pmod n$ 且 $\gcd(c,n)=d$，则

$$a \equiv b(\bmod (n/d))$$

证明：因为 $ac \equiv bc(\bmod n)$，由定理 15.9 可知 $n \mid (b-a)c$，由假设可推得 $n/d \mid (b-a)c/d)$。又因为 $\gcd(c,n)=d$，故有 $\gcd(c/d,n/d)=1$，因此 $n/d \mid (b-a)$，由定理 15.9 可推得结论 $a \equiv b(\bmod (n/d))$。

显然，由定理 15.10 可得如下推论。

推论 15.3 若 $ac \equiv bc(\bmod n)$ 且 $\gcd(c,n)=1$，则

$$a \equiv b(\bmod n)$$

令所有与 a 模 n 同余的整数所组成的集合为

$$[a]_n = \{a + kn : k \in \mathbf{Z}\}$$

它表示集合里每个整数模 n 的余数为 a。例如，$[3]_7 = \{\cdots, -11, -4, 3, 10, 17, \cdots\}$，该集合的其他表示有 $[-4]_7$ 和 $[10]_7$。因此 $a \in [b]_n$ 与 $a \equiv b(\bmod n)$ 是一样的。

按照整数模 n 所得的余数可以把整数分成 n 个等价类 $[0]_n, [1]_n, \cdots, [n-1]_n$，这 n 个等价类称作模 n 的剩余（同余）类。

定理 15.11 设 n 为一正整数，$[0]_n, [1]_n, \cdots, [n-1]_n$ 是模 n 的剩余类，则

（1）任一整数包含在一个 $[i]_n$，其中 $0 \leqslant i \leqslant n-1$。

（2）$[i]_n = [j]_n$ 的充要条件是 $i \equiv j(\bmod n)$。

（3）$[i]_n$ 与 $[j]_n$ 的交集为空集的充要条件是 i 和 j 模 n 不同余。

证明略，请读者思考。

定义 15.11 在模 n 的剩余类 $[0]_n, [1]_n, \cdots, [n-1]_n$ 中各取一数 $a_i \in [i]_n$，$0 \leqslant i \leqslant n-1$，此 n 个数 $a_0, a_1, \cdots, a_{n-1}$ 称为模 n 的一个完全剩余系。

由定义 15.11 和定理 15.11，可得以下定理。

定理 15.12 n 个整数 $a_0, a_1, \cdots, a_{n-1}$ 为模 n 的一个完全剩余系，当且仅当它们两两对模 n 不同余。

最常用的完全剩余系 $0, 1, \cdots, n-1$，称为模 n 的一个最小非负完全剩余系，记为 $Z_n = \{0, 1, \cdots, n-1\}$。

例如，以下是几个模 10 的完全剩余系：$0, 1, \cdots, 9$；$1, 2, \cdots, 10$；$-9, -8, \cdots, -1, 0$，其中 $0, 1, \cdots, 9$ 为模 10 的一个最小非负完全剩余系。

定义 15.12 小于或等于 n 且与 n 互素的正整数个数，称为欧拉（Euler）函数，记为 $\varphi(n)$。

由定义 15.12 可知 $\varphi(1)=1, \varphi(2)=1, \varphi(3)=2, \varphi(6)=2$。

定义 15.13 若模 n 剩余类中的数与 n 互素，则称该剩余类与模 n 互素，在与模 n 互素的所有剩余类中，各取一数所组成的集合，称为模 n 的一个缩剩余系。

例如，模 6 的缩剩余系为 1 和 5。

当 n 为素数时，$\varphi(n) = n-1$，并且 $1, 2, \cdots, n-1$ 是模 n 的缩剩余系。例如 $\varphi(7)=6$，而 $1, 2, \cdots, 6$ 是模 7 的缩剩余系。

由上面定义 15.12 和定义 15.13,可推出下面两个定理。

定理 15.13 若 $a_1,a_2,\cdots,a_{\varphi(n)}$ 是 $\varphi(n)$ 个与 n 互素的整数,$a_1,a_2,\cdots,a_{\varphi(n)}$ 是模 n 的一个缩剩余系,当且仅当它们两两对模 n 不同余。

定理 15.14 若 $\gcd(b,n)=1,a_1,a_2,\cdots,a_{\varphi(n)}$ 是模 n 的一个缩剩余系,则 ba_1,$ba_2,\cdots,ba_{\varphi(n)}$ 也是模 n 的一个缩剩余系。

证明:只需证明 $ba_1,ba_2,\cdots,ba_{\varphi(n)}$ 两两互不相同,且都与 n 互素即可。假设 $ba_i\equiv ba_j(\bmod n)$,其中 $1\leqslant i,j\leqslant\varphi(n)$。由于 $\gcd(b,n)=1$,根据推论 15.3,有 $a_i\equiv a_j(\bmod n)$,这与 $a_1,a_2,\cdots,a_{\varphi(n)}$ 是模 n 的一个缩剩余系矛盾,故假设不成立,即 $ba_1,ba_2,\cdots,ba_{\varphi(n)}$ 两两互不相同。

假设 p 是 n 和某 ba_i 的公共素因子,其中 $1\leqslant i\leqslant\varphi(n)$,因 p 是素数,故 $p\mid b$ 或 $p\mid a_i$。因此,p 是 b 和 n 的公因子,或是 a_i 和 n 的公因子,这与 $\gcd(b,n)=\gcd(a_i,n)=1$ 矛盾。因此有 $\gcd(ba_i,n)=1,1\leqslant i\leqslant\varphi(n)$,即 ba_i 与 n 互素。

由定理 15.14 可得著名的欧拉定理。

定理 15.15 设 $n>2$,且 $\gcd(b,n)=1$ 则 $b^{\varphi(n)}\equiv 1(\bmod n)$。

证明:设 $a_1,a_2,\cdots,a_{\varphi(n)}$ 是模 n 的一个缩剩余系,则由定理 15.14 知,ba_1,$ba_2,\cdots,ba_{\varphi(n)}$ 也是模 n 的一个缩剩余系,故

$$(ba_1)(ba_2)\cdots(ba_{\varphi(n)})\equiv a_1a_2\cdots a_{\varphi(n)}(\bmod n)$$

即

$$b^{\varphi(n)}a_1a_2\cdots a_{\varphi(n)}\equiv a_1a_2\cdots a_{\varphi(n)}(\bmod n)$$

由于 $\gcd(a_i,n)=1,1\leqslant i\leqslant\varphi(n)$,根据推论 15.3 有 $b^{\varphi(n)}\equiv 1(\bmod n)$。

注意到,当 n 为素数时,可以得到著名的费马定理。

定理 15.16 若 n 为素数,则 $b^{n-1}\equiv 1(\bmod n)$。

证明可参考习题 15.12 的解答。

这个结论在大整数的素性测试中将用到。

下面给出同余式及其解的概念。

定义 15.14 设 $f(x)=\sum_{i=0}^{m}a_ix^i$,其中 $m>0,a_i$ 是整数,又设 n 是正整数,则

$$f(x)\equiv 0(\bmod n)\tag{15.1}$$

称为模 n 的同余方程。若 a_m 不能被 n 整除,则 m 称为方程(15.1)的次数。若 x_0 满足 $f(x_0)\equiv 0(\bmod n)$,则 $x\equiv x_0(\bmod n)$ 称为(15.1)的解。不同解是指互不同余的解。

由定义 15.14 可知,要求方程(15.1)的解,只要逐个把 $0,1,\cdots,n-1$ 代入(15.1)中进行验算即可。但当 n 很大时,巨大的计算量使得求解非常困难。

下面就来讨论一次同余式的求解问题。

定理 15.17 设 $\gcd(a,n)=1,n>0$,则

$$ax \equiv b(\bmod n) \tag{15.2}$$

恰有唯一解，且 $x \equiv ba^{\varphi(n)-1}(\bmod n)$。

证明：因为 $0,1,\cdots,n-1$ 是模 n 的一个完全剩余系，且 $\gcd(a,n)=1$，故 $0a,1a,\cdots,(n-1)a$ 也是模 n 的一个完全剩余系，所以其中恰有一整数 j，满足 $aj \equiv b(\bmod n)$，因此 $x \equiv j(\bmod n)$ 是方程(15.2)的唯一解。

因为 $a^{\varphi(n)} \equiv 1(\bmod n)$，所以有 $a^{\varphi(n)}b \equiv b(\bmod n)$，即 $aa^{\varphi(n)-1}b \equiv b(\bmod n)$，故 $x \equiv ba^{\varphi(n)-1}(\bmod n)$ 是(15.2)的唯一解。

定理 15.18 设 $\gcd(a,n)=d, n>0$，则方程(15.2)有解当且仅当 $d|b$。

证明：若方程(15.2)有解，则由 $d|a$ 以及 $d|n$，可以推出 $d|b$。反过来，若 $d|b$，则因 $\gcd(a/d,n/d)=1$，由定理 5.17 知，$(a/d)x \equiv b/d(\bmod n/d)$ 有唯一解，即方程(15.2)有解。

定理 15.19 设 $\gcd(a,n)=d, n>0$，且 $d|b$，则方程(15.2)恰有 d 个解。

证明：由定理 15.18 知方程(15.2)有解。显然，c 为方程(15.2)的解当且仅当 c 也是

$$(a/d)x \equiv b/d(\bmod n/d) \tag{15.3}$$

的解。令 c 满足方程(15.3)，若 $x \equiv c(\bmod n/d)$ 是方程(15.3)的唯一解，则有 d 个模 n 互不同余的整数 $c, c+n/d, c+2n/d, \cdots, c+(d-1)n/d$ 是方程(15.2)的解。由于 $ac \equiv b(\bmod n)$ 且显然有 $ain/d \equiv 0(\bmod n)$，$i=0,1,2,\cdots,d-1$，故 $a(c+in/d) \equiv b(\bmod n)$，于是 $x \equiv (c+in/d)(\bmod n)$ 是方程(15.2)的解。又 $c, c+n/d, c+2n/d, \cdots, c+(d-1)n/d$ 两两模 n 互不同余，且对于其他解，均可以在以上 d 个解中找到一个数与之模 n 同余，即方程(15.2)只有 d 个解。

根据定理 15.19，我们可以得到求解方程(15.2)的算法如下：

```
ModularLinearEquation(a, b, n)
1   (d, x', y') ← ExtendEuclid(a, n)
2   if d|b then
3       c ← x'(b/d) mod n
4       for i ← 0 to d − 1 do
5           print (c + i(n/d)) mod n
6   else print "no solutions"
```

例如求方程 $14x \equiv 30(\bmod 100)$ 的解，其中 $a=14, b=30, n=100$。执行第 1 行，可得 $(d,x',y')=(2,-7,1)$。因为 $2|30$，执行第 3 行可得 $c=(-7)(15) \bmod 100=95$，执行第 4 行～第 5 行可得两个解 95 和 45。

由于 ExtendEuclid(a, n) 算法的时间复杂度为 $O(\lg n)$，而第 4 行执行的次数取决于 d，即 $\gcd(a,n)=d$，因此 ModularLinearEquation(a, b, n) 算法的时间复杂度为 $O(\lg n+\gcd(a,n))$。

下面介绍一次同余方程组的解法。关于解同余方程组的问题，我国古代数学家孙子发明了中外驰名的中国剩余定理(Chinese Remainder Theorem)。

定理 15.20 设 $m_1, m_2, \cdots, m_k$ 是两两互素的 k 个正整数，$k>2$，并且 $m=m_1m_2\cdots m_k$，$m=m_iM_i$，$i=1,2,\cdots,k$，则对任意的整数 $b_1, b_2, \cdots, b_k$，同余方程组

$$\begin{cases} x \equiv b_1 (\bmod m_1) \\ x \equiv b_2 (\bmod m_2) \\ \cdots \\ x \equiv b_k (\bmod m_k) \end{cases} \tag{15.4}$$

有唯一解 $x \equiv M_1'M_1b_1 + M_2'M_2b_2 + \cdots + M_k'M_kb_k (\bmod m)$，其中 $M_i'M_i \equiv 1 (\bmod m_i)$，$i=1,2,\cdots,k$。

证明：因为 $m_1, m_2, \cdots, m_k$ 是两两互素的正整数，因此当 $i \neq j$，有 $\gcd(m_i, m_j)=1$，因而有 $\gcd(m_i, M_i)=1$，于是对每一个 M_i，存在 M_i'，使得

$$M_i'M_i \equiv 1 (\bmod m_i), i = 1,2,\cdots,k \tag{15.5}$$

另一方面，当 $i \neq j$，由 $\gcd(m_i, m_j)=1$ 且 $m=m_iM_i$，$i=1,2,\cdots,k$ 可得 $m_i \mid M_j$，因此有

$$b_jM_j'M_j \equiv 1 (\bmod m) \tag{15.6}$$

由式(15.5)和式(15.6)，有

$$\sum_{j=1}^{k} b_jM_j'M_j \equiv b_iM_i'M_i \equiv b_i (\bmod m_i), 1 \leqslant i \leqslant k \tag{15.7}$$

因此式(15.7)为同余方程组(15.4)的解。

现证明解的唯一性，假设 x 和 y 是同余方程组(15.4)的两个解，则有

$$x \equiv y \equiv b_i (\bmod m_i), 1 \leqslant i \leqslant k$$

因为 $m_1, m_2, \cdots, m_k$ 是两两互素的正整数，故 $m \mid (x-y)$，即 $x \equiv y (\bmod m)$，因此解是唯一的。

例如，有某部队，若列成五行纵队，则末行一人；若列成六行纵队，则末行五人；若列成七行纵队，则末行四人；若成十一行纵队，则末行十人，求兵数。

设 x 是所求兵数，则依题意有

$$x \equiv 1 (\bmod 5), x \equiv 5 (\bmod 6), x \equiv 4 (\bmod 7), x \equiv 10 (\bmod 11)$$

令 $m_1=5, m_2=6, m_3=7, m_4=11, b_1=1, b_2=5, b_3=4, b_4=10$，于是 $m=m_1m_2m_3m_4=5\times6\times7\times11=2310$，$M_1=2310/5=62$，$M_2=385$，$M_3=330$，$M_4=210$，分别求解同余方程

$$M_i'M_i \equiv 1 (\bmod m_i), i = 1,2,\cdots,4$$

可得 $M_1'=3$，$M_2'=1$，$M_3'=1$，$M_4'=1$，故解为

$$\begin{aligned} x \equiv{}& 1\times3\times462 + 1\times5\times385 + 1\times4\times330 + \\ & 1\times10\times210 \equiv 6731 \equiv 2100 (\bmod 2310) \end{aligned}$$

由中国剩余定理,可得如下推论。

推论 15.4 设 $m_1, m_2, \cdots, m_k$ 是两两互素的 k 个正整数,并且 $m = m_1 m_2 \cdots m_k$,对任意的整数 x 和 a,有 $x \equiv a \pmod{m_i}$,$1 \leqslant i \leqslant k$ 当且仅当 $x \equiv a \pmod{m}$。

15.4 模幂运算

在数论算法中,例如素数的测试问题,经常碰到模幂(Modular Exponentiation)运算 $a^b \bmod n$。事实上,$a^b \bmod n$ 可以直接计算,但是直接计算 a^b,就需要 $\Theta(n)$ 次乘法,计算量比较大。改进的办法是对 a^b 进行平方运算,其结果与它的指数 b 乘以 2 等价。此外,两个数 a^b 和 a^h 相乘等价于计算 a^{b+h}。令 b 的二进制表示为

$$\begin{aligned} b &= b_k b_{k-1} \cdots b_0 \\ &= b_k 2^k + b_{k-1} 2^{k-1} + \cdots + b_1 2 + b_0 \end{aligned}$$

其中,$b_i \in \{0,1\}$,$0 \leqslant i \leqslant k$。令 c_j 表示 b 的最左边的 j 位数,即 $c_j = b_k b_{k-1} \cdots b_{k-(j-1)}$,则有

$$c_j = \begin{cases} b_k & j = 1 \\ 2c_{j-1} + b_{k-(j-1)} & j > 1 \end{cases}$$

再令 $d_j = a^{c_j} \bmod n$,则有

$$d_j = \begin{cases} a^{b_k} \bmod n & j = 1 \\ (d_{j-1}^2 \bmod n)\, a^{b_{k-(j-1)}} \bmod n & j > 1 \end{cases}$$

根据上述递归方程,便可以得到计算 $a^b \bmod n$ 的迭代算法如下:

```
ModularExponentiation(a, b, n)
1  c ← 0
2  d ← 1
3  for i ← k downto 0 do
4          c ← 2c
5          d ← (d × d) mod n
6          if b_i = 1 then
7                  c ← c + 1
8                  d ← (d × a) mod n
9  return d
```

最初,$i = k$,$c = 0$。c 在 ModularExponentiation(a, b, n)算法中没有什么用,只起解释的作用。ModularExponentiation(a, b, n)算法从 $d = 1$ 开始,从左到右扫描二进制数,执行 $d \leftarrow (d \times d) \bmod n$。如果当前位二进制数值为 1,还要执行 $d \leftarrow (d \times a) \bmod n$。

例如,$a = 7$, $b = 560$, $n = 561$, 计算 $a^b \bmod n$。

$b = 560 = 1000110000$

i	9	8	7	6	5	4	3	2	1	0
b	1	0	0	0	1	1	0	0	0	0
c	1	2	4	8	17	35	70	140	280	560
d	7	49	157	526	160	241	298	166	67	1

最后结果 $d=1$。

如果输入 a,b,n 是 k 位的数，则算法需要执行的算术运算次数为 $O(k)$，总共需要执行的位运算的次数为 $O(k^3)$。

15.5 数论的应用

15.5.1 Hash 函数

为定位网络上某台计算机的位置，我们用一个 IP 地址来表示其位置。IP 地址由 32 位二进制数组成，为便于使用，常以 XXX. XXX. XXX. XXX 形式表示，每组 XXX 代表一个小于或等于 255 的 10 进制数。例如 59. 77. 16. 229。假设要维护一个拥有 250 个 IP 地址的列表，我们可以直接开辟一个数组，用 IP 地址来做索引，这种方法可以很方便地进行查询、插入和删除运算，但是由于 IP 地址的范围很大，开辟这么大的一个数组，空间浪费太大，也不可行。如果用链表来实现，上述运算的速度又太慢。这时候，最合适的算法自然是使用 Hash 表。Hash 就是把任意长度的输入，通过 Hash 算法，变换成固定长度的输出，该输出就是 Hash 值。

实现 Hash 函数的方法很多，其中借助模运算 mod 而获得的除法 Hash 函数最有效，除法 Hash 函数为

$$h(k) = k \bmod T$$

其中，k 为关键字，T 是 Hash 表的大小(即位置数)。Hash 表中的位置号从 0 到 $T-1$，每一个位置称为桶。若 k 不是正整数类型，则在计算 $h(k)$ 之前必须把 k 转换成非负整数。如果某元素 x 的关键字为 k，那么某元素在 Hash 表中的位置为 $h(k)$。要搜索关键字为 k 的元素，首先要计算出 $h(k)$，然后看表中 $h(k)$ 处是否有元素。如果有，便找到了该元素。如果没有，说明该表中不包含该元素。在前一种情况中，如果要删除该元素，只需把表中 $h(k)$ 位置清空即可。在后一种情况中，可以通过把元素放在 $h(k)$ 位置以实现插入，从而方便地实现元素的搜索、插入和删除。

由于 Hash 表的容量是有限的，如果几个 IP 地址在 Hash 表中对应的位置相同怎么办？解决该问题的方法很多，一个办法就是用链表，在 Hash 表的每个入口挂一个链表，保存所有对应的 IP 就可以了。如图 15.2 所示，图中元素 x 和 y 都映射

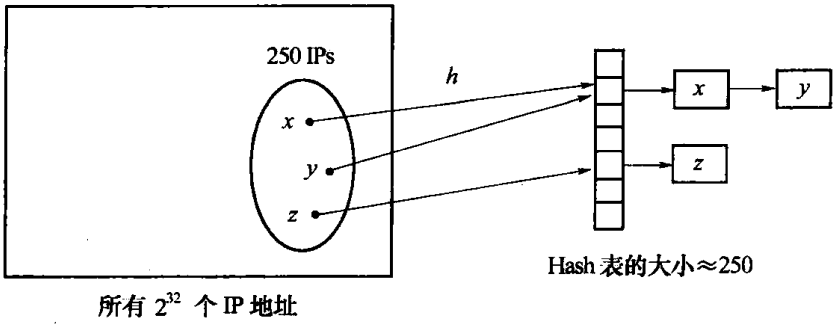


图 15.2

到 Hash 表的同一个位置,因此用链表保存。

此外,Hash 函数在信息安全中有重要的应用。关于 Hash 函数的详细介绍,这里就不再赘述,读者可参考文献[21]。

15.5.2 RSA 公钥加密系统

RSA 算法由美国麻省理工学院的学者 Ron Rivest、Adi Shamir 和 Leonard Adleman 于 1977 年提出并于 1978 年公布。算法 RSA 取名自三位发明者的名字,是第一个既能用于数据加密也能用于数字签名的算法。RSA 易于理解和操作,也很流行,是研究最广泛的公钥算法。从提出到现在已 30 多年,RSA 经历了各种攻击的考验,逐渐为人们接受,被人们普遍认为是目前最优秀的公钥方案之一。但 RSA 的安全性一直未能得到理论上的证明。RSA 算法的理论基础在于找到一个大素数很容易,而将一个大整数分解为两个大素数因子的乘积却是非常困难的。

在公钥加密系统中,每个参加者都用一把公开的密钥(简称公钥)和一把机密的密钥(简称私钥)。每把密钥都是一条信息。例如,在 RSA 公钥加密系统中,每把密钥均是由一对整数组成。令 P_A 和 S_A 分别表示参加者 A 的公钥和私钥,令 P_B 和 S_B 分别表示参加者 B 的公钥和私钥。每个参加者均自己创建其公钥与私钥,其中私钥不公开,而公钥则可以对任何人公开。设 D 表示允许的信息集合,例如,某些字符串的集合。为了达到加密解密的目的,我们需要构造对应公钥和私钥的函数:一种从 D 到其自身的一一对应的函数。令 $P_A(\,)$ 表示对应于 A 的公钥 P_A 的函数(可看作加密函数),$S_A(\,)$ 表示对应于 A 的私钥 S_A 的函数(可看作解密函数)。假定如果已知密钥 P_A 和 S_A 就能够有效地计算出函数 $P_A(\,)$ 与 $S_A(\,)$。任何参加者的公钥和私钥都是一个“匹配对”,它们指定的函数互为反函数。即对任意信息 $M \in D$,有

$$M = S_A(P_A(M)) \tag{15.8}$$

$$M = P_A(S_A(M)) \tag{15.9}$$

无论用哪一种次序,运用两把密钥 P_A 和 S_A 对 M 相继进行变换后,最后仍然得到信息 M。

在公钥加密系统中，一个假设就是除 A 外没有人能在较短的时间内计算出函数 $S_A()$。送给 A 加密的邮件的保密程度与对 A 的数字签名的证明均依赖于假设：只有 A 能够计算出 $S_A()$。设计一个可行的公钥加密系统的困难性在于：如何创建一个系统，在该系统中我们可以公开加密函数 $P_A()$，而不至于因此而公开计算其相应解密函数 $S_A()$ 的方法。

在公钥加密系统中，按下列过程进行加密。假定 B 希望给 A 发一条加密信息 M，使得窃听者无法获取该信息的真正含义，发送信息的方案如下：

（1）B 取得 A 的公钥 P_A（可以直接向 A 索取）。

（2）B 计算出相应于 M 的密文 $C=P_A(M)$，并把 C 发送给 A。

（3）当 A 收到密文 C 后，运用自己的私钥恢复出原始信息：$M=S_A(C)$。

图 15.3 说明了这一过程。因为 $S_A()$ 和 $P_A()$ 互为反函数，所以 A 能够根据 C 计算出 M。因为只有 A 能够计算出 $S_A()$，所以也只有 A 能根据 C 计算出 M。运用 $P_A()$ 对 M 进行加密可以使 M 的内容不会泄露给除 A 以外的任何人。

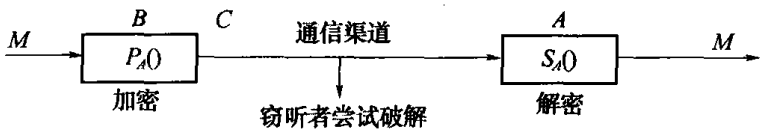


图 15.3

类似地，在公钥加密系统中可以很容易地实现数字签名。假设现在 A 希望把经过数字签名的回答 M' 发送给 B。数字签名的过程如下：

（1）A 运用私钥 S_A 计算出信息 M' 的数字签名 $\sigma=S_A(M')$。

（2）A 把信息签名对 (M',σ) 发送给 B。

（3）当 B 收到 (M',σ) 时，首先利用 A 的公钥 P_A 计算 $P_A(\sigma)$，然后验证等式 $M'=P_A(\sigma)$ 是否成立，以证实该信息的确是否由 A 发出的（假设 M' 包含有 A 的名字，这样 B 就知道应该使用谁的公钥）。如果等式不成立，那么 B 就得出结论，要么是信息 M' 或数字签名 σ 在信息传输过程中有误，要么信息对 (M',σ) 就是一个有意的伪造。图 15.4 说明了上述过程。因为数字签名既是对签名者身份的证明，也是对所签名信息内容的证明，类似对文件末尾的手写签名。

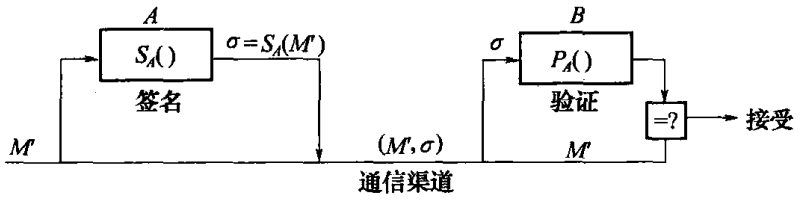


图 15.4

数字签名的一个优点是它可以被任何能取得签名者公钥的人所验证。一条签名过的信息可以被一方确认后再传送到其他各方，他们同样能对该签名进行验证。例如，这条信息可能是 A 发送给 B 的一张电子支票。当 B 确认了支票上 A 的签名

后，可以把该支票送交银行，而银行也可以对签名进行验证，然后决定是否支付相应的资金。

值得注意的是，被签名的信息并没有加密，该信息是“公开”的，没有受到保护。如果把上述有关加密和签名的两种方案结合起来使用，我们就可以构造同时被签名并加密的信息。其过程如下：签名者首先把他的数字签名附加在信息后面，然后再用希望的接受者的公钥对得到的信息签名对进行加密。接受者运用他的机密钥匙对收到的信息进行解密以同时获得原始信息签名对，然后他可以用签名者的公钥对签名进行验证。

RSA 公钥加密系统过程如下：

(1) B 创建其公钥与私钥

① 随机选取两个大素数 p 和 q，例如，素数 p 和 q 可能是上百位的十进制数；

② 根据 $n = p \times q$ 计算出 n 的值；

③ 选取一个与 $\varphi(n)$ 互素的小奇数 e，其中 $\varphi(n) = (p-1) \times (q-1)$；

④ 找到满足 $d \times e = 1 \bmod \varphi(n)$ 的 d，可以根据定理 15.19 来求；

⑤ 公开对 $P = (e,n)$，并把它作为 RSA 公钥；

⑥ 对 $S = (d,n)$ 进行保密，并把它作为 RSA 私钥；

(2) A 送信息 M 给 B

⑦ A 查询 B 的公钥 $P = (e,n)$，然后送密文 $W = M^e \bmod n$ 给 B；

⑧ 通过计算 $W^d \pmod n$，B 解密密文 W。

其中，步骤④可以用扩展的欧几里得算法计算，而步骤⑦可以调用 ModularExponentiation(M,e,n)，步骤⑧可类似步骤⑦计算。步骤⑦中与公钥 $P = (e,n)$ 相关联的信息 M 的加密函数为

$$P(M) = M^e \bmod n \tag{15.10}$$

与私钥 $S = (d,n)$ 相关联的密文 W 的解密函数为

$$S(W) = W^d \bmod n \tag{15.11}$$

式(15.10)和式(15.11)对加密与签名都适用。为了创建一个签名，签名人把其私钥应用于签名的信息而不是密码报文。为了确认签名，可以对签名而不是对被加密的信息应用签名人的公钥。

根据创建公钥与私钥的过程，我们可以给出一个签名的例子，来说明具体过程：

(1) 若 B 选择了 $p = 11$ 和 $q = 13$。

(2) 那么，$n = 11 \times 13 = 143$，$\varphi(n) = 10 \times 12 = 120$。

(3) 再选取一个与 $\varphi(n) = 120$ 互素的数，例如 $e = 7$。

(4) 找到 $d = 103$ 满足 $d \times e = 1 \bmod \varphi(n)$。

(5) $P = (e,n) = (7,143)$ 为公钥，$S = (d,n) = (103,143)$ 为私钥。

(6) B 公开公钥 $P = (7,143)$。

(7) 现假设 B 想发送消息 85 给 A,B 用自己的私钥 $S=(d,n)=(103,143)$ 进行签名:$M^e \bmod n = 85^{103} \bmod 143 = 6$,于是发送消息 85 和签名 6 给 A。

(8) 当 A 接收到消息 85 和签名 6 时,用 B 公开的公钥 $P=(7,143)$ 进行验证:$7 \pmod{143} = 85$,跟 B 发送的消息一致,于是确定该消息是由 B 所发送,且没有被修改。

从上面的分析可以看出,如果窃听者能对公钥中的模 n 进行分解,就可以根据公钥推导出私钥,并可以与公钥的创建者用一样的方法来运用因子 p 和 q。因此如果能够容易地分解大整数,也就能容易地攻破 RSA 加密系统。迄今为止,人们还没有找到快速分解大整数的方法,而且也没有发现比分解大整数 n 更容易的方法来攻破 RSA 公钥加密系统。因此 RSA 仍然是目前最优秀的公钥方案之一。

15.6 小 结

本章内容主要取材于文献[1],部分内容参考了文献[20,6],主要介绍了数论的基础知识以及基本的数论算法。

长期以来,数论只具有纯粹数学中的一些性质,而被认为没有直接的应用价值。计算机的产生与发展给科学技术带来了巨大而深刻的变革,这使数论有了非常广泛的应用途径。最著名的应用就是信息安全,整个数论算法是计算机安全的基础。一些著名的数论算法,例如大整数的分解以及素数的测试等问题中采用的随机算法,这里就不详细介绍,有兴趣的读者可参考文献[4]。

习题

15.1 证明推论 15.2。

15.2 证明如果 $a|b$ 且 $b|c$,则 $a|c$。

15.3 证明如果 p 是素数,并且 $0<k<p$,则 $\gcd(k,p)=1$。

15.4 证明对任意整数 a 和 b,如果 $a|b$ 且 $b>0$,则对任意 x 有 $(x \bmod b) \bmod a = x \bmod a$。

15.5 设 F_i 和 F_{i+1} 分别是第 i 次调用 Euclid(a,b) 算法中的第一个参数和第二个参数,且假定 $F_{i+1}<F_i$,则 $F_{i+2}<\frac{1}{2}F_i$。(提示:按照 $F_{i+1}\leqslant\frac{1}{2}F_i$ 以及 $F_{i+1}>\frac{1}{2}F_i$ 两种情况给予证明。)

15.6 将 ExtendedEuclid(a, b) 算法改写为迭代算法。

15.7 证明定理 15.9。

15.8 设计一个 β 位整数除以一个短整数及其余数的有效算法。要求所设计算法的时间复杂度为 $O(\beta)$。

15.9 设计一个能把β位二进制整数转化为相应十进制数的有效算法。并证明：如果长度至多为β位的整数的乘法与除法运算所需时间为$M(\beta)$，则执行二进制到十进制转换所需的时间为$\Theta(M(\beta)\lg\beta)$。（提示：类似5.4节大整数的乘法。）

15.10 类似图15.1，给出ExtendedEuclid(899,943)的输出值(d,x,y)。

15.11 证明对任意整数a,k和n，$\gcd(a,n)=\gcd(a+kn,n)$。

15.12 证明定理15.16。

15.13 定义$\text{lcm}(a_1,a_2,\cdots,a_n)$是整数$a_1,a_2,\cdots,a_n$的最小公倍数，即每个$a_i$的倍数中的最小非负整数。说明如何用算法$\gcd(a,b)$有效地计算出$\text{lcm}(a_1,a_2,\cdots,a_n)$。

15.14 求方程$35x\equiv 10 \bmod 50$的所有解。

15.15 将ModularLinearEquation(a, b, n)的第3行修改为

3 $c\leftarrow x'(b/d) \bmod (n/d)$

修改后的算法能正确运行吗？试分析原因。

15.16 计算出使方程$x\equiv 4 \pmod 5$和$x\equiv 5 \pmod{11}$同时成立的所有解。

15.17 试找出被9,8,7除时，余数分别为1,2,3的所有整数x。

15.18 修改模取幂算法，要求该算法从右向左检查b的各位的顺序，而不是从左向右检查。

15.19 设计模取幂算法的递归算法并分析其时间复杂度。

15.20 假设已知$\varphi(n)$，试说明如何运用ModularExponentiation(a, b, n)算法计算出对任意$a\in\mathbf{Z}_n^*$，$a^{-1} \bmod n$的值，这里$\mathbf{Z}_n^*=\{[a]_n\in\mathbf{Z}_n:\gcd(a,n)=1\}$。

15.21 考虑一个RSA密钥集合，其中$p=11$，$q=29$，$n=319$，$e=3$。在私钥中用到的d值是多少？对信息$M=100$加密后得到什么？

15.22 证明如果A的$e=3$，并且对方获得了A的私钥值d，则对方能够在关于$\lg n$的位数的多项式时间内对A的模n进行分解。

实验题

15.23 完成XOJ如下题目：1009,1016,1036,1044,1048,1051,1052,1079。

15.24 完成POJ如下题目：1006,1023,1061,1150,1152,1604,1730,1995,2115,2262,2635,2739,3090,3372。

第 16 章　计算几何

计算几何(Computational Geometry)作为计算机科学的一个分支,主要研究解决几何问题的算法。在现代工程和数学领域,例如计算机图形学、机器人技术、超大规模集成电路设计、模式识别以及统计学等,计算几何都有着十分重要而广泛的应用。计算几何问题就是给定一组几何物体的描述,如二维、三维或更高维空间中的一组点、线段,或者一个具有 n 个顶点的多边形,要求给出与这些物体有关的问题的求解算法,以满足实际问题的需要,例如是否有直线相交,是否可能产生一个新的物体,如顶点集的凸包等。下面我们给出计算几何中的一些常用的基本算法。

16.1　叉积及其应用

二维几何问题中,点 p_i 可用坐标(x_i,y_i)表示,给定两点 $p_1=(x_1,y_1)$,$p_2=(x_2,y_2)$,用$\overline{p_1p_2}$表示线段,线段上任意一点的坐标可定义为

$$p_3=(\alpha x_1+(1-\alpha)x_2,\alpha y_1+(1-\alpha)y_2),0\leqslant\alpha\leqslant1$$

如果一条线段的端点是有次序的,则称线段为有向线段。例如,线段从点 p_1 指向 p_2,则记为$\overrightarrow{p_1p_2}$,特别地,如果有向线段$\overrightarrow{p_1p_2}$的起点 p_1 是原点,则可以把线段简记为矢量 $\boldsymbol{p}_2$。

在设计与直线和线段相关的算法中,经常用到计算矢量的叉积。给定矢量 $\boldsymbol{p}_1$ 和 $\boldsymbol{p}_2$,矢量的叉积可表示为由原点(0,0),$p_1=(x_1,y_1)$,$p_2=(x_2,y_2)$ 以及 $p_1+p_2=(x_1+x_2,\ y_1+y_2)$所组成的平行四边形的带符号的面积,即 $\boldsymbol{p}_1\times\boldsymbol{p}_2=x_1y_2-x_2y_1$,其结果是一个标量,如图 16.1 所示。由叉积的定义可知 $\boldsymbol{p}_1\times\boldsymbol{p}_2=-\boldsymbol{p}_2\times\boldsymbol{p}_1$。利用叉积的符号,可以很方便地判断两矢量相互之间的顺或逆时针关系。

由图 16.1 可知,利用叉积可以求三角形的面积。例如,已知三个顶点分别为 $p_1=(x_1,y_1)$,$p_2=(x_2,y_2)$和 $p_3=(x_3,y_3)$,则我们可以将三角形平移,直到点 p_1 与原点重合,因此三角形面积 S 为 $S=\frac{1}{2}|(\boldsymbol{p}_2-\boldsymbol{p}_1)\times(\boldsymbol{p}_3-\boldsymbol{p}_1)|$,即矢量 $\boldsymbol{p}_2-\boldsymbol{p}_1$ 和 $\boldsymbol{p}_3-\boldsymbol{p}_1$ 叉积绝对值的一半。由于避免了开平方运算,因此,这种求面积的方法比海伦公式或者其他方法要简单。值得注意的是,面积应该取绝对值。此外,利用面积是否为 0 也可以考察三点共线问题。类似地,我们可以利用矢量的叉积来判断折线段的拐向:

如果$(\boldsymbol{p}_2-\boldsymbol{p}_1)\times(\boldsymbol{p}_3-\boldsymbol{p}_1)>0$,则说有向线段$\overrightarrow{p_1p_3}$在有向线段$\overrightarrow{p_1p_2}$的逆时针方向上,或者说以$p_1$为轴,$\overrightarrow{p_1p_2}$可左拐到$\overrightarrow{p_1p_3}$,如图16.2(a)表示。

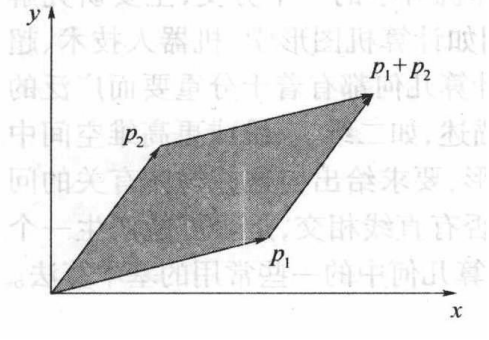


图16.1

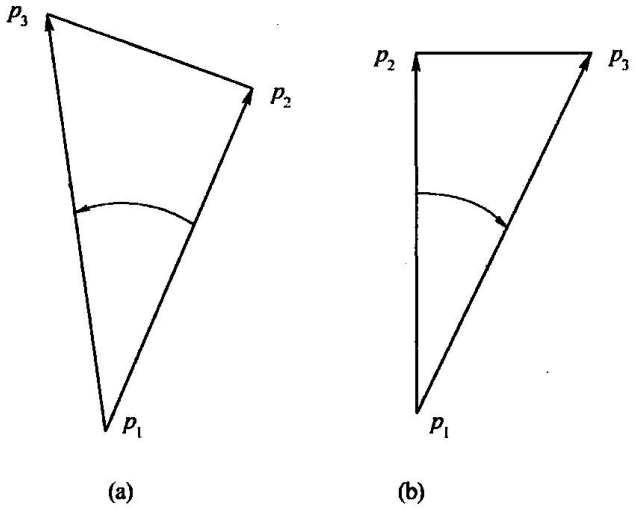


图16.2

如果$(\boldsymbol{p}_2-\boldsymbol{p}_1)\times(\boldsymbol{p}_3-\boldsymbol{p}_1)<0$,则说有向线段$\overrightarrow{p_1p_3}$在有向线段$\overrightarrow{p_1p_2}$的顺时针方向上,或者说以$p_1$为轴,$\overrightarrow{p_1p_2}$可右拐到$\overrightarrow{p_1p_3}$,如图16.2(b)表示。

如果$(\boldsymbol{p}_2-\boldsymbol{p}_1)\times(\boldsymbol{p}_3-\boldsymbol{p}_1)=0$,则说有向线段$\overrightarrow{p_1p_3}$和有向线段$\overrightarrow{p_1p_2}$共线(同向或反向),可以由此判断三点$p_1$,$p_2$和$p_3$共线。

利用叉积,我们还可以确定两条线段是否相交。为了确定两线段是否相交,我们只需检查两线段是否相互跨立含有对方线段的直线。线段$\overrightarrow{p_1p_2}$跨立一条直线,意味着线段的一个端点p_1在直线的一边,而点p_2在直线的另一边。此外还必须考虑点p_1或者点p_2在直线上这个边界条件。因此,两条线段相交当且仅当满足下列任一条件:

(1) 两线段相互跨立含有对方线段的直线。

(2) 一个线段的端点在另一个线段上。

为了判断两线段相互跨立含有对方线段的直线,我们可以利用叉积的符号来判断。如果线段$\overline{p_1p_2}$跨立含有$\overline{p_3p_4}$的直线,则$(\boldsymbol{p}_1-\boldsymbol{p}_3)\times(\boldsymbol{p}_4-\boldsymbol{p}_3)<0$且$(\boldsymbol{p}_2-\boldsymbol{p}_3)\times(\boldsymbol{p}_4-\boldsymbol{p}_3)>0$,或者$(\boldsymbol{p}_1-\boldsymbol{p}_3)\times(\boldsymbol{p}_4-\boldsymbol{p}_3)>0$且$(\boldsymbol{p}_2-\boldsymbol{p}_3)\times(\boldsymbol{p}_4-\boldsymbol{p}_3)<0$,如图16.3(a)所示。同理,如果线段$\overline{p_3p_4}$跨立含有$\overline{p_1p_2}$的直线,则一定有叉积$(\boldsymbol{p}_4-\boldsymbol{p}_1)\times(\boldsymbol{p}_2-\boldsymbol{p}_1)<0$,且$(\boldsymbol{p}_3-\boldsymbol{p}_1)\times(\boldsymbol{p}_2-\boldsymbol{p}_1)>0$,或者$(\boldsymbol{p}_4-\boldsymbol{p}_1)\times(\boldsymbol{p}_2-\boldsymbol{p}_1)>0$且$(\boldsymbol{p}_3-\boldsymbol{p}_1)\times(\boldsymbol{p}_2-\boldsymbol{p}_1)<0$,如图16.3(b)所示。

对于条件(2),一条线段(Segment)的端点r在线段$\overline{pq}$上,意味着r,p,q三点共线,即叉积$(\boldsymbol{q}-\boldsymbol{r})\times(\boldsymbol{p}-\boldsymbol{r})=0$。此外还必须满足$r$在$p$,$q$两点之间。图16.3(c)

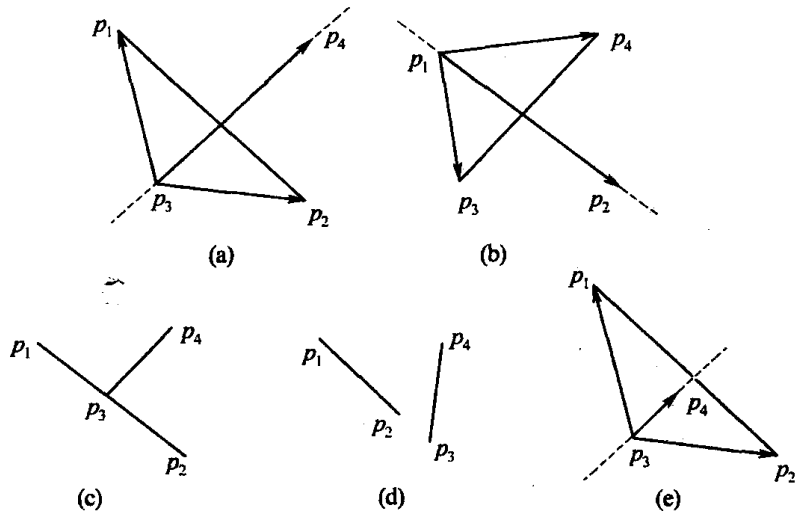


图 16.3

中，p_1, p_3, p_2 三点共线，因此有$(\boldsymbol{p}_3 - \boldsymbol{p}_1) \times (\boldsymbol{p}_2 - \boldsymbol{p}_1) = 0$ 且 OnSegment(p_1, p_2, p_3) = True。点 r 在 p, q 两点之间的判断，可实现如下：

```
OnSegment(p, q, r)
1  if min{x_p, x_q} ≤ x_r ≤ max{x_p, x_q} and min{y_p, y_q} ≤ y_r ≤ max{y_p, y_q} then
2          return True
3  else return False
```

如果条件(1)和(2)均不成立，则两线段不交叉。图 16.3(d)中，虽然 p_1, p_2, p_3 三点共线，但是点 p_3 不在线段$\overline{p_1p_2}$之间，因此，两线段不交叉。而图 16.3(e)中，虽然线段$\overline{p_1p_2}$跨立含有$\overline{p_3p_4}$的直线，但是线段$\overline{p_3p_4}$不跨立含有$\overline{p_1p_2}$的直线。因此两线段同样不交叉。

综上所述，判断两个线段相交(Segments Intersect)的算法如下：

```
SegmentsIntersect(p_1, p_2, p_3, p_4)
1  d_1 ← (p_1 - p_3) × (p_4 - p_3)
2  d_2 ← (p_2 - p_3) × (p_4 - p_3)
3  d_3 ← (p_3 - p_1) × (p_2 - p_1)
4  d_4 ← (p_4 - p_1) × (p_2 - p_1)
5  if d_1 d_2 < 0 and d_3 d_4 < 0 then
6       return True
7  else if d_1 = 0 and OnSegment(p_3, p_4, p_1) = True then return True
8       else if d_2 = 0 and OnSegment(p_3, p_4, p_2) = True then return True
9            else if d_3 = 0 and OnSegment(p_1, p_2, p_3) = True then
10                 return True
11                 else if d_4 = 0 and OnSegment (p_1, p_2, p_4) = True then
12                           return True
13                           else return False
```

16.2 计算任意线段的交点

给定平面上 n 条线段的集合 $S=\{l_1,l_2,\cdots,l_n\}$，确定是否有任意一对线段相交。为求解这个问题，我们介绍计算几何里常用的扫描技术。

在扫描技术中，经常假设有一条垂线 $x=a$ 从左到右扫描这个给定的几何物体的集合，通过几何物体与垂线的相交情况，可以方便地对几何物体进行排序。

为方便起见，这里假设没有垂直线段，没有三条线段交于同一点的情况。根据这个假设，我们能够按照扫描线与线段交点的高低，来排序集合 S 的线段。设 l_i 和 l_j 是 S 中的任意两条线段，如果 l_i 和 l_j 分别与垂线 $x=a$ 相交于点 p_i 和 p_j。假设点 p_i 在 p_j 的上方，则我们说 l_i 在 l_j 的上方，记为 $l_i >_a l_j$。关系 $>_a$ 定义了所有与垂线 $x=a$ 相交的线段集的全序。例如，图 16.4 中，我们有

$$l_3 >_a l_2 >_a l_1$$
$$l_5 >_b l_2 >_b l_1 >_b l_3$$

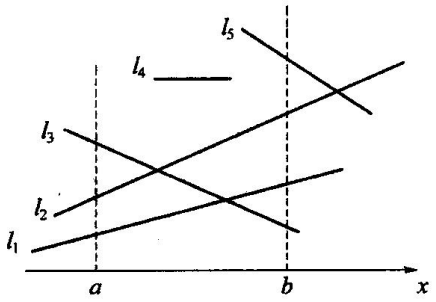


图 16.4

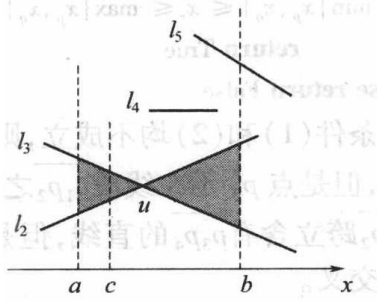


图 16.5

值得注意的是，如果两条线段相交，则交点左右线段的顺序关系相反，并且存在某些扫描线使得两条线段在交点某一边的顺序关系保持连续，即不变。例如图 16.5 中交点 u 的左边，$l_3 >_a l_2$，而在 u 的右边，$l_2 >_b l_3$。当扫描线 $x=c$ 在交点 u 到 $x=a$ 之间滑动时，始终有 $l_3 >_c l_2$。

扫描算法始终维护两个数据集：扫描线的状态以及事件点进度表。前者给出了与扫描线相交线段的关系，而后者是一个从左到右排列的 x 坐标的序列，定义了扫描线的暂停位置。事件点的进度表随着算法的运行而动态地确定。为了表示扫描线状态的全序 T 以及与 T 有关的运算，我们需要一个支持以下运算的数据结构：

(1) Insert(T,s)：插入线段 s 到 T 中。

(2) Delete(T,s)：从 T 中删除线段 s。

(3) Above(T,s)：返回紧接在线段 s 上方的线段。

(4) Below(T,s):返回紧接在线段 s 下方的线段。

上述运算显然可以用红黑树等数据结构实现,而且每个运算能够在 O(lg$n$)内完成。下面给出具体的扫描线算法:

```
AnySegmentsIntersect(S)
1  T ←∅
2  sort the endpoints of the segments in S from left to right
3  for each point p in the sorted list of endpoints do
4      if p is the left endpoint of a segment s then
5          Insert(T, s)
6          if (Above(T, s) exists and intersects s) or (Below(T, s)
           exists and intersects s) then
7              return True
8      if p is the right endpoint of a segment s then
9          if both Above(T, s) and Below(T, s) exist and Above(T, s)
           intersects Below(T, s) then
10             return True
11         Delete(T, s)
12 return False
```

AnySegmentsIntersect(S)算法中,T 表示扫描线状态的全序。第 2 行,线段集合 S 中的线段按照末端点的 x 坐标从左到右排序。如果有几个末端点具有同样的 x 坐标,则把左端点的 x 坐标放在右端点 x 坐标的前面。如果有几个左端点或右端点具有同样的 x 坐标,则端点 y 坐标小的放前面。第 3 行对每个扫描到的点 p,判断:

(1) 如果点 p 是线段 s 的左端点时,s 被加到 T,同时判断是否有一条线段 Above(T, s) 紧接在 s 的上面,并与 s 相交,或者有一条线段 Below(T, s)紧接在 s 的下面,并与 s 相交,如果有,则返回 True。

(2) 如果点 p 是线段 s 的右端点时,同时测试紧接在 s 上下的线段 Above(T, s)和 Below(T, s),若线段 Above(T, s)和 Below(T, s)相交,则返回 True。从 T 删除 s。

如果 **for** 循环执行完后,没有发现线段相交,则返回 False。

如图 16.6 所示,当扫描线到达 $x=a$ 时,碰到的是线段 l_1 的左端点,l_1 被加到 T。当到达 $x=b$ 时,碰到的是线段 l_2 的左端点,l_2 被加到 T,l_1 与上面的线段 l_2 不相交。类似地,扫描线由左至右,当扫描线到达 $x=c$ 时,碰到的是线段 l_1 的右端点,删除 l_1,同时,l_1 上下的线段 l_3 和 l_4 相交,返回 True。算法终止。

现在我们分析一下算法的运行时间,第 2 行用合并排序算法,其时间复杂度为 O($n$lg$n$)。第 3 行 **for** 循环最多执行 O(n),每步中运算最多耗时 O(lg$n$),因此 **for** 循环用时 O($n$lg$n$),因此我们有下列结论。

定理 16.1 AnySegmentsIntersect(S)算法的时间复杂度为 O($n$lg$n$)。

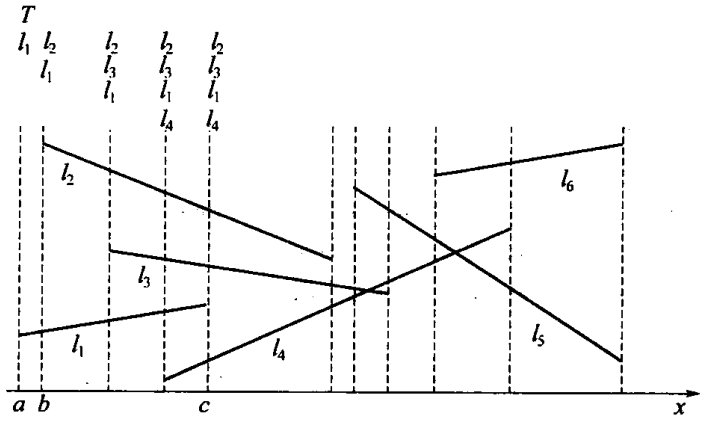


图 16.6

16.3 凸 包

一个有 n 个顶点的多边形 P 可用顶点的逆时针顺序 $<p_0,p_1,p_2,p_3,\cdots,p_n>$ 表示。其中，$p_n=p_0$，$\overline{p_{i-1}p_i}$ 为多边形的边。如果多边形的边除了连接顶点外，之间与其他边没有公共点，则称该多边形为简单多边形，否则称非简单多边形。如果简单多边形任意两个顶点的连线均在多边形的内部，则称该多边形为凸多边形，否则称非凸多边形，如图 16.7 所示，(a)代表一个简单多边形，(b)表示一个非简单多边形，(c)是一个凸多边形，(d)表示一个非凸多边形。

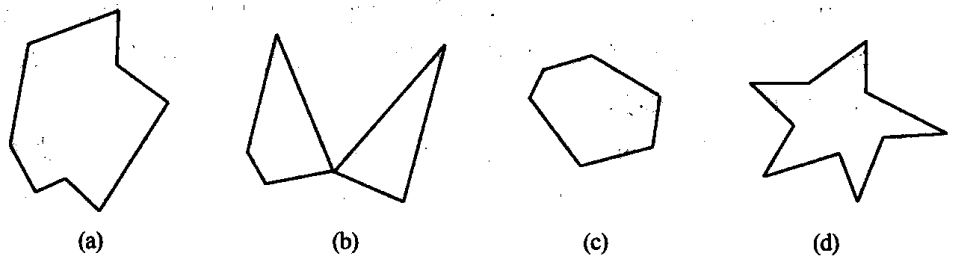


图 16.7

点集 Q 的凸包(Convex Hull)，记为 CH(Q)，是一个面积最小的凸多边形，使得 Q 中的点在多边形边上或者在多边形内部。图 16.8 中，多边形 $<p_0,p_1,p_3,p_6,p_8,p_9,p_0>$ 就是点集 $Q=\{p_0,p_1,p_2,p_3,\cdots,p_9\}$ 的凸包。

计算一个点集的凸包是计算几何领域中一个非常有趣的问题，在机器人运动规划、模式识别、划分问题等许多领域都有应用。下面我们介绍一下凸包的基本性质。

点集 Q 的凸包有下列性质：

性质 16.1 设 $p,q \in Q$,Q 中的线段 pq 是 CH(Q)的一条边当且仅当点集 Q 的所有其他点都包含在经过 p 和 q 的直线的一边。

性质 16.2 Q 中的点 p 是 CH(Q)的一个顶点当且仅当存在过点 p 的直线 l,使得点集 Q 的所有其他点都包含在直线 l 的一边。

性质 16.3 Q 中的点 p 不是 CH(Q)的顶点当且仅当点 p 包含在由 Q 中其他三个点所形成的三角形内部,或者在由 Q 中其他两个点所形成线段的内部。

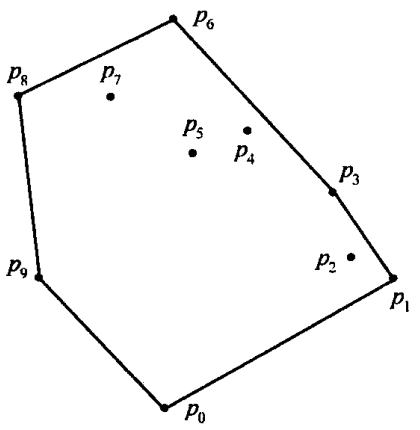


图 16.8

根据上述性质,我们很容易验证,对于平面上任意点集 Q,Q 中具有最小 x 坐标,最大 x 坐标,最小 y 坐标,最大 y 坐标的点均在 CH(Q)的边界上,这些点也依次称为最左极点,最右极点,最下极点和最上极点。

下面,我们介绍求凸包的几种常用算法。

16.3.1 礼物包装算法

对于平面上任意点集 Q,假设这些点没有三点共线的情况,把这些点设想为平面上的一根根柱子。由于 Q 中具有最小 y 坐标的点在 CH(Q)的边界上,因此我们把一根绳子一头系在具有最小 y 坐标的柱子上,然后拉动绳子,并沿逆时针方向旋转,直至碰到另一根柱子,继续沿逆时针方向旋转绳子,直到绳子被起始柱子挡住为止。如图 16.8 所示,围住的区域就是所形成的凸包,挡住绳子的柱子就是凸包的顶点。图 16.9(a)表示人拉着的绳子碰到柱子 p_1。图 16.9(b)表示人拉着的绳子依次碰到柱子 p_3,p_6。图 16.9(c)表示人拉着的绳子碰到柱子 p_8,p_9。图 16.9(d)表示人拉着的绳子碰到初始柱子 p_0,过程结束,多边形 $<p_0,p_1,p_3,p_6,p_8,p_9,p_0>$ 就是所求的凸包。

每次当绳子绕着当前柱子旋转时,会碰到另一根柱子,进行称为包装步骤的运算,这就是礼物包装算法的由来。从当前柱子寻找绳子要碰到的下一根柱子,并不需要复杂的计算。只需要找当前柱子所在的凸边与其他柱子跟当前柱子所形成边的夹角中,找最小角度的一根柱子即可。例如图 16.10 中,当前柱子 p 所在的凸边为 p_0p,在其他柱子与当前柱子 p 形成的边中,边 pr 与 p_0p 的夹角最小。因此,绳子要碰到的下一根柱子为 r。

详细的礼物包装(Gift Wrapping)算法可以描述如下:

```
GiftWrapping(Q)
1  p ← the lowest point p0
2  let x axis with origin p be initial hull edge
```

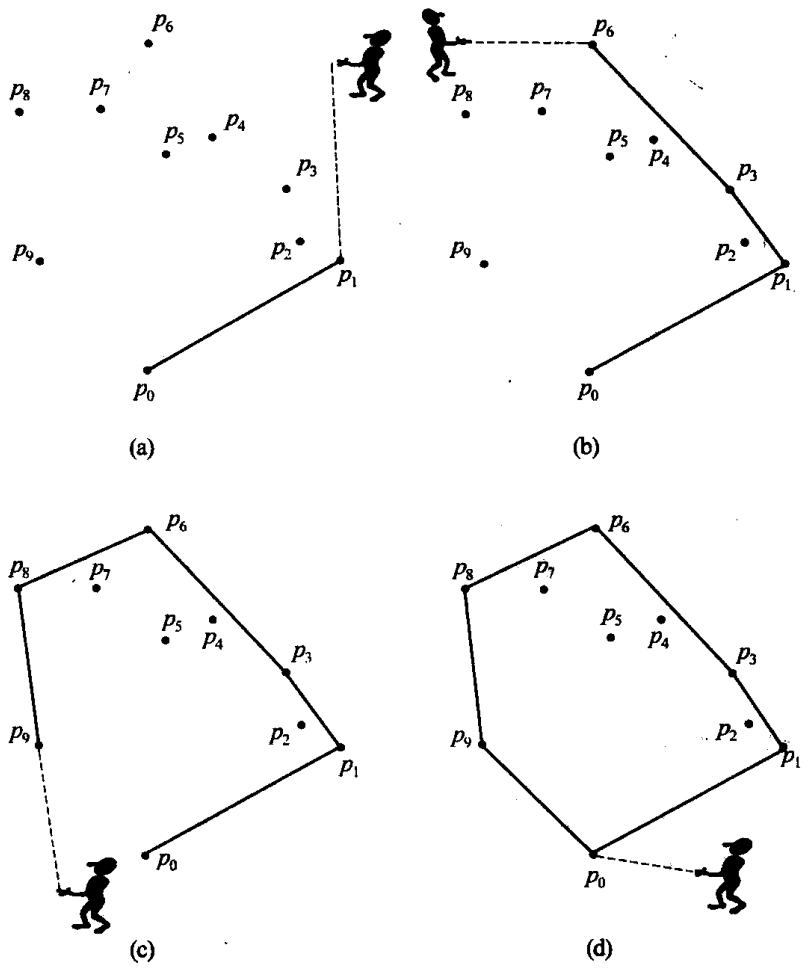


图 16.9

```
3  repeat
4      for each q ∈ Q and q ≠ p do
5          compute counterclockwise angle θ from previous hull edge
6      let r be the point with smallest θ
7      output (p, r) as a hull edge
8      p ← r
9  until p = p0
```

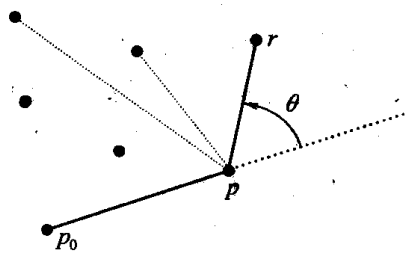


图 16.10

GiftWrapping(Q)算法从 x 轴出发，沿着逆时针(counterclockwise)方向寻找当前点 p 所在的凸边与其他点跟 p 所形成边的最小夹角的那个点 r(第 4 行 ~ 第 6 行)，然后输出凸包的边(p, r)。重复上述过程，直到找到初始的出发点 p_0。

现在我们分析一下 GiftWrapping(Q)算法的运行时间。设点集 Q 的大小为 n，凸包 CH(Q)的大小为 $h(\leqslant n)$。算法 **repeat** 循环执行的次数取决于凸包的大小。每次 **for** 循环最多执行 n 次，因此我们有下述结论。

定理 16.2 设点集 Q 的大小为 n，凸包 CH(Q)的大小为 $h(\leqslant n)$。则 GiftWrapping(Q)算法的时间复杂度为 $O(nh)$。

从定理 16.2 可知，GiftWrapping(Q)算法是一种运行时间取决于输出大小的算法，也就是说取决于给定点集 Q 的好坏。当点集 Q 的所有点都在凸包上时，出现最坏情形，时间复杂度为 $O(n^2)$。虽然礼品包扎算法最坏情形下不是很有效，然而这个算法在实际中还是很有效的，因为许多实际问题中 CH(Q)的大小 h 是相对较小的。下面，我们给出一些虽然有点复杂，但是对所有凸包大小都有效的算法。

16.3.2 Graham 扫描法

Graham 扫描法的基本思想是先找出一个最低位置且最左的点，将其作为原点，其他点用极坐标(r,θ)表示，并且将各点按照极角 θ 的大小从小到大排序，如果两个点具有同样大小的极角，则 r 小的排在前面。以原点为中心的扫描线从水平射线开始以逆时针方向扫描整个平面，每当扫描线碰到一个点，就会决定该点是否包括在凸包中。为了跟踪凸包的构造过程，我们用栈来记录当前扫描到的点并判断其是否包括在凸包中。

设经过排序后的点集 $Q=\{p_0,p_1,p_2,p_3,\cdots,p_{n-1}\}$，初始栈 S 中只包括(p_0,p_1,p_2)，p_2 在栈顶，然后扫描线扫描各点，从 p_3 到 p_{n-1}结束。在任何时刻，设栈 S 中含有$\{p_0,p_1,p_2,\cdots,p_{i-2},p_{i-1},p_i\}$，这时 p_i 是栈顶元素，而 p_{i-1}是紧接 p_i 的元素。当扫描线碰到 p_k，如果线段$\overline{p_{i-1}p_i}$可以左拐到$\overline{p_{i-1}p_k}$，则点 p_k 进栈，即 Push(p_k, S)。否则，点 p_i 出栈，即 Pop(S)，然后重新测试，直到满足条件。最后，Push(p_k, S)。当 p_k 处理完后，扫描线移动到下一个点。值得注意的是，这里的 p_i 不一定是 p_{k-1}，p_{i-1}也不一定是 p_{k-2}，详细的 Graham 扫描(Scan)法可以描述如下：

```
GrahamScan(Q)
1    S←∅
2    sort Q by polar angle in counterclockwise (break ties by r)
3    Push(p0, S)
4    Push (p1, S)
5    Push (p2, S)
6    i ← 2
7    for k ← 3 to n − 1 do
8        while the segment p_{i−1}p_i turns right to p_{i−1}p_k or p_{i−1}, p_i, p_k
         are collinear do
```

```
9              Pop(S)
10             i ← i − 1
11         Push(p_k, S)
12 return S
```

算法 GrahamScan(Q)第 8 行 ~ 第 10 行是一个反复执行的过程，直到$\overline{p_{i-1}p_i}$可以左拐到$\overline{p_{i-1}p_k}$为止。算法结束后返回的 S 中含有的由底至顶的元素就是 Q 的凸包顶点按逆时针排列的点序列。需要注意的是，我们对点按极角逆时针排序时，并不需要真正求出极角，只需要求出任意两点的次序就可以了，具体可以用叉积来实现。

由于算法第 2 行的时间复杂度为 $O(n\lg n)$，第 3 行 ~ 第 6 行需要常数时间。**for** 循环至多迭代 $n-3$ 次。因为 Push 花常数时间，除了第 8 行 **while** 循环外，**for** 循环至多执行 $O(n)$。现在我们用分摊分析中的合计分析来证明，**while** 循环总共执行 $O(n)$。**for** 循环中，每个点只进栈一次。对每个 Push 运算，至多有一个 Pop 运算相对应。由于每次栈中至少有 3 个点，因此，Pop 运算至多执行$n-2$次。在 **while** 循环的每次迭代中，只执行一次 Pop 运算，因此 **while** 循环总共执行 $O(n)$次迭代。分摊到 **for** 循环的每次迭代的次数为常数时间 $\Theta(1)$，因此有如下结论。

定理 16.3 GrahamScan(Q)算法的时间复杂度为 $O(n\lg n)$。

图 16.11 给出了一个例子，描述了 Graham 扫描法的执行过程。图 16.11(a)给出了一个排序后的点集 $Q=\{p_0,p_1,p_2,p_3,p_4,p_5,p_6,p_7,p_8,p_9\}$，初始栈中含有点 p_0,p_1,p_2，用粗线条连接，当前扫描到的顶点为 p_3，用带圈的点表示(下同)，如图 16.11(b)所示。由于线段$\overline{p_1p_2}$可以右拐到$\overline{p_1p_3}$，因此点 p_2 出栈。由于线段$\overline{p_0p_1}$可以左拐到$\overline{p_0p_3}$，因此 p_3 进栈，当前扫描点为 p_4，如图 16.11(c)所示。由于线段$\overline{p_1p_3}$可以左拐到$\overline{p_1p_4}$，因此点 p_4 进栈，当前扫描点为 p_5，如图 16.11(d)所示。类似地，p_5 进栈，当前扫描点为 p_6，如图 16.11(e)所示。由于$\overline{p_4p_5}$可以右拐到$\overline{p_4p_6}$，因此点 p_5 出栈，如图 16.11(f)所示。类似地，p_4 出栈，如图 16.11(g)所示。由于$\overline{p_1p_3}$可以左拐到$\overline{p_1p_6}$，因此点 p_6 进栈，当前扫描点为 p_7，如图 16.11(h)所示。类似地，p_7 进栈，当前扫描点为 p_8，如图 16.11(i)所示。由于$\overline{p_6p_7}$可以右拐到$\overline{p_6p_8}$，因此点 p_7 出栈，p_8 进栈，当前扫描点为 p_9，如图 16.11(j)所示。由于$\overline{p_6p_8}$可以左拐到$\overline{p_6p_9}$，因此点 p_9 进栈，当前扫描点为 p_0，如图 16.11(k)所示。算法结束，构造的凸包如图 16.11(l)所示。

16.3.3 分治算法求凸包

第 5 章介绍了分治算法，我们已经体会到分治算法是解决许多问题的一种非常简单而有效的算法，这节介绍分治算法在求凸包中的应用。

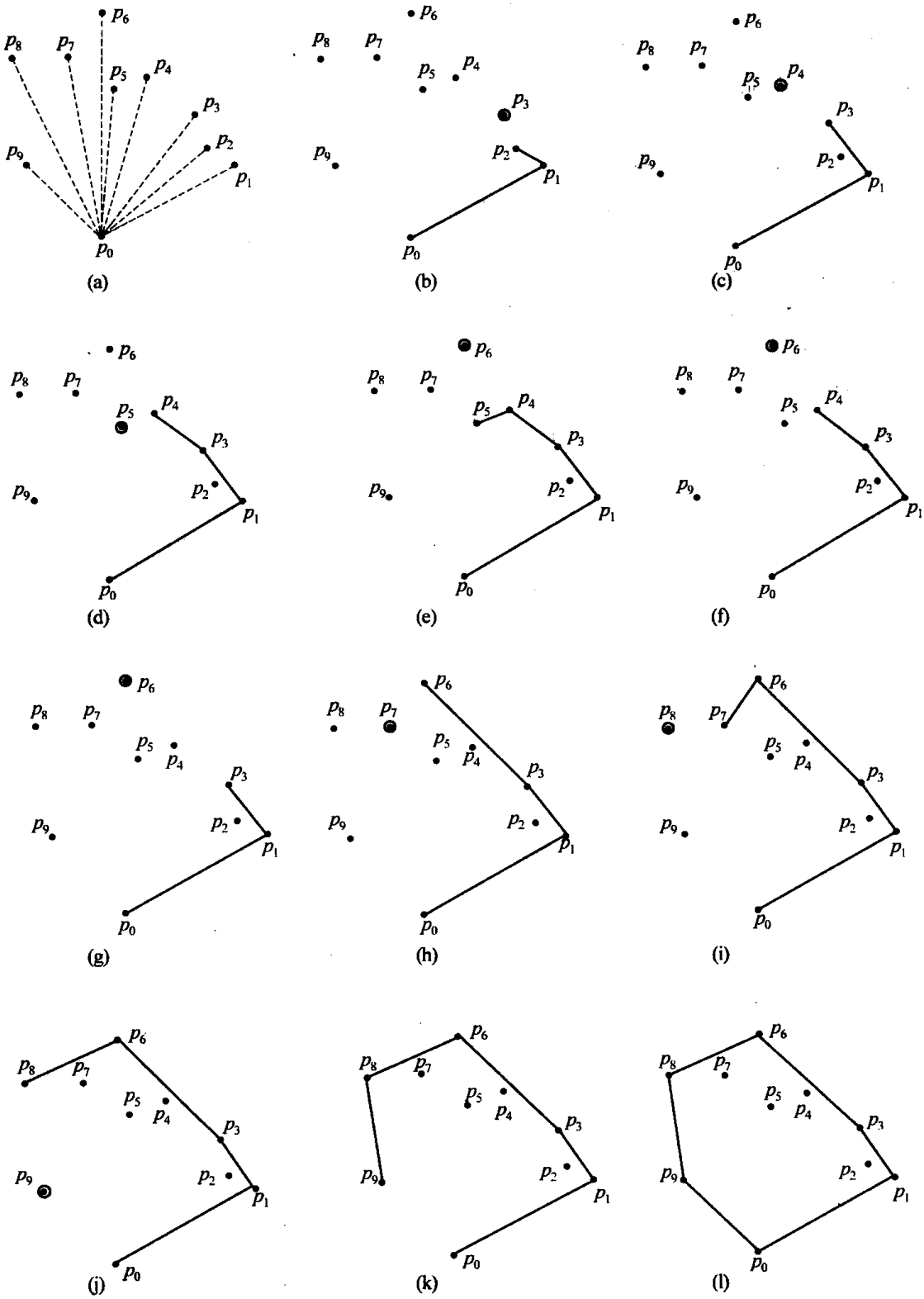


图 16.11　Graham 扫描法的计算过程

1. 快速凸包算法

快速凸包算法类似快速排序算法，它的基本思想是尽可能多地丢掉凸包内部的点，集中考虑那些距离边界比较近的点。算法首先找到凸包的两个极点，例如最右最低的极点 u 以及最左最高的极点 v，这样就能保证所取的两个极点互不相同，如图 16.12所示。然后将整个点集 Q 分成两部分：上半平面 Q_u 和下半平面 Q_l，然后分别求每个半平面的凸包顶点，整个凸包由两个半平面的凸包顶点构成。快速凸包算法就是对上、下半平面分别进行递归求解。考虑一般情况，假设现在需要对半平面 S 进行求解，即直线 uw 右下区域。我们首先从这个区域里找一个离直线 uw 最远的点 y，直线 wy 和直线 yu 将 S 分成半平面 S_l（即直线 wy 右下区域）、S_r（即直线 yu 右下区域）以及三角形区域。然后我们只要从这两个区域 S_l 和 S_r 找凸包的顶点即可，而三角形 $<u,w,y>$ 中，除了顶点 u,w,y 之外，其他顶点包括三角形内部以及在三角形边上的点都被去掉，如图 16.12 阴影部分所示。对两个半平面 S_l 和 S_r 求凸包的顶点，类似上述过程，可递归求解。

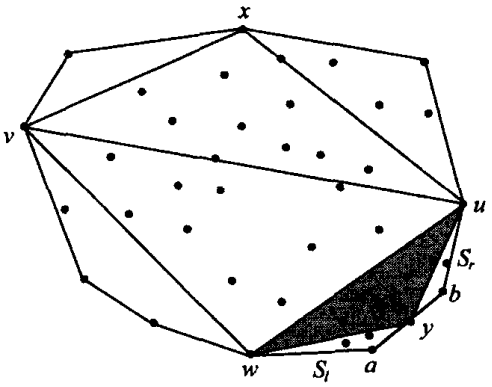


图 16.12

详细的快速凸包（Quick Hull）算法可以描述如下：

```
QuickHull(u, w, S)
1  if S = {u, w} then return {u, w}
2  else
3        y ← the furthest point from line uw
4        S_l ← points in the right of line wy
5        S_r ← points int the right of line uy
6        return QuickHull(y, w, S_l) ∪ QuickHull(u, y, S_r)
```

我们现在分析一下 QuickHull(u, w, S)算法的时间复杂度。找初始的极点 u，v，然后划分点集 Q 能够在 O(n)时间内完成。假设划分 Q 后得到 Q_u 的大小为 n_u，Q_l 的大小为 n_l，且有 $n_u + n_l \leqslant n$，而且递归调用的时间取决于子问题的大小 n_u, n_l。令 $T(n)$ 表示 QuickHull(u, v, Q)算法找 n 个点的凸包所需要的时间，则有下述递归方程

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 2 \\ T(n_u) + T(n_l) + O(n) & \text{if } n > 2 \end{cases}$$

对于上述递归方程，如果我们不知道 n_u, n_l 的大小，则显然无法求解。

下面，我们考虑两种特殊情况：

（1）均衡划分，此时 $n_u = n_l = n/2$，则原递归方程变为

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 2 \\ 2T(n/2) + O(n) & \text{if } n > 2 \end{cases}$$

利用第4章的公式法可得 $T(n) = O(n\lg n)$。

(2) 极端划分，$n_u = 0, n_l = n - 1$，则原递归方程变为

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 2 \\ T(n-1) + O(n) & \text{if } n > 2 \end{cases}$$

利用第4章的替换方法可得 $T(n) = O(n^2)$。由此可见，虽然快速凸包算法平均情形下是非常快的，但是在最坏情形下，时间复杂度仍然比较高。

2. 合并凸包法

前面我们介绍的求凸包的分治算法，不需要合并两个子问题的解。本节我们利用另外一种分治算法来求解点集的凸包，求出两个子问题的凸包后，需要合并两个凸包。为方便起见，我们假定没有三点共线以及没有两点位于一条垂线的情形。分治算法的思路是将大小为 n 的点集 Q 分成两个子集 Q_l, Q_r，其中点集 Q_l 的大小为 $\lceil n/2 \rceil$，点集 Q_r 的大小为 $\lfloor n/2 \rfloor$，然后分别递归地寻求 Q_l 和 Q_r 的凸包 $CH(Q_l)$ 和 $CH(Q_r)$，最后求出 $CH(Q_l) \cup CH(Q_r)$ 的凸包，其算法过程如下：

```
DivideConquerHull(Q)
1  if |Q| <6 then
2      use exhaustive search to find CH(Q)
3      return CH(Q)
4  else
5      compute separation line l that divides Q into Q_l, Q_r
6      CH(Q_l)←DivideConquerHull(Q_l)
7      CH(Q_r)←DivideConquerHull(Q_r)
8      CH(Q)←Merge(CH(Q_l),CH(Q_r))
9      return CH(Q)
```

Merge($CH(Q_l)$,$CH(Q_r)$)算法能否快速合并两个子凸包是 DivideConquerHull(Q) 算法求凸包的关键。这里我们介绍两种合并两个凸包的方法：

第一种是使用 Graham 扫描法。其基本思想是先从 $CH(Q_l)$ 中选择一个内部点 p，然后从该点引一条水平线，如果该水平线与 $CH(Q_r)$ 相交，则在 $CH(Q_r)$ 中找出纵坐标最大的顶点 u 以及纵坐标最小的顶点 v。如果该水平线与 $CH(Q_r)$ 不相交，则在 $CH(Q_r)$ 中找出纵坐标与点 p 形成最大极角的一个顶点 u 以及与点 p 形成最小极角的一个顶点 v。找出 $CH(Q_r)$ 的顶点 u, v 后，我们可以从 v 开始，沿 $CH(Q_r)$ 的逆时针方向，依次找出顶点，直到碰到 u，从而得到一个顶点序列 P_1。同样从 v 开始，沿 $CH(Q_r)$ 的顺时针方向，依次找出顶点，直到碰到 u，从而得到一个顶点序列 P_2。对凸包 $CH(Q_l)$，我们可以逆时针给出顶点序列 P_3。这三个序列按照极角的大小，都是递增序列，这样我们可以按照合并排序算法中的合并程序，对这三个序列进行合并，得到合并序列 P。然后我们可以按照 Graham 扫描方法，逐

步检查这些点，如果连续三点形成右拐，则可以删除其中间点，依次类推，剩下的顶点即为凸包的顶点。

例如，给出两个凸包 $\mathrm{CH}(Q_l)$，$\mathrm{CH}(Q_r)$，如图 16.13 所示，按照上述方法，我们可以得到 $u=p_3$，$v=p_0$，顶点序列 P_1 为 p_0,p_1,p_2,p_3；顶点序列 P_2 为 p_5,p_4；顶点序列 P_3 为 p_6,p_7,p_8,p_9,p_{10}，合并这三个有序序列可得顶点序列

$$p_6,p_7,p_0,p_5,p_1,p_4,p_2,p_3,p_8,p_9,p_{10}$$

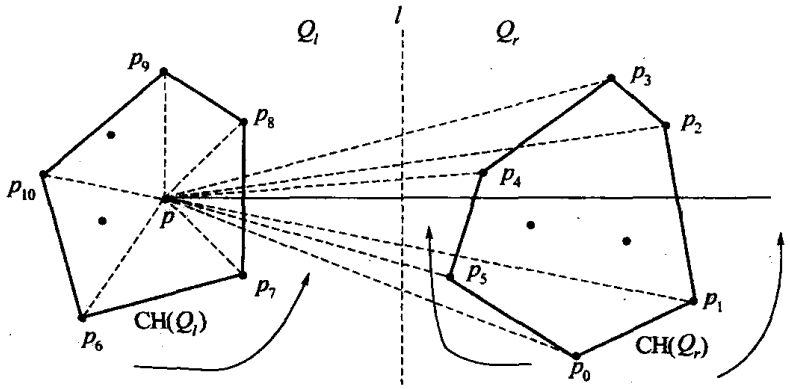


图 16.13

如图 16.14 所示。由于顶点 p_6,p_7,p_0 形成右拐，因此可以删除顶点 p_7，按照此办法，依次删除顶点 p_7,p_5,p_4,p_8，可得到凸包 $\mathrm{CH}(Q)$，如图 16.15 所示。

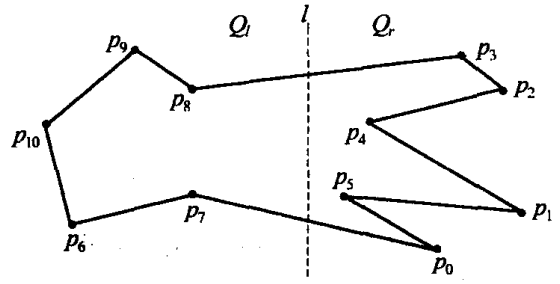


图 16.14

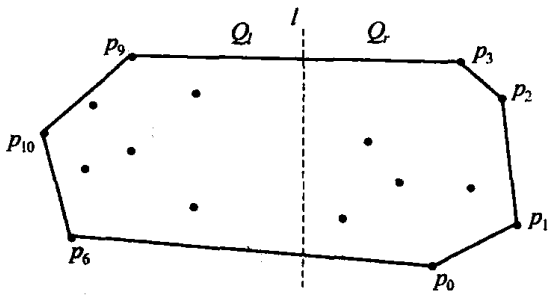


图 16.15

在上述合并方法中，对三个有序序列进行合并，需要花费的时间最多为 $O(n)$。删除点的运算可在 $O(n)$ 时间内完成。令 $T(n)$ 表示 DivideConquerHull(Q) 算法找 n 个点的凸包所需要的时间，子问题 Q_l,Q_r 的大小分别为 $\lceil n/2\rceil$ 和 $\lfloor n/2\rfloor$，因此有

$$T(n)=\begin{cases}\Theta(1) & \text{if } n<6\\ T(\lceil n/2\rceil)+T(\lfloor n/2\rfloor)+O(n) & \text{if } n\geqslant 6\end{cases}$$

利用第 4 章所介绍的公式法，我们很容易得到下述结论。

定理 16.4 基于 Graham 扫描法的 DivideConquerHull(Q) 算法的时间复杂度为 $O(n\lg n)$。

第二种方法是寻找两个凸包的切线，一条是从下面支撑两个凸包的切线，称为下切线（lower tangent）；另一条从上面覆盖两个凸包的切线，称为上切线（upper tangent）。为方便计算，令凸包 $CH(Q_l)$ 的顶点序列为 $\{p_0, p_1, \cdots, p_{n_l}\}$，$CH(Q_r)$ 的顶点序列为 $\{q_0, q_1, \cdots, q_{n_r}\}$。我们先介绍如何求下切线，类似地可以求上切线。

找下切线思路是：首先找到 $CH(Q_l)$ 的最右极点 p_i，然后找出 $CH(Q_r)$ 的最左极点 q_j，如果线段 p_iq_j 不是 $CH(Q_r)$ 的切线，则 j 增加，找下一点，直到找到 $CH(Q_r)$ 的切线 p_iq_k。然后找 $CH(Q_l)$ 的切线，如果 p_iq_k 不是，则 i 减少，找下一点，直到找到 $CH(Q_l)$ 的切线 p_hq_k。如果 p_hq_k 不是两个凸包的下切线，则重复上述过程，直到找到下切线。

例如，对于图 16.16，凸包 $CH(Q_l)$ 的最右极点为 p_3，$CH(Q_r)$ 的最左极点 q_7。线段 p_3q_7 不是下切线，也不是凸包 $CH(Q_r)$ 的切线，此时 $j=7$，j 逐步增加到 9，此时 p_3q_9 为凸包 $CH(Q_r)$ 的切线，但不是 $CH(Q_l)$ 的切线。此时 $i=3$，i 逐步减少直到 0，此时 p_0q_9 为凸包 $CH(Q_l)$ 的切线，但还不是 $CH(Q_r)$ 的切线，此时，$j=9$，增加 1，找到 q_{10} 即 q_0，此时 p_0q_0 即为两个凸包的下切线。

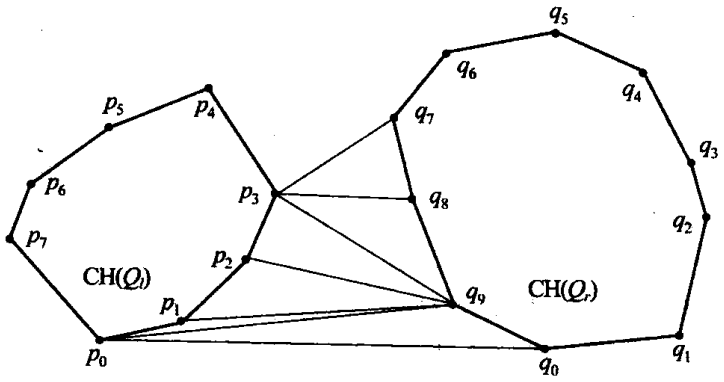


图 16.16

具体算法可以描述如下：

```
FindLowerTangent(CH(Q_l),CH(Q_r))
1  let p_i be the rightmost point of CH(Q_l)
2  let q_j be the leftmost point of CH(Q_r)
3  while p_iq_j is not lower tangent of CH(Q_l) and CH(Q_r) do
4        while p_iq_j not lower tangent to CH(Q_r) do
5              j ← j+1
6        while p_iq_j not lower tangent to CH(Q_l) do
7              i ← i-1
8  return p_iq_j
```

类似 Graham 扫描方法，我们有下列结论。

定理 16.5 基于找下、上切线的 DivideConquerHull(Q)算法的时间复杂度为 $O(n\lg n)$。

证明可参考习题 16.6 的解答。

16.4 最近点对

给定点集 $Q=\{p_1,p_2,p_3,\cdots,p_n\}$，$p_i=(x_i,y_i)$，$i=1,2,\cdots,n$，要求找出其中距离最近的两个点。这里两点间距离公式为 $d(p_i,p_j)=\sqrt{(x_i-x_j)^2+(y_i-y_j)^2}$。

对于这个问题，我们可以直接对 n 个点两两之间的距离进行比较，这样可以找出距离最近的点对，但是这种算法的时间复杂度为 $O(n^2)$。对于规模比较小的问题，可以这样直接求解，但是对于规模比较大的问题，则需要寻找更有效的算法，下面我们给出一个求解最近点对的分治算法。分治算法的思想比较直接，将大小为 n 的点集 Q 分成两个子集 Q_l,Q_r，其中点集 Q_l 的大小为 $\lceil n/2\rceil$，点集 Q_r 的大小为 $\lfloor n/2\rfloor$，然后求每个点集距离最近的点对与原问题相似，可以递归求解。值得注意的是，划分点集的时候，要保证两个点集能完全分开，即 $Q_l\cap Q_r=\varnothing$。对点集 Q 分成两个子集 Q_l,Q_r 后，最近点对可能出现三种情形：

（1）最近点对都在 Q_l 中；

（2）最近点对都在 Q_r 中；

（3）最近点对，一点在 Q_l，另一点在 Q_r 中。

不管是哪种情形，我们可以分别把这三种情形的最近点对都求出来，然后选择其中距离最小的一对点。对于情形(1)，我们只需要对点集 Q_l 递归求解，求出其最近点对的距离 δ_l；而对情形(2)，可对 Q_r 进行递归求解，求出其最近点对的距离 δ_r。为了确定情形(3)下的最近点对，假设我们对 Q 中所有点的 x_i 求中值，然后在中值处划一条垂线 l，垂线左边的点属于 Q_l，垂线右边的点属于 Q_r。位于垂线上的点可在 Q_l 和 Q_r 之间均匀分配，如图 16.17 所示。

设 δ 是 Q_l 的最近点对的距离以及 Q_r 的最近点对的距离的最小距离，即 $\delta=\min\{\delta_l,\delta_r\}$。若情形(3)下的最近点对 p 和 q 的距离比 δ 小，则点 p,q 到垂线 l 的距离分别小于 δ，这样，我们就可以丢掉那些到垂线距离不小于 δ 的点。令 P_1,P_2 分别表示垂线 l 两边宽度为 δ 的带形区域，则点 p 一定在 P_1 中，q 一定在 P_2 中。因此，我们只要考虑 P_1,P_2 区域内部的点即可，如图 16.18 所示。

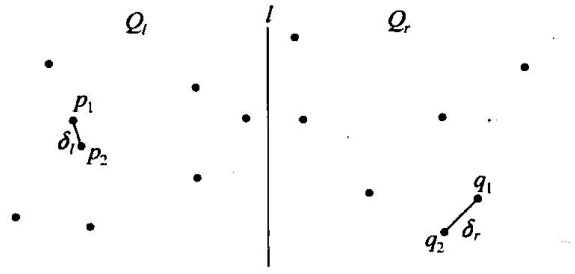


图 16.17

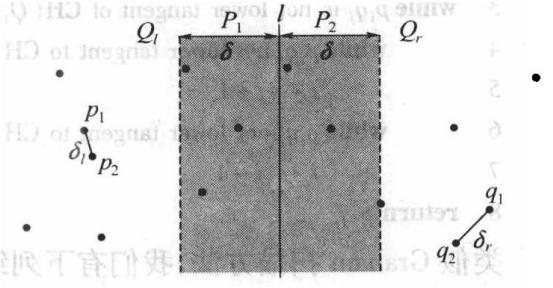


图 16.18

如果存在点对 p 和 q，其中 $p \in Q_l$，$q \in Q_r$，且它们的距离 $d(p, q) < \delta$，则一定有 $p \in P_1$，$q \in P_2$，因此，我们可以通过依次检查 P_1 中一个点 p，然后在 P_2 中寻找一个距离点 p 最近的点 q 即可。现在的问题是，对于 P_1 中一个点 p，我们是否有必要搜索 P_2 中所有的点？事实上，对于 P_1 中一个点 $p = (x_p, y_p)$，我们只需要检查 P_2 中，其纵坐标满足条件 $y_p - \delta < y < y_p + \delta$ 的点即可。如图 16.19所示白色区域中的点。

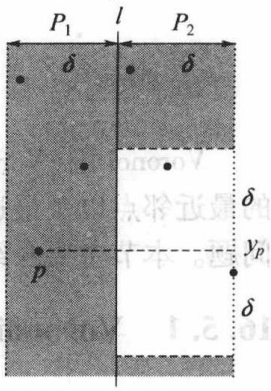


图 16.19

经过上述分析后，分治算法求解最近点对的算法可以描述如下：

```
ClosestPair(Q)
1    if |Q| <4 then
2        compute the distance of each pair point
3        return the minimum distance
4    else
5        compute separation line l that divides Q into Q_l, Q_r
6        δ_l ← ClosestPair(Q_l)
7        δ_r ← ClosestPair(Q_r)
8        δ ← min{δ_l, δ_r}
9        let P_1 be the set of points of Q_l that are within δ of the dividing line l
10       let P_2 be the set of points of Q_r that are within δ of the dividing line l
11       sort the points in P_1 by y - coordinate and obtain P_1*
12       sort the points in P_2 by y - coordinate and obtain P_2*
13       scan P_1* and for each point in P_1* inspect the points of P_2* within δ
14       let δ' be the smallest distance of any pair thus examined
15   return min{δ, δ'}
```

令 $T(n)$ 表示 ClosestPair(Q)算法找 n 个点的最近点对所需要的时间，ClosestPair(Q)算法中，第 11 行～第 12 行进行预处理，时间最多 $O(n)$，其他每行需要花费的时间最多为 $O(n)$。子问题 Q_l，Q_r 的大小分别为$\lceil n/2 \rceil$，$\lfloor n/2 \rfloor$，因此有

$$T(n) = \begin{cases} \Theta(1) & \text{if } n < 4 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) & \text{if } n \geqslant 4 \end{cases}$$

利用第 4 章所介绍的公式法，我们很容易得到下述结论。

定理 16.6 ClosestPair(Q)算法的时间复杂度为 $O(n\lg n)$。

16.5 Voronoi 图

Voronoi 图(Voronoi Diagrams)是一种有趣的几何结构,它包含了有关平面中点的最近邻点以及最远邻点的信息,可用来解决计算几何中大量的近邻性和远邻性问题。本节主要介绍最近点 Voronoi 图。

16.5.1 Voronoi 图的定义及其性质

定义 16.1 设 $Q=\{p_1,p_2,\cdots,p_n\}$ 是平面上 n 个点的集合,平面中与点 p_i 的距离不大于与 Q 中任何其他点的距离的点的集合,称为 p_i 的 Voronoi 区域,记为 VD(p_i)。即

$$\mathrm{VD}(p_i)=\{p:|p-p_i|\leqslant|p-p_j|,\forall j\neq i\}$$

由定义 16.1 可知,VD(p_i)可能是一个凸多边形,也可能是无界的。如果 VD(p_i)是多边形,则最多有 $n-1$ 条边,每条边在 p_i 和 Q 中另一点的垂直平分线上。定义 16.1 表明,每个 $p\in Q$ 的点属于唯一的区域 VD(p),而且 VD(p)内部的任意点 q,q 离 p 比离 Q 中的其他任何点都近。对于 Q 中每个点 p_i,我们都可以定义一个 VD(p_i),因此我们有下述定义。

定义 16.2 平面上 n 个点的 Voronoi 区域的集合,即 $\bigcup\limits_{1\leqslant i\leqslant n}\mathrm{VD}(p_i)$,称为点集 Q 的最近点 Voronoi 图,记为 VD(Q)。

例如,图 16.20(a)给出了平面点集 $Q=\{p_1,p_2,p_3,p_4,p_5,p_6\}$ 中,点 p_2 的 Voronoi 区域 VD(p_2),如图 16.20(a)中阴影部分所示,是一个凸多边形。图 16.20(b)给出了平面点集 $Q=\{p_1,p_2,p_3\}$ 的 Voronoi 图 VD(Q),由 p_1 的 Voronoi 区域 VD(p_1)、p_2 的 Voronoi 区域 VD(p_2)和 p_3 的 Voronoi 区域 VD(p_3)组成,VD(p_1),VD(p_2)和 VD(p_3)由三线共点的垂直平分线分开,而且均是无界的。

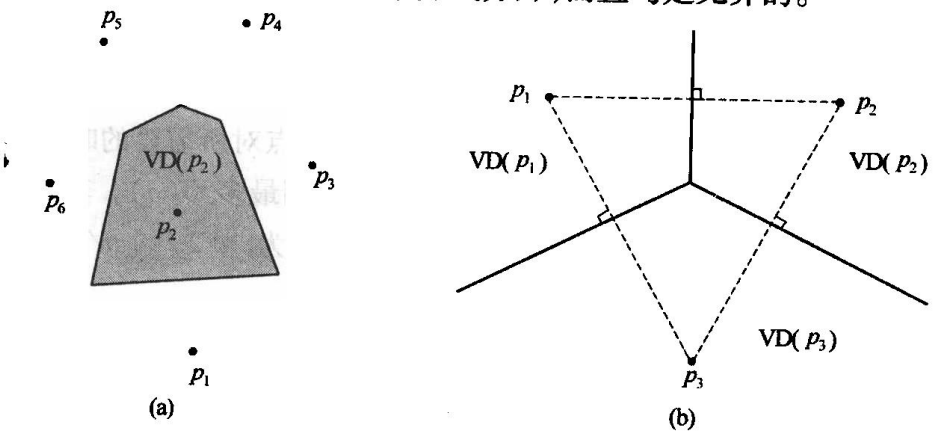


图 16.20

一般地，设 p_i 和 p_j 是平面点集 Q 中的两点。由 p_i 和 p_j 的垂直平分线定义的包含 p_i 的半平面 $H(p_i,p_j)$ 是平面内离 p_i 比离 p_j 近的点所在的区域。点 p_i 的 Voronoi 区域 $\mathrm{VD}(p_i)$ 是 $n-1$ 个半平面的交，即

$$\mathrm{VD}(p_i) = \bigcap_{i\neq j} H(p_i,p_j)$$

上述定义给出了如何在平面上，构造出 $\mathrm{VD}(p_i)$。按照定义 16.2，Voronoi 区域 $\mathrm{VD}(p_i)(1\leqslant i\leqslant n)$ 的集合定义了点集 Q 的 Voronoi 图 $\mathrm{VD}(Q)$。图 16.21(a)给出了一个平面点集 Q 的 Voronoi 图。

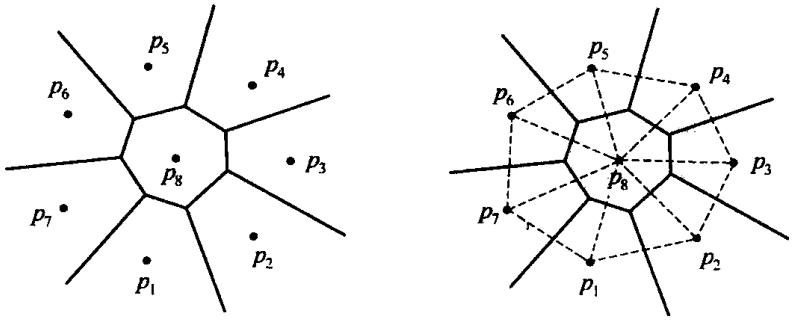


图 16.21

定义 16.3 点集 Q 的 Voronoi 图是一个平面图，Voronoi 图的顶点称为 Voronoi 顶点，Voronoi 图的边称为 Voronoi 边。

下面，我们介绍 Delaunay 三角剖分，它是一个非常有用的概念。

为方便起见，下面我们假设点是在一般位置上，即要求三点不共线，四点不共圆(四点不在一个圆的圆周上)。

给定平面点集 Q，以及相应的 Voronoi 图 $\mathrm{VD}(Q)$，我们构造 $\mathrm{VD}(Q)$ 的对偶图 G 如下：G 中的顶点为 Q 中的点。任给两个顶点 $u,v\in Q$，如果 u,v 的 Voronoi 区域 $\mathrm{VD}(u)$ 和 $\mathrm{VD}(v)$ 共享一条 Voronoi 边，则 u 和 v 之间用一条弧连接。

定义 16.4 给定平面点集 Q，如果 Voronoi 图 $\mathrm{VD}(Q)$ 的对偶图 G 的边是直线，则称对偶图 G 为 Voronoi 图 $\mathrm{VD}(Q)$ 的 Delaunay 三角剖分(Delaunay triangulation)，记为 $\mathrm{DT}(Q)$。

从定义 16.4 可以看出，Voronoi 图 $\mathrm{VD}(Q)$ 的 Delaunay 三角剖分 $\mathrm{DT}(Q)$ 是一个平面图，而且 Delaunay 三角剖分中的每个三角形与 Voronoi 图 $\mathrm{VD}(Q)$ 中度数为 3 的一个顶点对应。由定义 16.3 可知，$\mathrm{VD}(Q)$ 中边的对偶是 $\mathrm{DT}(Q)$ 中的边。值得注意的是，$\mathrm{DT}(Q)$ 的边不一定与其对应 Voronoi 图中的边相交。

Delaunay 三角剖分 $\mathrm{DT}(Q)$ 有下列有趣的性质：

性质 16.4 $\mathrm{DT}(Q)$ 是 $\mathrm{VD}(Q)$ 的直线对偶。

性质 16.5 如果 Q 中的点没有四点共圆，则 $\mathrm{DT}(Q)$ 是一个三角剖分，每一个面是一个三角形。

性质 16.6 DT(Q)的每一个面对应 VD(Q)的一个顶点。

性质 16.7 DT(Q)的每一条边对应 VD(Q)的一条边。

性质 16.8 DT(Q)的每一个顶点对应 VD(Q)的一个区域。

性质 16.9 DT(Q)的边界是点集 Q 的凸包 CH(Q)。

性质 16.10 DT(Q)每一个面的内部不含 Q 中的点。

上述性质比较直观,读者可给予验证。图 16.21 给出了 Voronoi 图(图 16.21(a))及其相应的 DT(Q)(图 16.21(b)),其中虚线平面图就是 DT(Q),DT(Q)中每个三角形是一个面,例如三角形 $<p_1p_2p_8>$ 是一个面。

定理 16.7 给定平面点集 Q,Voronoi 图 VD(Q)以及相应的 DT(Q),令 $|Q| = n$,设 m 和 r 分别是 DT(Q)中的边数和三角形区域数,则

(1) DT(Q)中的边数最多是 $3n-6$。

(2) VD(Q)中的边数和顶点数分别最多是 $3n-6$ 和 $2n-4$。

证明:由于 DT(Q)是一个平面图,由欧拉公式可得

$$n - m + r = 2 \tag{16.1}$$

且有

$$m \leqslant 3n - 6 \tag{16.2}$$

因此结论(1)成立。

由性质 16.7,DT(Q)中的每条边对应 VD(Q)中的一条边,因此 VD(Q)中的边数也不多于 $3n-6$。

由式(16.1)和式(16.2)可得

$$r = m - n + 2 \leqslant 2n - 4$$

由性质 16.6,DT(Q)中的每个三角形区域(无界区域除外)对应 VD(Q)中的一个顶点,因此 VD(Q)中的顶点数最多是 $2n-4$。故所证成立。

由定理 16.7 可得,存储 DT(Q)或 VD(Q)只需要 O(n)的空间。

平面点集 Q 的 Voronoi 图也具有一些有趣的性质:

性质 16.11 VD(p_i)是无界的当且仅当点 p_i 是凸包 CH(Q)的顶点。

性质 16.12 如果 Voronoi 顶点 v 是三个 Voronoi 区域 VD(p_1)、VD(p_2)和 VD(p_3)的交点,则 v 是由三点 p_1、p_2 和 p_3 所确定的圆 $C(v)$的圆心。

性质 16.13 $C(v)$是对应顶点 v 的 Delaunay 三角形的外接圆。

性质 16.14 $C(v)$的内部不含 Q 中的点。

性质 16.15 如果点 p_i 是 p_j 的最近点,则边(p_i,p_j)是 DT(Q)的一条边。

性质 16.16 如果过点 p_i 和 p_j 的圆不含 Q 中其他的点,则边(p_i,p_j)是 DT(Q)的一条边。反过来仍然成立。

这里我们简单证明几个性质。

性质 16.12 的证明:因为 v 是三个 Voronoi 区域 VD(p_1)、VD(p_2)和 VD(p_3)的

交点，恰好是三条垂直平分线的交点，是 Voronoi 图三条边的公共交点，这些边位于由这三点定义的三角形的边的垂直平分线上，因此，Voronoi 顶点是通过这三点的圆的圆心。

性质 16.14 的证明：用反证法证明。如果有某个其他的点 p_4 在 $C(v)$ 内，则 v 离 p_4 比离 p_1，p_2 和 p_3 中的任意一点都近，这就意味着 v 一定落在 $\mathrm{VD}(p_4)$，而这与 v 是 $\mathrm{VD}(p_1)$、$\mathrm{VD}(p_2)$ 和 $\mathrm{VD}(p_3)$ 的交点矛盾，由此可见 $C(v)$ 内部不包含 Q 中的点。

性质 16.16 的证明：设过点 p_i 和 p_j 的圆的圆心为 x，则圆 $C(x)$ 的圆心 x 一定在 p_ip_j 的垂直平分线上，因为过点 p_i 和 p_j 的圆不含 Q 中其他的点，因此 $x \in \mathrm{VD}(p_i) \cap \mathrm{VD}(p_j)$，$x$ 在 $\mathrm{VD}(p_i)$ 和 $\mathrm{VD}(p_j)$ 共用的一条边上，因此 p_i 和 p_j 是最近的邻居，故 (p_i, p_j) 是 $\mathrm{DT}(Q)$ 的一条边。

其他的几个性质，读者可以类似验证。

16.5.2 Voronoi 图的构造

给定大小为 n 的点集 Q，直接构造 Voronoi 图需要每次构造一个区域。由于每个区域是 $n-1$ 个半平面的交集，每个区域的构造可以在 $O(n\lg n)$ 时间内完成，因而直接法构造 Voronoi 图的时间复杂度为 $O(n^2 \lg n)$。对于规模比较小的问题，可以这样直接求解，但是对于规模比较大的问题，则需要寻找更有效的算法。

事实上，这个问题可以利用分治算法来求解。这个想法比较直接，因为将大小为 n 的点集 Q 分成两个子集 Q_l，Q_r，其中点集 Q_l 的大小为 $\lceil n/2 \rceil$，点集 Q_r 的大小为 $\lfloor n/2 \rfloor$，类似前面的讨论，我们只要根据 Q 中 x 坐标的中值划分，使得 $Q_l \cap Q_r = \varnothing$ 即可。然后构造每个子集的 Voronoi 图 $\mathrm{VD}(Q_l)$ 和 $\mathrm{VD}(Q_r)$，最后合并两个 Voronoi 图即可。详细的算法可以描述如下：

```
DivideConquerVoronoi(Q)
1  if |Q| <4 then
2      use exhaustive search to construct VD(Q)
3      return VD(Q)
4  else
5      compute separation line l that divides Q into Q_l, Q_r
6      VD(Q_l) ← DivideConquerVoronoi(Q_l)
7      VD(Q_r) ← DivideConquerVoronoi(Q_r)
8      VD(Q) ← Merge(VD(Q_l), VD(Q_r))
9      return VD(Q)
```

DivideConquerVoronoi(Q) 算法在第 1 行判断 $|Q|$ 是否小于 4，如果是，则在第 2 行利用枚举搜索（exhaustive search）构造出 $\mathrm{VD}(Q)$，否则在第 5 行计算划分线 l，将 Q 分成两个子集 Q_l，Q_r。算法的关键是如何合并两个 Voronoi 图。为了合并 $\mathrm{VD}(Q_l)$ 和 $\mathrm{VD}(Q_r)$，按照 Voronoi 图的定义，两个 Voronoi 图中远离直线 l 的顶点互不影响，而在直线 l 附近的顶点才有影响。此外，Voronoi 图的边界是点集 Q 中相

邻两顶点的垂直平分线，因此，我们只要依次找出两个 Voronoi 图中相邻顶点的垂直平分线，然后将这些垂直平分线连接起来形成一条折线，就可以得到 Voronoi 图的边界。这条折线常称为分段线性超平面，简记为 HP，它具有如下性质：如果点 p 在 HP 的左半平面，那么点 p 的最近邻点一定是 Q_l 中的点，如果点 p 在 HP 的右半平面，那么点 p 的最近邻点一定是 Q_r 中的点。

找到了线性超平面 HP，直接删除 $VD(Q_l)$ 中越过 HP 到右边的部分，同样删除 $VD(Q_r)$ 中越过 HP 到左边的部分，就可以得到合并后的 Voronoi 图。图 16.22 给出了两个集合 Q_l，Q_r 的 Voronoi 图。图 16.23 给出了分段线性超平面，显然这条折线在两个 $VD(Q_l)$ 和 $VD(Q_r)$ 的交界处，而且 HP 由一条射线、一些线段和另一条射线组成，它们都是最终 Voronoi 图的一部分。

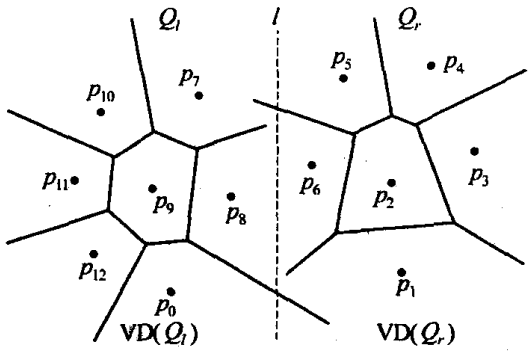


图 16.22

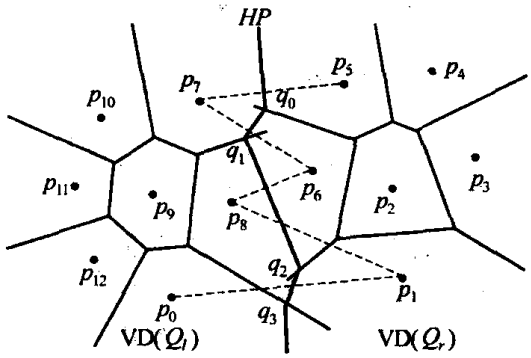


图 16.23

关于 HP，有一个有趣的性质。

性质 16.17 HP 关于 y 是单调的，即每条水平线与 HP 相交至多一次。

证明：假设相交两次，即存在一条折线使得 HP 在 q_1 折向上方，到达 q_2，如图 16.24所示。q_1q_2 是 p_1p_2 的垂直平分线，q_1q_0 是 p_1p_3 的垂直平分线。由于 HP 划分 Q 中的点为两部分，因此 p_2，$p_3 \in Q_r$，而划分直线 l 是将 Q 中的点划分为两部分 Q_l，Q_r，将使得 p_1，p_2 在同一个集合，p_3 在另一个集合，这样就产生了矛盾。故所证成立。

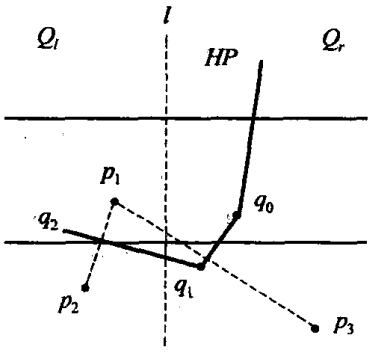


图 16.24

为了找到 HP，我们需要先找到两个 Voronoi 图中最接近的处于图最上部的一对顶点 p_i，p_j，然后从这一对顶点出发，作这一对顶点的垂直平分线，记下该垂直平分线最先与 Voronoi 图的一条射线的交点 u，然后在相应 Voronoi 图中找一个最接近 u 的顶点，然后作该顶点与另一个 Voronoi 图中最接近顶点的垂直平分线。重复上述过程，直到作出两个 Voronoi 图中最接近

的处于 Voronoi 图最下部的一对顶点的垂直平分线为止。上述过程中，最关键的是找出两个 Voronoi 图中最接近的处于 Voronoi 图最下部和最上部的两对顶点。VD(Q_l)和 VD(Q_r)的对偶分别是 DT(Q_l)和DT(Q_r)，根据 Delaunay 三角剖分的性质 16.9，可知 DT(Q_l)和 DT(Q_r)的边界分别构成一个凸包，而两个凸包的上切线和下切线所连接的两对顶点正是要找的顶点。综上所述，合并两个 Voronoi 图的算法如下：

```
Merge(VD(Q_l),VD(Q_r))
1    find the convex hulls CH(Q_l) and CH(Q_r) of VD(Q_l) and VD(Q_r) respectively
2    find the upper tangent p_ip_j and lower tangent p_up_v of CH(Q_l) and CH(Q_r)
3    SG←p_ip_j; HP←∅
4    while SG≠p_up_v do
5        BS←the perpendicular bisector of SG
6        HP← HP∪{BS}
7        if BS first intersects with the ray l from VD(Q_l) then
8            find p_z ∈ VD(Q_l) such that l is a perpendicular bisector of p_ip_z
9            i←z
10           SG←p_ip_j
11       else
12           find p_z ∈ VD(Q_r) such that l is a perpendicular bisector of p_jp_z
13           j←z
14           SG←p_ip_j
15   BS←the perpendicular bisector of SG
16   discard the edges of VD(Q_l) which extend to the right of HP
17   discard the edges of VD(Q_r) which extend to the left of HP
18   return the resulting graph VD(Q)
```

Merge(VD(Q_l),VD(Q_r))算法第 7 行中，SG 的垂直平分线 BS 一定与 VD(Q_l)和 VD(Q_r)相交，如果 BS 首先与来自 VD(Q_l)的射线 l 相交，则执行第 8 行～第 10 行，如果 BS 首先与来自 VD(Q_r)的射线 l 相交，执行第 12 行～第 14 行。第 1 行中，如果按照 16.3 节构造凸包的算法，时间复杂度至少为 O($n$lg$n$)，事实上，构造 Voronoi 图的凸包有下列更有效的算法：根据 Voronoi 图性质 16.11，依次检查 Q 中每个点 p_i，如果 VD(p_i)是无界的，则 p_i 是凸包的顶点。因此，Voronoi 图构造完之后，可在 O(n)时间内构造出凸包。第 2 行中，可利用 16.3 节找上、下切线的方法，也可在 O(n)时间内找到上、下切线的两对顶点。第 4 行～第 15 行是构造 HP 的过程，由定理 16.7，VD(Q_l)和 VD(Q_r)的边数最多为 $3n-6$，而根据 HP 的性质 16.17，HP 中的边不会重复，因此 HP 可在 O(n)时间内构造出来。因此我们有下

列结论。

定理 16.8 Merge(VD(Q_l),VD(Q_r))算法的时间复杂度为 O(n)。

现在我们给出图 16.25 中两个 Voronoi 图 VD(Q_l)和 VD(Q_r)的合并过程。图 16.25(a)给出了两个凸包 CH(Q_l)和 CH(Q_r)的下切线 p_0p_1 以及上切线 p_7p_5，SG = p_7p_5，BS 就是 p_7p_5 的垂直平分线，与 VD(Q_r)的一条射线交于 q_0，此时 $p_z = p_6$，SG = p_7p_6。作 p_7p_6 的垂直平分线，与来自 VD(Q_l)的射线交于 q_1，如图 16.25(b)所示。此时 $p_z = p_8$，SG = p_8p_6。作 p_8p_6 的垂直平分线，与来自 VD(Q_r)的射线交于 q_2，如图 16.25(c)所示。此时 $p_z = p_1$，SG = p_8p_1。作 p_8p_1 的垂直平分线，与来自 VD(Q_l)的射线交于 q_3，如图 16.25(d)所示。此时 $p_z = p_0$，SG = p_0p_1。**while** 循环终止。最后作 p_0p_1 的垂直平分线，得到图 16.23。

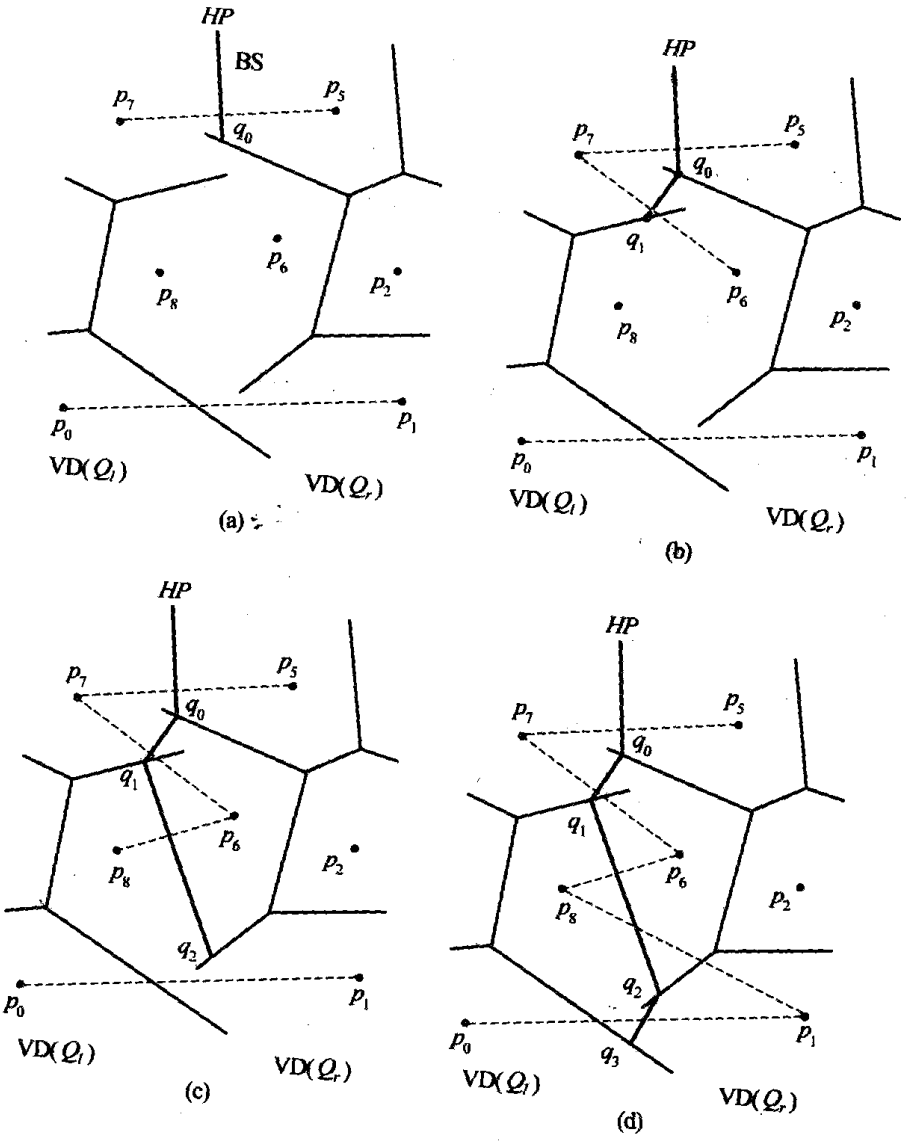


图 16.25

令 $T(n)$ 表示 DivideConquerVoronoi(Q)算法找 n 个点的 Voronoi 图所需要的时间，子问题 Q_l，Q_r 的大小分别为$\lceil n/2 \rceil$，$\lfloor n/2 \rfloor$，因此有

$$T(n) = \begin{cases} \Theta(1) & \text{if } n < 6 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) & \text{if } n \geqslant 6 \end{cases}$$

利用第 4 章所介绍的公式法，我们很容易得到下述结论。

定理 16.9 基于 Merge(VD(Q_l)，VD(Q_r))算法的 DivideConquerVoronoi(Q)算法的时间复杂度为 O($n$lg$n$)。

16.5.3 Voronoi 图的应用

在 Voronoi 图的构造一节，我们已经介绍了如何利用 Voronoi 图来计算凸包。下面，我们介绍点集 Voronoi 图的更多应用。由于 Voronoi 图含有几乎所有的关于近邻性的信息，因此它可以用来解决许多与近邻性有关的问题，例如最近邻点问题、最小生成树问题、旅行商问题等等。通过首先构造出 Voronoi 图，可以有效地解决这些问题。

1. *所有最近邻点*

定义 16.5 设 Q 是平面上的一个点集，对于 Q 中的两个点 p 和 q，q 是 p 的最近邻点当且仅当 $d(p,q) = \min_{r \in Q - \{p\}} d(p,r)$，其中 $d(p,r)$ 是 p 和 r 间的欧几里得距离。

定义 16.5 表明，如果 q 是 p 的最近邻点，则在 $Q - \{p\}$ 的所有点中，q 距离 p 是最近的。

最近邻点是集合 Q 上的一个关系，可写作 $p \rightarrow q$：p 的最近邻点是 q。值得注意的是，这个关系不一定是对称的，例如图 16.26(a)所示，$p_2 \rightarrow p_3$，$p_3 \rightarrow p_2$，p_2 和 p_3 是对称的。其他的有以下关系：$p_1 \rightarrow p_2$，$p_4 \rightarrow p_3$，$p_5 \rightarrow p_4$。

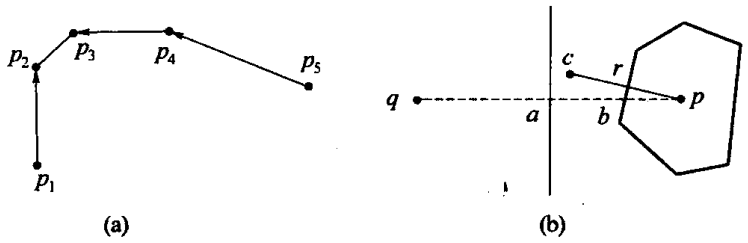


图 16.26

所有最近邻点问题可以描述如下：给定一个平面点集 Q，对于 Q 中的每一点找出其最近邻点。对于这个问题，如果用直接法，则时间复杂度为 O(n^2)。利用 Voronoi 图，我们可以得到一个更有效的算法。下面给出一个有用的结论。

定理 16.10 设 Q 是平面中的一个点集，对于 Q 中的两点 p 和 q，如果 $p \rightarrow q$，则 $VD(p)$ 和 $VD(q)$ 共用一条 Voronoi 边。

证明：可用反证法证明。假设 VD(p)和 VD(q)不共用一条 Voronoi 边，则根据 Voronoi 图的定义，p 和 q 的垂直平分线一定在 VD(p)的外部。令线段$\overline{pq}$与垂直平分线交于点 a，与某 Voronoi 边交于 b。假设 Voronoi 边在 p 和 c 之间，且$\overline{pc}$与 Voronoi 边交于点 r，如图 16.26(b)所示。根据图 16.26(b)，可得

$$d(p,r) < d(p,b) < d(p,a)$$

由上式，立即可得

$$d(p,c) < d(p,q)$$

这表明 $p \rightarrow c$ 与已知 $p \rightarrow q$ 矛盾，故所证成立。

根据定理 16.10，所有最近邻点问题可通过检查 Voronoi 图的每一条边来解决。由于两个 Voronoi 区域共用一条 Voronoi 边，检查 Voronoi 边不会多于两次。因此，Voronoi 图构造好之后，所有最近邻点问题可在线性时间内解决，而 Voronoi 图的构造，可在 O($n\lg n$)时间内完成，即有下述结论。

定理 16.11 借助 Voronoi 图，所有最近邻点问题可在 O($n\lg n$)时间内解决。

2. 最小生成树问题

给定一个平面点集 Q，以及 Q 中每对点之间的欧几里得距离。欧几里得最小生成树问题是寻找一棵最小生成树 T，使得

$$d(T) = \sum_{(p,q)\in T} d(p,q)$$

是最小的。

利用第 8 章介绍的 Kruskal 算法，时间复杂度为 O($|E|\lg n$)，其中 $|E|$ 为平面中相连的边数，如果平面中每对点均互相连接，则 $|E| = n^2$，从而算法时间复杂度为 O($n^2\lg n$)。下面，我们利用点集的 Delaunay 三角剖分，可以避免考虑所有的边，从而可以得到更有效的算法。

定理 16.12 设 Q 是平面中的一个点集，最小生成树 T 是 DT(Q)的一个子集，即 $T \subseteq \mathrm{DT}(Q)$。

证明：要证明 $T \subseteq \mathrm{DT}(Q)$，也就是要证明，任给边 $(p,q) \in T$，则 $(p,q) \in \mathrm{DT}(Q)$。用反证法证明。假设 $(p,q) \notin \mathrm{DT}(Q)$，看是否能得出与已知矛盾的结果。

根据 Voronoi 图的性质 16.16，如果 $(p,q) \notin \mathrm{DT}(Q)$，则通过 p 和 q 的圆一定还包含 Q 中的其他点。特别地，以 (p,q) 为直径的圆也包括除 p 和 q 之外的其他 Q 中的点，假设这点为 r，如图 16.27 所示。由图 16.27，有 $d(p,r) < d(p,q)$，$d(q,r) < d(p,q)$，即使 r 在圆上，上述不等式仍然成立。删除边 (p,q)，将使原来的树分成两棵子树，点 p 所在的子树为 T_p，点 q 所在的子树为 T_q。不失一般性，假定点 r 在子树 T_p 上，现在把边 (q,r) 加进来，从而得到一棵新树 $T' = T_p \cup (q,r) \cup T_q$。由于 $d(q,r) < d(p,q)$，因此 $d(T') < d(T)$，这与 T 是最小生成树矛盾。故所证成立。

由定理 16.12，立即可以得到找欧几里得最小生成树的算法：首先构造 Delaunay 三角剖分，然后对 Delaunay 三角剖分的边进行排序，然后调用 Kruskal 算法。

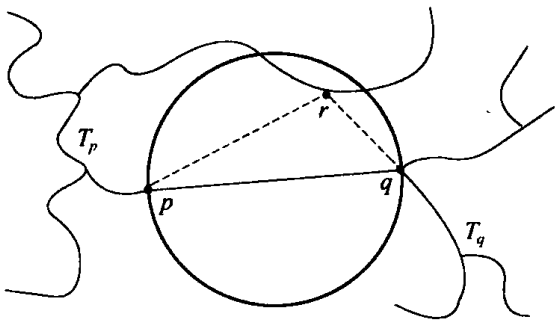


图 16.27

构造 Delaunay 三角剖分能够在 $O(n\lg n)$ 时间内完成，Delaunay 三角剖分的边数为 $O(n)$，对边进行排序，其时间复杂度为 $O(n\lg n)$，调用 Kruskal 算法费时 $O(n\lg n)$，因此可得如下结论。

定理 16.13 利用 Delaunay 三角剖分，然后调用 Kruskal 算法求解欧几里得最小生成树问题的时间复杂度为 $O(n\lg n)$。

16.6 小 结

本章主要取材于文献[1，5，19]，主要介绍计算几何的一些基础知识，包括计算凸包、最近点对的常用算法以及 Voronoi 图。本章介绍的内容都非常经典，其中一些算法已经得到了广泛的应用。对于计算凸包的算法，当前最有效的算法是 Graham 扫描法，其时间复杂度等于凸包问题的下界 $O(n\lg n)$。对于三维凸包的计算、Voronoi 图的应用、计算几何最新的以及更全面的内容，大家可以参考专门介绍计算几何的文献[19]。

习题

16.1 修改 AnySegmentsIntersect(S)算法，以便找出所有线段的交点集合。

16.2 给定一个由 n 个点组成的序列 $<p_1,p_2,\cdots,p_n>$，试按序列中各点对某给定原点 p_0 的极角大小对该序列进行排序，写出排序的伪代码。所设计算法的时间复杂度应为 $O(n\lg n)$，并要求用叉积来比较极角的大小。

16.3 试说明如何在 $O(n^2\lg n)$ 的时间内确定具有 n 个点的集合中任意三点是否共线。

16.4 试说明如何在 $O(n)$ 的时间内计算出 n 个顶点组成的简单多边形的面积，其中简单多边形不一定是凸多边形。

16.5 试说明如何在 $O(n\lg n)$ 的时间内判断两个具有 n 个顶点的简单多边形是否相交。

16.6 证明定理 16.5。

16.7 有一种求解凸包的增量算法，其基本思路是从一个三角形凸包出发，一次考虑一个顶点，然后用一个函数 merge 实现下列过程：如果该顶点在凸包上，则丢掉该顶点，否则，将该顶点加入凸包，同时对原来凸包进行修改。其基本算法如下：

IncrementalHull(Q)

1 $H_2 \leftarrow \{p_0, p_1, p_2\}$

2 **for** $k \leftarrow 3$ **to** $n-1$ **do**

3 $\quad H_k \leftarrow \text{merge}(H_{k-1}, p_k)$

问题是：

(1) 如何判断点 p_k 在当前凸包 H_{k-1} 上？

(2) 如果点 p_k 不在凸包 H_{k-1} 上，如何对凸包 H_{k-1} 进行修改，以便得到新的凸包 H_k？

(提示：过点 p_k 作当前凸包 H_{k-1} 的两条切线，然后考虑如何删除两切点之间 H_{k-1} 上的点。)

(3) 分析 IncrementalHull(Q)算法的时间复杂度。

16.8 当所有点都在 x 轴上时，设计出一个有效的算法来寻找最近点对。

16.9 在平面上，点 p_1 和 p_2 的 L_m 距离由下式给出：$((x_1-x_2)^m+(y_1-y_2)^m)^{1/m}$，前面定义的欧几里得距离实际上是 L_2 距离。试用 L_1 距离来修改最近点对算法。

16.10 除了书中介绍的利用点集 Q 中点 x 的坐标的中值来划分 Q 外，还有别的有效方法吗？谈谈你的看法。

16.11 证明 Voronoi 图性质 16.11。

16.12 证明 Voronoi 图性质 16.16 的第二部分。

实验题

16.13 编程实现 ClosestPair(Q)算法。

16.14 用实验分析方法比较三种求解凸包算法的效率。

16.15 完成 XOJ 如下题目：1071，1082。

16.16 完成 POJ 如下题目：1039，1106，1113，1118，1151，1228，1379，1654，1687，1693，1873，2187，2826，3608。

参考文献

[1] Cormen T H, Leiserson C E, Rivest R L. Introduction to Algorithms. 第2版影印版. 北京: 高等教育出版社, 2003.

[2] Lee R C T, Tseng S S, Chang R C 等. 算法设计与分析导论. 王卫东, 译. 北京: 机械工业出版社, 2008.

[3] Ellis Horowitz, Sartaj Sahni, Sanguthevar Rajasekaran. 计算机算法(C++版). 冯博琴, 叶茂, 高海昌, 等译. 北京: 机械工业出版社, 2006.

[4] 张德富. 算法设计与分析(高级教程). 北京: 国防工业出版社, 2007.

[5] Alsuwaiyel M H. 算法设计技巧与分析. 吴伟昶, 方世昌, 等译. 北京: 电子工业出版社, 2004.

[6] Michael T. Goodrich, Roberto Tamassia. 算法分析与设计. 霍红卫, 译. 北京: 人民邮电出版社, 2006.

[7] Sartaj Sahni. 数据结构, 算法与应用——C++语言描述. 汪诗林, 孙晓东, 等译. 北京: 机械工业出版社, 2003.

[8] Karger D R, Klein P N, R E Tarjan. A randomized linear-time algorithm for finding minimum spanning trees. Journal of the ACM, 1995, 42(2): 321 - 328.

[9] Ahuja R K 等. 网络流: 理论, 算法与应用. 影印版. 北京: 机械工业出版社, 2005.

[10] Sedgewick Robert. Algorithms in C++, Part 5 Graph Algorithms. 第3版影印版. 北京: 高等教育出版社, 2002.

[11] Even S, Tarjan R E. Network Flow and Testing Graph Connectivity. SIAM Journal on Computing, 1975, 4(4): 507 - 518.

[12] Silvio Micali, Vijay V, Vazirani. An $O(\sqrt{|V|}|E|)$ Algorithm for Finding Maximum Matching in General Graphs. FOCS 1980: 17 - 27.

[13] Saul I Gass. Linear Programming Methods and Applications. Oversea Publishing House, 2003.

[14] Hochbaum D S. Approximation Algorithms for NP-hard Problems. 影印版. 世界图书出版公司, 1998.

[15] 堵丁柱, 葛可一, 王洁. 计算复杂性导论. 北京: 高等教育出版社, 2002.

[16] 王晓东. 计算机算法设计与分析. 北京: 电子工业出版社, 2005.

[17] Nils J Nilsson. 人工智能. 郑扣根, 庄越挺, 译. 北京: 机械工业出版社, 1999.

[18] George F Luger. Artificial Intelligence: Structures and Strategies for Complex Problem Solving. 第4版影印版. 北京: 机械工业出版社, 2003.

[19] Joseph O' Rourke. 计算几何C语言描述. 第2版. 北京: 机械工业出版社, 2005.

[20] Dasgupta S, Papadimitriou C H, Vazirani U V. Algorithms. McGraw-Hill, 2006.

[21] Brassard G, Bratley P . Fundamentals of Algorithms. 影印版. 北京：清华大学出版社, 2005.
[22] Jon Kleinberg, éva Tardos. Algorithm Design. 影印版. 北京：清华大学出版社, 2005.
[23] Sara Baase, Allen Van Gelder. Computer Algorithms: Introduction to Design and Analysis. 第3版影印版. 北京：高等教育出版社, 2001.
[24] http://www. cs. uiuc. edu/class/fa05/cs473g/lectures/07 - approx. pdf.
[25] 吴文虎, 王建德. 实用算法的分析与程序设计. 北京：电子工业出版社,1998.
[26] Beasley J E. http://people. brunel. ac. uk/ ~ mastjjb/jeb/info. html.
[27] Jeffrey J McConnell. Analysis of algorithms: An active Learning Approach. Jones and Bartlett Publishers,2007.